PLANE COMMUNITY EDITION V1.4.2 · GIT-SYNC 0.1.0

git-sync for Plane

Developer & Admin Manual

Plane for tasks, GitHub for code, and git-sync keeping the two connected — from your first work item to a merged pull request, and from installing the service to keeping it healthy.

Bespo EMS PLC
git-sync 0.1.0 · commit 95cc855 · Plane CE v1.4.2
15 October 2026

Contents

00 Introduction 9

Part A · Developer guide 14

1 Welcome 15

2 Getting in 21

3 Finding your way 31

4 Work items 42

5 Views and layouts 65

6 Planning 77

7 Notifications 89

8 The four connections and your GitHub account 95

9 Linking branches, commits and pull requests 108

10 Magic words and WEB-NEW 120

11 Smart Commits 127

12 The Development panel 141

13 Issue mirroring and what syncs 152

14 PR automation from a developer's view 164

15 A day in the life 173

16 Team conventions 193

17 Troubleshooting 197

18 FAQ and glossary 208

19 Cheat sheet 218

Part B · Admin guide 219

20 Architecture and requirements 220

21 Install git-sync 232

22 Register the GitHub App 254

23 God mode settings (instance admin) 263

24 Connect GitHub and map repositories (workspace admin) 277

25 Mapping settings and conflict policies 290

26 Rules: lifecycle table, branch-aware rules, YAML, versions and Test rules 300

27 User links 316

28 Sync health and the audit log 321

29 Operations: update, rotate, replay, back up, uninstall 333

30 Admin troubleshooting 341

Appendix A · All routes 349

Appendix B · Settings and environment variables 358

Appendix C · Reference, magic-word and Smart Commit grammar 364

Appendix D · Rule reference: triggers, conditions, actions 371

Appendix E · Known limitations and defects 378

Appendix F · Version and how this manual was made 384

List of figures 4

Index of UI labels 387

List of figures

0.1 Diagram: Plane holds tasks, GitHub holds code, and git-sync connects them through identifiers and events. 9
1.1 Diagram: outcome, ownership, workflow, planning, relationships and development are separate parts of a work it 16
1.2 Diagram: Plane holds tasks, GitHub holds code, and git-sync connects them through identifiers and events. 17
1.3 Diagram: join, find, read, connect, practice and verify form the first-day journey. 19
2.1 Screenshot (test Plane): Accept the invitation: 22
2.2 Screenshot (test Plane): Sign in: 23
2.3 Screenshot (test Plane): Personal menu: 24
2.4 Screenshot (test Plane): Profile: 25
2.5 Screenshot (test Plane): Preferences: 26
2.6 Screenshot (test Plane): Email notifications: 27
2.7 Screenshot (test Plane): Join Website: 29
3.1 Screenshot (test Plane): Home: 31
3.2 Screenshot (test Plane): Workspace switcher: 33
3.3 Screenshot (test Plane): Your work: 34
3.4 Screenshot (test Plane): Command palette: 36
3.5 Screenshot (test Plane): Keyboard shortcuts: 37
4.1 Diagram: outcome, ownership, workflow, planning, relationships and development are separate parts of a work it 42
4.2 Screenshot (test Plane): WEB-42: 44
4.3 Screenshot (test Plane): Create a work item: 45
4.4 Screenshot (test Plane): Quick add: 46
4.5 Screenshot (test Plane): Description: 48
4.6 Diagram: state groups classify project-specific state names. 49
4.7 Screenshot (test Plane): State: 50
4.8 Screenshot (test Plane): Priority: 51
4.9 Screenshot (test Plane): Labels: 52
4.10 Screenshot (test Plane): Due date: 54
4.11 Screenshot (test Plane): Relations: 56
4.12 Screenshot (test Plane): Choose a related item: 56
4.13 Screenshot (test Plane): Add link: 57
4.14 Screenshot (test Plane): Attachment: 58
4.15 Screenshot (test Plane): Comment editor: 59
4.16 Screenshot (test Plane): Item menu: 60
4.17 Screenshot (test Plane): Unfinished item: 61
4.18 Screenshot (test Plane): Drafts: 62
4.19 Screenshot (test Plane): Starting work: 63
5.1 Screenshot (test Plane): List layout: 66
5.2 Screenshot (test Plane): Board layout: 67
5.3 Screenshot (test Plane): Calendar: 68

5.4 Screenshot (test Plane): Spreadsheet: 69
5.5 Screenshot (test Plane): Gantt: 70
5.6 Screenshot (test Plane): Filters: 72
5.7 Screenshot (test Plane): Display: 74
5.8 Screenshot (test Plane): Project Views: 75
6.1 Diagram: project, module, cycle, view, page and intake serve different planning purposes. 77
6.2 Screenshot (test Plane): Cycles: 78
6.3 Screenshot (test Plane): Inside a cycle: 79
6.4 Screenshot (test Plane): Modules: 80
6.5 Screenshot (test Plane): Inside a module: 81
6.6 Screenshot (test Plane): Pages: 82
6.7 Screenshot (test Plane): Shared page: 82
6.8 Screenshot (test Plane): Intake: 84
6.9 Screenshot (test Plane): Estimates: 85
6.10 Screenshot (test Plane): Project states: 86
6.11 Screenshot (test Plane): Project analytics: 87
7.1 Screenshot (test Plane): Inbox: 90
7.2 Screenshot (test Plane): Email notifications: 93
8.1 Screenshot (demo instance): A member opening the admin page: 97
8.2 Screenshot (demo instance): Admins also reach the card from the workspace page: 97
8.3 Screenshot (test Plane): Personal GitHub account: 98
8.4 Screenshot (demo instance): My GitHub account before connecting: 99
8.5 Screenshot (demo instance): Right after GitHub sends you back: 100
8.6 Screenshot (demo instance): Connected: 100
8.7 Screenshot (demo instance): GitHub refused the stored token: 102
8.8 Strip (demo instance): My GitHub account in three states: not connected → connected → needs re-authorization. 103
8.9 Screenshot (demo instance): After you disconnect: 104
8.10 Screenshot (demo instance): You pressed Cancel on GitHub: 105
8.11 Screenshot (demo instance): That GitHub account is already linked to another member: 106
8.12 Diagram: the four connections. Connections 1–3 are shared team configuration; connection 4 is personal. 96
9.1 Screenshot (demo instance): A non-closing reference (Refs WEB-41): 112
9.2 Diagram: branch, PR title and body contribute references; the strongest tier wins for each work item. 114
9.3 Screenshot (GitHub): Real PR #117: 115
9.4 Screenshot (GitHub): Real GitHub branches: 117
9.5 Diagram: how one key becomes a reference with a tier. 113
10.1 Screenshot (demo instance): A work item created from a WEB-NEW pull request: 123
11.1 Diagram: how git-sync reads one commit message. 129
12.1 Screenshot (test Plane): The Development section embedded in a Plane work item's sidebar, with historical link 141
12.2 Screenshot (demo instance): A project with no mapped repository: 143
12.3 Screenshot (demo instance): A failing check: 144
12.4 Screenshot (demo instance): WEB-42 before any code: 146
12.5 Screenshot (demo instance): After Create branch: 147
12.6 Screenshot (demo instance): Pressing Create branch again: 148

12.7 Screenshot (test Plane): The standalone Development page on the earlier test instance: historical links and th 150
13.1 Screenshot (demo instance): A GitHub issue mirrored into Plane: the Development panel has no PR — mirrored iss 154
13.2 Diagram: events travel through signed webhooks, queues, rules and durable writes before a refreshed UI reflect 160
14.1 Diagram: the daily loop from picking WEB-42 to verifying Done and MERGED. 164
14.2 Screenshot (demo instance): Two PRs close WEB-36: 169
14.3 Diagram: the merge guard, checked when a linked PR merges. 168
15.1 Strip (demo instance): The WEB-42 loop in four frames: push → draft PR → review → merged. 173
15.2 Screenshot (test Plane): WEB-42: 175
15.3 Screenshot (test Plane): Starting work: 176
15.4 Screenshot (demo instance): WEB-42 before any code: 177
15.5 Screenshot (demo instance): After Create branch: 177
15.6 Screenshot (demo instance): After the first push: 179
15.7 Screenshot (GitHub): A real Smart Commit on the test repository (from the earlier test, not this run): 179
15.8 Screenshot (demo instance): Draft PR opened: 181
15.9 Screenshot (GitHub): A real PR on the test repository (PR #117 for WEB-42 from the earlier test, not this run) 181
15.10 Screenshot (demo instance): CI started: 182
15.11 Screenshot (demo instance): CI passed and a preview deployed: 183
15.12 Screenshot (demo instance): Ready for review and a reviewer requested: 184
15.13 Screenshot (demo instance): The reviewer asked for changes: 185
15.14 Screenshot (GitHub): A real review on the test repository (from the earlier test; it is a COMMENTED walkthroug 186
15.15 Screenshot (demo instance): Approved with passing checks: 187
15.16 Strip (demo instance): Review badges on the PR line: REVIEW → REJECTED → APPROVAL. 187
15.17 Screenshot (demo instance): Merged: 188
15.18 Screenshot (demo instance): A work item created from a WEB-NEW pull request: 191
17.1 Screenshot (demo instance): A project with no mapped repository: 198
17.2 Screenshot (demo instance): Errors only (URL parameter errors=1): 201
17.3 Screenshot (demo instance): A failing check: 203
17.4 Screenshot (demo instance): GitHub refused the stored token: 205
20.1 Diagram: Plane holds planned work, GitHub holds code and issues, and git-sync joins selected work across them. 220
20.2 Diagram: two event directions through signed ingress, queues, reducers and the ordered outbox. 225
20.3 Diagram: runtime components and durable state. Shared infrastructure, separate database and queue namespace. 226
20.4 Diagram: the four connections. One installation belongs to exactly one workspace. 221
21.1 Diagram: installer stages — detection, safety proof, configuration and startup. 232
21.2 Screenshot (test Plane): the Sign up page with only an example bot address entered; nothing was submitted. 234
21.3 Screenshot (test Plane): Workspace settings → Members, where you invite the bot. 234
21.4 Screenshot (test Plane): the avatar menu with Settings. 236
21.5 Screenshot (test Plane): Profile settings → Personal Access Tokens. No token value is visible. 236
21.6 Screenshot (test Plane): the Create token dialog with an example title, cancelled afterwards. 237
21.7 Screenshot (test Plane): Workspace settings → Webhooks. The listed addresses are old test sinks, not the addre 239
21.8 Screenshot (test Plane): the Create webhook dialog filled with the internal URL and Send me everything (cancel 239
21.9 Screenshot (terminal): real installer output through a healthy start (from the disposable installer test; path 242
21.10 Screenshot (terminal): the next steps and the full future start command. 243
21.11 Screenshot (terminal): ./install.sh --help lists every option. 245

21.12 Screenshot (test Plane): Workspace settings → GitHub Sync with the embedded page loaded. 248
22.1 Illustration: the required permissions and nine event subscriptions. 258
23.1 Screenshot (test Plane): God mode navigation with GitHub Sync. 263
23.2 Screenshot (demo instance): Git sync: instance settings — 264
23.3 Screenshot (demo instance): a setup check that needs attention — 266
23.4 Screenshot (demo instance): GitHub App — 268
23.5 Screenshot (demo instance): GitHub edition — 268
23.6 Screenshot (demo instance): Workspace GitHub App installations — 270
23.7 Screenshot (demo instance): Disabled repository mappings — 271
23.8 Screenshot (demo instance): Manual mapping recovery — 272
23.9 Screenshot (demo instance): Magic words — 273
23.10 Screenshot (demo instance): Defaults — 274
24.1 Screenshot (test Plane): Workspace settings → GitHub Sync with the embedded page. 277
24.2 Screenshot (demo instance): a new workspace — 279
24.3 Screenshot (demo instance): Workspace GitHub connection — 280
24.4 Screenshot (demo instance): The workspace page: 281
24.5 Screenshot (demo instance): Send a link to an organization owner — 283
24.6 Screenshot (demo instance): Map a repository — 285
24.7 Screenshot (demo instance): Open and closed states — 286
24.8 Screenshot (demo instance): right after Add mapping — 287
25.1 Screenshot (demo instance): Mapping Settings tab — 292
25.2 Screenshot (demo instance): Conflict policies — 297
26.1 Screenshot (demo instance): Rules — 303
26.2 Screenshot (demo instance): Branch-aware rules — 304
26.3 Screenshot (demo instance): Rules (YAML) — 306
26.4 Screenshot (demo instance): an invalid rule is refused — 307
26.5 Screenshot (demo instance): Rule versions — 310
26.6 Screenshot (demo instance): Test rules — 311
26.7 Screenshot (demo instance): Dry-run result — 312
26.8 Screenshot (demo instance): Dry-run with no match — 313
26.9 Screenshot (demo instance): Testing unsaved rules: 314
26.10 Diagram: rule evaluation. The first matching rule wins; reference tiers gate only state changes. 300
27.1 Screenshot (demo instance): User links — 317
27.2 Screenshot (demo instance): Link a user — 318
27.3 Screenshot (demo instance): After Link account: 318
28.1 Screenshot (demo instance): Mapped repositories with sync health — 322
28.2 Screenshot (demo instance): Sync health while GitHub rate-limits the App — 323
28.3 Screenshot (demo instance): a disabled mapping — 323
28.4 Screenshot (demo instance): Audit log — 325
28.5 Screenshot (demo instance): Show raw — 326
28.6 Screenshot (demo instance): errors only (URL parameter errors=1) — 328
28.7 Screenshot (demo instance): Audit log filtered by URL parameters (mappingId=1, hours=24): 329
28.8 Screenshot (demo instance): Instance audit across all workspaces — 330

28.9 Screenshot (demo instance): Instance audit by default: 331
29.1 Screenshot (terminal): the health endpoint of a running service. 333
29.2 Screenshot (demo instance): Replay — 337
29.3 Screenshot (demo instance): a failed write — 338
30.1 Screenshot (demo instance): stored App credentials that cannot be decrypted (GITSYNC_TOKEN_KEY changed) — 344

00 · Introduction

Your team keeps its plans in **Plane** and its code in **GitHub**. **git-sync** is the service that connects the two. It reads Plane work-item keys such as `WEB-42` in branches, pull requests and commits. It then adds development links to the work item, moves it through the workflow your team configured, and can mirror issues between the two tools.

This manual explains how to use all three together, from your first sign-in to your next merged pull request. It also explains how administrators install and run git-sync.

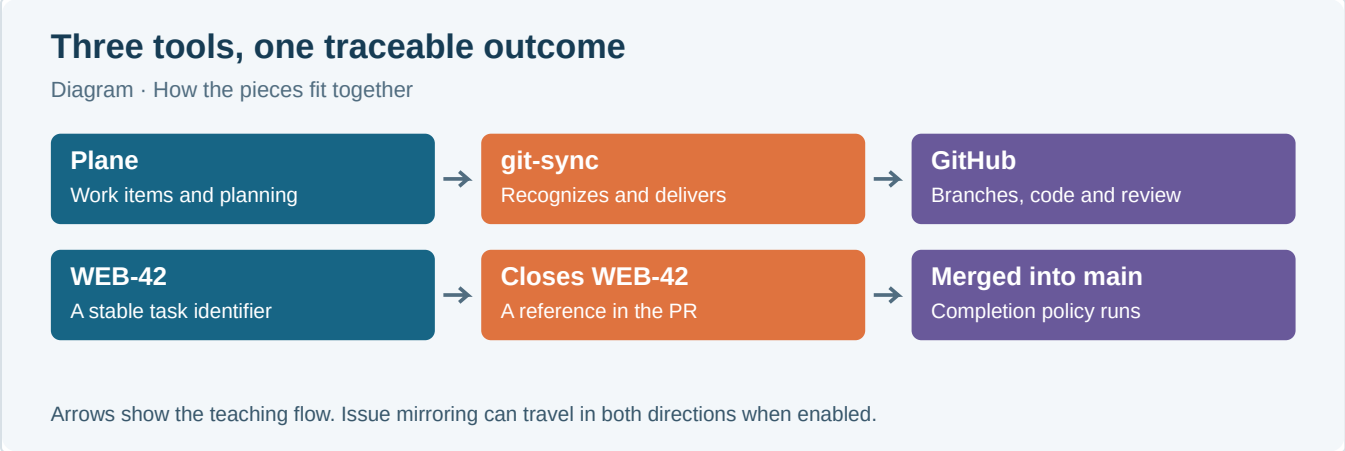


Figure 0.1 Diagram: Plane holds tasks, GitHub holds code, and git-sync connects them through identifiers and events.

Who this manual is for

Part A — Developer guide (chapters 1–19) is written for a developer who has just joined a team that uses Plane for tasks and GitHub for code. You do not need to install anything on a server. After reading Part A you should be able to do all of the following without asking anyone:

- Understand Plane's building blocks (workspace, project, work item, cycle, module, view, page, intake) and what git-sync adds to them.
- Sign in, set up your profile and find and work on your tasks every day.
- Connect your personal GitHub account, and tell apart the four different "connections" git-sync uses.
- Link branches, commits and pull requests to work items, use magic words and Smart Commits correctly, and read the **Development** panel.
- Know exactly what synchronizes, in which direction, and what never does.
- Fix common problems yourself, and send an administrator a report they can act on.

Part B — Admin guide (chapters 20–30) is for workspace administrators and instance (God mode) administrators. It covers installation, registering the GitHub App, mapping repositories, rules and conflict policies, user links, sync health, the audit log and day-to-day operations.

The *appendices* are reference material: every route, setting and environment variable; the exact grammar of references, magic words and Smart Commits, with valid and invalid examples; the full rule reference; known limitations; and the exact version this manual describes.

How to read it

If you are new, read chapters 1–3 before you change anything. Then read chapter 8 (connections and your GitHub account) and chapter 15 (a day in the life), which shows the whole loop end to end with one screenshot per step. Use the other chapters as reference when you need them.

If you already know Plane, skip to chapter 8. If you only need a reminder, use the one-page cheat sheet in chapter 19. It is also built as a separate printable PDF.

Most procedures follow the same pattern:

- **Goal** — what you will achieve.
- **Before you start** — what must already be true, such as your role or an existing mapping.
- **Steps** — numbered actions. Where a screenshot has orange numbered badges (callouts), the step numbers match the badges.
- **Result** — what you should see, usually with a screenshot.
- **If it went wrong** — the common symptoms, their causes and their fixes.

Boxes highlight special information:

> **Tip:** a useful habit or shortcut.

> **Note:** extra detail that helps you understand a behavior.

> **Warning:** a consequence to think about before you continue.

> **Example:** sample wording or a worked scenario.

> **Admin only:** a step that needs a workspace or instance administrator.

On-screen labels are shown in **bold** exactly as they appear, for example **Workspace settings → GitHub Sync**. Keys, branch names, commands and file paths are shown in `code`. Every chapter ends with **What you learned**, a short summary, and **Source files**, a list of the repository files that implement the behavior. Source files are there for maintainers and curious readers; you do not need to open them.

The example team

All examples use one fictional team. Names, addresses and data are invented.

Thing	Name in this manual
Plane workspace	**Acme** (address slug `acme`)
Plane project	**Website**, identifier `WEB`
Example work item	`WEB-42` *Make the checkout button visible on small screens*
GitHub repository	`acme/website`
Developer	Alex Developer (GitHub `alex-dev`)
Reviewer	Sam Reviewer (GitHub `sam-reviews`)
Workspace administrator	Jordan Admin (GitHub `jordan-acme`)
A brand-new member	Riley New (not yet connected to GitHub)
Plane address	`plane.example.com`

About the screenshots

The manual uses three kinds of figures. The caption always tells you which kind you are looking at.

Screenshot (test Plane) and **Screenshot (GitHub)** are real browser captures made for the earlier developer handbook, on a running Plane Community Edition v1.4.2 test instance and on GitHub. They show Plane's own screens: sign-in, profile, work items, layouts, cycles, modules, pages, intake and the inbox. Visible addresses, emails and unrelated test data were replaced with the fictional values above before capture; the underlying records were not changed.

Screenshot (demo instance) figures show git-sync's own pages: the **Development** panel, **My GitHub account**, the workspace **Git sync** settings and God mode's **Git sync: instance settings**. No signed-in browser sessions for the test system were available for this edition. These pages were therefore captured from a *demo instance*: the real git-sync service and the real git-sync user interface, built from this repository and run locally on a throwaway database. The demo data was produced by feeding realistic GitHub and Plane webhook events through git-sync's real webhook endpoints, handlers, rules and write paths. Only Plane's and GitHub's APIs were replaced by small stand-ins that answer the way those services do. The screens, labels, badges, states, error messages and audit entries are what the code really produces. The demo instance shows the git-sync pages on their own, outside Plane's surrounding window. In a real installation, administrators see the same pages embedded in Plane's settings, and members see the Development panel inside the work item's sidebar.

Diagram and **Illustration** figures are drawings. A diagram explains a flow or a concept. An illustration is a drawing of a screen we could not capture, such as GitHub's App registration form; it is never presented as a screenshot.

Some places show a marker like this:

> **Screenshot needed** — example: a real capture that is still owed. Capture later with `npm run manual:capture -- --only <id>`.

A marker means a real screenshot of that exact screen is still owed, usually because it needs a signed-in GitHub or Plane browser session. The text around the marker still describes the behavior accurately from the code and the demo run. Administrators can capture the missing images later with the command shown in the marker.

Orange numbered badges in a screenshot point at the controls discussed in its caption and steps. They were added to the image; they are not part of the product.

Version

This manual describes *git-sync 0.1.0* running with **Plane Community Edition v1.4.2** and the patched Plane web and admin images that git-sync's installer builds. The exact repository commit is printed on the cover and in Appendix F. Where older documentation disagrees with the code, this manual follows the code. The differences are listed in Appendix E.

> **Note:** Plane Community Edition has no built-in GitHub integration; that feature is part of Plane's commercial editions. git-sync is a separate, self-hosted service that adds one to Community Edition. When this manual says "git-sync does X", it means this service, not a feature of Plane itself.

What you learned

- Part A is for developers using Plane and GitHub every day; Part B is for administrators; the appendices are reference.
- Procedures follow Goal → Before you start → Steps → Result → If it went wrong, and step numbers match the orange callouts.
- Figures are real test-Plane/GitHub captures, demo-instance captures of git-sync's real UI, or labelled drawings.
- A `[[SCREENSHOT NEEDED]]` marker shows a capture that is still owed; the text beside it is still accurate.

Source files

- `package.json` (git-sync version)
- `src/ui/client/` (the git-sync pages shown in demo-instance screenshots)
- `manual/demo/server.ts`, `manual/demo/seed.ts`, `manual/demo/events.ts` (the demo instance)
- `docs/handbook/HANDBOOK_NOTES.md` (provenance of the test-Plane screenshots)

PART A

Developer guide

Everything a developer needs to plan work in Plane, connect it to code on GitHub, and let git-sync keep the two in step.

01 · Welcome

Your team keeps its plans in Plane and its code in GitHub. This chapter explains the handful of Plane ideas you meet every day. It then explains, at a high level, what git-sync adds on top of Plane and where to read more. It ends with a one-page checklist for your first day.

You do not need to install anything on a server. You need an invitation to the workspace, membership of the right project, and your usual development tools.

1.1 Plane in one page

Plane is a shared place to describe and organize work. These are the ideas you will meet first.

Idea	What it is	Example
Workspace	The broad team area you are invited to. Everything else lives inside one workspace.	**Acme**
Project	A focused area of work inside the workspace, with its own identifier, states, labels and members.	**Website** (`WEB`)
Work item	One record of an outcome the team wants: a bug to fix, a feature to add, something to investigate or write.	`WEB-42` *Make the checkout button visible on small screens*
State	Where a work item is in the workflow. Each state belongs to a **state group** (Backlog, Unstarted, Started, Completed, Cancelled, plus Triage for intake).	Todo, In Progress, In Review, Done
Cycle	A time window for planning delivery; some teams call it a sprint.	**October customer experience**
Module	A collection of work around a feature area or larger outcome, possibly spanning several cycles.	**Checkout experience**

Idea	What it is	Example
View	A saved selection of work items with filters, grouping and layout.	**Website delivery overview**
Page	A shared document for knowledge that outlives a single task.	**Website working agreement**
Intake	An inbox for new requests that the team has not yet accepted into its plan.	A pending bug report waiting for triage

Each work item has a short **identifier**, also called its **key**. In `WEB-42`, `WEB` is the project identifier and `42` is the item's sequence number in that project. The title can change over time; the identifier stays the same. People use it in conversation, and git-sync uses it to connect code to the task.

These ideas are independent of each other. Changing from a list to a board does not move work to another project. Applying a filter does not change the items it hides. Adding an item to a cycle does not copy it.

> **Example:** "WEB-42 is In Progress, assigned to Alex, in the October cycle" gives four separate facts. The identifier selects the record, the state describes the workflow, the assignee names who is responsible, and the cycle names a planning window.

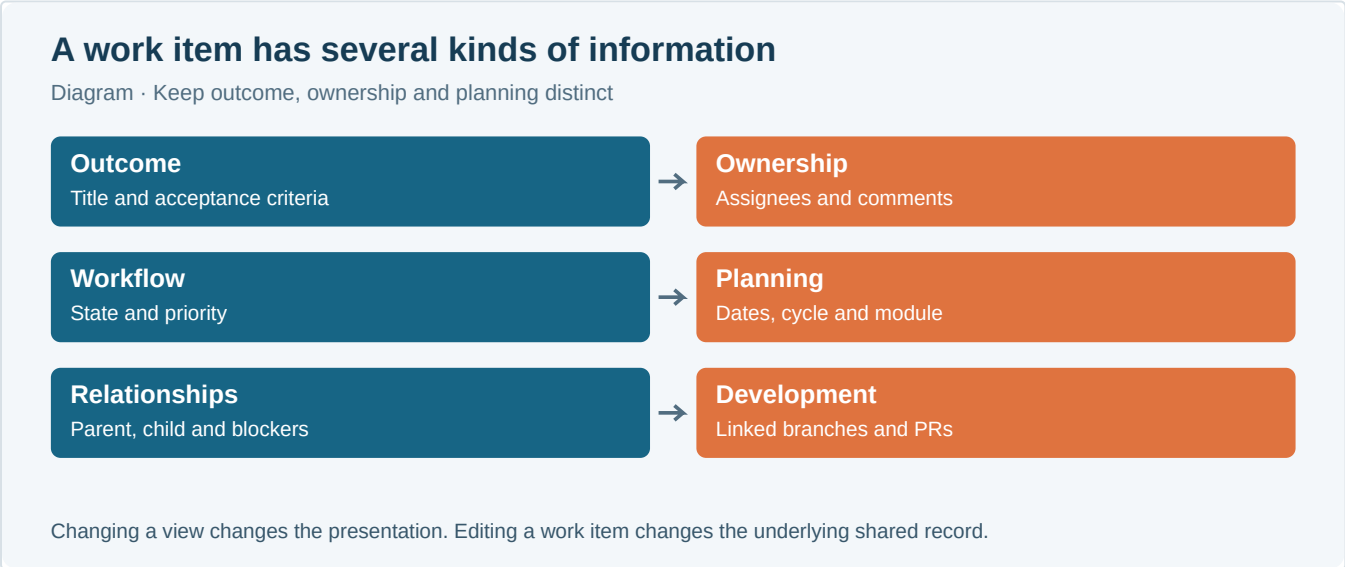


Figure 1.1 Diagram: outcome, ownership, workflow, planning, relationships and development are separate parts of a work item.

1.2 Git, GitHub and pull requests in one paragraph

Git is the version-control tool on your computer. It records changes as **commits** and lets you work on a **branch** without changing the team's main branch. GitHub hosts the shared repository and the discussion of proposed changes. That discussion happens in a **pull request**, or **PR**. A PR asks to merge a branch (the **head branch**) into another branch (the **base branch**, often `main`). Reviewers can comment, request changes or approve, and automated checks (**CI**) run on the PR's commits. Merging the PR puts its changes into the base branch.

1.3 What git-sync adds

Plane Community Edition v1.4.2 has no GitHub integration of its own. **git-sync** is a separate self-hosted service that your administrators installed next to Plane. It listens to events from both GitHub and Plane and does the bookkeeping that would otherwise mean copying information between two tools by hand.

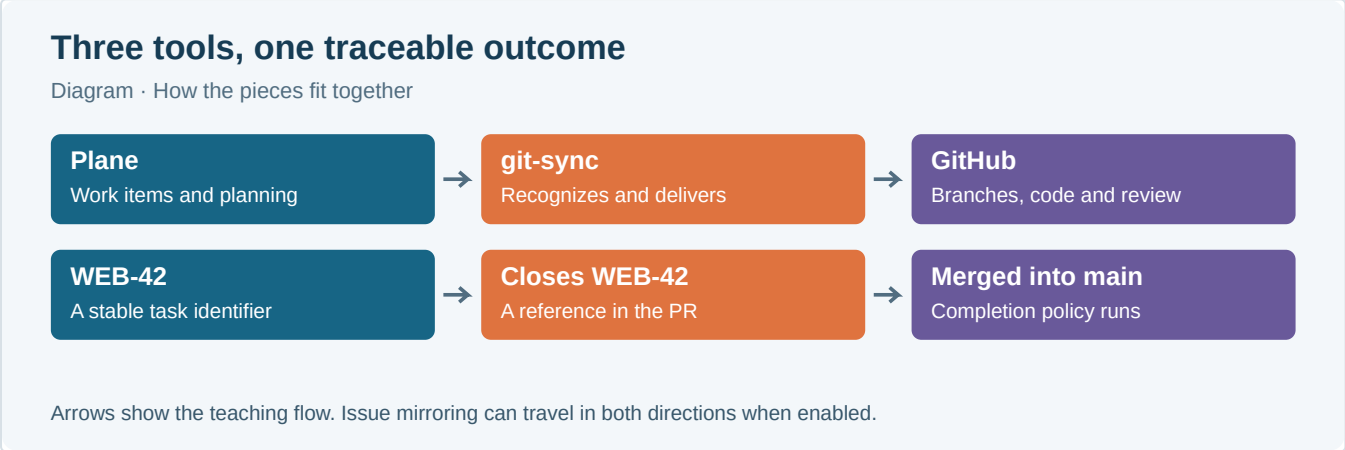


Figure 1.2 Diagram: Plane holds tasks, GitHub holds code, and git-sync connects them through identifiers and events.

At a high level, git-sync adds six things.

1. **Links from code to work items.** When a branch name, PR title or PR body contains a work-item key such as `WEB-42`, git-sync links that branch or PR to the work item. Words before the key, called **magic words**, decide how strong the link is: `Closes WEB-42` means "this PR finishes the item", `Refs WEB-42` means "this PR contributes to it", `Relates to WEB-42` means "this is useful context". See chapters 9 and 10.
2. **A Development panel on each work item.** The work item's sidebar gains a **Development** section. It shows the linked pull requests with their status and review badges, CI builds, deployments, preview links and branches. Before any code exists, it offers **Copy branch name** and **Create branch**. See chapter 12.
3. **Automatic workflow states.** With the shipped rules, pushing a new branch or opening a PR moves the item to **In Progress**, requesting a review moves it to **In Review**, and merging into `main` moves it to **Done**. Administrators can change these rules per repository. See chapter 14.

4. **Smart Commits.** A commit message such as `WEB-42 #comment Checked on small phones` or `WEB-42 #time 1h 30m` adds a comment to the work item. A command such as `#in-review` changes its state. See chapter 11.
5. **Issue mirroring.** A GitHub issue labelled `plane` gets a matching Plane work item; a Plane work item labelled `github` gets a matching GitHub issue. Title, description, state, assignees, labels and new comments then stay in step in both directions. See chapter 13.
6. **Your own GitHub identity.** You can connect your personal GitHub account, so that changes git-sync makes on GitHub for you appear under your name. See chapter 8.

Chapter 15, *A day in the life*, puts these together. It follows `WEB-42` from **Create branch** to merged PR, with a screenshot of the Development panel after every step.

git-sync is not a replacement for reading the task, testing your change or asking for review. It only records and moves information. If a link or a state looks wrong, the chapters above explain why it happened and chapter 17 explains how to fix it.

> **Note:** git-sync only works for repositories that an administrator has **mapped** to a Plane project. A key in a repository that is not mapped does nothing. Chapter 8 explains the four connections that must all be in place.

1.4 Who sets what up

Most of the setup is done for you before you arrive.

Already arranged by administrators	Done by you
Plane installed, sign-in configured	Accept the invitation and sign in
git-sync installed and the GitHub App registered	Set your profile, preferences and notifications
GitHub App installed on the organization and connected to the workspace	Join your project
Repository mapped to the Plane project, rules and policies chosen	Connect your personal GitHub account, if your team uses it
Optional: your GitHub account linked to your Plane account by an admin	Use keys, magic words and Smart Commits in your daily work

If installing or configuring the server is your job, read Part B. Never paste a server token, a Plane personal access token or a GitHub secret into a work item, a commit, a screenshot or a support message.

1.5 How to use this part of the manual

Chapters 2–7 teach Plane itself: getting in, finding your way, work items, layouts, planning and notifications. Chapters 8–15 teach the GitHub connection. Chapters 16–19 give team conventions, troubleshooting, answers to common questions and a printable cheat sheet.

Some actions need a project or workspace administrator. If your menu has fewer entries than a screenshot, first check your role and project membership before assuming something is broken.

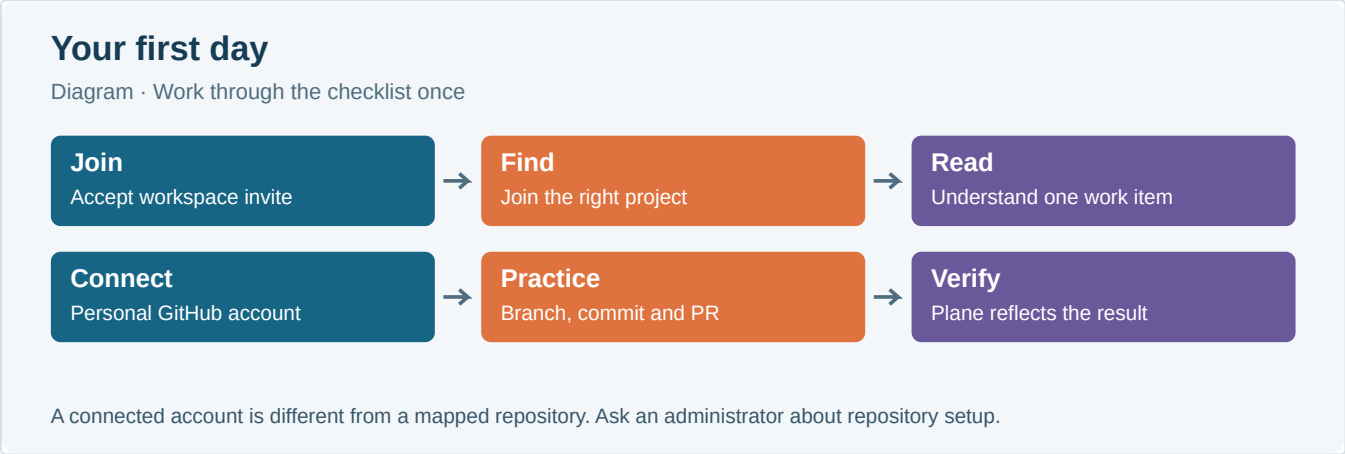


Figure 1.3 Diagram: join, find, read, connect, practice and verify form the first-day journey.

1.6 Your first day: one-page checklist

Work through this list with a teammate. Tick a box only when you have checked the result, not merely opened the menu.

- [ ] Accept the invitation to the intended workspace (chapter 2).
- [ ] Sign in with your own Plane account.
- [ ] Set your name and a recognizable display name.
- [ ] Check your timezone in **Preferences**.
- [ ] Choose a theme and notification settings that suit you.
- [ ] Join the project that contains your team’s work.
- [ ] Find **Home**, **Your work**, **Drafts** and the inbox (chapter 3).
- [ ] Open the command palette and the shortcuts dialog once.
- [ ] Read one assigned work item completely (chapter 4).
- [ ] Identify its state, priority, owner and any blockers.
- [ ] Open **My GitHub account** for the workspace (chapter 8).
- [ ] Connect your own GitHub account if your team uses it.
- [ ] Check that the work item’s **Development** panel shows the mapped repository (chapter 12).

- [ ] Practice the branch, commit and PR loop (chapter 15).
- [ ] Confirm that the merged PR and the completed work item agree.

Ask your onboarding teammate three questions, and write the answers somewhere shared:

1. Which project view is the team's source for choosing work?
2. Which state means "ready for review"?
3. Should a PR *finish* its referenced item (`Closes`) or only *contribute* to it (`Refs`)?

These are team decisions, not product settings. Chapter 16 suggests sensible defaults.

> **Tip:** Keep `WEB-42`, its Plane link, the repository name and your branch name together in a scratch note while you work. The identifiers help you find your place again when many tabs are open.

What you learned

- A workspace contains projects; a project contains work items, each with a stable key such as `WEB-42`.
- States, cycles, modules, views, pages and intake answer different planning questions and are independent of each other.
- git-sync is a separate service that links branches and PRs to work items, adds a Development panel, moves states by rule, runs Smart Commits, mirrors labelled issues and can act as you on GitHub.
- Nothing happens for a repository that an administrator has not mapped to a Plane project.

Source files

- `docs/superpowers/specs/2026-10-10-plane-github-sync-design.md` (design of git-sync)
- `src/parse/references.ts`, `src/app/pr.ts`, `src/app/push.ts` (linking)
- `src/ui/client/DevPanel.tsx` (Development panel)
- `src/github/smart-commits.ts` (Smart Commits)
- `src/sync/mirror.ts` (issue mirroring)

02 · Getting in

Three separate steps put you in front of your team’s work. An **invitation** gives your account membership of a workspace. **Signing in** proves which account you are using. **Joining a project** gives you a place in that project’s work. They can happen at different times, so signing in successfully does not mean every project is already in your sidebar.

This chapter walks through all three, then shows how to set your profile, preferences and email notifications.

2.1 Accept the invitation

Goal: become a member of the team’s workspace

Before you start: your team has sent you an invitation link by email, or told you to open Plane and look for a pending invitation. Use the Plane address your administrator gave you; the examples use `plane.example.com`. If you have accounts at more than one organization, check which email you are signed in with.

Steps

1. Open the invitation link from your team.
2. Sign in, or create your own account when Plane asks.
3. Select the checkbox for the intended workspace. ①
4. Select **Continue**. ②

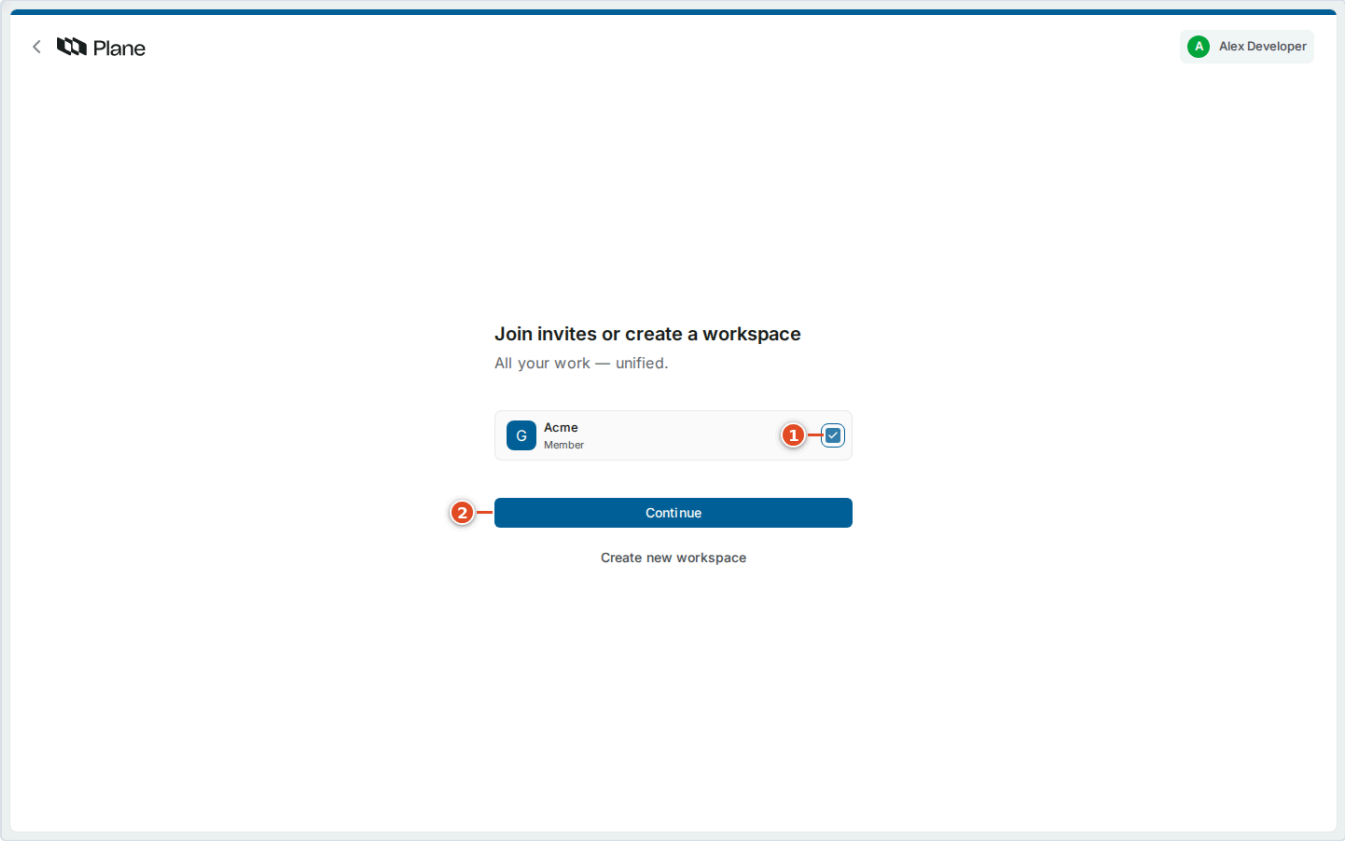


Figure 2.1 Screenshot (test Plane): Accept the invitation: (1) select the workspace checkbox; (2) Continue.

Result: Plane opens the workspace. The first-time invitation screen shows the workspace name and your membership role. Select the checkbox itself; clicking the text beside it is not reliable. **Continue** stays disabled until an invitation is selected. An existing account may reach the invitation through **Workspace invites** instead of this onboarding screen.

If it went wrong:

- *No invitation is listed* → the invitation was sent to a different address, perhaps an alias. Ask the inviter to check the exact email. Do not create a new workspace just to get past onboarding.
- *An old link fails* → open the workspace invitations screen, or ask for a fresh invitation.
- *You joined the wrong workspace* → use the workspace switcher (chapter 3) and tell the inviter.

2.2 Sign in

The test instance uses email and password. Your administrator may enable other sign-in methods; use whichever methods your sign-in page shows. Signing in to Plane is separate from authorizing git-sync on GitHub (chapter 8). Having one does not give you the other.

Goal: sign in with your own account

Steps

1. Open your team's Plane address.
2. Enter your account email. ①
3. Select **Continue**. ②
4. Enter your password when asked, and submit the form.

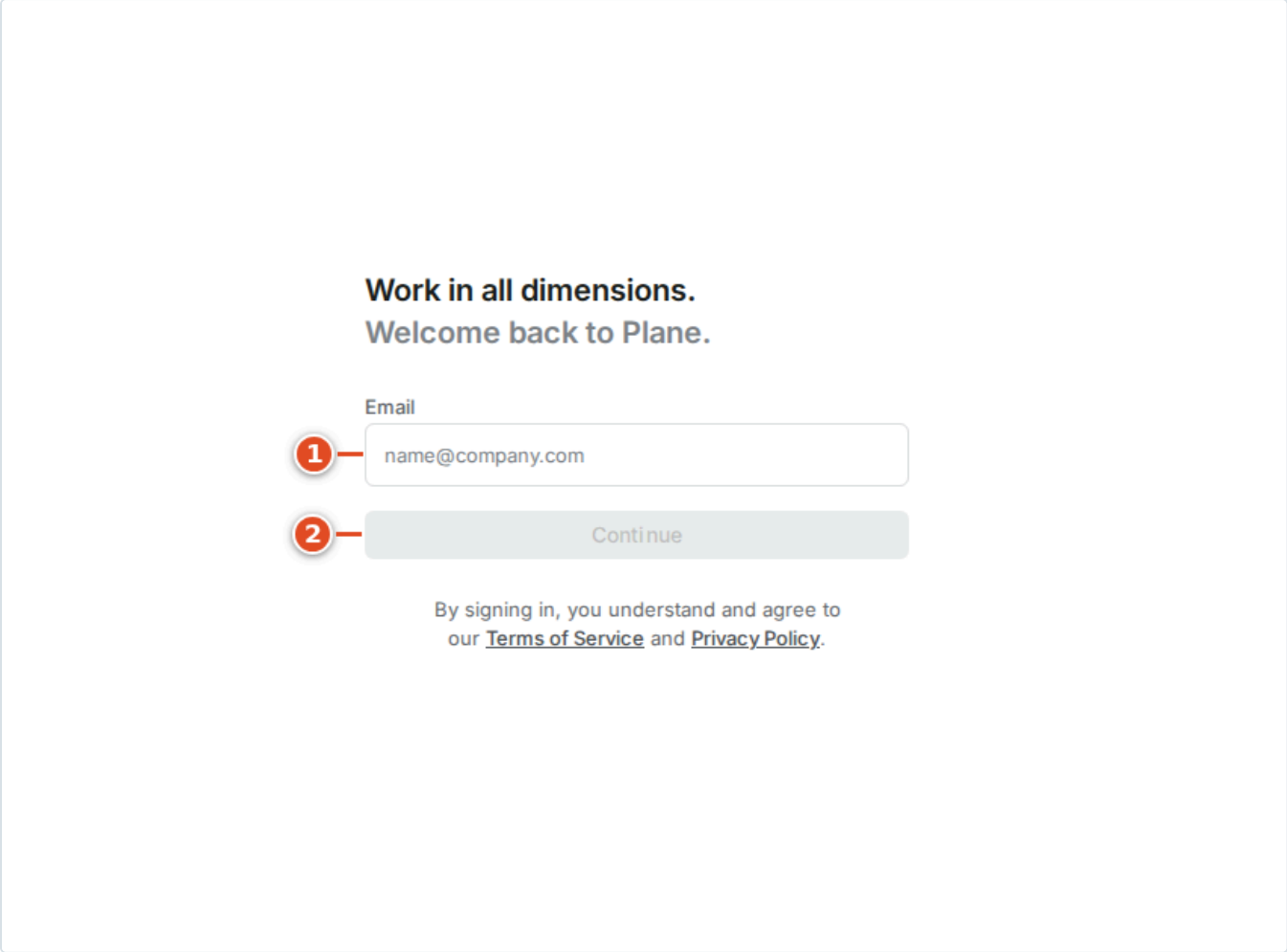


Figure 2.2 Screenshot (test Plane): Sign in: (1) email field; (2) Continue. The next screen asks for a password.

Result: you see your workspace's Home page.

If it went wrong:

- *Wrong password* → use the password-recovery control your instance offers. Never send a password to a teammate.
- *No recovery email arrives* → ask an administrator whether outgoing email is configured. The handbook's test instance had no mail server, so email delivery was not verified there. A success message on screen is not proof that a mailbox received anything.

> **Warning:** git-sync uses a separate **service account** (a bot user) to write to Plane. It deliberately ignores changes made by that account, to avoid sync loops. Never use the service account for your normal work: your edits would look ignored.

2.3 Set your profile

Your profile helps teammates recognize who is working and who is commenting. Your **full name** is your human-readable identity. Your **display name** is the shorter name shown in many places. Your **avatar** is the small picture or initials shown beside assignments and comments.

Goal: make yourself recognizable

Steps

1. Select your avatar in the top-right corner. ①
2. Select **Settings**. ②
3. Select **Profile** if another section is open.
4. Update **First name** and, if you use one, **Last name**.
5. Update **Display name**.
6. Select **Save changes**.

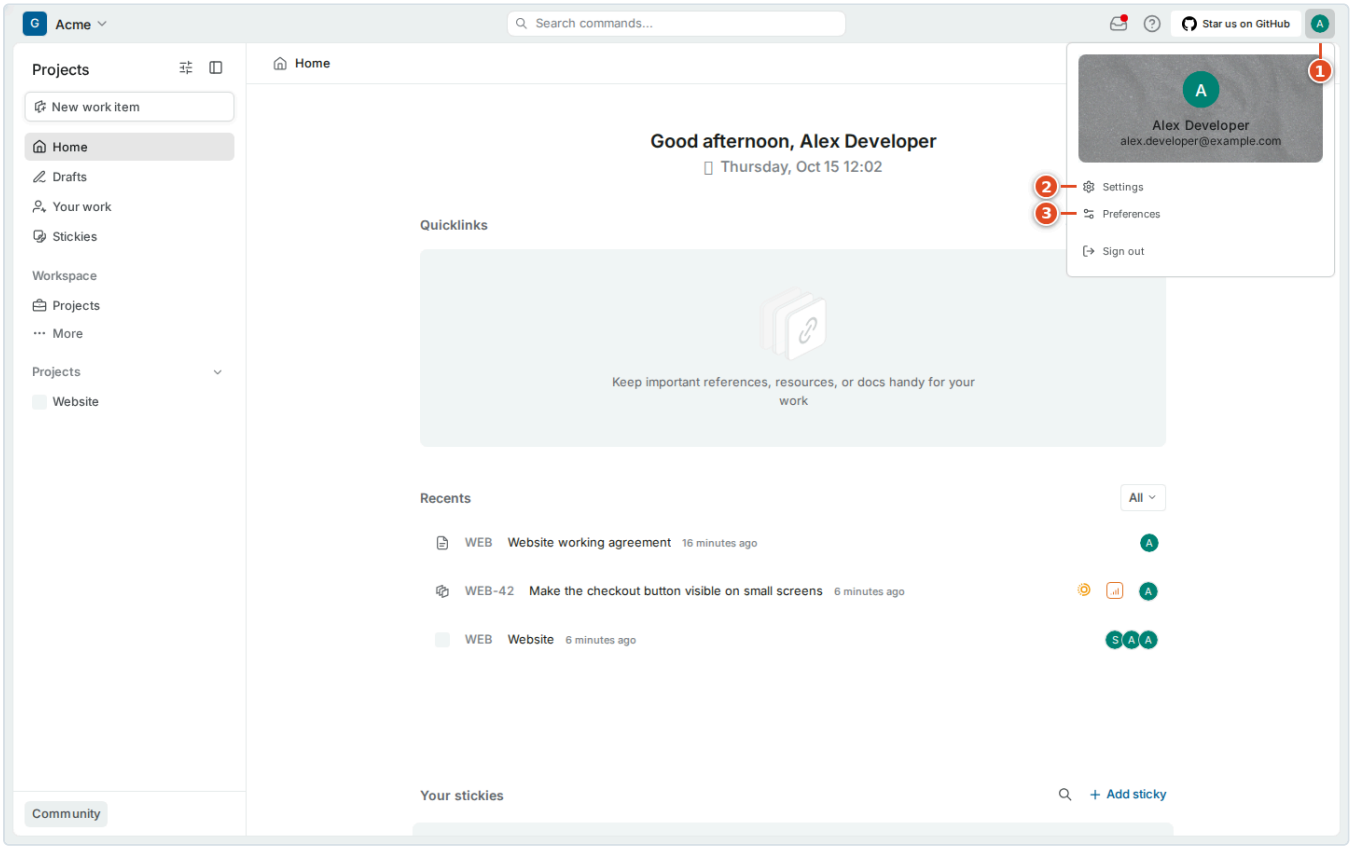


Figure 2.3 Screenshot (test Plane): Personal menu: (1) avatar; (2) Settings; (3) Preferences.

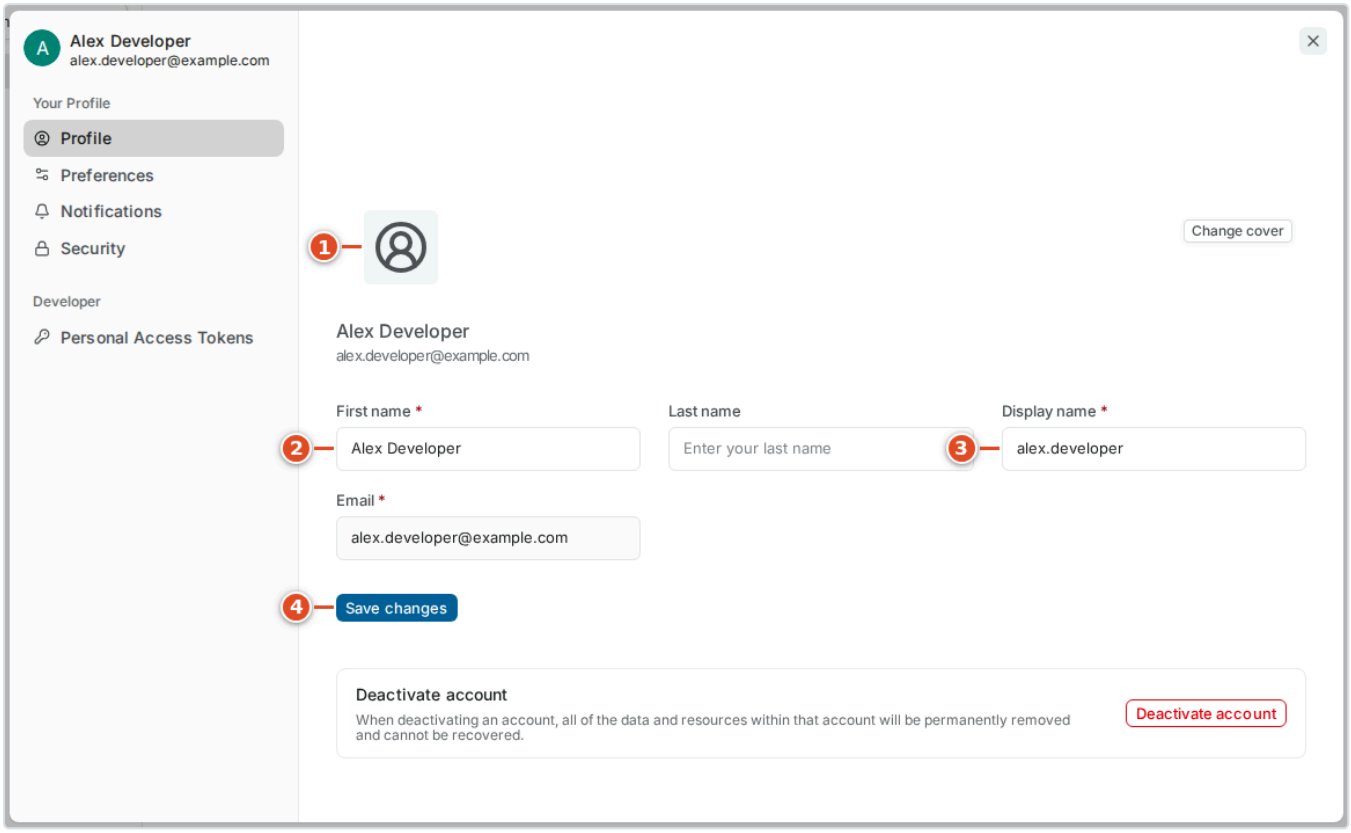


Figure 2.4 Screenshot (test Plane): Profile: (1) avatar; (2) name; (3) display name; (4) Save changes.

Result: your new name appears beside your comments and assignments.

In Plane CE v1.4.2 the profile editor opens as a dialog over the current page, so you can close it and carry on where you were. Use the avatar control (① in the profile screenshot) to upload a picture, or keep the initials. Do not upload images that contain customer data or credentials.

The profile also shows your email address. Changing it can affect future invitations and how you sign in, so ask your administrator about the team's account policy first. Do not create a second account to fix a display-name problem.

> **Tip:** Choose a display name that teammates can match to your GitHub username. It makes reviews and mentions easier to follow, and it helps an administrator who has to link your accounts by hand (chapter 27).

2.4 Set theme, timezone and first day of the week

Your **timezone** controls how dates and times are displayed to you. Your **theme** only changes how Plane looks to you; it changes no data and no other member's view.

Goal: make dates and times match where you work

Steps

1. Open your avatar menu and select **Preferences**.
2. Open the theme selector and choose a theme. ①
3. Open the timezone selector and choose your working timezone. ②
4. Set the first day of the week if the default differs from your calendar. ③

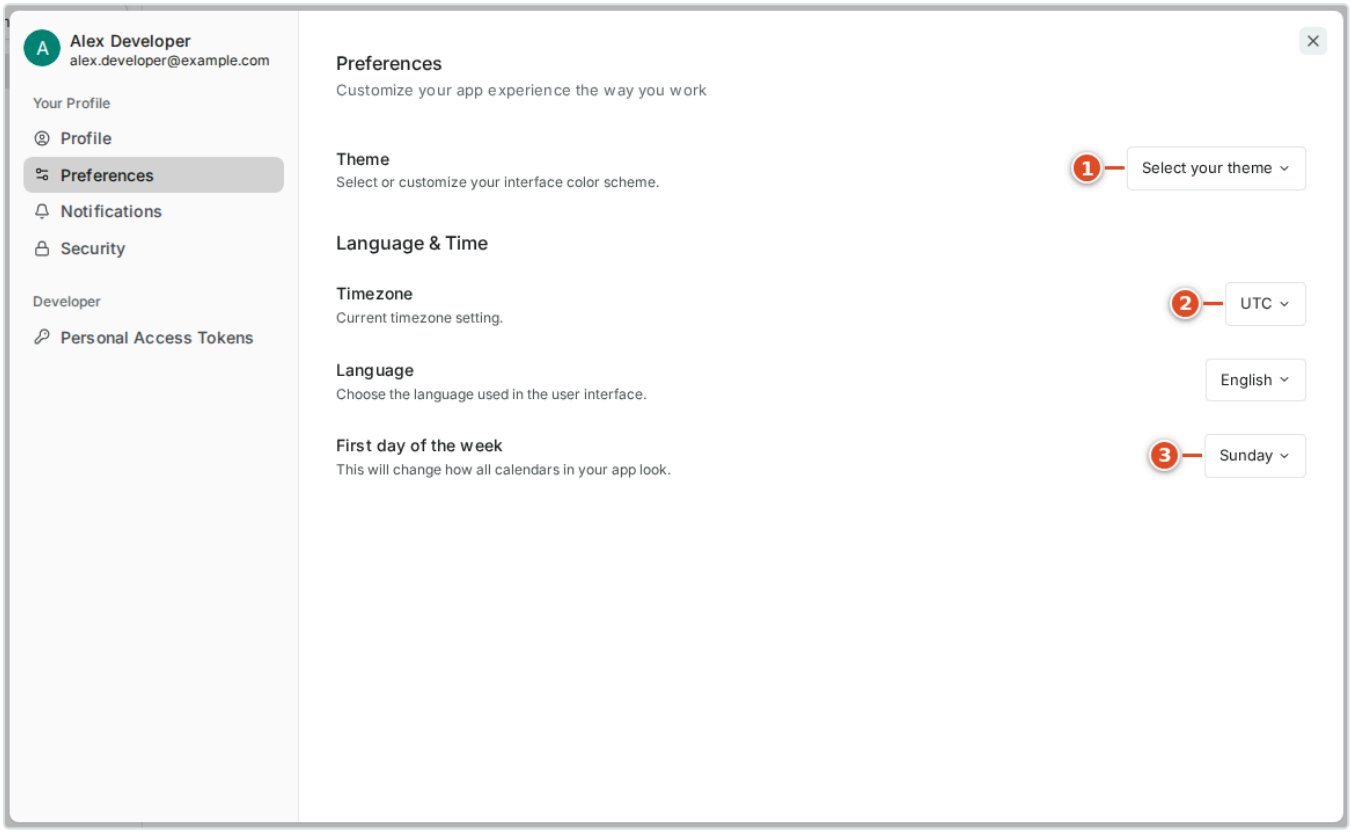


Figure 2.5 Screenshot (test Plane): Preferences: (1) theme; (2) timezone; (3) first day of the week.

Result: due dates and notification times are shown in your local time.

The same area has a language setting. This manual uses the English labels; control positions stay the same in other languages.

> **Tip:** If a meeting time or a deadline looks a few hours off, compare timezone settings before you change the work item's date. A display setting and a planning mistake need different fixes.

2.5 Set email notification preferences

A **notification** tells you that something relevant changed. A **subscription** says you want updates about a particular work item. **Email notification preferences** decide which of those updates also reach your mailbox. Turning off an email category is not the same as unsubscribing from an item or being unassigned.

Goal: get the emails that matter, not every small edit

Steps

1. Open personal **Settings** and select **Notifications**.
2. Review **Property changes**. ①
3. Review **State change**, **Work item completed** and **Comments**.
4. Review **Mentions**. ②

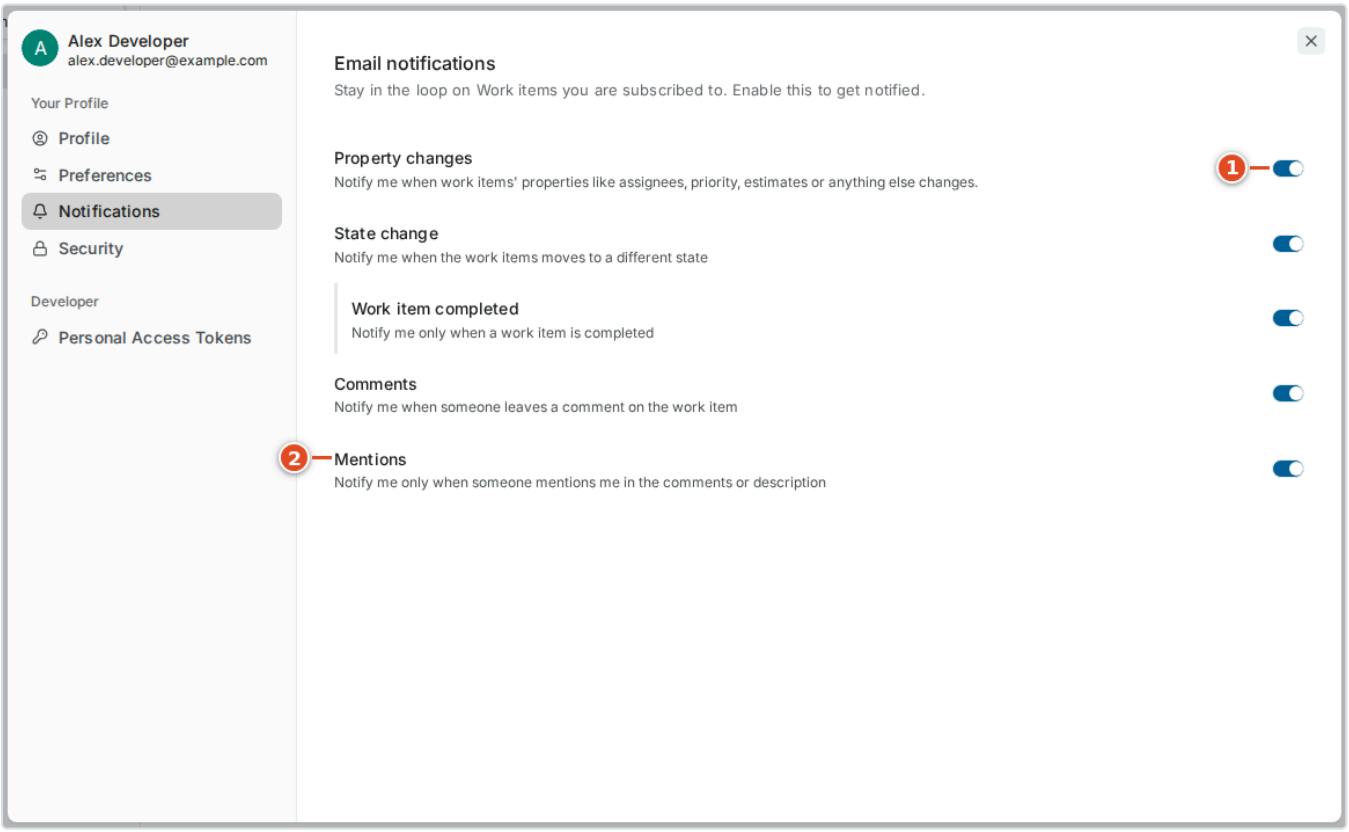


Figure 2.6 Screenshot (test Plane): Email notifications: (1) property-change switch; (2) Mentions category and its switch.

Result: Plane emails you only for the categories you left on. In-app notifications in the inbox continue regardless (chapter 7).

A good starting point is to keep **Mentions** and **Comments** on, and decide about **Property changes** after a few days of normal activity. git-sync's automatic updates (state changes, development links, Smart Commit comments) are ordinary Plane changes made by the service account, so they produce notifications like any other change.

If it went wrong:

- *An expected email never arrives* → first check that the change appears in Plane itself, then your categories, then your mailbox filters. If all three look right, ask an administrator about mail delivery. Do not change instance settings to debug your own notifications.

2.6 Join your project

The **Projects** page lists the projects you are allowed to discover. A project can be visible but still show a **Join** action because you are not a member yet. Private projects need an explicit invitation from a project administrator.

Goal: become a member of the Website project

Steps

1. Select **Projects** in the workspace navigation.
2. Find the project your teammate named.
3. Select its **Join** action.
4. Check the project name in the confirmation dialog. ①
5. Select **Join Project**. ②
6. Open **Work items** under the project in the sidebar.

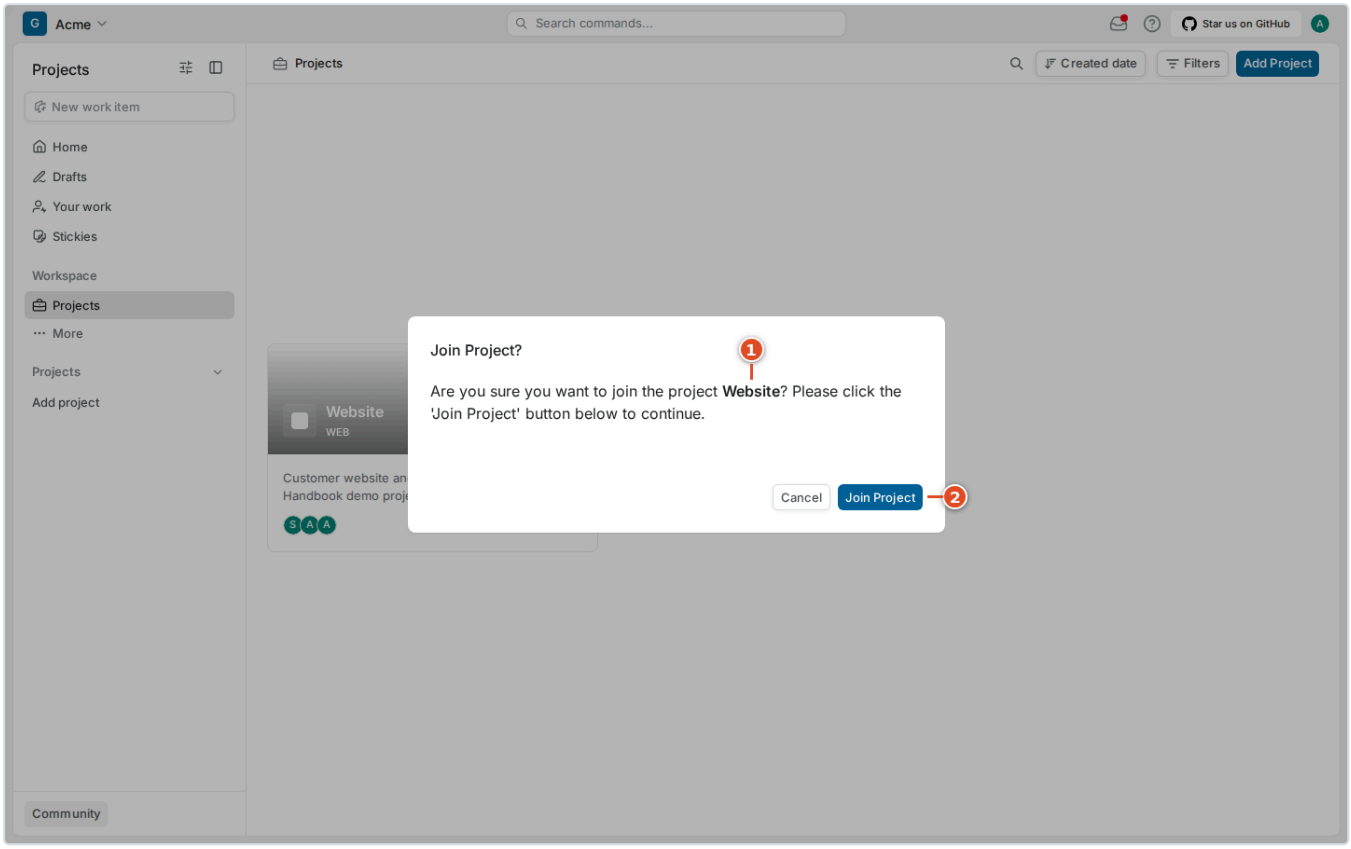


Figure 2.7 Screenshot (test Plane): Join Website: (1) confirm the project name; (2) Join Project.

Result: the project appears in your sidebar and you can see and edit its work items.

If it went wrong:

- *The project is not listed* → check the workspace switcher first; you may be in another workspace. Otherwise the project is private and you need an invitation.
- *You cannot join* → tell the project administrator which account you are using and what you see. Send the project name and the message, not a screenshot of your whole account settings.

> **Note:** Project membership matters for git-sync too. The **Create branch** button in the Development panel is only available to project members (chapter 12).

Joining the right workspace does not mean you should join every visible project. Start with the projects you actually work in.

What you learned

- Invitation, sign-in and project membership are three separate steps.
- Your profile, display name, timezone and theme are personal settings; they change nothing for others.
- Email preferences control which notifications reach your mailbox; they do not unsubscribe or unassign you.
- Never use git-sync's service account for normal work, and never share passwords or tokens.

Source files

- Plane CE v1.4.2 web application (`apps/web/`), observed in the running test instance
- `docs/handbook/HANDBOOK_NOTES.md` (screenshot provenance)
- `src/queue/workers.ts` (ingress filter: why the service account's own changes are ignored)

03 · Finding your way

Plane has two scales of navigation. **Workspace navigation** takes you to places that span the whole team: Home, Your work, Drafts, the inbox and the list of projects. **Project navigation** takes you to the work, plans and documents inside one project. Knowing which scale you are on prevents a common first-day mistake: looking for an item in a view that belongs to another project.

3.1 Read the screen from the outside in

- The **workspace switcher** at the top left names the current workspace.
- The **sidebar** holds Home, Drafts, Your work and the workspace's projects. Expanding a project shows its enabled features: Work items, Cycles, Modules, Views, Pages and Intake.
- The **main area** shows the selected page.
- The *top-right controls* include the inbox, help and your avatar.

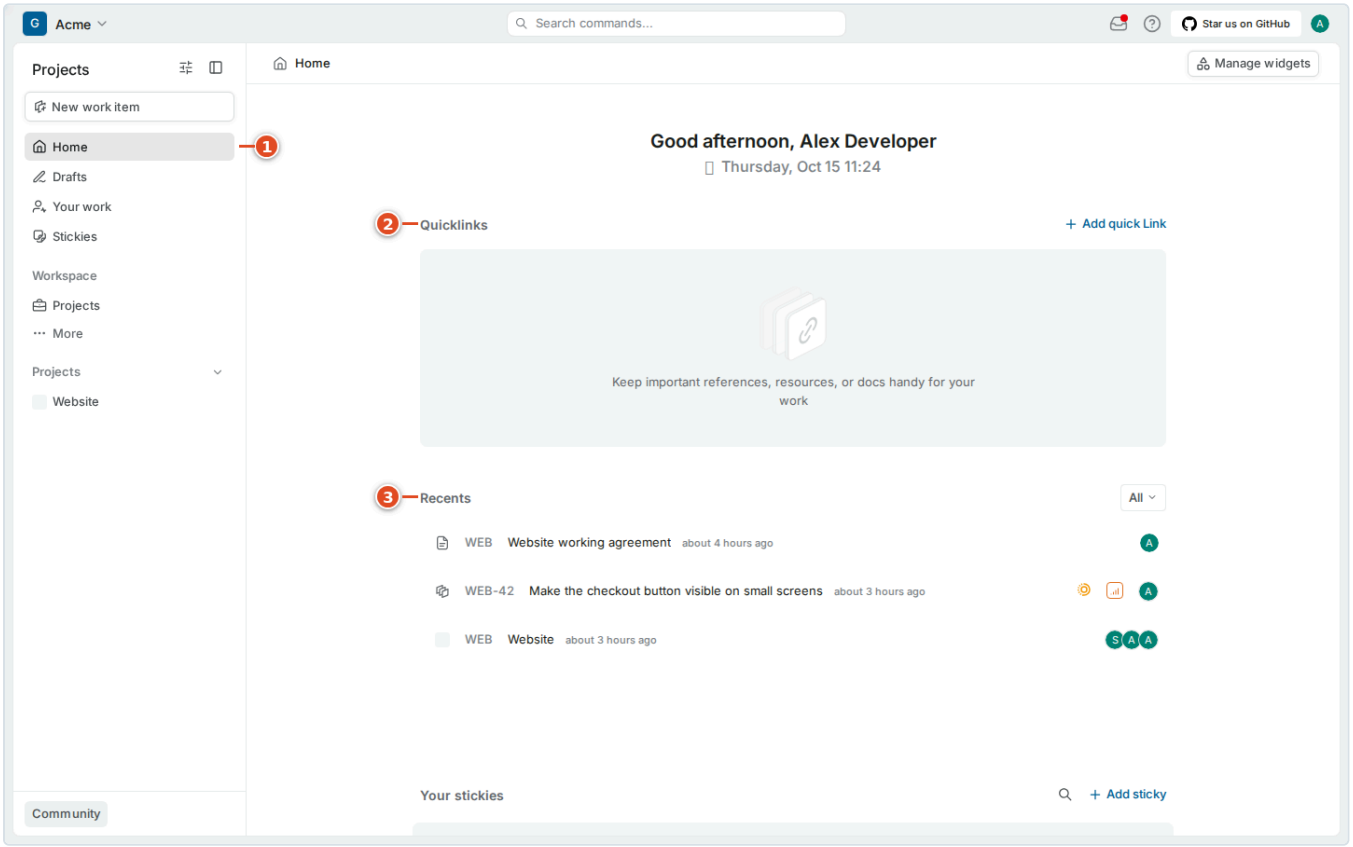


Figure 3.1 Screenshot (test Plane): Home: (1) sidebar Home; (2) Quicklinks; (3) Recents.

Goal: get oriented from Home

Steps

1. Check the workspace name at the top left.
2. Select **Home** in the sidebar. ①
3. Look at **Quicklinks** ② and **Recents** ③.
4. Before opening a recent item, check its project and identifier.

Result: you can jump back to what you were last working on.

Home is a convenient starting point, but it is not a complete list of your unfinished work, and your widgets can differ from a teammate's. Use **Your work** or a project view when you need a deliberate list.

Quicklinks keep useful destinations close at hand. A **sticky** is a personal note. Keep durable task information and acceptance criteria in a shared work item, not in a personal note that nobody else can find.

3.2 Switch workspace carefully

Goal: make sure you are in the right workspace before creating anything

Steps

1. Select the workspace switcher. ①
2. Check which workspace is selected.
3. Choose the workspace you need.

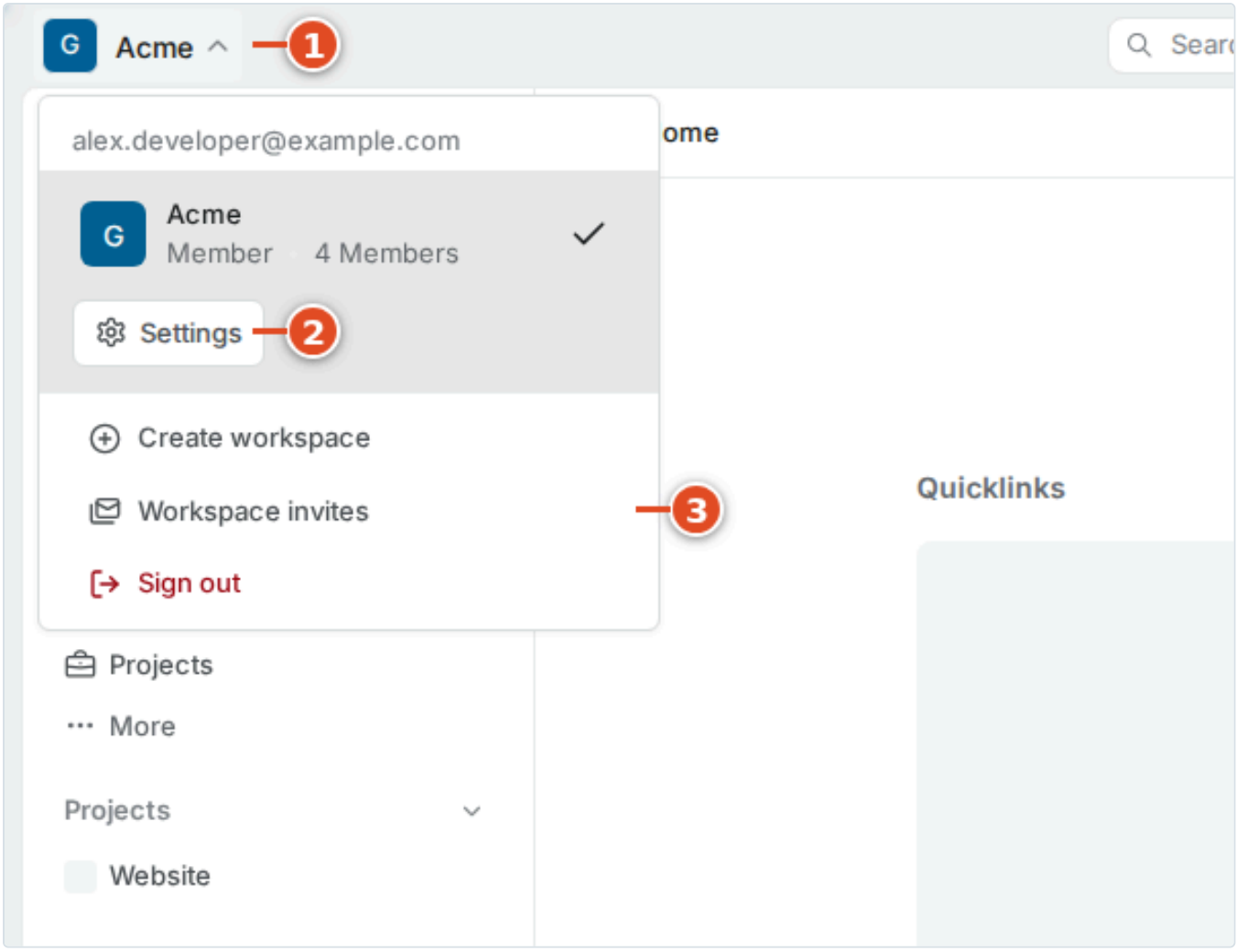


Figure 3.2 Screenshot (test Plane): Workspace switcher: (1) current workspace control; (2) Settings; (3) Workspace invites.

Result: the sidebar now shows the projects of the chosen workspace.

The same menu leads to workspace **Settings** ② and **Workspace invites** ③. Creating a new workspace makes a separate team area; it does not fix a missing project invitation. If you are in the wrong workspace, switch back before you create a work item.

> **Note:** An identifier such as `WEB-42` is only meaningful inside its own workspace. git-sync also resolves keys only inside the workspace that a repository is mapped to. The same key typed against a repository mapped to another workspace refers to a different item, or to none.

> **Admin only:** In a workspace's **Settings**, workspace administrators see a **GitHub Sync** entry. Ordinary members do not, so they reach their personal GitHub page through a direct link instead (chapter 8).

3.3 Find Your work and the project list

Your work is a personal page. It shows work related to you, such as items assigned to you, created by you or that you subscribed to, without scanning every project. Its filters can hide items, so check the selected category and filters before concluding that nothing is assigned to you.

Goal: see the work assigned to you

Steps

1. Select **Your work** in the sidebar.
2. Check the selected category, for example **Assigned**. ①
3. Check the **Display** controls and filters. ③
4. Open the item you want, here `WEB-42`. ②

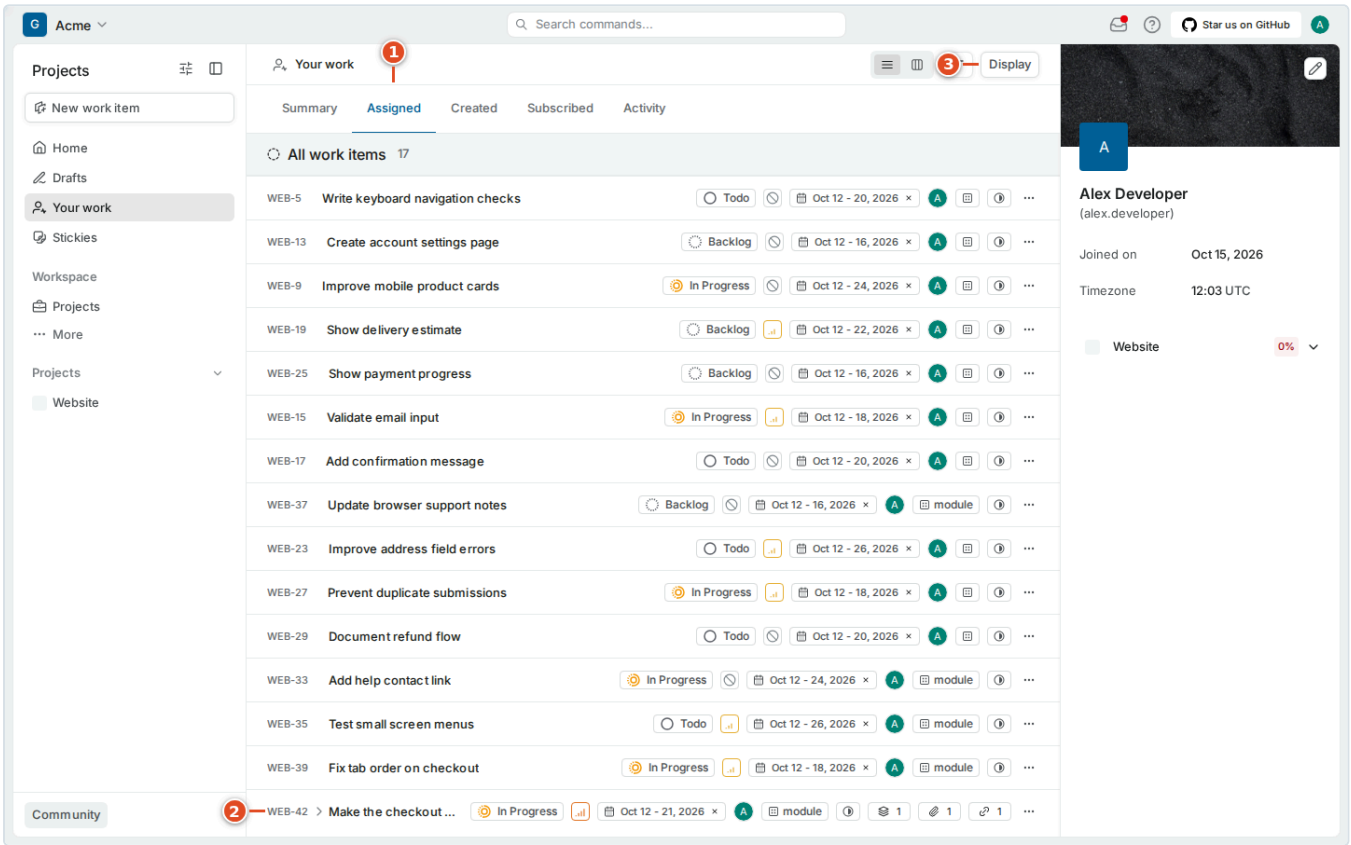


Figure 3.3 Screenshot (test Plane): Your work: (1) Assigned category; (2) matching item WEB-42; (3) Display controls.

Result: you see your assigned items across projects.

Goal: open a project’s work items

Steps

1. Select **Projects** under the workspace.
2. Find **Website** and open it.
3. Select **Work items** in its expanded navigation.

Result: you see the project’s work items in its current layout (chapter 5).

A project administrator can turn off optional features. If **Cycles** or **Intake** is missing from a project’s navigation, the team may simply not use it. Ask before assuming the installation is broken. The Community and billing pages are not evidence that a paid feature is included in your edition.

3.4 Use the command palette

The **command palette** is a searchable menu of destinations and actions. Use it when you know the name of something but not which menu contains it.

Goal: find an action from the keyboard

Steps

1. Press **Ctrl + K** (Windows, Linux) or **Cmd + K** (macOS).
2. Type part of the action or item you want into the search field. ①
3. Read the highlighted result, for example a creation action. ②
4. Press **Enter** to run it, or **Escape** to close the palette without acting.

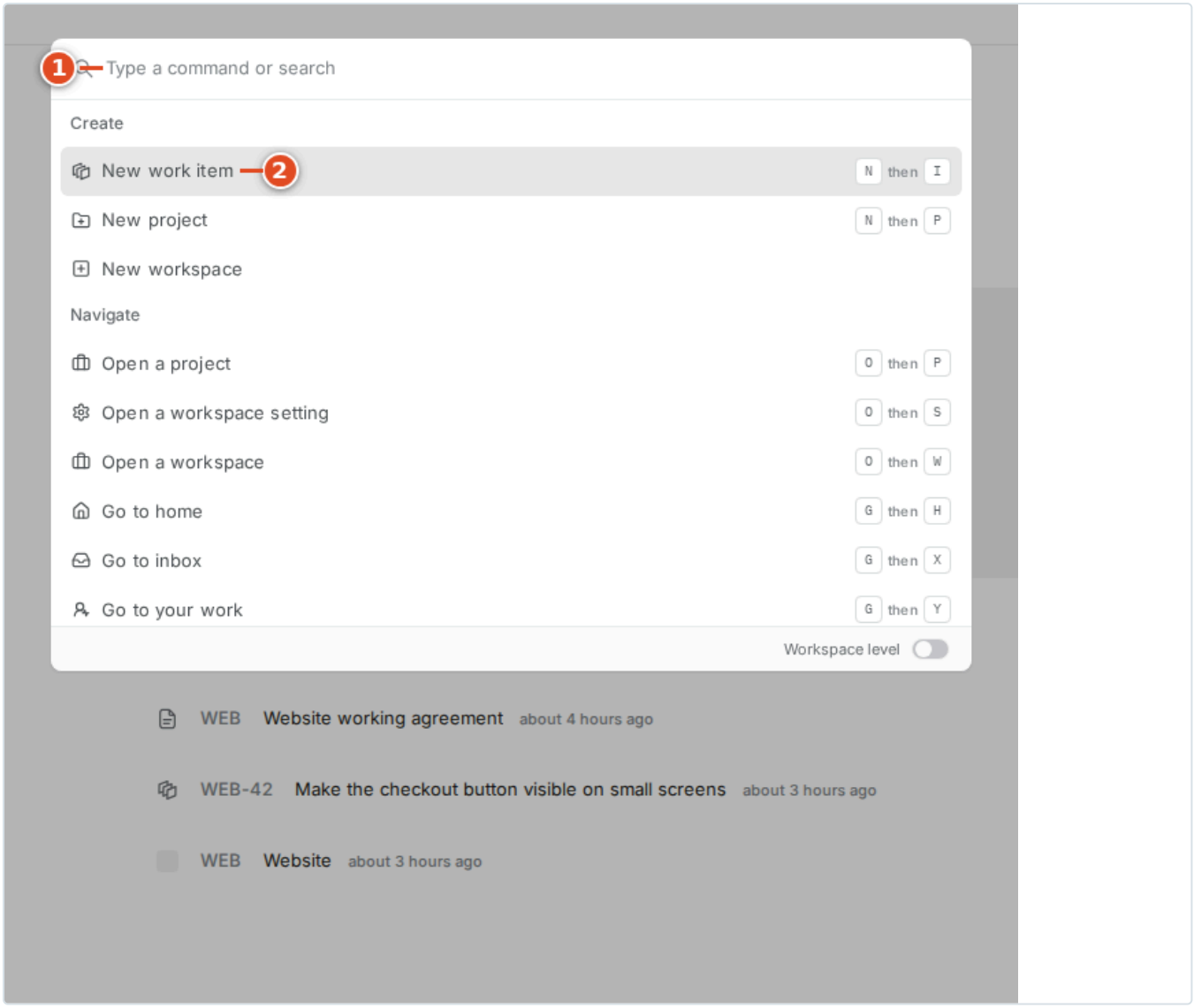


Figure 3.4 Screenshot (test Plane): Command palette: (1) search field; (2) a creation action in the results.

Result: Plane goes to the destination or opens the action.

> **Warning:** Some palette results change data, not just navigate. Read the label before you press Enter. If you are typing in a description or code block, leave the editor first; otherwise the editor may treat the keystroke as text or formatting.

3.5 Learn the keyboard shortcuts

The shortcuts dialog is the reference for your running version and operating system. Open it from the help control in the top bar, or with **Ctrl + /**.

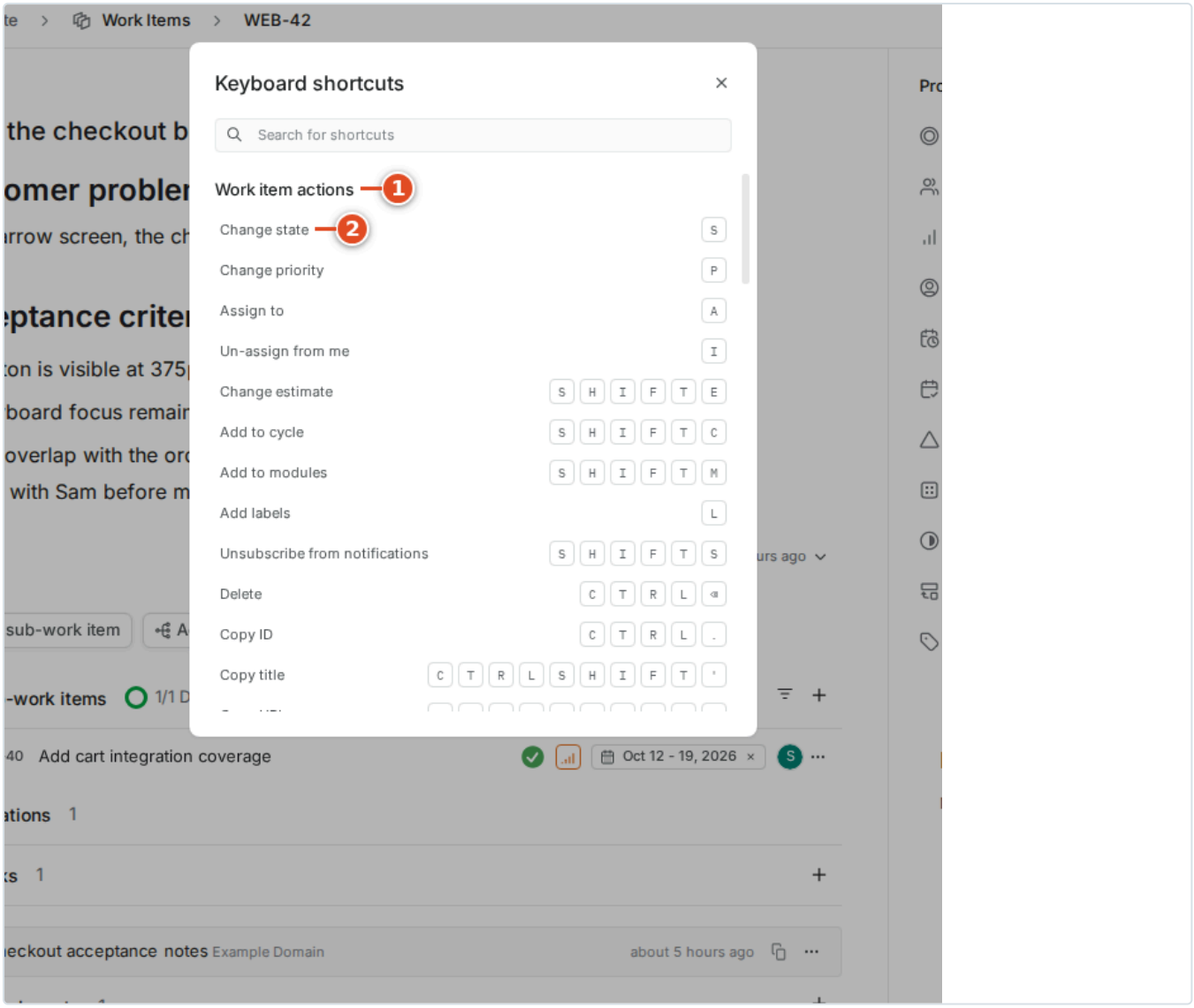


Figure 3.5 Screenshot (test Plane): Keyboard shortcuts: (1) Work item actions category; (2) action labels with their adjacent keys.

The tables below were copied from the running CE v1.4.2 dialog on Linux. On macOS, the dialog shows the Command key where appropriate. “Then” means press and release the first key, then press the next. A plus sign means hold the first key while pressing the second. Some combinations appear in more than one context; the page you are on decides which applies.

Work item, cycle, module and page actions

Context	Action	Keys
Work item	Change state	S
Work item	Change priority	P
Work item	Assign to	A

Context	Action	Keys
Work item	Un-assign from me	I
Work item	Change estimate	Shift + E
Work item	Add to cycle	Shift + C
Work item	Add to modules	Shift + M
Work item	Add labels	L
Work item	Unsubscribe from notifications	Shift + S
Work item	Delete	Ctrl + Backspace
Work item	Copy ID	Ctrl + .
Work item	Copy title	Ctrl + Shift + ’
Work item	Copy URL	Ctrl + Shift + ,
Cycle	Add to favorites	Shift + F
Cycle	Copy URL	Ctrl + Shift + ,
Module	Add/remove members	M
Module	Change status	S
Module	Add to favorites	Shift + F
Module	Copy URL	Ctrl + Shift + ,
Page	Lock	Shift + L
Page	Make public	Shift + A
Page	Archive	Shift + R
Page	Add to favorites	Shift + F
Page	Copy URL	Ctrl + Shift + ,

Create, open and go to

Context	Action	Keys
Create	New work item	N then I
Create	New page	N then D
Create	New view	N then V
Create	New cycle	N then C
Create	New module	N then M
Create	New project	N then P
Open	A cycle	O then C
Open	A module	O then M
Open	A project view	O then V
Open	A project setting	O then S
Open	A project	O then P
Open	A workspace setting	O then S
Open	A workspace	O then W
Go to	Home	G then H
Go to	Inbox	G then X
Go to	Your work	G then Y
Go to	Work items	G then I
Go to	Pages	G then D
Go to	Cycles	G then C
Go to	Modules	G then M
Go to	Project views	G then V
Go to	Intake	G then K
Go to	Project settings	G then S
Go to	Project archives	G then R

Context	Action	Keys
Go to	Workspace analytics	G then A
Go to	Workspace settings	G then S
Go to	Workspace drafts	G then J
Go to	Workspace archives	G then R
Go to	Projects list	G then P
Miscellaneous	Toggle app sidebar	Ctrl + B
Miscellaneous	Copy current page URL	Ctrl + Shift + C
Miscellaneous	Focus search input	Ctrl + F
Help	Open keyboard shortcuts	Ctrl + /

You do not need to memorize these on day one. Start with the palette, **Escape** and **N then I** (new work item). Add navigation shortcuts once you recognize the destinations. A shortcut never grants access: it cannot open a private project or run an action your role does not allow.

> **Tip: Ctrl + .** copies the open work item's ID. It is the fastest way to get an exact key such as `WEB-42` into a branch name or commit message without typos.

> **Tip:** If a shortcut seems to do nothing, check whether a text editor, the browser address bar or a dialog has focus. Use the mouse once to confirm the destination before troubleshooting the keys.

3.6 Keep a reliable route back

A direct link to a work item is the most useful thing to share in a discussion: it points to one record instead of describing the clicks to find it. Always put the identifier next to the link so people recognize it in a message, commit or PR.

Goal: share a work item

Steps

1. Open the work item.
2. Use its copy-link control, or press **Ctrl + Shift + ,**.
3. Paste the link where you need it.

4. Check that the text next to the link includes the correct identifier.

Result: your teammate opens the same record.

A link does not override permissions. If a teammate sees “not found”, ask them to check their workspace and project membership. Do not make a project public to work around one missing invitation.

> **Warning:** The link contains your team’s Plane address. Do not paste private project links into public GitHub repositories unless your team’s policy allows it. git-sync’s own link-back comments on GitHub also contain Plane links, so administrators should enable them only where that is acceptable (chapter 25).

What you learned

- The workspace switcher, sidebar, main area and top-right controls frame every page.
- Home and Your work are personal starting points; a project’s Work items page is the complete list.
- The command palette (Ctrl/Cmd + K) and the shortcuts dialog (Ctrl + /) speed up daily work; Ctrl + . copies an item’s ID.
- Share work with a direct link plus its identifier; links never bypass permissions.

Source files

- Plane CE v1.4.2 web application (`apps/web/`), observed in the running test instance
- `apps/web/core/components/settings/workspace/sidebar/item-categories.tsx` (the admin-only **GitHub Sync** settings entry)
- `docs/handbook/03-finding-your-way.md` (original transcription of the shortcuts dialog)

04 · Work items

A work item is a shared record of an outcome. Its **description** explains what needs to change. Its **properties** (state, priority, assignees, labels, dates, estimate, cycle, module) help the team organize and track it. Its **comments** and **activity** keep the history of decisions. You can edit an item from many layouts, but they all change the same record.

This chapter is the reference for a single work item. It also notes, where relevant, how git-sync interacts with each property.

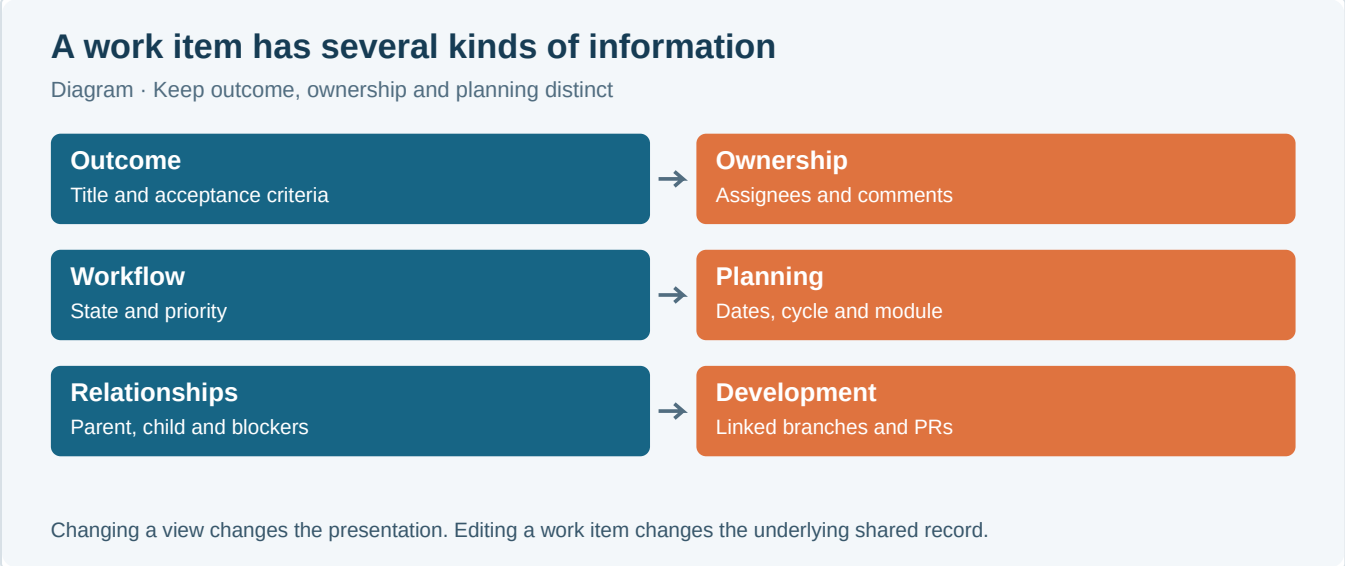


Figure 4.1 Diagram: outcome, ownership, workflow, planning, relationships and development are separate parts of a work item.

4.1 Anatomy of a work item

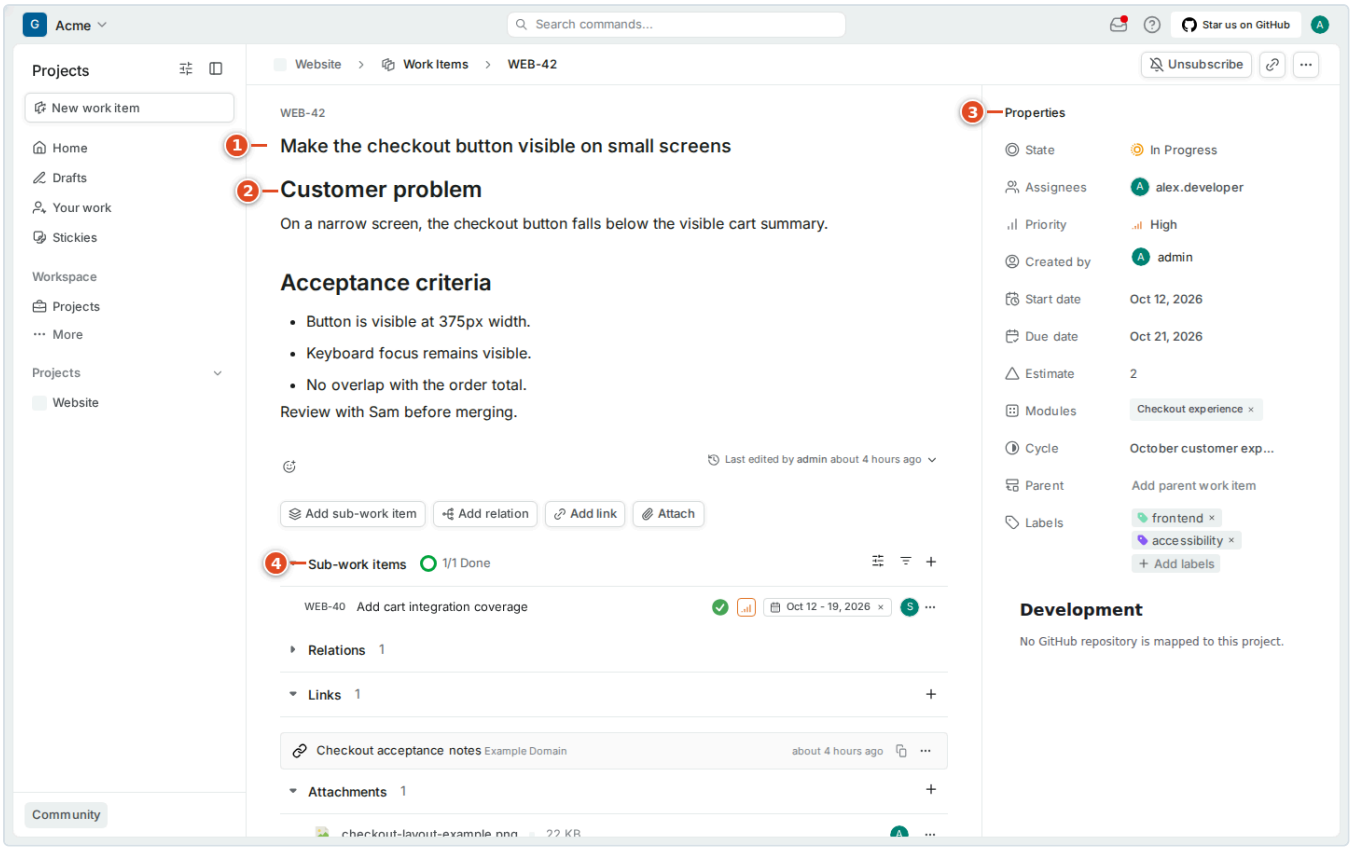


Figure 4.2 Screenshot (test Plane): WEB-42: (1) title and identifier; (2) description; (3) properties; (4) sub-work items.

1. **Title and identifier.** The identifier (`WEB-42`) never changes; the title can be improved.
2. **Description.** The problem, the acceptance criteria and supporting notes.
3. **Properties.** State, assignees, priority, labels, dates, estimate, cycle, module and parent, in the right-hand sidebar. When a repository is mapped to the project, the sidebar also contains git-sync's **Development** section (chapter 12).
4. **Sub-work items.** Smaller items that belong to this one.

Below the description are links, attachments, relations, and the activity feed with comments.

4.2 Create a work item with enough context

Before you create an item, search for an existing one. Two records for the same bug split the discussion and can cause duplicate work. If the request already exists, add your evidence there, or link a separate follow-up with a relation (section 4.9).

Goal: create a new work item

Before you start: you are a member of the project.

Steps

1. Open the project and select **Add work item** in its header, or press **N then I**.
2. Check that the right project is selected in the form.
3. Enter a title that describes the outcome. ①
4. Enter the description. ②
5. Set the properties: state, priority and assignee if known. ③
6. Select **Save**. ④

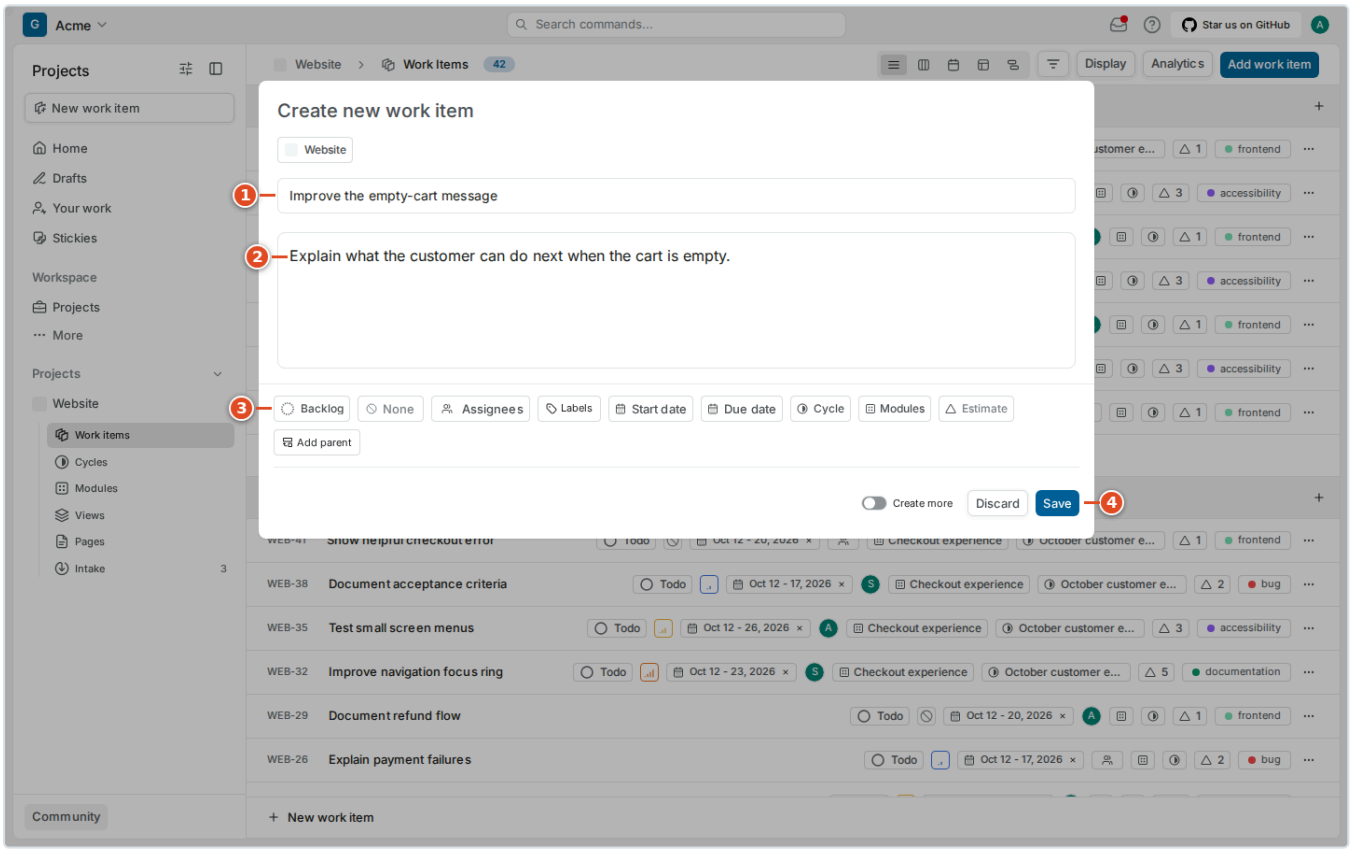


Figure 4.3 Screenshot (test Plane): Create a work item: (1) title; (2) description; (3) properties; (4) Save.

Result: Plane creates the item and gives it the next identifier, for example `WEB-46`.

If it went wrong:

- *It landed in the wrong project* → open the item and change its project, or ask an administrator. Do not create a second copy.
- *You were interrupted* → close the form and choose **Save to Drafts** (section 4.13).

A good title says what should become true: “Keep the checkout button visible on small screens”. “Checkout” on its own forces everyone to open the description. Keep long troubleshooting notes in the description and comments, not in the title.

> **Example:** A small bug description can have four short parts: what happened, what should happen, how to reproduce it, and how to confirm the fix. For `WEB-42` the key facts are the screen widths affected and whether the button stays keyboard-accessible.

> **Note:** You can also create a work item from GitHub. A pull request whose title or body contains `WEB-NEW` makes git-sync create a new item in the `WEB` project, titled after the PR (chapter 10). A GitHub issue labelled `plane` is mirrored into a new item (chapter 13).

Quick add

The quick-add controls in a list or board are useful when you already know the project and the group. They create an item with only a title; the group you add it in can supply the state.

1. In the group you want, select the plus or **New work item** control.
2. Type the title in the inline input. ①
3. Press **Enter** to create the item, or **Escape** to cancel.

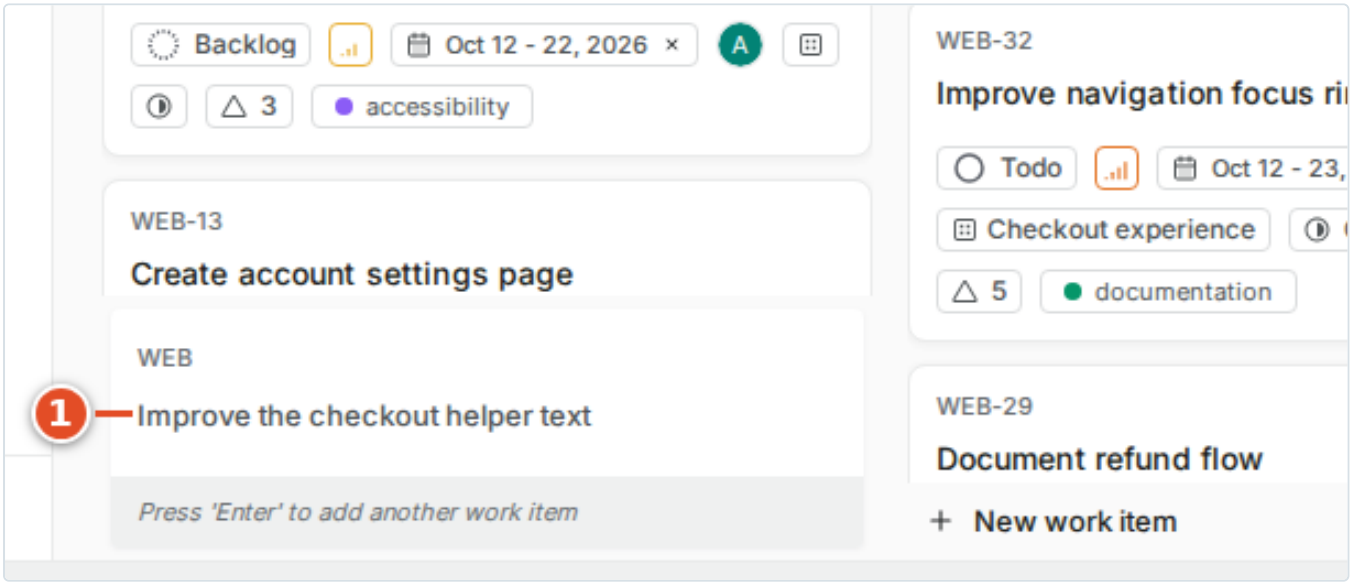


Figure 4.4 Screenshot (test Plane): Quick add: (1) inline title input. Enter submits; Escape cancels.

Open the new item afterwards and add the context that did not fit. A fast creation path does not remove the need for acceptance criteria.

4.3 Understand and copy the identifier

The project supplies the short identifier (`WEB`); Plane adds the sequence number to make `WEB-42`. Use the exact key in conversation and in the places git-sync reads: branch names, PR titles and bodies, and commit messages.

- Copy it from the item (press **Ctrl + .**) rather than typing it.
- Keep the hyphen and the digits. `WEB_42` and `WEB42` are different strings, and git-sync ignores them.
- Do not guess the identifier of an unsaved draft; it does not have one yet.
- Upper and lower case matter in some places: a lower-case `web-42` works in a branch name, but in a PR title or commit message the key must be upper case. Chapter 9 explains the exact rules.

4.4 Write and maintain the description

The description editor supports headings, lists, checklists, code blocks, links, images and mentions. Use structure to separate the customer problem, acceptance criteria and implementation notes.

Goal: write a description others can act on

Steps

1. Select the description area.
2. Write the customer problem in plain language. ①
3. Add concrete acceptance criteria, for example as a checklist. ②
4. Add reproduction steps or references.
5. Click outside the editor and wait for the saved indication.

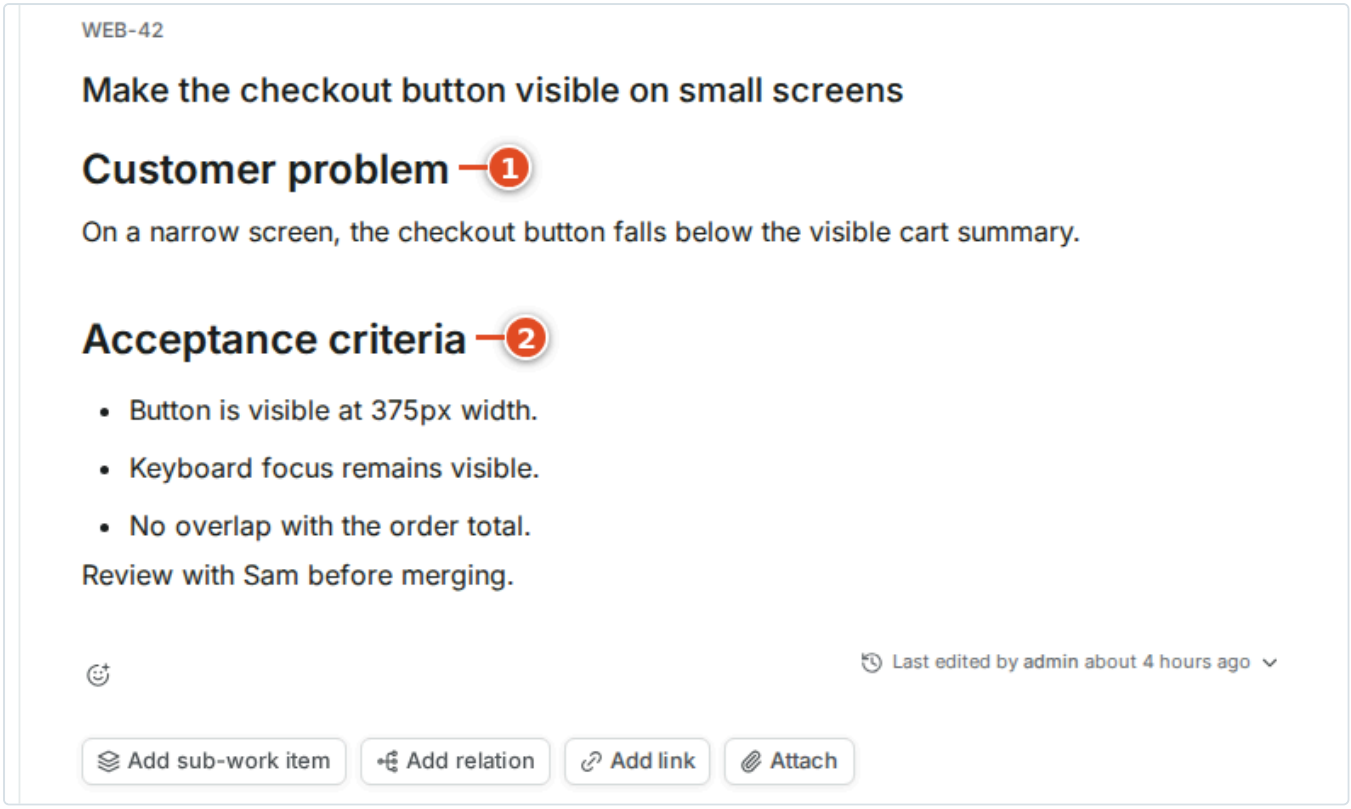


Figure 4.5 Screenshot (test Plane): Description: (1) customer problem; (2) acceptance criteria.

Result: teammates and reviewers can tell when the work is done.

Select text to show the formatting controls. Use a real checklist block instead of imitating one with symbols. Put code and literal error messages in a code block. Remove credentials, customer data and machine-specific secrets from copied logs before saving.

An image can explain a visual bug better than paragraphs. Include enough of the screen to identify the problem, and add a sentence saying what to notice.

A **mention** names a teammate. Type `@` and choose the person from the suggestions. Text that you never select from the list is just text. Put your question next to the mention (“@Sam, may the label wrap at 320 px?”).

Keep the description current as decisions change. A comment can explain why the team changed its mind; the description should still say what is expected now.

> **Note:** If the item is mirrored to a GitHub issue (chapter 13), its description syncs with the issue body. By default GitHub’s version wins when both change between syncs. Ask your administrator which side is the source of truth before making large edits on both.

4.5 States and state groups

A **state** is a named point in the workflow, such as Todo, In Progress, In Review or Done. A **state group** is the broader class that Plane and git-sync use. The Website project has the groups Backlog, Unstarted, Started, Completed and Cancelled. Its Todo state belongs to Unstarted; both In Progress and In Review belong to Started.

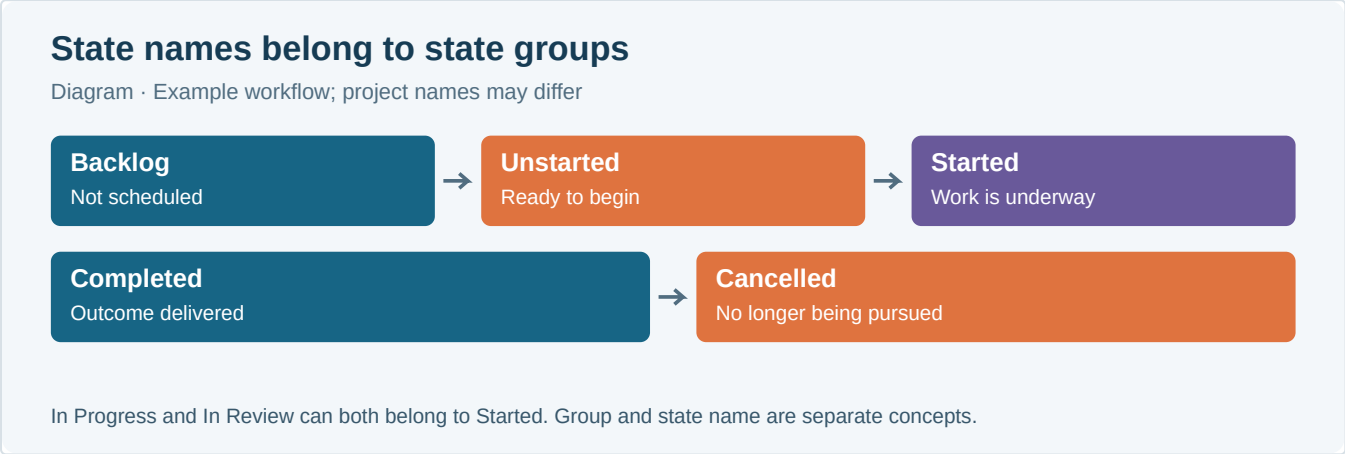


Figure 4.6 Diagram: state groups classify project-specific state names.

Goal: change the state

Steps

1. Open the **State** property, or press **S**. ①
2. Read the available states.
3. Select the state that matches reality, for example **In Review**. ②
4. Check the new value on the item.

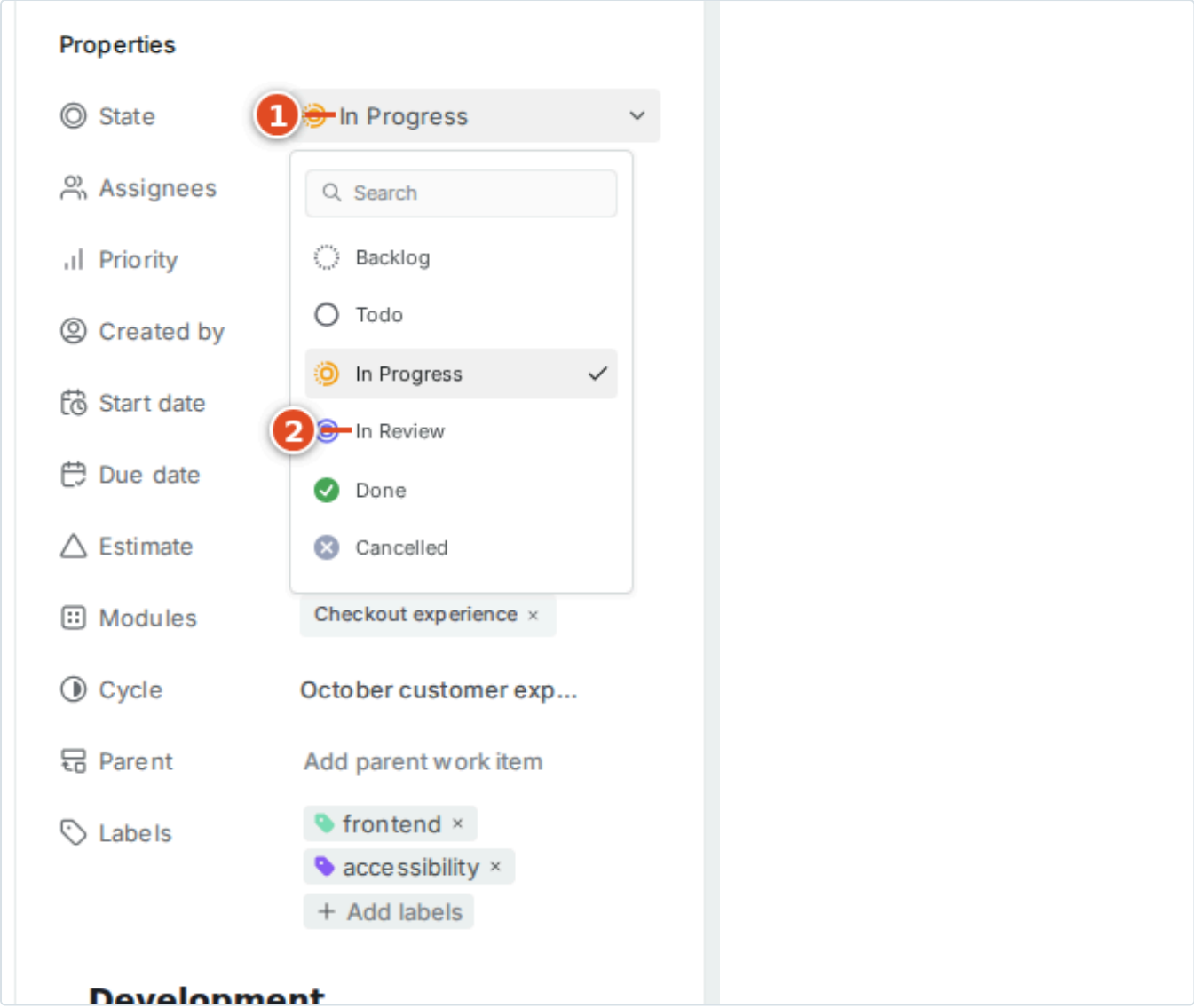


Figure 4.7 Screenshot (test Plane): State: (1) current state control; (2) In Review among the project choices.

Result: the item shows the new state and the change appears in its activity.

Done means the agreed outcome is complete. It is not the same as archiving. **Cancelled** means the team is no longer pursuing the work; add a comment with the reason. A blocked task usually keeps its workflow state and gets a **Blocked by** relation; do not invent a state for every temporary obstacle.

> **Note:** git-sync can change states for you. With the shipped rules, a new branch or an opened PR moves a linked item to **In Progress**, a review request moves it to **In Review**, and a merge into `main` moves it to **Done**. A Smart Commit such as `WEB-42 #in-review` also changes the state. The activity feed shows these changes as made by the git-sync service account. If an item keeps moving back, read chapter 14 before fighting the rule.

4.6 Priority, assignees and labels

Priority is relative importance: None, Urgent, High, Medium or Low. A high priority does not assign, unblock or schedule an item by itself.

Goal: set the priority

1. Open **Priority**, or press **P**. ①
2. Choose the agreed priority, for example **Urgent**. ②
3. Add a comment if the reason for a big change is not obvious.

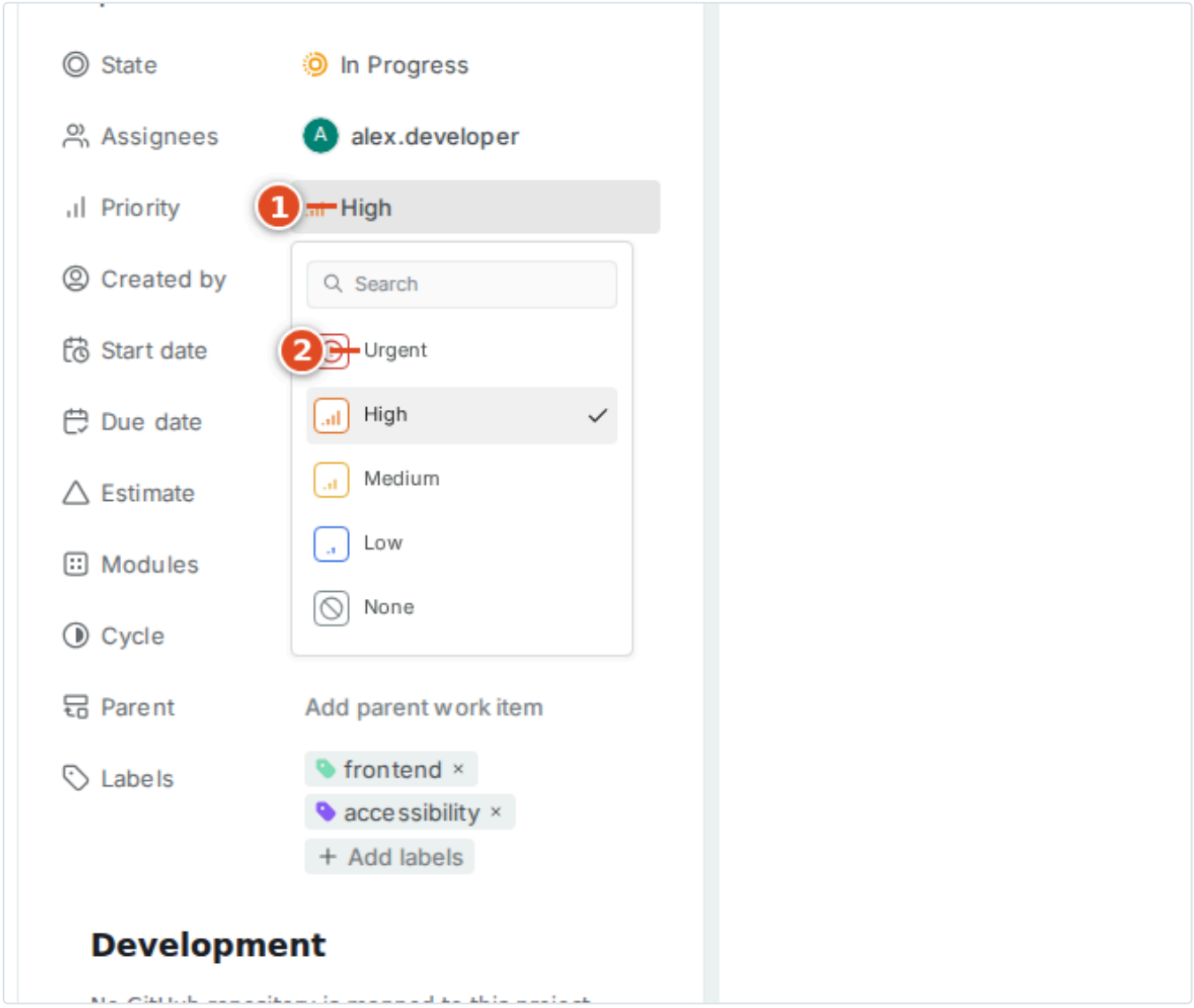


Figure 4.8 Screenshot (test Plane): Priority: (1) current priority; (2) Urgent among the available priorities.

An **assignee** is a person responsible for moving the item forward. You can assign more than one person, but the team should still know who gives the next update.

Goal: assign the item

1. Open **Assignees**, or press **A**.
2. Search for the project member and select them.
3. Close the selector and check the names or avatars.

Labels are reusable categories such as frontend, bug or accessibility. Use existing labels rather than creating near-duplicates with different capitalization.

Goal: add labels

1. Open **Labels**, or press **L**. ①
2. Select the relevant labels, for example **accessibility**. ②
3. Close the selector.

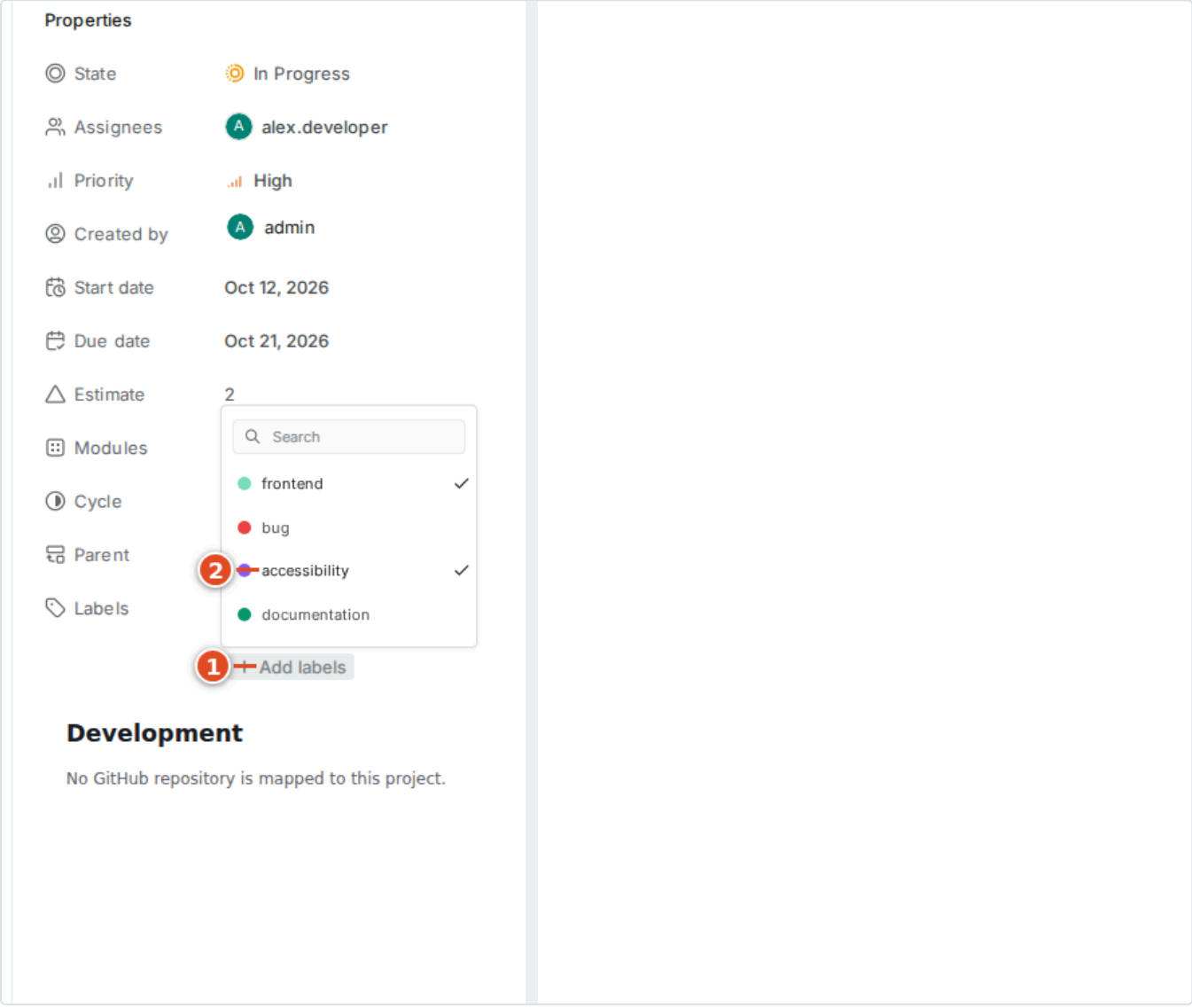


Figure 4.9 Screenshot (test Plane): Labels: (1) Add labels; (2) accessibility in the project label choices.

> **Note:** Two labels have a special meaning for git-sync. Adding the label **github** to a Plane work item in a mapped project asks git-sync to create a matching GitHub issue and keep the two in sync (chapter 13). The label **plane** plays the same role on the GitHub side. On mirrored items, assignees and labels also sync, with GitHub users translated through their linked Plane accounts.

4.7 Dates and estimates

A **start date** marks when work should begin; a **due date** marks the target. They are planning properties and do not create calendar invitations. The Calendar and Gantt layouts use them, so an item with no dates may be missing from those layouts.

Goal: set the due date

1. Open **Start date** and select a date, if you use one.
2. Open **Due date**. Use the month selector ① and pick the day ②.
3. Check that the range makes sense.

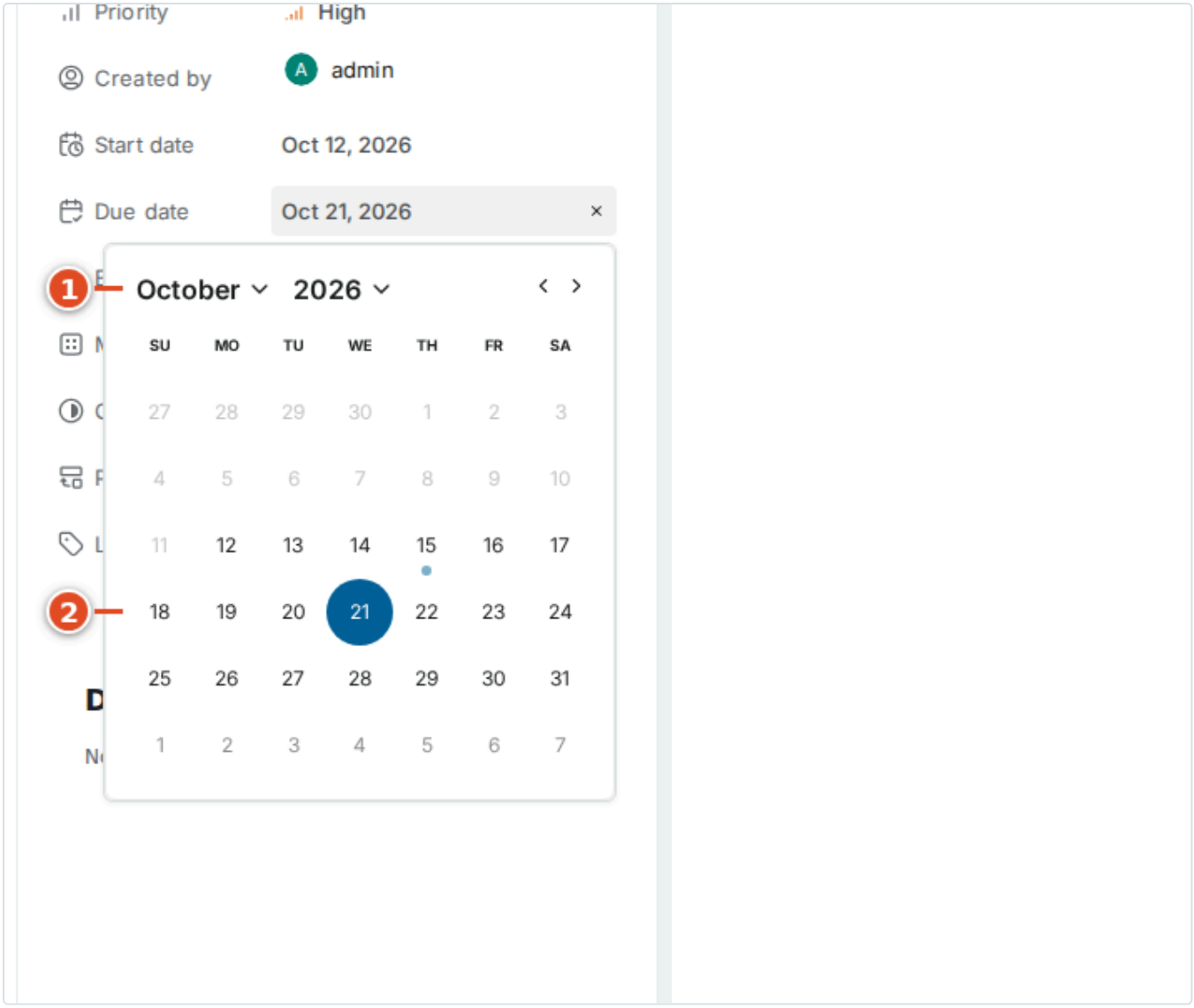


Figure 4.10 Screenshot (test Plane): Due date: (1) month selector; (2) selected day, 21 October.

An **estimate** is the team's agreed measure of size or effort. A project administrator chooses the scale (chapter 6). Do not assume that 3 means three hours; ask what the scale means. If the project has no estimate system, the **Estimate** field is not shown.

> **Warning:** Estimates, dates and priority never sync with GitHub, even on mirrored items. The Smart Commit command `#time 1h 30m` does not set an estimate or a native worklog either: Plane CE has no worklog, so git-sync posts a comment such as "Time logged: 1h 30m (90 minutes)" (chapter 11).

4.8 Parents and sub-work items

A **parent** groups related work under a larger outcome. A **sub-work item** is a full work item with its own identifier, state and discussion. Use sub-work items when pieces can be implemented, reviewed or assigned separately. Use a checklist in the description for small checks that do not need their own lifecycle.

Goal: add a sub-work item

1. Open the parent item.
2. Select **Add sub-work item**.
3. Choose an existing item or create a new one in the form.
4. Check that the child appears in the parent's sub-work items section (④ in the anatomy screenshot).

You can also set the **Parent** property on the child. A completed child does not prove the whole parent is complete.

> **Note:** git-sync does not cascade between parents and children. Merging a PR that closes `WEB-42` does not close its sub-work items, and parent/child structure never syncs to GitHub. Reference each item a change actually affects, for example `Closes WEB-40, WEB-42`.

4.9 Relations: dependencies and duplicates

Relations connect two items without making one a child of the other. Plane offers **Relates to**, **Duplicate of**, **Blocked by** and **Blocking**. Direction matters: if `WEB-42` cannot proceed until `WEB-41` is done, then `WEB-42` is *blocked by* `WEB-41`.

Goal: record that an item is blocked

Steps

1. Select **Add relation**. ①
2. Choose the relation type, for example **Blocked by**. ②
3. Search for the other item by identifier. ①
4. Select the correct result. ②
5. Select **Add selected work items**. ③

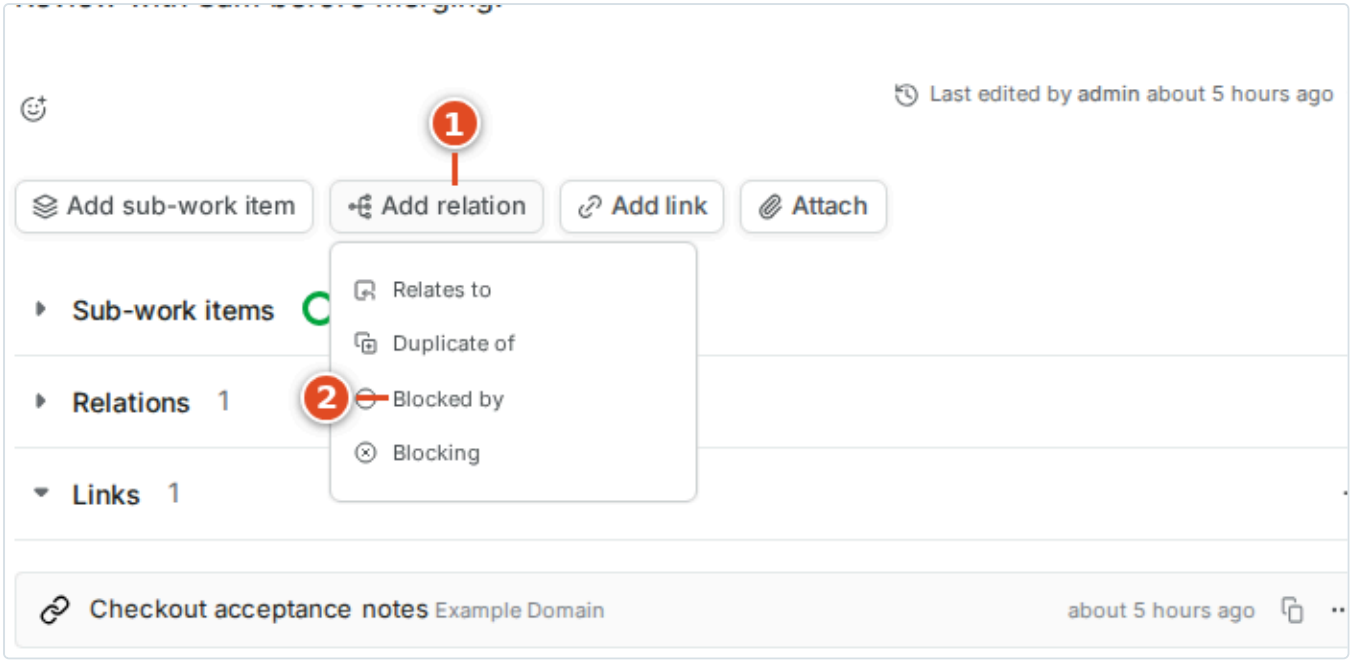


Figure 4.11 Screenshot (test Plane): Relations: (1) Add relation; (2) Blocked by, alongside Relates to, Duplicate of and Blocking.

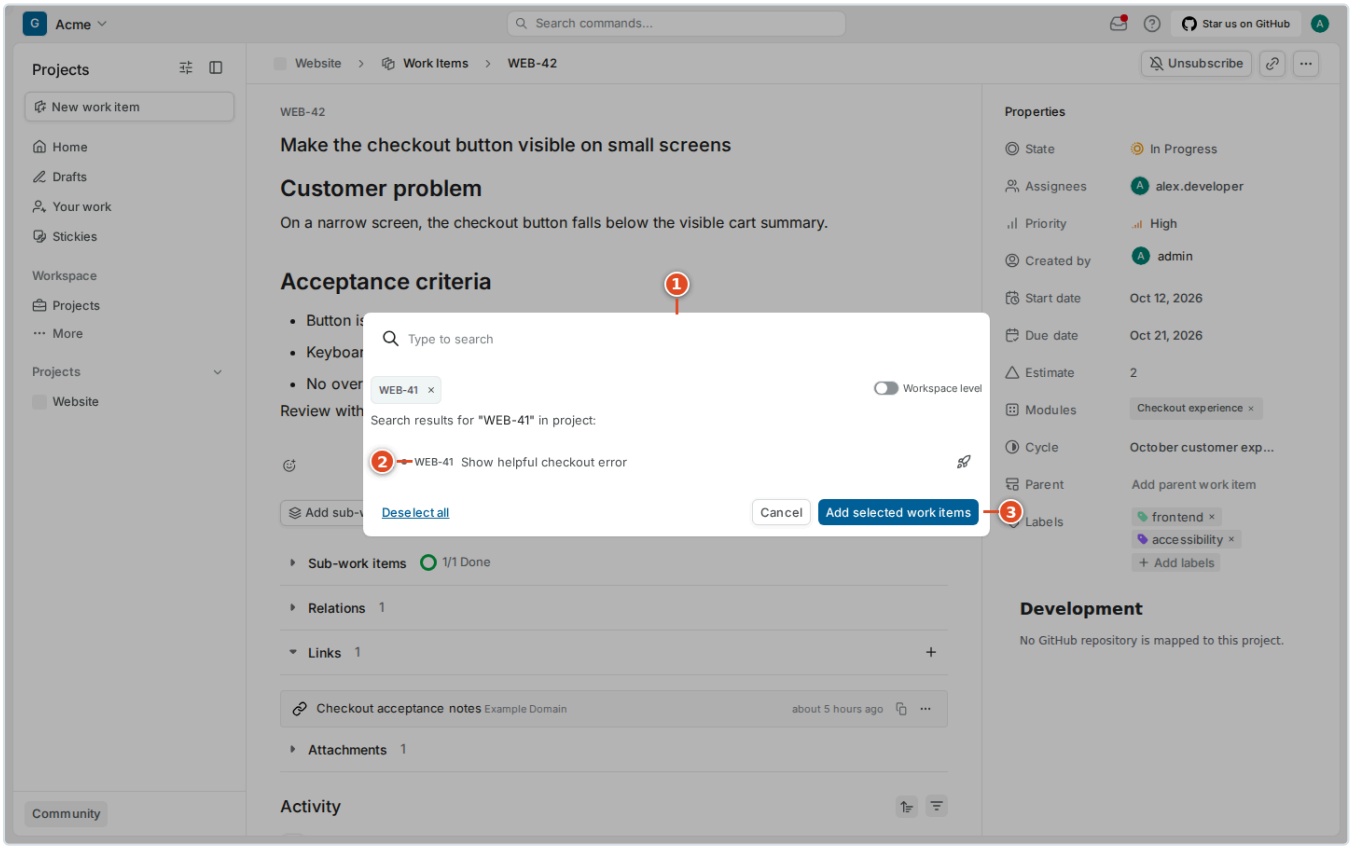


Figure 4.12 Screenshot (test Plane): Choose a related item: (1) search by identifier; (2) selected result; (3) Add selected work items.

Result: both items show the relation.

Add a short comment when a dependency is not obvious. Mark duplicates deliberately and keep the useful evidence on the main item rather than deleting the second report at once.

> **Note:** A Plane relation does not block a GitHub merge, and relations never sync to GitHub. They are also unrelated to git-sync's **relation tier** (`Relates to WEB-42` in a PR body), which is a way to link a PR to an item *without* changing the item's state (chapter 9).

4.10 Links and attachments

A **link** points to a web page. An **attachment** is an uploaded file. Use a link for a document that keeps changing, and an attachment for a stable copy of evidence.

Goal: add a link

1. Select **Add link**.
2. Enter the URL. ①
3. Enter a useful display title. ②
4. Select **Add Link**. ③

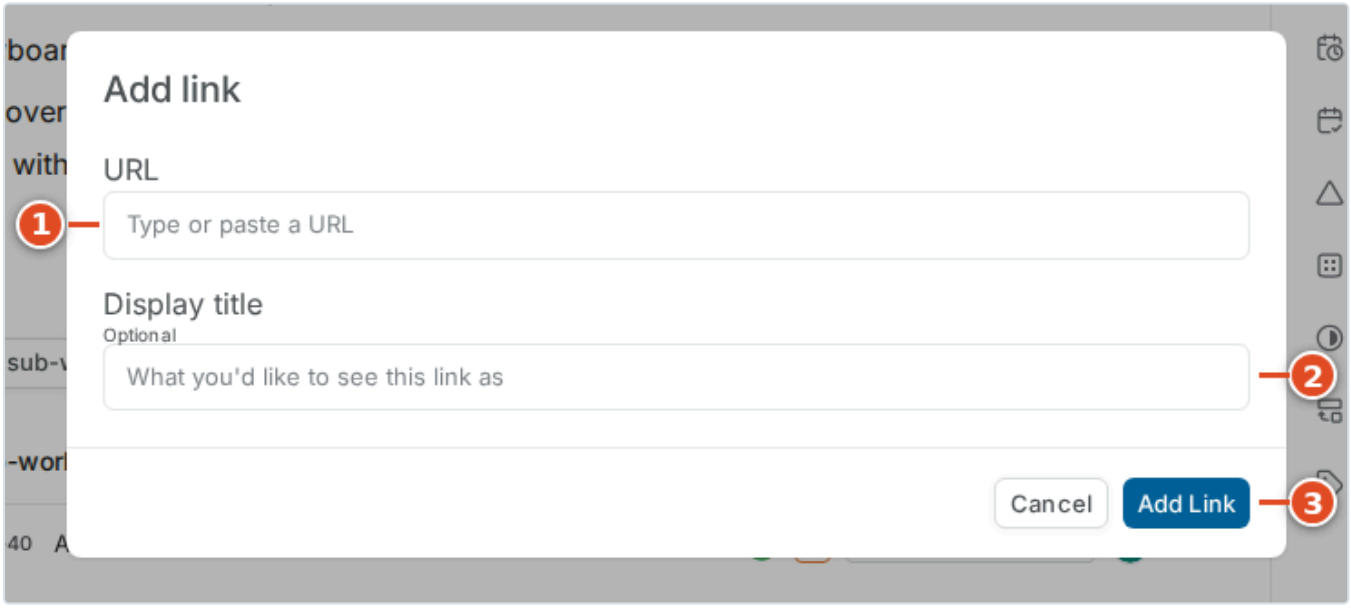


Figure 4.13 Screenshot (test Plane): Add link: (1) URL field; (2) display-title field; (3) Add Link.

Goal: attach a file

1. Select **Attach** and choose the file.
2. Wait for the upload to finish.
3. Open the attachment entry to check it is the right file. ①②

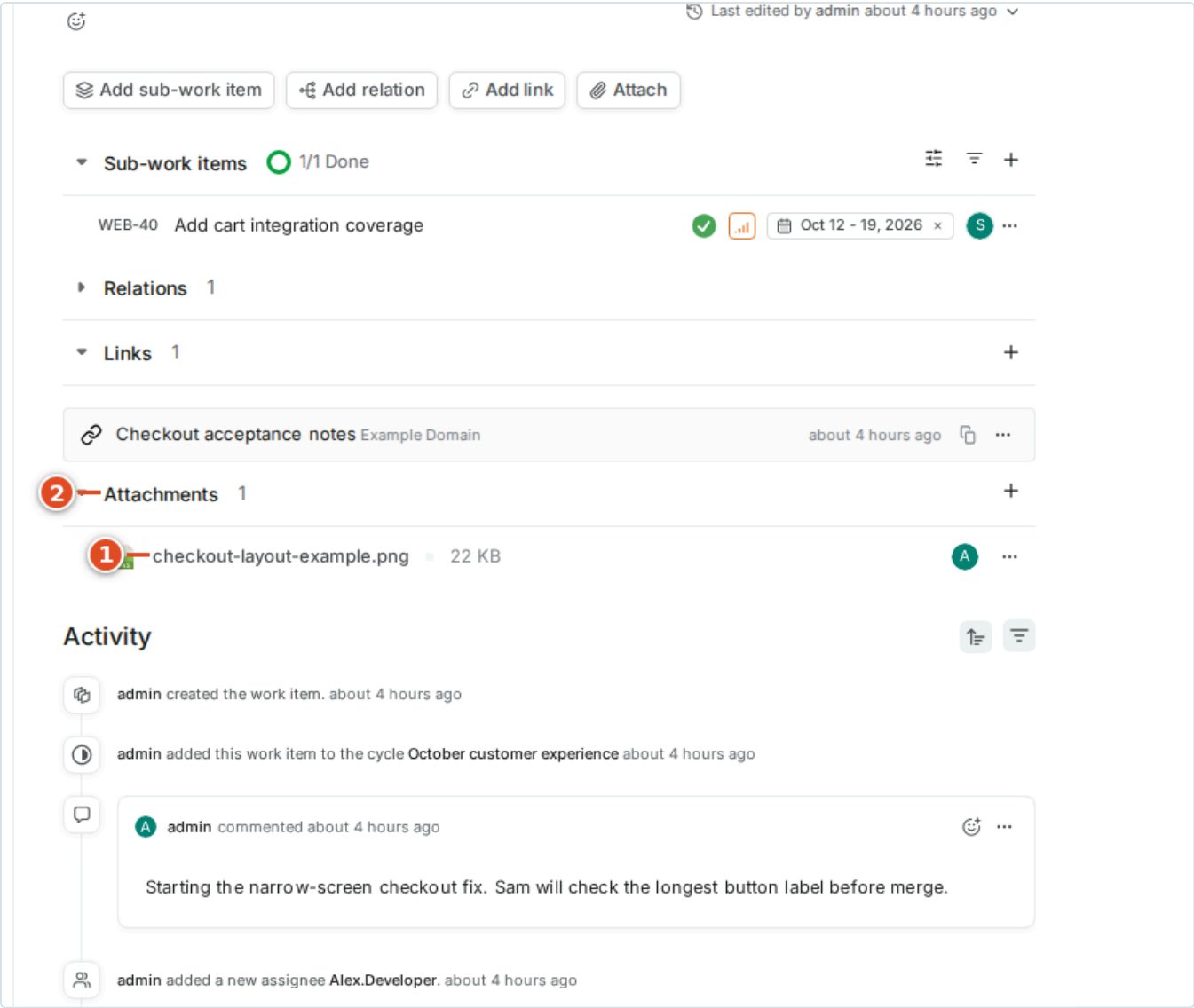


Figure 4.14 Screenshot (test Plane): Attachment: (1) uploaded checkout-layout-example.png; (2) attachment section.

The server can limit file size and type. On the test instance, a plain-text file was refused as an invalid file type while a PNG image uploaded normally. If a large file fails, link to the team's approved storage instead.

> **Note:** git-sync stores its development information as ordinary Plane links on the work item, with titles that start with a category in brackets: `[PR]`, `[Branch]`, `[CI]`, `[Deploy]` and `[Preview]`. For example: `[PR] #121 WEB-42 Keep the checkout button visible on small screens · MERGED`. git-sync keeps these links up to date and the Development panel reads them (chapter 12). Do not edit or delete them by hand. Attachments never sync to GitHub.

4.11 Comments, reactions and activity

Use a **comment** to report progress, ask a focused question or record a decision. “I need a decision about the maximum label width” is more useful than “Any thoughts?”. A **reaction** is a lightweight acknowledgment on a comment; it does not replace an explicit approval when one is needed.

Goal: post a comment

1. Open the comment editor below **Activity**. ①
2. Write the update or question.
3. Select **Comment**. ②
4. Check the posted comment.

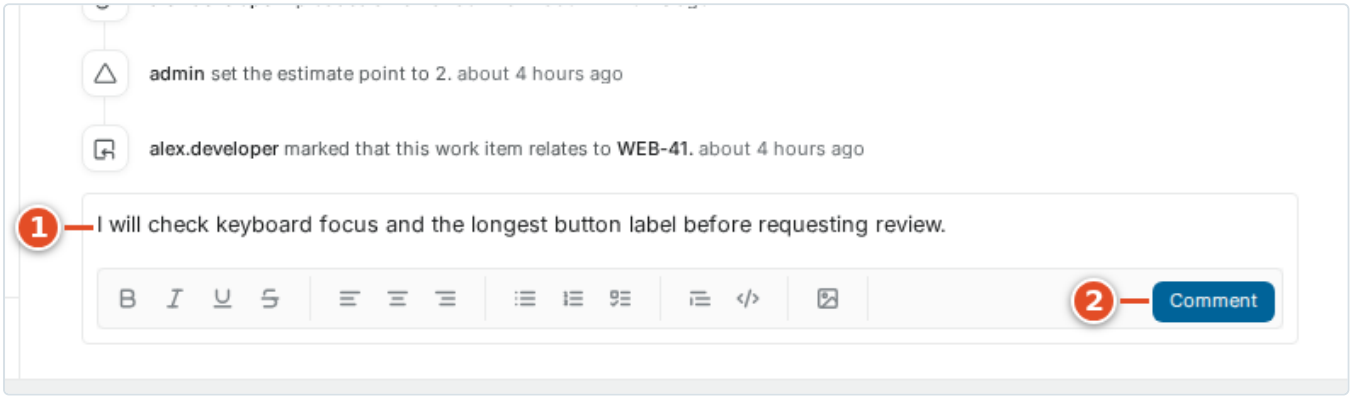


Figure 4.15 Screenshot (test Plane): Comment editor: (1) write the update; (2) Comment submits it.

Use a posted comment’s own menu to edit or delete it. Deleting can remove useful context; often it is better to post a correction.

Activity records property changes alongside comments: who changed the state, when an assignee was added, which cycle or module changed. Read it before concluding that an unexpected value is a sync fault.

> **Note:** git-sync writes comments too, as the service account. A Smart Commit `#comment` appears with a small “via `3b2335d` by alex-dev” line that links to the commit. A comment mirrored from a GitHub issue starts with “*sam-reviews* commented on GitHub:”. On a mirrored item, new comments sync in both directions, but edits and deletions of existing comments do not (chapter 13).

4.12 Subscribe, share, archive and delete

Subscribe asks Plane to keep you informed about an item; the button then reads **Unsubscribe**. Subscribing does not assign the item to you.

Use the header's copy controls or action menu to copy the item's link or ID. The item menu also offers **Make a copy**, **Open in new tab**, **Archive** and **Delete**.

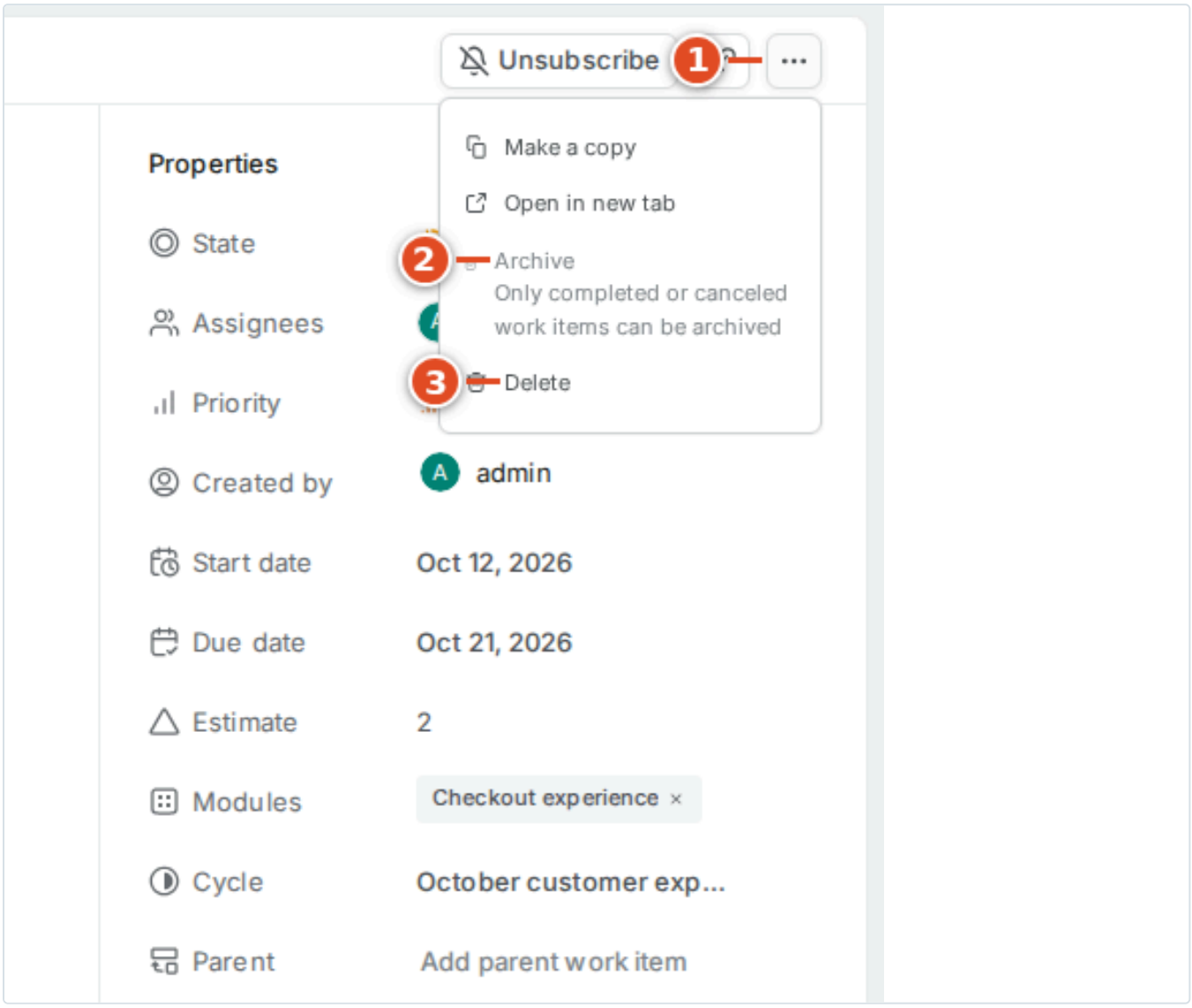


Figure 4.16 Screenshot (test Plane): Item menu: (1) menu button; (2) archive availability; (3) Delete.

In CE v1.4.2, **Archive** is disabled for work that is not completed or cancelled. Archive finished work when the team wants it out of active lists but kept on record. **Delete** is stronger: read the confirmation and check the identifier. Never delete a task just to undo a state change or an accidental comment.

> **Warning:** Archive, delete, cancel and complete are different actions. Closing a GitHub PR without merging is yet another. Use the one that describes what actually happened.

> **Warning:** Deleting a mirrored Plane work item does **not** close its GitHub issue. Plane v1.4.2 sends no delete event; git-sync's periodic reconciliation only marks the link as orphaned and comments on the GitHub issue. Close the issue on GitHub yourself, or move the Plane item to Cancelled instead of deleting it (chapter 13).

4.13 Keep unfinished writing in Drafts

If you close the creation form before saving, Plane can keep your unfinished writing in **Drafts**. Use this when you need more information before creating a shared task. A draft has no identifier yet, and git-sync cannot link to it.

Goal: save and resume a draft

Steps

1. Close the unfinished creation form.
2. Select **Save to Drafts**. ①
3. Later, open **Drafts** from the sidebar (or press **G then J**).
4. Open the saved draft to keep editing. ①
5. Check its project and properties before publishing it as a work item.

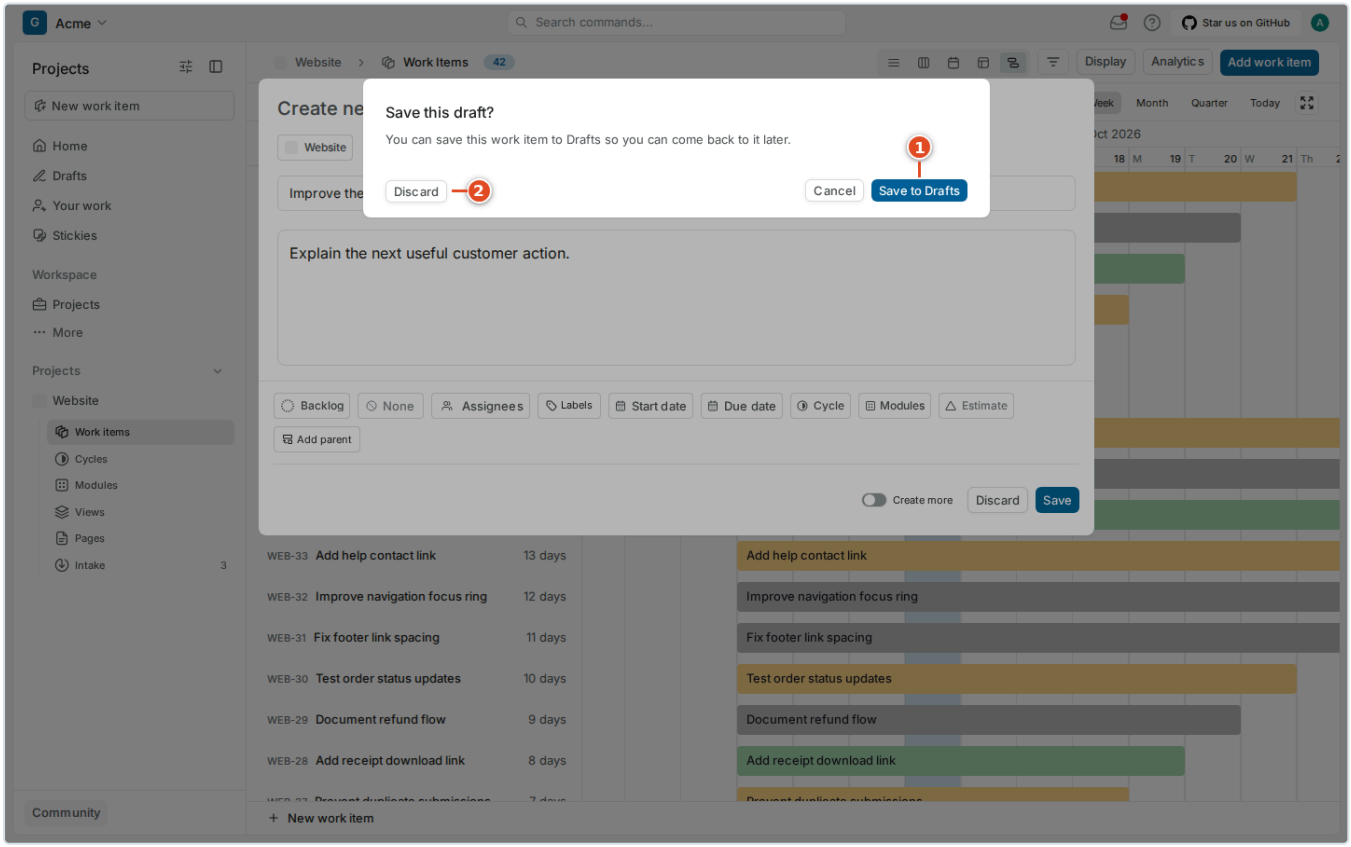


Figure 4.17 Screenshot (test Plane): Unfinished item: (1) Save to Drafts preserves the writing; (2) Discard removes it.

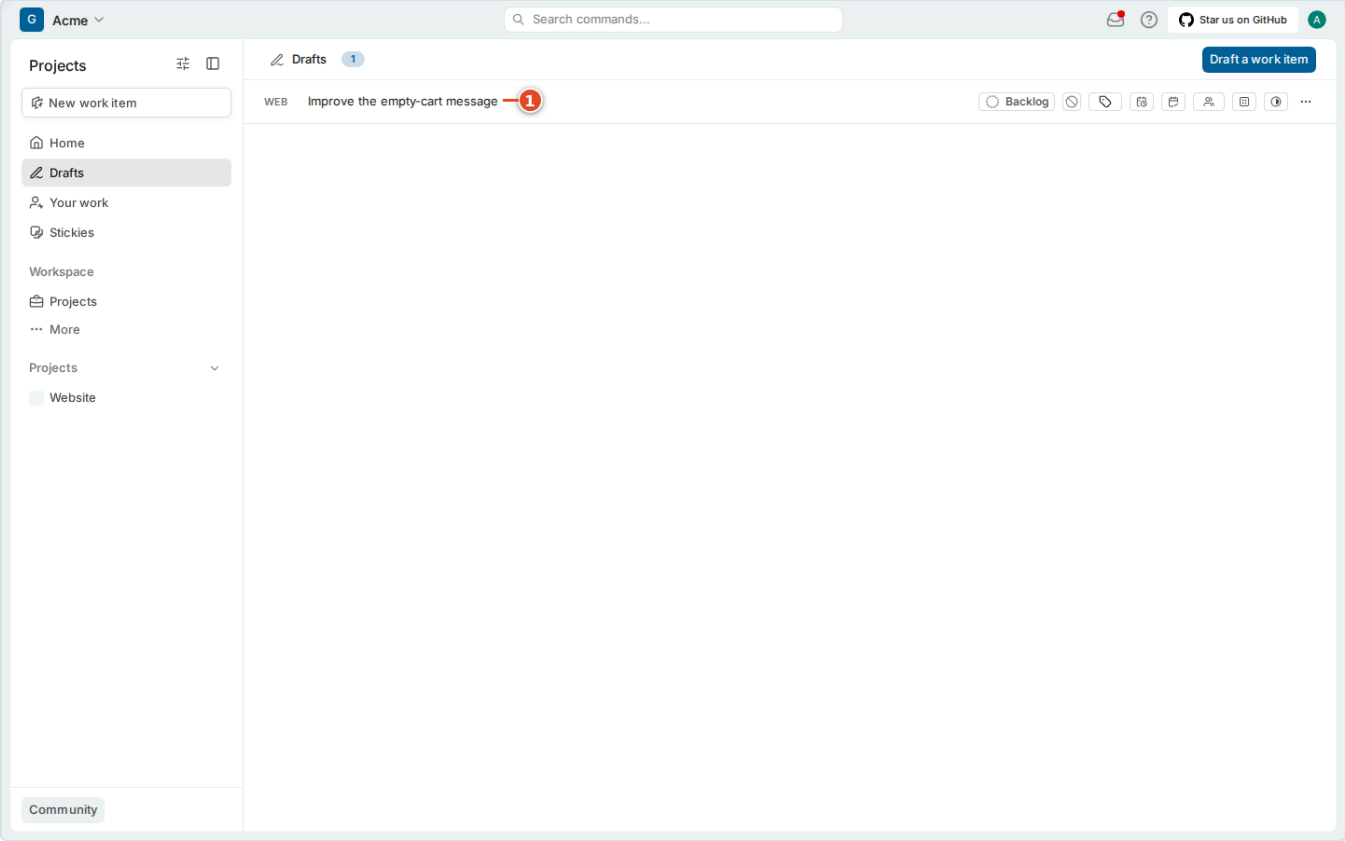


Figure 4.18 Screenshot (test Plane): Drafts: (1) the saved unfinished item. Open it to continue editing.

Result: your text is kept until you publish or discard it.

Discard ② deletes the unfinished draft. Before publishing an old draft, search again: a teammate may have created the item meanwhile.

4.14 Before you start coding: a ready check

When you pick up an item, take a moment to make sure it is really ready.

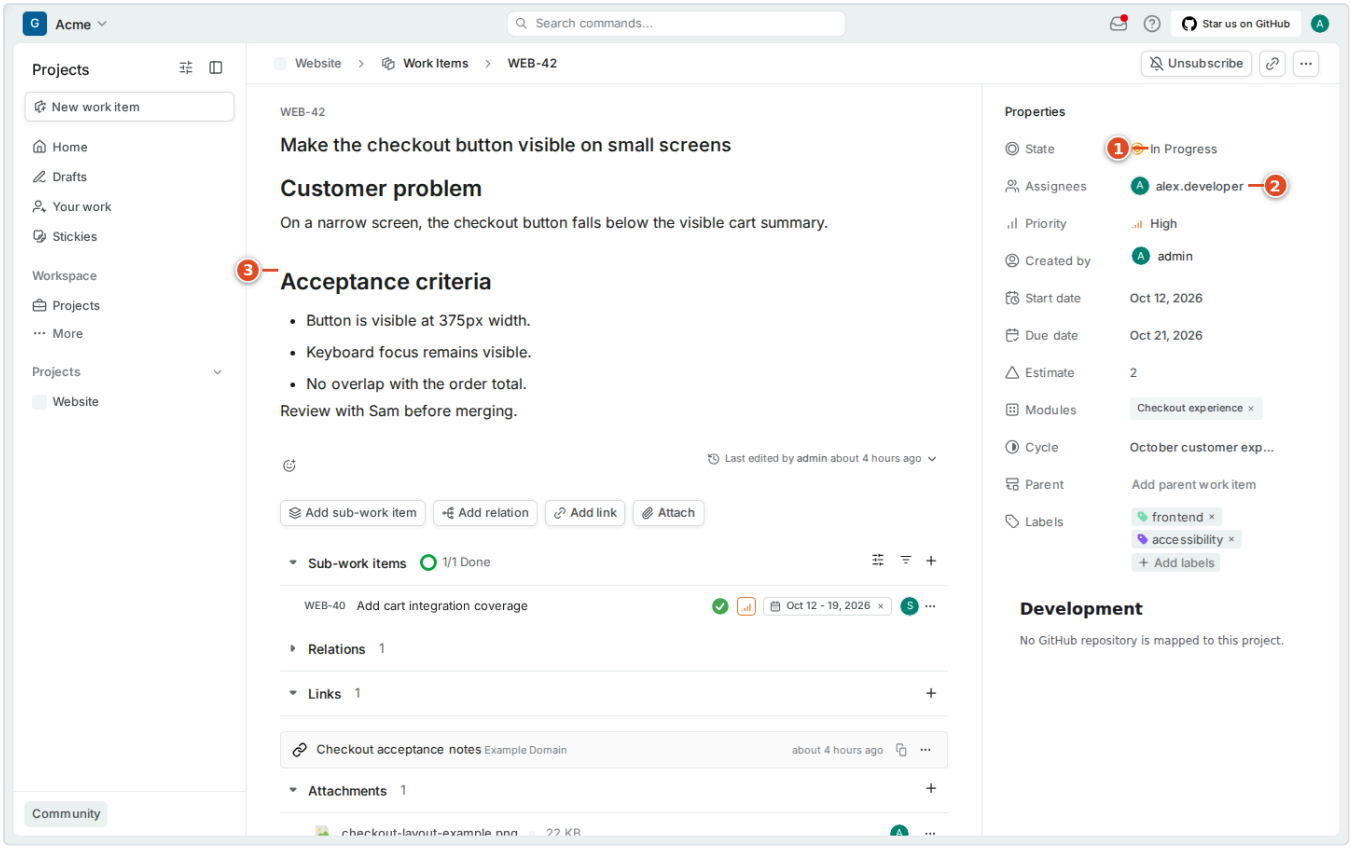


Figure 4.19 Screenshot (test Plane): Starting work: (1) In Progress state; (2) assigned developer; (3) acceptance criteria.

- The acceptance criteria are clear, and you know how to confirm them.
- You are an assignee, and any blocker is resolved or recorded.
- The project is mapped to the repository you will change. The Development panel shows it (chapter 12).
- You know whether your PR will **close** the item or only **contribute** to it (chapter 9).

You do not need to set **In Progress** by hand if your team uses the shipped rules: your first push to a branch named after the key does it for you. Moving it yourself does no harm.

What you learned

- A work item has a permanent identifier, a description, properties, relations, links, attachments and an activity feed.
- States belong to groups; git-sync's rules and Smart Commits change states for you, and the activity feed shows them.
- Labels **github** and **plane** opt items into issue mirroring; priority, dates, estimates, parents, relations and attachments never sync.
- git-sync stores development information as `[PR]`, `[Branch]`, `[CI]`, `[Deploy]` and `[Preview]` links — leave them alone.
- Deleting a mirrored item does not close its GitHub issue; cancel it instead.

Source files

- Plane CE v1.4.2 web application (`apps/web/`), observed in the running test instance
- `src/sync/dev-info.ts` (bracketed development links)
- `src/github/smart-commits.ts` (`#comment`, `#time`, state commands)
- `src/sync/mirror.ts`, `src/sync/reconcile.ts` (mirroring, orphaned links)
- `src/parse/references.ts` (key format and case rules)

05 · Views and layouts

A project can hold more work than you can usefully read at once. Four controls help:

- A **layout** decides how items are arranged: list, board, calendar, spreadsheet or Gantt chart.
- A **filter** decides which items appear.
- **Grouping** splits the matching items into sections, and **ordering** decides which comes first.
- A **saved view** keeps a useful combination so you can return to it.

These controls change only how *you* see shared work. Hiding the Due date column does not remove due dates; filtering to In Progress does not move hidden items anywhere. But editing a property value in any layout changes the real work item, and, for mirrored items, can be synced to GitHub.

5.1 Choose a layout

Plane CE v1.4.2 has five layout buttons in the Work items header. Hover over an icon to read its tooltip.

Layout	Good for answering	Remember
List	What is ready for me to pick up?	Grouping and ordering decide the sequence.
Board (Kanban)	Where is work piling up?	A column usually represents a state; check the grouping first.
Calendar	What has a date in this period?	Items without dates do not appear.
Spreadsheet	How do properties compare across many items?	Editable cells change the real item.
Gantt	How do planned date ranges overlap?	A bar shows planned dates, not progress.

5.2 List: a readable queue

The list shows one row per item with its identifier, title and the properties you chose to display.

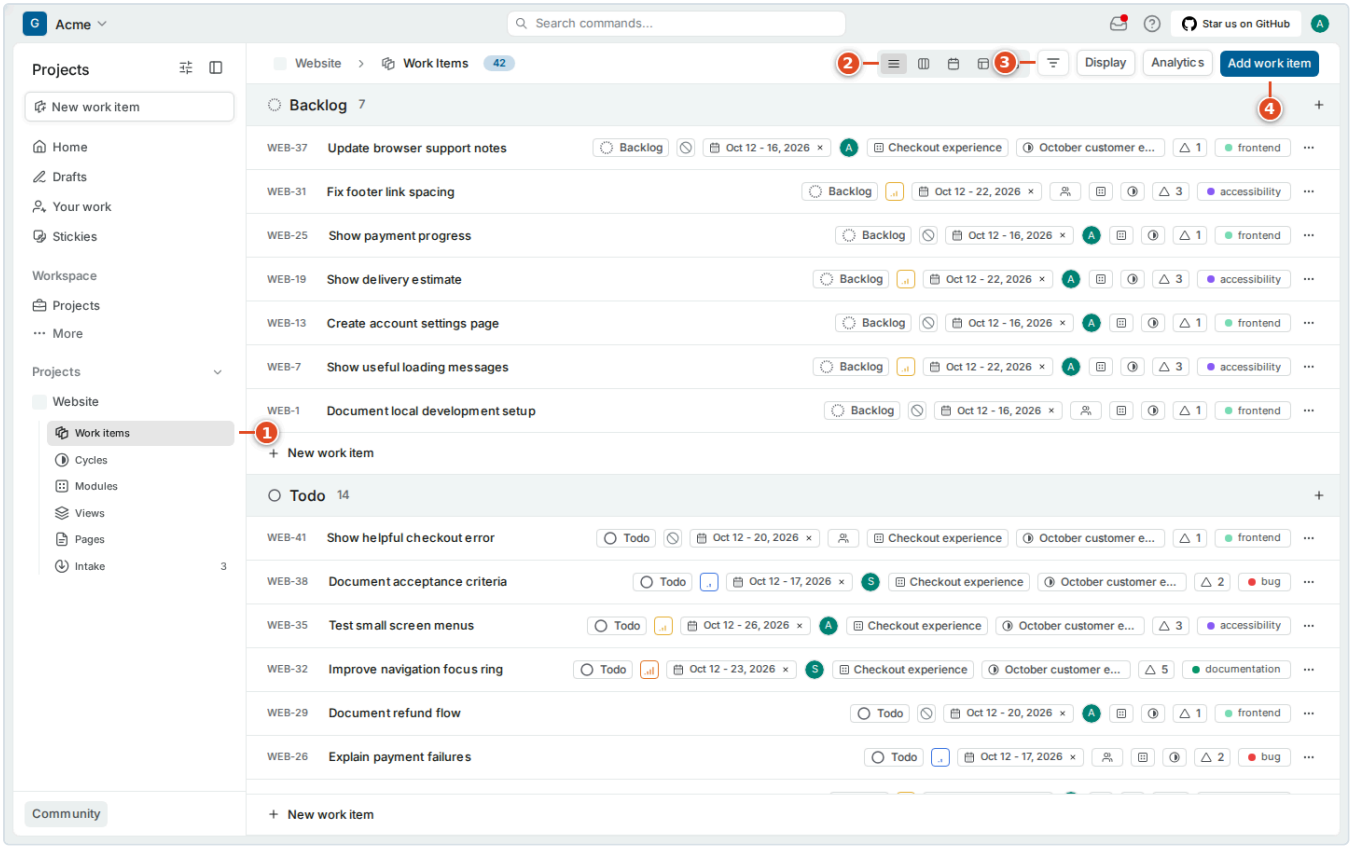


Figure 5.1 Screenshot (test Plane): List layout: (1) project navigation; (2) layout buttons; (3) filter button; (4) Add work item.

Goal: scan the project's work as a list

Steps

1. Open the project's **Work items** page from the project navigation. ①
2. Select the list icon among the layout buttons. ②
3. Read the group heading above the rows and check the ordering in **Display** (section 5.8).
4. Use the filter button ③ to narrow the list, or **Add work item** ④ to create one.
5. Open an item to read its description before deciding what it means.

Result: a sorted, grouped list of matching items.

The count next to a group counts items that match the current view. If an item seems missing, check filters, collapsed groups and sub-work-item visibility before creating a duplicate.

5.3 Board: discuss the flow of work

A board shows items as cards in columns. Grouped by state, it is a good shared picture for a stand-up: what is waiting, what is being built and what is ready for review.

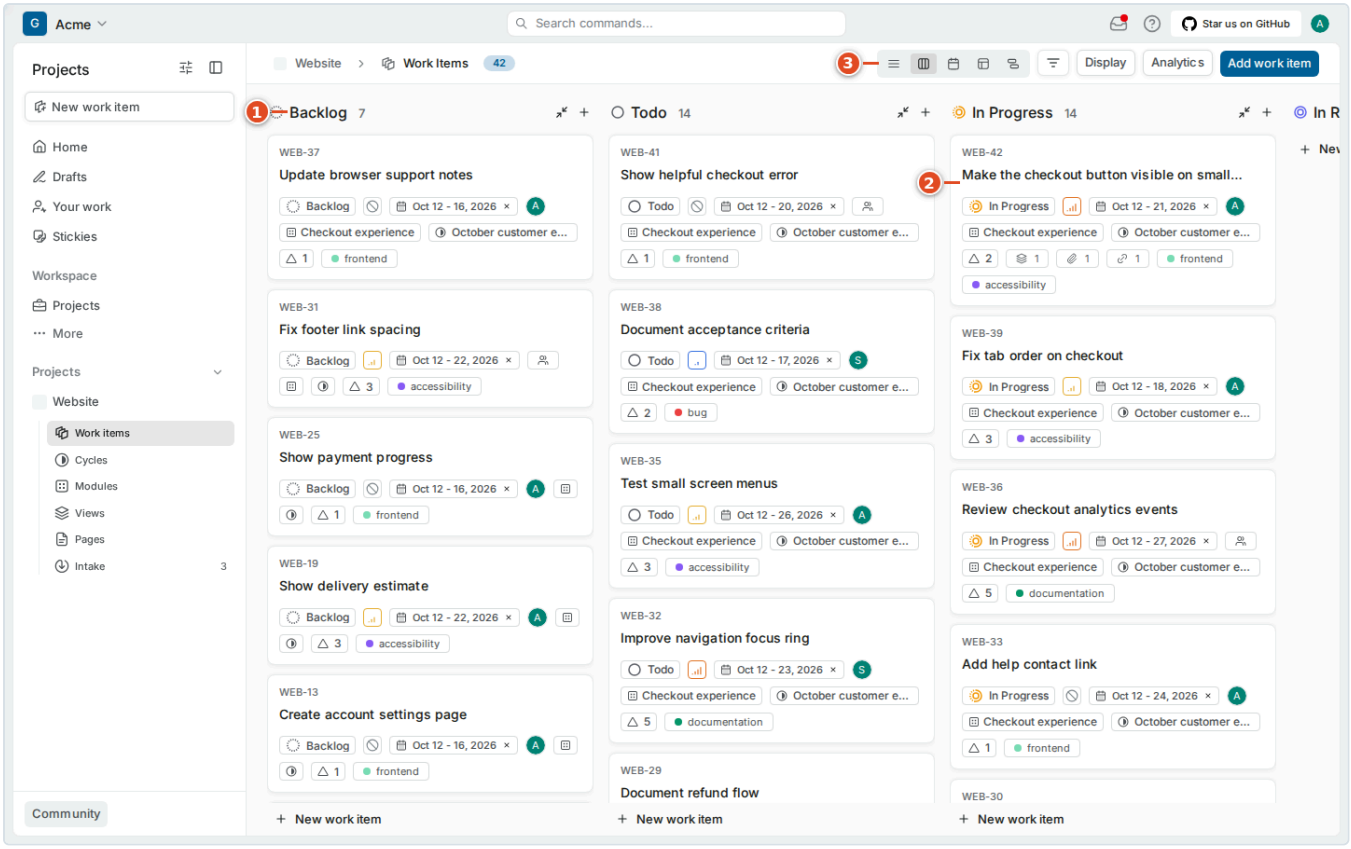


Figure 5.2 Screenshot (test Plane): Board layout: (1) state columns; (2) work item cards; (3) layout selector.

Goal: move work across the board

Steps

1. Select the board icon. ③
2. Check what the column headings represent ①; usually states.
3. Open a card ② before you change its state.
4. Drag the card to another column, or change its state in the item.
5. Check that the new value is correct.

Result: the card sits in the new column and the item's state has changed.

If the board is grouped by a different property, such as assignee, dragging a card changes that property instead. A wide board scrolls sideways; scroll before deciding that Done or Cancelled is missing.

> **Tip:** With git-sync's shipped rules you rarely need to drag cards between In Progress, In Review and Done: pushing a branch, requesting a review and merging do it for you. Watching the board after a merge is a quick way to confirm that sync is working.

5.4 Calendar: dated work

The calendar places items on their dates. It helps you review a delivery week or notice several items due on the same day. It is not a complete backlog, because undated items have nowhere to go.

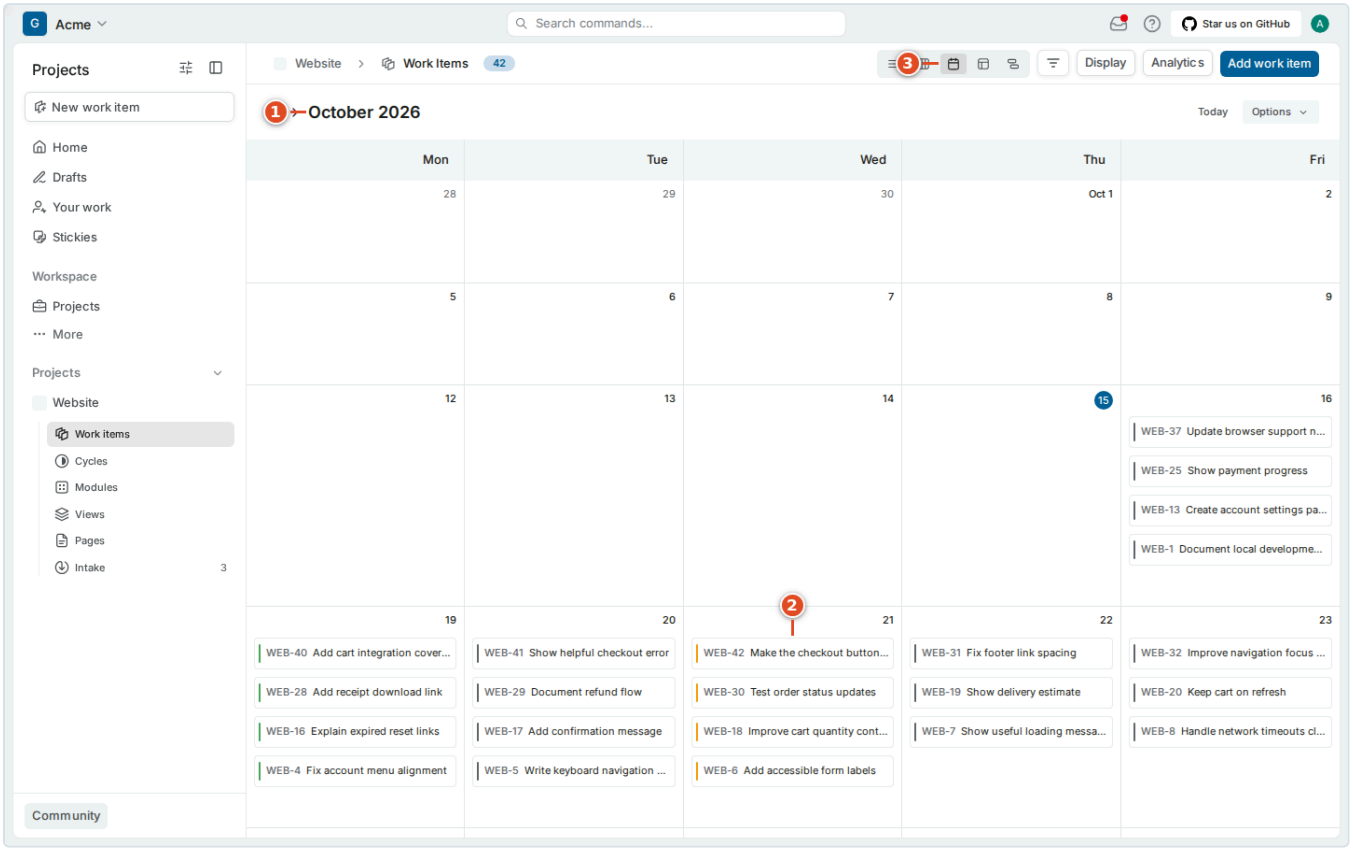


Figure 5.3 Screenshot (test Plane): Calendar: (1) displayed month; (2) WEB-42 on its due date; (3) calendar layout button.

Goal: review what is due this month

Steps

1. Select the calendar icon. ③
2. Check the displayed month ① and navigate to the period you need.
3. Open a dated item, such as `WEB-42` ②.
4. Check its date in the item's properties before changing it.

Result: you see the items due in the period.

Changing a deadline is a planning decision. Do not drag dates just to make a crowded calendar look tidier; discuss scope or capacity with the team instead.

5.5 Spreadsheet: careful property edits

The spreadsheet shows properties in columns. It is good for comparing state, priority, assignees, dates, modules and cycles across many items.

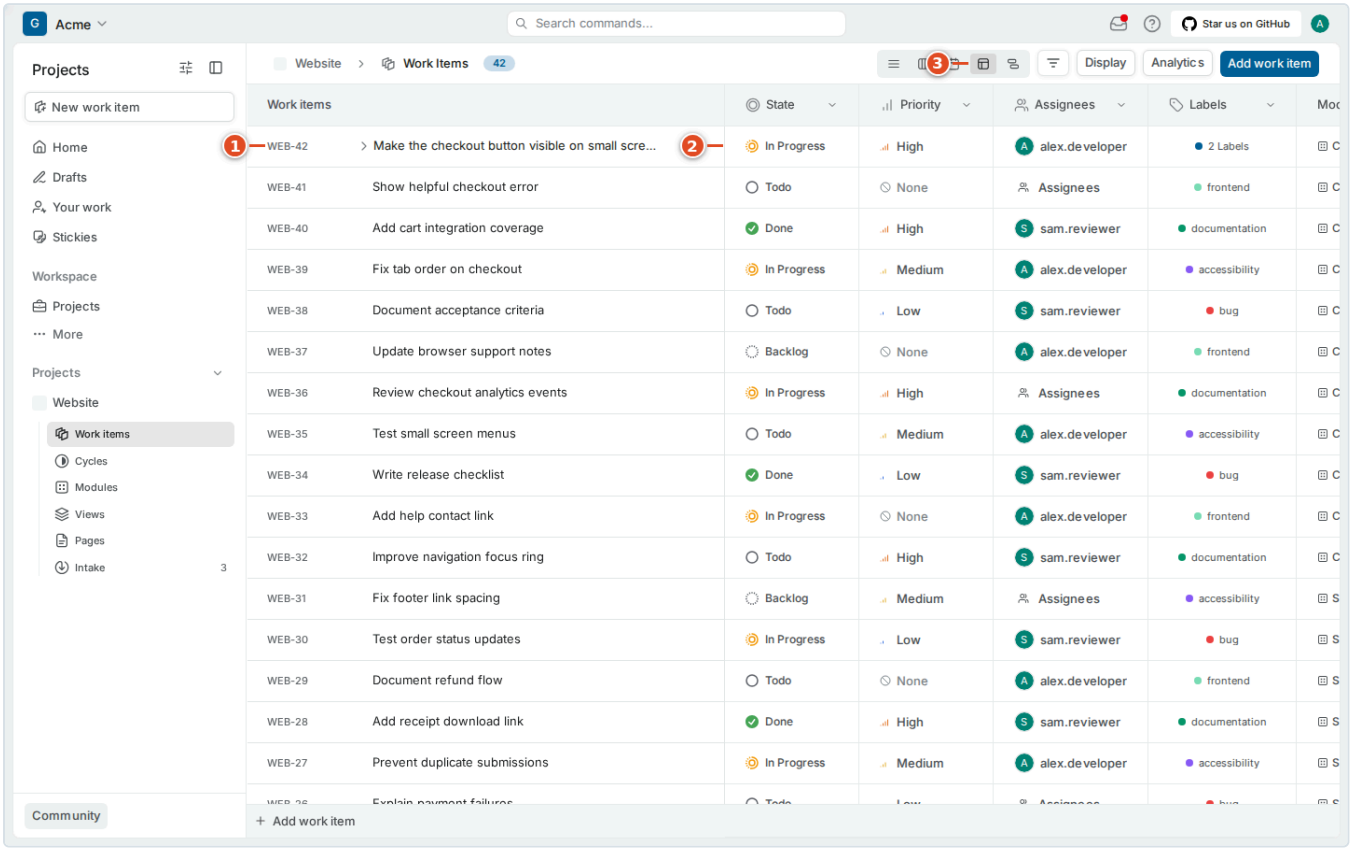


Figure 5.4 Screenshot (test Plane): Spreadsheet: (1) WEB-42 row; (2) state cell; (3) spreadsheet layout button.

Goal: change a property for one item

Steps

1. Select the spreadsheet icon. ③
2. Find the row by its identifier at the left, here `WEB-42`. ①
3. Select the cell to change, for example the state cell. ②
4. Choose the new value and wait for the update.
5. Check the row before editing another one.

Result: the item has the new value.

Some columns show counts or derived values and cannot be edited. Selecting several rows for a bulk action is different from editing one cell; read the scope of a bulk action before applying it.

> **Warning:** The spreadsheet is not a private draft. Every edit is a real change. On a mirrored item, a change to state, assignees or labels is sent to the linked GitHub issue just as if you had made it on the detail page.

5.6 Gantt: compare date ranges

Gantt shows items as bars along a time scale. Use it to see overlapping plans or to discuss start and target dates.

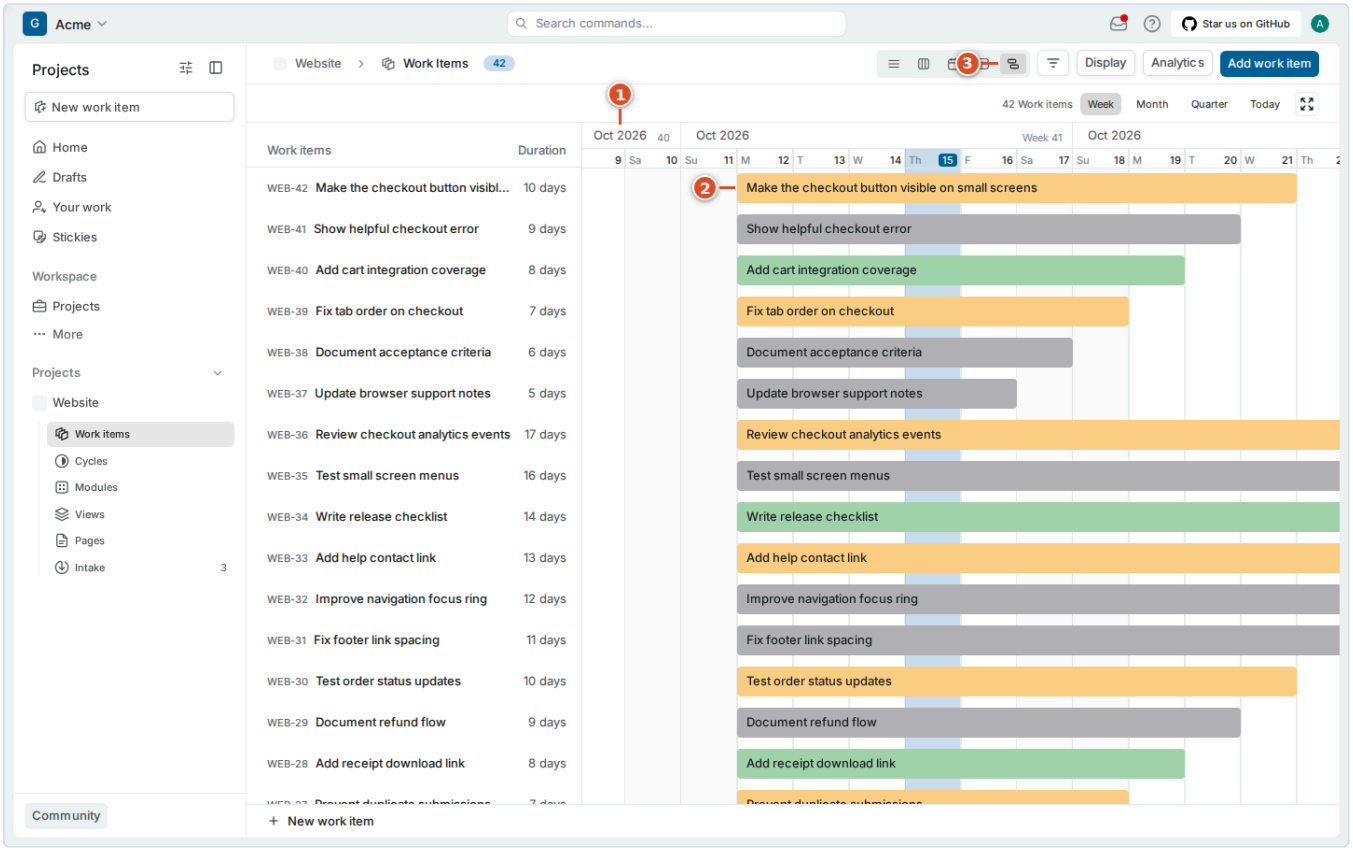


Figure 5.5 Screenshot (test Plane): Gantt: (1) time scale; (2) dated work-item bar; (3) Gantt layout button.

Goal: compare planned dates

Steps

1. Select the Gantt icon. ③
2. Check the time scale in the header. ①
3. Find the item and compare its bar ② with the dates.
4. Open the item for exact dates or dependencies.

Result: you see how planned ranges overlap.

A bar is a plan. It does not show that anyone has started, and it does not replace a **Blocked by** relation or a comment.

5.7 Filter without losing your place

The filter control offers State, State Group, Assignees, Priority, Mentions, Label, Cycle, Module, Start date, Target date, Created at, Updated at and Created by. Ask the narrowest useful question: “assigned to me and not finished” is easier to reason about than a dozen conditions.

Goal: show only the items you care about

Steps

1. Open the filter control beside the layout buttons.
2. Search ① or choose a property category ②.
3. Select one or more values.
4. Check the resulting items and count.
5. Remove the filter when you need the whole project again.

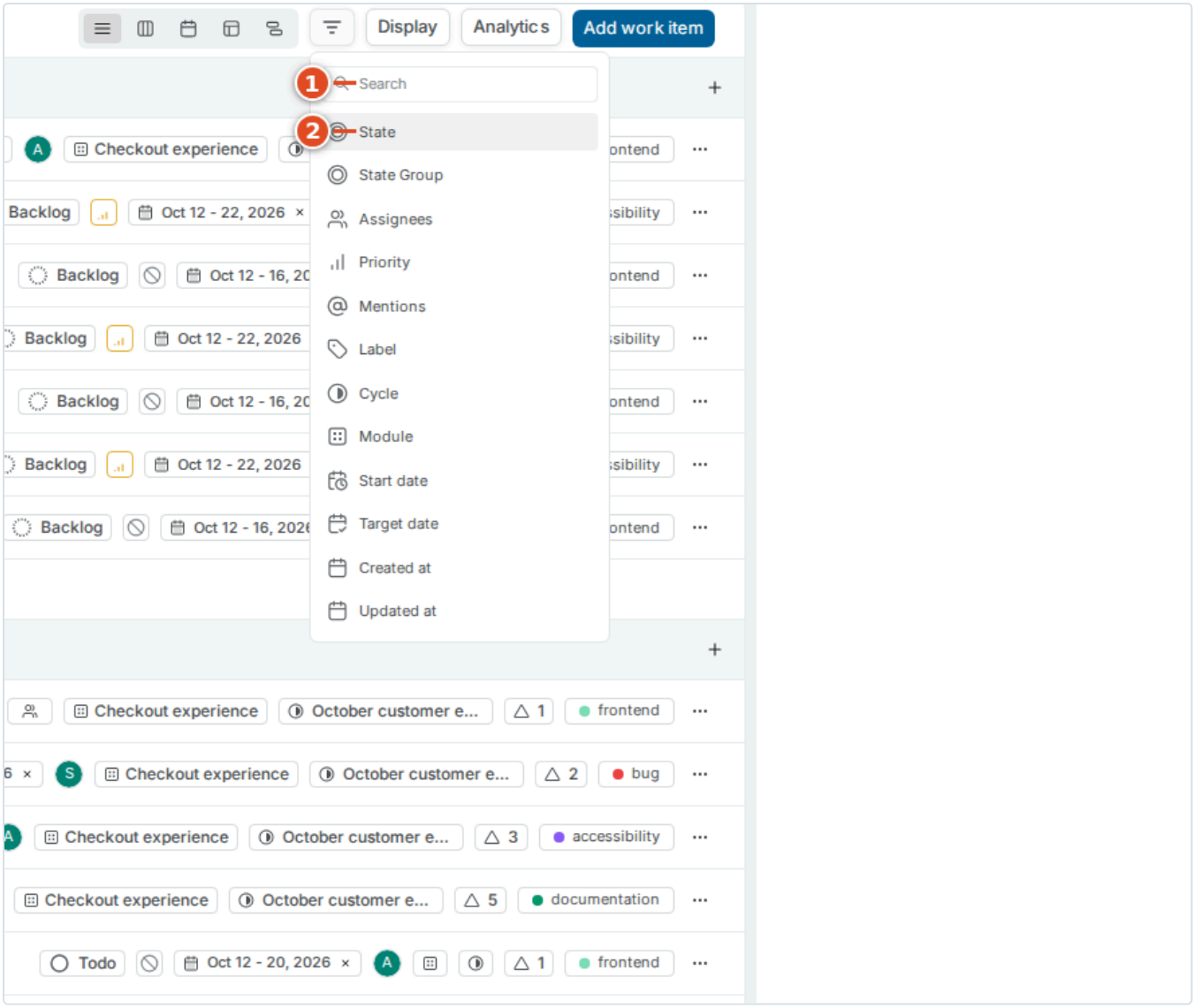


Figure 5.6 Screenshot (test Plane): Filters: (1) search field; (2) available property categories. Choose a category to select its values.

Result: only matching items are shown.

Read the displayed selection rather than assuming every click adds an “and”. Relative date filters such as “this week” show different items as time passes, even if nobody edits the view.

> **Tip:** To see everything your team is working on in GitHub right now, filter **State Group** to Started. Items that git-sync moved to In Progress or In Review because a branch or PR exists will show up there.

When a teammate says an item is missing, compare project, filters, layout and visible period. Sending the direct link quickly separates a display problem from an access problem.

5.8 Display: properties, grouping and ordering

Display controls which properties appear and how items are grouped and ordered. In the list layout you can show ID, Assignee, Start date, Due date, Labels, Priority, State, sub-work item count, attachment count, link, Estimate, Module and Cycle.

Goal: set up the list the way you work

Steps

1. Open **Display**.
2. Choose the properties to show. ①
3. Choose **Group by**. ②
4. Choose **Order by**.
5. Turn **Show sub-work items** on or off. ③
6. Close the menu.

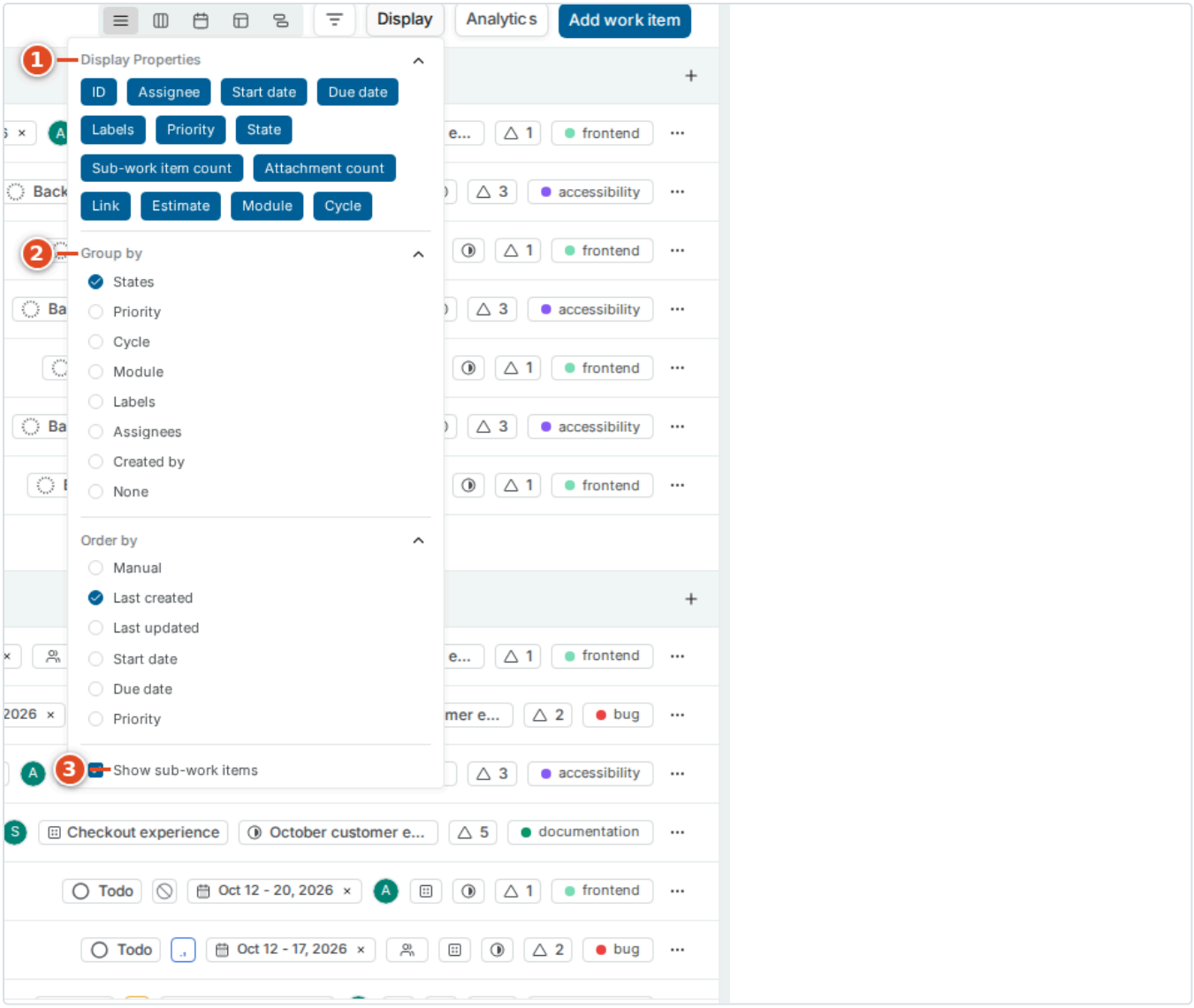


Figure 5.7 Screenshot (test Plane): Display: (1) visible properties; (2) Group by; (3) Show sub-work items. Ordering is also in this menu.

Result: the layout shows the properties and grouping you chose.

You can group by state, priority, cycle, module, labels, assignees or creator, or not at all. Ordering includes manual order, created, updated, dates and priority. Choices differ by layout. Hiding sub-work items does not detach them from their parents.

> **Note:** The **link** count includes the development links git-sync adds (`[PR]` , `[Branch]` , `[CI]` …). An item with a link count but no manual links is usually one that already has code connected to it.

5.9 Save a useful view

A **saved view** names a useful query. Choose a name that says what it shows, such as “Frontend work needing review”, not “My view 2”.

Goal: save a view the team can reuse

Steps

1. Open **Views** under the project.
2. Start a new view with **Add view**. ②
3. Enter a descriptive name and a short explanation.
4. Configure the filters.
5. Save the view.
6. Open it again to check the items it shows. ①

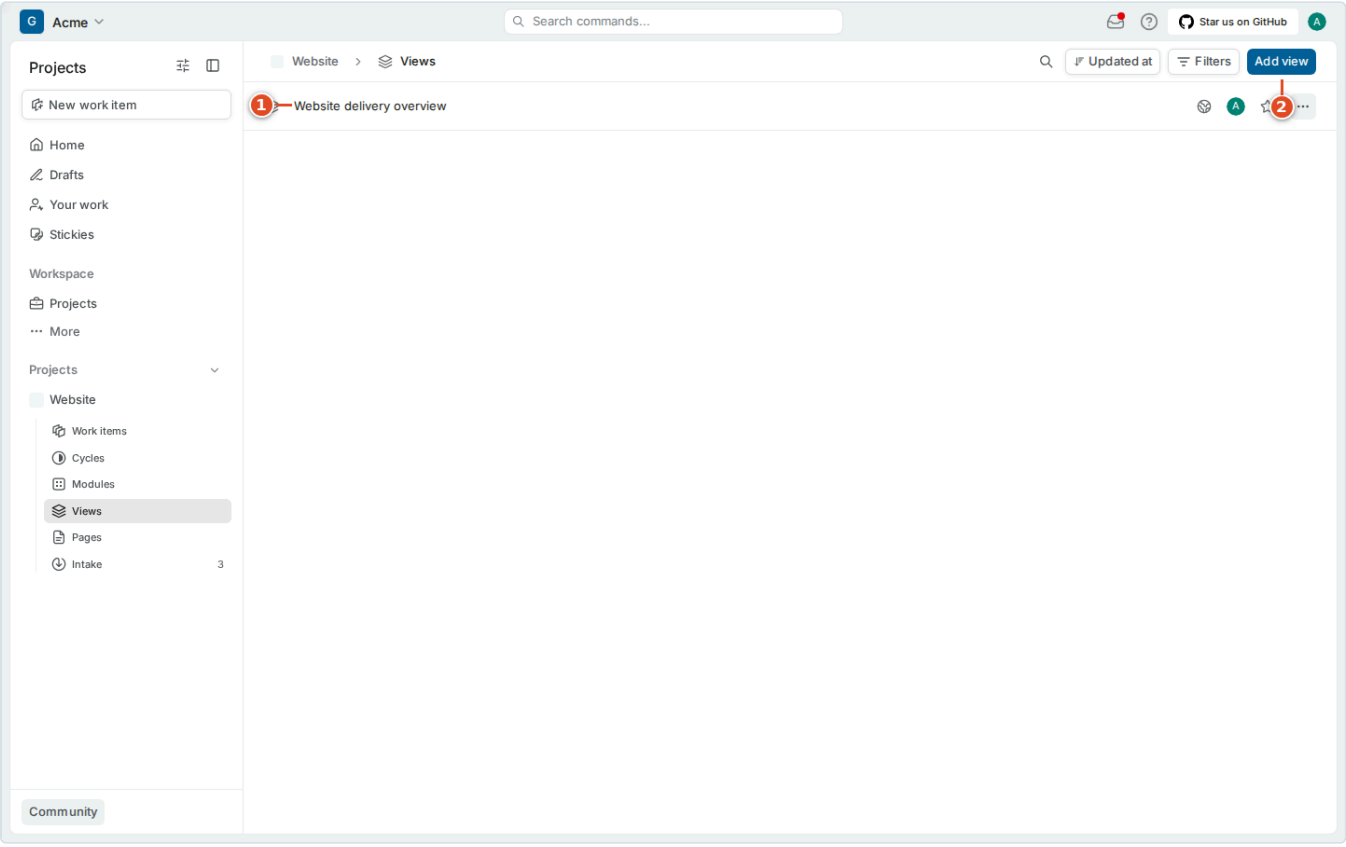


Figure 5.8 Screenshot (test Plane): Project Views: (1) Website delivery overview; (2) Add view creates another saved selection.

Result: the view appears in the list and anyone with access can open it.

A view does not copy items. An item can appear in many views, and a change made through one view shows wherever the item matches. Before changing the meaning of a shared view, ask the team; others may rely on it every day.

> **Note:** Saved views and pages are Plane-only. git-sync never syncs them to GitHub.

What you learned

- Layouts, filters, grouping and ordering change only your view; edits to property values change the real item.
- List, Board, Calendar, Spreadsheet and Gantt answer different questions.
- Every edit in any layout is a real change, and on mirrored items it can sync to GitHub.
- Saved views name a reusable query and never copy items.

Source files

- Plane CE v1.4.2 web application (`apps/web/`), observed in the running test instance
- `src/sync/mirror.ts` (which mirrored fields sync)
- `src/sync/dev-info.ts` (development links that count as item links)

06 · Planning

Plane's planning tools answer different questions:

- A **cycle** is a time window: what will we deliver in these two weeks?
- A **module** is a feature area or larger outcome: how is the checkout work going overall?
- A **page** holds written knowledge: how do we review code here?
- **Intake** receives new requests before the team accepts them: what came in, and do we want it?
- A **saved view** is a reusable query (chapter 5).
- **Estimates**, **states** and **analytics** help the team size work, agree on a workflow and see trends.

All of these are present in the Plane CE v1.4.2 test project. A project administrator can turn optional ones on or off.

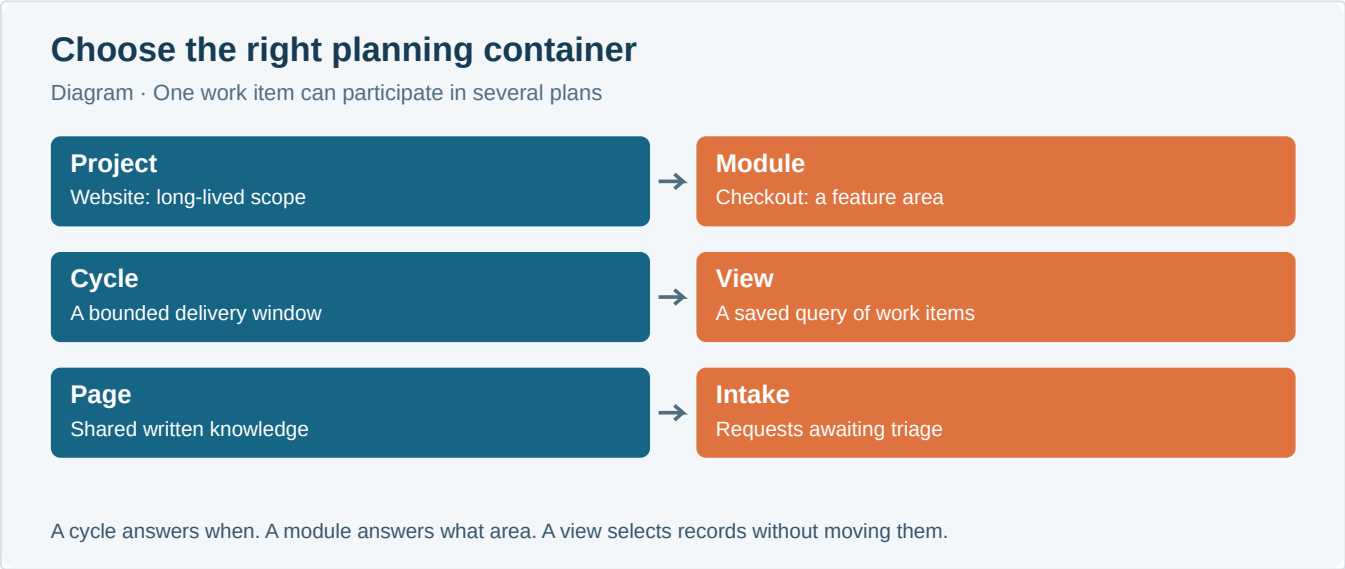


Figure 6.1 Diagram: project, module, cycle, view, page and intake serve different planning purposes.

A work item can be in a module and a cycle at the same time. Adding it to either does not copy it. The original item stays the place for its implementation and discussion.

> **Note:** None of these planning containers sync with GitHub. git-sync never copies cycles, modules, pages, views, estimates or intake requests, and it never sets an item's cycle or module. Your team's plan lives only in Plane.

6.1 Work within a cycle

A **cycle** is a bounded period used to plan and check delivery; some teams call it a sprint. The example cycle **October customer experience** has start and end dates and contains the Website items chosen for that period.

Goal: see what your team committed to this cycle

Steps

1. Open **Cycles** under the project.
2. Find the current cycle by name ① and check its dates ②.
3. Open the cycle.
4. Read its work items ① and its dates and progress ②.

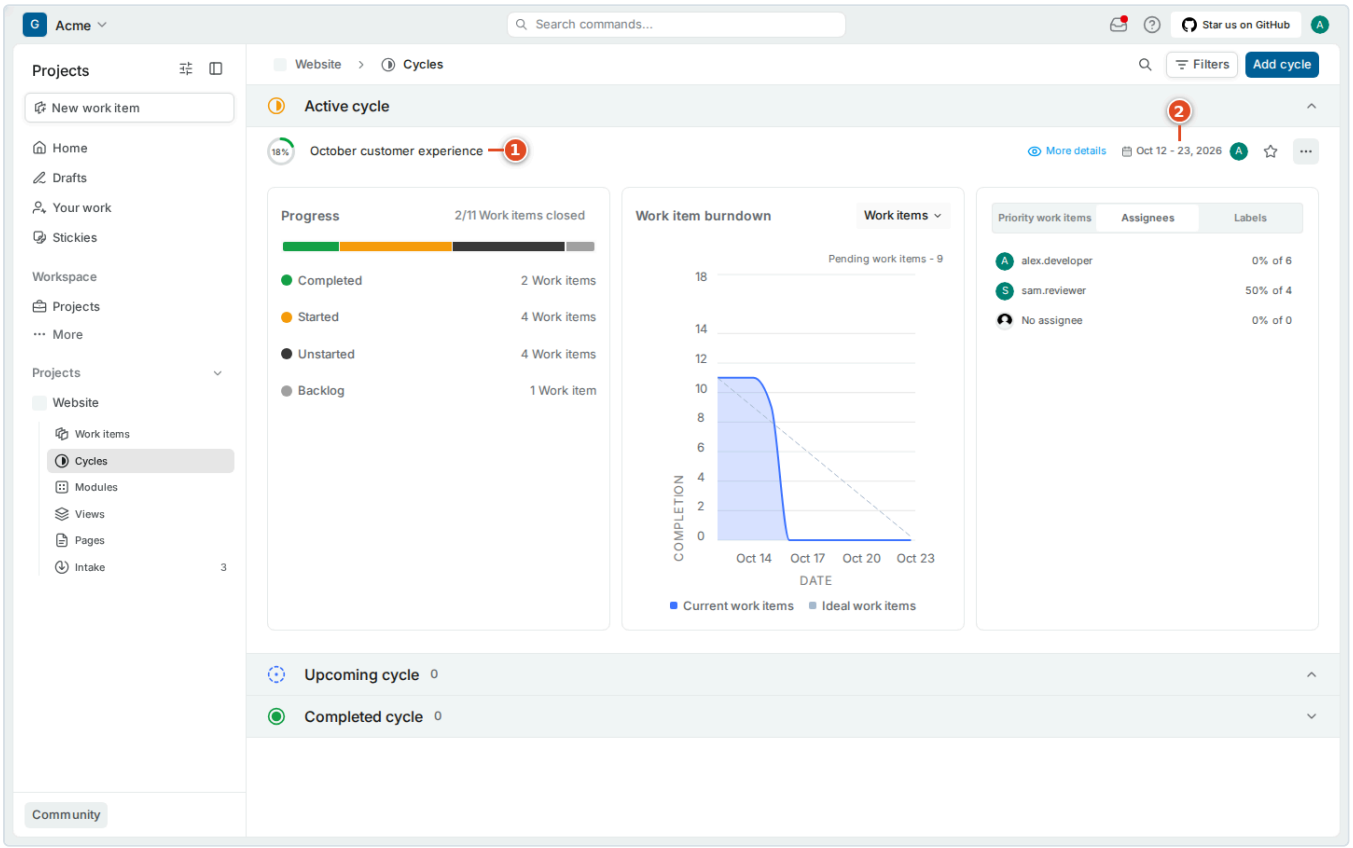


Figure 6.2 Screenshot (test Plane): Cycles: (1) cycle name; (2) scheduled date range.

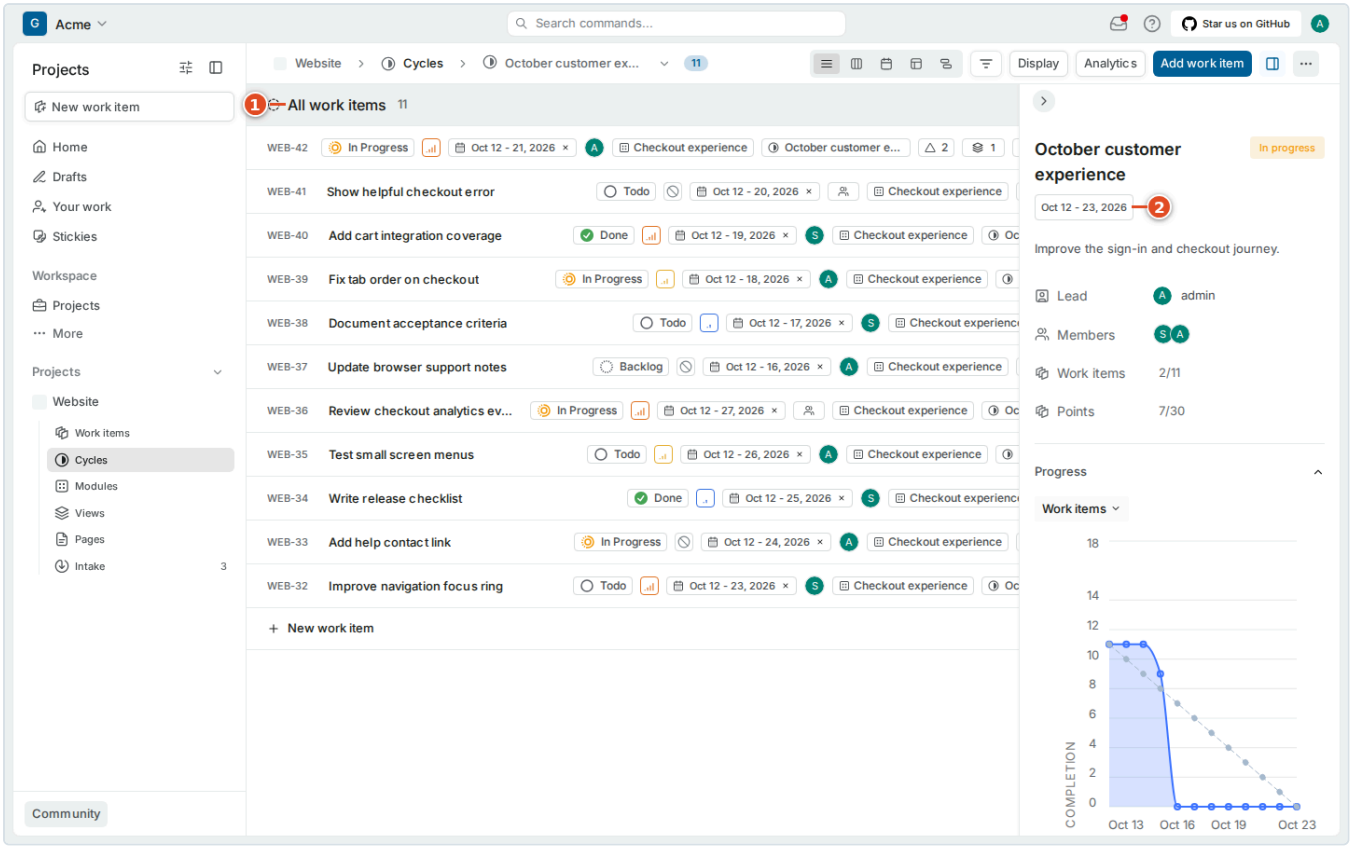


Figure 6.3 Screenshot (test Plane): Inside a cycle: (1) included work items; (2) dates and progress.

Result: you see the cycle's committed work and how far it has progressed.

To add an item to a cycle, set its **Cycle** property (or press **Shift + C** on the item). Check the dates so you do not pick a similarly named past cycle.

Cycle progress comes from the included items and their states. It changes when items are added, removed or move between states, which includes state changes made by git-sync when PRs open and merge. A percentage is not a promise about the remaining time; one unresolved dependency can matter more than several finished small tasks.

> **Tip:** In planning, when an urgent item enters a committed cycle, ask what moves out. Changing the cycle is easy; making the capacity decision visible is the useful part.

6.2 Use modules for a feature area

A **module** groups work around an area or larger outcome. The example module **Checkout experience** contains several Website improvements and can span more than one cycle. The module has its own status and dates; each item keeps its own state.

Goal: follow progress on a feature area

Steps

1. Open **Modules** under the project.
2. Find the module ① and check its status ②.
3. Open the module.
4. Read its description, properties and progress ②, and review its items ①.

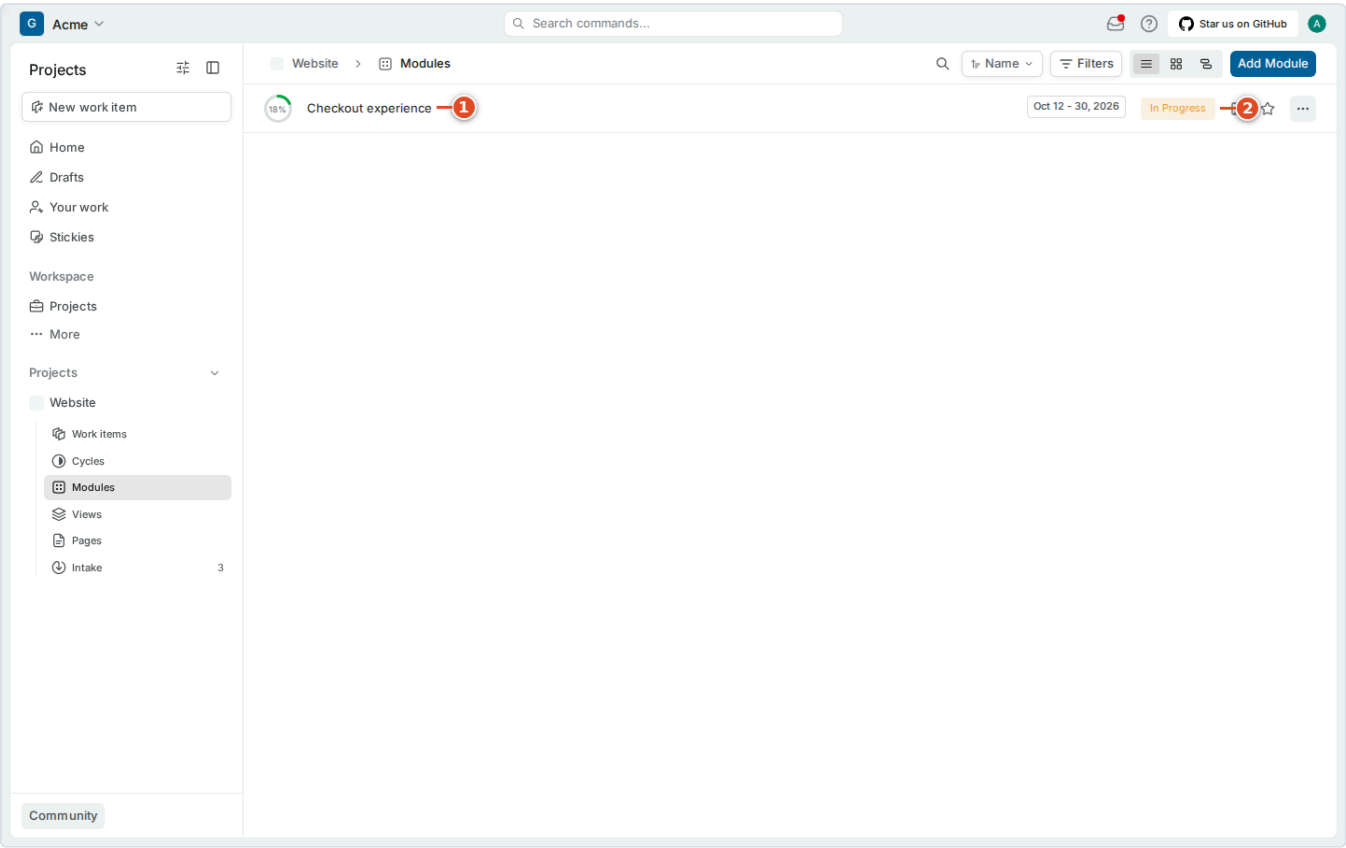


Figure 6.4 Screenshot (test Plane): Modules: (1) Checkout experience; (2) In Progress status.

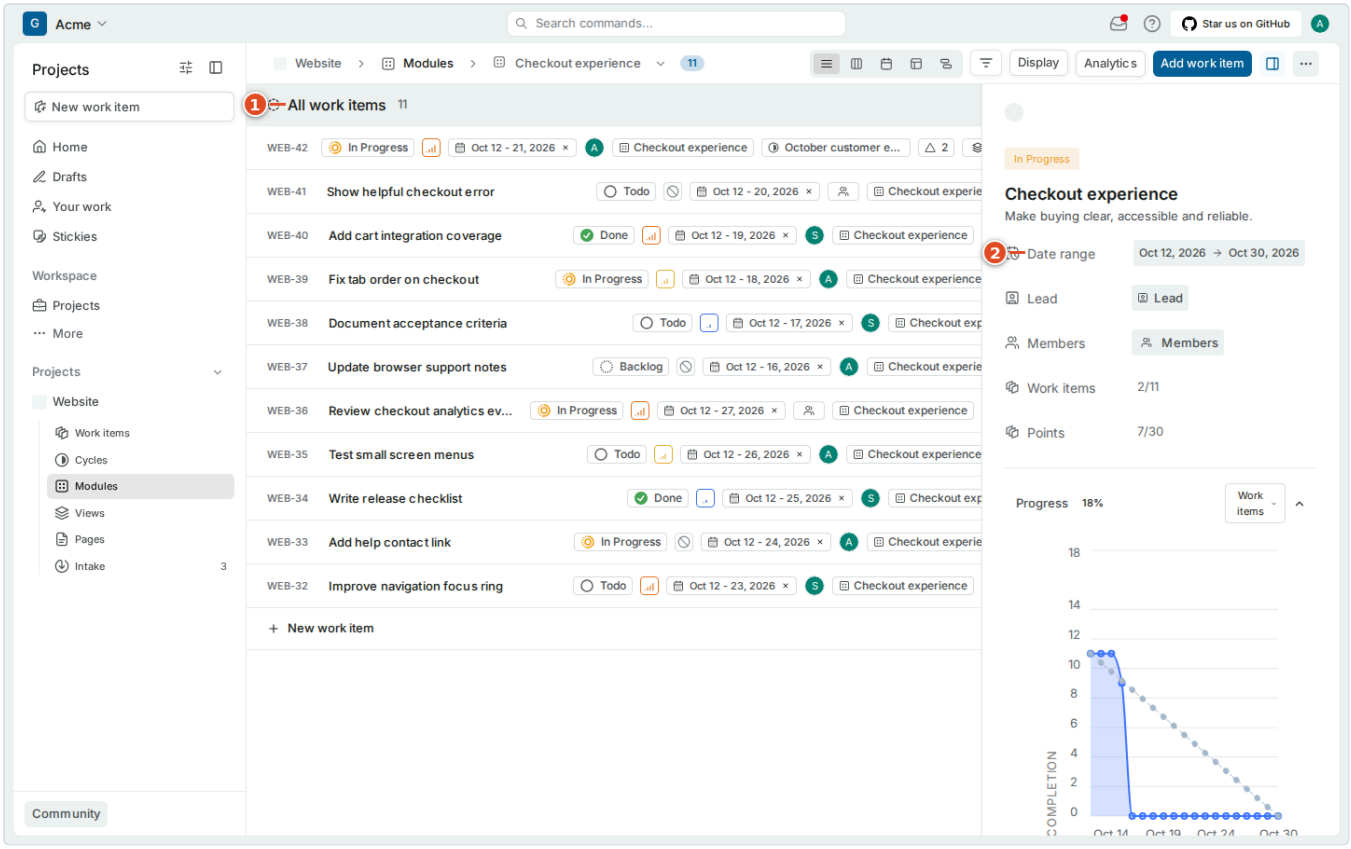


Figure 6.5 Screenshot (test Plane): Inside a module: (1) included work items; (2) module properties and progress.

Result: you see every item that contributes to the area.

Use the item's **Modules** property (**Shift + M**) to add it to a module. A parent relationship describes how one outcome breaks down into parts; a module is a planning collection. Do not build a huge parent item just to reproduce a module. Marking a module complete does not merge PRs or complete its open items.

6.3 Keep team knowledge in Pages

A **page** is a shared document: working agreements, design notes, review checklists, onboarding instructions. A work item's description holds its own acceptance criteria but can link to a page for a convention that many items share.

Goal: read the team's working agreement

Steps

1. Open **Pages** under the project.
2. Find the document, here **Website working agreement** ①.
3. Open it and read its title ① and checklist ②.

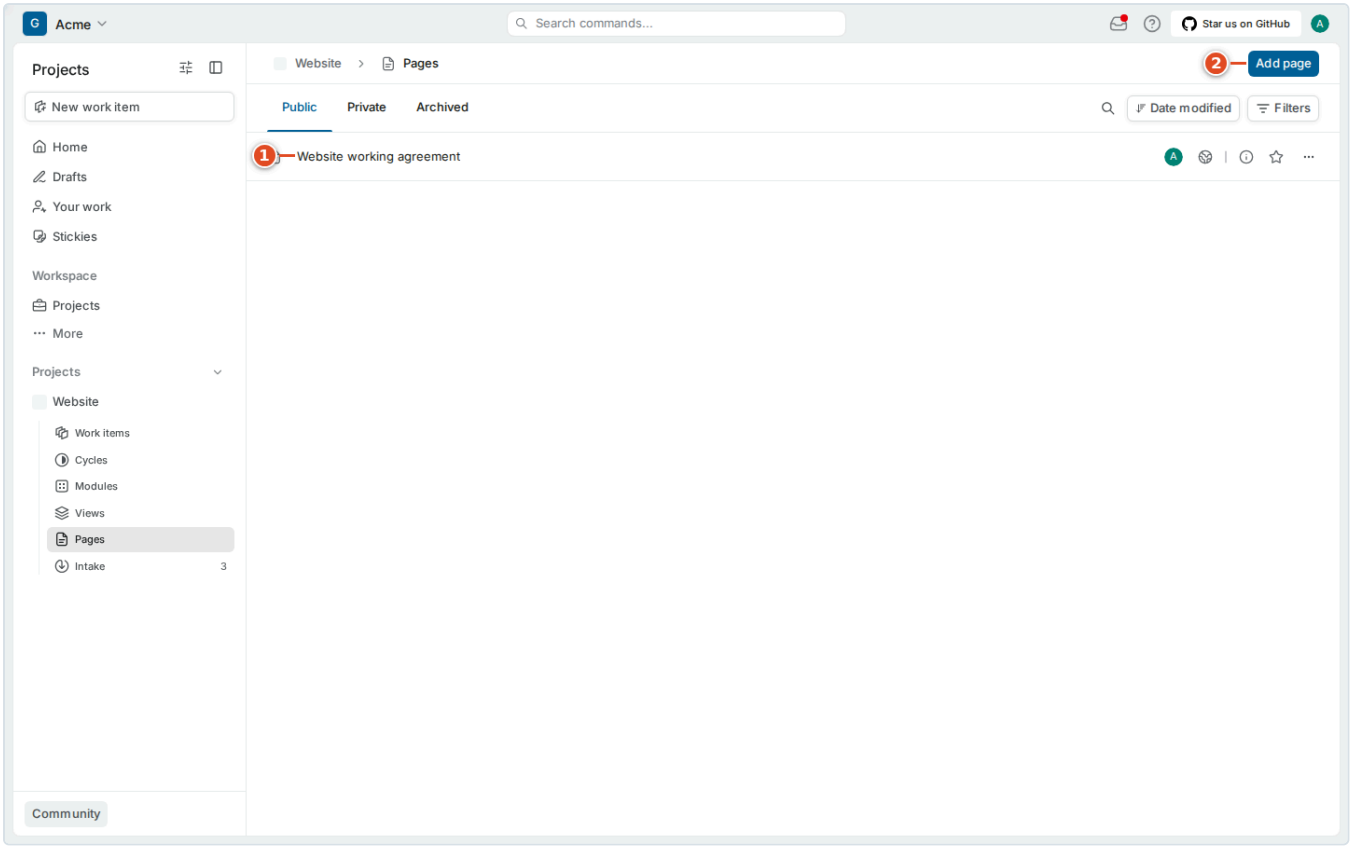


Figure 6.6 Screenshot (test Plane): Pages: (1) the shared working agreement; (2) Add page.

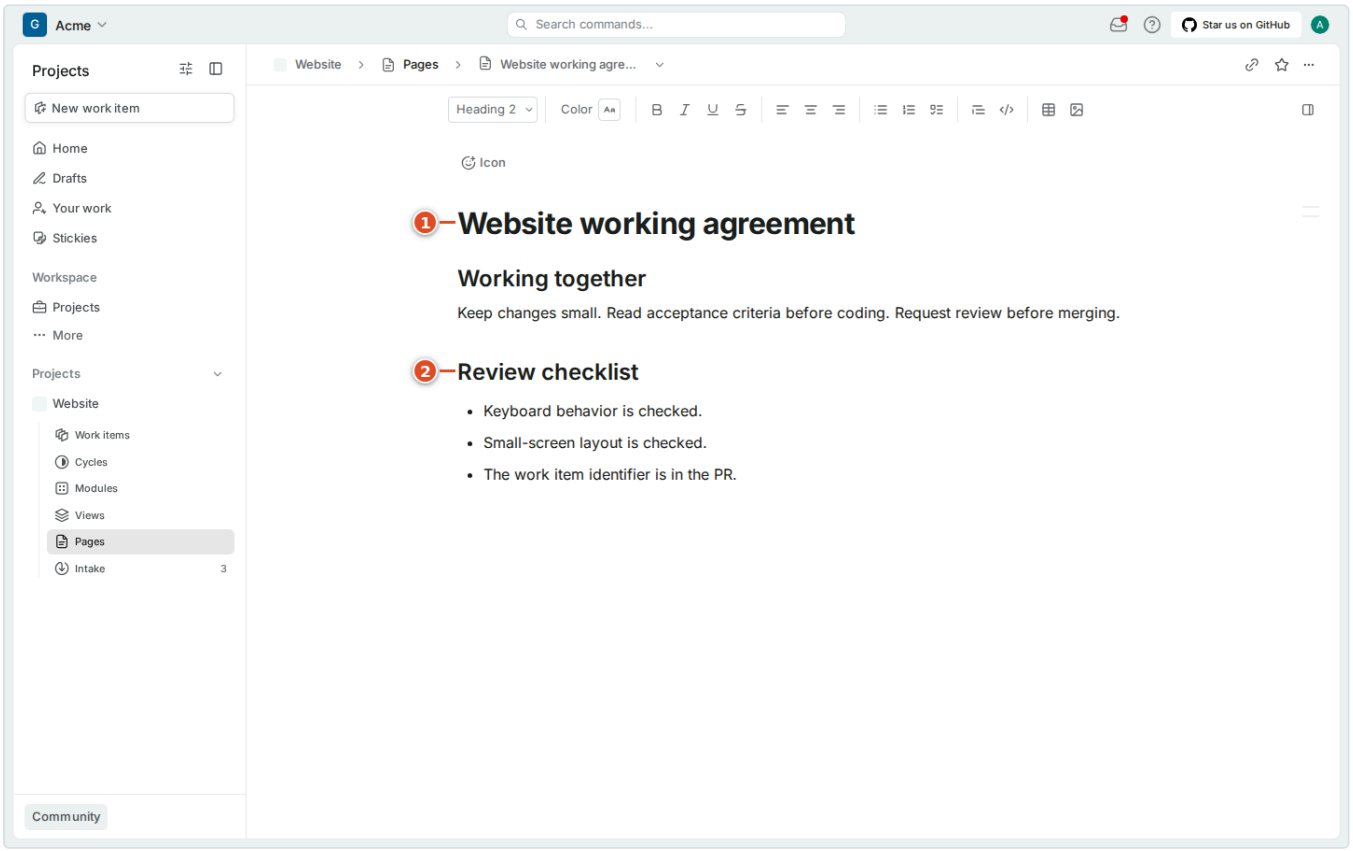


Figure 6.7 Screenshot (test Plane): Shared page: (1) document title; (2) review checklist.

Result: you know the team’s agreed way of working.

To write a page, select **Add page**, give it a clear title and say who it is for. Use headings, and keep checklists short enough to be used. Link to a work item for a specific decision instead of copying a long comment thread.

Page actions such as favorites, locking, archiving and sharing depend on your role. Public sharing can expose content outside the team; follow your organization’s policy. A shortcut listed in the keyboard dialog does not prove your account may use the action.

> **Tip:** A page is a good home for your team’s git-sync conventions: branch naming, when to write `Closes` versus `Refs`, and which Smart Commit commands to use. Chapter 16 has a starting template.

6.4 Triage new requests through Intake

Triage means deciding what to do with a new report before it becomes planned work. **Intake** keeps incoming bugs, ideas and feedback apart from the regular queue. A new request is **Pending** and its state is **Triage**. The detail screen offers **Accept** and **Decline**.

Goal: submit a request through Intake

Steps

1. Open **Intake** under the project.
2. Select **Add work item**.
3. Enter a specific title and describe what you observed.
4. Select **Create work item**.
5. Find the new request in the pending list ①.

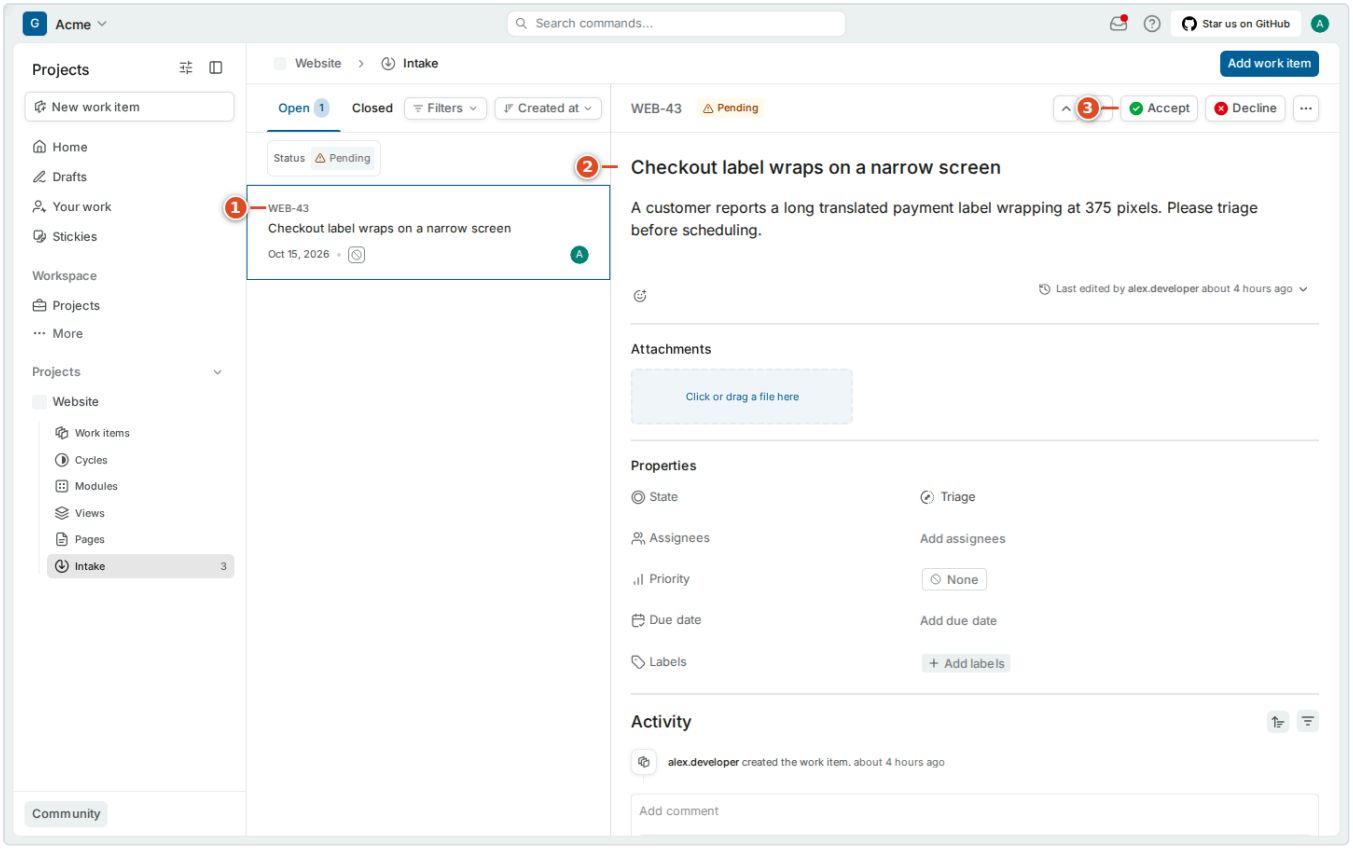


Figure 6.8 Screenshot (test Plane): Intake: (1) pending request WEB-43; (2) report title; (3) Accept action.

Result: the request waits for triage. Whoever triages it checks for duplicates, clarifies it and then selects **Accept** ③ or **Decline**.

Accepting is a decision, not a way to clear an unread count. Declining is not the same as completing. Leave a reason so the reporter knows what happened. After accepting, check the item's regular state, priority and owner.

> **Note:** git-sync never moves a work item *into* a Triage state, and its normal rules work on accepted items. Have a request accepted before you put its key in a branch or PR and expect automation to apply.

If **Intake** is missing from the project navigation, the team may not use it. Ask who handles incoming requests instead of starting a private backlog.

6.5 Understand the estimate system

A project administrator chooses the estimate system. The **Points** templates offer Fibonacci `1, 2, 3, 5, 8, 13`, Linear `1, 2, 3, 4, 5, 6` and Squares `1, 4, 9, 16, 25, 36`, plus a custom scale. The Website project uses Fibonacci points.

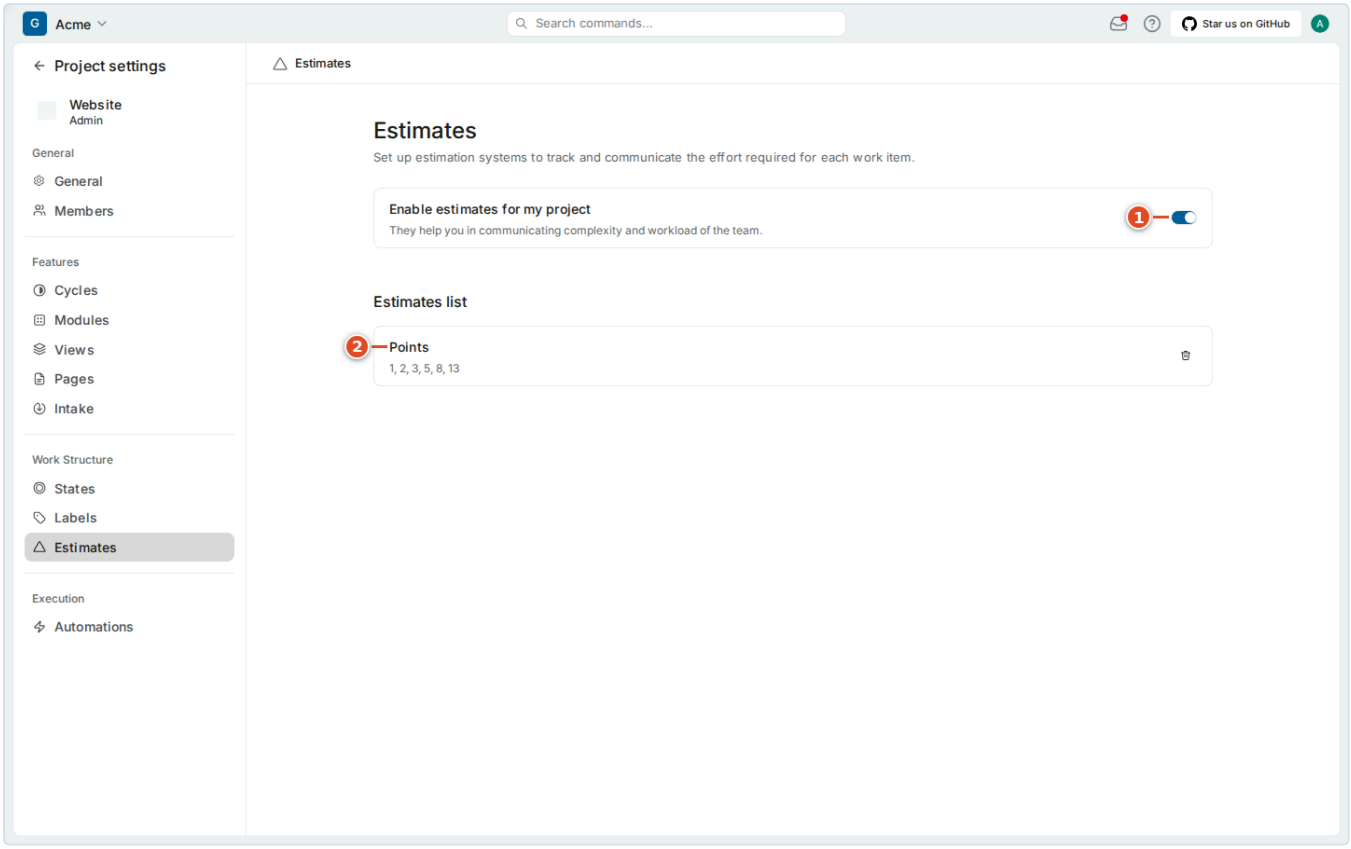


Figure 6.9 Screenshot (test Plane): Estimates: (1) project enable control; (2) configured Fibonacci Points scale. Administrator view.

As a member, set an item's **Estimate** (**Shift + E**) by comparing it with items the team has already sized. If the team has not agreed what a point means, do not convert points to hours in a report. Changing the estimate system affects how existing values are read, so it is a team and administrator decision.

> **Note:** Estimates do not sync to GitHub, and git-sync's Smart Commit `#time` command does not change them. Time logged with `#time` appears only as a comment (chapter 11).

6.6 Read the project's states and labels

States and labels are the team's shared vocabulary. Project settings show states organized by group, and the project's labels. Your role decides whether you can edit them.

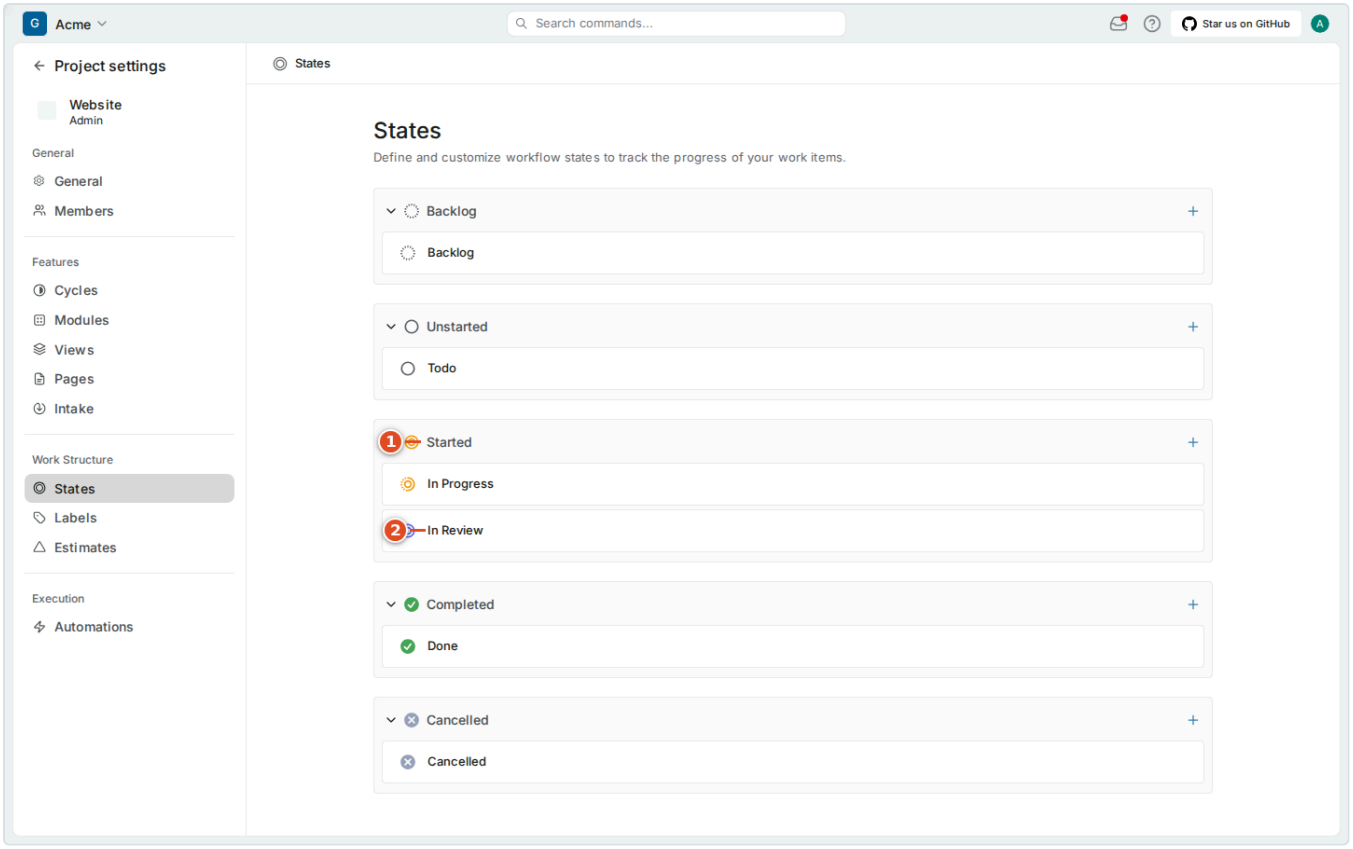


Figure 6.10 Screenshot (test Plane): Project states: (1) Started group; (2) In Review state in that group. Administrator view.

> **Warning:** git-sync's rules and Smart Commits refer to states by group and name, for example `started:In Review` or `#in-review`. Renaming or deleting a state that a rule uses can make that rule fail, and the failure appears in the audit log. Ask the project administrator and the git-sync administrator before renaming states such as In Progress, In Review or Done. Likewise, keep the labels **github** and **plane** if your team mirrors issues (chapter 13).

The **Automations** entry in Plane's project settings is separate from git-sync's rules. A rule configured in one place does not appear in the other. For any state change triggered from GitHub, the git-sync mapping and its rules are the place to look (chapters 14 and 26).

6.7 Use the project analytics

The project's Work items header has an **Analytics** button. It opens a summary with the total number of work items, counts by state group, a **Created vs Resolved** chart, breakdown controls, and priority and assignee tables with CSV export.

Goal: look at the project's trend

Steps

1. Open the project's **Work items** page.
2. Select **Analytics**.
3. Read the summary ① and the **Created vs Resolved** chart ②, checking the scope and labels.
4. Compare with the project's records before drawing conclusions.

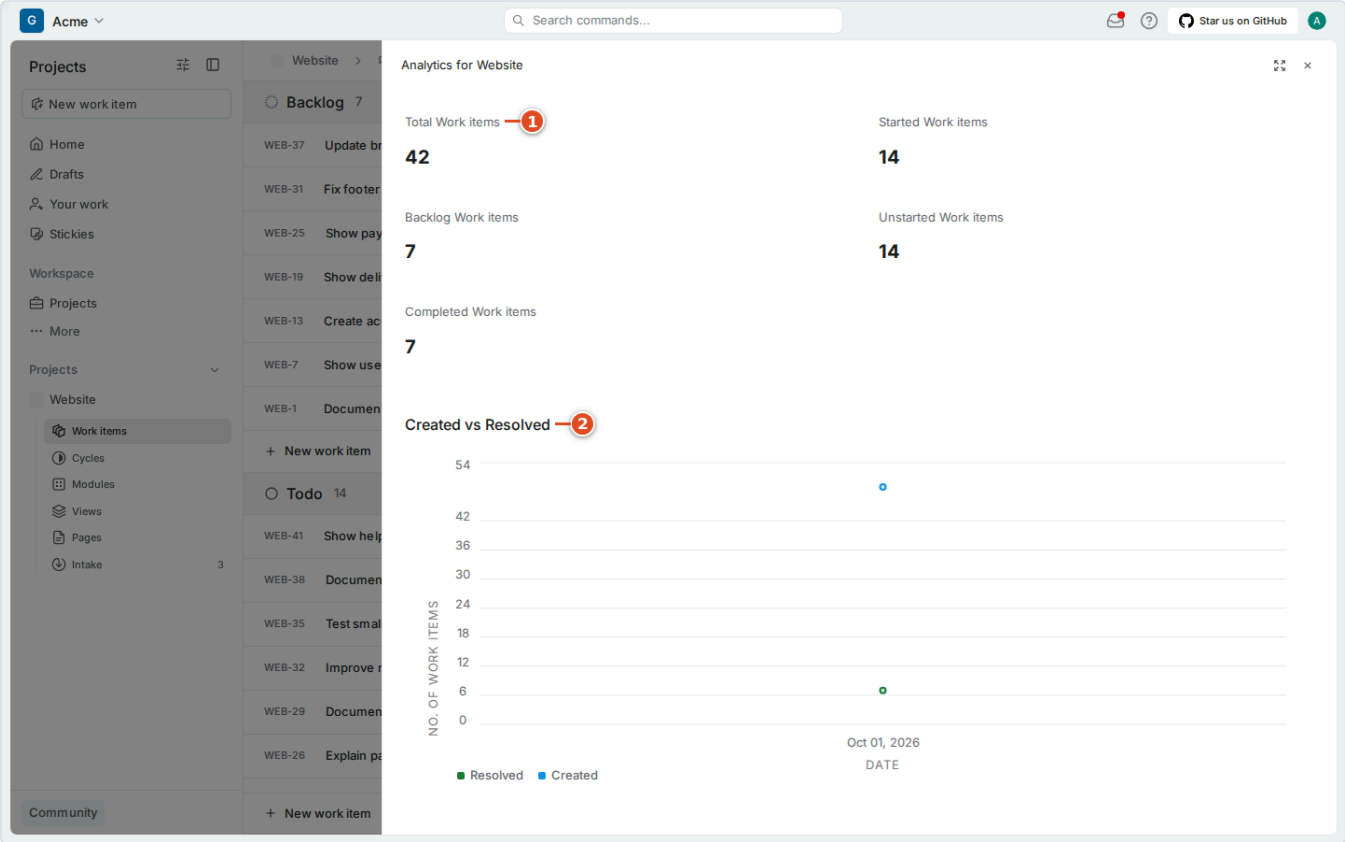


Figure 6.11 Screenshot (test Plane): Project analytics: (1) total work-item summary; (2) Created vs Resolved chart.

Result: you can see whether work is being resolved as fast as it arrives.

Counts describe a selection of records. They do not measure individual productivity or quality. Use charts to ask planning questions, such as why started work is piling up or whether there is enough review capacity. Workspace-level **Analytics** in the workspace navigation covers all projects; check its scope before comparing numbers. This manual does not assume reporting features advertised for other Plane editions.

What you learned

- Cycles are time windows, modules are feature areas, pages hold knowledge, and Intake holds requests awaiting triage.
- An item can be in a cycle and a module at once; neither copies it.
- Planning data (cycles, modules, pages, views, estimates, intake) never syncs to GitHub.
- Renaming states that git-sync rules use can break automation; coordinate first.
- Analytics show trends in the work, not individual performance.

Source files

- Plane CE v1.4.2 web application (`apps/web/`), observed in the running test instance
- `src/github/pr-automation.ts` (state targets such as `started:In Review`)
- `src/github/smart-commits.ts` (state commands; never into triage)
- `src/sync/mirror.ts` (fields that are never synced)

07 · Notifications

The **inbox** helps you notice relevant changes without rereading every item in every project. A notification usually needs one of three responses: read it for context, answer a question, or take on a next action. Opening it is only the first step; decide which response it needs.

This chapter covers the inbox, mentions, subscriptions and email, and how git-sync's automatic updates show up among your notifications.

7.1 Open the inbox

The inbox control is in the top bar; the shortcut is **G then X**. Your notifications depend on your assignments, subscriptions and activity, so your list differs from a teammate's.

Goal: deal with today's notifications

Steps

1. Select the inbox icon in the top bar.
2. Check the selected category, for example **All** ①, or switch to **Mentions** ③.
3. Open a notification, such as Sam's comment ②.
4. Read the work item and the surrounding discussion.
5. Respond on the work item if a response is needed.

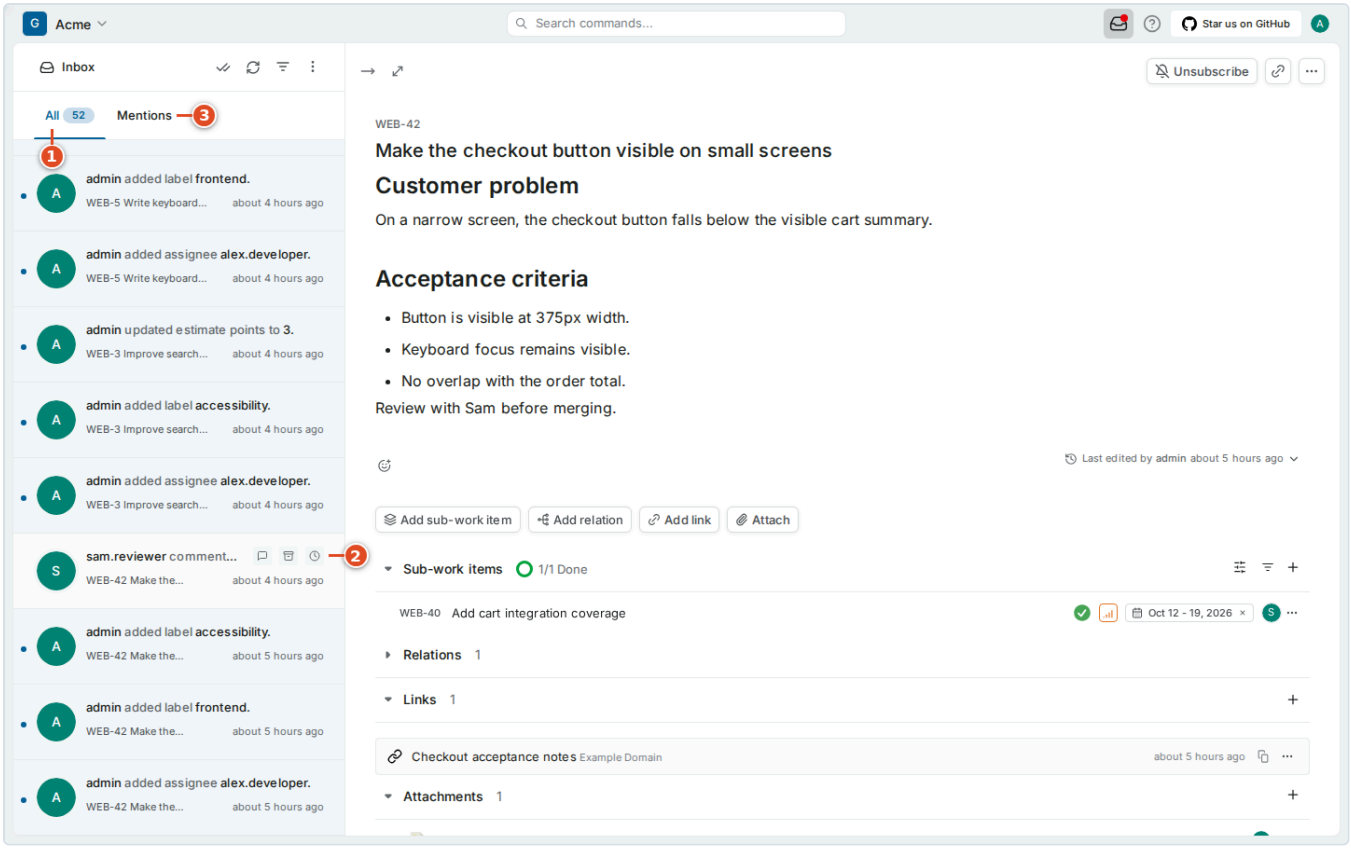


Figure 7.1 Screenshot (test Plane): Inbox: (1) All category; (2) Sam's comment notification; (3) Mentions category.

Result: you have read the update in context and answered where needed.

A notification can describe something that has since changed again. Read the current work item before acting on an old state or assignment. If someone has already answered a question, a short acknowledgment is more helpful than repeating the answer.

Use the inbox's read, archive and filter controls as labelled on your account. Marking a notification read does not complete the work item. An archived notification and an archived work item are different things. If a notification disappears, check the inbox category and filters before assuming the item was deleted.

7.2 Notifications that come from git-sync

git-sync changes Plane through a **service account**, the bot user your administrator created for it. Its changes are ordinary Plane changes, so they produce ordinary notifications. On items you are assigned to or subscribed to, expect notifications like these:

What happened on GitHub	What you see in Plane
A branch named after the item was pushed, or a PR was opened	State changed to **In Progress**
A review was requested on the PR	State changed to **In Review**

What happened on GitHub	What you see in Plane
The PR was merged into `main`	State changed to **Done**
A commit contained `WEB-42 #comment …` or `#time …`	A new comment with a “via `<commit>` by `<login>`” line
A new comment on a mirrored GitHub issue	A new comment beginning “**login** commented on GitHub:”
A PR, build, deployment or preview changed	A development link was added or updated (`[PR]` , `[CI]` , `[Deploy]` , `[Preview]` , `[Branch]`)

These are the shipped defaults; your administrator may have changed the rules (chapter 14). The actor shown for these changes is the git-sync service account, not you, even when you made the commit. Plane-side writes are made by that account.

> **Tip:** If you receive many property-change emails because of automatic state changes, turn off **Property changes** email (section 7.4) but keep **State change** and **Work item completed**. You will still hear when your item reaches Done.

> **Note:** A busy PR can update its `[PR]` link several times: when it opens, when reviews arrive and when it merges. That is git-sync keeping the link current, not a fault.

7.3 Mentions are requests for attention

A **mention** directs a comment or description at a specific person. Type `@` in a supported editor and pick the person from the suggestions. Give enough context for them to answer without redoing your investigation.

> **Example:** “@Alex, should the button keep its full translated label at 320 pixels, or may it wrap onto two lines?” asks for a decision. “@Alex, please look” does not say what decision is needed.

Typing someone’s name without selecting it from the list does not create a mention. Use mentions sparingly so they stay meaningful; if something is urgent, use the priority and your team’s normal process.

> **Note:** On mirrored issues, git-sync translates mentions. A GitHub `@sam-reviews` in an issue comment becomes a real Plane mention of Sam if Sam's GitHub account is linked to Sam's Plane account (chapters 8 and 27). If the GitHub login is not linked to anyone, git-sync keeps it as a link to the GitHub profile instead of guessing who it is. Linking your GitHub account is how you make sure GitHub mentions reach your Plane inbox.

7.4 Subscribe to work you need to follow

Assignment says who is responsible. **Subscription** says who wants updates. You can subscribe to a dependency, a review or a cross-team change you are not implementing.

Goal: follow an item without being assigned

Steps

1. Open the work item.
2. Select **Subscribe** near its header.
3. Check that the button now reads **Unsubscribe**.

Result: you receive notifications for the item's changes.

To stop, select **Unsubscribe**. That does not unassign you or remove your comments. When you follow several dependencies, a saved view (chapter 5) gives you one place to check them together.

7.5 Understand email notifications

Personal **Settings** → **Notifications** has email categories for property changes, state changes, work-item completion, comments and mentions.

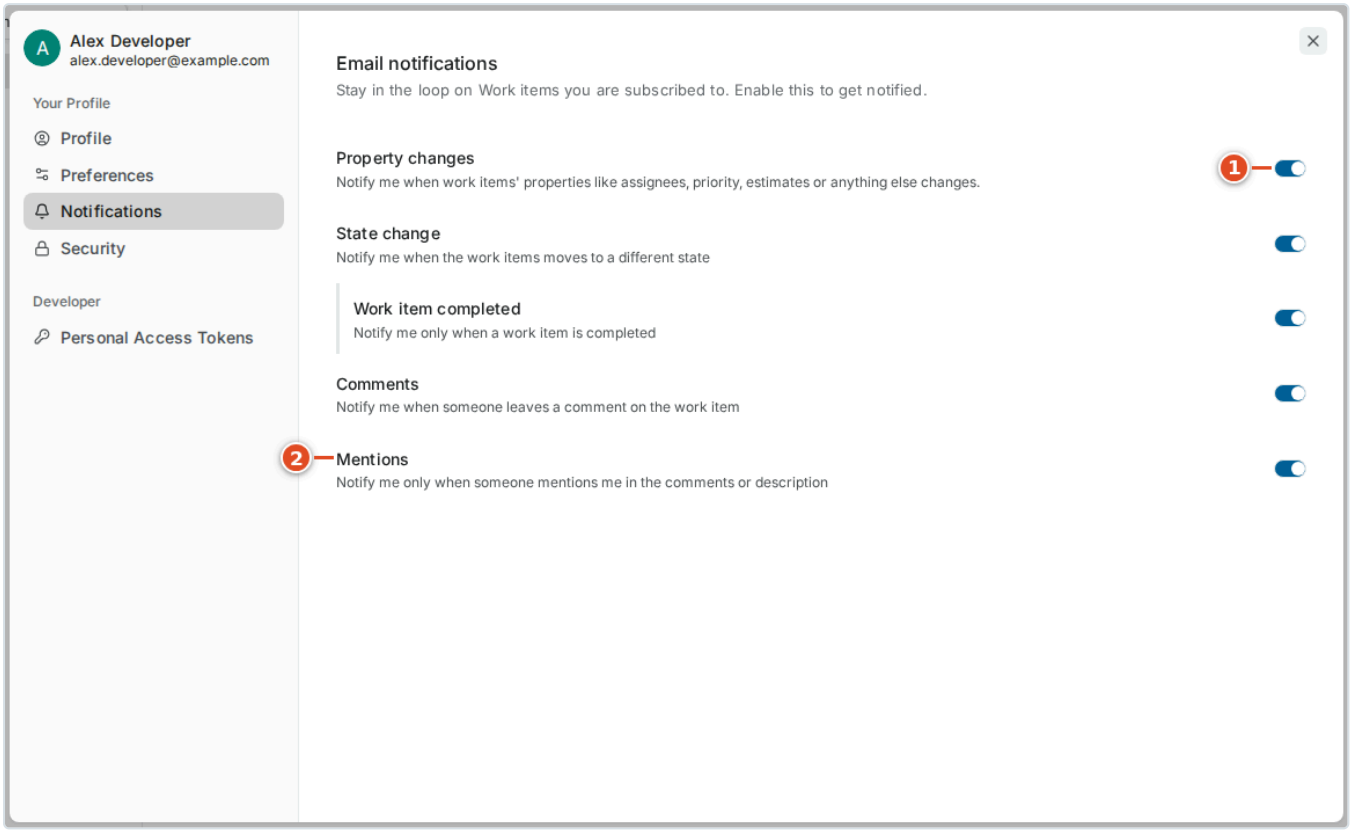


Figure 7.2 Screenshot (test Plane): Email notifications: (1) property-change switch; (2) Mentions category and its switch.

Goal: tune which notifications reach your mailbox

Steps

1. Open personal **Settings** and select **Notifications**.
2. Turn the property-change switch ① on or off.
3. Review **State change**, **Work item completed** and **Comments**.
4. Keep the **Mentions** switch ② on.

Result: email arrives only for the categories you kept.

Email also needs the server's mail configuration. The handbook's test instance had no mail server, so mailbox delivery was not tested there; the preference controls and in-app notifications were.

If it went wrong:

- *An expected email is missing* → first check that the event appears on the work item. Then check your subscription, your preferences and your mailbox filters. If the event is there but mail never arrives, send an administrator one concrete example: item identifier, approximate time and expected category. Never include a password or token.
- *An expected git-sync change never happened at all* → it is not a notification problem. See chapter 17.

7.6 Keep notification habits sustainable

A suggested habit: check direct questions before starting a long coding session, and again before you stop for the day. When you hand work over, leave a comment with the next action. A teammate in another timezone should be able to see what you checked, what is still uncertain and who needs to decide.

The inbox is a prompt to return to the shared record. Keep the decision on the work item, the code discussion on the PR, and a link between them; git-sync maintains that link for you when the PR mentions the key. Then the next developer can reconstruct what happened long after the notification has been read.

What you learned

- The inbox (**G then X**) collects notifications; reading one is not the same as handling the work.
- git-sync's state changes, comments and development links arrive as normal notifications from the service account.
- Mentions on mirrored GitHub issues reach you only if your GitHub account is linked to your Plane account.
- Subscription and assignment are different; email categories control only what reaches your mailbox.

Source files

- Plane CE v1.4.2 web application (`apps/web/`), observed in the running test instance
- `src/sync/text.ts` (mention and comment translation)
- `src/github/smart-commits.ts` (comment provenance line)
- `src/github/pr-automation.ts` (shipped state rules)
- `src/sync/dev-info.ts` (development links)

08 · The four connections and your GitHub account

Your Plane account tells the team who owns work. Your GitHub account tells the team who changes code. git-sync connects the two, but only after several separate connections are in place. Most confusion with the integration comes from mixing these connections up: “I connected GitHub” can mean four different things, managed by three different people.

This chapter first explains all four connections, so you know which ones are your job and which ones belong to an administrator. It then walks through the one connection every developer makes personally: linking **your** GitHub account on the **My GitHub account** page.

The four connections

Think of the integration as a chain. Each link must exist before the next one is useful.

#	Connection	Who sets it up	Where	What it gives
1	The **GitHub App** is registered and installed on a GitHub organization or personal account	A GitHub organization owner (install) and the Plane instance admin (registration in God mode)	GitHub, then **God mode → GitHub Sync**	git-sync may read and write the repositories the installation covers
2	That *installation is connected to one Plane workspace*	A workspace admin, sometimes with a link sent to an organization owner	**Workspace settings → GitHub Sync → Workspace GitHub connection**	The workspace may use that installation. One installation belongs to exactly one workspace.
3	A *repository is mapped to a Plane project*	A workspace admin	**Workspace settings → GitHub Sync → Map a repository**	Events from that repository reach that project, with its rules, policies and Smart Commit setting

#	Connection	Who sets it up	Where	What it gives
4	*Your Plane user is linked to your GitHub account*	You (or an admin on your behalf)	**My GitHub account**	git-sync recognizes you as the same person on both sides

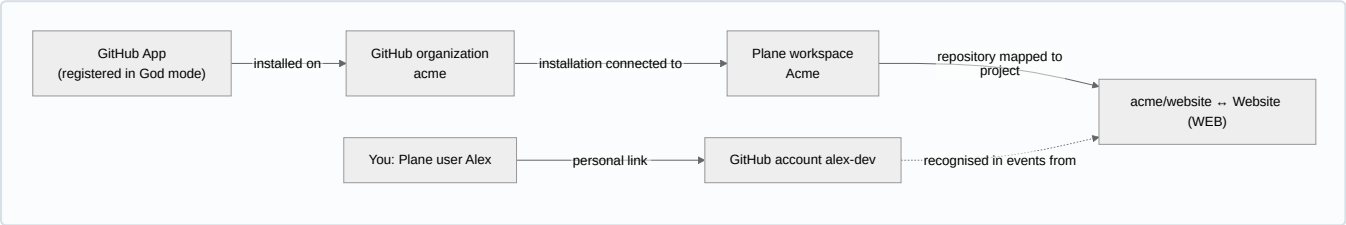


Figure 8.12 Diagram: the four connections. Connections 1–3 are shared team configuration; connection 4 is personal.

A few consequences are worth stating plainly:

- Connecting your personal account (4) **cannot** give git-sync access to a repository. Access comes only from the App installation (1). If your repository is missing, linking again will not help; ask an admin.
- One installation is connected to **one** workspace (2). If an organization's installation is already connected to another Plane workspace, the connection screen says **That installation is connected to another workspace.** An instance admin must move it deliberately.
- A repository that is not mapped (3) produces no links, no Smart Commits and no automation, even if you are linked and the App is installed. The Development panel says **No GitHub repository is mapped to this project.**
- Your link (4) is per workspace. If you work in two Plane workspaces, connect in each.

> **Admin only:** Connections 1–3 are described in Part B: Register the GitHub App, God mode settings and Connect GitHub & map repositories. As a developer you never need the App's private key, webhook secret or client secret. If someone asks you to paste one into a work item or chat, stop and ask your admin.

Find the My GitHub account page

The page is called **My GitHub account**. Its route is:

```
https://<your Plane address>/git-sync/ui/workspaces/<workspace slug>/account
```

For the demo workspace in this manual that is `https://plane.example.com/git-sync/ui/workspaces/acme/account`. The *slug* is the short workspace name in your Plane URLs (the part after the host, for example `acme` in `https://plane.example.com/acme/projects/`). The page uses your existing Plane session; it is not a separate sign-up site.

The navigation wrinkle: members do not see the GitHub Sync menu

The Plane fork adds **GitHub Sync** to **Workspace settings**, but that settings entry is restricted to workspace **admins**. Plane hides it from ordinary members, and the admin pages behind it refuse members. If a member opens the workspace's admin page directly, git-sync refuses:

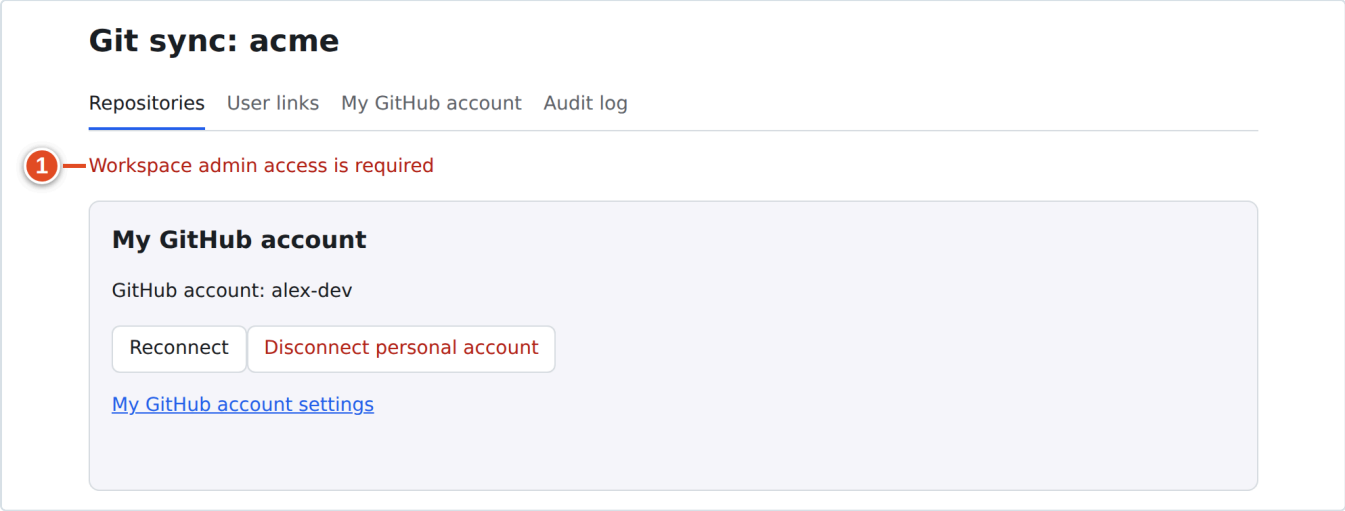


Figure 8.1 Screenshot (demo instance): A member opening the admin page: (1) the refusal. Members use the direct My GitHub account link instead.

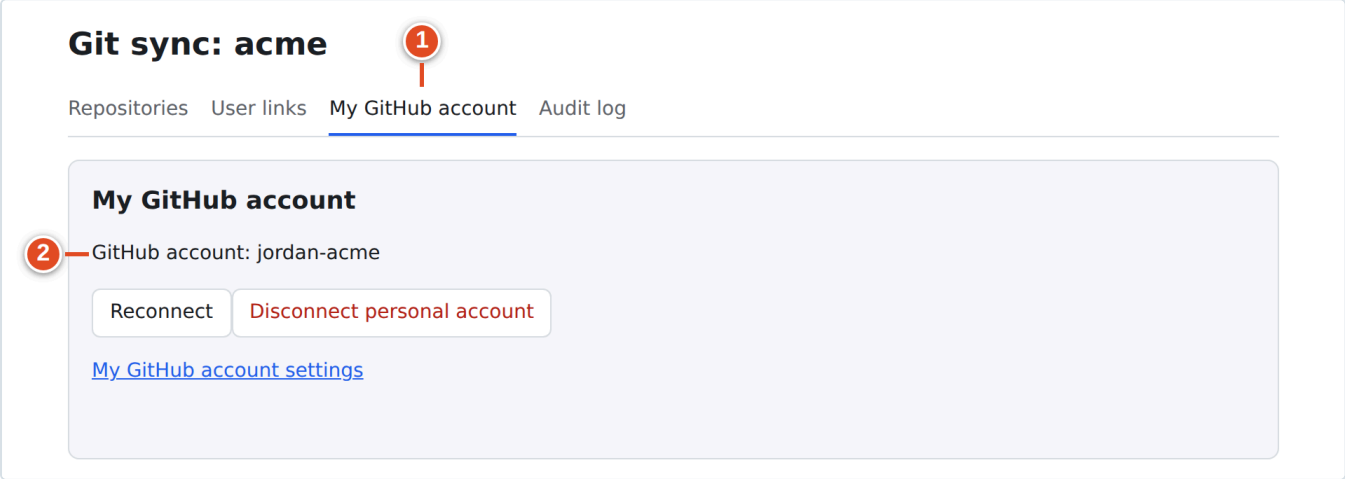


Figure 8.2 Screenshot (demo instance): Admins also reach the card from the workspace page: (1) the My GitHub account tab; (2) the linked login.

The personal account page itself is open to every workspace member. So:

1. Ask your admin for the direct **My GitHub account** link (or build it from the pattern above).
2. Bookmark it. You will need it again if your authorization expires or you change accounts.

Admins see the same card in two places: on the **My GitHub account** tab of **Git sync: <slug>** and at the top of the **Repositories** tab.

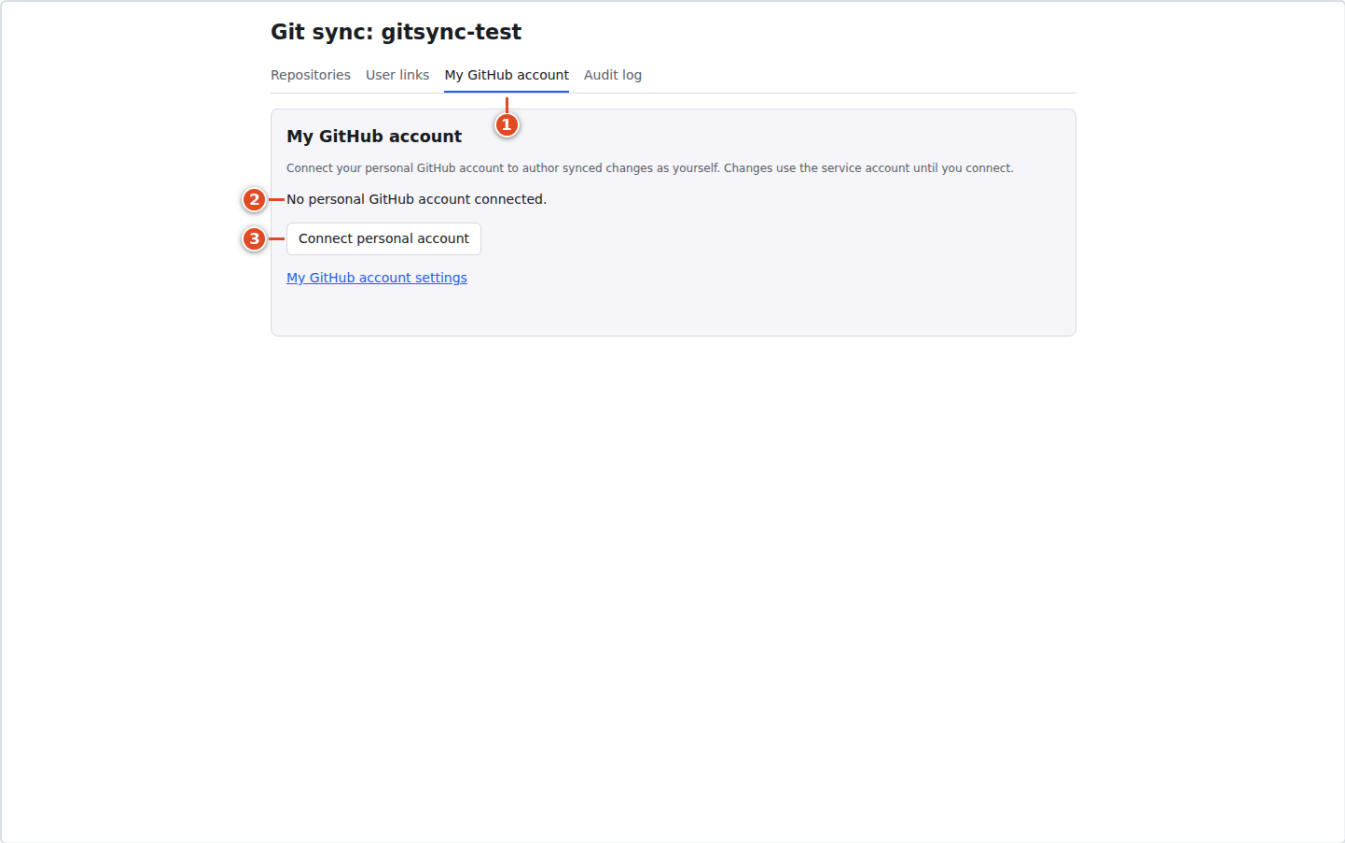


Figure 8.3 Screenshot (test Plane): Personal GitHub account: (1) member account tab; (2) current connection status; (3) Connect personal account.

The states of the My GitHub account card

The card always shows one of these states. The wording is exactly what you will see.

State	What the card says	What it means
Not connected	**No personal GitHub account connected.** with the hint “Connect your personal GitHub account to author synced changes as yourself. Changes use the service account until you connect.”	No link exists for you in this workspace
Connected	**GitHub account:** followed by your login	A link exists, keyed by your GitHub account’s stable numeric ID

State	What the card says	What it means
Needs re-authorization	**Needs re-authorization. Reconnect your GitHub account to author changes as yourself.**	The link exists, but GitHub refused or could not refresh the stored token
OAuth not configured	**The instance admin must configure the GitHub App OAuth credentials.** (the button is disabled)	The App's client ID and secret are not set in God mode
Not a member	**Personal account settings need workspace membership.**	You are signed in to Plane but not a member of this workspace

The buttons are **Connect personal account** (when not connected), **Reconnect** (when connected or needing re-authorization) and **Disconnect personal account**.

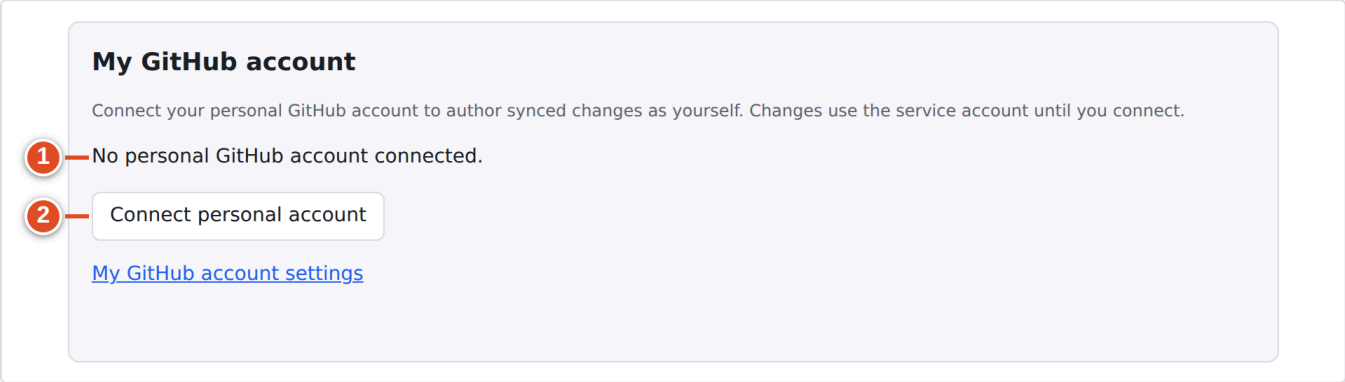


Figure 8.4 Screenshot (demo instance): My GitHub account before connecting: (1) status line; (2) Connect personal account.

Connect for the first time

Goal: link your Plane user to your GitHub account

Before you start:

- An admin has completed connections 1–3 (the App is installed, connected to the workspace, and at least one repository is mapped). You can still link before mapping, but nothing will happen yet.
- You are signed in to Plane in the intended workspace.
- In the same browser, you are signed in to GitHub with the account your team expects. Many people have a personal and an employer account; GitHub's authorization page shows which one is being used. Your local `git config user.email` does **not** choose the account.

Steps

1. Open your **My GitHub account** link. The card shows **No personal GitHub account connected.** ①
2. Select **Connect personal account**. ②
3. Your browser leaves Plane and opens GitHub's authorization page. This full-page navigation is expected (the page cannot authorize inside Plane's frame).
4. On GitHub, check the account name shown, then approve the authorization.
5. GitHub sends you back to the **My GitHub account** page.

> **Screenshot needed** — gh-03-oauth-consent — GitHub's authorize screen during Connect personal account (needs a signed-in GitHub browser session). Capture later with npm run manual:capture -- --only gh-03-oauth-consent

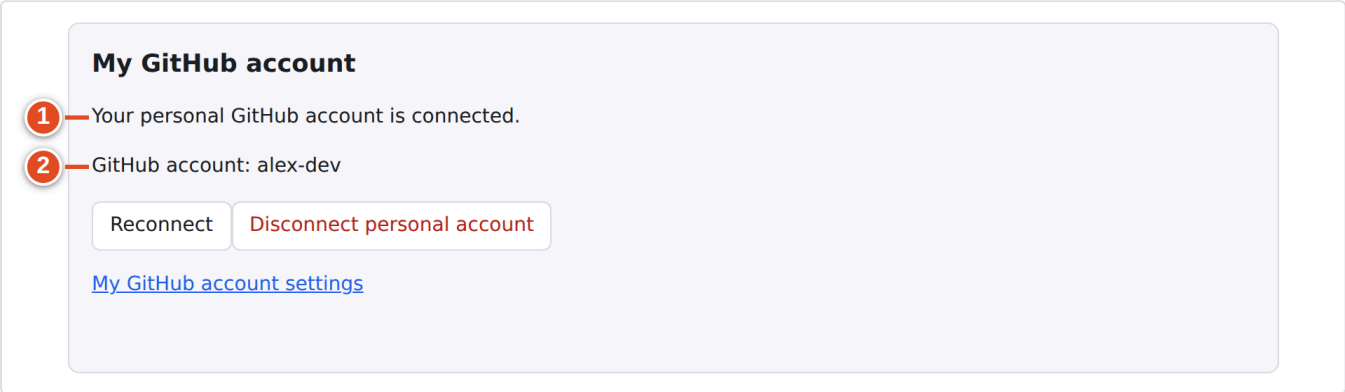


Figure 8.5 Screenshot (demo instance): Right after GitHub sends you back: (1) success message; (2) the linked GitHub login.

Later visits show the card without the notice:

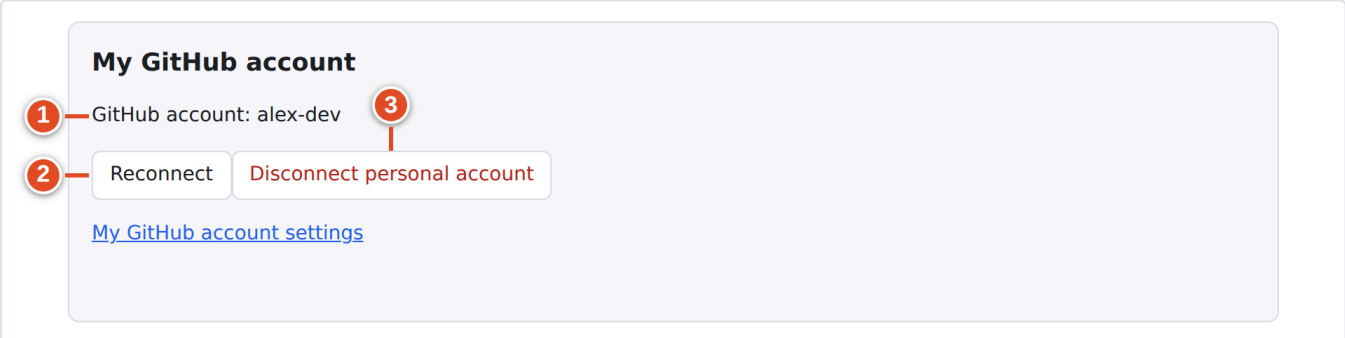


Figure 8.6 Screenshot (demo instance): Connected: (1) GitHub account login; (2) Reconnect; (3) Disconnect personal account.

If it went wrong: see Connection errors below.

What happens behind the button

- git-sync creates a single-use, expiring connection link bound to your Plane user (with PKCE, so the code GitHub returns is useless to anyone else).
- After you approve, GitHub returns a code; git-sync exchanges it for a user token and asks GitHub who you are.
- It stores your GitHub login *and its stable numeric account ID*. The ID is what counts: if you later rename your GitHub account, the link still points at the same account, and nobody who later takes your old username inherits your link.
- The GitHub token is stored encrypted. git-sync refreshes it automatically when GitHub issues expiring tokens.
- If an admin previously unlinked you, connecting again lifts that block.

Other ways a link can be created

- *An admin links you* on the **User links** page (Part B, User links). The admin types your GitHub login; git-sync checks it with GitHub and stores the numeric ID. No token is stored in that case, so the link identifies you but cannot act as you on GitHub.
- **Automatic email match.** When an event comes from a GitHub user who is not linked, and that user's *public* GitHub email exactly matches the email of exactly one workspace member, git-sync links them and records **Account matched by email** in the audit log. Disconnecting yourself (below) prevents this from re-linking you automatically.

What your link changes — and what it does not

Being precise here saves a lot of "why does it say the bot did this?" questions.

Your link lets git-sync recognise you. Events from GitHub carry an authenticated `sender` (the GitHub account that caused the event). With your link, git-sync can map that sender to your Plane user. This is used for:

- Smart Commit provenance: comments end with "via `<sha>` by `<your login>`".
- Translating GitHub `@mentions` and assignees on mirrored issues into Plane users (an unmapped login in a mention becomes a GitHub profile link; an unmapped assignee is simply dropped).
- Deciding whether problem reports may contain details (some reports are only shown in full to linked members).
- The repository picker an admin sees when mapping repositories (it lists repositories *their* account can access).
- The `{username}` placeholder in a branch-name template, if your team uses one.

Changes that go from Plane to GitHub can be authored as you. When you change a mirrored Plane work item, and your stored GitHub token works for that repository, git-sync writes the GitHub side with your token. If the token is missing, expired beyond refresh or refused, it falls back to the App's own identity (`<app-slug>[bot]`). Some steps, such as link-back comments, always come from the App.

Changes that go from GitHub to Plane appear as the git-sync service account in this release. The code can write to Plane as an individual user only when it holds that user's *Plane* personal access token (PAT). No screen in this release stores one, so every Plane comment, state change, link and mirrored item created by git-sync shows the service (bot) user as its author. The audit log records this as **Service account attribution** with the reason `no_stored_token`. This is expected behavior, not a failure. The provenance line in Smart Commit comments tells readers who really did it.

> **Note:** A Plane PAT and your GitHub link are two separate credentials. Linking GitHub does not create a Plane PAT, and the member account page has no field for one. Never paste a Plane PAT (or any token) into a commit, comment, chat or screenshot.

> **Example:** Alex pushes `WEB-42 #comment Checked at 320, 375 and 390 px widths`. In Plane the comment appears under the git-sync service user and ends with "via 3b2335d by alex-dev". Sam then edits the title of a mirrored Plane item; because Sam's GitHub token works, the GitHub issue's history shows `sam-reviews` made the edit.

Reconnect when the card says Needs re-authorization

GitHub can refuse a stored token: you revoked the authorization on GitHub, an organization policy changed, or the refresh token expired. git-sync then marks your link and the card shows:

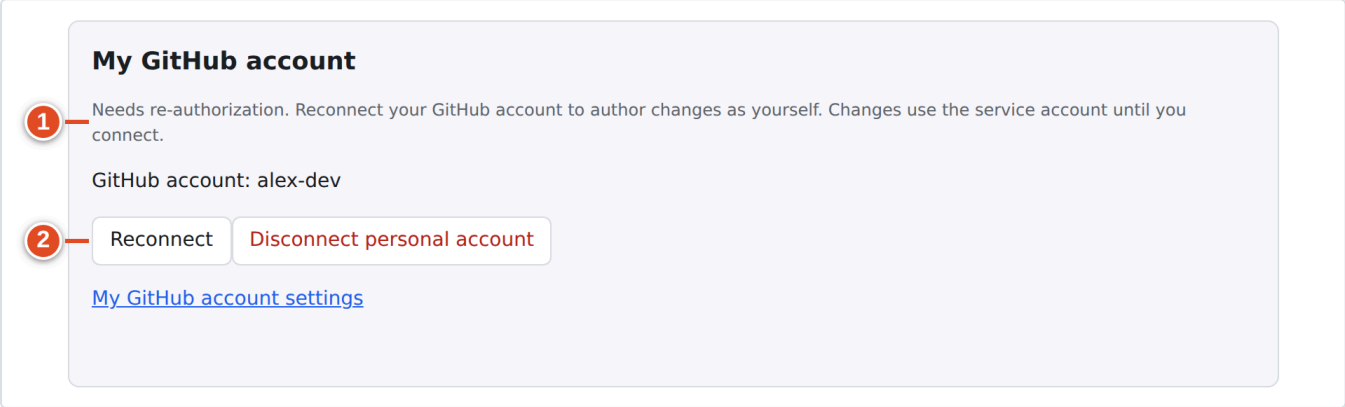


Figure 8.7 Screenshot (demo instance): GitHub refused the stored token: (1) the re-authorization notice; (2) Reconnect.

The three states side by side:

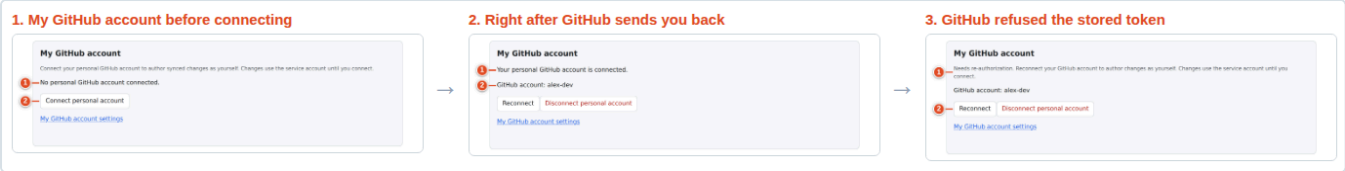


Figure 8.8 Strip (demo instance): My GitHub account in three states: not connected → connected → needs re-authorization.

Goal: restore a working authorization

Steps

1. Open **My GitHub account**. The card shows **Needs re-authorization. Reconnect your GitHub account to author changes as yourself.** ①
2. Select **Reconnect**. ②
3. On GitHub, check the account and approve.
4. Back on the card, confirm **GitHub account:** shows your login and the notice is gone.

Result: the same link is kept (same numeric ID) with a fresh token.

While the notice is shown, nothing breaks: git-sync keeps recognizing you as the event sender, and GitHub writes fall back to the App identity. The admin's **User links** page shows the same flag next to your name.

> **Tip:** Reconnecting with a *different* GitHub account replaces your link with that account, if it is not linked to someone else in the workspace.

Disconnect your personal account

Disconnecting is a personal action. It is different from the **Disconnect** button on an organization card in **Workspace GitHub connection**, which an admin uses to disconnect the whole installation and disables its mappings.

Goal: remove your personal link

Steps

1. Open **My GitHub account**.
2. Select **Disconnect personal account**.
3. Read the confirmation: "Disconnect your GitHub account? Synced writes will use the service account until you reconnect." ① Confirm. ②
4. The card returns to **No personal GitHub account connected.**

Your browser shows this confirmation (text recorded from the running UI):

> **Note:** “Disconnect your GitHub account? Synced writes will use the service account until you reconnect.” Choose **OK** to disconnect or **Cancel** to keep the link.

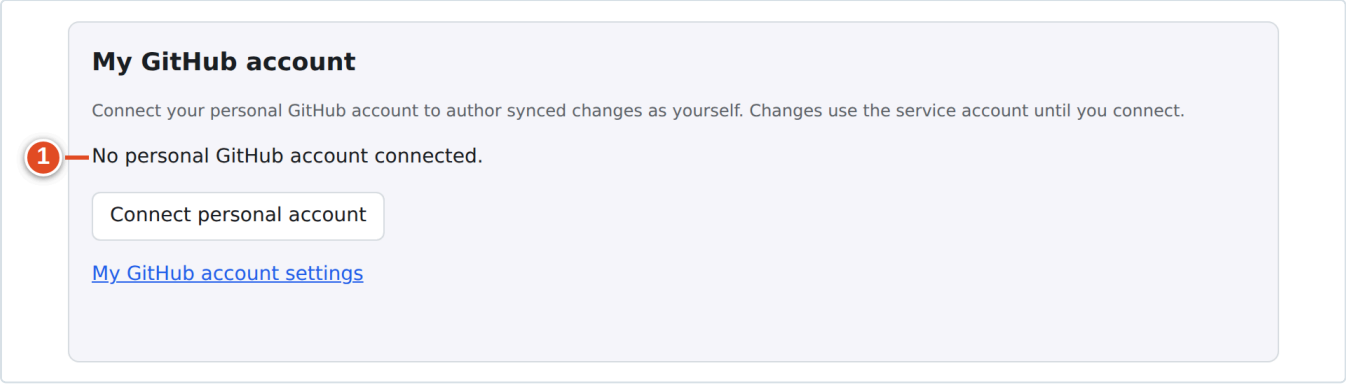


Figure 8.9 Screenshot (demo instance): After you disconnect: (1) the page reports no connected account again.

What disconnect does:

- Asks GitHub to revoke the authorization (best effort; if GitHub cannot be reached, the local token is deleted anyway).
- Deletes your stored GitHub token. Tokens are stored per Plane user, so this also ends personal GitHub authorship in your other workspaces until you connect again.
- Removes your link in this workspace.
- Records that you unlinked, so the automatic email match does not silently link you again.

What disconnect does not do: it does not delete branches, commits, PRs, comments, links on work items, or repository mappings. Your future events are simply treated as coming from an unmapped GitHub user. To stop the whole team’s integration, ask a workspace admin.

Connection errors

When the OAuth round trip fails, GitHub sends you back with an error, and the page shows one of these notices. The text is exact.

Notice on the page	Cause	What to do
This connection link has expired, was already used, or belongs to another Plane account. Start again.	The single-use link timed out, was reused (for example the back button), or you returned signed in as a different Plane user	Select **Connect personal account** again in the right Plane session
GitHub authorization was declined. You can connect again when ready.	You pressed **Cancel** on GitHub	Connect again when ready

Notice on the page	Cause	What to do
GitHub authorization could not be completed. Start again.	The code exchange with GitHub failed	Try once more; if it repeats, send the admin the time and this message
This GitHub account is already linked to another member in this workspace.	Another Plane user in this workspace is linked to that GitHub account	Check you used the right GitHub account; otherwise ask an admin to inspect **User links**
Ask the instance admin to configure the GitHub App client ID, client secret and App slug.	OAuth is not configured for the App	Ask the instance admin (God mode → GitHub App)
GitHub could not complete the connection. Check your account access and try again.	GitHub refused the user lookup after authorization	Retry; check your GitHub account is in good standing
The connection could not be completed. Try again; ask an instance admin to check the service if it keeps failing.	Unexpected server error	Retry once, then report
Personal account settings need workspace membership.	You are not a member of this workspace	Ask for membership or open the right workspace's link

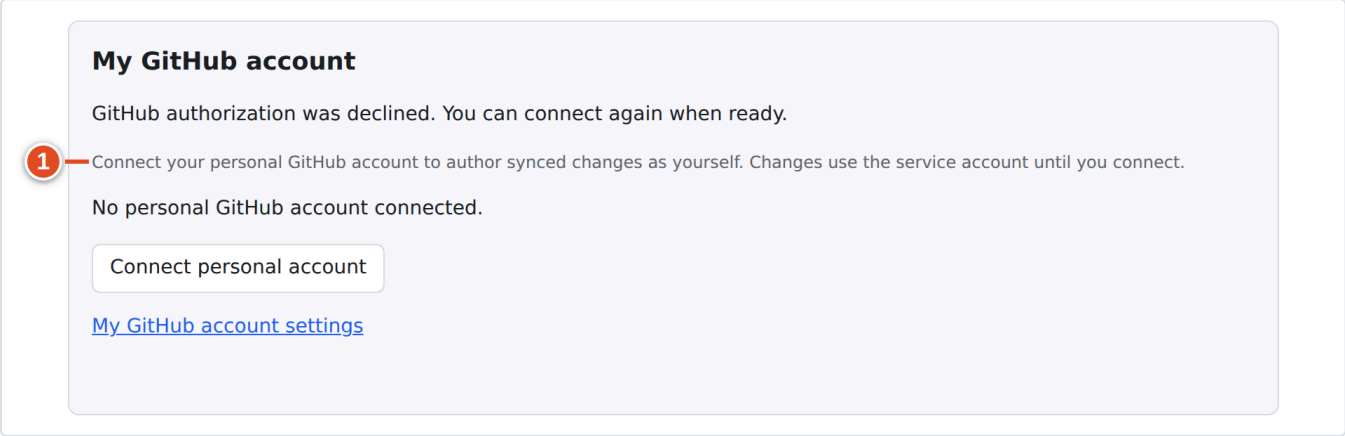


Figure 8.10 Screenshot (demo instance): You pressed Cancel on GitHub: (1) the error explains what happened.

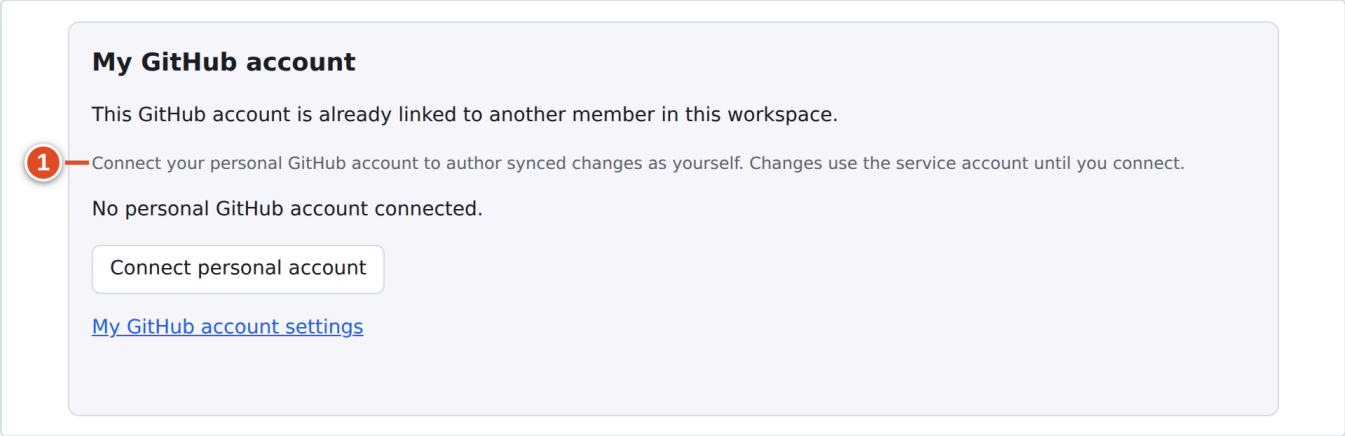


Figure 8.11 Screenshot (demo instance): That GitHub account is already linked to another member: (1) the error.

If linking succeeds but your repository still does not sync, the problem is in connections 1–3, not in your link. Give your admin the repository name, the work item key and what you expected to happen.

A first-day example

Riley joins the `acme` workspace. Riley's admin sends the link `https://plane.example.com/git-sync/ui/workspaces/acme/account`. The card says **No personal GitHub account connected.** Riley selects **Connect personal account**, checks that GitHub shows `riley-codes`, and approves. Back in Plane the notice **Your personal GitHub account is connected.** appears.

Later that week Riley's push contains `WEB-30 #comment Draft layout pushed`. The comment appears on WEB-30 under the git-sync service user, ending with "by riley-codes". Sam, the reviewer, has their own link; Riley's link does not map anyone else.

What you learned

- There are four connections: App installation, installation ↔ workspace, repository ↔ project, and your personal link. Only the last one is yours.
- Members reach **My GitHub account** through a direct link, because the **GitHub Sync** settings menu is admin-only.
- The link stores your GitHub account's stable numeric ID; **Reconnect** refreshes authorization and **Disconnect personal account** removes the link and the token.
- Plane-side changes made by git-sync show the service account as author in this release; GitHub-side mirror writes can be authored as you.
- Each OAuth error notice tells you exactly which step failed.

Source files

- `src/ui/client/Connect.tsx` — the My GitHub account card, notices and buttons
- `src/ui/oauth-flow.ts` — start, callback, disconnect, revoke
- `src/auth/github-user.ts` — OAuth exchange, token refresh, `needs_reauth`
- `src/auth/identity.ts`, `src/app/identity.ts` — sender → Plane user, email auto-match
- `src/ui/install-flow.ts` — installation ↔ workspace connection
- `src/app/context.ts`, `src/sync/mirror.ts`, `src/sync/github.ts` — attribution (service account, `asUser`)
- `packages/constants/src/settings/workspace.ts` — admin-only **GitHub Sync** settings entry

09 · Linking branches, commits and pull requests

A work item's key is its short address in code discussions. `WEB-42` means work item 42 in the project whose identifier is `WEB`. When you put that key in a branch name or a pull request, git-sync links the code to the task: the PR shows up in the work item's **Development** panel, a link-back comment appears on GitHub, and the team's automation rules can move the item through its states.

This chapter explains exactly where git-sync looks for keys, how strong each kind of reference is, and how to avoid accidental links. Chapter 10 covers the magic words and `WEB-NEW` in detail, and chapter 11 covers commands inside commit messages.

> **Note:** Everything here applies only to repositories an admin has mapped to a Plane project. In an unmapped repository nothing is scanned.

The key format

A key is a project prefix, a hyphen and a number:

- The **prefix** starts with a letter, followed by 1–11 letters or digits (so it is 2–12 characters long): `WEB`, `API2`, `MOBILEAPP`.
- The **number** is decimal digits. Leading zeros are dropped: `WEB-0042` is `WEB-42`.
- A key must not start right after a letter or digit, and must not be followed directly by a letter, digit or underscore.

Text	Result
`WEB-42`	Key `WEB-42`
`API2-103`	Key `API2-103`
`WEB-0042`	Key `WEB-42`
`feature/WEB-42-login`	Key `WEB-42` (slash and following hyphen are fine boundaries)
`X-42`	Ignored: the prefix is only one character
`WEB_42`	Ignored: underscore instead of hyphen
`WEB-42more`	Ignored: a letter follows the digits

Text	Result
`WEB-42_extra`	Ignored: an underscore follows the digits
`42-WEB`	Ignored: wrong order

Recognizing the shape is only the first step. git-sync then asks Plane whether the work item exists in the workspace. A key with the right shape that does not exist (for example `WEB-999`) is not linked; see Unknown keys below.

Prefixes that are never keys

Some technical abbreviations look like keys (`SHA-256`, `UTF-8`, `RFC-7231`). git-sync ignores these prefixes everywhere, even after a magic word:

`UTF`, `SHA`, `ISO`, `AES`, `RSA`, `CVE`, `CWE`, `RFC`, `TLS`, `SSL`, `MD`, `ES`, `PEP`, `USB`, `HTTP`, `IPV`, `SSH`, `GPT`, `RS`, `CRC`, `UTC`, `WIN`.

If your project's identifier is one of these, its keys will never link. Ask an admin to choose a different project identifier.

Where git-sync looks

Different places are scanned with different rules. Two properties matter: whether a **bare** key (one without a magic word in front) must be uppercase, and what **tier** a bare key gets.

Where	Bare key must be uppercase?	Tier of a bare key	Used for
PR **head branch** name (the branch with your changes)	No — `web-42-fix` works	**Closing**	PR links; branch pushes
PR **title**	Yes	**Closing**	PR links
PR **body** (description)	Yes	**Relation**	PR links
Commit message	Yes	**Relation**	Smart Commits and push rules only — never PR links
A key right after a **magic word** (anywhere above)	No — `Refs web-42` works	The magic word's tier	As for the place it appears

Some practical consequences:

- **The PR handler reads only the head branch, the title and the body.** It does not read the PR's commits. A key that appears only in a commit message never links the PR to the item. Put the key in the branch name or the PR itself.
- The **base** branch (where the PR merges into, such as `main`) is never scanned for keys. Admin rules can *inspect* the base branch (for example "merged into `staging`"), but that is a condition, not a reference.
- The scan is plain text. Markdown code blocks, quotes and links are still text: a pasted example containing `WEB-99` in a PR body creates a relation to WEB-99.

```
Branch:   feature/web-42-login        → WEB-42, closing
Title:    WEB-42 Improve login        → WEB-42, closing
Title:    web-42 Improve login        → nothing (bare lowercase in a title)
Body:     Related discussion WEB-42   → WEB-42, relation
Body:     Related discussion web-42   → nothing (bare lowercase in a body)
Body:     Refs web-42                 → WEB-42, non-closing (magic word)
Commit:   WEB-42 tidy CSS             → push rules only; no PR link
```

Text	Where	Result (real parser)	Why
`WEB-42-checkout-button`	branch	WEB-42 (closing)	bare key in a branch
`feature/web-42-login`	branch	WEB-42 (closing)	branch keys are case-insensitive
`WEB-42 Keep the checkout button visible`	title	WEB-42 (closing)	bare key in a PR title
`Closes WEB-42`	body	WEB-42 (closing)	closing magic word
`Refs WEB-41`	body	WEB-41 (non-closing)	non-closing magic word
`Relates to WEB-40`	body	WEB-40 (relation)	relation magic word
`See WEB-40 for context`	body	WEB-40 (relation)	bare uppercase key in a body
`Closes WEB-42, WEB-43 and API2-7`	body	WEB-42 (closing); WEB-43 (closing); API2-7 (closing)	several keys after one word
`Closes: WEB-42`	body	WEB-42 (closing)	optional colon, then a space

Text	Where	Result (real parser)	Why
`Closes:WEB-42`	body	WEB-42 (relation)	no space after the colon: only the bare key counts
`X-42 WEB_42 WEB-42more 42-WEB`	body	no reference	not keys
`UTF-8 SHA-256 CVE-2024`	body	no reference	deny-listed prefixes
`Closes WEB-42⏎skip WEB-42`	body	skipped: WEB-42	skip removes the key from this parse

The three tiers

Each reference has a **tier**, which says how much automation it allows. The tier is separate from the work item's state; it decides which rule actions may change that state.

Tier	Plain meaning	Typical wording	Can automation change the item's state?
Closing	This code finishes the item	`Closes WEB-42`, a key in the branch or title	Yes, including the "merged" step — but only when every linked PR is merged or closed (the merge guard, chapter 14)
Non-closing	This code is part of the work	`Refs WEB-42`, `Part of WEB-42`	Yes for every step **except** "merged"
Relation	Useful context only	`Relates to WEB-42`, a bare key in the body	Never

The gate applies only to *state changes*. If an admin writes a rule that also comments, adds a label or assigns someone, those actions run for every tier. All three tiers show the PR in the Development panel.

The demo run shows the difference. PR #118 says `Refs WEB-41`; opening it moved WEB-41 to **In Progress** (non-closing allows the "opened" step). PR #119 says `Relates to WEB-40`; WEB-40 stayed in **Todo**. Both items still list their PR:

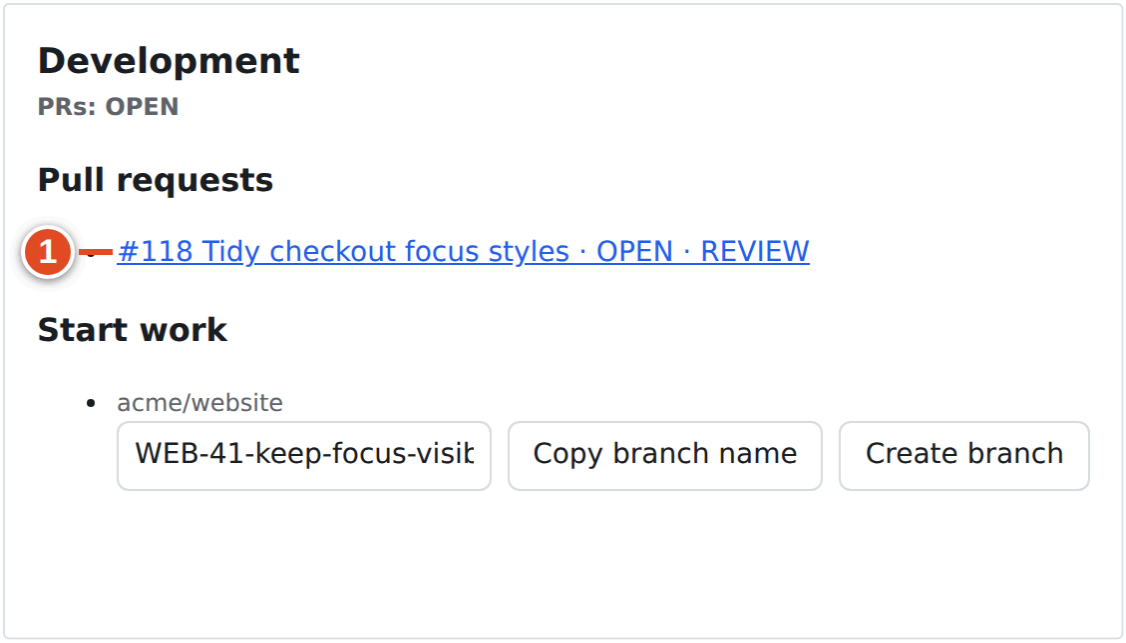


Figure 9.1 Screenshot (demo instance): A non-closing reference (Refs WEB-41): (1) the PR is linked and shown like any other.

When the same key appears several times, the strongest wins

git-sync collects every reference in the branch, title and body, then keeps **one** tier per work item: closing beats non-closing, which beats relation. If two references have the same tier, the record notes where it came from, preferring branch, then title, then body, then commit. That preference is bookkeeping only; it never makes a weaker tier win.

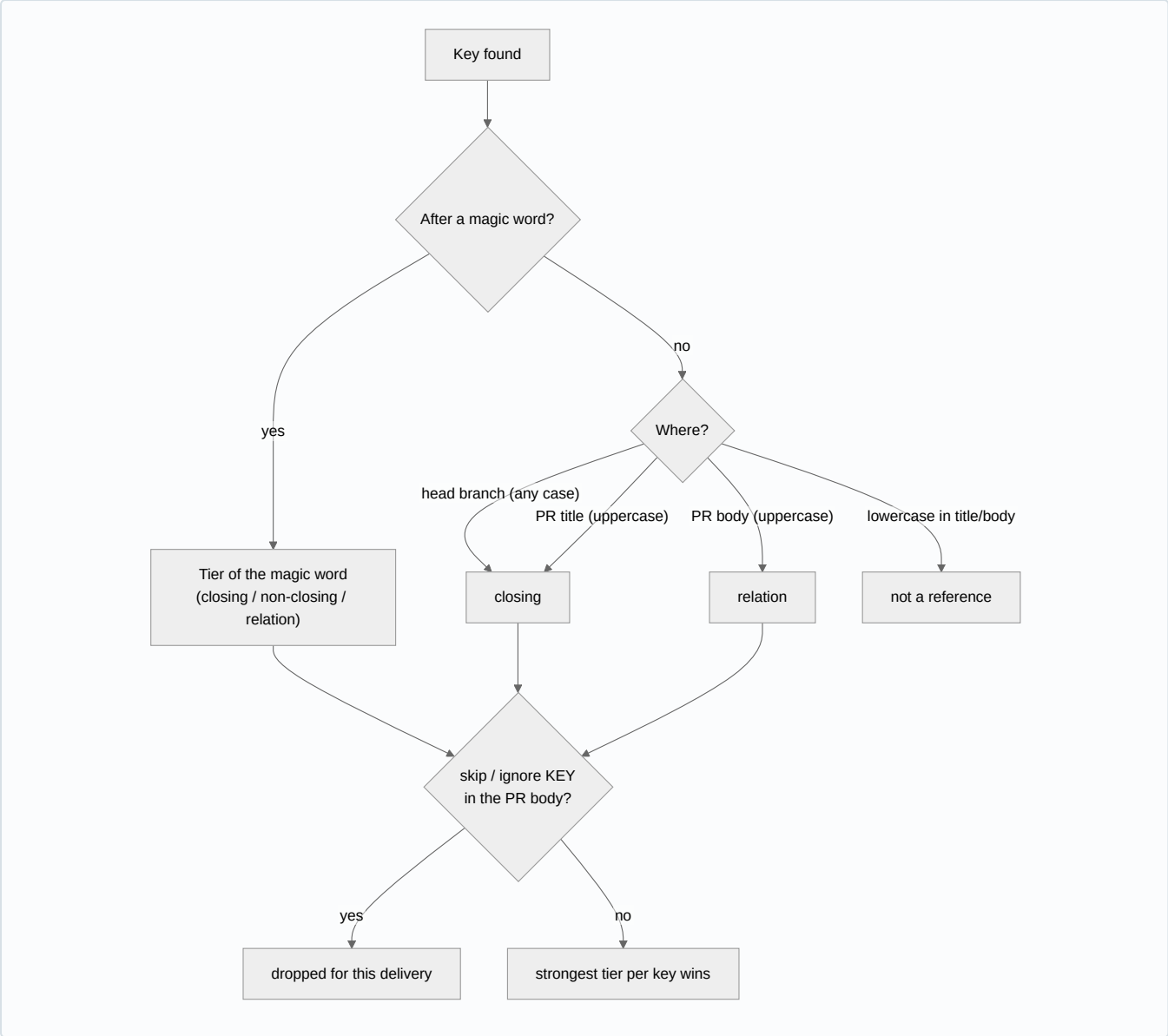


Figure 9.5 Diagram: how one key becomes a reference with a tier.

> **Warning:** A bare key in the branch name is already **closing**. If a PR should only contribute part of an item, writing `Refs WEB-42` in the body is not enough when the branch is `web-42-part-one`: the branch's closing reference wins. Use a branch without the key, such as `checkout-layout-part-one`, and `Refs WEB-42` in the title or body.

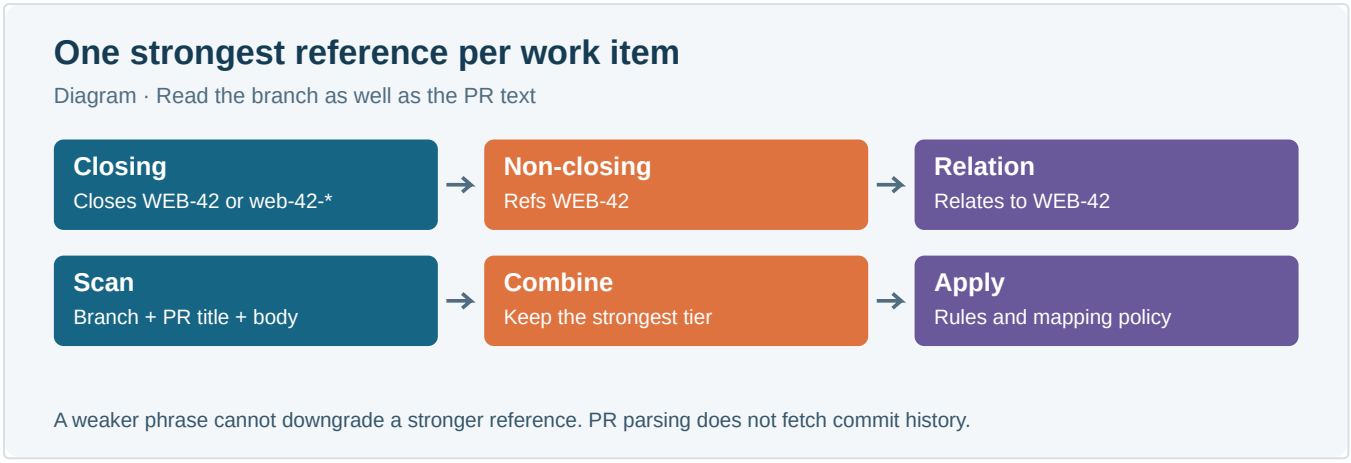


Figure 9.2 Diagram: branch, PR title and body contribute references; the strongest tier wins for each work item.

Link a pull request to a work item

Goal: a PR that shows up on WEB-42 and finishes it when merged

Before you start: the repository is mapped to the project (the Development panel shows a **Start work** section, not “No GitHub repository is mapped to this project.”).

Steps

1. Create your branch with the key in it. The easiest way is **Copy branch name** or **Create branch** in the work item’s **Development** panel (chapter 12), which gives `WEB-42-make-the-checkout-button-visible-on-small-screens`.
2. Push your commits.
3. Open the pull request on GitHub. Put the key in the title, for example `WEB-42 Keep the checkout button visible on small screens`. ①
4. In the body, state the intent on its own line: `Closes WEB-42`. ②
5. Check the branch, title and body for keys you did *not* mean to link.
6. Create the PR, then open WEB-42 in Plane and look at **Development**.

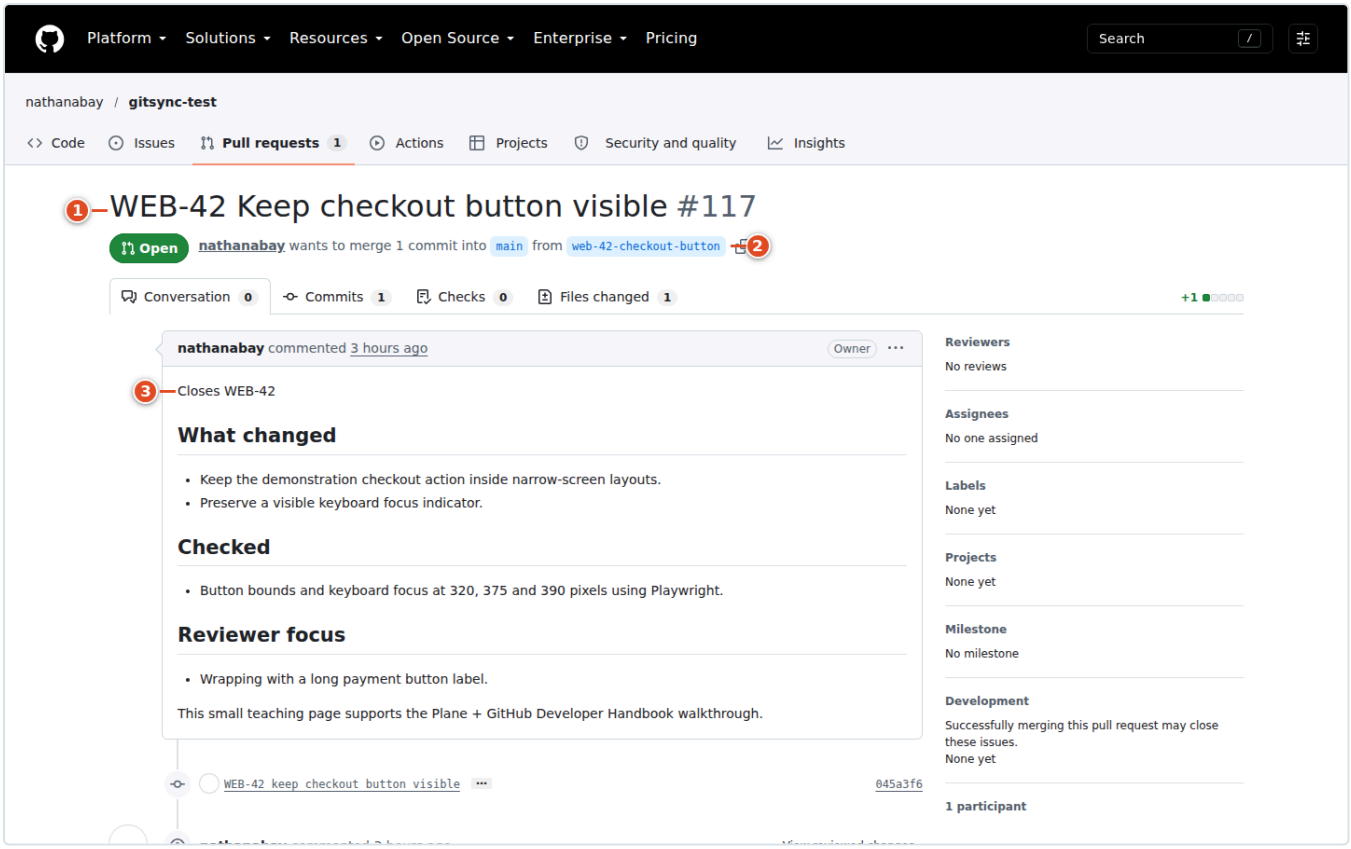


Figure 9.3 Screenshot (GitHub): Real PR #117: (1) title; (2) feature branch targeting main; (3) Closes WEB-42.

Result:

- The work item's **Development** panel lists the PR under **Pull requests** with its status (for example `OPEN · REVIEW`).
- A link titled `[PR] #121 WEB-42 Keep the checkout button visible on small screens · OPEN · REVIEW` is added to the item's links in Plane.
- If the mapping has **Post link-back comments on GitHub** turned on, git-sync comments on the PR once per newly linked item.
- The team's rules run. With the shipped defaults, opening a non-draft PR moves a closing or non-closing item to **In Progress**; a draft PR does the same.

The link-back comment text (verbatim from the demo run):

```
Linked Plane work item:
- WEB-42: http://plane.example.com/acme/projects/bb57e81d-c161-441f-b50e-907af28f08c2/issues/53486230-16eb-4c87-afc4-ffeb2339a1a9
```

Text git-sync posted to GitHub in the demo run.

When a PR links several items at once, the heading becomes “Linked Plane work items:” with one line per item. The comment is posted only for links that are **new** in that delivery, so editing the PR does not repeat it.

> **Screenshot needed** — gh-04-pr-linkback — A PR on GitHub with git-sync's 'Linked Plane work item' comment (needs a GitHub session and a live webhook). Capture later with npm run manual:capture -- --only gh-04-pr-linkback

- *Nothing in Development* → the key is lowercase in the title or body without a magic word, the key exists only in commits, or the repository is not mapped.
- *Linked, but the state did not change* → the reference is a relation, or your team’s rules have no step for this event. The audit log (ask an admin) shows **Automation rule matched** entries.
- *Linked to the wrong item* → see Removing a wrong link.

Branches

A branch name is the strongest and most convenient place for a key: it is closing, case-insensitive, and used both for PR links and for branch pushes.

When you *push a new branch* whose name contains a key, git-sync:

- adds a `[Branch] <branch name>` link to the work item (shown under **Branches** in the Development panel);
- runs the “branch pushed” rule, which by default moves the item to **In Progress**. A new branch never moves an item that is already completed or cancelled back to In Progress.

A good convention is key first, then a short description: `WEB-42-sticky-checkout-button`. The Development panel suggests the team’s configured name; see chapter 12.

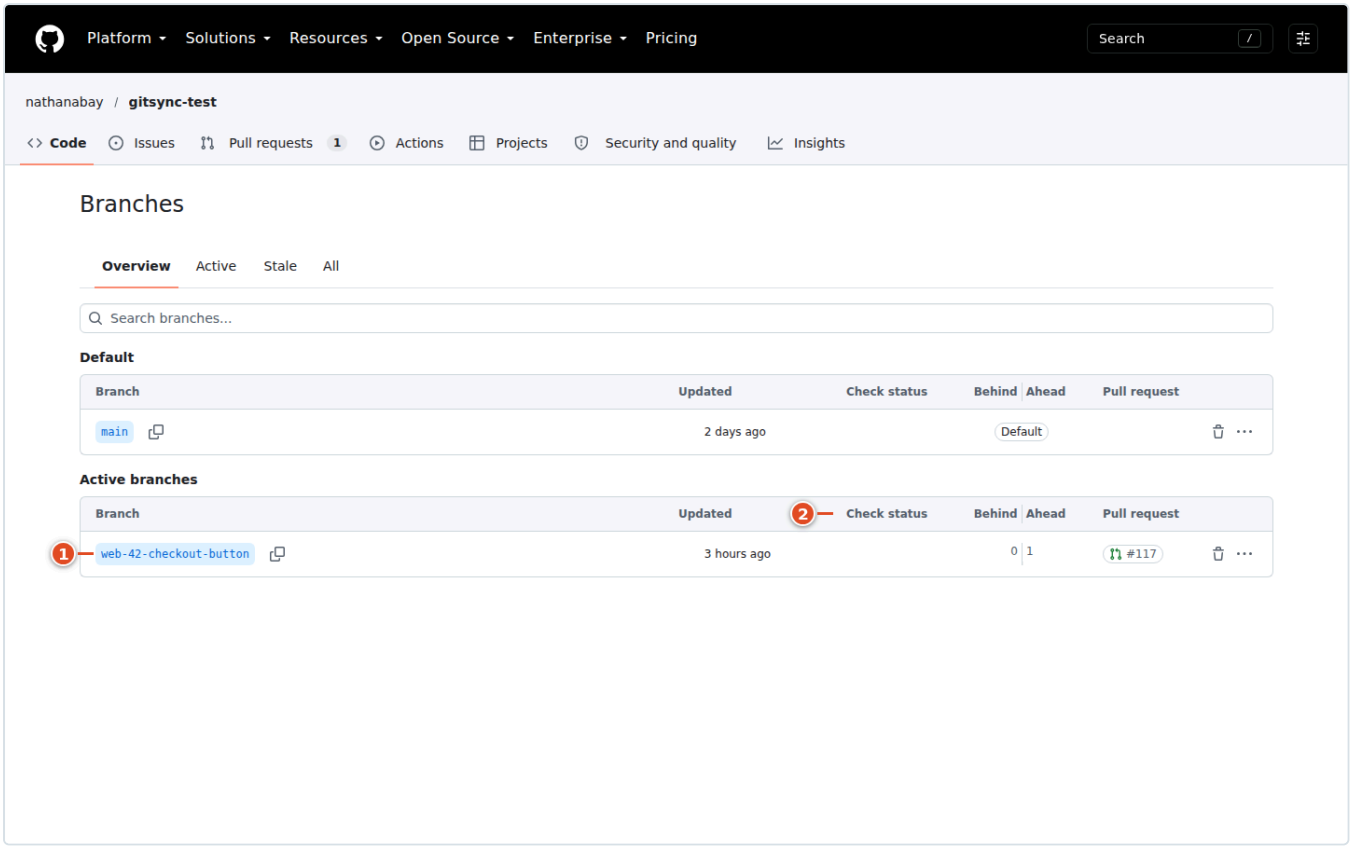


Figure 9.4 Screenshot (GitHub): Real GitHub branches: (1) web-42-checkout-button; (2) Check status column. The branch row also links PR #117.

Commits

A key in a commit message does two things, and only two:

1. **Smart Commits** — if the line has a `#command` after the key, git-sync runs it (chapter 11).
2. **Push rules** — an admin may write rules for `push` or `commit.pushed_to_default` events; bare uppercase keys in commit messages are their targets (as relations, so they cannot change state).

A commit message never links a PR, and there is no “Commits” list in the Development panel. To see a PR’s commits, open the PR on GitHub.

Opting out: skip and ignore

In a PR **body**, `skip WEB-42` or `ignore WEB-42` removes WEB-42 from the result of that scan, wherever else it appeared (branch, title or body). The words are case-insensitive. Each phrase covers exactly one key; `skip WEB-42 WEB-43` skips only WEB-42.

```
Closes WEB-42

This replaces the approach from WEB-17's spike.
skip WEB-17
```

Here WEB-17 is mentioned as context but not linked.

> **Warning:** `skip` only affects new scans. git-sync remembers links it has already made. Adding `skip WEB-42` (or deleting the key) after the PR was linked does **not** remove the stored link, and later events on the PR still reach WEB-42. Use `skip` in the PR body *before* you create the PR.

`skip` and `ignore` work only in the PR body. In a title or branch they are ordinary words.

Unknown keys

If a reference has the right shape but no such work item exists in the workspace, nothing is linked. git-sync records **Work item reference not found** in the audit log. When the mapping has **Report problems as comments on GitHub and Plane** turned on, and the PR's sender is a linked member of the workspace, git-sync also comments once on the PR (the comment is edited in place, not repeated):

```
WEB-99 does not match a work item in the linked Plane workspace, so this
pull request is not linked to it.
```

The report is limited to linked members on purpose: otherwise anyone who can open a PR could probe which keys exist in your Plane workspace.

Removing a wrong link

There is no button for developers to remove a link that git-sync has stored. Editing the PR text does not remove it either. If a PR is linked to the wrong item:

1. Tell your workspace admin the PR URL and the wrong key, before the PR is merged.
2. Meanwhile, avoid closing words for that key. A wrong *closing* link can move the wrong item to **Done** on merge.

One PR, several items; several PRs, one item

One PR, several items. State the intent per item on separate lines:

```
Closes WEB-42
Refs WEB-43
Relates to API-9
```

WEB-42 can complete on merge, WEB-43 moves along but does not complete, and API-9 only shows the PR. Check that the branch and title do not contain a closing key for WEB-43 or API-9.

Several PRs, one item. Each PR links on its own. The merge guard decides when the item may complete: at least one linked PR merged and none still open (chapter 14).

What you learned

- Keys look like `WEB-42`; a fixed deny-list (`SHA`, `UTF`, `RFC` …) is never treated as keys.
- PR links come from the head branch (any case, closing), the PR title (uppercase, closing) and the PR body (uppercase, relation). Commits never link PRs.
- Tiers gate state changes: closing can complete on merge, non-closing moves but never completes, relation never changes state. The strongest tier per key wins.
- `skip KEY` / `ignore KEY` in the PR body prevents a link, but cannot remove one that already exists.
- Link-back comments and unknown-key reports appear on GitHub only when the mapping enables them.

Source files

- `src/parse/references.ts` — key format, deny-list, scan rules, tiers, `skip` / `ignore`, `-NEW`
- `src/parse/magicWords.ts` — default magic words and tier order
- `src/app/pr.ts` — PR handler: sources scanned, stored links, link-back comment, unknown-key report
- `src/app/push.ts` — branch link on first push, Smart Commits, push rules
- `src/app/rules.ts`, `src/github/pr-automation.ts` — tier gating of `set_state`
- `tests/parse/references.test.ts`

10 · Magic words and WEB-NEW

A **magic word** is a short phrase in front of a key that says how the code relates to the task: “this finishes it”, “this is part of it”, or “this is related”. You already met the three tiers in chapter 9. Magic words are how you choose a tier deliberately, instead of relying on where a bare key happens to appear.

This chapter lists the shipped magic words, the exact grammar, how admins can change the lists, and the special `WEB-NEW` marker that creates a new work item from a pull request.

> **Note:** Magic words and Smart Commits are different tools. `Closes WEB-42` in a PR describes a relationship that rules act on later (for example on merge). `WEB-42 #done` in a commit message is a command that runs when the commit is pushed. See chapter 11.

The default magic words

These are the lists git-sync ships with (`src/parse/magicWords.ts`). Your instance admin can replace them; the God mode **Magic words** section shows the current lists.

Tier	Default phrases
Closing (20)	`close`, `closes`, `closed`, `closing`, `fix`, `fixes`, `fixed`, `fixing`, `resolve`, `resolves`, `resolved`, `resolving`, `complete`, `completes`, `completed`, `completing`, `implement`, `implements`, `implemented`, `implementing`
Non-closing (7)	`ref`, `refs`, `references`, `part of`, `contributes to`, `toward`, `towards`
Relation (2)	`relates to`, `related to`

Words that are *not* on the list, such as `see`, `addresses` or `for`, are just text. The key after them is then treated as a bare key, with the tier that its location gives it (closing in a branch or title, relation in a body).

Grammar

A magic word matches when all of these hold:

1. The phrase is a separate word: no letter, digit or underscore directly before or after it. `unfixes WEB-42` does not contain the magic word `fixes`.

2. The phrase is case-insensitive: `CLOSES`, `Closes` and `closes` are the same. Words in multi-word phrases may be separated by any whitespace (`part of`).
3. An optional colon may follow the phrase.
4. *At least one whitespace character* must come before the key. `Closes: WEB-42` works; `Closes:WEB-42` does not, because there is no space after the colon.
5. The key right after the phrase is case-insensitive: `Refs web-42` links WEB-42.

One phrase can cover several keys. Separate them with a comma, `and`, `, and`, `&` or plain whitespace:

```
Fixes WEB-42, WEB-43 and API-9
Refs WEB-42 WEB-43
Relates to WEB-42 & API-9
Closes: WEB-42
```

Anything else between the phrase and a key breaks the phrase. In `Refs (WEB-42)` the parenthesis stops the phrase; WEB-42 is then a bare key, and its tier depends on where it is.

Tier	Default words and phrases (from `src/parse/magicWords.ts`)
closing	`close`, `closes`, `closed`, `closing`, `fix`, `fixes`, `fixed`, `fixing`, `resolve`, `resolves`, `resolved`, `resolving`, `complete`, `completes`, `completed`, `completing`, `implement`, `implements`, `implemented`, `implementing`
non-closing	`ref`, `refs`, `references`, `part of`, `contributes to`, `toward`, `towards`
relation	`relates to`, `related to`

> **Example:** `Closes:WEB-42` in a PR **title** still links WEB-42 as *closing*, but only because a bare uppercase key in a title is closing anyway. The same text in a PR **body** is just a bare key: a *relation*. This is a common surprise. Always put a space after the colon.

Same key, several phrases

When a key is mentioned with different strengths, the strongest wins (closing > non-closing > relation). If the branch is `web-42-login` and the body says `Refs WEB-42`, the item is linked as **closing** because of the branch. A key that is already classified by a magic word is not counted a second time as a bare key: a title containing only `Refs WEB-42` stays non-closing.

Choosing the right word

You want to say	Write	Effect with the shipped rules
This PR finishes the task	`Closes WEB-42` (or the key in the branch/title)	Item moves to **In Progress** when the PR opens, **In Review** when review is requested, **Done** when merged into `main`
This PR is one part of the task	`Refs WEB-42` / `Part of WEB-42`	Item moves along with the PR, but merge never marks it **Done**
This PR is only related	`Relates to WEB-42`	Item shows the PR in Development; its state never changes
Do not link this key at all	`skip WEB-42` in the PR body	No link from this scan
There is no task yet	`WEB-NEW` in the PR title or body	git-sync creates the task (below)

> **Tip:** Split work across lines so every key's intent is visible to reviewers: `Closes WEB-42` / `Refs WEB-43` / `Relates to API-9`.

How admins change the lists

> **Admin only:** In **God mode → GitHub Sync → Magic words** there is one box per tier, one phrase per line, saved with **Save magic words**. The lists apply to the whole instance.

The validator refuses lists that would be ambiguous or unsafe:

- phrases must be lowercase letters and single spaces only, at most 40 characters;
- each tier must have between 1 and 100 phrases;
- a phrase may appear in only one tier. Otherwise saving fails with a message such as `"refs" is in both closing and non_closing`.

When your instance has custom words, the defaults in this chapter no longer apply. If you are unsure, ask the admin for the current lists, or check the **Magic words** screenshot in chapter 23.

Create a work item from a pull request: WEB-NEW

Sometimes the code comes first: a quick fix, a dependency bump, a refactor someone asked for in a hallway. If your team accepts PR-first work, put `<PROJECT>-NEW` in the PR title or body. git-sync creates a new work item in that project and links the PR to it.

Goal: create and link a new WEB item from a PR

Before you start: the repository is mapped to the project whose identifier you use (here `WEB`). Your team agrees PR-first items are acceptable.

Steps

1. Open a pull request whose **title or body** contains `WEB-NEW`. For example, title `WEB-NEW Add rate limiting to the sign-up form`. ①
2. Create the PR (or edit or reopen an existing one: the marker is checked on opened, edited and reopened events).
3. Open the Plane project. A new work item has appeared.

Result: in the demo run, PR #122 created WEB-46:

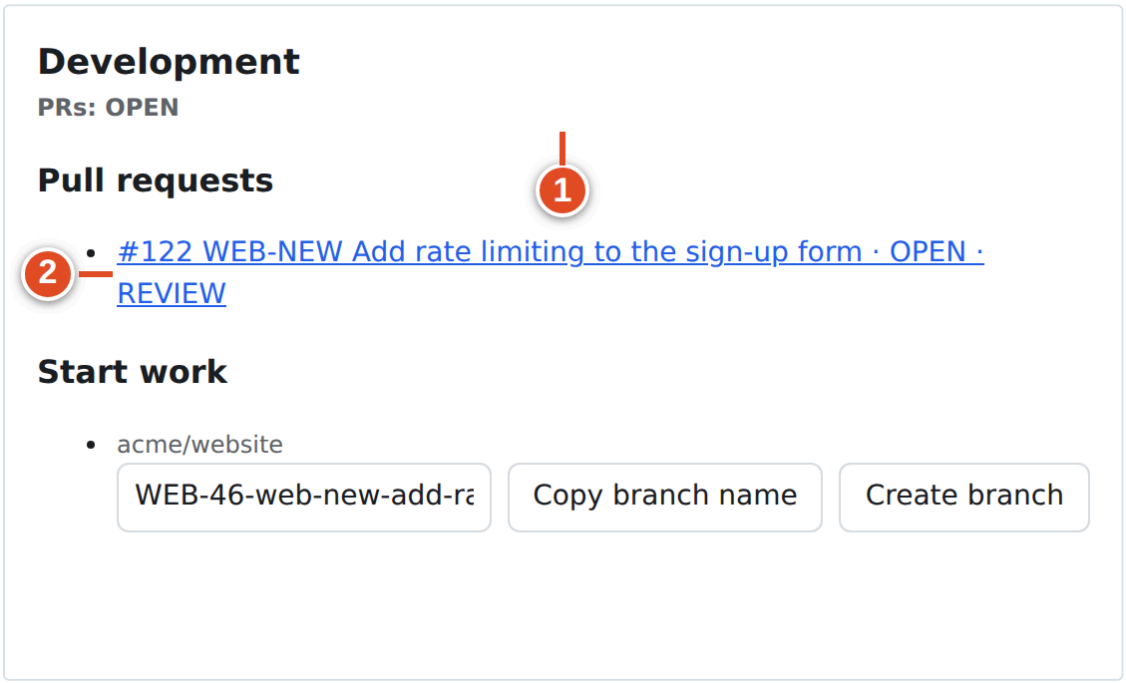


Figure 10.1 Screenshot (demo instance): A work item created from a WEB-NEW pull request: (1) title taken from the PR; (2) the PR link.

What git-sync does:

- The new item's *name is the PR title*, exactly as written (in the demo: "WEB-NEW Add rate limiting to the sign-up form"). Rename it in Plane afterwards if you like; the link stays.
- The item starts in a state from the project's **started** group (In Progress in the demo project).

- The PR is linked to the item with a **closing** link, so merging can complete it like any other closing PR.
- The PR body is not copied into the item's description.
- Repeated deliveries of the same PR do not create duplicates: one item per PR and project.

Rules for the marker:

- `NEW` must be uppercase. The project part may be any case: `web-NEW` works, `WEB-new` does not.
- Only the PR title and body are scanned; a branch called `WEB-NEW-…` or a commit message with `WEB-NEW` creates nothing.
- Prefixes on the deny-list (`SHA-NEW`, `RFC-NEW` …) are ignored.
- If no project has that identifier, nothing is created and the audit log records the failure ("no Plane project has the identifier …").

> **Warning:** Known defect in this release: for an item created by `WEB-NEW`, the link-back comment on GitHub names the project instead of the new key. The demo run posted:
>
> ```
> Linked Plane work item:
> - WEB: http://plane.example.com/acme/projects/bb57e81d-c161-441f-b50e-907af28f08c2/issues/50b5ef6b-6077-4578-a943-8bb8db2d6cc9
> ```
>
> The URL points to the right item (WEB-46). Only the label is wrong. This is listed in Appendix E.

> **Tip:** `WEB-NEW` names a project, not a number. If the task already exists, use its real key instead; otherwise you end up with a duplicate.

Worked examples

Each row shows what the shipped word lists make of a piece of text, and where the text appears. "Bare" means the key is read without a magic word.

Text	Where	Result
`Fixes WEB-42`	PR body	WEB-42, closing
`FIXES web-42`	PR body	WEB-42, closing (phrase and key are case-insensitive)
`Completing: WEB-42`	PR body	WEB-42, closing

Text	Where	Result
`Part of WEB-42`	PR title	WEB-42, non-closing (the phrase wins over the bare-title rule for this occurrence)
`Contributes to WEB-42 and API-9`	PR body	WEB-42 and API-9, both non-closing
`Related to WEB-42, WEB-43`	PR body	WEB-42 and WEB-43, both relation
`Closes:WEB-42`	PR body	WEB-42, relation (phrase does not match; bare uppercase key in a body)
`Closes:WEB-42`	PR title	WEB-42, closing (bare uppercase key in a title)
`Refs (WEB-42)`	PR body	WEB-42, relation (parenthesis breaks the phrase)
`unfixes WEB-42`	PR body	WEB-42, relation (`fixes` is not a separate word)
`See WEB-42`	PR body	WEB-42, relation (`see` is not a magic word)
`Addresses WEB-42`	PR title	WEB-42, closing (bare key in a title)
`refs web-42`	PR body	WEB-42, non-closing
`web-42`	PR body	nothing (bare lowercase)
`Fixes SHA-256`	anywhere	nothing (deny-listed prefix)
`Refs WEB-42` with branch `web-42-part-one`	PR	WEB-42, closing (the branch is stronger)

The generated tables in Appendix C run every such example through the real parser, so they always match the code.

> **Example:** A reviewer asks why WEB-43 moved to **Done** although the PR said `Refs WEB-43`. The title was `WEB-42 WEB-43 Checkout fixes`. Bare keys in a title are closing, and the strongest tier wins, so both items were closing. Writing the title without WEB-43 would have kept it non-closing.

A preview of the merge guard

Closing words have one more safety net. When a PR merges, a closing item moves to **Done** only if **at least one** linked PR has merged and **none** is still open. A PR that was closed without merging neither blocks nor counts. If another linked PR cannot be read from GitHub, git-sync treats it as open, to be safe.

The guard is evaluated only at the moment of a merge. If WEB-36 has two PRs, merging the first one does not complete it (the second is open), and later closing the second PR *without* merging does not re-run the check. Close obsolete PRs before the final merge, or move the item by hand. Chapter 14 shows this with screenshots.

What you learned

- There are 20 closing, 7 non-closing and 2 relation phrases by default; admins can replace them per instance.
- A phrase is case-insensitive, may have a colon, and needs whitespace before the key (`Closes: WEB-42`, not `Closes:WEB-42`).
- One phrase can cover several keys separated by commas, `and`, `&` or spaces.
- `WEB-NEW` in a PR title or body creates a work item named after the PR title, in a started state, linked as closing.
- Closing words complete an item only when the merge guard passes.

Source files

- `src/parse/magicWords.ts` — default lists and tier order
- `src/parse/references.ts` — magic-word regex, separators, colon rule, `-NEW` marker
- `src/ui/validate.ts` — magic-word validation (`parseMagicWords`)
- `src/app/pr.ts` — `createRequested` (WEB-NEW), merge guard input (`otherPrStates`)
- `src/github/pr-automation.ts` — merge guard on `set_state`
- `tests/parse/references.test.ts`, `tests/a8/grammar.test.ts`

11 · Smart Commits

A **Smart Commit** is a command inside a commit message. When you push the commit to a mapped repository, git-sync reads the message and acts on the Plane work item: it adds a comment, records time, or moves the item to another state. You never leave the terminal.

```
WEB-42 #in-review #time 45m #comment Ready for Sam to review
```

This one line moves WEB-42 to **In Review**, posts a “Time logged” note of 45 minutes, and posts the comment “Ready for Sam to review”.

> **Admin only:** Smart Commits are switched on per repository mapping with the **Smart Commits** checkbox (chapter 25). New mappings start with the instance default **Enable Smart Commits for newly mapped repositories** (on by default). If commands in your repository do nothing at all, check this setting first.

The grammar in one picture

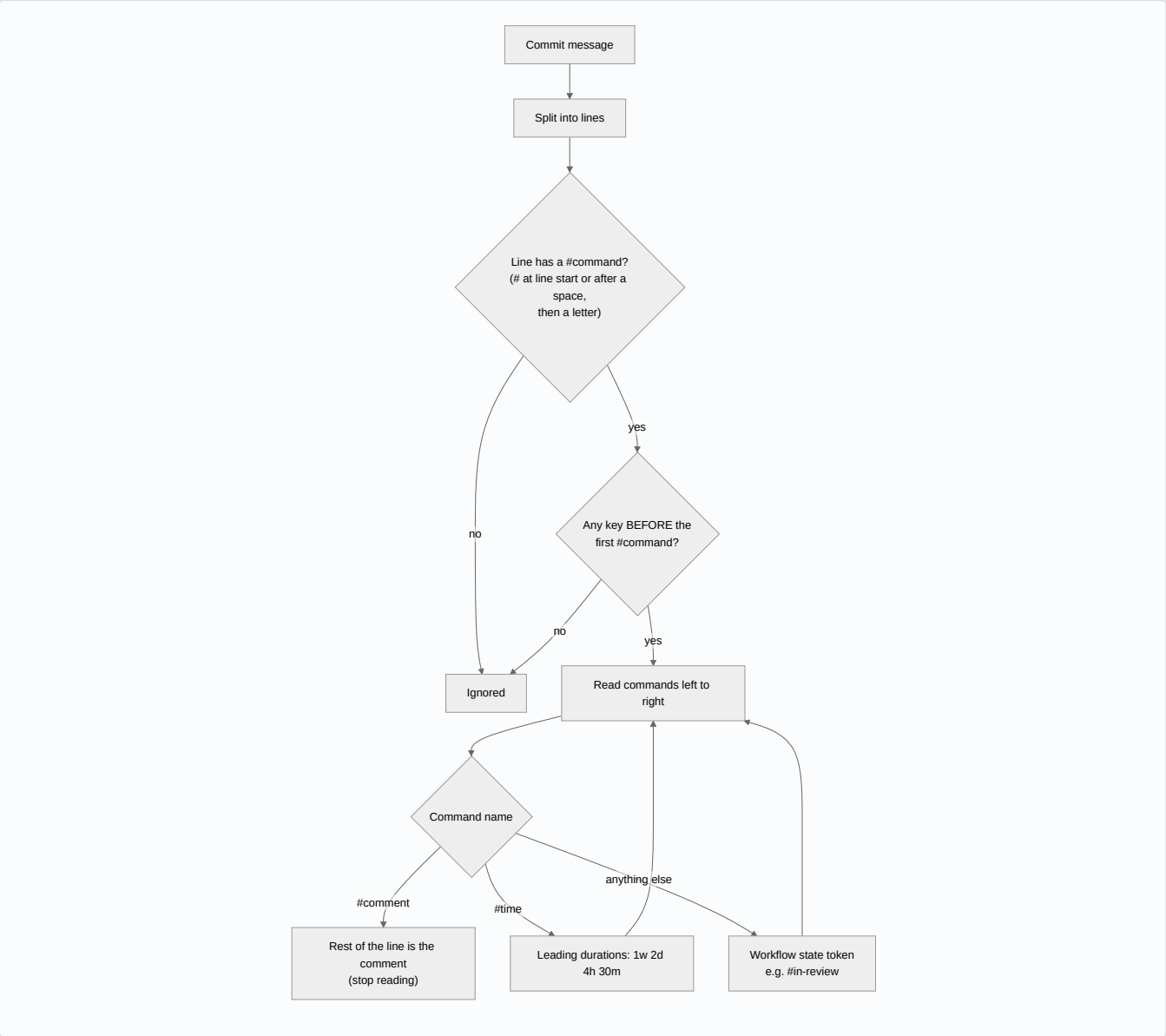


Figure 11.1 Diagram: how git-sync reads one commit message.

The rules, precisely:

1. **One line is one command context.** A command never continues onto the next line. Each line that should act needs its own key.
2. **Keys must come before the first command on that line.** Text before and between the keys can be ordinary prose: `Fix layout for WEB-42 #comment …` works.
3. **A command** starts with `#` at the beginning of the line or after whitespace, followed by a letter, then letters, digits, `_`, `-` or `'`. So `#123` and `abc#def` are not commands.
4. **Keys** are found like in a PR body: an uppercase bare key (`WEB-42`), or any-case key after a magic word (`refs web-42`). Several keys on one line all receive the same commands.

5. `#comment` and `#time` are reserved and case-insensitive. *Every other command name is a workflow-state token.* There is no `#assign`, `#label` or `#close` command; `#assign` would look for a state called “assign”.

Commit message	Result (real parser)
`WEB-42 #comment Checked at 320, 375 and 390 px`	keys WEB-42 · comment “Checked at 320, 375 and 390 px”
`Fix layout WEB-42 #time 1h 30m`	keys WEB-42 · time 1h 30m = 90 min
`WEB-42 #time 1w 2d 4h 30m`	keys WEB-42 · time 1w 2d 4h 30m = 3630 min
`WEB-42 #in-review #time 45m #comment Ready for Sam`	keys WEB-42 · state #in-review · time 45m = 45 min · comment “Ready for Sam”
`WEB-33 WEB-34 #in-progress #comment Shared validation helper`	keys WEB-33, WEB-34 · state #in-progress · comment “Shared validation helper”
`WEB-42 #comment see #12 and WEB-7 — this stays text`	keys WEB-42 · comment “see #12 and WEB-7 — this stays text”
`WEB-42 #time 3x`	keys WEB-42 · error: #time needs a duration such as 1w 2d 4h 30m, got “3x”
`WEB-42 #comment`	keys WEB-42 · error: #comment has no text
`#comment no key before the command`	nothing runs
`WEB-42 updated the docs`	nothing runs
`WEB-42 #done⏎WEB-43 #comment Second line, own key`	keys WEB-42 · state #done ‖ keys WEB-43 · comment “Second line, own key”

Lines that work and lines that do not

```
Improve the checkout flow

WEB-42 #in-progress #comment Started redirect handling     ← acts on WEB-42
API-9 #comment Client needs the new error shape            ← acts on API-9
```

```
WEB-42
#comment Started redirect handling     ← no key on this line: nothing
happens
```

```
#comment WEB-42 added tests            ← key is after the first command:
nothing happens
```

```
WEB-42 #comment We discussed #done but need another review
                                       ← one comment containing the text
"#done"; no state change
```

Because `#comment` takes the rest of the line, put state and time commands **before** it.

The three kinds of command

#comment — add a comment

`WEB-42 #comment Checked at 320, 375 and 390 px widths; the button stays visible.`

git-sync adds a Plane comment with your text and a provenance line naming the commit and the GitHub account that pushed it. Verbatim from the demo run (Plane comment HTML):

```
<p>Checked at 320, 375 and 390 px widths; the button stays visible.</p><p>
<small>via <a
href="https://github.com/acme/website/commit/3b2335d710bf1b0ba856a9ca334a2fd
2e8a27ac6">3b2335d</a> by alex-dev</small></p>
```

Text git-sync wrote in Plane in the demo run.

The text is plain text: Markdown and HTML are escaped, not rendered. An empty `#comment` is reported as a problem.

#time — record time spent

Plane Community Edition has no worklog feature that git-sync could write to. `#time` therefore posts a **comment** that starts with “Time logged:”, followed by what you typed and the total in minutes:

```
<p>Time logged: 1h 30m (90 minutes)</p><p><small>via <a
href="https://github.com/acme/website/commit/7b7ebfdd2c7657165f8c339cbace6ce
f32e140b6">7b7ebfd</a> by alex-dev</small></p>
```

Text git-sync wrote in Plane for `WEB-42 #time 1h 30m` in the demo run.

It does not change estimates, cycle capacity or any report. Durations:

Unit	Means	Example	Minutes
`m`	minute	`45m`	45
`h`	hour	`1.5h`	90
`d`	working day = 8 h	`1d`	480
`w`	working week = 5 d	`1w`	2,400
combined	added up	`1w 2d 4h 30m`	3,630

- Units are case-insensitive; decimals are allowed; the total is rounded to whole minutes.
- Only the **leading** durations count: in `#time 1h 30m investigating redirects`, the time is `1h 30m` and the words are dropped. Use `#comment` for the explanation.
- Several `#time` commands on one line are added into one note: `#time 15m #time 20m` posts 35 minutes.
- `1:30`, `two hours` or a missing duration are not durations. The problem is reported as "#time needs a duration such as 1h 30m."
- Each push of a *new* commit with `#time` posts another note; notes are never replaced.

#state — move the item

Any other command is a **state token**. git-sync looks it up among the states of the target item's own project, in three passes, stopping at the first pass that finds anything:

Pass	How the state name is compared	`#in-review` finds …
1. Exact	lowercase, spaces → hyphens	"In Review"
2. Ignoring hyphens and apostrophes	`#wontdo` = "Won't Do"	—
3. Leading whole words	the token equals the first words of the name	`#in` would match both "In Progress" and "In Review"

If a pass finds **exactly one** state, that state is used. If it finds **more than one**, the token is ambiguous and refused; git-sync never guesses. If no pass finds anything, the token is refused as unknown.

In the demo project (Backlog, Todo, In Progress, In Review, In QA, Done, Cancelled):

Token	Result
`#done`, `#DONE`	Done

Token	Result
`#in-review`	In Review
`#in-qa`	In QA
`#in`	refused: matches several states
`#prog`	refused: matches no state (prefixes must be whole words)
`#in review`	token `#in` (ambiguous); the word “review” is ignored — use hyphens
`#shipped`	refused: matches no state

Two more rules:

- **Never into triage.** Automation cannot move an item into a state of the triage group.
- **Each key uses its own project’s states.** `WEB-42 API-9 #in-review` can succeed for WEB-42 and fail for API-9 if API has no “In Review” state.

> **Warning:** A Smart Commit state change happens at **push** time, before review and merge. Do not use `#done` to mean “this commit is finished” if your team completes items on merge. Use a comment, or an intermediate state, and let the PR’s closing reference do the rest.

Write and push a Smart Commit

Goal: comment on WEB-42 and log time from the terminal

Before you start: the repository is mapped with **Smart Commits** on; you know the item’s key and, for state commands, the exact state names of its project.

Steps

1. Make and stage your change.
2. Commit with the command on its own line in the message body:

```
git commit -m "Move Place order button into a sticky footer" \
           -m "WEB-42 #comment Checked at 320, 375 and 390 px widths" \
           -m "WEB-42 #time 1h 30m"
```

3. Push to the mapped repository: `git push`.
4. Open WEB-42 in Plane and look at its activity.

Result: two new comments, both authored by the git-sync service user and ending in “via `<sha>` by `<your GitHub login>`”.

> **Screenshot needed** — plane-02-smart-comment — A Smart Commit #comment and Time logged comment in Plane's activity (needs a signed-in Plane session on an instance with a live GitHub webhook). Capture later with npm run manual:capture -- --only plane-02-smart-comment

If it went wrong: see When a command fails and the troubleshooting list at the end of this chapter.

Several keys, several commands, partial success

```
WEB-33 WEB-34 #in-progress #comment Shared validation helper for the
checkout form
```

In the demo run this line moved **both** WEB-33 and WEB-34 to In Progress and added the same comment to each. For each key the order is: state changes, then the time note, then the comment.

Commands are independent. If one fails, the others still run:

```
WEB-35 #time 3x #shipped
```

Here `#time 3x` is malformed and `#shipped` matches no state, so nothing is changed on WEB-35, and both problems are reported. A line like `WEB-42 #shipped #comment done` would still post the comment even though the state token failed.

Who did it: the actor

git-sync decides who performed a Smart Commit from the **authenticated webhook sender**: the GitHub account that pushed, as GitHub reports it. It never trusts the commit’s author or committer fields, which anyone can set with `git commit --author` or `git config user.email`.

- The sender is shown in the provenance line (“by alex-dev”).
- Bots, GitHub’s `ghost` account and senders who are not linked in the workspace are handled as the **service identity**; the audit log records **Service account attribution**.
- In this release every Plane change made by git-sync is written as the service (bot) user, even for linked members, because no per-user Plane token is stored (chapter 8). The provenance line is how readers see who did it.

Each commit runs once

git-sync remembers every commit SHA it has processed in the workspace.

- Pushing the same commit again (to another branch, a fork, or after a failed push) does **not** repeat its commands. If an earlier attempt was interrupted, the unfinished steps are resumed, not repeated.
- **Rewriting history creates new SHAs.** After `git commit --amend`, `rebase`, `cherry-pick` or a squash merge, the rewritten commits are new, and their commands **run again**. Remove or change old `#time` and `#comment` lines when you rewrite, or you will get duplicate notes.

Very large pushes: a GitHub push event lists at most 2,048 commits. For bigger pushes git-sync fetches the rest (up to 10,000) itself. If it still cannot see every commit, it reports “This push was incomplete; some Smart Commit commands may not have been processed.”

Signed commits: what is actually enforced

Some teams want Smart Commits only from commits GitHub shows as **Verified** (signed).

- Enforcement is controlled by the environment variable `REQUIRE_VERIFIED_COMMITS=true` in the service’s `git-sync.env` (then restart the service). With it, git-sync asks GitHub for each commit’s verification result and *ignores the commands* in unverified commits. The audit log records the rejection, and with **Report problems** on, the commit gets the notice “This commit is not signature-verified; Smart Commit commands were ignored.” If GitHub cannot be reached to check, the commit is retried later, never assumed verified.
- God mode also has a checkbox **Require signed commits for Smart Commits** under **Defaults**. *In this release the checkbox is saved but not used*: the push handler reads only the environment variable. Ticking the box changes nothing at runtime. This is listed in Appendix E.

Your local `git log` showing your name is not verification. Only GitHub’s “Verified” badge counts.

When a command fails

Failures are always recorded in the audit log. Developers see them only if the mapping has **Report problems as comments on GitHub and Plane** turned on (it is **off** by default). Then git-sync posts:

- **On GitHub:** one comment on the commit, listing every problem in that commit.
- **In Plane:** a notice on each affected work item that git-sync could identify, listing the problems that concern that item.

Verbatim from the demo run, for the commit `WEB-35 #time 3x #shipped` (GitHub commit comment):

```
Git sync could not complete this action:

- ` WEB-35: #shipped matches no Plane state. `
- ` WEB-35: #time needs a duration such as 1h 30m. `
```

Text git-sync posted to GitHub in the demo run. (A hidden marker line `<!-- git-sync-problem:… -->` follows, so the comment is updated in place instead of duplicated.)

The Plane notice on WEB-35 listed only the problem tied to that item's state:

```
<p>Git sync could not complete this action:</p><ul><li><code>WEB-35:
#shipped matches no Plane state.</code></li></ul>
```

Text git-sync wrote in Plane in the demo run.

And for a key that does not exist (`WEB-999 #comment typo in the key`), only GitHub can be told, because there is no item:

```
Git sync could not complete this action:

- ` Unknown work item WEB-999. `
```

Text git-sync posted to GitHub in the demo run.

The possible messages:

Message	Meaning
`KEY: #token matches no Plane state.`	No state of that project matches the token
`KEY: #token matches several Plane states.`	The token is ambiguous (for example `#in`)
`KEYS: #time needs a duration such as 1h 30m.`	`#time` without a valid leading duration
`KEYS: a Smart Commit command is malformed.`	For example `#comment` with no text
`Unknown work item KEY.`	The key does not exist (or git-sync cannot see it)
`This commit is not signature-verified; Smart Commit commands were ignored.`	`REQUIRE_VERIFIED_COMMITS` is on and the commit is unverified

Message	Meaning
`This push was incomplete; some Smart Commit commands may not have been processed.`	Very large push; some commits could not be listed
`Some Smart Commit commands could not be applied. Check the sync audit log with a linked workspace member.`	The pusher is not a linked member, so details about Plane items are withheld
`Could not apply the Smart Commit command for KEY. Check the sync audit log for details.`	Other failure (for example Plane refused the write)

The withheld-details message exists on purpose: without it, anyone who can push to the repository could learn which work items and states exist in your Plane workspace.

Screenshot needed — gh-05-problem-comment — A commit comment from git-sync reporting a Smart Commit problem (needs a GitHub session and a live webhook). Capture later with npm run manual:capture -- --only gh-05-problem-comment

More examples

Commit message line	Result when Smart Commits are on
`WEB-42 #comment Added a regression test`	One comment
`WEB-42 #time 20m #comment Investigated redirect loop`	Time note (20 minutes), then a comment
`WEB-42 #in-review #comment Ready for Sam`	Move to In Review, then a comment
`WEB-42 WEB-43 #time 30m`	A 30-minute note on **each** item (not split)
`WEB-42 #time 15m #time 20m`	One 35-minute note
`Fixes web-42 #comment done`	Works: lowercase key after a magic word
`web-42 #comment done`	Nothing: bare lowercase key
`WEB-42 #comment Looks good #time 1h`	One comment with the literal text “Looks good #time 1h”
`WEB-42 #time`	Reported: #time needs a duration
`WEB-42 #comment`	Reported: malformed command
`WEB-42 #assign alex-dev`	Tries a state called “assign”; reported as no matching state
`WEB-42 #in-review` written in a PR body	Nothing: Smart Commits are read only from pushed commits

Troubleshooting

Symptom	Likely cause	Fix
Nothing happens for any command	Smart Commits off for the mapping, repository not mapped, or `REQUIRE_VERIFIED_COMMITS` on and the commit unsigned	Ask an admin; check GitHub's Verified badge
Comment missing, others fine	Key after the first command, or on a different line	Put the key first on every line
State did not change	Token ambiguous/unknown, or the target state is in triage	Use the full hyphenated name (`#in-review`)
Duplicate time notes	History rewritten (new SHAs)	Remove old commands when amending or rebasing
Comment shows the bot as author	Expected in this release	Read the "by <login>" provenance

What you learned

- Write `KEY #command …` on one line; keys must come before the first command, and every line stands alone.
- `#comment` takes the rest of the line; `#time` posts a "Time logged" comment; any other token is a state, resolved exactly, then ignoring hyphens, then by leading whole words, and never guessed.
- Commands run once per commit SHA; amending or rebasing creates new SHAs that run again.
- Only `REQUIRE_VERIFIED_COMMITS` enforces signed commits; the God-mode checkbox is not applied in this release.
- Problems are reported on GitHub and in Plane only when **Report problems** is on; the audit log always has them.

Source files

- `src/github/smart-commits.ts` — parser (`parseSmartCommits`), durations, state resolution, execution, problem texts
- `src/app/push.ts` — push handler; reads `REQUIRE_VERIFIED_COMMITS` (line 55)
- `src/github/problems.ts` — the “Git sync could not complete this action:” reports
- `src/db/parity.ts` — stored (unused) **Require signed commits** setting
- `src/auth/identity.ts` — sender → actor
- `tests/github/smart-commits.test.ts`, `tests/a8/grammar.test.ts`

12 · The Development panel

Every work item in a project with a mapped repository has a **Development** section in its sidebar. It answers the questions a developer and a reviewer ask most often: *Is there a branch? Is there a PR, and what state is it in? Did the build pass? Where can I try it?* It also gives you the fastest way to start work: a ready-made branch name, or a button that creates the branch on GitHub for you.

Where to find it

Open any work item in Plane. The **Development** section is in the item's detail sidebar, below the standard properties. It is a small embedded page served by git-sync (an iframe), about 24 rem high, and scrolls on its own.

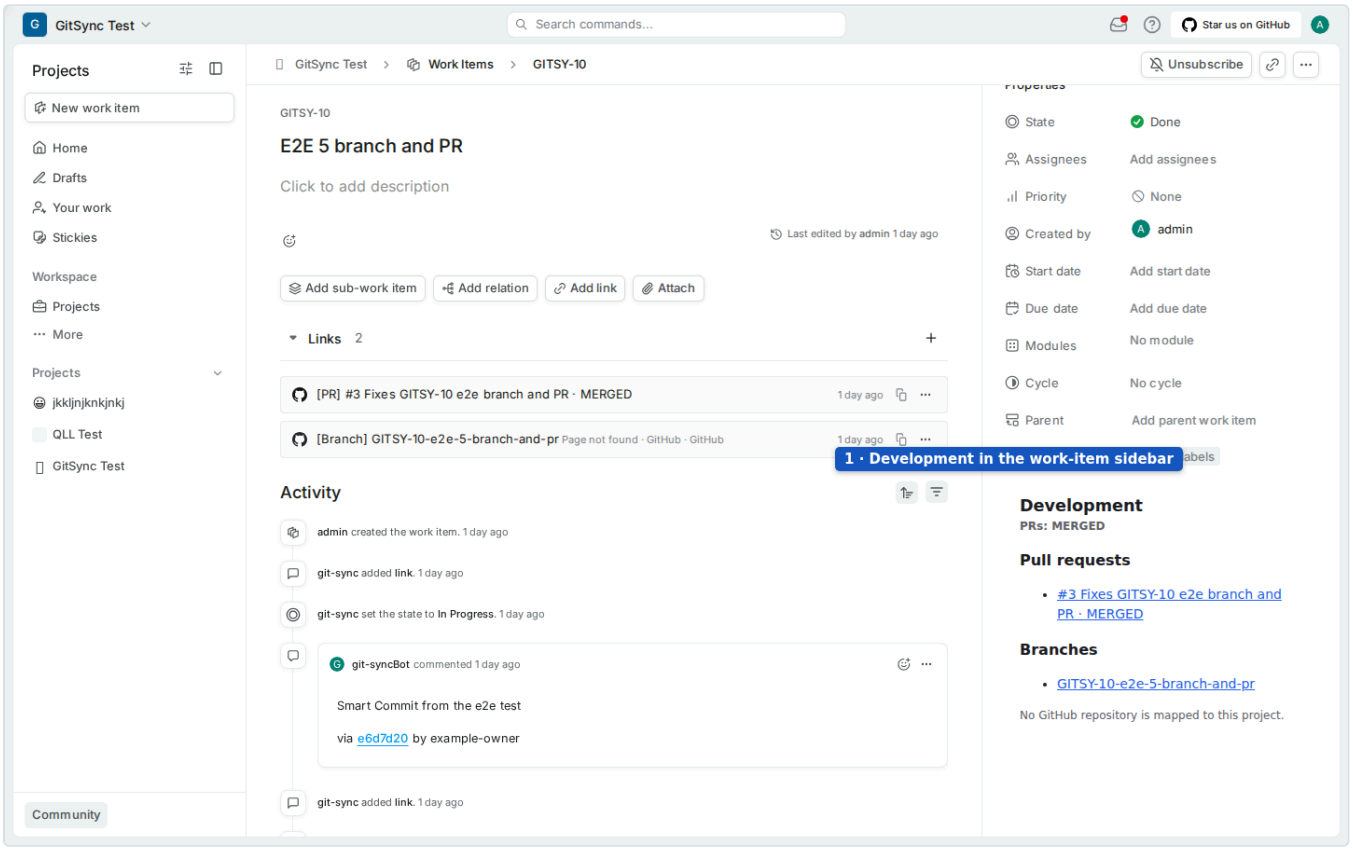


Figure 12.1 Screenshot (test Plane): The Development section embedded in a Plane work item's sidebar, with historical links on the item.

> **Screenshot needed** — plane-01-dev-sidebar — The Development section inside a real Plane work item sidebar with PR, build and preview (needs a signed-in Plane session on an instance with a live GitHub webhook). Capture later with npm run manual:capture -- --only plane-01-dev-sidebar

```
/git-sync/ui/workspaces/<slug>/projects/<projectId>/work-
items/<issueId>/development
```

The demo-instance screenshots in this chapter show that direct page. Its content is identical to the sidebar version.

The panel reads with **your** Plane session. If you cannot see the work item in Plane (for example a private project you are not a member of), the panel says **You cannot see this work item.**

What the panel shows

The panel heading is **Development**, followed by a roll-up of the item's pull requests: **PRs: OPEN**, **PRs: MERGED** or **PRs: DECLINED**. Below it come up to six sections, in this order. A section with nothing to show is hidden.

Section	Shows	Comes from
Pull requests	Each linked PR: `#number title · status · review`	PR events on GitHub
Builds	The latest CI result for the linked PR	Check suites, check runs and commit statuses
Deployments	Deployment environments and their status	GitHub deployment statuses
Previews	Preview sites from recognized hosting providers	Deployment URLs
Branches	Branches whose name contains the key	First push of a new branch
Start work	Suggested branch name, **Copy branch name**, **Create branch**	The project's repository mappings

Every entry is a link that opens GitHub (or the preview site) in a new tab.

When the project has no mapped repository at all, the panel shows only the sentence **No GitHub repository is mapped to this project.** Ask a workspace admin to map one (Chapter 24).

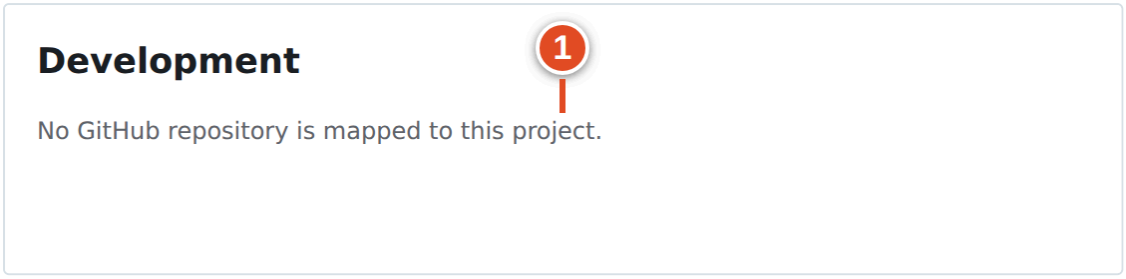


Figure 12.2 Screenshot (demo instance): A project with no mapped repository: (1) the Development panel says so.

Pull requests and their badges

Each PR line is built from the PR's title and two badges.

Badge	Meaning
`OPEN`	The PR is open (drafts too)
`MERGED`	The PR was merged
`DECLINED`	The PR was closed without merging
`REVIEW`	Open PR with no standing approval or change request (or reviews could not be read completely)
`APPROVAL`	Open PR with at least one standing approval and no standing change request
`REJECTED`	Open PR where at least one reviewer's latest decisive review requests changes

The review badge is shown only for open PRs. For each reviewer, only their latest *decisive* review counts (approve, request changes, or dismissal). A plain review comment does not cancel an earlier "request changes"; a dismissal does.

The heading's roll-up is `OPEN` if any linked PR is open, otherwise `MERGED` if any merged, otherwise `DECLINED`. It summarizes PRs; it is not the item's Plane state.

Builds

A build line reads `Build · pass`, `Build · fail` or `Build · pending` (from check suites and runs) or `Commit statuses · …` (from the older commit status API).

Result	When
`pass`	All runs concluded `success`, `neutral` or `skipped`

Result	When
`fail`	Any run concluded `failure`, `timed_out`, `cancelled`, `action_required`, `startup_failure` or `stale`
`pending`	Runs are still in progress, or there are none yet

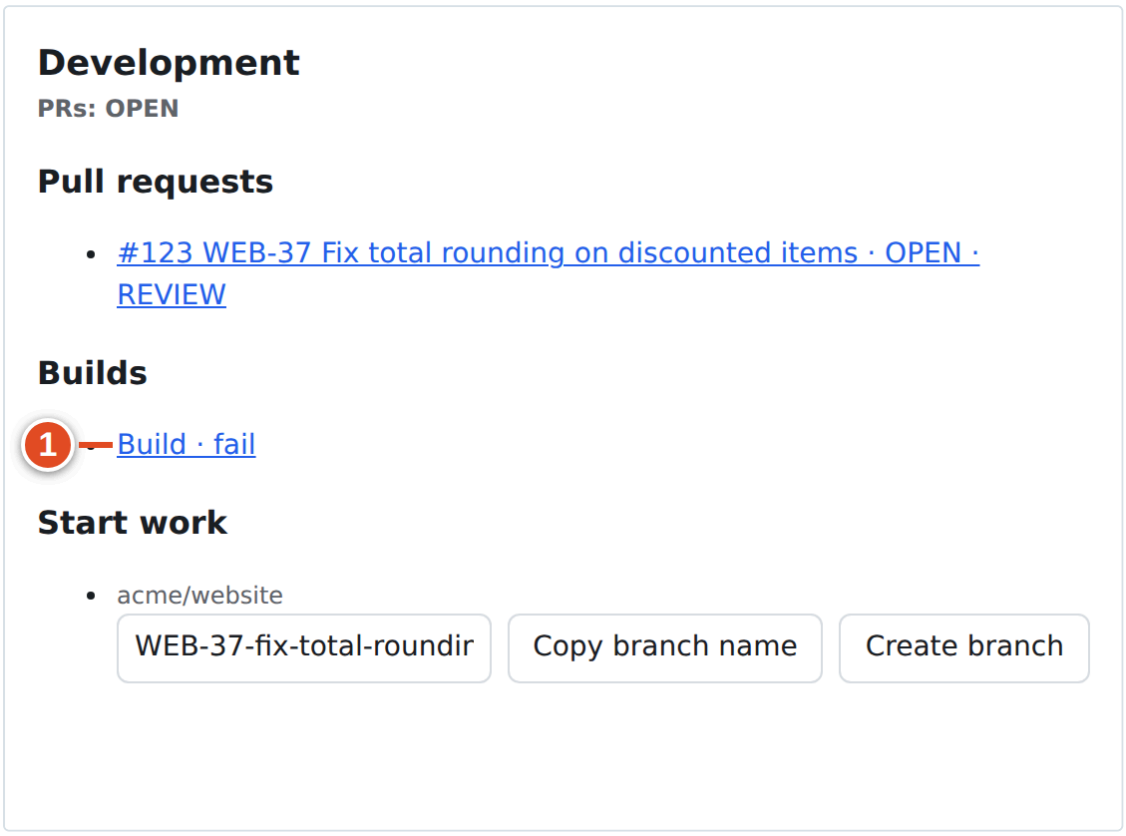


Figure 12.3 Screenshot (demo instance): A failing check: (1) Builds shows fail. Open it to see the CI log on GitHub.

The link opens the PR's **Checks** tab on GitHub, where the logs are.

Deployments and previews

When your CI or hosting provider creates GitHub **deployments** for the PR's commit, each environment appears under **Deployments** as `<environment> · <state>`, for example `Preview · success`.

A deployment whose URL belongs to a recognized preview host also appears under **Previews**, named after the provider:

Provider	URL host ends with
Vercel	`.vercel.app`
Netlify	`.netlify.app`

Provider	URL host ends with
Cloudflare Pages	`.pages.dev`
AWS Amplify	`.amplifyapp.com`

Verbatim links from the demo run, after Vercel deployed WEB-42's branch:

```
[Deploy] Preview · success -> https://website-git-web-42-acme.vercel.app
[Preview] Vercel -> https://website-git-web-42-acme.vercel.app
```

Links git-sync kept on the Plane work item in the demo run.

> **Note:** Previews are detected *only from GitHub deployment URLs* in this release. A link such as `[Website Preview](https://preview.example.com)` in the PR description is not picked up, even though older documentation said so: the detection code exists but is not called for PR bodies. A preview on another host (your own domain) appears under **Deployments** but not **Previews**.

git-sync does not deploy anything itself. If your pipeline does not create GitHub deployments, these two sections stay hidden.

Branches

When a **new** branch whose name contains the key is pushed, the branch appears under **Branches** (`[Branch] <name>`). Branches are recorded on their first push only; deleting the branch on GitHub does not remove the line.

There is no commits list

The panel does not list individual commits. Commit activity shows up in other ways: Smart Commit comments carry the commit hash (chapter 11), and the PR on GitHub has the full commit list. A key that appears only in commit messages does not add anything to this panel.

Start work: Copy branch name and Create branch

The **Start work** section has one row per repository mapped to the project. Each row shows the repository, a suggested branch name, and buttons.

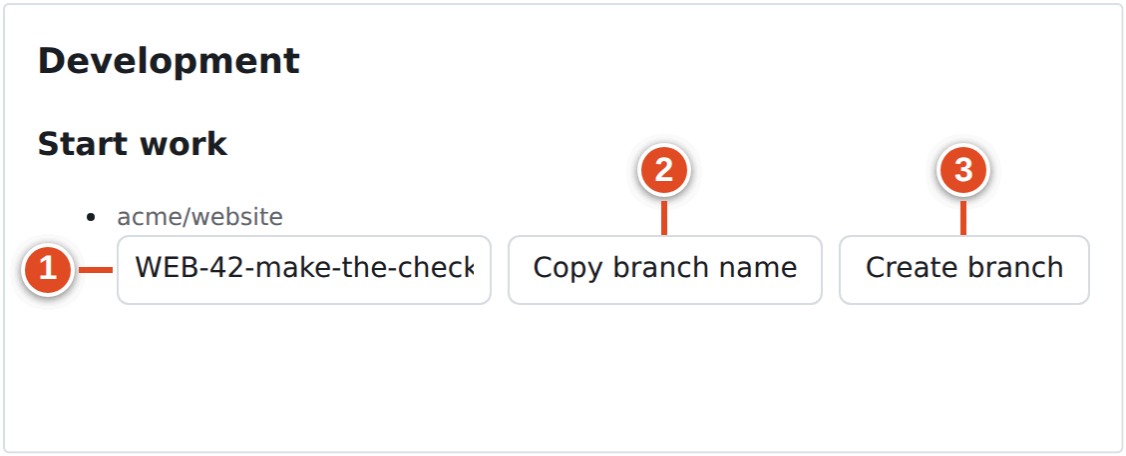


Figure 12.4 Screenshot (demo instance): WEB-42 before any code: (1) suggested branch name; (2) Copy branch name; (3) Create branch.

The suggested branch name

The name comes from the mapping's branch **template**. The default template is:

```
{key}-{title-slug}
```

Placeholder	Becomes
`{key}`	The work item key, e.g. `WEB-42`
`{title-slug}`	The title in lowercase, words joined by hyphens, cut at a word boundary to at most 50 characters
`{username}`	Your linked GitHub login (dropped together with its separator if you are not linked)

For WEB-42 "Make the checkout button visible on small screens" this gives `WEB-42-make-the-checkout-button-visible-on-small-screens`. Because the key is in the branch name, the branch is a **closing** reference (chapter 9). The whole name is limited to 200 characters.

> **Note:** There is no screen or API to change the template in this release. It is stored per mapping and can only be changed directly in the service database, so most teams use the default. If the stored template does not give a valid Git branch name, that repository's row is not shown.

Goal: start work on WEB-42 with a correctly named branch

Before you start: you are a member of the project (Create branch needs project role **Member** or higher; guests and viewers see only **Copy branch name**). The App installation has **Contents: write** permission (an admin setting).

Steps (Create branch — remote)

1. Open WEB-42 and find **Development** → **Start work**.
2. Check the repository name above the row, if the project has more than one.
3. Select **Create branch**. ③ The button shows **Creating…** while it works.
4. When it finishes, the status line says **Branch created**. The text is a link to the new branch on GitHub.
5. In your local checkout, fetch and switch to it:

```
git fetch origin
git switch WEB-42-make-the-checkout-button-visible-on-small-screens
```

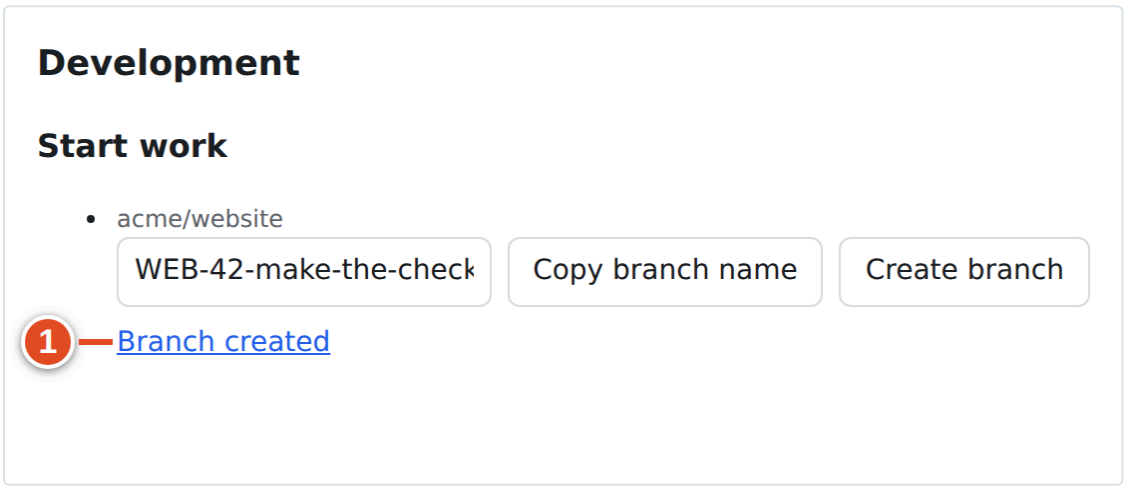


Figure 12.5 Screenshot (demo instance): After Create branch: (1) the confirmation with the new branch.

Steps (Copy branch name — local)

1. Select **Copy branch name**. ② The status line says **Copied**.
2. In your terminal: `git switch -c <paste>` and later `git push -u origin <paste>`.

If the browser blocks the clipboard, the status says **Copy failed; select the name and copy it**. Click into the name field (it selects itself) and copy by hand.

Result: a branch with the key in its name. Create branch also:

- adds the `[Branch]` link to the item straight away (otherwise it appears on your first push);
- records **branch_created** in the audit log.

Copying a name changes nothing in Plane. The state changes only when GitHub reports the branch: with the shipped rules, the first push of a new branch moves the item to **In Progress** (unless it is already completed or cancelled).

How Create branch works: git-sync uses the GitHub App (not your personal account) to create the branch from the *current tip of the repository's default branch* (usually `main`). It never overwrites anything. Pressing the button again is harmless:

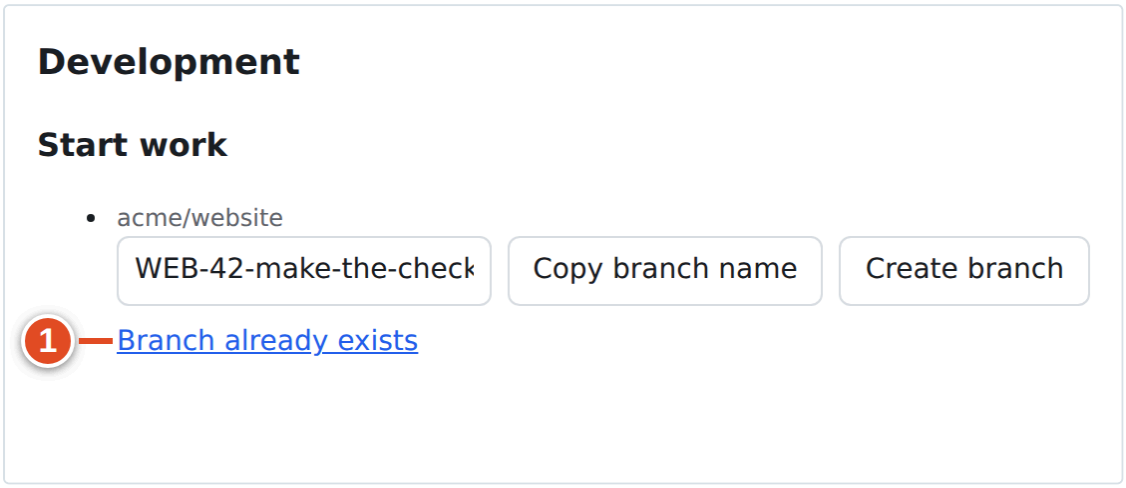


Figure 12.6 Screenshot (demo instance): Pressing Create branch again: (1) the branch already exists — nothing is overwritten.

If it went wrong:

Message	Cause	Fix
Branch already exists	A branch with that name is already on GitHub	Use it: `git fetch` and switch to it
The repository is empty; push a first commit before creating branches	No default branch yet	Push an initial commit to the repository
The GitHub App cannot create branches in this repository (it needs Contents: write)	App permission missing, or the repository is not covered by the installation	Ask an admin
Only project members can create branches	Your project role is below Member	Use **Copy branch name**, or ask to be added as a member
The mapping's branch template does not give a valid branch name	Custom template produced an invalid name	Ask an admin

How the panel stays up to date

The panel is not a live view of GitHub. git-sync keeps a set of *links on the Plane work item*, one per PR, build, deployment, preview and branch, and updates each link in place as GitHub events arrive. The panel reads those links (with your Plane session) and groups them by their title prefix.

The same links are visible in Plane's ordinary **Links** list on the work item, with these prefixes:

Prefix	Example title (verbatim from the demo run)
`[PR]`	`[PR] #121 WEB-42 Keep the checkout button visible on small screens · MERGED`
`[Branch]`	`[Branch] WEB-42-make-the-checkout-button-visible-on-small-screens`
`[CI]`	`[CI] Build · pass`
`[Deploy]`	`[Deploy] Preview · success`
`[Preview]`	`[Preview] Vercel`

> **Warning:** Do not rename or delete these links by hand. git-sync finds them by URL and will recreate or rename them on the next event; editing them only causes confusion.

Consequences:

- If GitHub could not reach git-sync for a while (an outage, a webhook problem), the panel shows the last recorded state until the next event. An admin can redeliver missed webhooks.
- Reload the work item to see changes; the panel does not refresh itself.

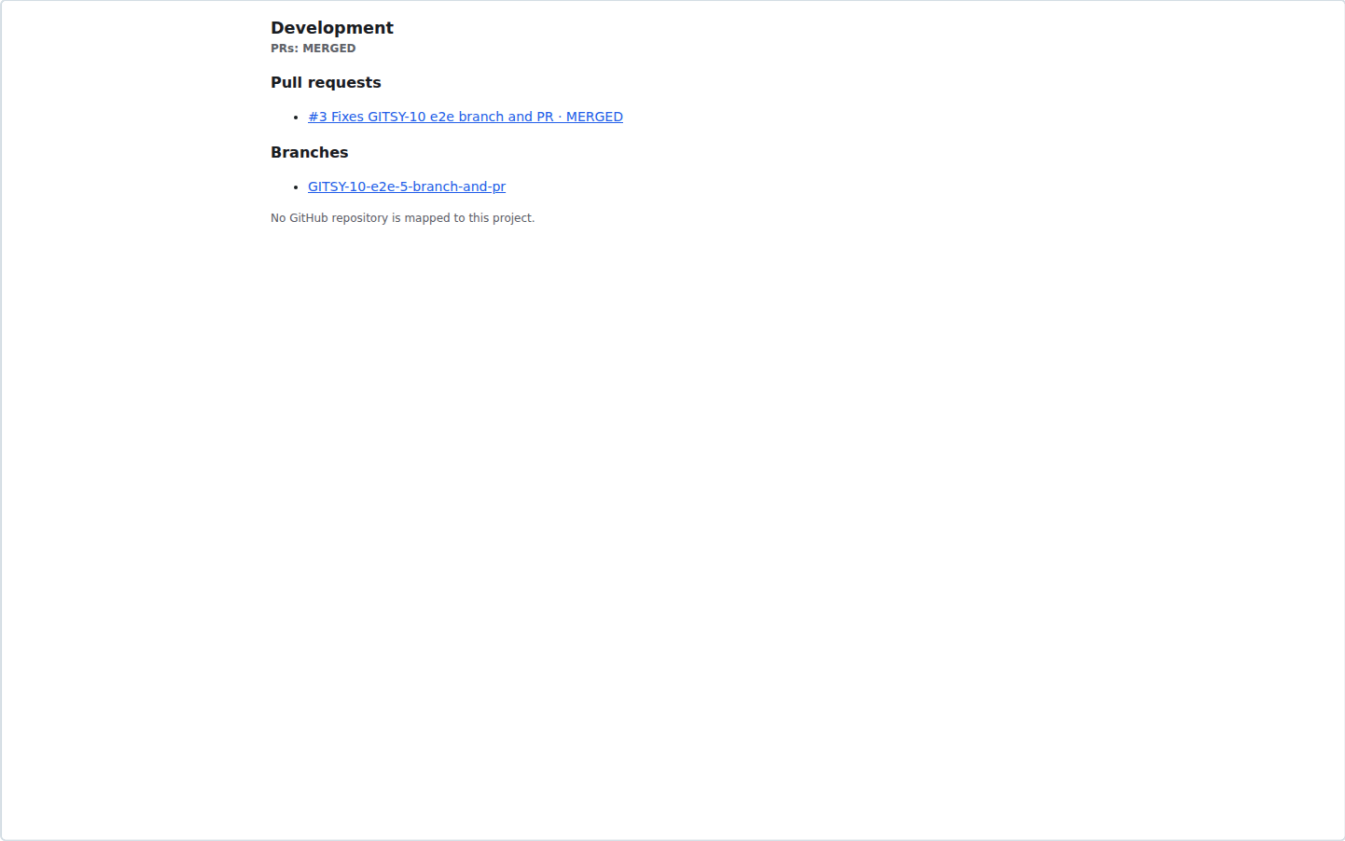


Figure 12.7 Screenshot (test Plane): The standalone Development page on the earlier test instance: historical links and the no-mapping message.

Reading the panel through a PR's life

The full sequence is shown step by step in chapter 15. In short, for WEB-42 in the demo run:

Moment	Panel shows	Item state (shipped rules)
Create branch / first push	**Branches**	In Progress
Draft PR opened	**PRs: OPEN**, `OPEN · REVIEW`	In Progress
CI running → passed	**Builds** `pending` → `pass`	unchanged
Preview deployed	**Deployments**, **Previews**	unchanged
Review requested	`OPEN · REVIEW`	In Review
Changes requested	`OPEN · REJECTED`	unchanged
Approved	`OPEN · APPROVAL`	unchanged
Merged into `main`	**PRs: MERGED**	Done

What you learned

- **Development** shows Pull requests, Builds, Deployments, Previews, Branches and Start work; empty sections are hidden.
- PR badges: `OPEN` / `MERGED` / `DECLINED`, plus `REVIEW` / `APPROVAL` / `REJECTED` for open PRs; builds are `pass` / `fail` / `pending`.
- Previews come only from deployment URLs on Vercel, Netlify, Cloudflare Pages or AWS Amplify.
- **Copy branch name** uses the template `{key}-{title-slug}` (not editable in the UI); **Create branch** creates the branch on GitHub from the default branch and never overwrites.
- The panel shows links git-sync maintains on the work item (`[PR]`, `[Branch]`, `[CI]`, `[Deploy]`, `[Preview]`); there is no commits list.

Source files

- `src/ui/client/DevPanel.tsx` — the panel and its messages
- `src/ui/dev-panel.ts` — panel API, access checks, Create branch
- `src/sync/dev-info.ts` — link titles, badges, build status, preview hosts, branch template
- `src/app/checks.ts` — builds, commit statuses, deployments, previews
- `src/app/push.ts` — `[Branch]` link on first push
- `apps/web/core/components/issues/issue-detail/sidebar.tsx` — the iframe in Plane's sidebar

13 · Issue mirroring and what syncs

git-sync connects Plane and GitHub in three separate ways. Each has its own rules, so keep them apart in your head:

Relationship	How it starts	What it does
Reference	A key such as `WEB-42` in a branch name, PR title or PR body	Links the PR or branch to the work item and lets PR automation change its state. No fields are copied.
Smart Commit	A key plus `#command` in a pushed commit message	Runs that one command (comment, time, state) in Plane.
Mirror pair	The `plane` label on a GitHub issue, or the `github` label on a Plane work item	Links a GitHub issue and a Plane work item, then copies six fields between them.

Chapters 9–11 cover references and Smart Commits. This chapter covers **mirror pairs**: what starts one, which fields are copied and in which direction, and what never syncs. It ends with how deliveries, queues, retries and echo suppression work. These explain why a change can take a moment to appear and why nothing loops forever.

> **Example:** `Closes WEB-42` in a PR body lets a merge move WEB-42 to Done, but the PR title never replaces the work item's title. Adding the `plane` label to GitHub issue #131 creates a new Plane work item and keeps its title, description, state, assignees, labels and new comments in step.

Start a mirror pair

Mirroring is **opt-in**: no issue is mirrored unless someone asks for it with a gate label. Gate-label names are compared case-insensitively.

Direction	Gate label	Where you add it	Needs
GitHub → Plane	`plane`	On the GitHub issue, when you open it or later	A repository mapping to a Plane project

Direction	Gate label	Where you add it	Needs
Plane → GitHub	`github`	On the Plane work item	A mapping whose **Sync direction** is **Two-way**

Goal: bring a GitHub issue into Plane

Before you start: the repository must be mapped to a Plane project. Ask your workspace admin if you are unsure.

Steps

1. Open the issue in the mapped repository on GitHub.
2. Add the label `plane`. If the label does not exist yet, create it with that exact name or ask a maintainer to.
3. Wait a moment. git-sync receives the `issues` webhook, queues it and creates the work item.
4. In Plane, open the mapped project and find the new item (filter by the newest items, or search the title).

Result: a new work item exists with the issue's title and description. Its first comment points back to the issue. In the demo run, GitHub issue #131 *Coupon field accepts expired codes* became *WEB-47*, and git-sync wrote this comment on it:

```
<p>Mirrored from GitHub issue <a
href="https://github.com/acme/website/issues/131">acme/website#131</a>.</p>
```

Text git-sync wrote in Plane in the demo run.

With **Post link-back comments on GitHub** on (the default), git-sync also adds a link on the Plane item and posts a comment on the GitHub issue:

```
Linked to Plane work item: http://plane.example.com/acme/projects/bb57e81d-
c161-441f-b50e-907af28f08c2/issues/7a2b5e69-aca5-49cb-8b4b-b71966c58b01
```

Text git-sync posted to GitHub in the demo run.

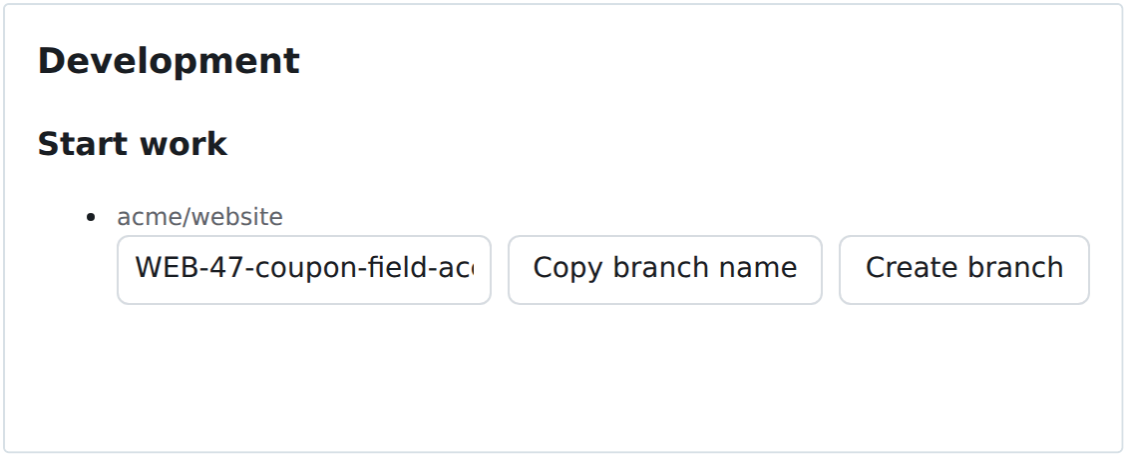


Figure 13.1 Screenshot (demo instance): A GitHub issue mirrored into Plane: the Development panel has no PR — mirrored issues appear as a link on the item, not here.

Screenshot needed — plane-03-mirrored-item — A Plane work item created from a GitHub issue labelled plane (WEB-47), showing the "Mirrored from GitHub issue" comment and the issue link. Capture later with npm run manual:capture -- --only plane-03-mirrored-item

- No work item appears → the repository is not mapped, the App cannot reach it, or the label is not exactly `plane`. Ask your admin to check **Errors in the last 24 hours** for the mapping.
- Two work items appear → someone created one by hand while the mirror was being made. Keep the mirrored one, because it holds the link.

Goal: send a Plane work item to GitHub

Before you start: the project's mapping must be **Two-way**. With **GitHub to Plane only**, git-sync ignores every Plane-side event, including the `github` label.

Steps

1. Open the work item in Plane.
2. Add the project label `github`.
3. Wait a moment, then open the mapped repository's **Issues** tab on GitHub.

Result: a new GitHub issue exists. In the demo run, **WEB-44** *Add rate limiting to the login form* became GitHub issue #120:

```
#120 Add rate limiting to the login form
Limit failed sign-ins to **5 per minute**.

---
Mirrored from Plane: http://plane.example.com/acme/projects/bb57e81d-c161-
441f-b50e-907af28f08c2/issues/f19a4a55-c98c-4829-af86-1505b5001d16
labels: ['plane']
```

The GitHub issue git-sync created in the demo run.

git-sync confirmed it on the Plane item with the comment `Mirrored to GitHub issue acme/website#120.` and a link to the issue.

What creation copies

The two directions do not copy the same things at creation time:

Created on	Copied when the pair is created
Plane (from a GitHub issue)	Title; description converted from Markdown to Plane's HTML; labels except `plane`; assignees whose GitHub accounts are linked to Plane members; state (an open issue gets the mapping's **Open state** or Plane's project default, a closed issue gets the **Closed state** or a completed-group state). git-sync also makes sure a `github` label exists in the project.
GitHub (from a Plane item)	Title; description converted to Markdown with a `Mirrored from Plane: <url>` footer; labels, with `plane` added and `github` removed. *Assignees and state are not set at creation.* Later assignment and state changes sync normally.

Old comments are not copied when a pair is created. Only comments made after that point are mirrored.

> **Note:** Removing the gate label later does **not** unlink the pair. The link is stored durably in git-sync. Pull requests are never mirrored as issues, even though GitHub treats them as issues internally, so editing a PR never renames its work item.

The six mirrored fields

Once a pair exists, exactly six things can cross between the two sides:

Field	GitHub → Plane	Plane → GitHub
Title	Issue title becomes the work-item name	Work-item name becomes the issue title
Description	Markdown is converted to Plane HTML	Plane HTML is converted to Markdown
State	Closing or reopening the issue picks a mapped Plane state	The Plane state group closes or reopens the issue
Assignees	Linked GitHub accounts become Plane assignees	Linked Plane members become GitHub assignees
Labels	Label names are found or created in Plane	Added and removed labels are applied on GitHub
New comments	Added as a new Plane comment that names its author	Added as a new GitHub comment that names its author

Everything else stays where it was written (see What never syncs).

Who wins when both sides change: conflict policies

Each mapping has one **conflict policy** per field. A policy decides which side's change is allowed to cross over when both sides could change a value. Workspace admins set policies on the mapping's **Conflict policies** tab (Chapter 25). The UI explains it as *"Which side wins when both change the same field between syncs."*

Policy (UI label)	Stored value	Meaning
Last write wins	`last_write_wins`	Changes from either side cross over
GitHub is the source of truth	`source_of_truth:github`	Only GitHub changes cross over to Plane
Plane is the source of truth	`source_of_truth:plane`	Only Plane changes cross over to GitHub

The shipped defaults are:

Field	Default
Title	**GitHub is the source of truth**
Description	**GitHub is the source of truth**
State	**Last write wins**
Assignees	**Last write wins**
Labels	**Last write wins**
Comments	**Last write wins**

In practice, with the defaults you should *edit a mirrored title or description on GitHub*. A Plane title edit is not sent to GitHub, and the next GitHub title edit overwrites it in Plane.

> **Warning:** A source-of-truth policy is not an editing lock. Plane still lets you type a new title. That local edit stays until the next eligible change arrives from GitHub and replaces it. Agree with your team which side owns which field.

Sync direction comes first. If the mapping is **GitHub to Plane only**, every Plane-side event is ignored: no state, comment or label from Plane ever reaches GitHub. Setting a field to **Plane is the source of truth** on such a mapping leaves that field syncing in neither direction. If you see that combination, tell your admin.

“Last write wins” is not a merge of two texts. For whole-value fields, git-sync reads the source object’s current value and its `updated_at` time before writing. A delivery that arrives late with an older snapshot does not overwrite a newer value. There is no conflict dialog, so make larger description edits on one side only.

Closing and reopening

GitHub issues are only open or closed, while Plane projects have many states. The mapping’s **Open state** and **Closed state** fields (shown as **Project default** when not set) decide where a GitHub close or reopen lands in Plane.

What happens	Result on the other side (when the State policy allows it)
GitHub issue closed	Plane item moves to the mapping’s **Closed state**, otherwise to a completed-group state
GitHub issue reopened	Plane item moves to the mapping’s **Open state**, otherwise to a not-started (unstarted-group) state

What happens	Result on the other side (when the State policy allows it)
Plane item moved into a **completed** state	GitHub issue closed with reason `completed`
Plane item moved into a **cancelled** state	GitHub issue closed with reason `not_planned`
Plane item moved into any other group	GitHub issue reopened (reason `reopened`) if it was closed

If the chosen state was deleted or no longer fits, git-sync falls back to a suitable default and records a `mirror_state_fallback` entry in the audit log. It never moves an item into **Triage**.

Moving an item from **In Progress** to **In Review** changes nothing on GitHub, because both are open states. That is expected: GitHub cannot show every Plane workflow step. Likewise, closing a GitHub issue as *not planned* does not choose a cancelled Plane state. Incoming closes always use the mapping's **Closed state** or the completed fallback.

People: assignees and mentions

People are translated through the workspace's user links: a member's personal link (Chapter 8) or an admin-made link (Chapter 27). Links are keyed by GitHub's stable numeric account ID, never by display name or commit email.

Situation	What git-sync does
GitHub assignee linked to a Plane member	That member becomes a Plane assignee
GitHub assignee with no link	Dropped, so the Plane item may show fewer assignees
Plane assignee linked to a GitHub account	Added on GitHub. git-sync only adds or removes linked accounts and leaves GitHub assignees it cannot represent alone.
`@login` in GitHub text, linked	Becomes a real Plane mention
`@login` in GitHub text, not linked	Becomes a link to the GitHub profile
Plane mention of a linked member	Becomes `@login` on GitHub
Plane mention of an unlinked member	Becomes their display name (or "Unknown user" if none is known)
`#123`, `owner/repo#123`, issue URLs	Turned into explicit GitHub links in Plane text

In the demo, Sam Reviewer commented on #131: *“Reproduced on staging. @alex-dev can you take this?”* Because Alex has linked their GitHub account, the Plane copy contains a real Plane mention of Alex, who gets a notification:

```
<p><strong>sam-reviews</strong> commented on <a
href="https://github.com/acme/website/issues/131#issuecomment-
77001">GitHub</a>:</p><p>Reproduced on staging. <mention-component
entity_identifier="6a57833d-4e43-49be-924c-f093cddfae76"
entity_name="user_mention"></mention-component> can you take this?</p>
```

Text git-sync wrote in Plane in the demo run.

Inline code and code blocks are not rewritten.

> **Note:** On the Plane side, every change git-sync makes is shown as made by the git-sync service account, even if it came from you. This release has no way to store a personal Plane token per member. Changes git-sync makes on GitHub *for* a Plane edit are made as you if your personal GitHub link works, and as the App otherwise. The comment text always names the original author (“**alex** commented in Plane:”, “*sam-reviews* commented on GitHub:”).

Labels and comments

Labels are matched by name, not by internal ID. A label missing on the target side is created where possible. Colors and label descriptions are not copied. GitHub label events send the issue’s current label set to Plane, without the `plane` gate label. Plane label events send the added and removed labels, resolved to names.

Comments: only **new** comments are mirrored.

- A GitHub comment appears in Plane starting with “**login** commented on GitHub:”, with a link to the original.
- A Plane comment appears on GitHub starting with “**Name** commented in Plane:”.
- Editing or deleting a comment is **not** mirrored. If both sides need a correction, add a new comment.
- PR review discussions are not mirrored. They belong to a PR, not to a mirrored issue.

What never syncs

The mirror is deliberately much smaller than Plane’s data model. These are **never** copied in either direction:

- priority, start date and due date;
- estimates (points);

- cycles and modules;
- parent and sub-work-item structure, and relations (blocked by, relates to, duplicate of);
- attachments as files (a link or image inside a description survives conversion, but no files are copied);
- subscribers, reactions, views, pages, intake, notification settings and project settings;
- edits and deletions of existing comments;
- label colors and descriptions.

Deleting is not closing

Plane v1.4.2 sends no webhook when a work item is deleted, so git-sync cannot react straight away. Instead, a **reconciliation** sweep runs every 60 minutes by default. It works out whether a linked item is archived, deleted or just hidden:

Plane-side situation	What reconciliation does
Item or its project **archived**	Marks the link archived. No comment, nothing closed.
Item **deleted** (confirmed, or missing on two sweeps in a row)	Marks the link **orphaned** and posts on GitHub: *"The linked Plane work item can no longer be found, so this link has been marked orphaned. Nothing was changed or closed here."*

So *never delete or archive a work item to close its GitHub issue.* Move it to the agreed completed or cancelled state instead. Deleting a GitHub issue or comment does not delete anything in Plane either.

How deliveries travel: queues, retries, echoes

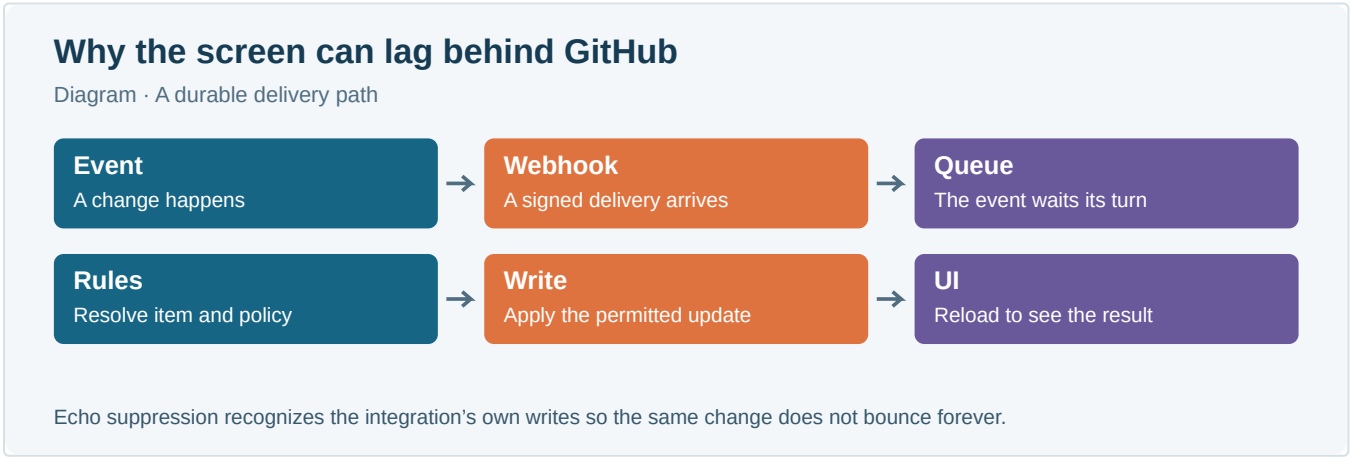


Figure 13.2 Diagram: events travel through signed webhooks, queues, rules and durable writes before a refreshed UI reflects the result.

A change takes this path:

1. **Webhook.** GitHub or Plane sends a short signed message to git-sync. git-sync checks the signature, records the delivery and queues it, then answers straight away.
2. **Inbound worker.** A worker reads the queued delivery, finds the mapping and decides what should change.
3. **Outbound writes.** Each change is registered in a durable, ordered outbox, with one queue for Plane-bound writes and one for GitHub-bound writes. If Plane is down, GitHub-bound work keeps flowing, and the reverse.
4. **Retries.** A write that fails for a temporary reason (network error, 5xx, rate limit) is retried with back-off. Writes to the same target run in order. A multi-step operation such as “create issue, remember its number, post link-back” resumes where it stopped, so a half-finished mirror never needs to be redone by hand.
5. **Audit.** Every delivery and result is written to the audit log, which your admin can read (Chapter 28).

No fixed delay is promised. A few seconds is normal, but rate limits and outages make it longer. *Do not keep editing to “wake up” the sync.* Each edit is a new event.

Four safeguards stop duplicates and loops:

Safeguard	What it prevents
Echo suppression	git-sync remembers each field it wrote (for 10 minutes by default). When the other side reports that same change back, git-sync recognizes it as an echo and ignores it, so nothing bounces back and forth.
Service-user filter	Plane events made by git-sync’s own service account are dropped on arrival. (GitHub bots in general are *not* ignored; their actions run under the service identity.)
Deduplication	A webhook delivered twice is processed once. A Smart Commit SHA runs its commands once.
Out-of-order protection	git-sync reads the current state before acting. An old PR event that arrives after a newer one never moves an item backwards, and the audit log records it as ignored.

Plane sends one webhook per changed field, so one edit can create several queue entries. That is normal.

> **Warning:** A consequence of the service-user filter: when *git-sync itself* changes a mirrored Plane item (for example a PR merge rule moves it to Done, or a Smart Commit `#done` runs), that change is **not** mirrored on to the GitHub issue, because it arrives as a service-account event. If the GitHub issue must close too, write `Closes #131` in the PR so GitHub closes it, or close it by hand.

> **Tip:** If a mirrored change is missing after a few minutes, collect the GitHub issue URL, the Plane key, the time with timezone, and what you expected. Send them to your admin, who can find the delivery in the audit log and **Replay** it if needed (Chapter 17 explains what to send).

A quick way to investigate a mismatch

1. Decide which relationship you are looking at: a mirror pair, a PR reference, or a Smart Commit.
2. Open the source object (the GitHub issue or the Plane item) and check that the change is really there.
3. Check the mapping's **Sync direction** and the field's conflict policy. Ask your admin if you cannot see them.
4. For people: check that both people have linked GitHub accounts.
5. For state: remember that only group changes (open, completed, cancelled) show on GitHub.
6. Record the expected and the actual result, and report them.

What you learned

- Mirroring is opt-in per issue: `plane` on GitHub, `github` in Plane (two-way mappings only). Removing the label does not unlink.
- Exactly six fields sync: title, description, state, assignees, labels and new comments, each under a conflict policy. By default GitHub owns title and description.
- State maps by group: completed → closed `completed`, cancelled → closed `not_planned`. Incoming closes use the mapping's **Closed state**.
- Priority, dates, estimates, cycles, modules, relations, attachments and comment edits never sync. Deleting a Plane item never closes anything.
- Deliveries are queued, retried, deduplicated and echo-suppressed, so wait and report instead of re-editing.

Source files

- `src/sync/mirror.ts`: mirror creation, six-field sync, policies, state mapping, assignees and mentions
- `src/app/issues.ts`: GitHub `issues` / `issue_comment` and Plane `issue` / `issue_comment` handlers
- `src/sync/text.ts`: Markdown ↔ Plane HTML conversion and reference links
- `src/sync/reconcile.ts`, `src/app/main.ts`: reconciliation, archive and orphan handling
- `src/sync/ophash.ts`, `src/db/ledgers.ts`: echo ledger and deduplication
- `src/queue/*`, `src/operations/outbox.ts`, `src/app/writers.ts`: queues, ordered outbox and retries
- `src/ui/validate.ts`: conflict-policy fields and defaults

14 · PR automation from a developer's view

When your pull request is opened, reviewed or merged, git-sync can move the linked work item to a new state. Your admin sets this up with **rules**; Chapter 26 shows how to edit them. This chapter is for developers. It covers which PR moments trigger a rule, what the shipped defaults do, why a reference's tier matters, how **ready for merge** is decided, and why a merge sometimes does *not* move an item to Done.

> **Note:** Everything here applies to work items linked by a **reference** (branch name, PR title or PR body; see Chapter 9). Mirrored GitHub issues follow Chapter 13 instead.

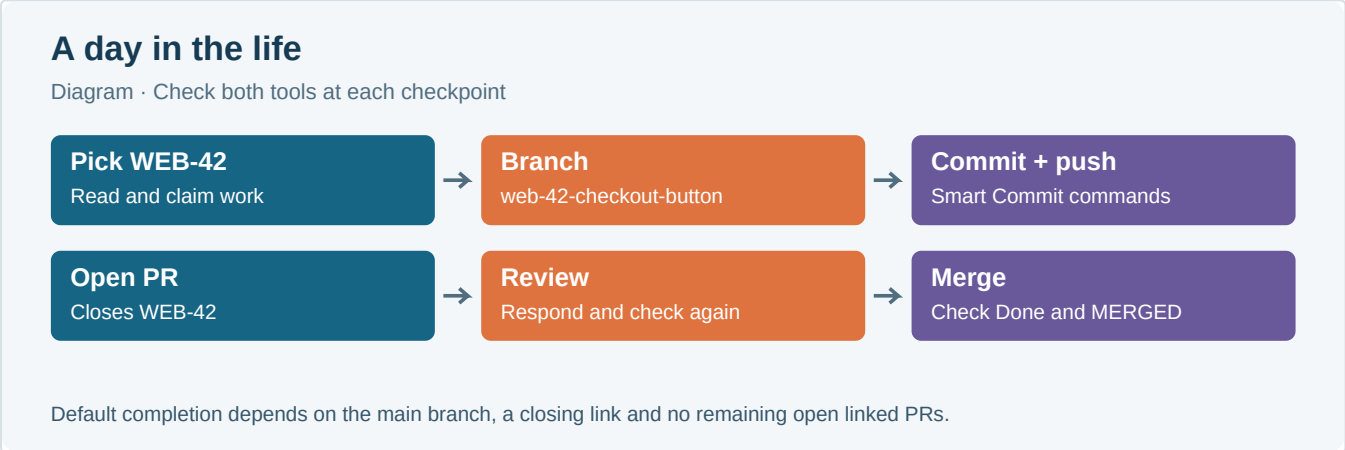


Figure 14.1 Diagram: the daily loop from picking WEB-42 to verifying Done and MERGED.

Lifecycle points

git-sync turns GitHub events into named **lifecycle points**. Each one is a rule trigger:

Lifecycle point (rules table)	Trigger name	What you did on GitHub
Branch pushed	`branch.created`	Pushed a new branch whose name contains the key
PR drafted	`pull_request.drafted`	Opened a draft PR, or converted a PR back to draft
PR opened	`pull_request.opened`	Opened a normal PR, or marked a draft **Ready for review**

Lifecycle point (rules table)	Trigger name	What you did on GitHub
Review requested	`pull_request.review_requested`	Requested a reviewer
Changes requested	`pull_request.changes_requested`	A reviewer submitted **Request changes**
Approved	`pull_request.approved`	A reviewer approved, but the PR is not yet ready for merge
Ready for merge	`pull_request.ready_for_merge`	git-sync checked that the PR is approved and green (see below)
PR merged	`pull_request.merged`	Merged the PR
Closed without merge	`pull_request.closed_unmerged`	Closed the PR without merging
Reopened	`pull_request.reopened`	Reopened a closed PR

Admins can also write rules for `push`, `commit.pushed_to_default`, `check_suite.*`, `deployment_status.*`, `issue.*` and `comment.*` (Appendix D). Those are less common.

What the shipped defaults do

A new mapping starts with this table. Lifecycle points that are not listed change nothing.

Lifecycle point	Default target
Branch pushed	**In Progress** (started group)
PR drafted	**In Progress**
PR opened	**In Progress**
Review requested	**In Review** (started group)
PR merged into a base matching `^main$`	**Done** (completed group)
Changes requested, approved, ready for merge, closed without merge, reopened	*no change*

Some details are easy to miss:

- **Approval does not move the item.** Neither does ready for merge, unless your admin adds a rule for it.

- **Merged into another branch does nothing by default.** A PR merged into `develop` or `staging` leaves the item where it was, unless a branch-aware rule says otherwise (see the staging example below).
- **A new branch never reopens finished work.** If the item is already in a completed or cancelled state, *Branch pushed* leaves it alone. Branches whose names start with `revert-` never trigger *Branch pushed* at all.
- **State names are matched inside their group.** The default `started:In Review` looks for a state called *In Review* in the started group. If your project has no such state but exactly one started state, that state is used. Otherwise the rule fails, and your admin sees the failure in the audit log. Tell your admin if your project names its states differently.
- **Rules evaluate in order and the first match wins.** Only one rule's actions run for each event.

Tiers decide whether the state may change

Chapter 9 introduced the three reference tiers. For PR automation they matter a lot, because they gate **state changes** (the `set_state` action):

Tier	Typical text	State changes on PR events	State change on merge
Closing	`Closes WEB-42`, a key in the branch name or PR title	Yes	Yes, if the merge guard passes
Non-closing	`Refs WEB-42`, `Part of WEB-42`	Yes	**No**
Relation	`Relates to WEB-42`, a bare key in the PR body	**Never**	**Never**

The demo run shows all three side by side:

- **WEB-41** was referenced with `Refs WEB-41`. Opening the PR moved it to **In Progress**, and a merge would leave it there.
- **WEB-40** was referenced with `Relates to WEB-40`. It stayed in **Todo**, even though the PR is linked and shown in its Development panel.
- **WEB-42** was closed by its branch name, title and `Closes WEB-42`. It went all the way to **Done** (Chapter 15).

Tier gating applies **only** to state changes. Other actions an admin puts in a rule, such as a comment, label, assignee, link or time note, run for every tier. So a relation-only PR can still receive a label from a rule.

When is a PR “ready for merge”?

git-sync computes readiness itself from GitHub’s current data, not from the event you just caused. A PR is ready for merge when **all** of these are true:

1. It is open and not a draft.
2. It has at least one **standing approval**. Only each reviewer’s latest decisive review counts (approved, changes requested or dismissed).
3. *No reviewer* still has changes requested. One person’s approval does not cancel another person’s change request.
4. Every relevant **check suite** has completed as success, neutral or skipped. Suites with no runs, and stale suites, are ignored.
5. The *combined commit status* is success, or no statuses are reported.

git-sync checks this after an approval, after a check suite completes successfully, and when a PR becomes ready for review or is reopened. It fires `pull_request.ready_for_merge` **once per head commit**: pushing a new commit means it can become ready again. In the demo run, the audit log recorded `ready_for_merge fired` for PR #121 when Sam approved it.

By default nothing happens at ready for merge. Admins often add a rule here, such as a comment that says the PR is ready, or a move to a *Ready to ship* state.

> **Note:** Readiness is git-sync’s own view. It does not replace GitHub’s branch protection, and GitHub may still block the merge button for reasons git-sync does not check (required reviewers, code owners, merge queues).

The merge guard: one item, several PRs

A work item can be linked to more than one PR. When one of them merges, a closing reference moves the item to Done only if:

- **at least one** linked PR has merged, **and**
- **none** of the linked PRs is still open.

A PR that was closed without merging neither blocks nor counts. A linked PR that git-sync cannot read (deleted repository, no access) is treated as **open**, because finishing work it cannot see would be unsafe.

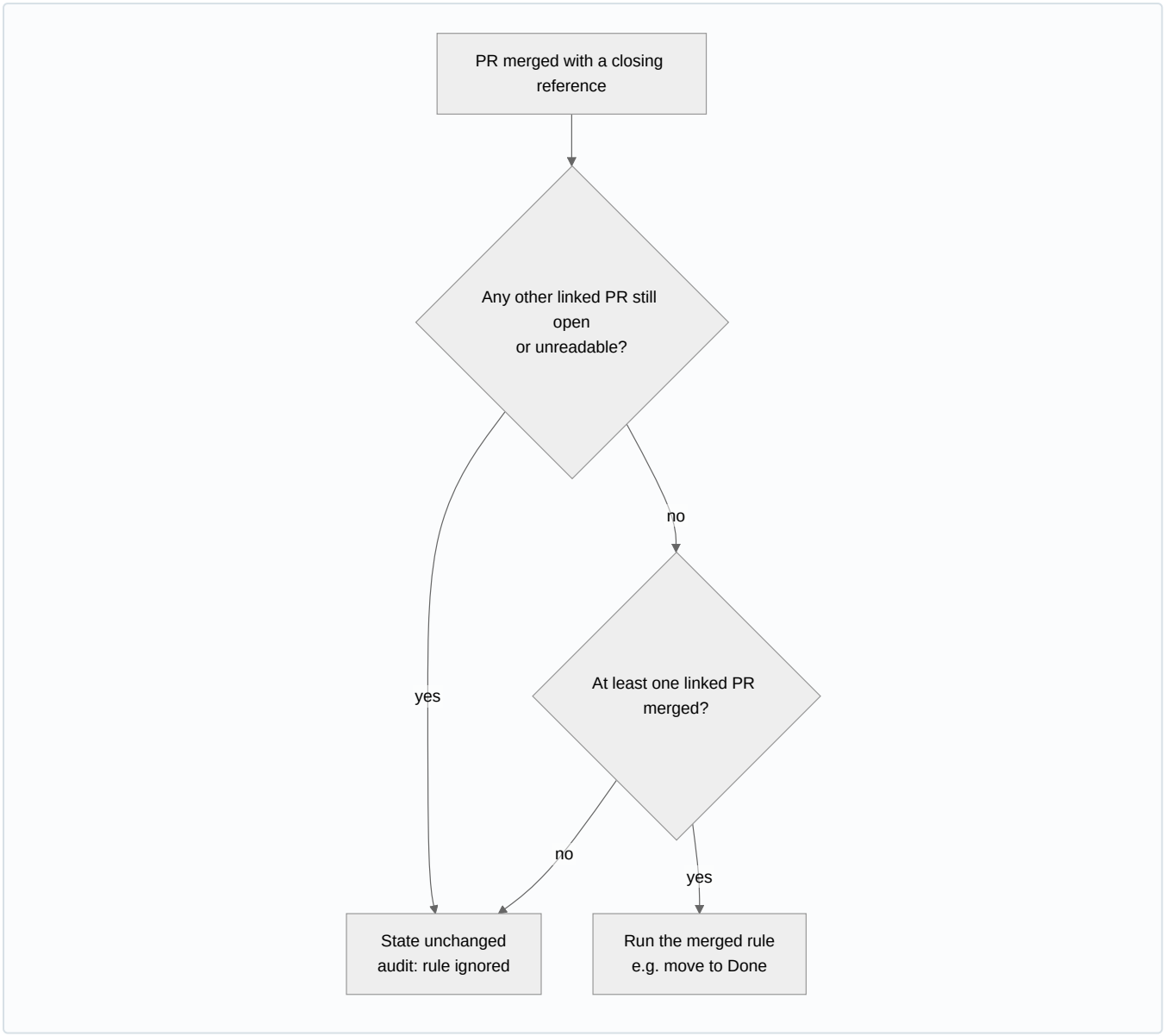


Figure 14.3 Diagram: the merge guard, checked when a linked PR merges.

In the demo run, WEB-36 had two closing PRs: #124 *Guest checkout: backend* and #125 *Guest checkout: form*. #124 merged while #125 was still open, so WEB-36 stayed **In Progress** and the audit log recorded the merged rule as `ignored`.

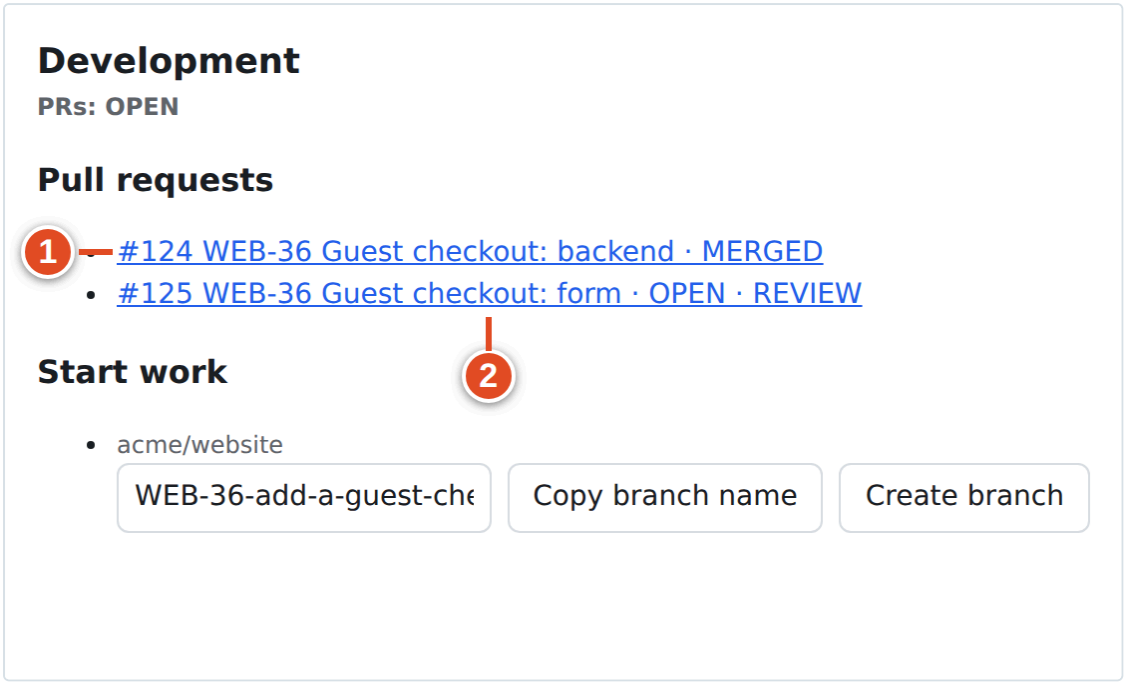


Figure 14.2 Screenshot (demo instance): Two PRs close WEB-36: (1) one merged; (2) one still open — the item does not move to Done yet.

> **Warning:** The guard is checked *only at the merge event*. If #125 is later closed *without* merging, git-sync does not go back and finish WEB-36. Close obsolete PRs **before** the final merge, or move the item to Done yourself after an abandoned PR.

Example: staging and main

Many teams merge feature branches into `staging` first and into `main` at release time. A **branch-aware rule** handles that. The demo mapping has one, added on the **Rules** tab:

Rule (first match wins)	When	If	Then
merged into `^staging$`	`pull_request.merged`	base branch matches `^staging$`	**In QA**
default: merged	`pull_request.merged`	base branch matches `^main$`	**Done**

In the demo run, PR #126 *WEB-39 Speed up the checkout page* targeted `staging`. On merge, the audit log recorded rule `merged into ^staging$` as applied, and WEB-39 moved to **In QA**. A later release PR from `staging` into `main` that references WEB-39 would move it to **Done**.

One PR, event by event

This table follows a single closing PR (title `WEB-42 …`, body `Closes WEB-42`) through every GitHub action it can see, using the shipped rules:

You do on GitHub	Event git-sync receives	Lifecycle point	WEB-42 afterwards	Development badge
`git push` of a new branch `WEB-42-…`	`push` (branch created)	Branch pushed	In Progress	Branch listed
`gh pr create --draft`	`pull_request` opened (draft)	PR drafted	In Progress	`OPEN · REVIEW`
`gh pr ready`	`pull_request` ready_for_review	PR opened	In Progress	`OPEN · REVIEW`
Add a reviewer	`pull_request` review_requested	Review requested	**In Review**	`OPEN · REVIEW`
Reviewer: **Request changes**	`pull_request_review` submitted	Changes requested	In Review	`OPEN · REJECTED`
Push a fix	`push` + `pull_request` synchronize	— (Smart Commits only)	In Review	`OPEN · REJECTED`
CI finishes green	`check_suite` completed	(readiness check)	In Review	`[CI] Build · pass`
Reviewer: **Approve**	`pull_request_review` submitted	Ready for merge (checks green) or Approved	In Review	`OPEN · APPROVAL`
Merge into `main`	`pull_request` closed, merged	PR merged	**Done**	`MERGED`
Close without merging (instead)	`pull_request` closed	Closed without merge	unchanged	`DECLINED`
Reopen	`pull_request` reopened	Reopened	unchanged	`OPEN · …`

Every row except the last two was verified on the demo instance (Chapter 15).

Rules your admin may add

The defaults are only a starting point. Rules can also add or remove a label, assign someone, post a comment, add a link, log time as a comment, or create another work item. A rule can be limited by base branch, head branch, PR labels, author, repository or project. Common additions:

Team wish	Rule idea
“Changes requested should send it back”	Changes requested → **In Progress**
“Tell us when it’s mergeable”	Ready for merge → comment *Ready to merge*
“Hotfix branches are urgent”	Head branch `^hotfix/` → add label `hotfix`
“Closed without merge means dropped”	Closed without merge → **Cancelled**
“Release branch means QA”	Merged into `^release/` → **In QA**

If a comment, label or assignee appears on your item after a PR event, a rule probably did it. Ask your admin to show you the rule list. The **Rules** tab and the **Test rules** dry-run are described in Chapter 26.

Reading what happened

When a state does or does not change, three places tell you why:

Where	What it shows
Plane work item: activity and state	The result. The change appears as made by the git-sync service account.
Development panel	Which PRs are linked and their status and review badges (`OPEN · REVIEW` , `OPEN · REJECTED` , `OPEN · APPROVAL` , `MERGED` , `DECLINED`)
Audit log (admin)	One `rule_match` entry per evaluated rule: `applied` , `ignored` (the tier or the merge guard skipped it) or `failed` (for example, the state does not exist)

Members cannot open the audit log, so send your admin the PR URL and the time (Chapter 17).

What you learned

- PR events become lifecycle points. By default: branch, draft or open → In Progress; review requested → In Review; merged into `main` → Done. Nothing else moves.
- Tiers gate only state changes: relation never moves an item, and non-closing never moves it on merge.
- Ready for merge means open, not draft, approved, no standing change request and green checks. It fires once per head commit and changes nothing by default.
- The merge guard needs at least one merged and no open linked PR, and it is only checked at the merge event.
- Branch-aware rules let merges into `staging` and `main` mean different states.

Source files

- `src/github/pr-automation.ts`: lifecycle table, defaults, `prEventToRuleEvent`, `isReadyForMerge`, tier gating, state resolution
- `src/app/pr.ts`: PR handler, merge guard (`otherPrStates`), ready-for-merge firing, link-back
- `src/app/rules.ts`: rule running, push and branch rules, `revert-` exclusion
- `src/rules/engine.ts`, `src/rules/types.ts`: triggers, conditions, actions, first-match evaluation

15 · A day in the life

This chapter follows one small change from start to finish. Alex Developer picks up *WEB-42 — Make the checkout button visible on small screens*, and Sam Reviewer reviews it. Each step says three things:

- **You do:** the exact action in Plane, git or GitHub;
- **git-sync does:** what the service does with the resulting event;
- **You see:** what to check, with a screenshot of the Development panel at that moment.

> **Note:** *About these screenshots.* The Development-panel images are real renders of git-sync's own UI, taken from the *demo instance*: the actual git-sync code running locally on a throwaway database, fed with the same webhook events GitHub would send for every step below. The states, badges, comments and links are what the real code produced. The GitHub screenshots come from an earlier real test on the authorized test repository (PR #117 for the same work item), not from this run. Plane's own work-item page for this run could not be captured; see the placeholders.

The run assumes:

- the shipped rules plus one branch-aware rule (Chapter 14);
- a two-way mapping of `acme/website` to the **Website** (`WEB`) project, with **Smart Commits**, **Post link-back comments on GitHub** and **Report problems as comments on GitHub and Plane** turned on;
- Alex and Sam have both linked their GitHub accounts (Chapter 8).

Your team's rules can differ. Treat every *You see* as a checkpoint against your own setup.

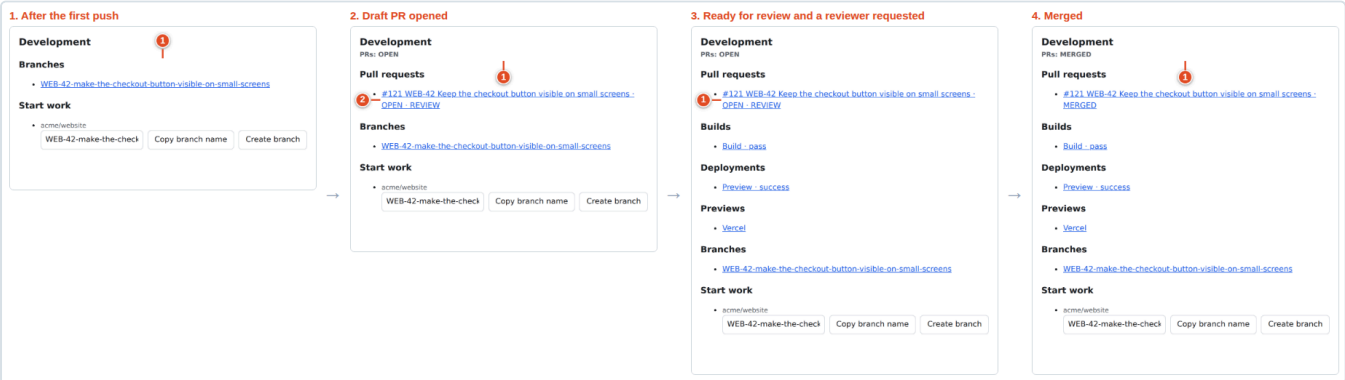


Figure 15.1 Strip (demo instance): The WEB-42 loop in four frames: push → draft PR → review → merged.

Step	Event	WEB-42 state afterwards	PR badge
1–2	Read and start the item	Todo → In Progress (by hand)	—
3	**Create branch**	(no event yet)	—
4	First push with Smart Commits	In Progress	—
5	Draft PR opened	In Progress	`OPEN · REVIEW`
6	CI runs, preview deploys	In Progress	`OPEN · REVIEW`
7	Ready for review + reviewer requested	**In Review**	`OPEN · REVIEW`
8	Changes requested	In Review	`OPEN · REJECTED`
9	Fix pushed, CI green, approved	In Review	`OPEN · APPROVAL`
10	Merged into `main`	**Done**	`MERGED`

The demo run, verified: each row is the state the real code produced.

Step 1 — Read the work item

You do: open **Website** → **Work items** in Plane and open `WEB-42`. Read the description, the comments, the assignee and the priority. Check for blocking relations.

The description gives the finish line: *Button visible at 320, 375 and 390 px. Focus indicator visible.* That is small, testable and clear. The item is assigned to Alex, its priority is **High**, and it carries the labels `frontend` and `accessibility`.

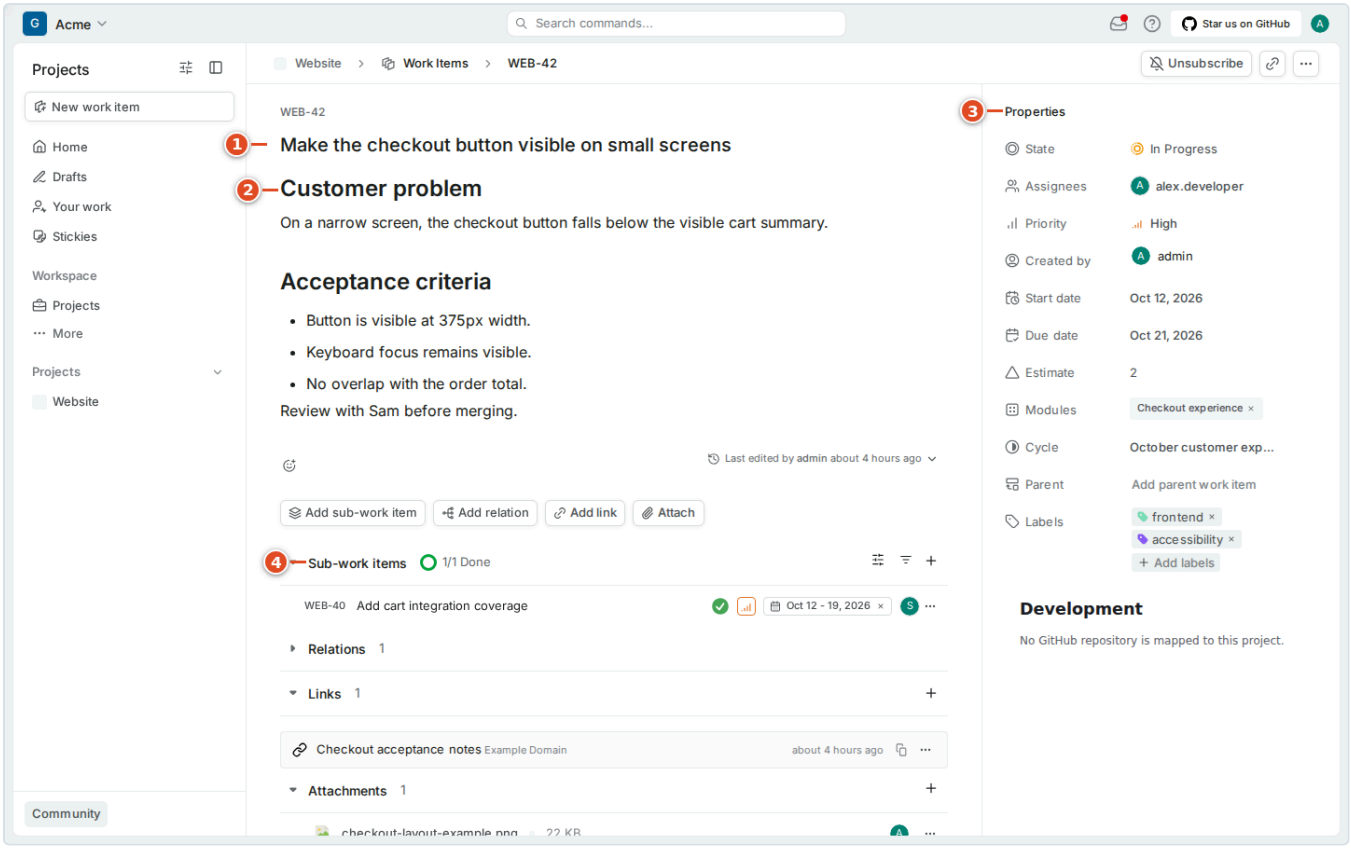


Figure 15.2 Screenshot (test Plane): WEB-42: (1) title and identifier; (2) description; (3) properties; (4) sub-work items.

Step 2 — Say that you started

You do: set the state to **In Progress** and add a short comment such as *“Starting the narrow-screen fix; I’ll keep the focus order and check the three widths.”*

git-sync does: nothing yet. Plane-side state changes of a referenced (not mirrored) item are not sent anywhere.

Why bother? The automation would also move the item to In Progress on the first push. Setting it yourself tells teammates now, before any code exists.

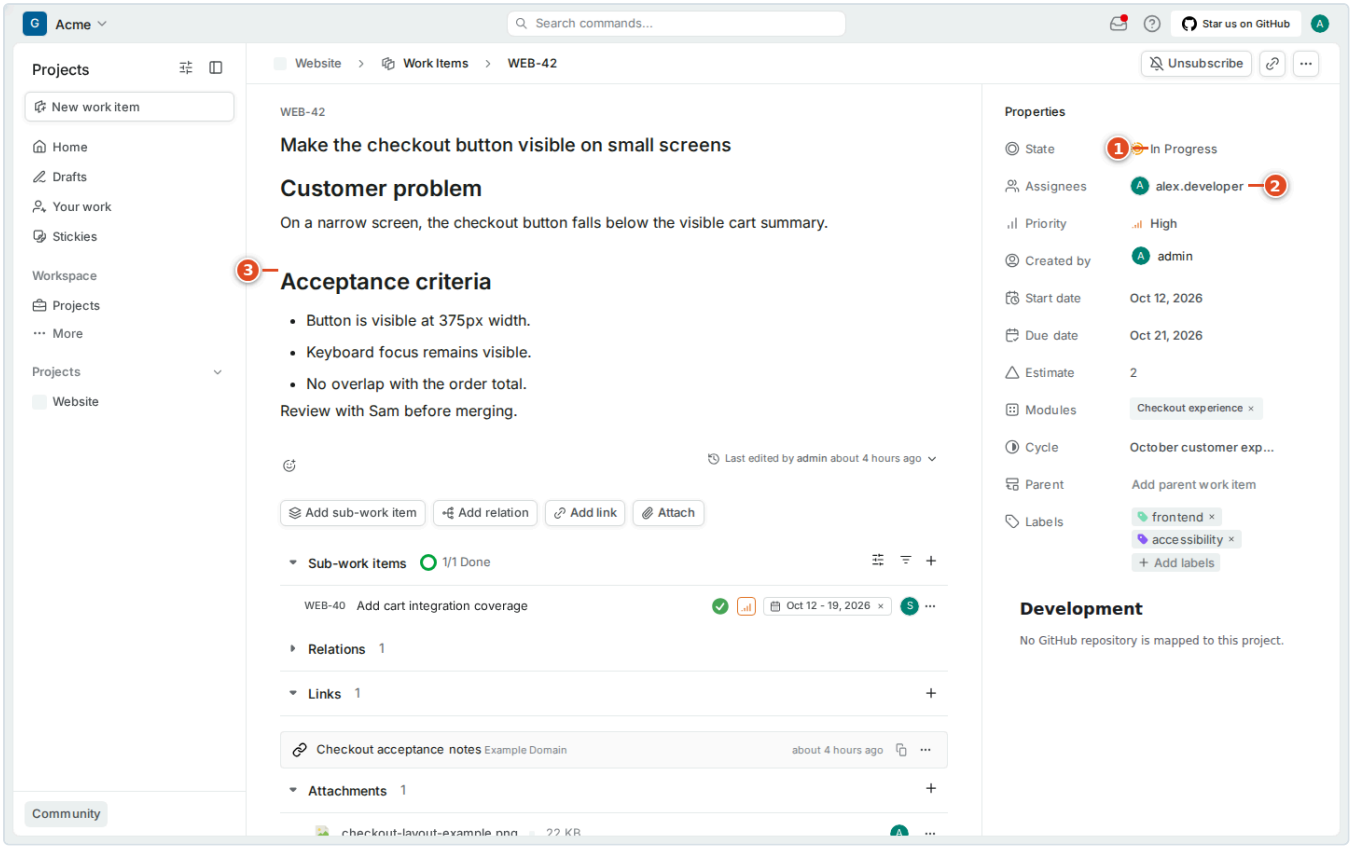


Figure 15.3 Screenshot (test Plane): Starting work: (1) In Progress state; (2) assigned developer; (3) acceptance criteria.

Step 3 — Create the branch from the Development panel

You do: in the work item's sidebar, find **Development**. Under **Start work** it suggests a branch name built from the default template `{key}-{title-slug}`:

```
WEB-42-make-the-checkout-button-visible-on-small-screens
```

Press **Create branch**. To create the branch yourself instead, press **Copy branch name** and use it in git.

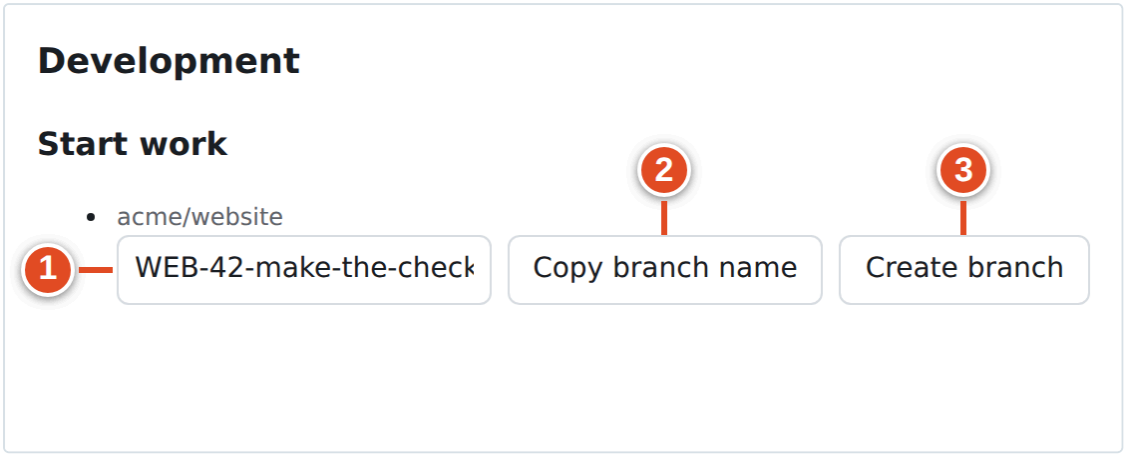


Figure 15.4 Screenshot (demo instance): WEB-42 before any code: (1) suggested branch name; (2) Copy branch name; (3) Create branch.

git-sync does: it reads the repository's default branch (`main`), then creates the branch on GitHub from main's latest commit using the App's installation. It records a `branch_created` audit entry and adds a `[Branch]` link to the item. Press the button again and it answers **Branch already exists**: nothing is overwritten.

You see: Branch created with the branch name.

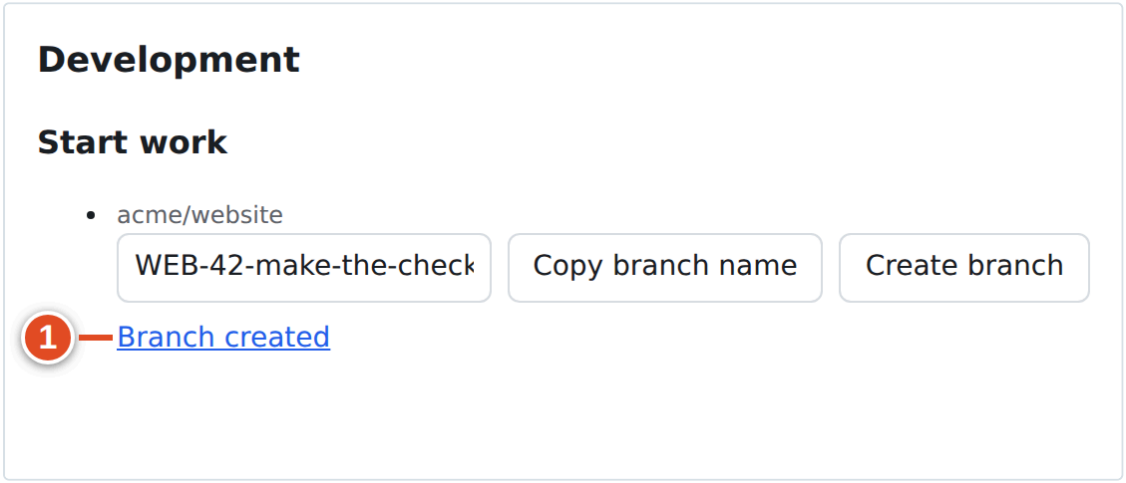


Figure 15.5 Screenshot (demo instance): After Create branch: (1) the confirmation with the new branch.

You do (terminal): fetch the new remote branch and switch to it. **Create branch** does not touch your laptop.

```
git fetch origin
git switch WEB-42-make-the-checkout-button-visible-on-small-screens
```

> **Tip:** The key at the start of the branch name is a **closing** reference: a merge of a PR from this branch can finish WEB-42. If your PR only does part of the work, use a branch name without the key (Variant B below).

Step 4 — Commit and push, with Smart Commits

You do: make the change, run your checks, then commit. Alex made three commits. The first explains the change. The second and third carry Smart Commit commands, with the key first and the command after it on the same line:

```
git add src/checkout/footer.css src/checkout/footer.test.ts
git commit -m "WEB-42 Move the Place order button into a sticky footer" \
           -m "Keeps the button inside the viewport on phones."
git commit --allow-empty -m "WEB-42 #comment Checked at 320, 375 and 390 px
widths; the button stays visible."
git commit --allow-empty -m "WEB-42 #time 1h 30m"
git push -u origin WEB-42-make-the-checkout-button-visible-on-small-screens
```

(In practice you would put the commands into real commits. `--allow-empty` here just keeps the example short.)

git-sync does: GitHub sends one `push` webhook for the whole push.

- **Branch pushed.** The first push of a branch whose name contains `WEB-42` triggers the default rule `branch.created` → **In Progress**. WEB-42 is already In Progress, so nothing visibly changes.
- **Smart Commits.** Each commit is read once, by its SHA. The first commit has no `#command`, so nothing runs for it. The second adds a comment. The third logs time **as a comment**, because Plane CE has no worklog. These are the exact comments the demo run wrote on WEB-42:

```
<p>Checked at 320, 375 and 390 px widths; the button stays visible.</p><p>
<small>via <a
href="https://github.com/acme/website/commit/3b2335d710bf1b0ba856a9ca334a2fd
2e8a27ac6">3b2335d</a> by alex-dev</small></p>
<p>Time logged: 1h 30m (90 minutes)</p><p><small>via <a
href="https://github.com/acme/website/commit/7b7ebfdd2c7657165f8c339cbace6ce
f32e140b6">7b7ebfd</a> by alex-dev</small></p>
```

Text git-sync wrote in Plane in the demo run.

The “via … by alex-dev” line is the **provenance**. `alex-dev` is the GitHub account that pushed (the webhook’s `sender`), not the author name inside the commit, which anyone can fake. In Plane the comment itself shows as written by the git-sync service account.

You see: Branches in the Development panel lists the branch, and the two comments appear in the item's activity.

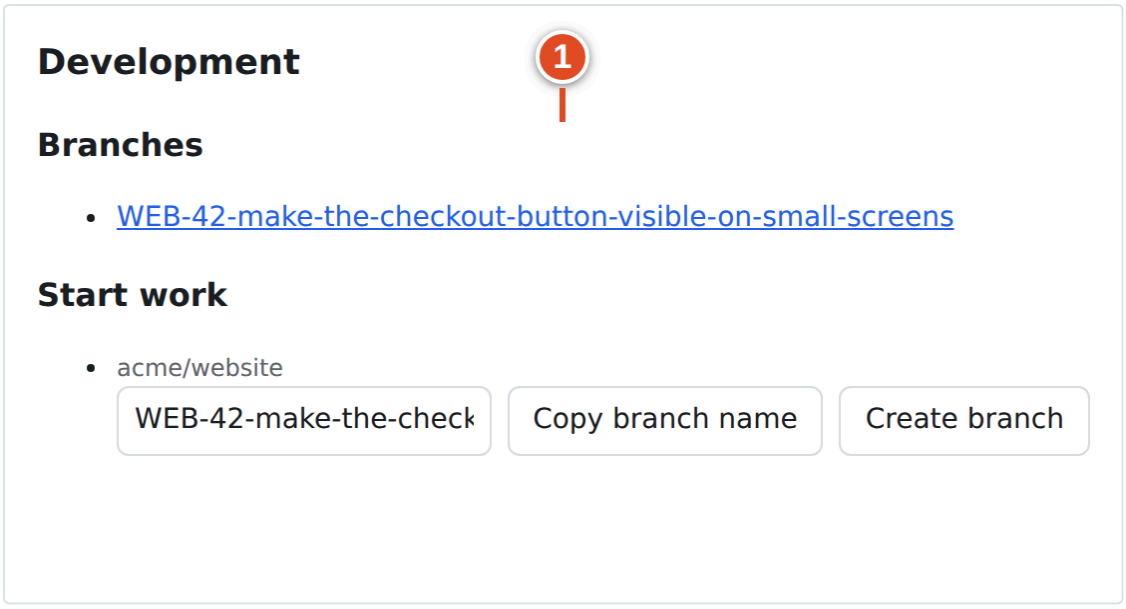


Figure 15.6 Screenshot (demo instance): After the first push: (1) Branches lists the branch.

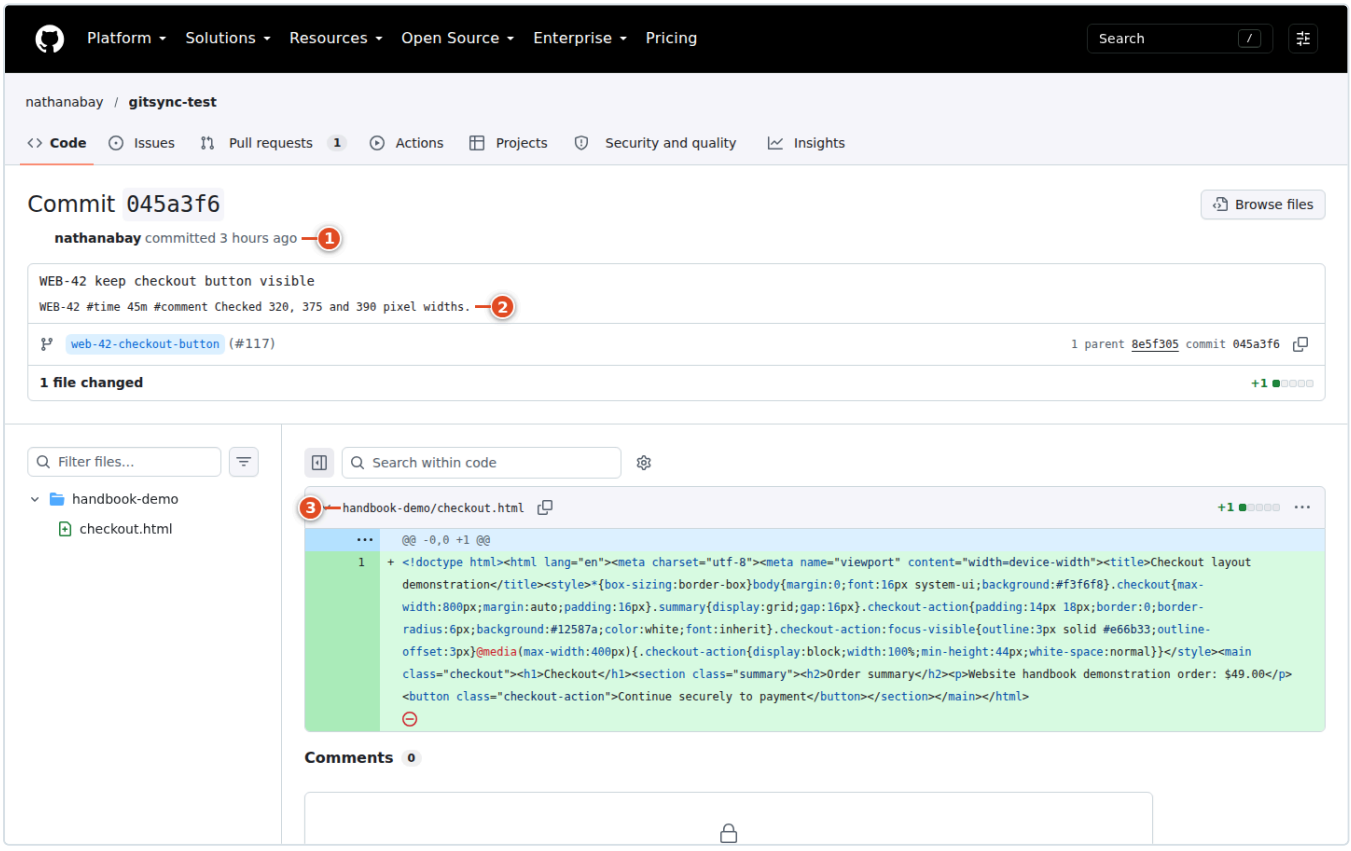


Figure 15.7 Screenshot (GitHub): A real Smart Commit on the test repository (from the earlier test, not this run): (1) author and commit time; (2) Smart Commit message; (3) changed file.

> **Screenshot needed** — plane-02-smart-comment — The two Smart Commit comments ("Checked at 320, 375 and 390 px…" and "Time logged: 1h 30m (90 minutes)") in WEB-42's activity in Plane. Capture later with npm run manual:capture -- --only plane-02-smart-comment

You do: open a **draft** PR from the branch into `main`. With the GitHub CLI:

```
gh pr create --draft --base main \
  --title "WEB-42 Keep the checkout button visible on small screens" \
  --body "Closes WEB-42

The **Place order** button now lives in a sticky footer on narrow screens.

- [x] 320 px
- [x] 375 px
- [x] 390 px"
```

git-sync does: reads the branch name, title and body. All three name `WEB-42` as **closing** (branch key, title key and `Closes`), and the strongest tier wins. It then:

- stores the link and runs the `pull_request.drafted` rule → **In Progress** (no visible change);
- adds a `[PR]` rollup link to the item: `#121 WEB-42 Keep the checkout button visible on small screens · OPEN · REVIEW`;
- because link-back is on, posts this comment on the PR:

```
Linked Plane work item:
- WEB-42: http://plane.example.com/acme/projects/bb57e81d-c161-441f-b50e-907af28f08c2/issues/53486230-16eb-4c87-afc4-ffeb2339a1a9
```

Text git-sync posted to GitHub in the demo run.

You see: Pull requests in Development, with **PRs: OPEN** as the roll-up and the PR's own `OPEN` badge. The review part shows `REVIEW` because nobody has decided yet.

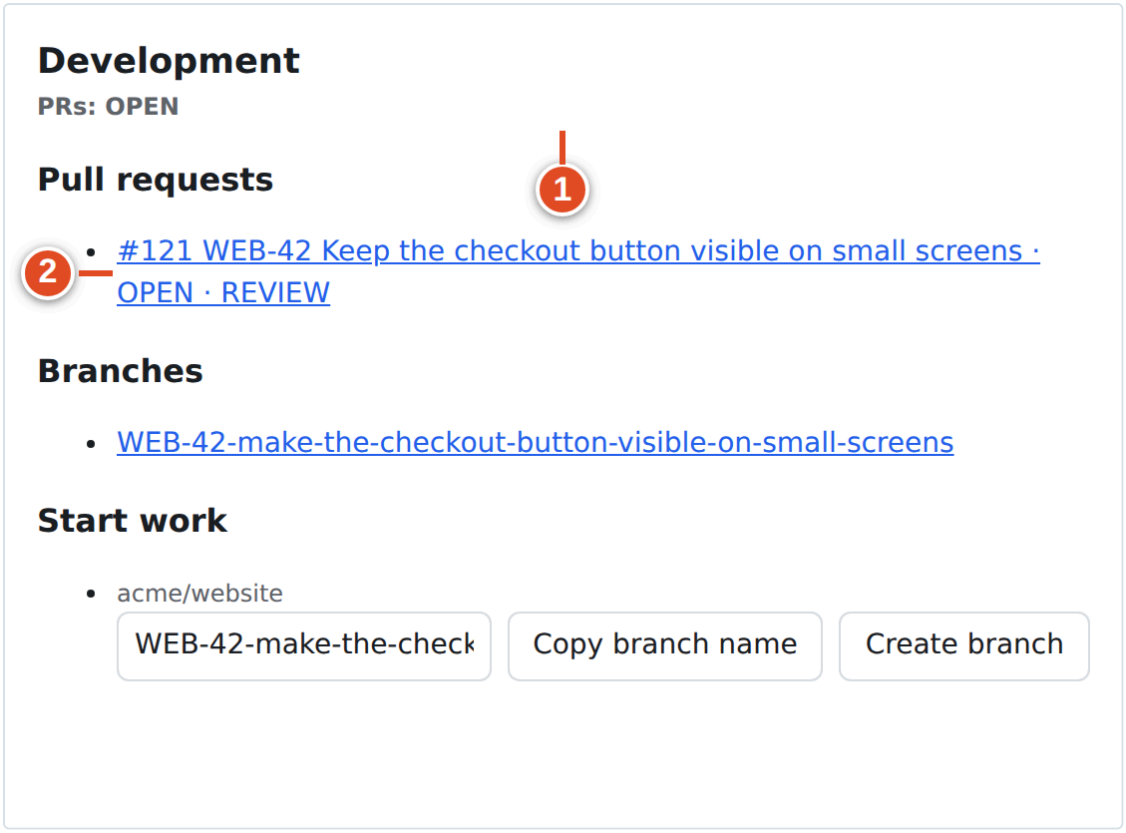


Figure 15.8 Screenshot (demo instance): Draft PR opened: (1) PR roll-up status; (2) the PR with its OPEN badge.

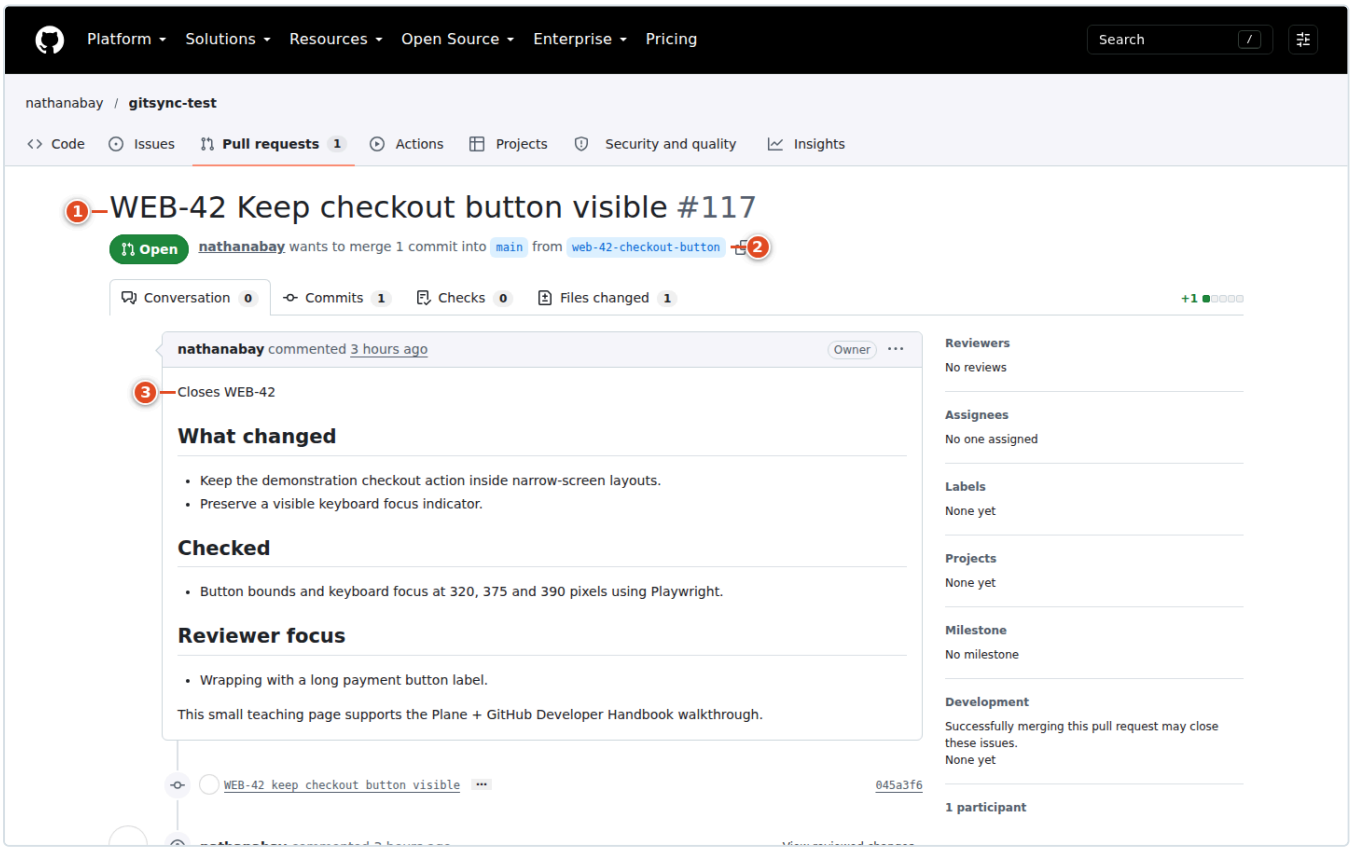


Figure 15.9 Screenshot (GitHub): A real PR on the test repository (PR #117 for WEB-42 from the earlier test, not this run): (1) title; (2) feature branch targeting main; (3) Closes WEB-42.

Step 6 — CI runs and a preview deploys

You do: nothing. Your CI (GitHub Actions in the demo) starts on the push, and your hosting provider (Vercel in the demo) deploys a preview.

git-sync does: `check_suite` events update a `[CI]` link: **pending** first, then **pass** (a **fail** looks like WEB-37 in Chapter 12). A `deployment_status` event whose URL ends in `.vercel.app`, `.netlify.app`, `.pages.dev` or `.amplifyapp.com` adds both a `[Deploy]` link and a `[Preview]` link. The demo run kept exactly these links on WEB-42:

```
[CI] Build · pass          ->
https://github.com/acme/website/pull/121/checks
[Deploy] Preview · success -> https://website-git-web-42-acme.vercel.app
[Preview] Vercel           -> https://website-git-web-42-acme.vercel.app
```

Links git-sync kept on WEB-42 in the demo run.

The rollup links are *updated in place*. The pending build becomes the passing build; no second link is added.

You see: Builds shows pending, then pass. **Deployments** and **Previews** appear.

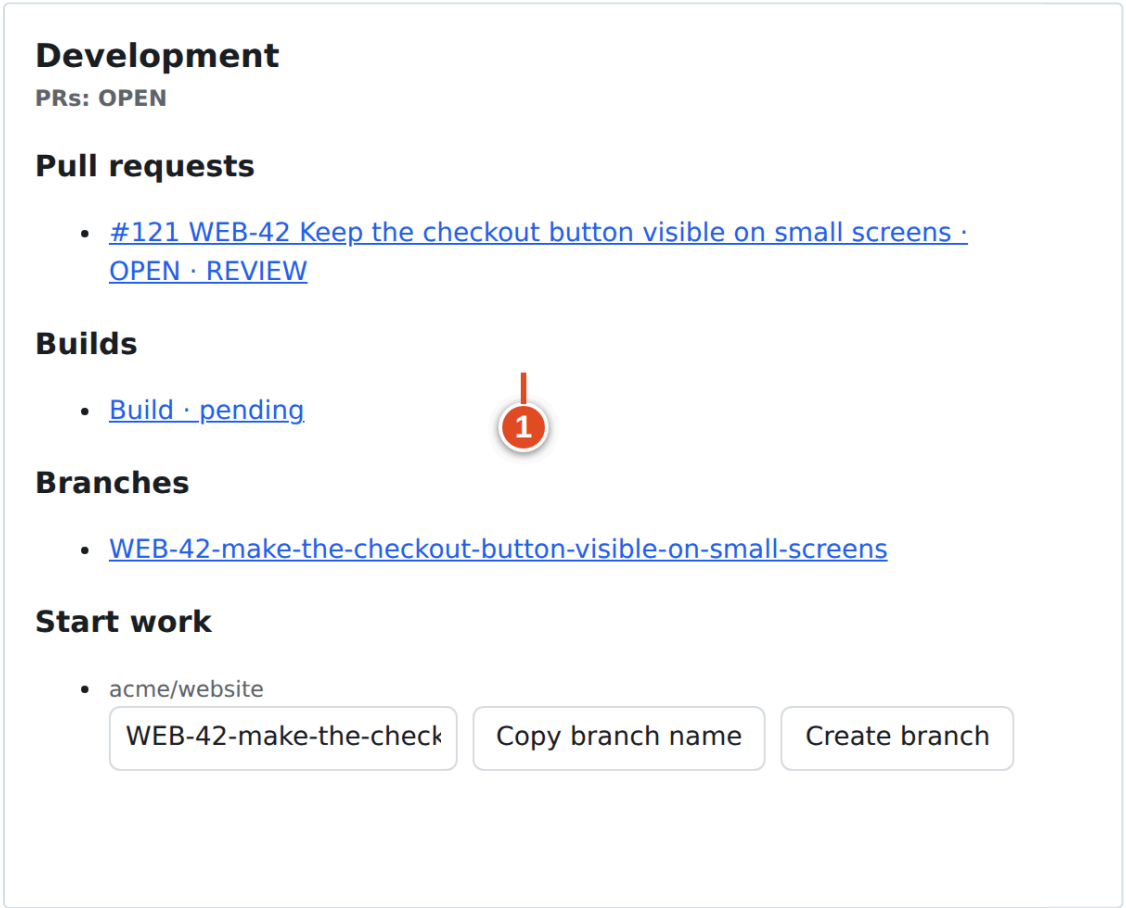


Figure 15.10 Screenshot (demo instance): CI started: (1) Builds shows pending.

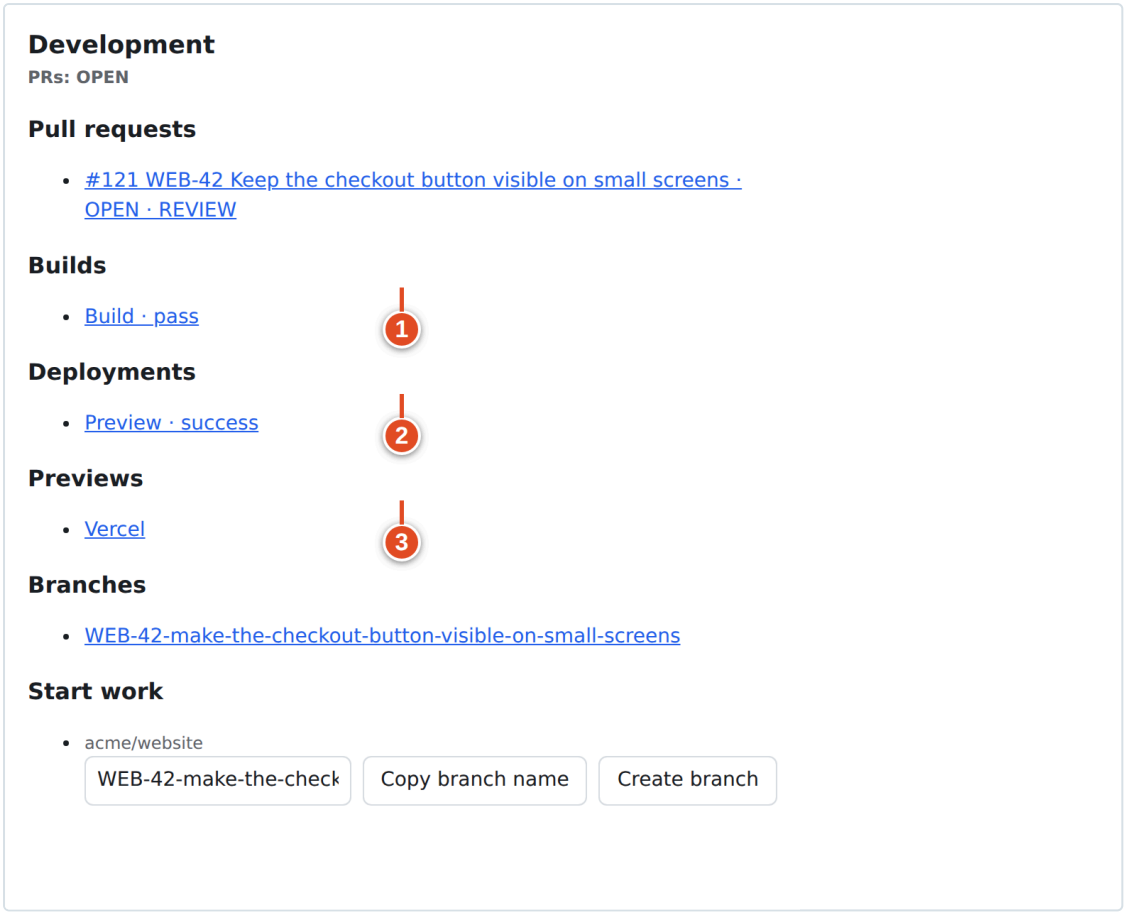


Figure 15.11 Screenshot (demo instance): CI passed and a preview deployed: (1) Builds pass; (2) Deployments; (3) Previews.

> **Tip:** Open the preview link from the work item to check the change on a real phone before asking for review. Product owners can do the same without opening GitHub.

Step 7 — Mark ready and request a review

You do: mark the PR ready and ask Sam to review:

```
gh pr ready 121
gh pr edit 121 --add-reviewer sam-reviews
```

git-sync does: *Ready for review* counts as `pull_request.opened` → **In Progress** (no change). *Review requested* runs the default rule `pull_request.review_requested` → **In Review**.

You see: WEB-42 moves to **In Review**. The PR still shows `OPEN · REVIEW`.

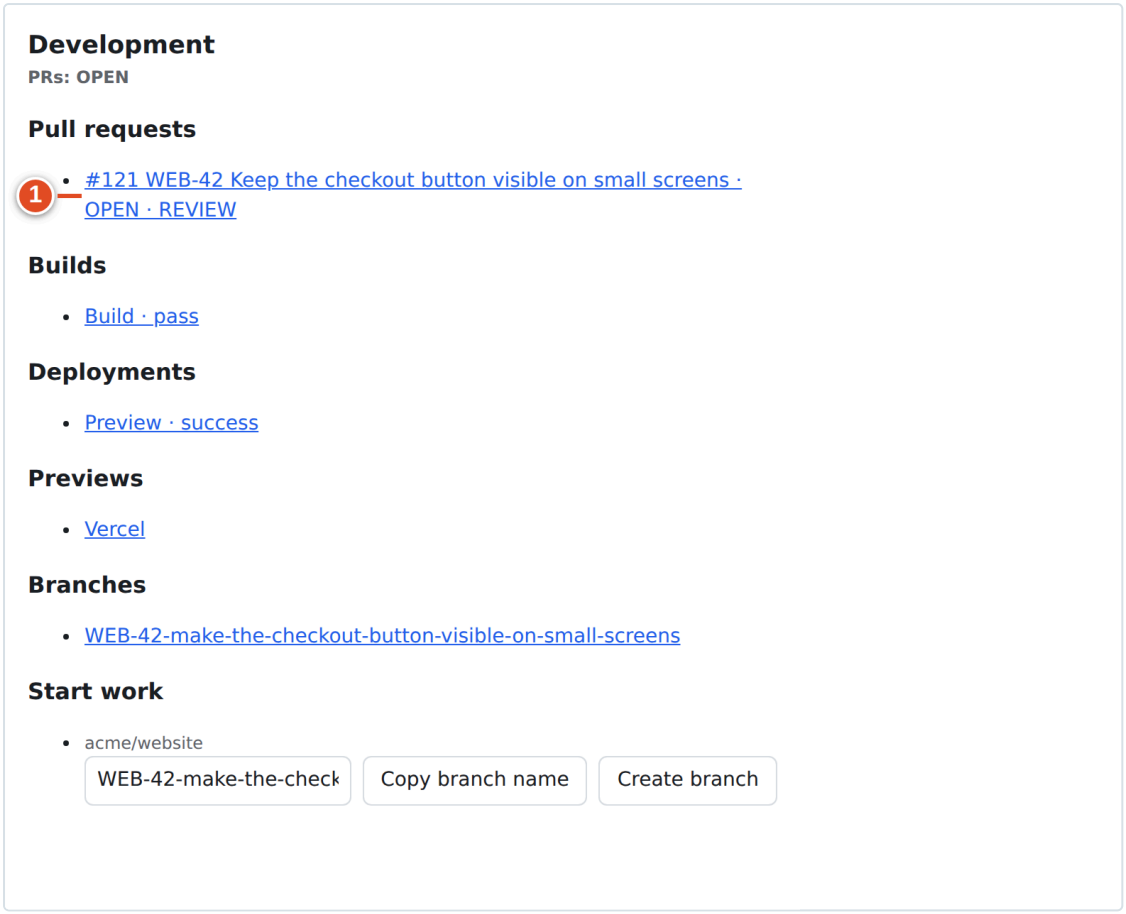


Figure 15.12 Screenshot (demo instance): Ready for review and a reviewer requested: (1) review badge.

> **Note:** Opening a PR is not the same as requesting a review. The item moves to In Review only when a reviewer is actually requested.

Step 8 — The reviewer asks for changes

You do (Sam): reviews on GitHub and chooses **Request changes**: *"Please keep the focus ring visible on the button."*

git-sync does: the `pull_request_review` event becomes `pull_request.changes_requested`. The shipped rules have no target for it, so the state stays **In Review**. The PR rollup link is rewritten to `OPEN · REJECTED`.

You see: the review badge changes to **REJECTED**. This means *a reviewer currently wants changes*. It does not mean the PR or the task was rejected.

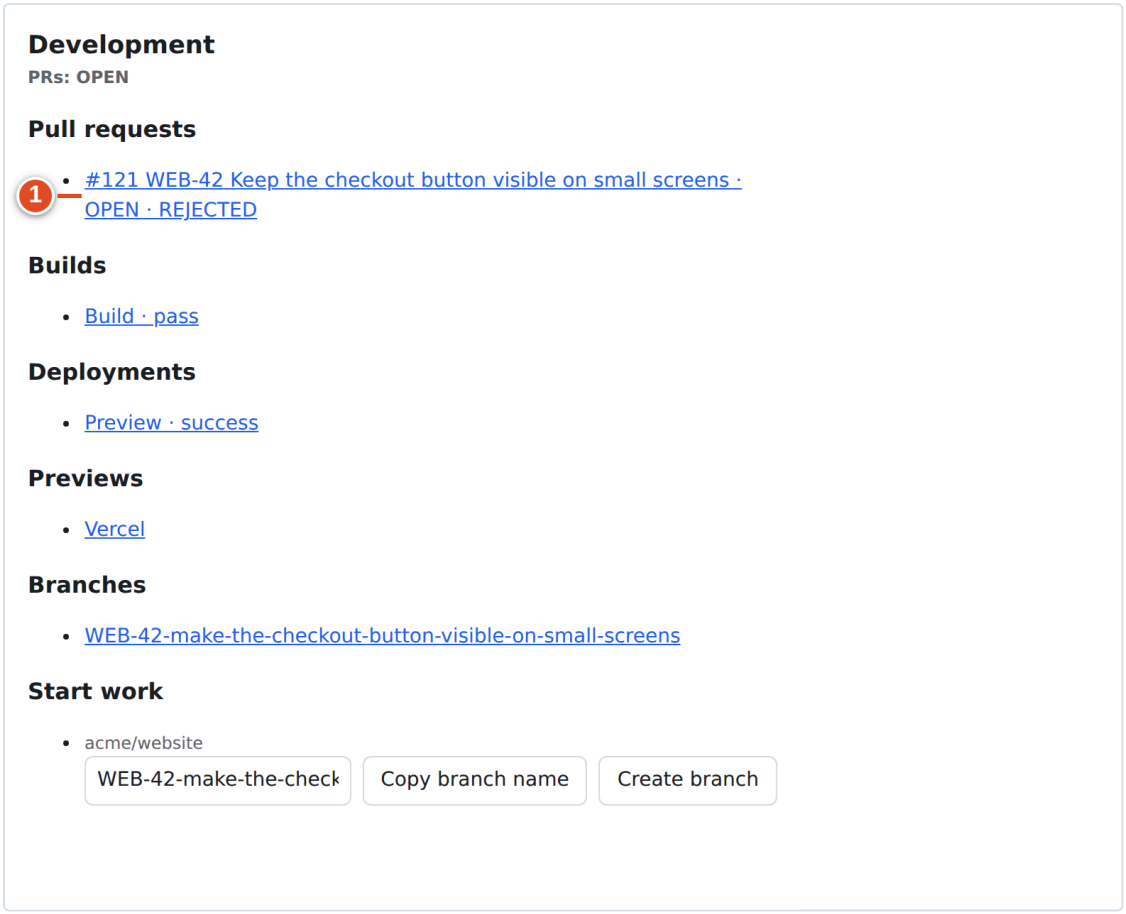


Figure 15.13 Screenshot (demo instance): The reviewer asked for changes: (1) the review badge changes.

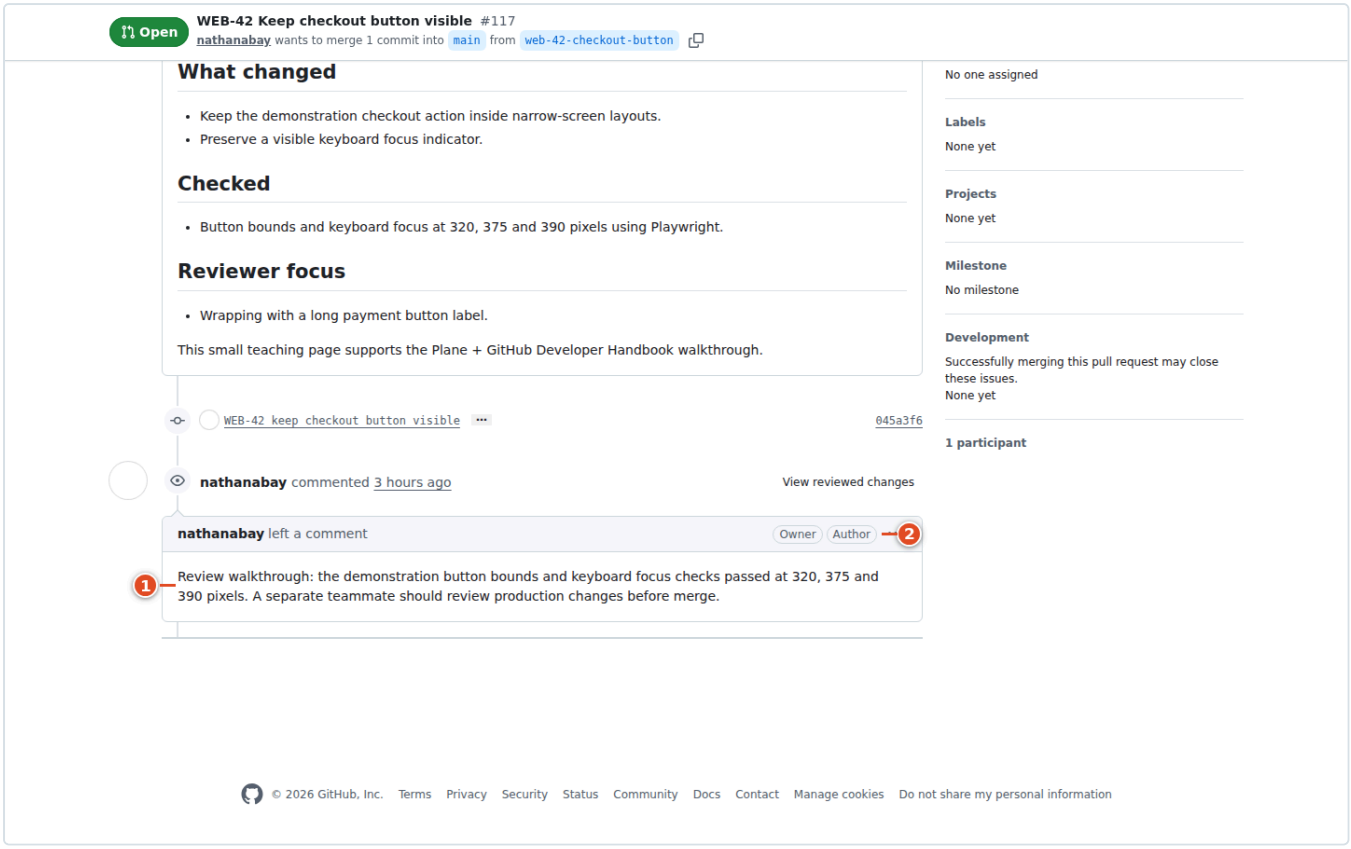


Figure 15.14 Screenshot (GitHub): A real review on the test repository (from the earlier test; it is a COMMENTED walkthrough review, not an approval): (1) review text; (2) reviewer identity.

Step 9 — Fix, push, get approved

You do: fix the focus ring and push. You can answer the review in Plane at the same time:

```
git commit -am "WEB-42 Keep the focus ring visible on the sticky button
#comment Addressed review: focus ring restored."
git push
```

git-sync does:

- adds the Smart Commit comment *"Addressed review: focus ring restored."* (with provenance) to WEB-42;
- GitHub sends `pull_request` `synchronize` for the new head commit; CI runs again and passes.

You do (Sam): Approve.

git-sync does: checks readiness against GitHub's current data: open, not draft, approved, no standing change request (Sam's approval replaced Sam's own change request), and checks green. Everything holds, so it fires **ready for merge** once for this head commit (audit: `ready_for_merge fired`). The shipped rules do nothing at that point, so the state stays **In Review**. The rollup becomes `OPEN · APPROVAL`.

You see: the review badge reads **APPROVAL**.

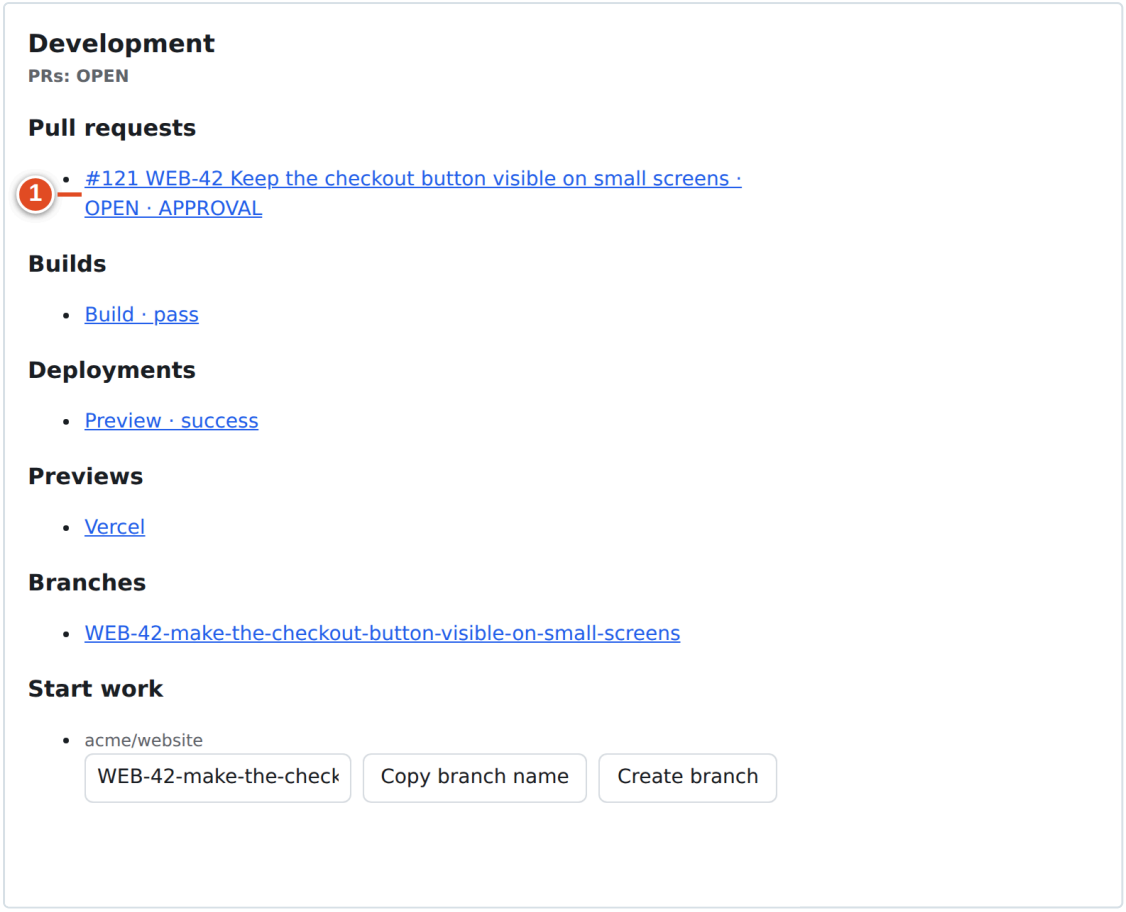


Figure 15.15 Screenshot (demo instance): Approved with passing checks: (1) review badge.

How the review badge changed during steps 7–9:

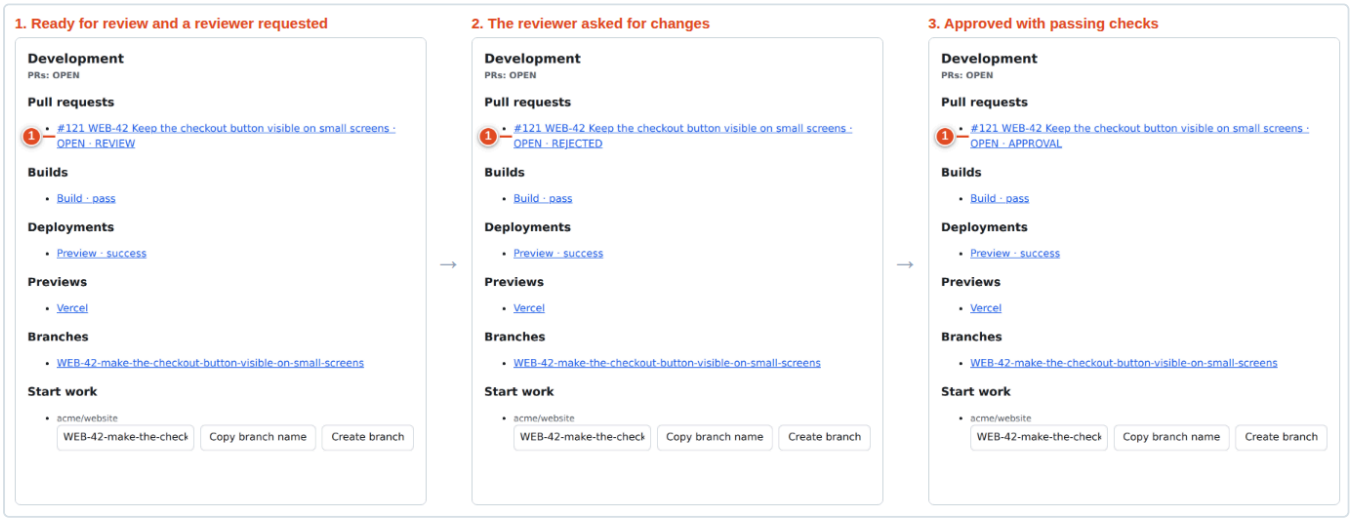


Figure 15.16 Strip (demo instance): Review badges on the PR line: REVIEW → REJECTED → APPROVAL.

Step 10 — Merge and verify Done

You do: merge the PR into `main` with your repository's merge method (button, or `gh pr merge 121`).

git-sync does: `pull_request` `closed` with `merged: true` becomes `pull_request.merged`. Before running the rule, the **merge guard** checks every PR linked to WEB-42. Only #121 is linked and it is merged, so the guard passes. The first matching rule is `default: merged` (base `main` matches `^main$`) → **Done**. The rollup becomes `MERGED`.

You see: WEB-42 is **Done**, and the Development roll-up reads **PRs: MERGED**.

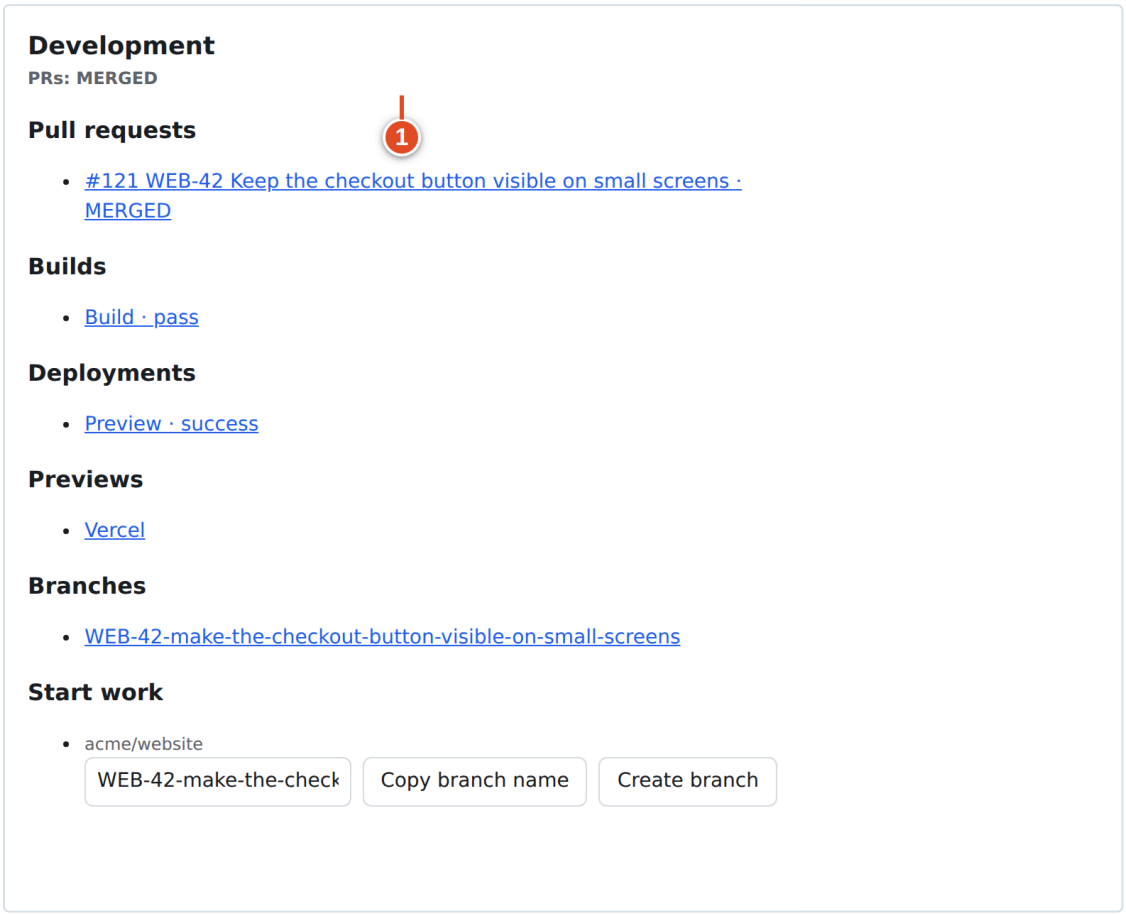


Figure 15.17 Screenshot (demo instance): Merged: (1) PR status MERGED; the work item moved to Done.

> **Screenshot needed** — plane-04-done-after-merge — WEB-42 in Done in Plane's own work-item page after the merge, with the Development section and the three Smart Commit comments. Capture later with npm run manual:capture -- --only plane-04-done-after-merge

```
git switch main
git pull --ff-only
git branch -d WEB-42-make-the-checkout-button-visible-on-small-screens
```

> **Warning:** *Merged is not deployed.* Done means “code merged into main” under the default rules. If your team defines Done as “live in production”, ask your admin to change the merged target and use a deployment rule or a manual step instead.

If it went wrong:

What you see	Likely reason	What to do
Stays In Review after merge	Another linked PR is still open (merge guard), or the PR was merged into a base other than `main`	Check **Pull requests** in Development, and the PR’s base branch
Stays In Progress, never In Review	No reviewer was requested, or the project has no *In Review* state	Request a reviewer; ask your admin about state names
No PR in Development	The key is missing from the branch, title and body, or is misspelled (`WEB_42`)	Fix the PR title, or ask your admin to **Replay** the delivery
No comment from `#comment`	The key came after the command, or Smart Commits is off	Keys first, same line (Chapter 11)

Variant A — A bug starts as a GitHub issue

A tester reports *#131 Coupon field accepts expired codes* on GitHub. The team plans in Plane.

1. On GitHub, add the label `plane` to #131.
2. git-sync creates *WEB-47* in the Website project. In the demo run it landed in **Backlog** (Plane’s project default, because no **Open state** is set on the mapping), with the label `bug` copied and Alex as the assignee (Alex’s GitHub account is linked).
3. Sam’s GitHub comment *“Reproduced on staging. @alex-dev can you take this?”* arrives in Plane as a new comment with a real mention of Alex.
4. Work on WEB-47 exactly like steps 1–10: branch `WEB-47-…`, PR `Closes WEB-47`.
5. When the PR merges, WEB-47 moves to Done. *That change does not close #131.* git-sync makes the change as its own service account, and Plane events from the service account are ignored on arrival (loop prevention). To close the GitHub issue as well, also write `Closes #131` in the PR body: GitHub closes #131 itself at merge.
6. If a person moves WEB-47 to a completed state in Plane instead, the mirror closes #131 with reason `completed`, because the State policy is **Last write wins**.

Do **not** create a second Plane item by hand while the mirror is being created.

Variant B — Split work across PRs (Refs)

WEB-42 is too big for one review: layout in one PR, keyboard support in another.

- Use branch names **without** the key: `checkout-layout-part-one`, `checkout-keyboard`.
- Put `Refs WEB-42` in each PR's title and body. Do not use a bare key in the title, because a bare title key is closing.
- **Effect:** each PR links to WEB-42 and can move it to In Progress or In Review, but *no merge moves it to Done*. That is what happened to WEB-41 in the demo (`Refs WEB-41`: linked, moved to In Progress on open).
- When the last part merges, move WEB-42 to Done yourself. Alternatively, write `Closes WEB-42` only in the final PR. While any earlier part is still open, the merge guard keeps the item open.

A branch such as `refs/web-42/layout` still contains a bare key and is **closing**. The slash is not the whitespace a magic word needs.

> **Example:** If the parts are separate sub-work items, close each child and refer to the parent: `Closes WEB-43` and `Refs WEB-42` in the same PR body.

Variant C — One PR finishes two items

```
Closes WEB-42, WEB-45
```

One magic word can take several keys separated by `,`, `and` or `&`. Each item is checked separately: WEB-45 might stay open because it has another PR still open. Verify both items after the merge.

Variant D — Reverting a merged change

A revert is new code and new events. git-sync has no "undo everything the old PR did" operation.

1. Create a work item that explains *why* the revert is needed, for example `WEB-51`.
2. Create the revert as usual. GitHub's **Revert** button creates a branch such as `revert-121-WEB-42-make-the-checkout-button-visible-on-small-screens` and pre-fills a PR form. Branches starting with `revert-` never trigger *Branch pushed*, so the old item is not restarted by the push alone.

3. *Before you submit the PR form*, edit its title and body. The pre-filled title `Revert "WEB-42 Keep…"` and the branch name both contain a bare `WEB-42`, and both are **closing** references when the PR is parsed. Write `Closes WEB-51` and, if the old item must not be linked at all, `skip WEB-42` in the body. `skip` removes that key from every source of the PR (branch, title and body). `Relates to WEB-42` alone is not enough here, because the branch name keeps WEB-42 a closing reference. Use `skip WEB-42`, or make the revert on a branch whose name has no key.
4. After merging, reopen WEB-42 yourself if its outcome is no longer delivered.

`skip WEB-42` only stops new linking. It does not remove a link that already exists. Smart Commit commands copied into the revert's commit messages run again, because a revert commit has a new SHA. Check for `#time` or `#done` before you push.

Variant E — PR first, work item later (WEB-NEW)

Sometimes the code exists before anyone wrote a task. Put `WEB-NEW` (uppercase `NEW`) in the PR title or body:

```
gh pr create --title "WEB-NEW Add rate limiting to the sign-up form" \
  --body "Limits sign-up attempts to 5 per minute per IP address."
```

git-sync creates a new work item in the project whose identifier is `WEB`. The item takes the PR's title and a **started**-group state, and is linked to the PR as closing. In the demo run this created *WEB-46*, already In Progress, with the PR in its Development panel. It runs on opened, edited and reopened events, and only once per PR.

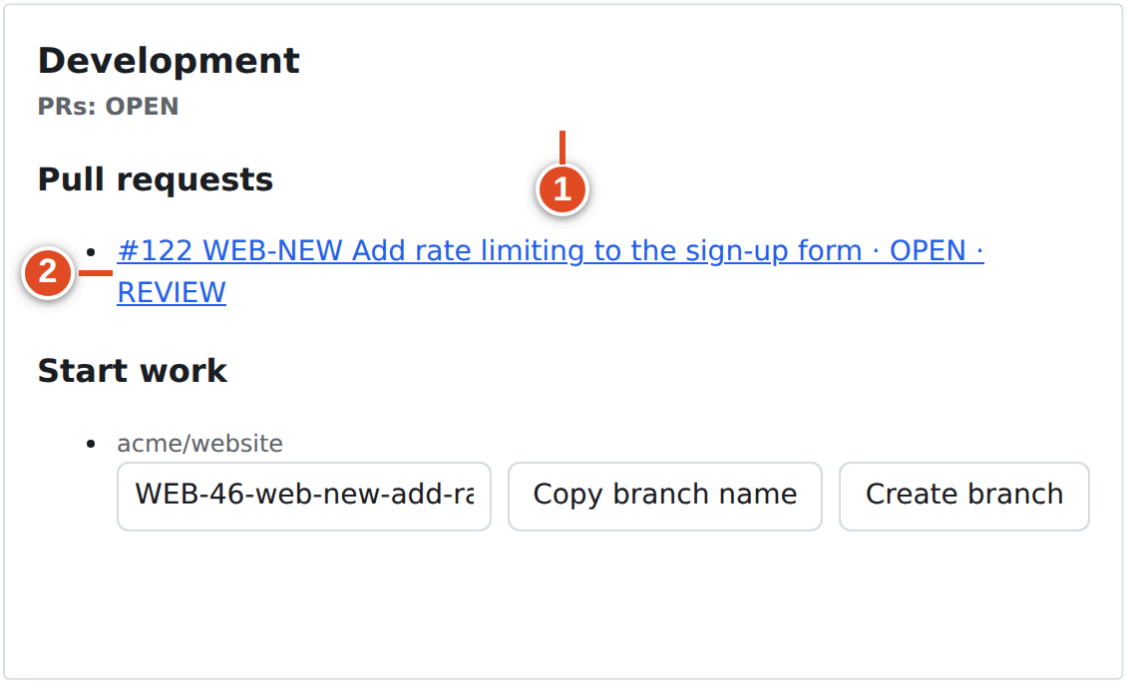


Figure 15.18 Screenshot (demo instance): A work item created from a WEB-NEW pull request: (1) title taken from the PR; (2) the PR link.

> **Note:** In this release, the link-back comment for an item created through `WEB-NEW` shows `-WEB:` instead of the new key (`WEB-46`). The link URL is correct. This is a known defect, listed in Appendix E.

Rename the item in Plane afterwards if `WEB-NEW` in its title bothers you. The title is not re-synced from the PR.

What you learned

- The full loop, as verified on the demo instance: branch → In Progress; draft and open → In Progress; reviewer requested → In Review; changes requested → `REJECTED` badge; approval → `APPROVAL` and ready for merge; merge into `main` → **Done**.
- **Create branch** creates the branch on GitHub only. Fetch it locally before you work.
- Smart Commits add comments and time notes with “via <sha> by <login>” provenance. Time is a comment, not a worklog.
- Use branches without the key plus `Refs` for partial work, and `Closes A, B` for one PR that finishes two items. Edit revert PR titles before opening them, and use `WEB-NEW` for PR-first work.
- Merged is not the same as deployed. Agree on what Done means in your team.

Source files

- `manual/demo/events.ts`: the exact events of this chapter’s run; `manual/demo/run-loop.sh` replays them
- `src/ui/dev-panel.ts`, `src/ui/client/DevPanel.tsx`: Development panel and **Create branch**
- `src/app/push.ts`, `src/github/smart-commits.ts`: pushes and Smart Commits
- `src/app/pr.ts`, `src/github/pr-automation.ts`: PR lifecycle, ready for merge, merge guard, `WEB-NEW`
- `src/app/checks.ts`, `src/sync/dev-info.ts`: builds, deployments, previews and rollup links
- `src/app/issues.ts`, `src/sync/mirror.ts`: Variant A mirroring

16 · Team conventions

Everything in this chapter is a **suggested team convention**. None of it is enforced by git-sync. These are habits that make the automation predictable and the history readable. Agree on a short list with your team, write it on a Plane **Page** (the demo project keeps *Website working agreement* for this), and review it when something surprises you.

Work items

- **Write an outcome, not a topic.** “Keep the checkout button visible on small screens” can be finished; “Checkout improvements” cannot.
- **Put acceptance checks in the description.** Say what proves the work is done, for example: *Button visible at 320, 375 and 390 px; focus indicator visible.* Keep the description current. Use comments for the conversation.
- **One owner per item.** Several people can help, but one person moves the item forward and asks for review.
- **For bugs:** steps to reproduce, actual result and expected result. Add a screenshot if it helps, but write enough text that the item makes sense without it.
- *Never paste secrets* (tokens, passwords, customer data) into items, comments or commit messages. Mirrored issues copy text to GitHub.

Branch names

Situation	Convention	Why
The branch finishes one item	Use **Create branch** or **Copy branch name** (`WEB-42-make-the-checkout-…`)	The key in the branch is a closing reference, and everyone’s names look the same
The branch is part of a bigger item	No key in the name: `checkout-layout-part-one`	Otherwise the branch key makes the PR closing
Revert	Keep GitHub’s `revert-…` name, but edit the PR text (Chapter 15, Variant D)	`revert-` branches never restart work by themselves
Spelling	`WEB-42` , with a hyphen and no underscore after the number	`WEB_42` and `WEB-42_fix` are not recognized

PR titles and bodies

Decide *before you open the PR* what it should do to the item, because the first parse creates the link:

Intent	Write	Avoid
Finish the item	`WEB-42 …` in the title and `Closes WEB-42` in the body	—
Contribute part	`Refs WEB-42` in the title and body, branch without the key	A bare `WEB-42` in the title or branch
Mention as context	`Relates to WEB-42` in the body	Any bare key in the title or branch
Copied template mentions a key	`skip WEB-42` in the body	Removing it later and expecting the link to vanish
No item yet	`WEB-NEW …` in the title	Creating the Plane item by hand at the same time

A useful PR body has four short parts: **purpose** (with the Plane reference first), *what changed*, *how it was checked*, and *what the reviewer should look at*.

Commits and Smart Commits

- Write a normal summary line first. Put commands in a separate paragraph or a separate commit.
- **Key first, then commands, on the same line**: `WEB-42 #in-review #time 30m #comment Ready for review.`
- **Put `#comment` last.** It swallows the rest of the line.
- *Use full state names* in hyphenated form (`#in-review`, not `#in` or `#review`).
- **Agree what `#time` means.** It becomes a comment ("Time logged: …"), not a timesheet entry. `1d` = 8 hours and `1w` = 5 days.
- **Avoid `#done` in intermediate commits.** It moves the item the moment the commit is pushed, before review or merge.
- **Do not rebase or amend pushed commits that carry commands.** A new SHA runs the commands again (two time notes, two comments).

Review and merge

- **Request a reviewer explicitly.** That is the event that moves the item to In Review.
- **Answer review feedback in the PR.** Add a one-line summary to the work item for people who do not read code (a `#comment` in the fix commit is enough).
- **Close obsolete PRs before the final merge.** An open linked PR blocks the merge guard, and git-sync does not re-check after a later close.
- **Agree on what Done means.** By default Done means "merged into `main`". If your team needs "deployed" or "verified in production", ask the admin to change the rules. Do not move items back and forth by hand.

Mirrored issues

- Decide *which side owns titles and descriptions*. The default is GitHub. Edit there.
- **Do not delete or archive items to close issues.** Move them to a completed or cancelled state.
- *Corrections are new comments*, because edited comments are not mirrored.
- Use the `plane` label for issues the team will actually plan, not for every question.

A team page template

Copy this into a Plane page and fill it in:

Topic	Our decision
Branch name	`{key}-{title}` from **Create branch** / no key for partial work
Review state	In Review = reviewer requested
Merge base(s) and meaning	`main` → Done; `staging` → In QA
Meaning of Done	
Time logging (`#time`)?	yes / no; what it counts
Who owns titles and descriptions of mirrored issues	GitHub (default)
Where to report sync problems	channel / person (workspace admin)

What you learned

- Conventions are team policy, not software rules. Write them down and review them.
- Branch names with a key close items. Use keyless branches and `Refs` for partial work.
- Smart Commits: key first, `#comment` last, full state names, and no rewriting of pushed command commits.
- Close obsolete PRs before the final merge, and agree what Done means.

Source files

- `src/parse/references.ts`, `src/parse/magicWords.ts`: what the conventions protect against
- `src/github/smart-commits.ts`: command grammar
- `src/github/pr-automation.ts`: default lifecycle rules

17 · Troubleshooting

Most “sync is broken” moments come from one of four causes:

1. **The key was not recognised.** It is misspelled, in the wrong place, or the item cannot be found.
2. **Nothing connects the repository to the project.** There is no mapping, or the App has no access.
3. **The rules did not allow the change.** The tier, the merge guard, the base branch or a missing state blocked it.
4. *The event has not been processed yet,* or it never arrived.

Work from what you can see towards what only an admin can see. Do not recreate items or push “wake-up” commits. Each new commit is a new event and can repeat Smart Commit commands.

> **Tip:** Ask three questions in order: *Is the code linked?* (look at **Development**) → *Did the event arrive?* (ask your admin for the audit log) → *Was the change allowed?* (tier, rules, merge guard).

Linking

Symptom	Likely cause	Fix
No PR in the work item's **Development** panel	The key is missing from the branch, PR title and PR body, or is misspelled	Copy the key from Plane, then edit the PR title or body. An edit triggers a new parse.
A key only in commit messages, but no PR link	Commit messages never create PR links	Put the key in the branch, title or body
`web-42` in the PR body does not link	Bare keys in the title and body must be uppercase. Only branch names are case-insensitive.	Write `WEB-42`, or use a magic phrase (`Refs web-42` works)
`WEB-42_fix` or `WEB_42` in a branch does not link	An underscore breaks the key	Use `WEB-42-fix`
`Closes:WEB-42` does not link	A space is required after the magic word (the colon is optional)	`Closes WEB-42` or `Closes: WEB-42`
A valid key still does nothing	The item does not exist, or git-sync's service account cannot see that project	Check you can open it; tell your admin (audit `reference_unresolved`)
`skip WEB-42` did not remove the link	`skip` only prevents *new* links	Ask your admin; there is no self-service unlink
No "Linked Plane work item" comment on the PR	**Post link-back comments on GitHub** is off, or the write is still queued	Check **Development** in Plane first
Development says **No GitHub repository is mapped to this project.**	The project has no mapping	Ask a workspace admin to map the repository (Chapter 24)

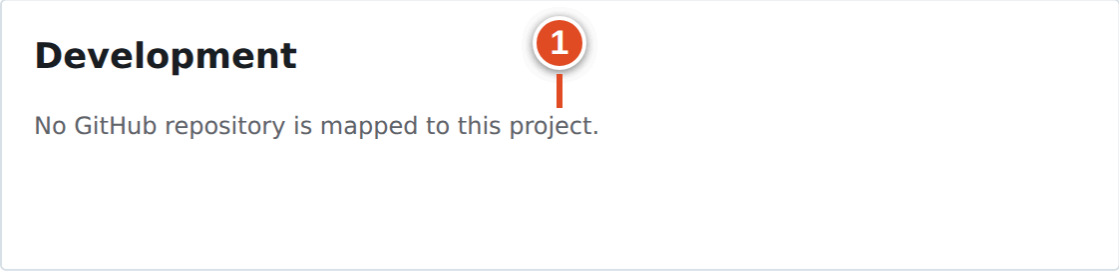


Figure 17.1 Screenshot (demo instance): A project with no mapped repository: (1) the Development panel says so.

State changes

Symptom	Likely cause	Fix
PR is linked, but opening it did not change the state	The reference is a **relation** (bare key in the body, `Relates to`), or the rule targets a state your project lacks	Use the tier you mean (Chapter 9); ask your admin to check the rule
Item moved to Done although the PR said `Refs`	A bare key in the **branch** or **title** made it closing, and the strongest tier wins	Partial work: no key in the branch or title
Merge did not move the item to Done	Another linked PR is still open (merge guard); the base is not `main`; or the reference is non-closing or relation	Check **Pull requests** in Development, the base branch and the wording
The other PR was later closed, but the item is still not Done	The guard is only checked at the merge event	Move the item yourself, or ask your admin to **Replay** the merge delivery
Approval did nothing	No default rule for approval or ready for merge	Expected. Merge to finish.
Never moved to In Review	No reviewer was requested, or there is no *In Review* state	Request a reviewer; ask your admin about state names
Item moved *back* after a merge	It did not: old events are ignored if the PR has changed since	Look for a manual change or a Smart Commit `#…` in a later push
A new branch did not reopen a Done item	By design, *Branch pushed* never moves completed or cancelled items	Reopen it yourself if the work really restarts
Revert PR closed or linked the old item	Its branch or title contains the old key	Use `skip WEB-42` in the body before submitting (Chapter 15, Variant D)

Smart Commits

When **Report problems as comments on GitHub and Plane** is on, git-sync tells you what went wrong. It posts a comment on the commit in GitHub, and on the work item in Plane when the key is known. These are the exact texts from the demo run:

```
Git sync could not complete this action:

- ` WEB-35: #shipped matches no Plane state. `
- ` WEB-35: #time needs a duration such as 1h 30m. `
```

```
Git sync could not complete this action:

- ` Unknown work item WEB-999. `
```

Text git-sync posted to GitHub in the demo run.

Symptom	Likely cause	Fix
Nothing happened after `git commit`	Commands run only after a **push**	Push
Nothing happened after a push	Smart Commits are off for this mapping; commits must be signature-verified (`REQUIRE_VERIFIED_COMMITS`); or the key came after the command	Keys first, same line; ask your admin about the mapping and signing
"… matches no Plane state"	The state name is wrong for this project	Use the full hyphenated name: `#in-review`
"… matches several Plane states"	The token is ambiguous (`#in` matches both In Progress and In Review)	Use more words: `#in-progress`
"#time needs a duration…"	Invalid duration (`3x`, `1 hour`)	`45m`, `1h 30m`, `0.5d`, `1w 2d`
"Unknown work item WEB-999."	A typo, or a key from another workspace	Fix the key in a *new* commit (the old one never re-runs)
`#done` ended up as comment text	`#comment` swallows the rest of the line	Put `#comment` last
Commands ran twice	The commit was rebased or amended (new SHA)	Expected. Delete the extra comment in Plane.
Item moved before review	A state command (`#done`) in an intermediate commit	Leave state changes to the PR lifecycle

Symptom	Likely cause	Fix
Comment shows the git-sync account as author	All Plane-side writes use the service account in this release; the “by <login>” line names you	Expected

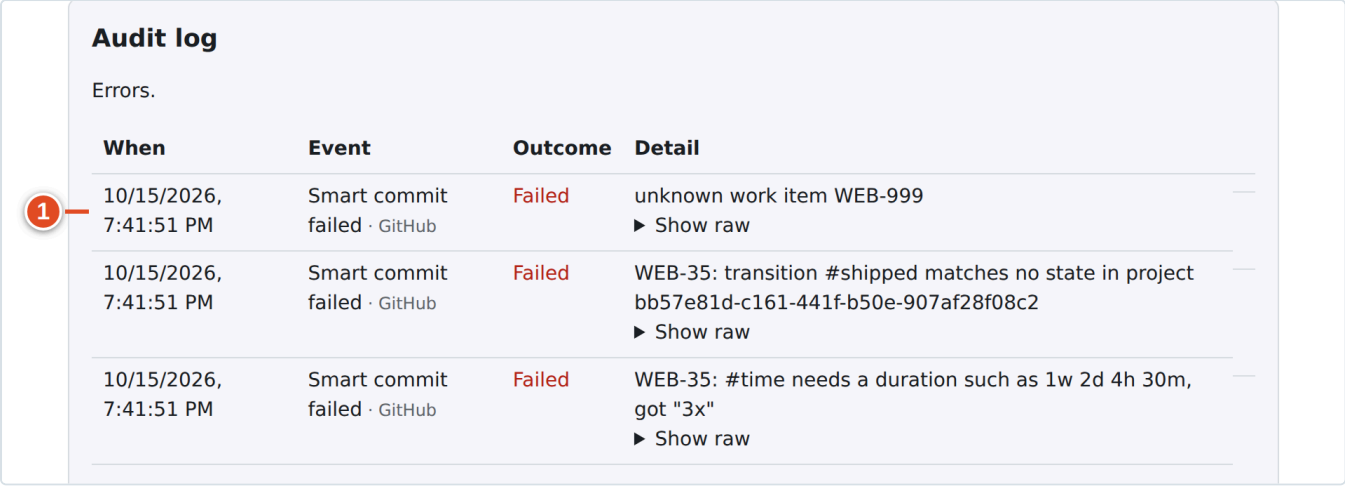


Figure 17.2 Screenshot (demo instance): Errors only (URL parameter errors=1): (1) failed Smart Commit commands.

Your admin’s view of the same failures in the audit log.

Development panel

Symptom	Likely cause	Fix
Builds stays pending	CI is still running, or the check suite never completed	Open the build link and look at GitHub Actions
Builds shows fail	A check failed	Open it, fix the failure and push
No **Previews** entry	Only deployment URLs on `.vercel.app`, `.netlify.app`, `.pages.dev` or `.amplifyapp.com` are recognized; a “Preview” link in the PR body is not	Use one of those hosts, or open the deployment link
No **Create branch** button	You are not a project member, or there is no mapping	Ask a project admin
“Branch already exists”	The branch was created before	Fetch it: `git fetch origin`
“The repository is empty; push a first commit before creating branches”	The repository has no commits	Push an initial commit
“The GitHub App cannot create branches in this repository (it needs Contents: write)”	App permission	Tell your admin
No commit list	The panel has no commits section in this release	Use the branch or PR link
“Sign in to Plane to continue”	Your Plane session expired	Sign in again

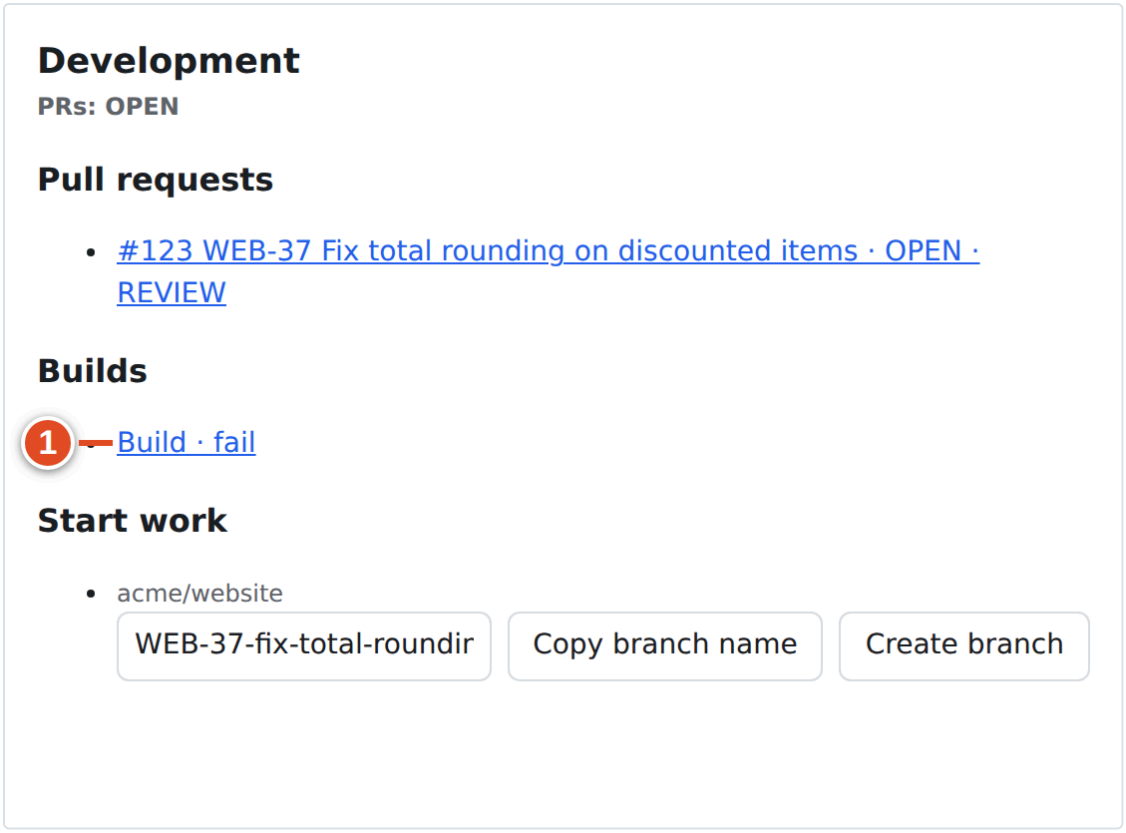


Figure 17.3 Screenshot (demo instance): A failing check: (1) Builds shows fail. Open it to see the CI log on GitHub.

Your GitHub account

Message on My GitHub account	Meaning	Fix
No personal GitHub account connected.	You have not linked yet	**Connect personal account**
Needs re-authorization. Reconnect your GitHub account to author changes as yourself.	GitHub refused the stored token (revoked, expired, App re-installed)	**Reconnect**
This connection link has expired, was already used, or belongs to another Plane account. Start again.	You waited too long, used the link twice, or were signed in to Plane as someone else	Start again from the page
GitHub authorization was declined. You can connect again when ready.	You pressed Cancel on GitHub	Connect again
This GitHub account is already linked to another member in this workspace.	Someone else linked that GitHub account	Ask your workspace admin (**User links**)
Ask the instance admin to configure the GitHub App client ID, client secret and App slug.	OAuth is not set up	Tell your instance admin
Personal account settings need workspace membership.	You are not a member of this workspace	Ask for an invitation
You cannot find **GitHub Sync** in workspace settings	That menu is for workspace admins only	Use the direct link `/git-sync/ui/workspaces/<slug>/account`

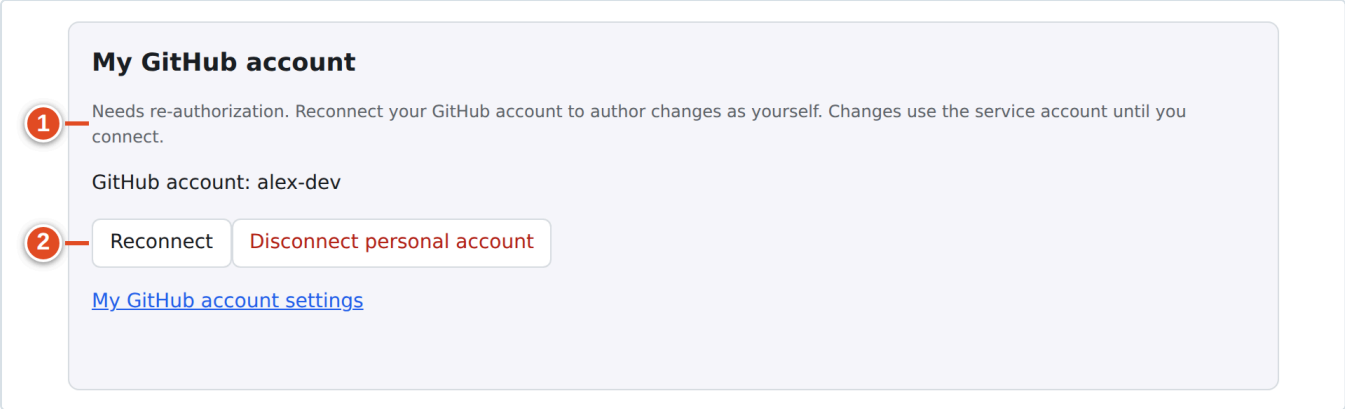


Figure 17.4 Screenshot (demo instance): GitHub refused the stored token: (1) the re-authorization notice; (2) Reconnect.

Your personal link is **not** the repository's access. Reconnecting does not give the App access to a private repository, and disconnecting does not break the team's sync.

Mirrored issues and delays

Symptom	Likely cause	Fix
GitHub issue with `plane` created nothing	The repository is not mapped, or the label is misspelled	Ask your admin
Plane item with `github` created nothing	The mapping is **GitHub to Plane only**	Expected; ask whether two-way is wanted
Title edit in Plane not sent to GitHub	Default policy: GitHub owns title and description	Edit on GitHub
A field changes on one side only	Direction or policy	Ask which side owns the field
Edited comment not updated on the other side	Only new comments are mirrored	Add a correction comment
Merge moved the item to Done, but the GitHub issue stayed open	git-sync's own changes are not mirrored back (service-user filter)	Write `Closes #N` in the PR, or close it by hand
Deleted Plane item, GitHub issue still open	Deleting is never closing; the link is only marked orphaned	Close the issue deliberately
Several minutes and nothing	Queue backlog, rate limit, outage, or a webhook that never arrived	Report it (below); your admin can **Replay** or ask GitHub to redeliver

How to send a useful report to an admin

Admins can see the audit log, deliveries, rules and health; you cannot. Give them enough to find the event in one search.

Include:

1. **The Plane key(s)**: `WEB-42`.
2. **The GitHub URL**: PR, issue or commit (`https://github.com/acme/website/pull/121`).
3. *The commit SHA*, for Smart Commit problems (`git log -1 --format=%H`).
4. **When**: the time and your **timezone** (for example *14:20 Africa/Addis_Ababa*).
5. *What you expected* and *what you saw*: one sentence each.
6. *What you already tried*, such as a new commit, a manual state change or a reconnect.
7. A **screenshot** of the Development panel or comment, with private data removed.
8. If you saw a problem comment, its exact text.

> **Example:** “WEB-36: PR #124 merged into main at 10:05 Africa/Addis_Ababa, but WEB-36 stays In Progress. Development shows #125 still OPEN. Is the merge guard blocking it, and should #125 be closed?”

Never send tokens, cookies, webhook secrets or `.env` files. An admin never needs them from you.

What your admin will do with it: find the delivery in the **Audit log** (filtered with `?errors=1`), open **Show raw**, check the mapping’s health and rules, and then **Replay** the delivery or **Retry** a failed write (Chapters 28–29).

What you learned

- Check in order: linked? → arrived? → allowed? Do not push “wake-up” commits.
- Most link problems are key spelling or placement. Most state problems are the tier, the merge guard or the base branch.
- Turn on **Report problems** to see Smart Commit errors on the commit itself.
- A good report has the key, URL, SHA, time with timezone, expected versus actual, and what you tried. It never contains secrets.

Source files

- `src/parse/references.ts`, `src/app/pr.ts`: linking and the merge guard
- `src/github/smart-commits.ts`, `src/github/problems.ts`: Smart Commit errors and problem reports
- `src/ui/client/Connect.tsx`, `src/ui/oauth-flow.ts`: account messages
- `src/ui/dev-panel.ts`: Development panel errors
- `src/queue/workers.ts`, `src/sync/mirror.ts`: delivery and mirroring

18 · FAQ and glossary

Short answers to the questions new developers ask most. Each answer points to the chapter with the details. If your workspace behaves differently, your admin's mapping and rules are the reason. Use Chapter 17 to collect evidence.

Getting started

Do I need to learn all of Plane on day one? No. Learn to find your assigned items (**Your work**), open one, read it, change its state and comment. Then connect your GitHub account (Chapter 8) and follow the daily loop (Chapter 15). Cycles, modules and pages can wait.

What is the difference between a workspace and a project? A workspace (here **Acme**, slug `acme`) holds people and projects. A project (here **Website**) holds work items and gives them their prefix: `WEB-42` is item 42 of the project with identifier `WEB`.

Which of the "four connections" do I set up myself? Only the last one: your personal GitHub account. The App, the workspace connection and the repository mappings are set up by admins (Chapter 8).

Where do I find the git-sync settings? I can't see GitHub Sync in workspace settings. That menu is for workspace admins. Members use the direct page `/git-sync/ui/workspaces/<slug>/account` (**My GitHub account**). The Development panel appears in every work item for everyone.

Your GitHub account

Why connect my GitHub account if the repository already syncs? It tells git-sync which Plane member you are on GitHub. With the link, your Smart Commits say "by <your login>", your GitHub assignments and @mentions become Plane assignments and mentions, and changes git-sync makes on GitHub for your Plane edits are made as you. Without it, git-sync uses the App's identity.

Will my Plane comments from Smart Commits show my name as the author? No. In this release every Plane-side change is made by the git-sync service account. Smart Commit comments name you in their "via <sha> by <login>" line.

Does connecting give git-sync access to my private repositories? No. Repository access comes only from the GitHub App installation that your organization approved. Your personal link adds identity, not access.

What does "Needs re-authorization" mean? GitHub refused the stored token: you revoked it, it expired, or the App was changed. Press **Reconnect**. Until you do, git-sync uses the App's identity for you.

Is Disconnect safe? **Disconnect personal account** removes only your link and revokes your token. The team's sync keeps working. It also stops git-sync from re-linking you automatically by email. The organization card's **Disconnect** is a different, admin-only button that disables mappings.

Linking code

Where must the key appear for the PR to link? In the branch name, the PR title or the PR body. Commit messages never create a PR link; they only run Smart Commits.

Why is `WEB-42` in my branch name treated as "closes"? A bare key in the branch name (any case) or in the PR title (uppercase) is a **closing** reference. A bare uppercase key in the body is only a relation. Magic words set the tier explicitly (Chapters 9–10).

I wrote `Refs WEB-42` but the merge still moved the item to Done. Why? The strongest tier wins. A key in the branch name or title made it closing. For partial work, keep the key out of both.

Can I unlink a PR by removing the key or adding `skip`? No. `skip WEB-42` (in the PR body) only prevents a *new* link. An existing link stays. Ask your admin.

Can one PR close two items? Yes: `Closes WEB-42, WEB-45` (or `and`, `&`). Each item is checked separately at merge.

Can one item have several PRs? Yes. The item moves to Done only when at least one linked PR has merged and none is still open. This is called the merge guard (Chapter 14).

What is `WEB-NEW`? Put `WEB-NEW` in a PR title or body and git-sync creates a new item in project `WEB`, titled after the PR, already In Progress and linked to the PR.

Does GitHub itself understand `WEB-42`? No. `WEB-42` is a Plane key that git-sync reads. GitHub's own `Closes #123` refers to GitHub issues. You can use both in one PR body.

Smart Commits

When do Smart Commit commands run? When the commit is **pushed** to the mapped repository, and only if the mapping has **Smart Commits** turned on.

Where does `#time` go? Into a Plane comment, "Time logged: 1h 30m (90 minutes)". Plane CE has no worklog. `1d` = 8h and `1w` = 5d.

Why did my commands run twice? You rebased or amended a pushed commit. The new SHA counts as a new commit. Each SHA runs once.

Why does `#in` fail? It matches both In Progress and In Review. Use `#in-progress` or `#in-review`.

Does the God mode setting "Require signed commits" protect us? Not in this release. The setting is stored but not enforced. Enforcement is the server setting `REQUIRE_VERIFIED_COMMITS=true` (Chapter 11).

Automation and the Development panel

Why didn't approval move my item? The shipped rules change nothing on approval or ready for merge. Merging into `main` moves it to Done.

What do `OPEN · REVIEW`, `REJECTED` and `APPROVAL` mean? `OPEN` is the PR state. `REVIEW` means no decisive review yet. `REJECTED` means a reviewer currently requests changes. `APPROVAL` means approved with no standing change request. They are PR badges, not Plane states.

Does MERGED mean deployed? No. Check **Deployments** and your release process.

Why does Create branch not show my branch locally? It creates the branch on GitHub only. Run `git fetch origin` and `git switch <branch>`.

Why is there no list of commits in the panel? The panel shows PRs, builds, deployments, previews and branches. A commits list is not part of this release.

Mirrored issues

Will every GitHub issue appear in Plane? No. Only issues with the `plane` label, and only in mapped repositories.

Why did my Plane title edit not reach GitHub? By default GitHub owns the title and description. Edit them on GitHub.

Why did a merge close the Plane item but not its GitHub issue? git-sync's own changes in Plane are not mirrored back. Add `Closes #N` to the PR so GitHub closes the issue.

What happens if I delete a mirrored Plane item? Nothing is closed. Within about an hour, reconciliation marks the link orphaned and comments on the GitHub issue.

Are priority, due dates, estimates or cycles synced? No. Only title, description, state, assignees, labels and new comments.

Trust and timing

How fast is sync? Usually seconds, but nothing is guaranteed: queues, rate limits and outages add delay. Wait, then report.

Can sync loop forever between Plane and GitHub? No. git-sync remembers what it wrote and ignores the echo. It also ignores Plane events from its own service account.

Can I fix a missing link by recreating the item? Do not. You would get a duplicate, and the original event is still in the audit log. Report it so an admin can **Replay** the delivery.

Glossary

Plain definitions of the terms used in this manual. Terms in **bold** are exact UI labels.

Term	Definition
Plane	The self-hosted project-management tool (Community Edition v1.4.2) where the team plans and tracks work.
Community Edition (CE)	The free, self-hosted edition of Plane. It has no built-in GitHub integration; git-sync adds one.
git-sync	The service in this repository. It runs beside Plane as one extra container and keeps Plane and GitHub in step.
Workspace	The top-level Plane space for a team or company (for example `acme`). Members, projects and settings live in it.
Workspace slug	The short name of a workspace used in URLs, such as `acme`.
Project	A group of work items inside a workspace, with its own identifier (for example **Website**, identifier **WEB**).
Project identifier	The letters before the number in a work-item key (`WEB` in `WEB-42`). 2–12 characters, starting with a letter.
Work item	One task, bug or story in Plane (Plane's name for an issue).
Work-item key	The project identifier plus a number, such as `WEB-42`. git-sync finds these keys in branch names, PRs and commits.
State	Where a work item is in the workflow, such as Todo, In Progress, In Review or Done.
State group	The fixed category a state belongs to: Backlog, Unstarted (shown as "Not started"), Started, Completed, Cancelled, or Triage. Rules can target a group.

Term	Definition
Triage	The state group used by Intake for unreviewed requests. git-sync never moves an item into triage.
Cycle	A time-boxed planning period (a sprint) that collects work items. Not synced.
Module	A feature-sized group of work items. Not synced.
Page	A Plane document (notes, specs, agreements). Not synced.
View	A saved set of filters and display options. Not synced.
Intake	The project inbox for requests that are triaged before they become regular work.
Estimate	A size value (for example Points 1, 2, 3, 5, 8, 13). Not synced.
God mode	Plane's instance-admin area at `/god-mode/`. git-sync's instance settings appear there under **GitHub Sync**.
Instance admin	The person who administers the whole Plane installation (God mode). Configures the GitHub App, magic words and defaults.
Workspace admin	A Plane workspace member with the Admin role. Connects GitHub, maps repositories, edits rules and user links.
Member	An ordinary workspace member. Can use the Development panel and **My GitHub account**, but cannot open **GitHub Sync** in workspace settings.
Service account (bot)	The Plane user git-sync acts as, using its Plane personal access token. Plane-side changes made by git-sync appear under this account.
Personal access token (PAT)	A Plane API key. The bot's PAT is set at install time; it is separate from any GitHub authorization.
GitHub App	The application registered on GitHub that gives git-sync permission to read and write repositories and receive webhooks.

Term	Definition
Installation	The GitHub App installed on one GitHub organization or user account. One installation can be connected to only one Plane workspace.
Allow-list	The instance admin's list that permits an installation to be used by a workspace (**Allow-list an installation**).
Owner handoff link	A single-use link a workspace admin sends to a GitHub organization owner so the owner can approve and connect the installation.
Repository mapping	The link between one GitHub repository and one Plane project, created with **Add mapping**.
Sync direction	A mapping setting: **Two-way**, or **GitHub to Plane only** (Plane changes are never written to GitHub).
Personal GitHub link	Your Plane account linked to your GitHub account through **Connect personal account**. Identified by GitHub's stable numeric user id.
OAuth	The standard sign-in-with-GitHub flow used by **Connect personal account**. git-sync never sees your GitHub password.
Re-authorization	Connecting your GitHub account again (**Reconnect**) after the stored authorization stopped working.
User link	The record that pairs a Plane member with a GitHub account. Created by the member (OAuth), by an admin (**Link a user**) or automatically by a unique public email match.
Tombstone	A record left when a link is removed, so automatic email matching does not link the same account again.
Reference	A work-item key found in a branch name, PR title, PR body or commit message.
Magic word	A word or phrase before a key that sets the reference's tier, such as `Closes`, `Refs` or `Relates to`. Instance admins can edit the list.

Term	Definition
Tier	How strong a reference is: closing, non-closing or relation. The strongest one found wins.
Closing reference	A reference that may move the item to its merged state (Done) when the PR merges. Bare keys in branch names and PR titles are closing.
Non-closing reference	A reference that links and can change state on PR events, but never on merge (`Refs WEB-42`).
Relation reference	A reference that only links; it never changes state (`Relates to WEB-42`, or a bare key in a PR body).
`skip` / `ignore`	A PR-body phrase (`skip WEB-42`) that stops one key from being linked by that PR. It does not remove an existing link.
`WEB-NEW`	A marker in a PR title or body that asks git-sync to create a new work item in project WEB and link it.
Merge guard	The rule that a closing item moves on merge only when at least one linked PR merged and none is still open.
Smart Commit	A commit-message command such as `WEB-42 #comment …`, `#time 30m` or `#in-review`, run when the commit is pushed.
Provenance	The “via <commit> by <login>” line git-sync adds to Smart Commit comments so readers can see their source.
Signed (verified) commit	A commit GitHub marks as signature-verified. Smart Commits can be limited to these with `REQUIRE_VERIFIED_COMMITS`.
Development panel	The **Development** box in a work item’s sidebar showing linked PRs, builds, deployments, previews, branches and **Start work**.
Rollup link	A categorized Plane link (`[PR]`, `[Branch]`, `[CI]`, `[Deploy]`, `[Preview]`) that git-sync creates and updates in place.

Term	Definition
Branch template	The pattern for suggested branch names, `{key}-{title-slug}` by default. Not editable in the UI in this release.
Issue mirroring	Opt-in two-way copying between a GitHub issue and a Plane work item, started by the `plane` or `github` label.
Gate label	The label that opts an item into mirroring: `plane` on a GitHub issue, `github` on a Plane work item.
Conflict policy	Per field, which side wins when both changed: **Last write wins**, **GitHub is the source of truth**, **Plane is the source of truth**.
Linkback comment	A comment git-sync posts on GitHub with the Plane link when a link is created (**Post link-back comments on GitHub**).
Problem report	A GitHub/Plane comment starting “Git sync could not complete this action:” posted when **Report problems as comments on GitHub and Plane** is on.
Rule	A YAML instruction: **when** a trigger happens, **if** conditions match, **then** run actions. The first matching rule wins.
Lifecycle point	A named PR or branch moment (Branch pushed, PR opened, Review requested, PR merged …) used in the rules table.
Trigger	The event name a rule listens for, such as `pull_request.merged` or `branch.created`.
Ready for merge	A lifecycle point reached when a PR is open, not a draft, approved, has no standing change request and all checks pass.
Branch-aware rule	A rule limited to a base-branch regex, such as “merged into `^staging$` → In QA”.
Dry run	**Test rules**: running a delivery or trigger through the rules without writing anything.
Backfill	Linking a newly mapped repository’s existing open PRs and issues to the work items they mention.

Term	Definition
Webhook	An HTTP message one system sends another when something changes. GitHub and Plane both send webhooks to git-sync.
Delivery	One received webhook, recorded in the audit log and replayable.
Replay	Running a recorded delivery again (**Replay**), which may write to Plane and GitHub.
Echo suppression	Ignoring the webhook git-sync caused itself, so a change does not bounce back and forth.
Reconciliation	A periodic sweep (default every 60 minutes) that detects missed changes and deleted or archived Plane items.
Orphaned link	A link whose Plane work item no longer exists. git-sync comments on GitHub but never closes anything.
Audit log	The list of everything git-sync received and did, with outcome and raw detail.
Sync health	Per-mapping timestamps and error counts shown on the Repositories tab.
Setup checks	God-mode checks of App credentials, permissions, events, deliveries and the Plane request rate.
`git-sync.internal`	The Docker network name Plane uses to send its webhooks to git-sync.

What you learned

- Members set up only their personal GitHub link. Everything else is admin work.
- Keys in branch names and titles close items, the strongest tier wins, and `skip` never unlinks.
- `#time` is a comment, commands run once per SHA, and signed-commit enforcement is the server variable `REQUIRE_VERIFIED_COMMITS`.
- Only six fields mirror, and git-sync's own Plane changes are not mirrored back.

Source files

- `manual/notes/glossary.md`: glossary source
- `src/parse/references.ts`, `src/github/smart-commits.ts`, `src/sync/mirror.ts`, `src/github/pr-automation.ts`: behavior behind the answers

19 · Cheat sheet

Daily loop: open the item → In Progress → **Create branch** → `git fetch && git switch <branch>` → commit + push → draft PR → ready + request a reviewer → fix → approve → merge → check that the item is **Done**.

Intent	Write in the PR	Merge moves the item?
Finish	branch `WEB-42-…`, title `WEB-42 …`, `Closes WEB-42`	Yes (merge guard)
Part of it	branch with no key, `Refs WEB-42`	No
Context	`Relates to WEB-42`	Never changes state
Exclude a copied key	`skip WEB-42` (body; does not unlink)	—
No item yet	`WEB-NEW` in the title	Creates the item
Two items	`Closes WEB-42, WEB-45`	Each checked separately

Keys: `WEB-42` (not `WEB_42`, not `WEB-42_x`). Branch keys are any case; title and body keys must be uppercase. `Closes:WEB-42` fails, because a space is needed. The strongest tier wins.

Smart Commit (runs on push, once per SHA)	Effect
`WEB-42 #comment Checked at 320 px.`	Plane comment “via <sha> by <login>”
`WEB-42 #time 1h 30m`	Comment “Time logged: 1h 30m (90 minutes)”; 1d = 8h, 1w = 5d
`WEB-42 #in-review`	Moves to In Review (full hyphenated name)
`WEB-33 WEB-34 #in-progress #comment …`	Both items; `#comment` last

Keys first, on the same line. A rebased or amended commit runs again.

Default rule	State
Branch pushed / PR drafted / PR opened	In Progress
Review requested	In Review
Merged into `main`	Done (no other linked PR still open)
Approved / ready for merge / closed without merge	no change

Badge	Means
`OPEN · REVIEW` / `REJECTED` / `APPROVAL`	Open: no decision / changes requested / approved
`MERGED` / `DECLINED`	Merged / closed without merge
`[CI]` `[Deploy]` `[Preview]`	Build, deployment, preview (.vercel.app, .netlify.app, .pages.dev, .amplifyapp.com)

Mirroring: `plane` label on a GitHub issue, `github` label on a Plane item. Six fields: title, description, state, assignees, labels, new comments. GitHub owns title and description by default. Deleting never closes.

Stuck? Send your admin the key, PR/commit URL, SHA, time + timezone, expected vs. seen. Never send tokens.

My GitHub account: `/git-sync/ui/workspaces/<slug>/account`

What you learned: the daily loop, tiers, Smart Commits, defaults, badges and mirroring on one page. **Source files:** `src/parse/references.ts`, `src/github/smart-commits.ts`, `src/github/pr-automation.ts`, `src/sync/dev-info.ts`

PART B

Admin guide

For workspace administrators and Plane instance (God mode) administrators. Install git-sync next to a self-hosted Plane, register and enter the GitHub App, connect GitHub to a workspace, map repositories to projects, tune conflict policies and rules, link people, and keep the service healthy: updates, key rotation, replay, uninstall and troubleshooting.

Chapters 20–30. Each procedure follows the same pattern: Goal → Before you start → Steps → Result → If it went wrong.

20 · Architecture and requirements

This chapter is for the people who install and run git-sync: the server operator, the Plane instance administrator who uses God mode, and the workspace administrators who map repositories. It explains what git-sync adds to a Plane server, how events travel, where data is kept and what you need before you install. Read it once before chapter 21; you will come back to it when something needs diagnosing.

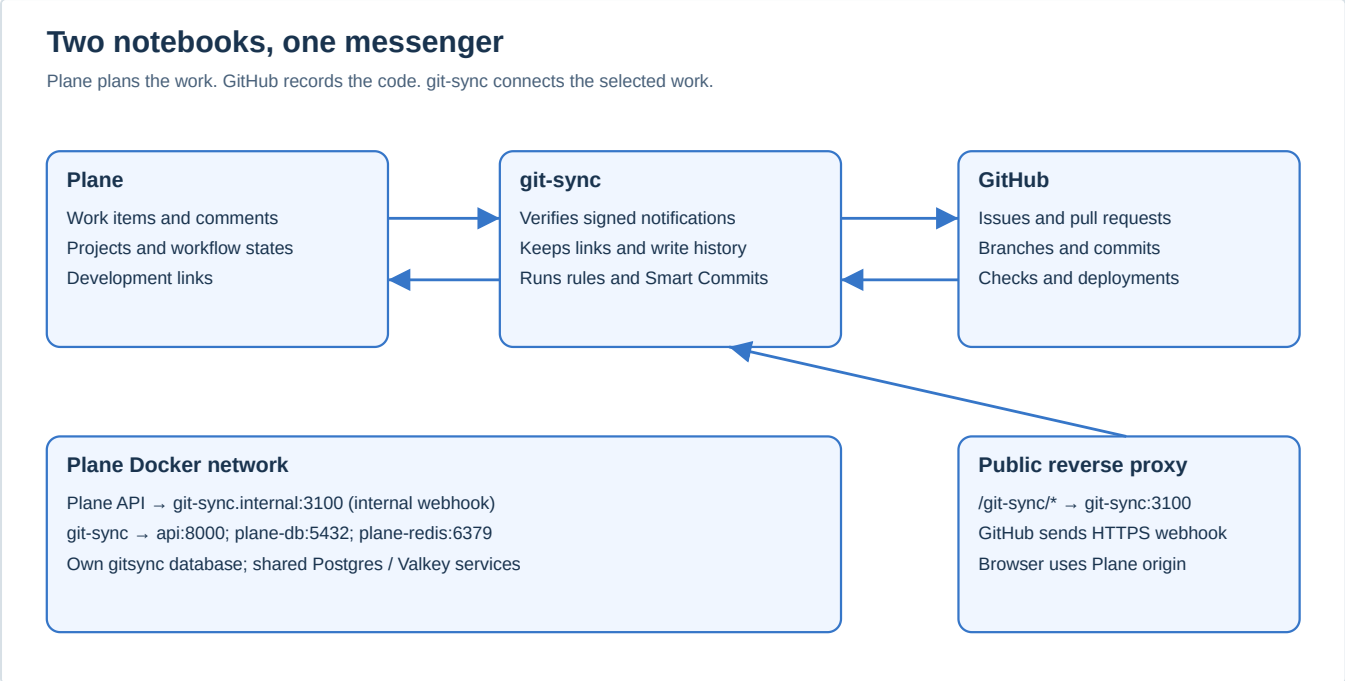


Figure 20.1 Diagram: Plane holds planned work, GitHub holds code and issues, and git-sync joins selected work across them.

What git-sync is

git-sync is **one extra container** that runs next to a self-hosted Plane Community Edition installed with Docker Compose. It is called `git-sync` in Compose and listens on port `3100`. It reuses three things Plane already has:

Plane part	What git-sync uses it for	What git-sync never does
Postgres (`plane-db`)	Its **own** database `gitsync` and role `gitsync`, created by the installer	It never reads or writes Plane's own tables. Every business change in Plane goes through Plane's API.
Valkey (`plane-redis`)	Work queues, under the prefix `gitsync` (setting `QUEUE_PREFIX`)	It does not touch Plane's own keys.

Plane part	What git-sync uses it for	What git-sync never does
Proxy (`proxy` , Caddy)	A route that sends `/git-sync/*` to `git-sync:3100` , inserted before Plane's catch-all	It does not change Plane's other routes.

Nothing else is added: no extra database server, no extra Redis and no extra public hostname. Everything git-sync shows in a browser lives under your Plane address, for example `https://plane.example.com/git-sync/ui/admin` .

On Plane v1.4.2 the installer can also build two small *Plane UI fork images*, `plane-frontend-gitsync:v1.4.2` and `plane-admin-gitsync:v1.4.2` . They add three things to Plane's own screens:

- **GitHub Sync** in **Workspace settings** (for workspace admins).
- **GitHub Sync** in God mode's sidebar (for instance admins).
- A **Development** section in every work item's sidebar (for everyone).

Each of these is an iframe that shows a git-sync page. The fork touches eight upstream files; the bulk of the UI lives in git-sync itself. If you keep Plane's stock UI (`--keep-plane-ui`), the same pages still open at their direct addresses (Appendix A lists them all).

> **Note:** The Plane menus always say **GitHub Sync**. The pages inside them have headings **Git sync:** `<workspace slug>` and **Git sync: instance settings**. Older notes and the installer's help text say "Git Sync"; that is the same thing.

The four connections

git-sync only syncs work when four separate connections exist. Different people own each one.

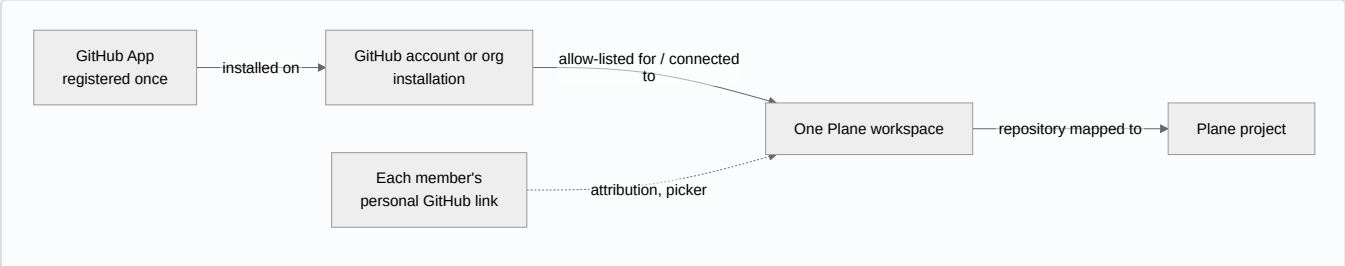


Figure 20.4 Diagram: the four connections. One installation belongs to exactly one workspace.

Connection	Who sets it up	Where	Chapter
1. The **GitHub App** is registered and its credentials are entered in git-sync	GitHub account/org owner registers it; the **instance admin** enters it	GitHub settings; God mode → **GitHub Sync**	22, 23
2. The App is **installed** on a GitHub account or organization, and that installation is **connected** to one Plane workspace	An org owner installs/approves; a **workspace admin** connects (or the instance admin allow-lists)	GitHub; **Workspace settings → GitHub Sync**	24, 23
3. A **repository is mapped** to a Plane project	**Workspace admin**	**Workspace settings → GitHub Sync → Map a repository**	24, 25
4. Each *member links their personal GitHub account*	Each member (or a workspace admin, manually)	**My GitHub account** / **User links**	8, 27

One installation can serve only one Plane workspace. If a second workspace tries to connect the same installation, it is refused with “That installation is connected to another workspace.” This boundary is deliberate: a webhook from that installation can only ever write into the one workspace that owns it.

Roles and permissions

Who	Plane role	What they can open in git-sync
Instance admin	Signed in to God mode (separate `admin-session-id` session)	`/git-sync/ui/admin`: App credentials, setup checks, defaults, magic words, installation allow-list, disabled mappings, manual mapping recovery, instance audit
Workspace admin	Workspace role **Admin** (role 20)	`/git-sync/ui/workspaces/<slug>`: connection, mappings, policies, rules, user links, audit log
Member	Workspace role **Member** (15) or higher	Only **My GitHub account** (`/git-sync/ui/workspaces/<slug>/account`) and the **Development** panel of items they can see
Guest	Guest	The Development panel only if Plane lets them read the work item

git-sync never keeps its own user list. On every request it asks Plane, with the browser's own session cookie, who the viewer is and what role they have. A normal workspace session is not enough for God mode pages: they need the God mode session.

> **Admin only:** Members do not see **GitHub Sync** in Workspace settings (Plane CE shows that settings area to admins only). Give members the direct link to **My GitHub account**: `https://<your Plane>/git-sync/ui/workspaces/<slug>/account`.

How an event travels

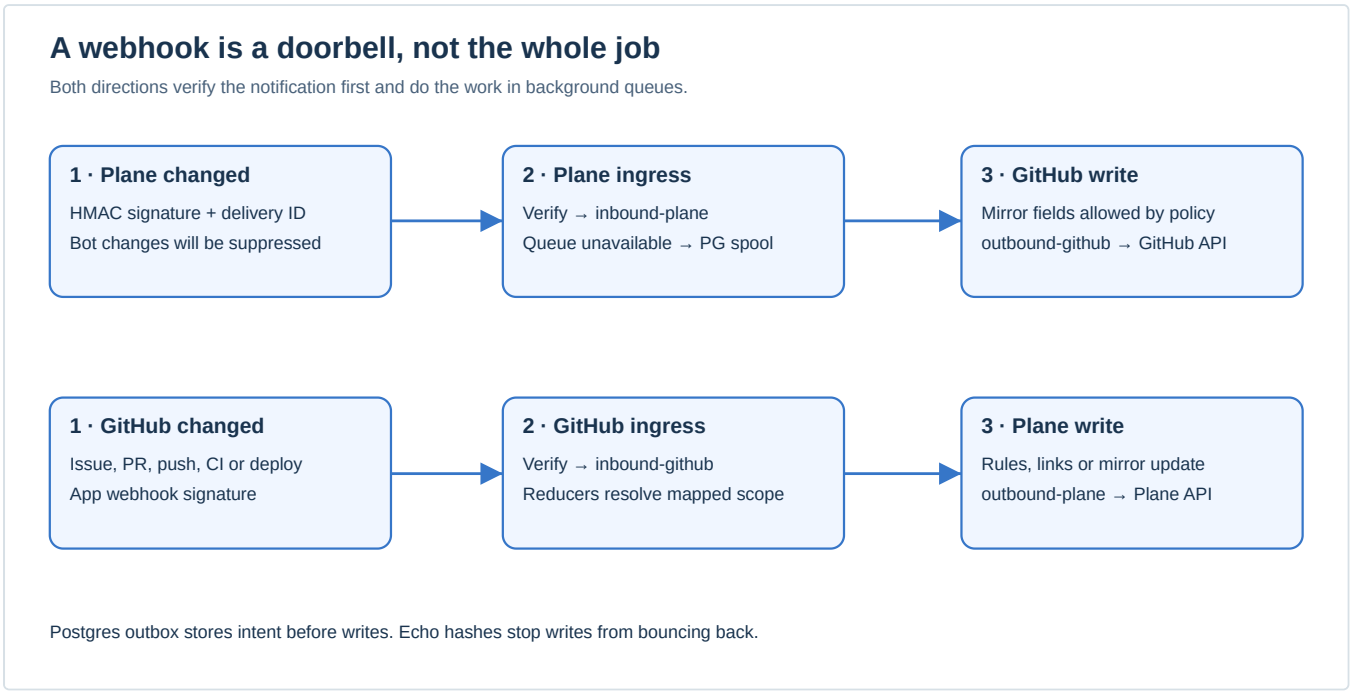


Figure 20.2 Diagram: two event directions through signed ingress, queues, reducers and the ordered outbox.

1. **A change happens.** Someone pushes, opens a PR or edits an issue on GitHub, or edits a work item in Plane.
2. **A signed webhook arrives.**
   - GitHub posts to `https://<your Plane>/git-sync/webhooks/github`, signed with the App's webhook secret (`X-Hub-Signature-256`).
   - Plane posts **inside Docker** to `http://git-sync.internal:3100/git-sync/webhooks/plane`, signed with the Plane webhook secret (`X-Plane-Signature`).
   - A missing or wrong signature gets `401` and is dropped.
3. **Durable acceptance.** The verified delivery is put on a queue (`inbound-github` or `inbound-plane`) and acknowledged.
   - If Valkey is down, a Plane delivery is kept in Postgres (`inbound_spool`) and replayed every minute once the queue is back.
   - A GitHub delivery gets `503` in that case and must be redelivered from GitHub later (chapter 29).
4. **Reducers decide.** A worker drops duplicates and echoes of git-sync's own writes, finds the mapping and checks that the mapping's installation is still allowed for that workspace. It then works out what should happen: link a PR, run Smart Commit commands, mirror a field, or run a rule.

5. **The ordered outbox writes.** Every intended write to Plane or GitHub is stored first in `outbound_writes`, then carried out in order per target by the `outbound-plane` / `outbound-github` workers.
   - Failed writes are retried with back-off, up to 30 attempts.
   - Rate limits pause the target until the limit's deadline.
6. **Everything is audited.** Each step leaves a row in the audit log (chapter 28).

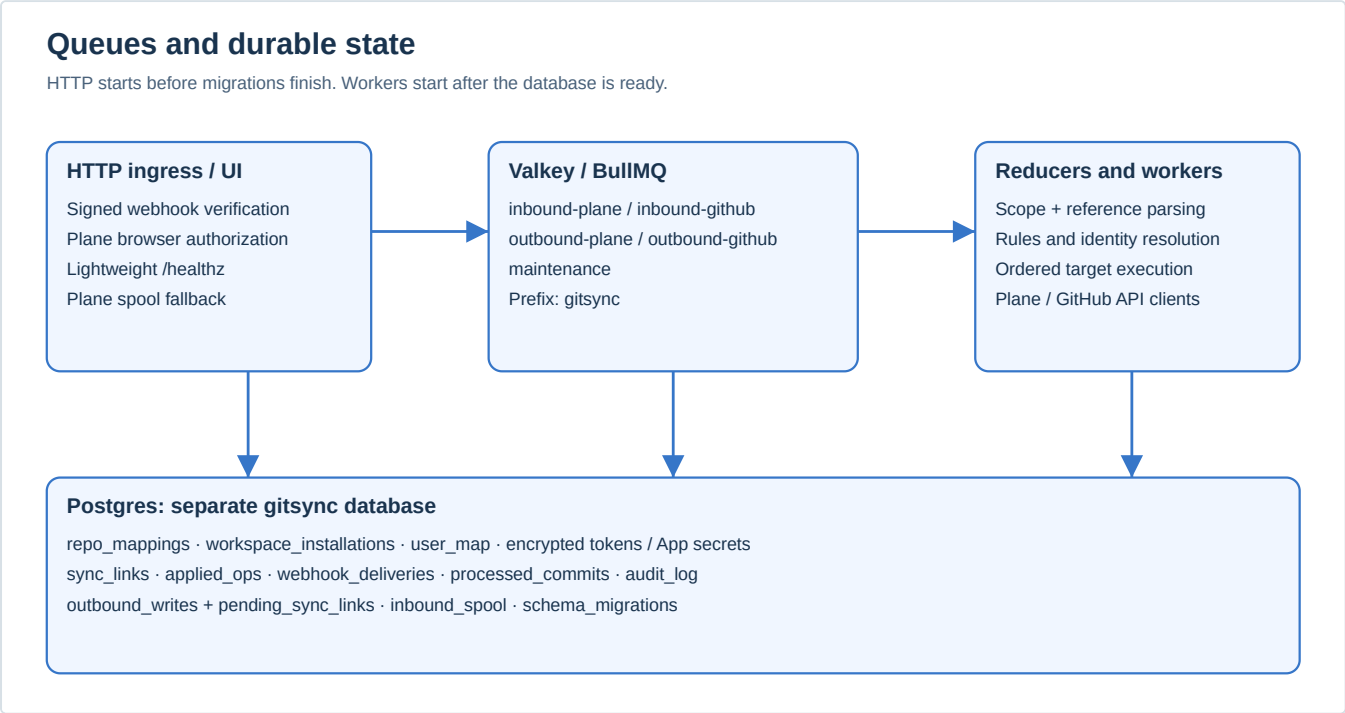


Figure 20.3 Diagram: runtime components and durable state. Shared infrastructure, separate database and queue namespace.

Why git-sync ignores its own changes

git-sync writes into Plane as a **service account** (a dedicated Plane user, usually called "GitHub Bot"). When Plane then sends a webhook for that change, git-sync recognizes the service user and the field-level fingerprint of its own write, and drops the echo. This is why the service account must be a **separate** Plane user: if you install git-sync with your own personal API key, your own edits look like echoes and never reach GitHub.

Background jobs

A `maintenance` queue runs recurring work:

- Replays the Plane spool every minute.
- Prunes echo records every ten minutes (default echo TTL ten minutes).
- Runs **reconciliation** every 60 minutes by default. It checks existing links, marks partners that were deleted in Plane as *orphaned*, and notices when Plane deliveries have stopped arriving.
- Prunes old ledgers hourly.

Both intervals are editable in God mode → **Defaults**.

Where data lives

Data	Where	Back it up?
Mappings, rules, policies, links, audit log, outbox, user links	`gitsync` Postgres database	Yes
GitHub App private key, webhook secret, OAuth client secret, members' GitHub tokens	Same database, **encrypted** with AES-256-GCM using `GITSYNC_TOKEN_KEY`	Yes, *together with the key*
Service settings (`DATABASE_URL`, `PLANE_SERVICE_PAT`, `GITSYNC_TOKEN_KEY`, …)	`git-sync.env` in Plane's directory, mode `0600`	Yes, privately
Optional App key bootstrap file and credential-retirement marker	`git-sync/` directory in Plane's directory	Yes
Compose wiring	`docker-compose.override.yaml` (merged, not replaced)	Yes
Queued jobs	Valkey under `gitsync:*`	Optional; Plane deliveries also spool to Postgres

> **Warning:** `GITSYNC_TOKEN_KEY` and the database belong together. Restoring the database with a different key makes every stored secret unreadable (God mode then shows **GitHub App credentials cannot be decrypted. Save the App secrets again.**). The installer always keeps the existing key; never change it by hand.

Network topology

There are two supported layouts.

Plane's bundled proxy (default, `--proxy plane`).

- The installer copies Plane's Caddyfile, inserts `reverse_proxy /git-sync/* git-sync:3100` before Plane's catch-all, and mounts the patched file into the `proxy` service.
- Browsers and GitHub reach git-sync through your normal Plane address.

Your own reverse proxy (`--proxy external --gateway-network NAME`).

- git-sync joins your proxy's Docker network under the alias `git-sync`.
- You add a route for `/git-sync/*` to `git-sync:3100` yourself, placed **before** the Plane route.

In both layouts:

- git-sync joins Plane's default Compose network with the alias **`git-sync.internal`**. Plane refuses webhooks to its own public domain and to host names without a dot, so the Plane webhook uses this internal name.
- git-sync talks to Plane's API directly at `http://api:8000`, without going through the public proxy.

Requirements checklist

Requirement	Why	Check
Self-hosted **Plane Community Edition v1.4.2**, installed with Docker Compose (the official `setup.sh` install works)	The UI forks are pinned to v1.4.2; other versions keep stock UI automatically	Plane opens normally
Docker Engine with the **Compose plugin**, run by a user who can use Docker and write Plane's directory (usually `sudo`)	The installer drives Compose	`docker compose version`
`git` on the server	Building the Plane UI fork images	`git --version`
~2 GB free disk and 10–20 minutes for the first UI image build	Fork build	—
A public **HTTPS** Plane address reachable from GitHub (or your GitHub Enterprise Server)	GitHub webhooks and OAuth callbacks	`curl https://<plane>/` from outside
A dedicated Plane **service account** with a Personal Access Token, member of every synced workspace	Writes into Plane; echo filtering	Chapter 21
`WEBHOOK_ALLOWED_HOSTS` containing `git-sync.internal` in Plane's env, applied to `api`, `worker`, `beat-worker`	Plane's webhook guard	Installer checks it
A GitHub account or organization where you may register and install a GitHub App	App registration	Chapter 22
Plane's `API_KEY_RATE_LIMIT` and git-sync's `PLANE_API_RATE_PER_MINUTE` agreed	Avoid 429s	God mode **Setup checks**

Security model in one page

- **Webhooks are signed.** A delivery with a wrong signature is refused before parsing. A signature proves who sent it; it does not grant write access by itself.
- **Every write is authorised again at execution time.** A queued write remembers the mapping, repository and installation that admitted it. The outbox checks that exact authority again before calling Plane or GitHub. A mapping removed or an installation revoked after the event means the write is refused.
- **Browser writes are CSRF-protected.** Every configuration change must come from your exact Plane origin (`PUBLIC_ORIGIN`) with a JSON body.
- **Secrets are encrypted and never shown again.** The App's private key, webhook secret and client secret are write-only in God mode: leave the fields empty to keep them.
- *Personal GitHub tokens* are stored encrypted per member. Members can disconnect at any time, which revokes the token at GitHub when possible.
- **Workspace identity is pinned.** A workspace slug is bound to Plane's immutable workspace ID. If a workspace is deleted and a new one reuses its slug, git-sync answers `409` until an instance admin clears the old data, so the new workspace cannot inherit old credentials.
- **The UI may only be framed by Plane itself.** git-sync pages send `frame-ancestors 'self'`. If your proxy adds `X-Frame-Options: DENY` to everything, the embedded pages go blank (chapter 21, "Blank GitHub Sync page").

What you learned

- git-sync is one container beside Plane that reuses Plane's Postgres, Valkey and proxy, with its own database and queue prefix.
- Four connections must exist: App credentials, an installation connected to one workspace, a repository mapping, and personal links.
- Events are verified, queued, reduced and written through an ordered, re-authorized outbox; everything is audited.
- The database and `GITSYNC_TOKEN_KEY` must always travel together.
- Before installing you need Docker Compose, a service account with a PAT, the internal webhook allow-list entry and a public HTTPS origin.

Source files

- `src/app/main.ts` — runtime composition
- `src/api/handler.ts` — signed webhook ingress
- `src/queue/` — queues, workers, maintenance
- `src/operations/outbox.ts`, `src/app/writers.ts` — ordered outbox
- `src/ui/plane-auth.ts` — role checks against Plane
- `apps/README.md` — the Plane UI fork surface
- `docs/superpowers/specs/2026-10-10-plane-github-sync-design.md` — design

21 · Install git-sync

This chapter takes you from a running Plane to a running git-sync. You prepare three things in Plane: a service account, its API key and a webhook. Then you run one command, `./install.sh`, on the Plane server. The GitHub side comes afterwards, in chapters 22–24.

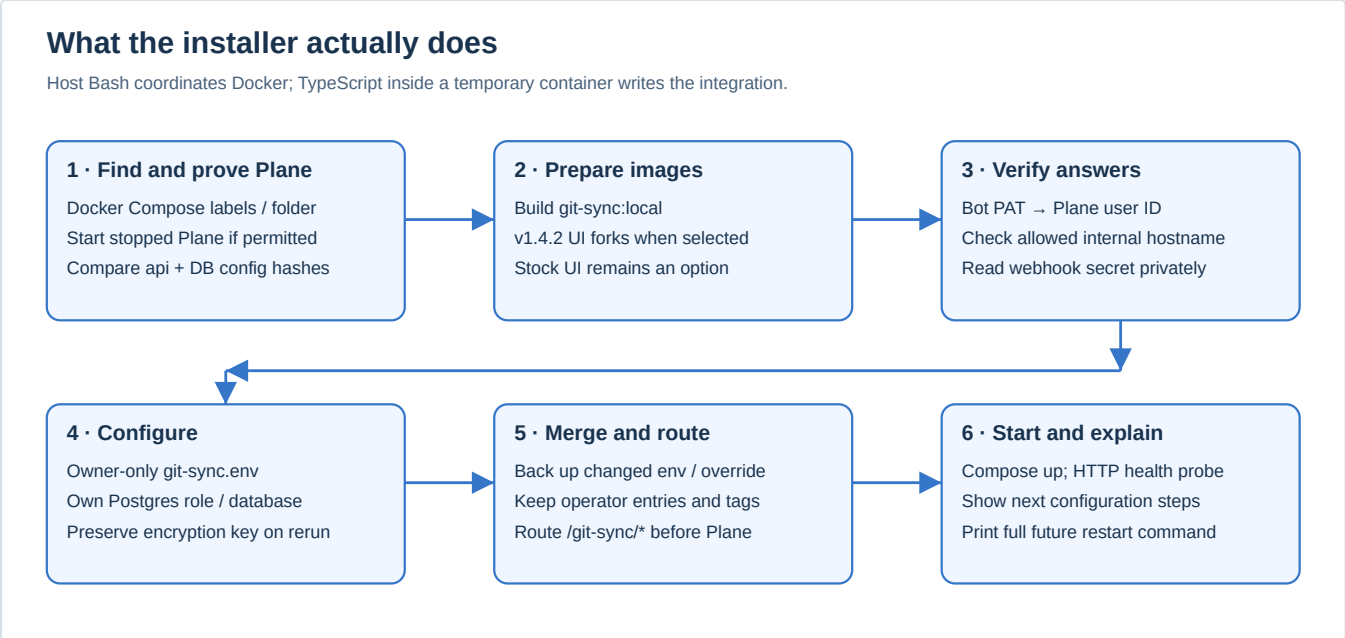


Figure 21.1 Diagram: installer stages — detection, safety proof, configuration and startup.

Overview

Step	Where	Time	Who
1. Create a service account	Plane sign-up + **Workspace settings → Members**	5 min	Workspace admin
2. Create its API key	Plane, signed in *as the service account*	2 min	Workspace admin
3. Allow the internal webhook host	`plane.env` on the server	5 min	Server operator
4. Create the Plane webhook	**Workspace settings → Webhooks**	2 min	Workspace admin
5. Run `./install.sh`	Server shell	5 min, plus 10–20 min for the first UI build	Server operator
6. Fix framing if the page is blank	Your proxy	as needed	Server operator

Step 1 — Create a separate service account

Goal: give git-sync its own Plane identity

Before you start: decide on an email address for the bot. It must be a different Plane user from every person.

> **Warning:** Do not use your own account. git-sync ignores Plane changes made by its service account, because they look like its own echoes. If the service account is you, your own edits never sync to GitHub. The installer prints this reminder too: *"git-sync ignores Plane changes made by this account (they look like its own). Use a dedicated account, not your personal one, or your own edits will not sync to GitHub."*

Steps

1. In a private browser window, open your Plane sign-in page and choose **Sign up** (if your instance allows self-registration). Create the user, for example display name **GitHub Bot**.
2. Back in your admin session, open **Workspace settings → Members** and invite the bot to every workspace you will sync.
3. Accept the invitation in the bot's session.
4. Join the bot to the projects you will map, so it can read and update their work items.

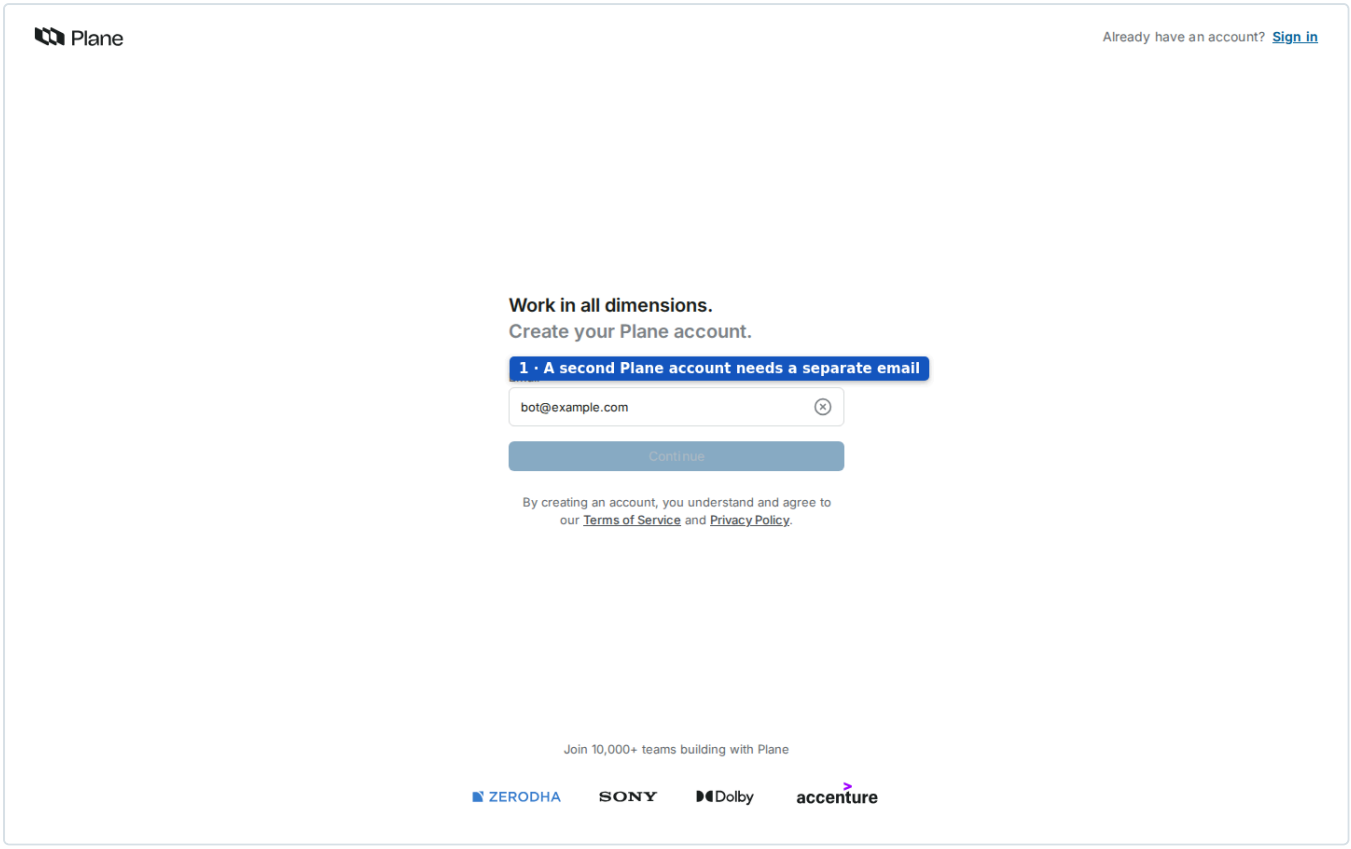


Figure 21.2 Screenshot (test Plane): the Sign up page with only an example bot address entered; nothing was submitted.

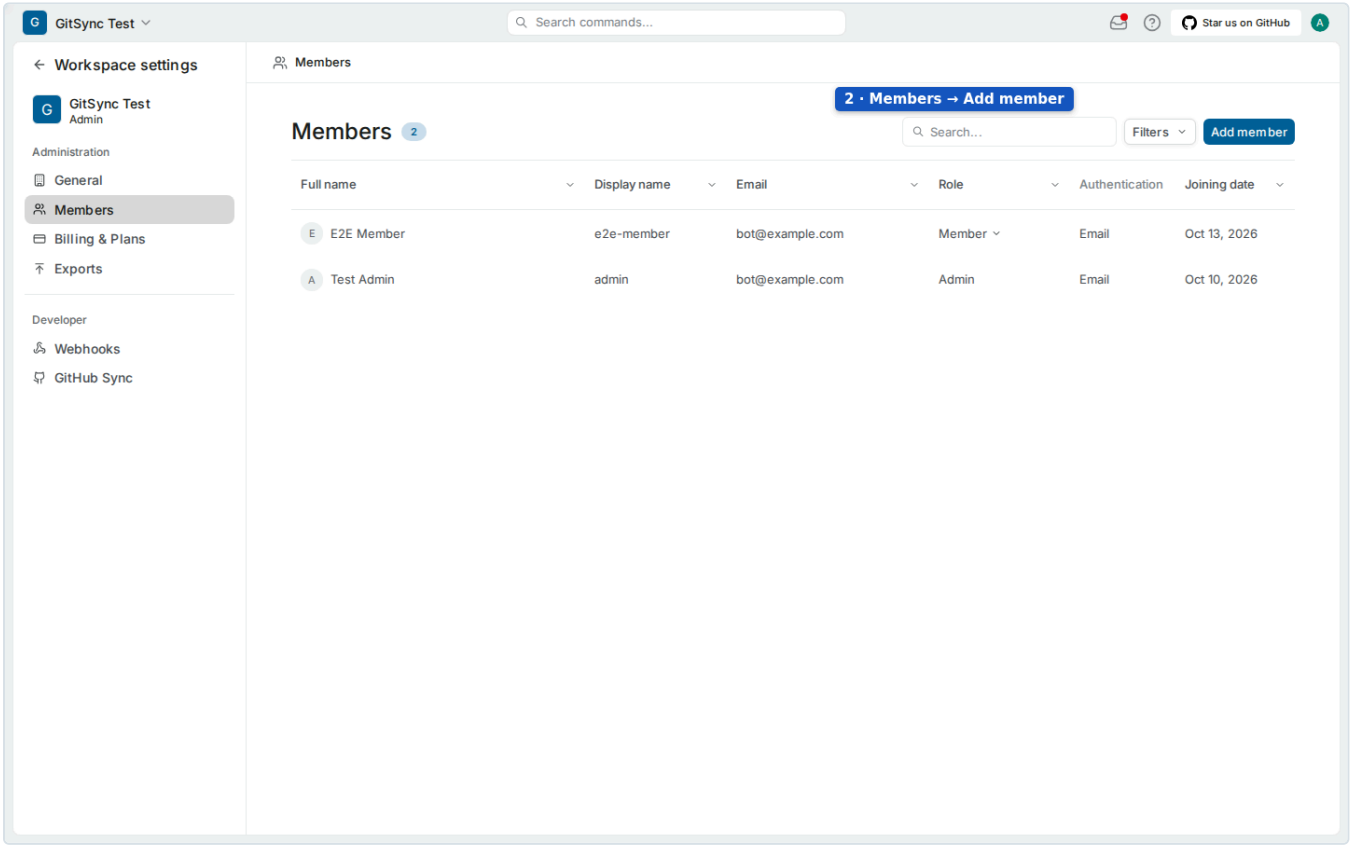


Figure 21.3 Screenshot (test Plane): Workspace settings → Members, where you invite the bot.

Result: the bot is a member of the workspace and its projects.

If it went wrong:

- *Sign up is not offered* → self-registration is disabled. Ask the instance admin to create the user through the instance's normal account process.
- *A Gmail address is the only one available* → a plus address such as `you+gitsync@example.com` is a distinct Plane user that still delivers to your mailbox. Use it only if your instance accepts it, and double-check you are signed in as the bot before step 2.

Step 2 — Create the service account's API key

Goal: a Plane Personal Access Token owned by the bot

Before you start: be signed in **as the bot**, not as yourself. A token made in your session belongs to you, whatever its title says.

Steps

1. Open the bot's avatar menu and choose **Settings**.
2. Choose **Personal Access Tokens**.
3. Select **Add access token**, enter a title such as `git-sync bot`, choose an expiry, and generate the token.
4. Copy the token into a safe temporary place. You will paste it into the installer's hidden prompt.

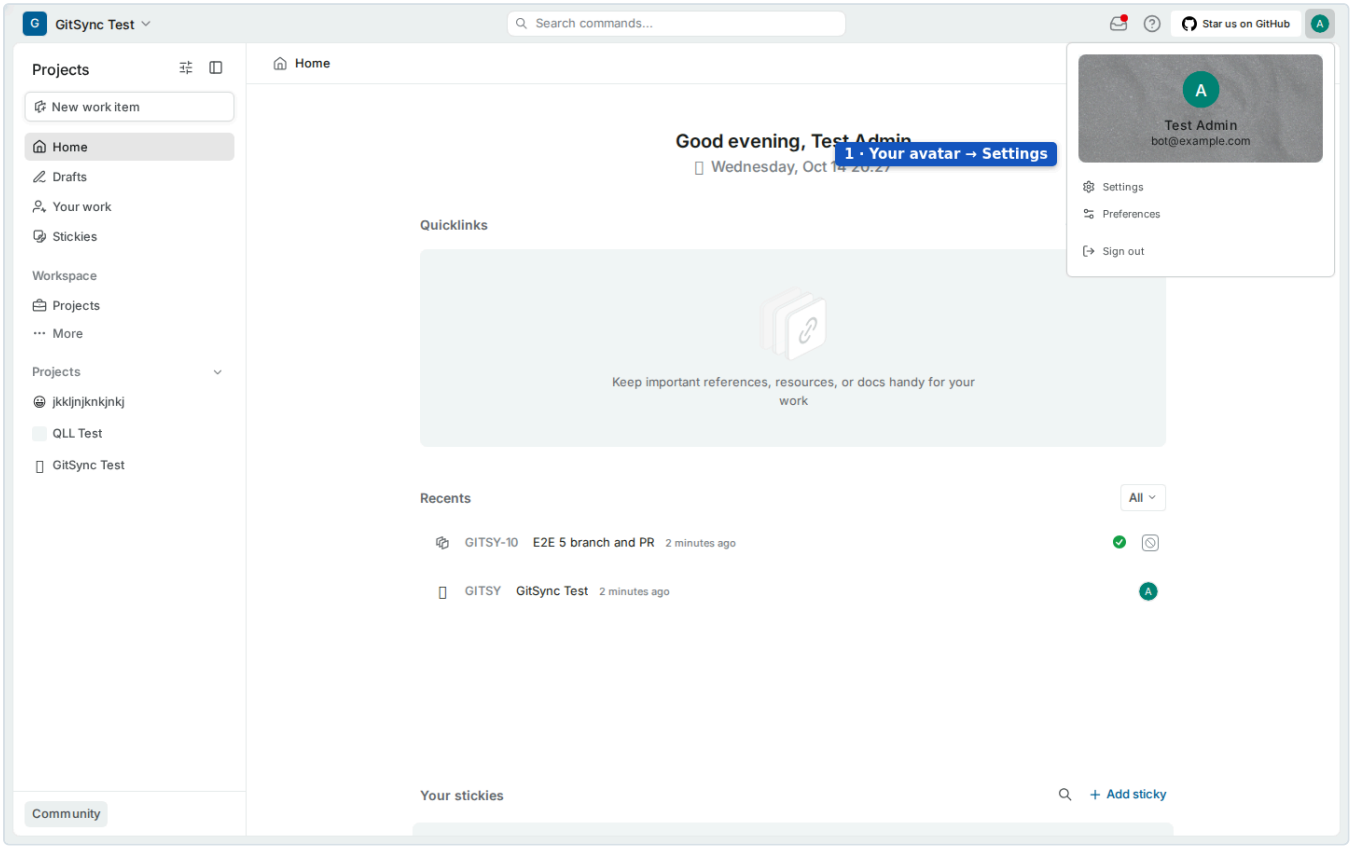


Figure 21.4 Screenshot (test Plane): the avatar menu with Settings.

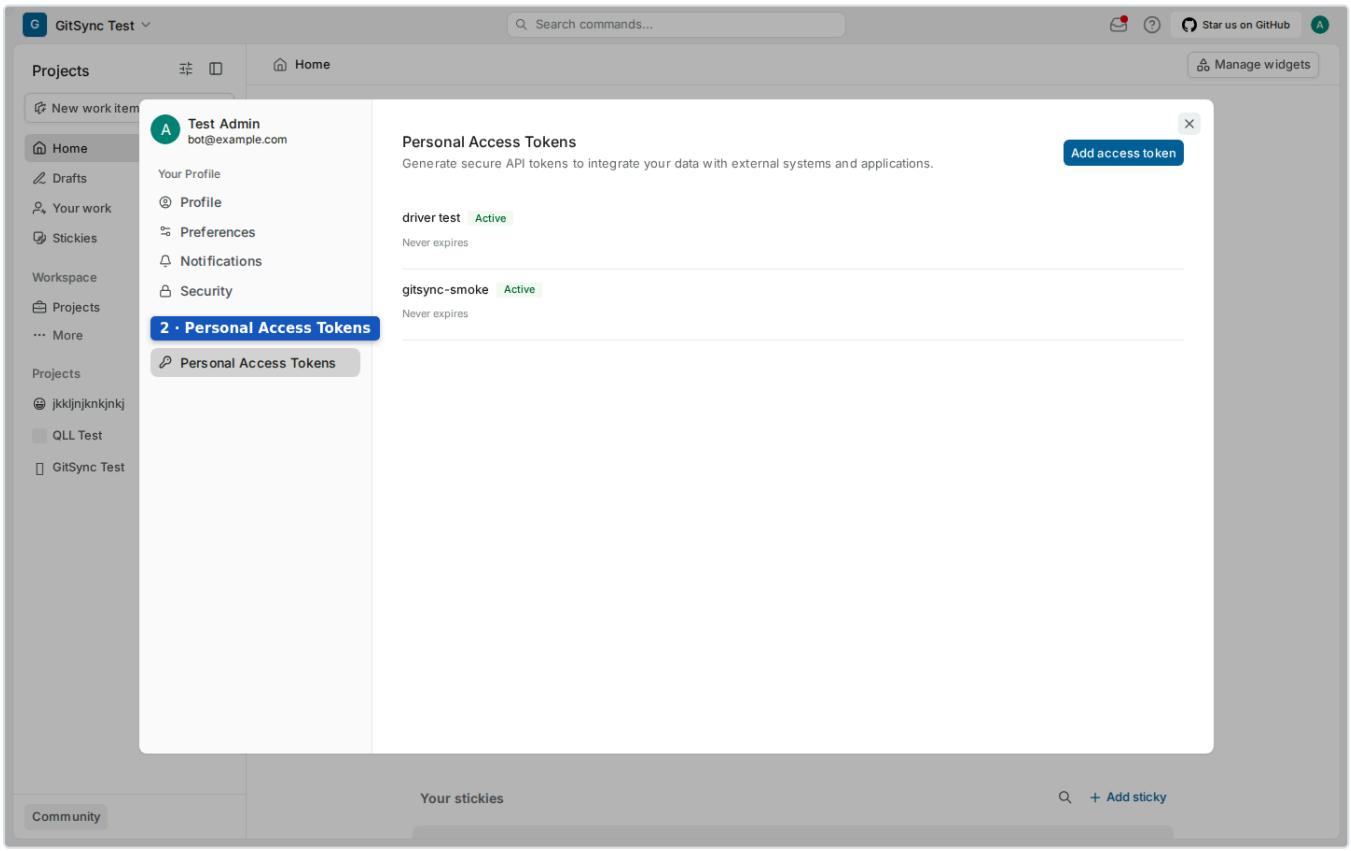


Figure 21.5 Screenshot (test Plane): Profile settings → Personal Access Tokens. No token value is visible.

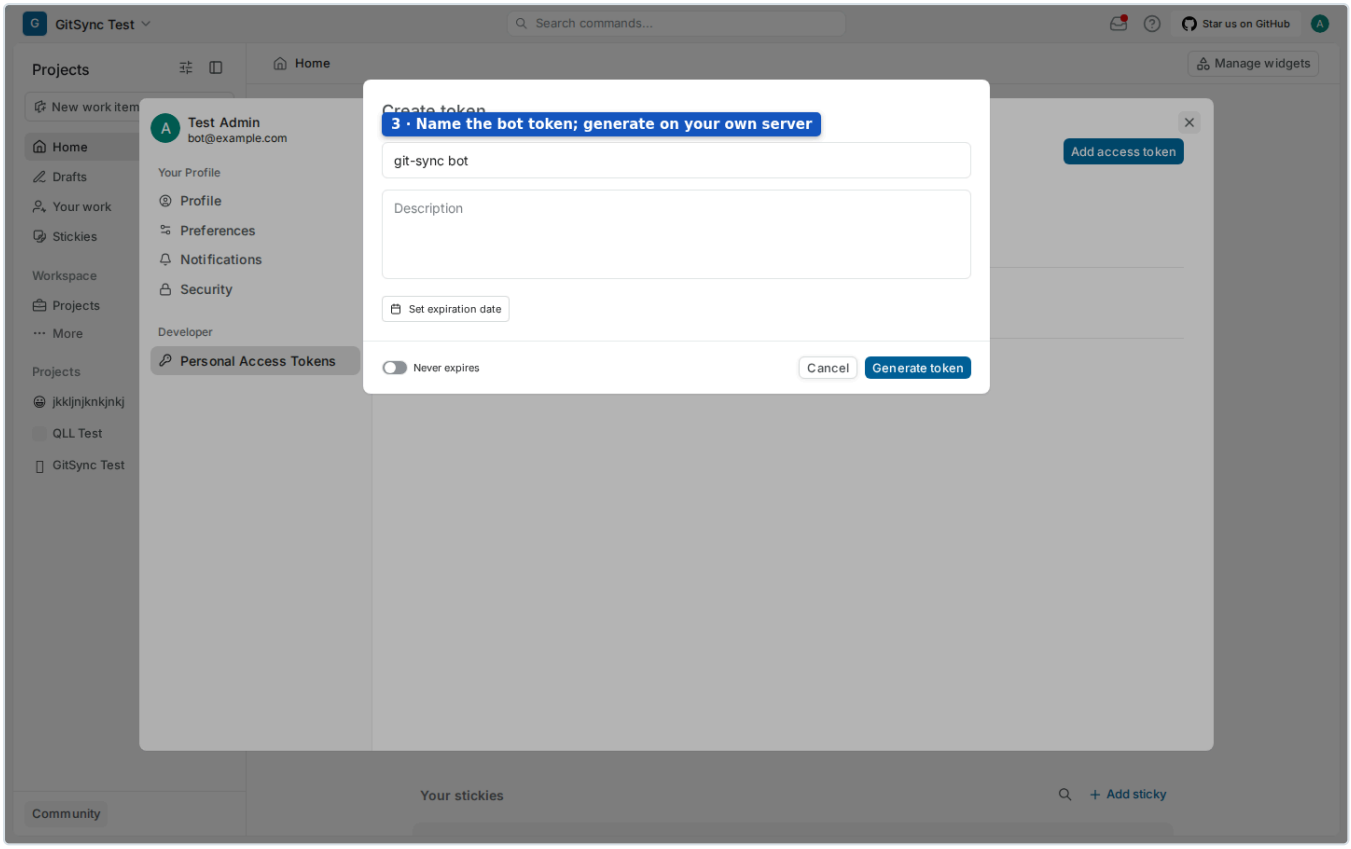


Figure 21.6 Screenshot (test Plane): the Create token dialog with an example title, cancelled afterwards.

Result: you have a token. The installer verifies it against Plane (`/api/v1/users/me/`) and records the bot's user ID by itself. You never type that ID.

If it went wrong: an expired or revoked token stops all Plane writes. Put a reminder in your calendar before the expiry date; rotation is in chapter 29.

Step 3 — Allow Plane to send webhooks to git-sync

Goal: `git-sync.internal` in Plane's webhook allow-list

Plane refuses webhooks to its own public domain and to host names without a dot. git-sync therefore receives Plane's webhooks inside Docker under the alias **`git-sync.internal`**, which the installer adds to Plane's network. Plane only sends to it if the name is in `WEBHOOK_ALLOWED_HOSTS`.

Steps (on the server, in Plane's directory)

1. Open Plane's env file (often `plane.env`).
2. Set `WEBHOOK_ALLOWED_HOSTS=git-sync.internal`, or append `,git-sync.internal` if the variable already has a value.
3. Apply it to the running API with your usual full Compose command, for example:

```
docker compose -f docker-compose.yaml --env-file plane.env up -d api worker
beat-worker
```

Use `up -d`, not `restart`: `restart` keeps the old environment.

Result: the running `api` container has the new value. The installer reads it from the running container, not from the file, and stops if it is missing:

```
Plane does not allow webhooks to git-sync yet. Add git-sync.internal to
WEBHOOK_ALLOWED_HOSTS in Plane's env file
  (e.g. WEBHOOK_ALLOWED_HOSTS=git-sync.internal, or append ,git-
sync.internal to an existing value),
  restart Plane's api, worker and beat-worker with your usual compose
command, then run ./install.sh again.
  Nothing has been changed yet.
```

Step 4 — Create the Plane webhook

Goal: Plane sends every workspace change to git-sync

Before you start: Step 3 is applied. Do this once **per workspace** you will sync.

Steps

1. Open **Workspace settings** → **Webhooks** and select **Add webhook**.
2. URL: `http://git-sync.internal:3100/git-sync/webhooks/plane` (exactly this; `http`, port `3100`).
3. Choose **Send me everything**.
4. Save, then copy the **secret key** Plane shows. Keep it with the bot token.

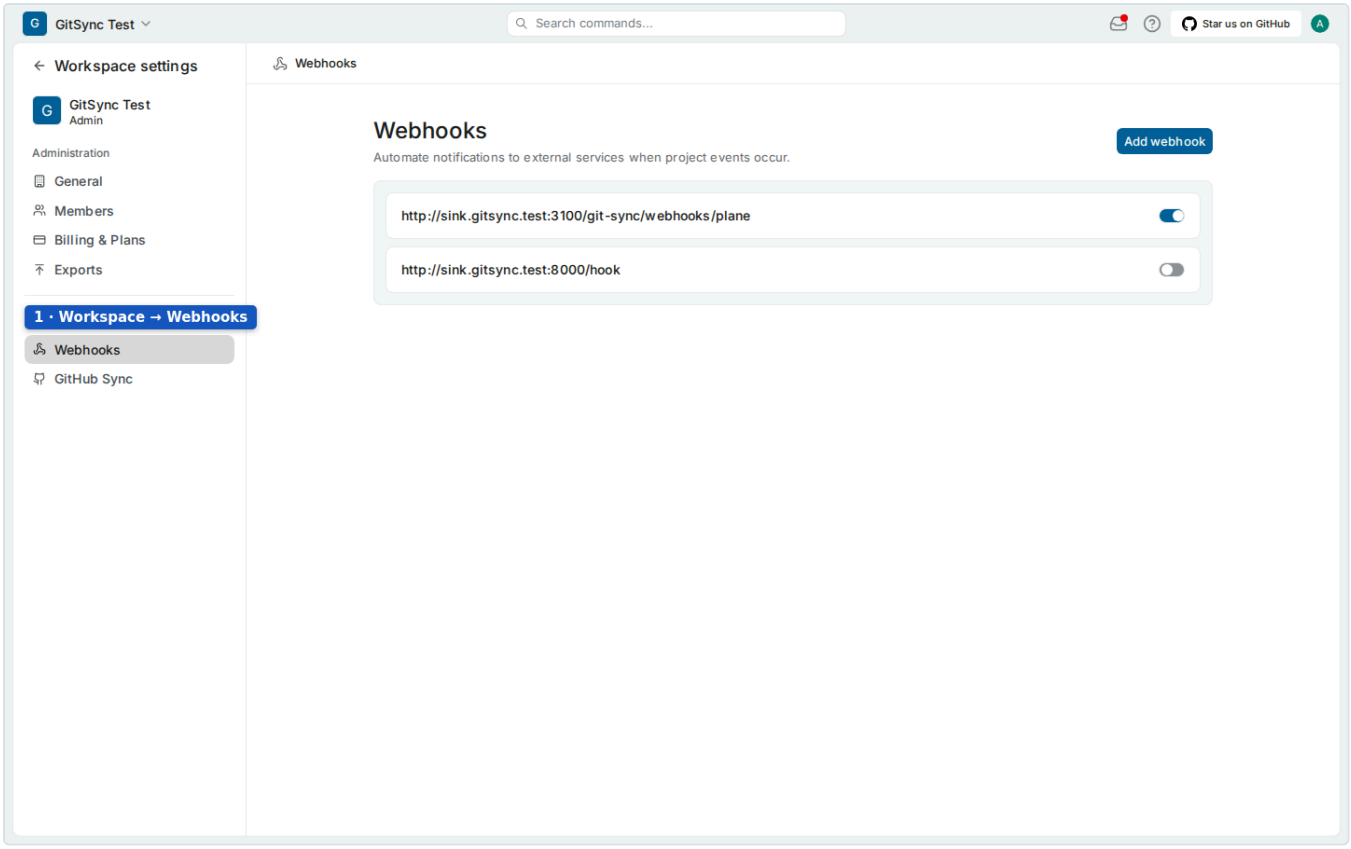


Figure 21.7 Screenshot (test Plane): Workspace settings → Webhooks. The listed addresses are old test sinks, not the address to use.

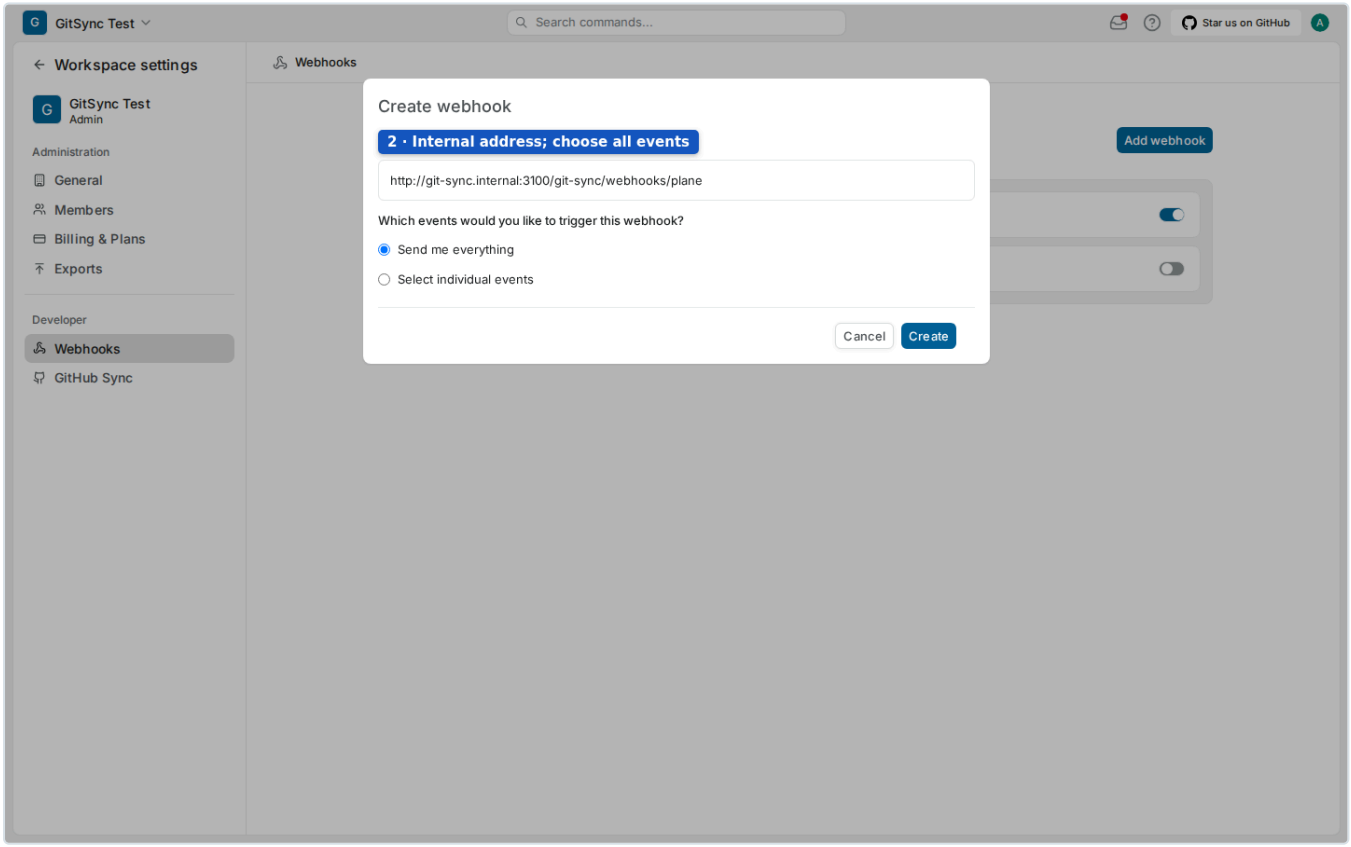


Figure 21.8 Screenshot (test Plane): the Create webhook dialog filled with the internal URL and Send me everything (cancelled).

Result: the webhook exists. Until git-sync is running, Plane cannot deliver to it; keep the gap short. Plane switches a webhook off after about 30–50 minutes of failed connections (chapter 29 explains how to re-enable it).

If it went wrong:

Message in Plane	Cause	Fix
“Webhook not created” / URL refused	You used your Plane domain, or a name without a dot, or `git-sync.internal` is not in the running `WEBHOOK_ALLOWED_HOSTS`	Use the exact internal URL; repeat step 3 with `up -d`

> **Note:** The Plane webhook secret, the bot’s API key and (later) the GitHub webhook secret are three different secrets. Never substitute one for another.

Step 5 — Run the installer

Goal: git-sync running and healthy next to Plane

Before you start:

- A shell on the server that runs Plane.
- The bot token and the Plane webhook secret.
- Docker access for your user. Use `sudo` only if root owns the Docker daemon Plane runs on. With rootless Docker, run as the owning user.

Steps

1. Get the code and enter it:

```
git clone https://github.com/nathanabay/git_plane-intergration.git git-sync
cd git-sync
```

2. Start the installer:

```
sudo ./install.sh
```

3. Answer the prompts (typed secrets are hidden):

Prompt (exact text)	Shown when	Enter
`Start Plane now and install git-sync on it? [Y/n]`	Plane is installed but stopped	`Y` only if this is the Plane you mean
`Plane API key of the git-sync service account:`	First install, or the stored key was rejected	The bot token from step 2
`Plane webhook secret:`	First install	The secret from step 4
`Public URL of your Plane site (e.g. https://plane.example.com):`	Plane's `WEB_URL` could not be read	Your public Plane address
`GitHub App webhook secret:`	Only with `--github-app-id` and `--github-private-key`	The App's webhook secret (chapter 22)

4. Wait. The installer prints one heading per stage:

Heading	What happens
`==> Checking Docker and Plane`	Finds Docker, Compose and Plane (running, stopped, or only its folder). Reproduces how Plane was started and refuses if it cannot.
`==> Building the git-sync image`	Builds `git-sync:local` from this checkout.
`==> Preparing the Plane UI images (adds Git Sync to Plane's menus)`	On Plane v1.4.2 only: builds `plane-frontend-gitsync:v1.4.2` and `plane-admin-gitsync:v1.4.2` (first time: often 10–20 minutes), or reuses them.
`==> Configuring git-sync`	Checks the bot token, checks `WEBHOOK_ALLOWED_HOSTS`, writes `git-sync.env`, creates the `gitsync` database and role, patches the proxy route, merges the Compose override.
`==> Starting git-sync`	Validates the merged Compose model, runs `up -d`, waits up to 2 minutes for `/git-sync/healthz`, checks the proxy answers.

Real disposable installer run · paths and origin simplified

```
==> Checking Docker and Plane
    Plane is installed in /root/plane but not running.
    Plane started
    Plane: /root/plane (project plane)
    Plane env file: /root/plane/plane.env

==> Building the git-sync image
    git-sync:local ready
    Keeping Plane's own UI (node:24-alpine); git-sync's settings open at <your Plane URL>/git-sync/ui/admin

==> Configuring git-sync
  - Plane site: https://plane.example.com
  - Plane service account: e2e bot
    Note: git-sync ignores Plane changes made by this account (they look like its own). Use a dedicated
    account, not your personal one, or your own edits will not sync to GitHub.
  - Wrote <plane>/git-sync.env (owner-only permissions); Plane API limit 120/minute
  - Created the gitsync database on Plane's Postgres
  - Routed /git-sync/* through Plane's proxy
  - Updated <plane>/docker-compose.override.yaml (backup: <plane>/docker-compose.override.yaml.bak-TIMESTAMP)

==> Starting git-sync
    git-sync is running and healthy

git-sync is installed.
```

Figure 21.9 Screenshot (terminal): real installer output through a healthy start (from the disposable installer test; paths simplified).

Result: the installer ends with **git-sync is installed.** and prints the next steps. It also prints the command you must use **from now on** to start Plane:

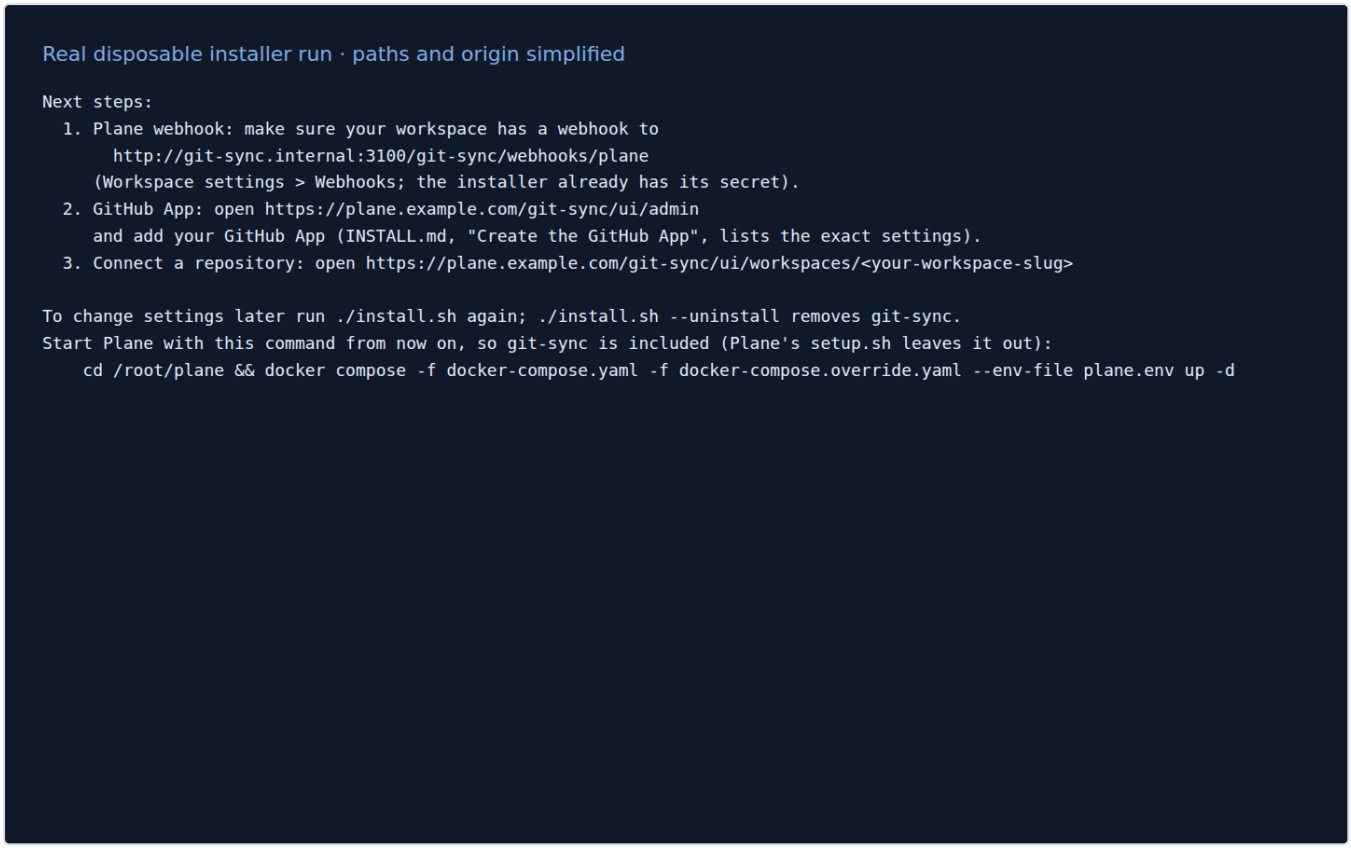


Figure 21.10 Screenshot (terminal): the next steps and the full future start command.

```
Start Plane with this command from now on, so git-sync is included (Plane's
setup.sh leaves it out):
    cd /root/plane && docker compose -f docker-compose.yaml -f docker-
compose.override.yaml --env-file plane.env up -d
```

> **Warning:** Plane's own `setup.sh` start/restart ignores the override file, so it starts Plane **without** git-sync. Copy the printed command into your runbook and use it instead.

All installer options

```
$ ./install.sh --help
Usage: ./install.sh [options]

Installs git-sync next to a running self-hosted Plane, or updates an existing install.
It asks for two things (a Plane API key and a Plane webhook secret) and works out the rest.

Options:
  --plane-dir DIR          Plane's install directory (found automatically when Plane is running)
  --env-file FILE          Plane's env file, if it cannot be detected (e.g. plane.env)
  --public-url URL         Public URL of the Plane site (default: Plane's WEB_URL)
  --keep-plane-ui          Keep Plane's stock web/admin images (git-sync pages stay reachable
                           at <url>/git-sync/ui/admin); skips the slow Plane UI image build
  --with-plane-ui          Always use the git-sync Plane UI images (adds Git Sync to Plane's menus)
  --proxy plane|external   plane (default): route /git-sync/* through Plane's own proxy.
                           external: you add the route to your own reverse proxy.
  --gateway-network NAME   With --proxy external: a Docker network your reverse proxy is on
  --github-app-id ID       Optional: configure the GitHub App now (otherwise do it later in the UI)
  --github-private-key F   ...its private key file (.pem); the webhook secret is asked for
  -y, --yes                Never prompt; take secrets from the environment (see INSTALL.md)
  --uninstall              Remove git-sync from Plane (keeps its database and settings)
  --purge                  With --uninstall: also delete git-sync's database and settings
  -h, --help               Show this help

Non-interactive secrets: GITSYNC_PLANE_PAT, GITSYNC_PLANE_WEBHOOK_SECRET, GITSYNC_GITHUB_WEBHOOK_SECRET.
```

Figure 21.11 Screenshot (terminal): ./install.sh --help lists every option.

Option	Use it when
`--plane-dir DIR`	More than one Plane runs on the machine (“Found more than one Plane. Choose one with --plane-dir DIR”), or Plane is in an unusual folder
`--env-file FILE`	The installer cannot tell which env file Plane was started with (“Could not work out how this Plane was started”)
`--public-url URL`	Plane’s `WEB_URL` is not the address people actually use
`--keep-plane-ui`	Keep Plane’s stock web/admin images and skip the slow UI build. git-sync pages stay reachable at `<url>/git-sync/ui/admin` and `<url>/git-sync/ui/workspaces/<slug>`
`--with-plane-ui`	Always use the git-sync Plane UI images (the default on v1.4.2 is automatic)
`--proxy plane`	Default: route `/git-sync/*` through Plane’s own proxy
`--proxy external --gateway-network NAME`	Your own reverse proxy serves Plane. git-sync joins that Docker network as `git-sync`; add the route yourself (see below)

Option	Use it when
`--github-app-id ID --github-private-key FILE`	Enter the GitHub App at install time instead of in God mode. The two go together; the webhook secret is asked for
`-y`, `--yes`	Scripted install: never prompt. Secrets come from `GITSYNC_PLANE_PAT`, `GITSYNC_PLANE_WEBHOOK_SECRET` and `GITSYNC_GITHUB_WEBHOOK_SECRET`; the public URL from `GITSYNC_PUBLIC_ORIGIN`
`--uninstall`	Remove git-sync from Plane, keeping its database and settings (chapter 29)
`--purge`	With `--uninstall`: also delete git-sync's database and settings
`-h`, `--help`	Show the help

Example: your own Caddy serves Plane over a Docker network called `gateway`.

```
sudo ./install.sh --plane-dir /root/plane --proxy external --gateway-network
gateway
```

The installer then prints the route to add to your proxy, placed **before** the Plane route:

```
handle /git-sync/* {
    reverse_proxy git-sync:3100
}
```

Example: scripted install without prompts.

```
read -r -s -p 'Bot PAT: ' GITSYNC_PLANE_PAT; export GITSYNC_PLANE_PAT
read -r -s -p 'Plane webhook secret: ' GITSYNC_PLANE_WEBHOOK_SECRET; export
GITSYNC_PLANE_WEBHOOK_SECRET
sudo --preserve-env=GITSYNC_PLANE_PAT,GITSYNC_PLANE_WEBHOOK_SECRET
./install.sh -y
unset GITSYNC_PLANE_PAT GITSYNC_PLANE_WEBHOOK_SECRET
```

With `-y`, a missing answer is an error that names the variable. The installer never invents a secret.

> **Note:** The non-interactive error for the webhook secret mentions a `--plane-webhook-secret` flag. `install.sh` has no such flag; set `GITSYNC_PLANE_WEBHOOK_SECRET` instead (logged in `manual/notes/discrepancies.md`).

What the installer changes on the server

Item	Change
`docker-compose.override.yaml`	Adds the `git-sync` service (and, with UI forks, `web` / `admin` image entries). Your own entries, comments and `!override` / `!reset` tags are kept. Backed up first as `.bak-<time>`.
`git-sync.env`	Created, mode `0600`, owner-only. Existing values and extra keys are kept on re-runs. Backed up first.
`git-sync/`	Private directory for the optional App key and the credential-state marker (owned by UID 1000).
Postgres	A `gitsync` role and `gitsync` database on Plane's database server. Plane's own database is untouched.
Proxy (bundled mode)	A patched Caddyfile with `reverse_proxy /git-sync/* git-sync:3100` before Plane's catch-all, mounted into `proxy`. Refuses if your proxy already mounts its own Caddyfile.
Generated values	Database password, `GITSYNC_TOKEN_KEY` (32 random bytes, base64), `PUBLIC_ORIGIN`, `PLANE_API_RATE_PER_MINUTE` (read from Plane's `API_KEY_RATE_LIMIT`), `PLANE_SERVICE_USER_ID`

Is it safe on a live Plane?

- **No change unless Plane can be reproduced.** The installer only touches Plane's containers if it can rebuild exactly how Plane was started. It compares Compose's configuration hash for `api` and `plane-db` with the running containers; if they differ, it stops before changing anything.
- **Plane's own services are not recreated.** Only the proxy and, with the UI images, the `web` and `admin` apps restart.
- **Backups first.** Changed files are backed up next to themselves before each change.
- **Plane's data is never touched.** git-sync uses its own database and role; `--purge` drops only those.
- **Secrets are never printed.** Do not run the installer with `bash -x`, which would print them.
- **Existing override files are flagged.** If an existing override file was not loaded when Plane started, the installer warns: "…exists but the running Plane was started without it; its other entries will now apply too".

Step 6 — Blank GitHub Sync page? Fix framing

Plane shows git-sync inside an iframe. git-sync allows framing by the same origin (`frame-ancestors 'self'`). If your reverse proxy adds `X-Frame-Options: DENY` or `Content-Security-Policy: frame-ancestors 'none'` to **every** response, the embedded page stays blank.

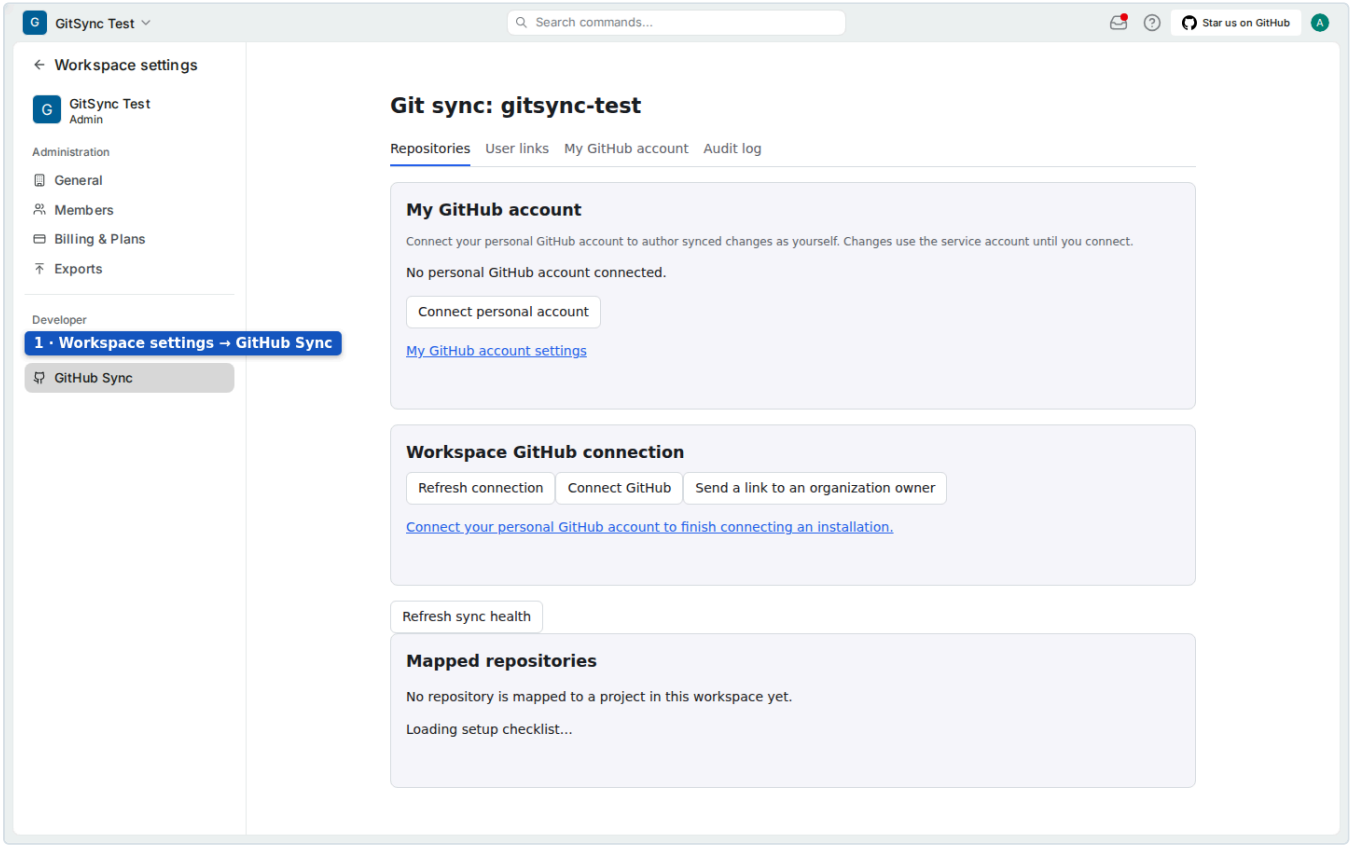


Figure 21.12 Screenshot (test Plane): Workspace settings → GitHub Sync with the embedded page loaded.

Goal: the embedded page loads

Steps

1. Open `https://<your Plane>/git-sync/ui/admin` in its own tab. If it loads there but not inside Plane, framing is the problem.
2. Exclude `/git-sync/*` from the blanket header. Caddy:

```
plane.example.com {
    handle /git-sync/* {
        header -X-Frame-Options
        reverse_proxy git-sync:3100
    }
    handle {
        header X-Frame-Options DENY
        reverse_proxy plane-proxy:80
    }
}
```

nginx:

```
location /git-sync/ {
    proxy_pass http://git-sync:3100;
    proxy_set_header Host $host;
    proxy_hide_header X-Frame-Options;
}
location / {
    proxy_pass http://plane-proxy:80;
    add_header X-Frame-Options DENY always;
}
```

3. Validate and reload the proxy, then reload the Plane page.

Result: GitHub Sync shows the git-sync page inside Plane.

If it went wrong: check every proxy layer (CDN, outer proxy, Plane proxy). One layer adding a denial is enough to block the frame. Keep git-sync's own `frame-ancestors 'self'`: do not remove all CSP.

Check the installation

```
curl -i https://plane.example.com/git-sync/healthz
```

The answer is `200` with `{"status":"ok"}`. This only proves the HTTP service answers. Next, open God mode → **GitHub Sync** and look at **Connection health** and **Setup checks** (chapter 23).

Troubleshooting the installer

Message	Cause	Fix
`Docker is not installed.` / `Cannot talk to the Docker daemon.`	No Docker or no permission	Install Docker; use `sudo` or the right rootless user
`The Docker Compose plugin is missing`	Old Docker	Install the Compose plugin
`Plane is not installed here (no Plane found in Docker or in the usual folders).`	No Plane found	Install Plane first, or pass `--plane-dir`
`Found more than one Plane. Choose one with --plane-dir DIR`	Several Plane projects	Pass `--plane-dir`
`Could not work out how this Plane was started (which env file it uses).`	Env file not detected	Pass `--env-file /path/plane.env`
`Running Plane with --env-file … does not reproduce the running Plane; check the file`	Wrong env file	Use the file Plane really runs with; never bypass
`Plane's directory … is not writable by this user (try sudo)`	Permissions	Run as the owner or with sudo
`Plane rejected the API key`	Expired, revoked or wrong token	Make a new bot token; run `GITSYNC_PLANE_PAT=<key> sudo -E ./install.sh` or answer the re-prompt
`Plane does not allow webhooks to git-sync yet…`	Step 3 not applied	Add `git-sync.internal`, `up -d api worker beat-worker`
`Plane's proxy service is not running. If you use your own reverse proxy, pass --proxy external`	External proxy	Use `--proxy external`

Message	Cause	Fix
`could not read the Caddyfile from Plane's proxy image; use --proxy external` / proxy already mounts its own Caddyfile	Custom proxy	Use `--proxy external` and add the route yourself
`building plane-…-gitsync:v1.4.2 failed (re-run, or use --keep-plane-ui)`	UI build failed (network, disk)	Re-run, or `--keep-plane-ui`
`the merged compose configuration is invalid (a backup of the override was kept next to it)`	Override conflict	Inspect the override and its `.bak-` file
`git-sync did not become healthy within 2 minutes (logs above)`	Start failure	Read the printed logs; `docker compose logs git-sync`
`GitHub App ID must be a number` / `--github-app-id and --github-private-key go together`	Bad App options	Pass both, numeric ID, readable PEM
Installer stops with no message	Shell error, lost terminal, Docker failure	Note the exit code, check Docker context and logs, re-run interactively (never with `bash -x`)

What you learned

- Prepare a dedicated service account, its API key, the `git-sync.internal` allow-list entry and a Plane webhook to `http://git-sync.internal:3100/git-sync/webhooks/plane` first.
- `./install.sh` finds Plane, proves it can reproduce it, builds images, writes `git-sync.env`, creates its own database, routes `/git-sync/*` and waits for health.
- Every option, prompt and message, and how to run it non-interactively with `-y`.
- Always start Plane afterwards with the printed full Compose command.
- A blank embedded page is a proxy framing header; exclude `/git-sync/*`.

Source files

- `install.sh` — host-side installer and options
- `src/install/main.ts`, `plane-env.ts`, `plane-api.ts`, `env-file.ts`, `compose.ts`, `caddy.ts`, `database.ts` — container-side configuration
- `INSTALL.md`, `deploy/APPLY.md` — installation reference and runbook
- `src/ui/static.ts` — UI security headers (`frame-ancestors 'self'`)

22 · Register the GitHub App

git-sync talks to GitHub as a **GitHub App**: a named integration with an explicit list of permissions and events. You register it *once per instance* on the GitHub account or organization that will own it. Then you install it on the repositories you want to sync. This chapter covers registration and the six values you copy into God mode. Installation and connection to a workspace are in chapter 24.

> **Note:** No GitHub session was available when this manual was produced, so GitHub's own forms are shown as *Illustrations* plus placeholders for real screenshots. The values and choices below come from the code (`src/ui/setup-check.ts` , `src/github/hosts.ts`) and the runbook.

Three different things on GitHub

Thing	What it is	Where you do it
Register the App	Create the App definition: name, URLs, permissions, events, keys	GitHub **Settings** → **Developer settings** → **GitHub Apps** → **New GitHub App** (for an organization: the organization's settings)
Install the App	Give the App access to an account's repositories	From git-sync's **Connect GitHub** button (chapter 24) or the App's public page
Authorize your personal account	Let git-sync know who you are on GitHub	**Connect personal account** in git-sync (chapter 8)

Before you start

- Your Plane public address, for example `https://plane.example.com`. Use the address people actually visit, with no trailing path and no invented port.
- Permission to create a GitHub App on the owning account or organization.
- A private shell where you can create a random secret:

```
openssl rand -hex 32
```

Keep the output private. It is the **webhook secret**, and you will enter it on GitHub and in God mode.

Register the App

Goal: a GitHub App with git-sync's URLs, permissions and events

> **Screenshot needed** — gh-01-new-app-form — GitHub's New GitHub App form (organization settings → Developer settings → GitHub Apps → New GitHub App), filled with the values below, not submitted, Client ID/slug masked. Capture later with npm run manual:capture -- --only gh-01-new-app-form

Steps

1. Open **New GitHub App** on the owning account or organization.
2. Fill in **General**:

Field	Value	Why
GitHub App name	A unique, recognizable name, e.g. `Acme Plane Sync`	Shown on installations and on comments the App posts (`<slug>[bot]`)
Homepage URL	`https://plane.example.com`	Where the integration is used
Callback URL	`https://plane.example.com/git-sync/oauth/github/callback`	Returns people after **Connect personal account**
Expire user authorization tokens	Leave on (GitHub's default)	git-sync stores expiry and refreshes tokens; a failed refresh asks the member to **Reconnect**
Request user authorization (OAuth) during installation	**Off**	When on, GitHub hides the Setup URL and sends installers to the OAuth callback instead
Enable Device Flow	Off	Not used
Setup URL	`https://plane.example.com/git-sync/install/github/callback`	Returns an App installation to the right Plane workspace
Redirect on update	**On**	Returns to git-sync after someone changes the installation's repository selection

3. Fill in **Webhook**:

Field	Value
Active	On
Webhook URL	`https://plane.example.com/git-sync/webhooks/github`
Webhook secret	The value from `openssl rand -hex 32`
SSL verification	Enabled

4. Set **Permissions** exactly as below. git-sync's setup check compares the App registration against this list (`REQUIRED_APP_PERMISSIONS`):

Scope	Permission	Access	Why git-sync needs it
Repository	**Pull requests**	Read and write	Read PR lifecycle; post link-back comments on PRs
Repository	**Issues**	Read and write	Mirror issues: fields, labels, assignees, comments
Repository	**Contents**	Read and write	Read commits and branches; **Create branch** from the Development panel
Repository	**Checks**	Read	Build results (check suites and runs)
Repository	**Commit statuses**	Read	Combined status (ready for merge) and the `status` event
Repository	**Deployments**	Read	Deployment results and preview URLs
Repository	**Metadata**	Read	Repository identity and installation checks (always required by GitHub)

Scope	Permission	Access	Why git-sync needs it
Organization	**Members**	Read	Prove an organization owner/admin when connecting an organization installation

Checks and **Commit statuses** are separate permissions; choosing one does not grant the other. Leave everything else at *No access*.

> **Note:** The runbook also lists *Administration: Read and write* for GitHub autolink references. The mapping form shows **GitHub autolink references (unavailable)** in this release, so do not grant it.

5. Under **Subscribe to events**, tick these nine events (`REQUIRED_APP_EVENTS`):

Event on GitHub's form	git-sync uses it for
Pull request	Linking, lifecycle rules, Development panel
Pull request review	Review badges, approved / changes requested, ready for merge
Push	Smart Commits, branch created, branch links
Issues	Issue mirroring
Issue comment	Mirrored comments
Check suite	Build status, ready for merge
Check run	Build status
Status	Commit status (legacy CI)
Deployment status	Deployments and previews

`installation` and `installation_repositories` events arrive automatically; you cannot and need not tick them.

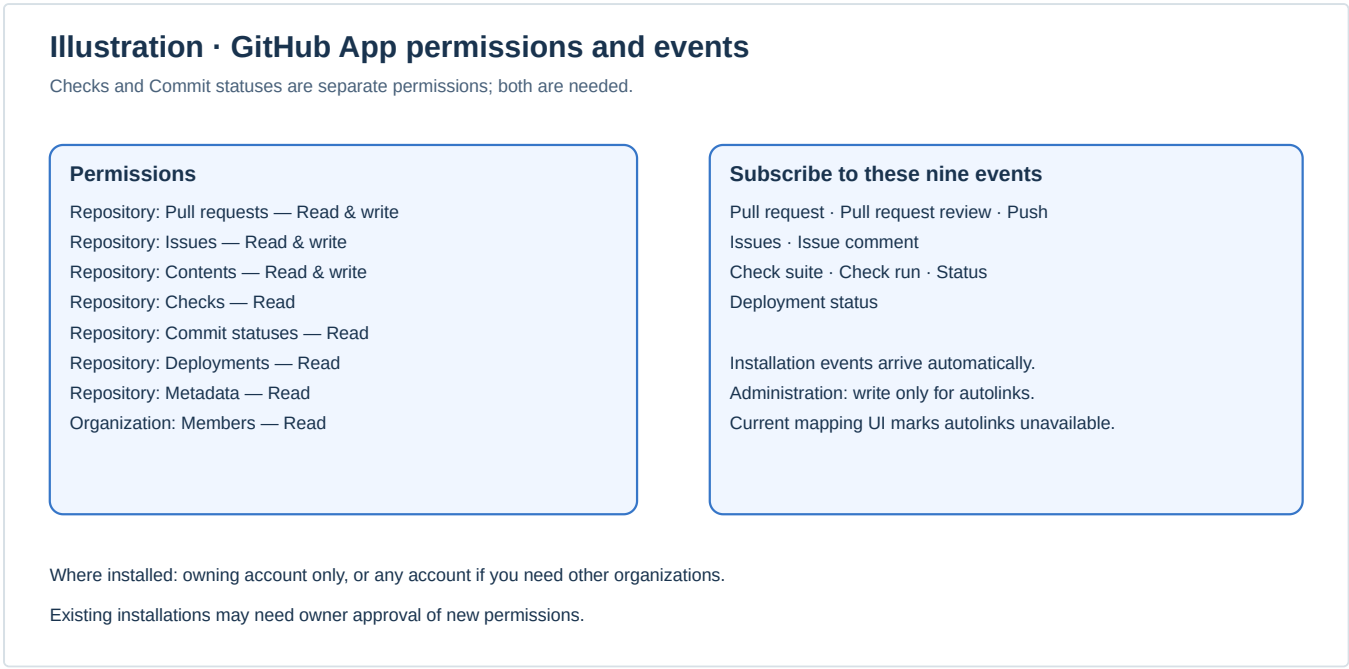


Figure 22.1 Illustration: the required permissions and nine event subscriptions.

6. *Where can this GitHub App be installed?*
   - Choose *Only on this account* if all repositories belong to the App's owner.
   - Choose *Any account* if other accounts or organizations will install it.
   - This choice does not grant access to any repository by itself.
7. Select **Create GitHub App**.

> **Screenshot needed** — gh-02-app-permissions — the created App's "Permissions & events" page showing the eight permissions and nine events, secrets/IDs masked. Capture later with npm run manual:capture -- --only gh-02-app-permissions

Collect the six values

On the App's settings page:

Value	Where on GitHub	Looks like	Goes to God mode field
App ID	"About" section	A number, e.g. `1234567`	**App ID**

Value	Where on GitHub	Looks like	Goes to God mode field
Client ID	“About” section	Text starting with `Iv` on GitHub.com (an OAuth App’s `Ov…` ID is the wrong kind of registration)	**OAuth client ID**
App slug	The last part of the App’s public link `https://github.com/apps/<slug>`	e.g. `acme-plane-sync`	**App slug**
Client secret	*Generate a new client secret*	Shown once	**OAuth client secret**
Private key	*Generate a private key* — a `.pem` file downloads	`-----BEGIN RSA PRIVATE KEY-----…`	**Private key (PEM)** (paste the file’s contents)
Webhook secret	You generated it	64 hex characters	**Webhook secret**

The App ID and the Client ID are different values and cannot be swapped. The slug is a URL name, not the numeric ID.

Secret	Authenticates
Private key	The App itself (JWT and installation tokens)
Client secret	Personal account authorization (OAuth code exchange)
Webhook secret	Incoming GitHub events

Keep all three private. If GitHub no longer shows a secret, generate a new one rather than trying to recover it.

Next: enter the values in God mode (chapter 23, “Enter the GitHub App”).

GitHub Enterprise

The same three git-sync URLs are used with GitHub Enterprise. Register the App *on that GitHub host*: a GitHub.com App cannot be installed on an unrelated Enterprise Server.

Edition (God mode GitHub edition)	Extra field	API base	Web/OAuth/install origin
GitHub.com	—	`https://api.github.com`	`https://github.com`
Enterprise Cloud with data residency	**Enterprise Cloud subdomain**, e.g. `octocorp` (“The enterprise name before .ghe.com.”)	`https://api.octocorp.ghe.com`	`https://octocorp.ghe.com`
Enterprise Server	**Enterprise Server URL**, e.g. `https://ghe.example.com`	`https://ghe.example.com/api/v3`	`https://ghe.example.com`

Changing the App later

- **Permissions or events changed on GitHub:** existing installations must accept the new permissions (GitHub asks the owner). God mode’s **App permissions** check reads the *registration*, not the installation’s acceptance. Ask owners to approve, then press **Refresh connection** in the workspace.
- **A new private key, client secret or webhook secret:** save it in God mode with the other secret fields left empty. For the webhook secret, change GitHub and God mode close together, then redeliver any failed deliveries from **Advanced** → **Recent deliveries** (chapter 29).
- *A different App* (new App ID): personal tokens issued for the old App no longer work. Every member must **Reconnect**, and installations must be connected again.

If it went wrong

Symptom	Cause	Fix
GitHub: `redirect_uri is not associated with this application`	Callback URL differs from `<PUBLIC_ORIGIN>/git-sync/oauth/github/callback`, or wrong Client ID	Compare character by character; use the App's Client ID
The Setup URL field is greyed out	**Request user authorization (OAuth) during installation** is on	Turn it off, then enter the Setup URL and **Redirect on update**
God mode **App permissions: Needs attention** — "The App registration is missing required permission levels…"	A permission missing or too low	Fix the permissions table, then owners approve
God mode **App webhook events: Needs attention** — "Subscribe the App to the missing webhook events."	An event not ticked	Tick all nine
Signed GitHub deliveries: Not verified	No GitHub event received yet, or wrong webhook URL/secret	Make a PR change; check **Advanced** → **Recent deliveries** on GitHub for 401/404
Deliveries show `401` on GitHub	Webhook secret differs between GitHub and God mode	Save the same secret in both; redeliver
Deliveries show `503`	git-sync cannot decrypt its stored secret	See chapter 30, "Credentials cannot be decrypted"

What you learned

- Register once, install per account, authorize per person — three separate GitHub steps.
- The three URLs: `/git-sync/oauth/github/callback`, `/git-sync/install/github/callback` (with **Redirect on update**), `/git-sync/webhooks/github`.
- Eight permissions (PRs, Issues, Contents read/write; Checks, Commit statuses, Deployments, Metadata read; Organization Members read) and nine events.
- Six values go to God mode: App ID, Client ID, slug, client secret, private key, webhook secret.

Source files

- `src/ui/setup-check.ts` — `REQUIRED_APP_PERMISSIONS`, `REQUIRED_APP_EVENTS`
- `src/github/hosts.ts` — editions and derived URLs
- `src/ui/oauth-flow.ts`, `src/ui/install-flow.ts` — callback handlers
- `deploy/APPLY.md` — “GitHub App registration”

23 · God mode settings (instance admin)

Everything that applies to the *whole Plane instance* lives on one page, **Git sync: instance settings**. That includes the GitHub App credentials, the setup checks, which installation belongs to which workspace, the magic words and the defaults. Only a Plane **instance administrator** signed in to God mode can open it.

Open the page

1. Open God mode on your Plane (`https://<your Plane>/god-mode/`) and sign in as an instance admin.
2. Choose **GitHub Sync** in the sidebar (“Manage GitHub synchronization.”).

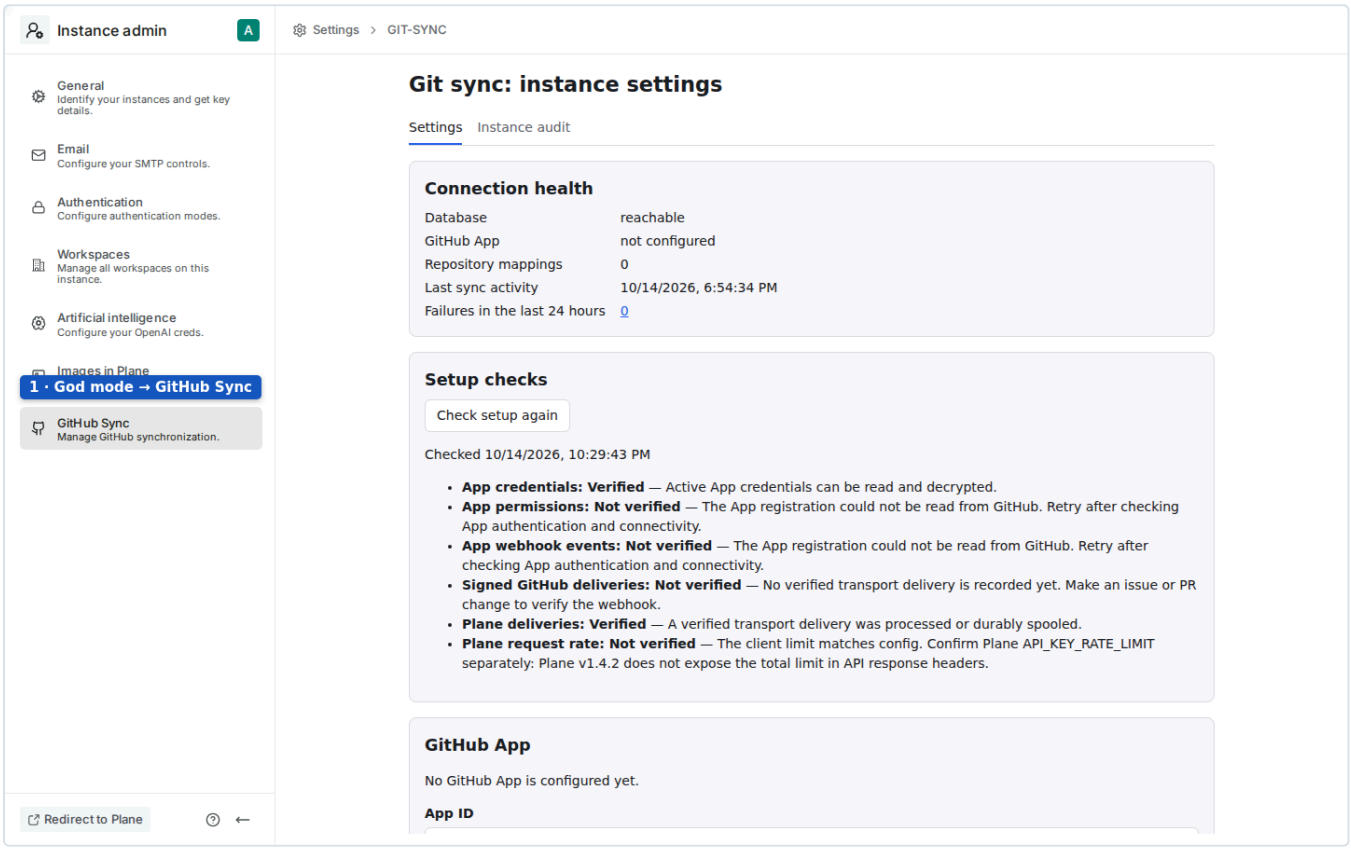


Figure 23.1 Screenshot (test Plane): God mode navigation with GitHub Sync.

Without the UI fork (installed with `--keep-plane-ui`), open `https://<your Plane>/git-sync/ui/admin` directly. You still need a God mode session in the same browser. A normal workspace session gets **Not authorised** with “The instance settings need a Plane instance-admin session.”

The page has two tabs: **Settings** (this chapter) and **Instance audit** (chapter 28). The Settings tab contains, top to bottom:

Section	Purpose
Connection health	Is the database reachable, is an App configured, how many mappings, latest activity, recent failures
Setup checks	Six read-only checks of credentials, App registration, deliveries and rate limit
GitHub App	Enter or replace the App's credentials and GitHub edition
Workspace GitHub App installations	Allow-list which installation belongs to which workspace
Disabled repository mappings	Mappings that cannot sync and why
Manual mapping recovery	Map a repository by ID when nobody can authorize personally
Magic words	The closing / non-closing / relation phrase lists
Defaults	Smart Commits default, signed-commit preference, reconciliation interval, echo TTL

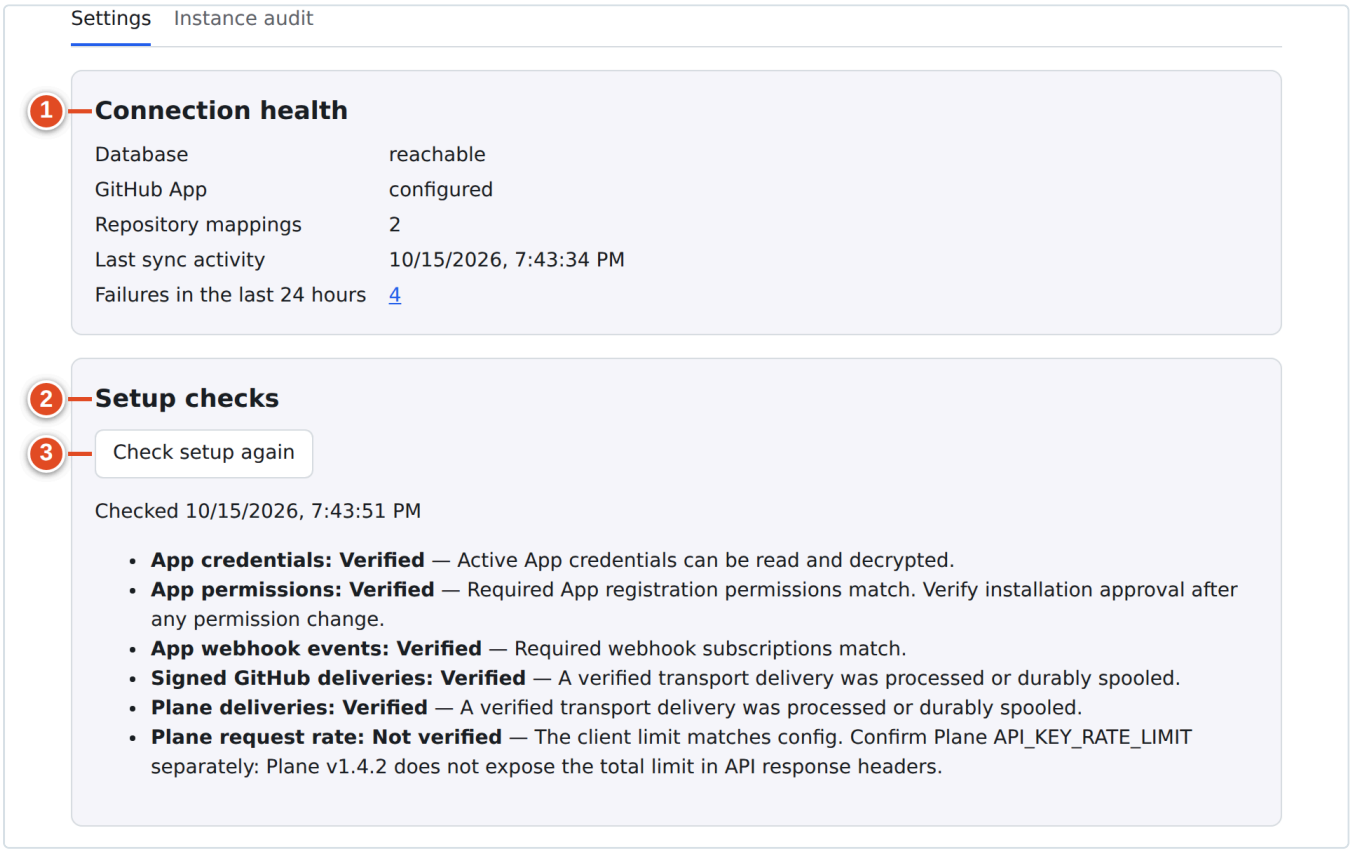


Figure 23.2 Screenshot (demo instance): Git sync: instance settings — (1) Connection health; (2) Setup checks; (3) Check setup again.

Connection health

Card	Meaning
Database — reachable / unreachable	git-sync can query its `gitsync` database
GitHub App — configured / not configured	App credentials are stored or supplied by the environment
Repository mappings	Number of mappings across all workspaces (Unknown if unreadable)
Last sync activity	Most recent recorded activity
Failures in the last 24 hours	Count; the link opens the instance audit filtered to errors

If the stored App secrets cannot be decrypted, a red notice appears: **GitHub App credentials cannot be decrypted. Save the App secrets again.** (chapter 30).

Setup checks

Setup checks runs six read-only checks when the page opens; **Check setup again** re-runs them. Each shows **Verified**, **Needs attention** or **Not verified**, with a one-line explanation. Nothing is written anywhere by running them.

Check	Verified when	Typical “Needs attention / Not verified” text and fix
App credentials	“Active App credentials can be read and decrypted.”	“No GitHub App is configured yet…” → fill **GitHub App**. “App credentials cannot be read or decrypted…” → see chapter 30.
App permissions	“Required App registration permissions match. Verify installation approval after any permission change.”	“The App registration is missing required permission levels. Update it and verify installation approval.” → fix permissions on GitHub (chapter 22)
App webhook events	“Required webhook subscriptions match.”	“Subscribe the App to the missing webhook events.” → tick all nine events

Check	Verified when	Typical “Needs attention / Not verified” text and fix
Signed GitHub deliveries	“A verified transport delivery was processed or durably spooled.”	“No verified transport delivery is recorded yet. Make an issue or PR change to verify the webhook.”
Plane deliveries	Same as above, for Plane	Same; check the Plane webhook exists and is enabled
Plane request rate	“Plane’s deployment setting and this service’s request limit match.”	Usually **Not verified**: “The client limit matches config. Confirm Plane API_KEY_RATE_LIMIT separately: Plane v1.4.2 does not expose the total limit in API response headers.” This is normal. **Needs attention** “The active client request limit differs from PLANE_API_RATE_PER_MINUTE. Restart with matching settings.” means the running service and its configuration disagree.

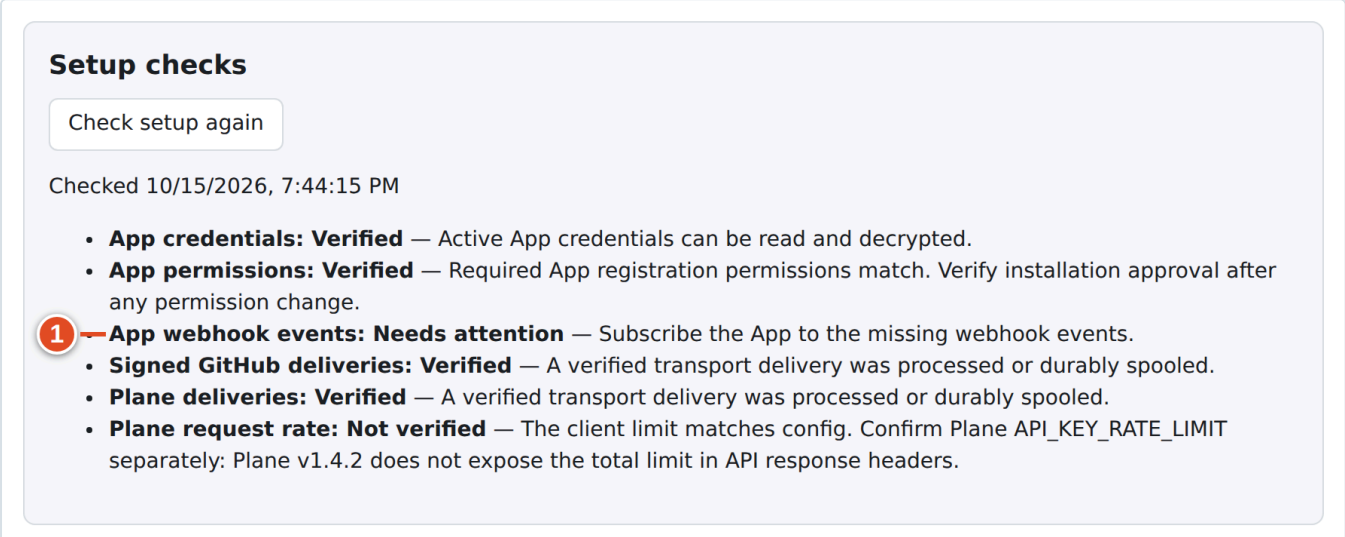


Figure 23.3 Screenshot (demo instance): a setup check that needs attention — (1) the failing check and what to fix.

> **Note: App permissions** and **App webhook events** read the App **registration** with the App’s own identity (`GET /app`). They cannot tell whether an organization owner has already *accepted* changed permissions on an installation. After changing permissions, also check each installation on GitHub.

> **Tip:** “Not verified” for the two delivery checks right after installation just means nothing has happened yet. Open a PR or edit a work item, then press **Check setup again**.

Enter the GitHub App

Goal: git-sync can authenticate as your GitHub App

Before you start: the six values from chapter 22: App ID, private key file, webhook secret, App slug, Client ID and client secret.

Steps

1. Scroll to **GitHub App**. A new instance says “No GitHub App is configured yet.”
2. **App ID** — the number. ①
3. **Private key (PEM)** — paste the whole contents of the downloaded `.pem` file, including the `BEGIN` / `END` lines. ②
4. **Webhook secret** — the value you set on GitHub. ③
5. **App slug** — “Used for the Connect GitHub installation link.” ④
6. **OAuth client ID** and **OAuth client secret** — “Required for personal account authorization.” ④
7. **GitHub edition** — **GitHub.com**, **Enterprise Cloud with data residency** (then **Enterprise Cloud subdomain**), or **Enterprise Server** (then **Enterprise Server URL**). ⑤
8. Select **Save GitHub App**. ⑥

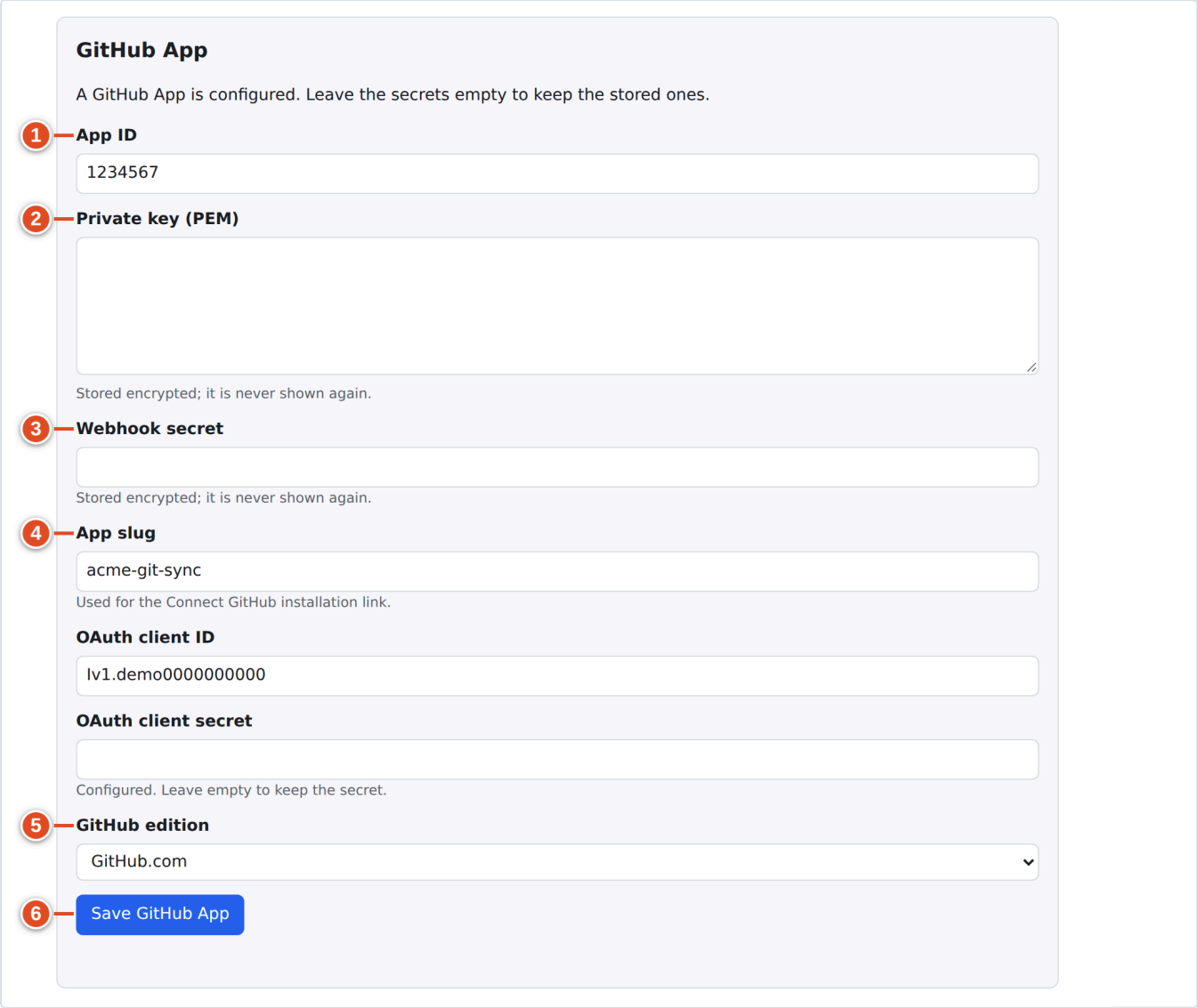


Figure 23.4 Screenshot (demo instance): GitHub App — (1) App ID; (2) Private key (PEM); (3) Webhook secret; (4) App slug and OAuth client; (5) GitHub edition; (6) Save GitHub App.

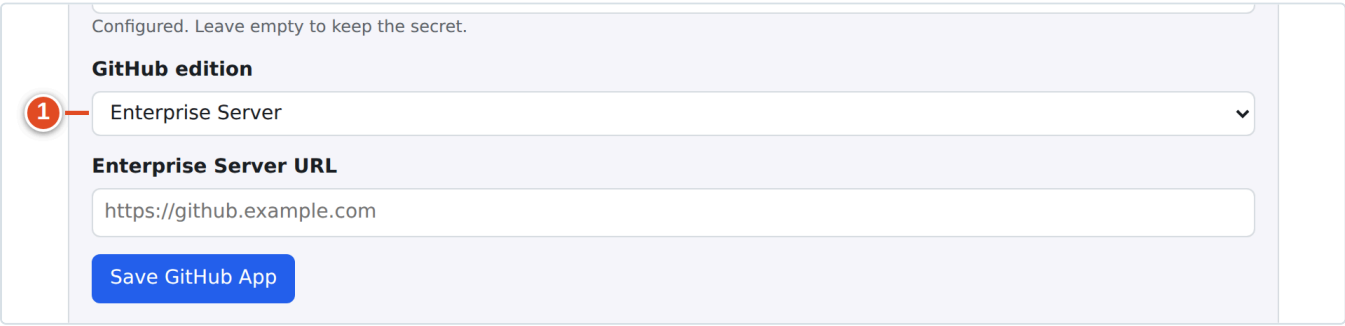


Figure 23.5 Screenshot (demo instance): GitHub edition — (1) GitHub.com, Enterprise Cloud with data residency, Enterprise Server.

Result: the panel now says “A GitHub App is configured. Leave the secrets empty to keep the stored ones.” Secret fields say “Stored encrypted; it is never shown again.” and “Configured. Leave empty to keep the secret.” Press **Check setup again**; **App credentials**, **App permissions** and **App webhook events** should turn **Verified**. The running service picks up new credentials within about five seconds; no restart is needed.

If it went wrong:

- *“appId must be a positive integer”* → you pasted the Client ID into **App ID**.
- *Connect GitHub stays disabled in workspaces* (“Ask the instance admin to configure the GitHub App client ID, client secret and App slug.”) → one of slug, Client ID or client secret is missing.
- *App permissions Not verified* “The App registration could not be read from GitHub…” → wrong private key, wrong App ID, wrong edition/host, or no outbound connectivity.

> **Admin only:** Once you have saved the App in God mode, the stored credentials take precedence over any `GITHUB_APP_ID` / `GITHUB_PRIVATE_KEY_PATH` / `GITHUB_WEBHOOK_SECRET` bootstrap values in `git-sync.env`. git-sync records that the bootstrap is retired, so deleting the stored row later does not quietly revive old environment secrets.

Workspace GitHub App installations (allow-list)

“Allow-list GitHub App installations per workspace. A repository can only be mapped if its installation is allow-listed for that workspace.”

Normally a workspace admin connects an installation with **Connect GitHub** (chapter 24), and git-sync records it here automatically. Use this panel when:

- you want to pre-approve an installation for a workspace,
- an organization owner installed the App directly on GitHub, or
- you need to move or remove an installation.

Goal: allow an installation for one workspace

Steps

1. Find the installation’s numeric ID. On GitHub it is the number at the end of the installation’s settings URL: `…/settings/installations/<id>`.
2. Under **Allow-list an installation**, enter **Installation ID** (“Numeric GitHub App installation ID.”). ②
3. Choose **Workspace** (“Workspace allowed to use this installation.”).
4. Optionally enter **Account / Org (optional)** (“GitHub org or user name for display.”).
5. Select **Allow-list installation**.

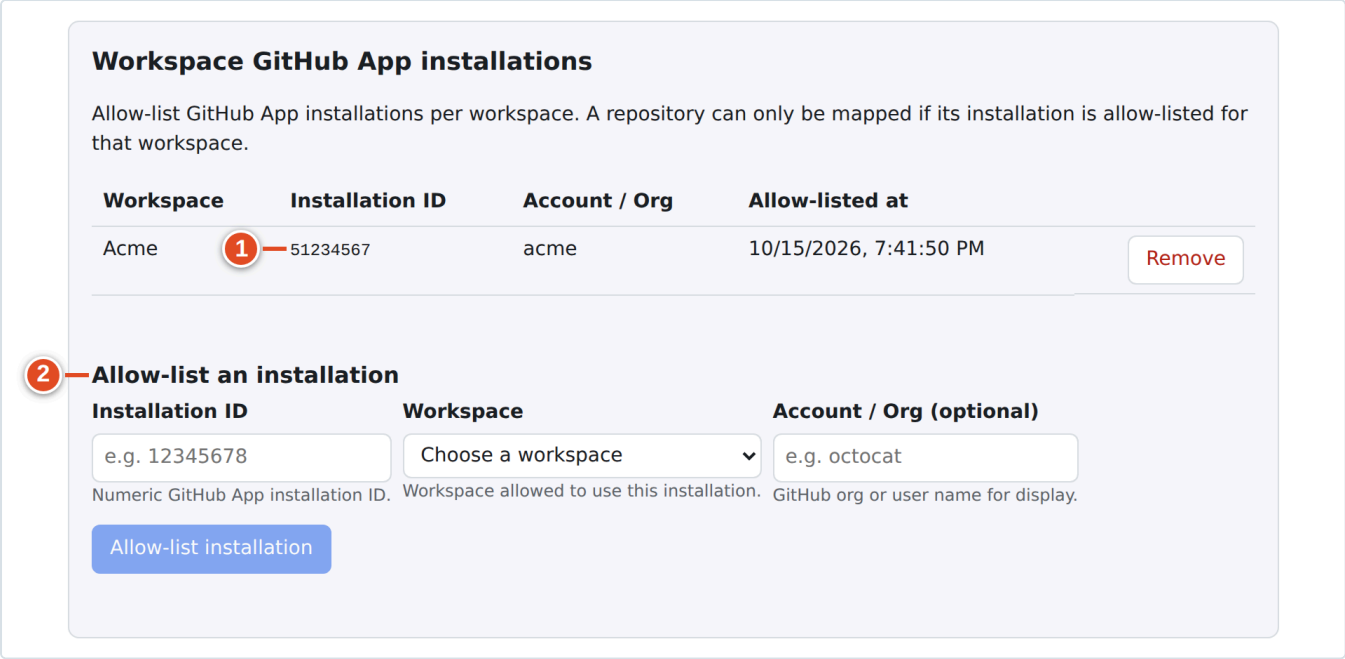


Figure 23.6 Screenshot (demo instance): Workspace GitHub App installations — (1) the allow-listed installation; (2) Allow-list an installation.

Result: the table (**Workspace**, **Installation ID**, **Account / Org**, **Allow-listed at**) shows the entry. git-sync immediately re-checks that workspace's mappings for the installation.

If it went wrong: "That installation is already allowed for another workspace; remove it there first" — one installation serves one workspace. **Remove** the other entry first ("Remove allow-list entry for installation <id>?"). Removing an entry **disables** every mapping that used it.

Disabled repository mappings

"Unresolved mappings and mappings whose installation is no longer allowed are disabled. Allow-list the installation for its workspace, then have a workspace admin save the mapping to verify it."

The table lists **Workspace**, **Repository**, **Project**, **Installation** and **Reason**. The reasons are:

Reason	Meaning	Fix
Save the mapping to verify its installation (installation **Unresolved**)	The mapping has no verified installation yet (for example, it was restored or created before the allow-list)	A workspace admin opens the mapping and presses **Save changes**
Installation is not allowed for this workspace	The installation was removed from the allow-list, uninstalled on GitHub, or moved	Allow-list it again (above), or reconnect it, then save the mapping

Refresh mappings reloads the list. A disabled mapping does not sync anything. Its sync health card in the workspace says **Disabled · <reason>** (chapter 28).

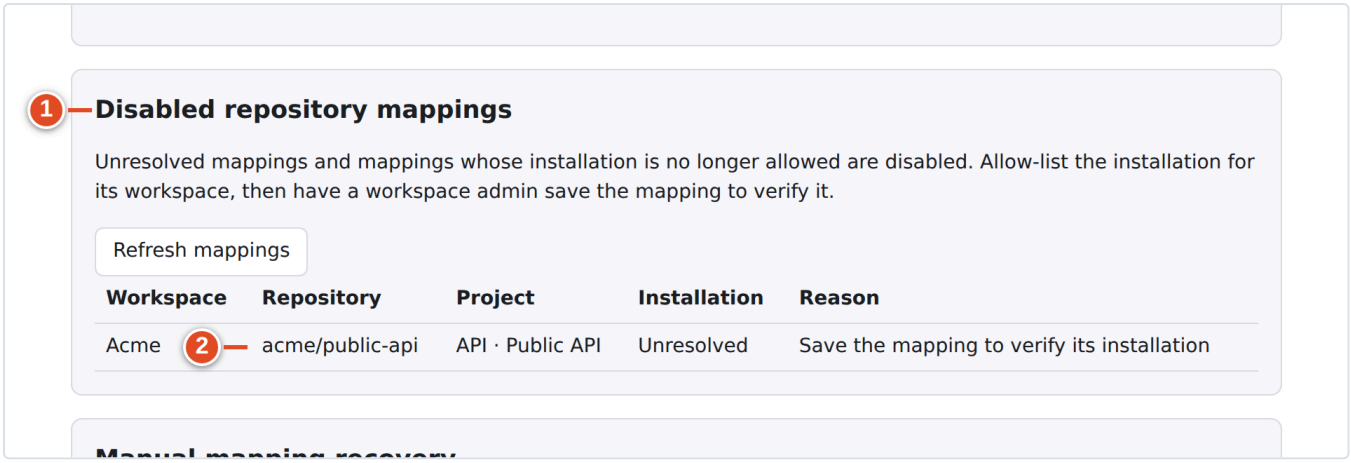


Figure 23.7 Screenshot (demo instance): Disabled repository mappings — (1) the disabled mapping and its reason.

Manual mapping recovery

“Instance admins can map a repository by ID when personal GitHub authorization is unavailable. The App installation and Plane project are still verified.”

Use this when a workspace admin cannot use the normal repository picker, for example because nobody with access can connect a personal GitHub account. It is a recovery tool; prefer **Map a repository** in the workspace.

Goal: map a repository without a personal GitHub link

Steps

1. Choose **Workspace**. ①
2. Choose **Plane project**. ②
3. Enter **GitHub repository** as `owner/repository`, and **GitHub repository ID**. The ID is the repository’s numeric `id` from `GET /repos/{owner}/{repo}` on the GitHub API. ②
4. Select **Add manual mapping**. ③



Figure 23.8 Screenshot (demo instance): Manual mapping recovery — (1) workspace; (2) repository and project; (3) Add manual mapping.

Result: the mapping is created with default settings: two-way, the instance's Smart Commits default, link-back comments on, problem reports off. A workspace admin can then tune it on its **Settings** tab.

If it went wrong:

- Not saved → git-sync still verifies that the repository's installation is allow-listed for that workspace, that the App can read the repository with that ID and name, and that the project belongs to the workspace.
- An ID and name that do not match → refused.

Magic words

The instance-wide phrases that turn a reference in a PR into a closing, non-closing or relation link (chapter 10 explains them for developers).

Box	Default
Closing words	close, closes, closed, closing, fix, fixes, fixed, fixing, resolve, resolves, resolved, resolving, complete, completes, completed, completing, implement, implements, implemented, implementing
Non-closing words	ref, refs, references, part of, contributes to, toward, towards

Box	Default
Relation phrases	relates to, related to

Goal: change the vocabulary your teams use

Steps

1. Edit the boxes: “One word or phrase per line. A word may belong to only one tier.” ①
2. Select **Save magic words**. ②

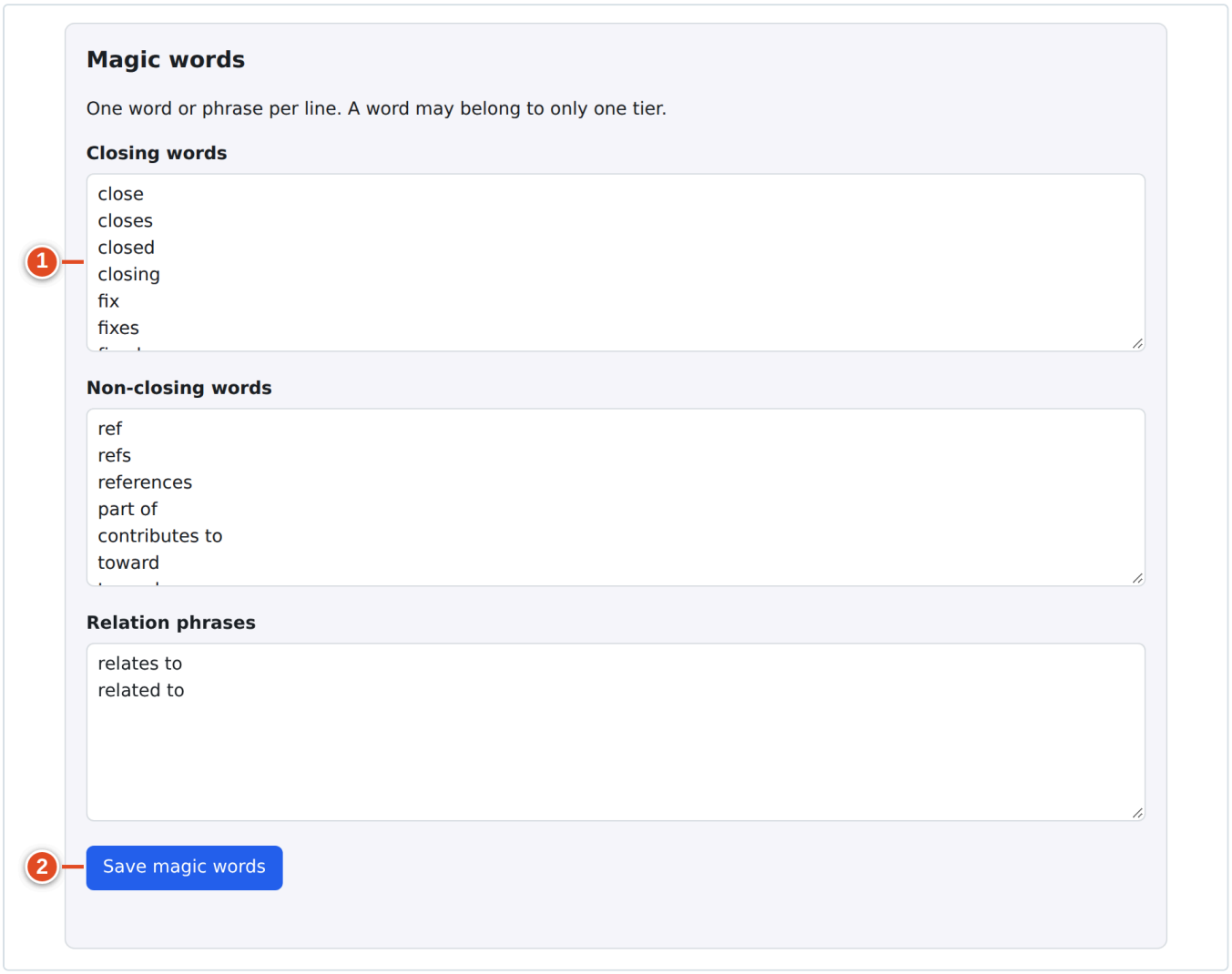


Figure 23.9 Screenshot (demo instance): Magic words — (1) one tier per box, one phrase per line; (2) Save magic words.

Rules the save enforces:

- Only lowercase letters and single spaces (no digits, hyphens or punctuation).
- At most 40 characters per phrase, and 1–100 phrases per tier.
- No tier may be empty.

- A phrase in two tiers is refused with `"<word>" is in both <tier> and <tier>`.

Result: the new lists apply to the next PR event in every workspace. Existing links keep their tier.

> **Warning:** Removing a closing word your teams use (for example `fixes`) silently turns `Fixes WEB-42` into a bare body reference, which is only a relation. Announce changes.

Defaults

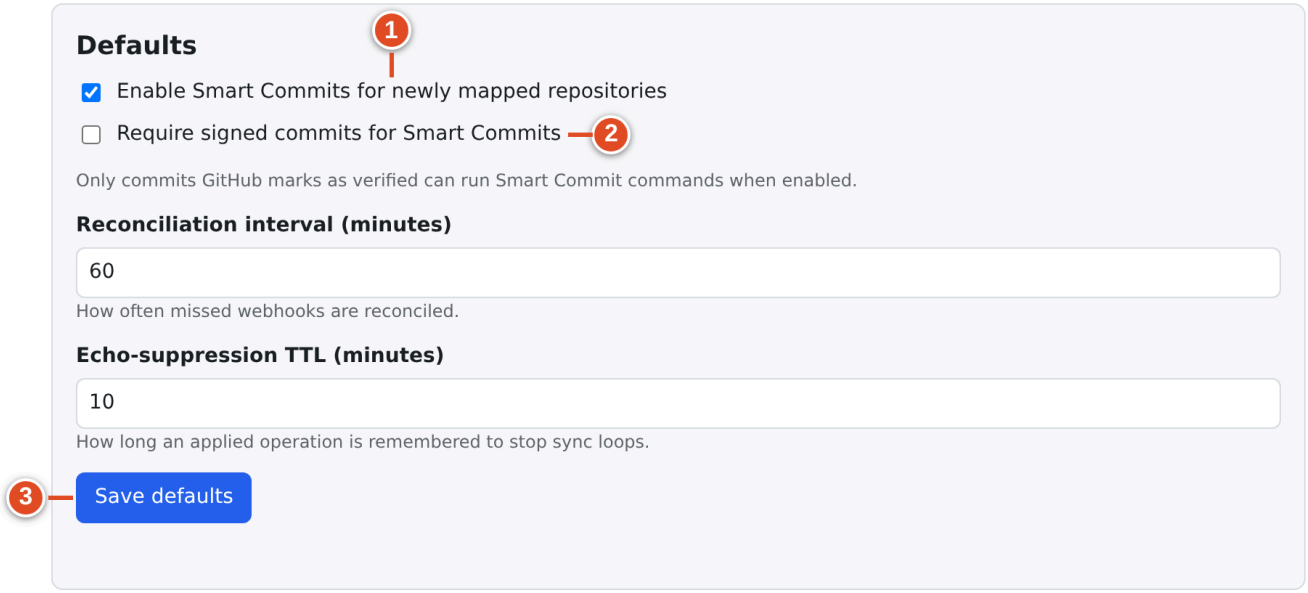


Figure 23.10 Screenshot (demo instance): Defaults — (1) Smart Commits for new mappings; (2) Require signed commits — stored but not enforced in this release; (3) Save defaults.

Field	Default	Effect
Enable Smart Commits for newly mapped repositories	On	Pre-ticks **Smart Commits** in the **Map a repository** form. Existing mappings are not changed.
Require signed commits for Smart Commits	The value of `REQUIRE_VERIFIED_COMMITS`	*Stored but not enforced in this release.* See the warning below.
Reconciliation interval (minutes)	60	“How often missed webhooks are reconciled.” Also how quickly a silent Plane webhook is noticed.

Field	Default	Effect
Echo-suppression TTL (minutes)	10	“How long an applied operation is remembered to stop sync loops.” Too long can swallow a genuine identical edit; too short can let an echo through.

Select **Save defaults** to store them.

> **Warning:** The checkbox **Require signed commits for Smart Commits** (“Only commits GitHub marks as verified can run Smart Commit commands when enabled.”) is saved, but the Smart Commit push handler does not read it. It only reads the environment variable `REQUIRE_VERIFIED_COMMITS` (`src/app/push.ts:55`). To enforce signed commits:
>
> 1. Set `REQUIRE_VERIFIED_COMMITS=true` in `git-sync.env`.
> 2. Restart git-sync with the full Compose command: `docker compose … up -d git-sync`.
>
> When it is on, an unsigned commit’s commands are skipped and audited. If **Report problems** is enabled on the mapping, git-sync posts “This commit is not signature-verified; Smart Commit commands were ignored.”

What you learned

- God mode → **GitHub Sync** (`/git-sync/ui/admin`) holds everything instance-wide and needs a God mode session.
- **Setup checks** verify credentials, App permissions, the nine events, deliveries and the rate limit, without writing anything.
- **GitHub App** secrets are encrypted and write-only; leave fields blank to keep them.
- The installation allow-list binds one installation to one workspace; removing it disables mappings, listed under **Disabled repository mappings**.
- Magic words are validated strictly, and the signed-commit checkbox is not enforced — use `REQUIRE_VERIFIED_COMMITS`.

Source files

- `src/ui/client/Admin.tsx` — the page
- `src/ui/setup-check.ts` — the six checks
- `src/ui/api.ts` — `/instance/*` routes (`github-app`, `installations`, `magic-words`, `settings`, `health`)
- `src/ui/validate.ts` — `parseGithubApp`, `parseMagicWords`, `parseInstanceSettings`
- `src/app/credentials.ts` — credential source and bootstrap retirement
- `src/app/push.ts` — signed-commit enforcement (environment only)

24 · Connect GitHub and map repositories (workspace admin)

Once the instance admin has entered the GitHub App (chapter 23), each workspace admin does two things:

- **connect** a GitHub installation to the workspace, and
- **map** each repository to a Plane project.

Both happen on the workspace's **GitHub Sync** page.

Open the workspace page

1. In Plane, open **Workspace settings → GitHub Sync**. Only workspace *Admins* see this entry.
2. Or open `https://<your Plane>/git-sync/ui/workspaces/<workspace-slug>` directly.

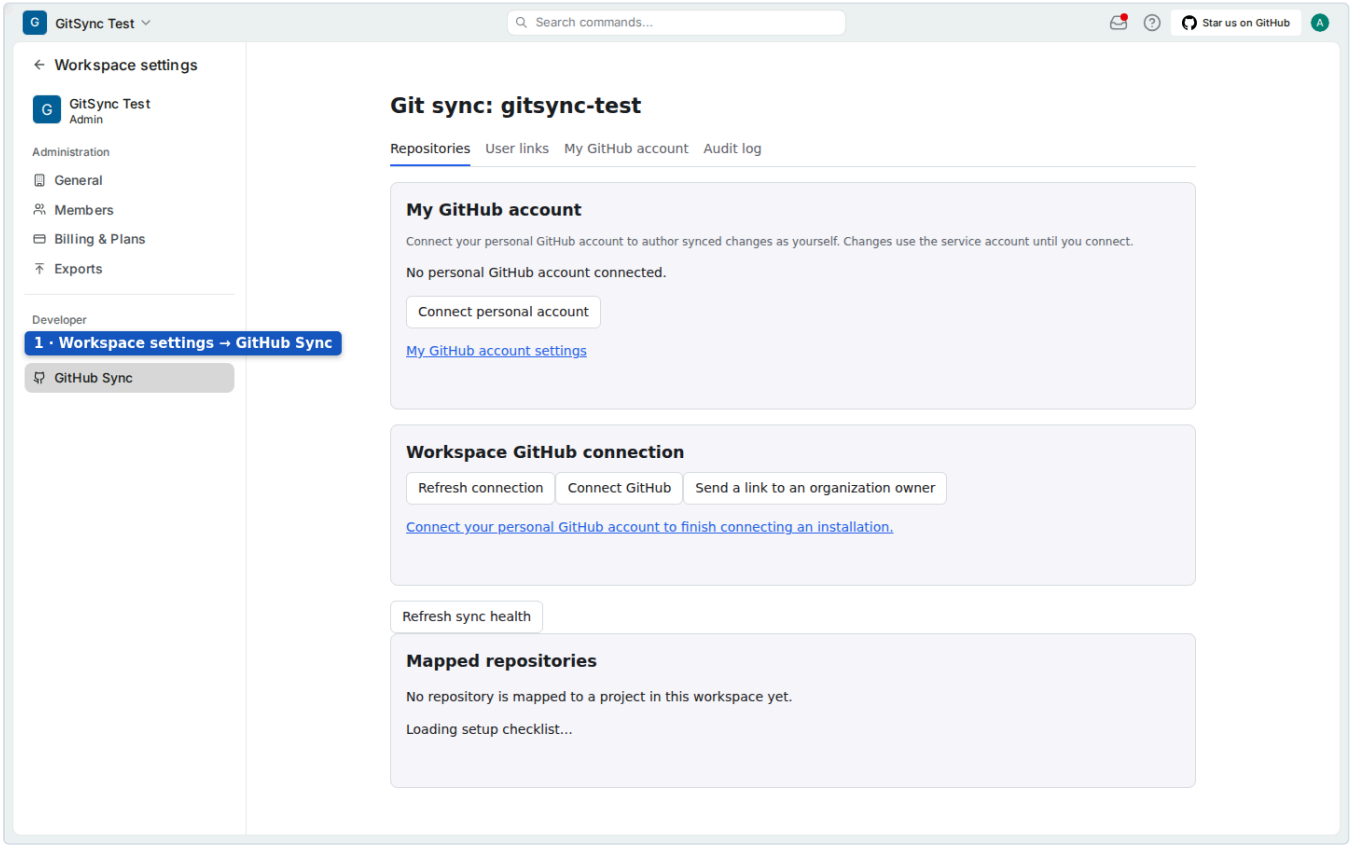


Figure 24.1 Screenshot (test Plane): Workspace settings → GitHub Sync with the embedded page.

The page heading is **Git sync: <slug>**, with four tabs:

Tab	What it holds	Chapter
Repositories	Your personal account, the workspace GitHub connection, mapped repositories with sync health, **Map a repository**	this chapter, 28
User links	Which Plane member is which GitHub account	27
My GitHub account	Your own personal link	8
Audit log	What git-sync did and why	28

A member who is not an admin sees **Not authorised** (“Workspace settings need a workspace admin.”). Members use the direct **My GitHub account** link instead.

The setup checklist

While no repository is mapped, the **Repositories** tab shows **Set up GitHub sync**, a checklist of what is ready:

Line	Meaning
Connect the GitHub App and choose a repository below.	The next action
Rules: shipping defaults are ready; customize them after adding a mapping.	Default lifecycle rules will apply (chapter 26)
Conflict policies: default policies are ready; customize them after adding a mapping.	Default field policies will apply (chapter 25)
Member accounts: *n* linked of *m*.	How many workspace members have linked GitHub accounts (chapter 27)
App events: verified. / need attention from an instance admin. / not yet verified; ask an instance admin to check setup.	The instance’s **App webhook events** setup check

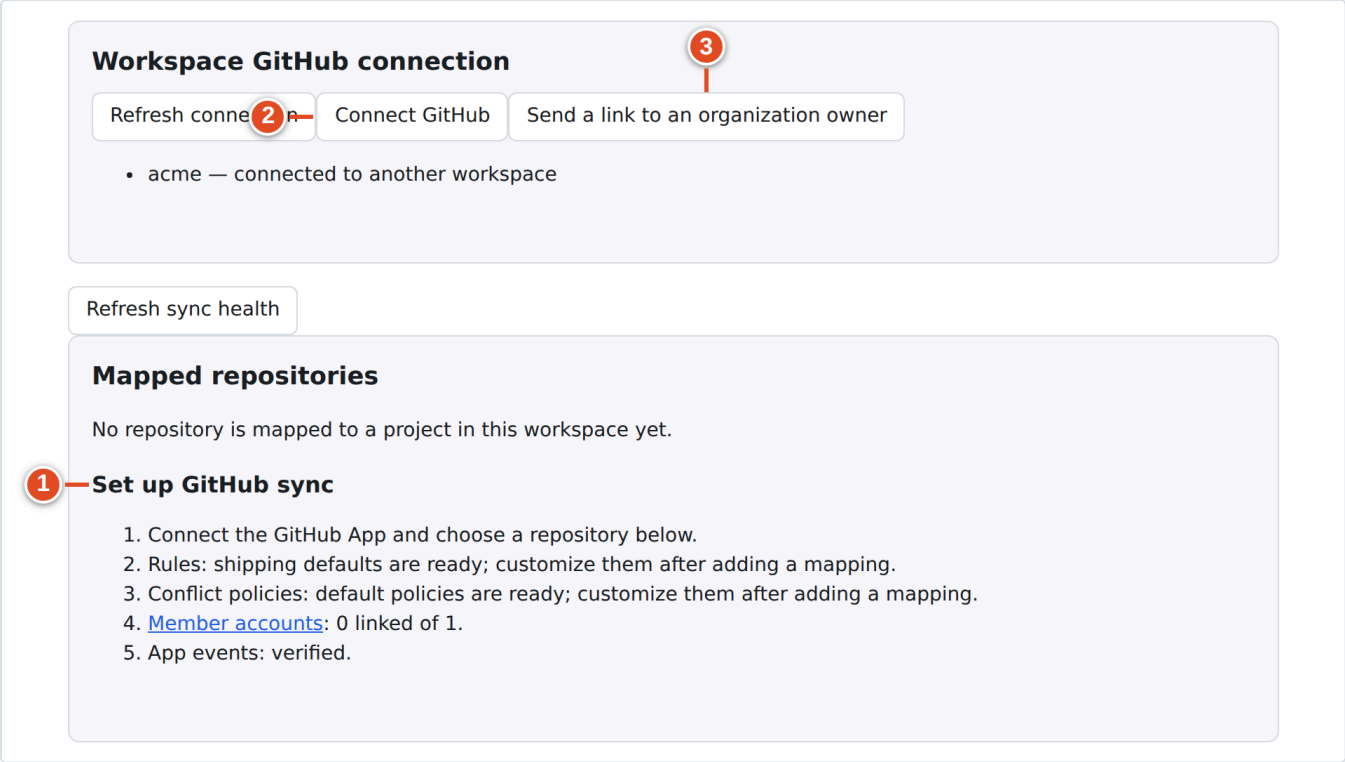


Figure 24.2 Screenshot (demo instance): a new workspace — (1) Set up GitHub sync checklist; (2) Connect GitHub; (3) Send a link to an organization owner.

Connect GitHub to the workspace

Goal: one GitHub installation connected to this workspace

Before you start:

- The instance admin has saved the App, including slug, Client ID and client secret. Otherwise the page says “Ask the instance admin to configure the GitHub App client ID, client secret and App slug.” and the buttons are disabled.
- Connect your **personal** GitHub account first (**My GitHub account** tab → **Connect personal account**). git-sync uses it to prove you may connect the installation, and to list repositories you can pick.

Steps

1. In **Workspace GitHub connection**, select **Connect GitHub**. GitHub’s App installation page opens.
2. On GitHub, choose the **account or organization**, then *All repositories* or *Only select repositories*, and select **Install** (or **Request** if you are not an owner of the organization).
3. GitHub returns you to the Setup URL. git-sync checks the installation:
   - **Personal account:** it must be *your* account.
   - **Organization:** you must be an active **owner or admin**, checked through your personal GitHub token and the App’s *Members: Read* permission.

4. You land back on the page with **GitHub is connected to this workspace.**

Result: the installation appears as a card: the account name, "*Organization* · installation <*id*>", **Manage repository access on GitHub** and **Disconnect**. The instance's allow-list now holds the installation for this workspace.

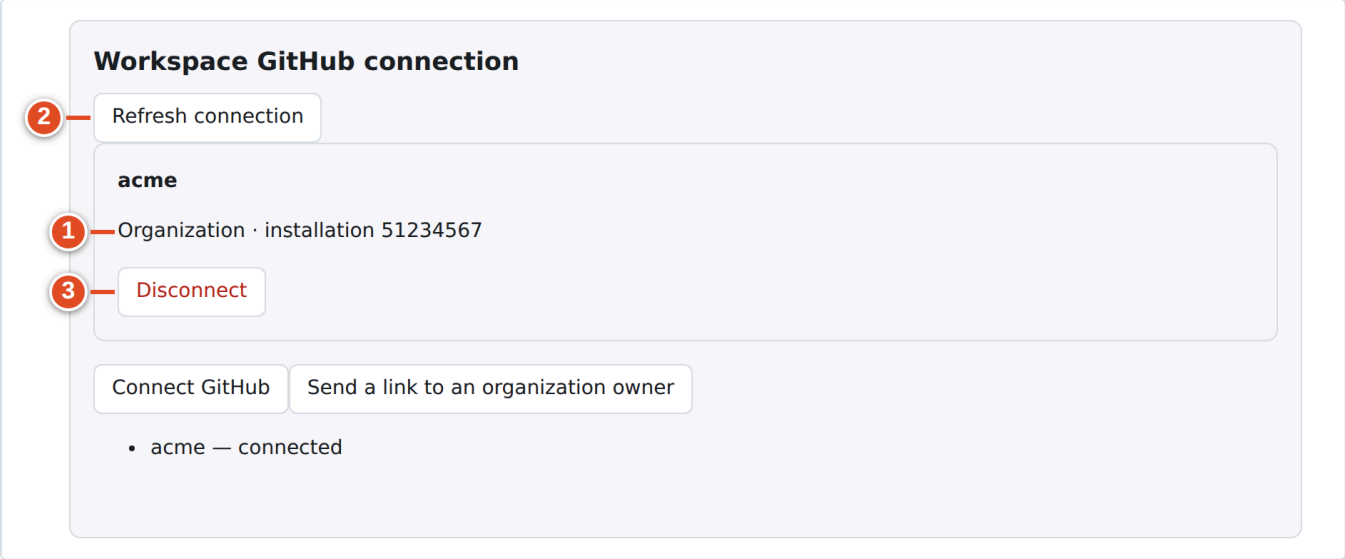


Figure 24.3 Screenshot (demo instance): Workspace GitHub connection — (1) connected organization and installation; (2) Refresh connection; (3) Disconnect.

The whole workspace page, with its four tabs:

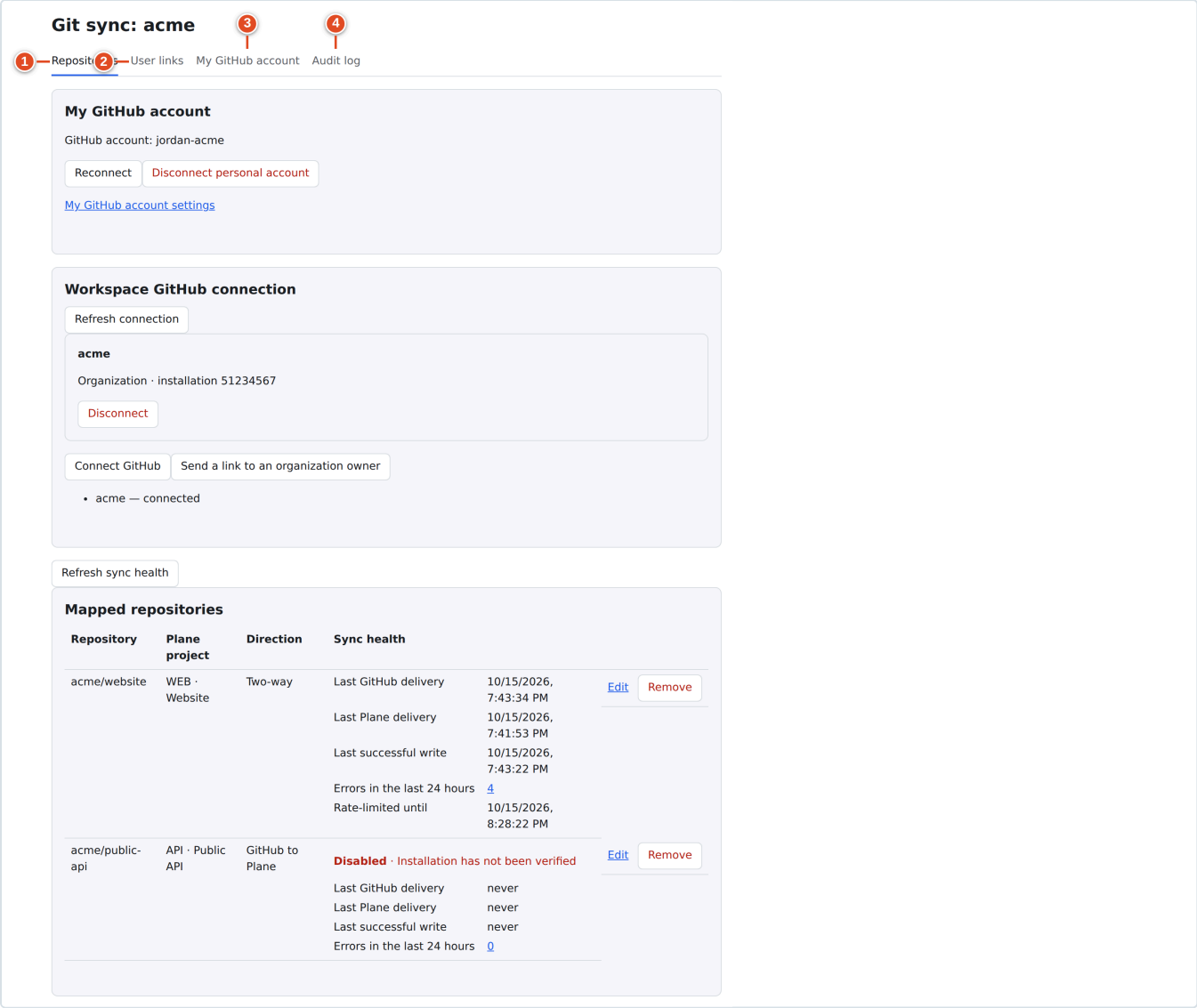


Figure 24.4 Screenshot (demo instance): The workspace page: (1) Repositories; (2) User links; (3) My GitHub account; (4) Audit log.

Other states in the same panel:

You see	Meaning	Do
acme — connected	Already connected to this workspace	Nothing
acme — connected to another workspace	One installation serves one workspace	Ask that workspace's admin, or the instance admin, to free it
acme — available + **Finish connecting**	An installation your GitHub account can access but not yet connected	**Finish connecting**

You see	Meaning	Do
acme — available + “Ask an organization owner to connect it” + **Send owner handoff link**	You are not an owner of that organization	Send a handoff link (below)
Pending organization approval — “*org* — requested by *login*. Waiting for an owner to approve.”	You pressed **Request** on GitHub	Send the owner a handoff link
“Connect your personal GitHub account to finish connecting an installation.”	No personal link yet	Link your account first

Press **Refresh connection** after anything changes on GitHub, for example after an owner approves or you add repositories to the installation.

Ask an organization owner (handoff link)

When the App must be installed or approved by an organization owner who does not use Plane:

1. Select **Send a link to an organization owner** (or **Send owner handoff link** next to a pending request).
2. If asked “Which GitHub organization should the owner connect? Enter its login.”, type the organization’s login, for example `acme`.
3. A panel **Send a link to an organization owner** shows the **Installation link**, **Copy link** and when it **Expires** (7 days). Send the link only to the intended owner.
4. The owner opens it, installs or approves the App on GitHub and completes GitHub authorization. They see “The account owner has connected GitHub to this workspace. You can close this page; the workspace admin can refresh the connection.” They need no Plane account.
5. You select **Refresh connection**.

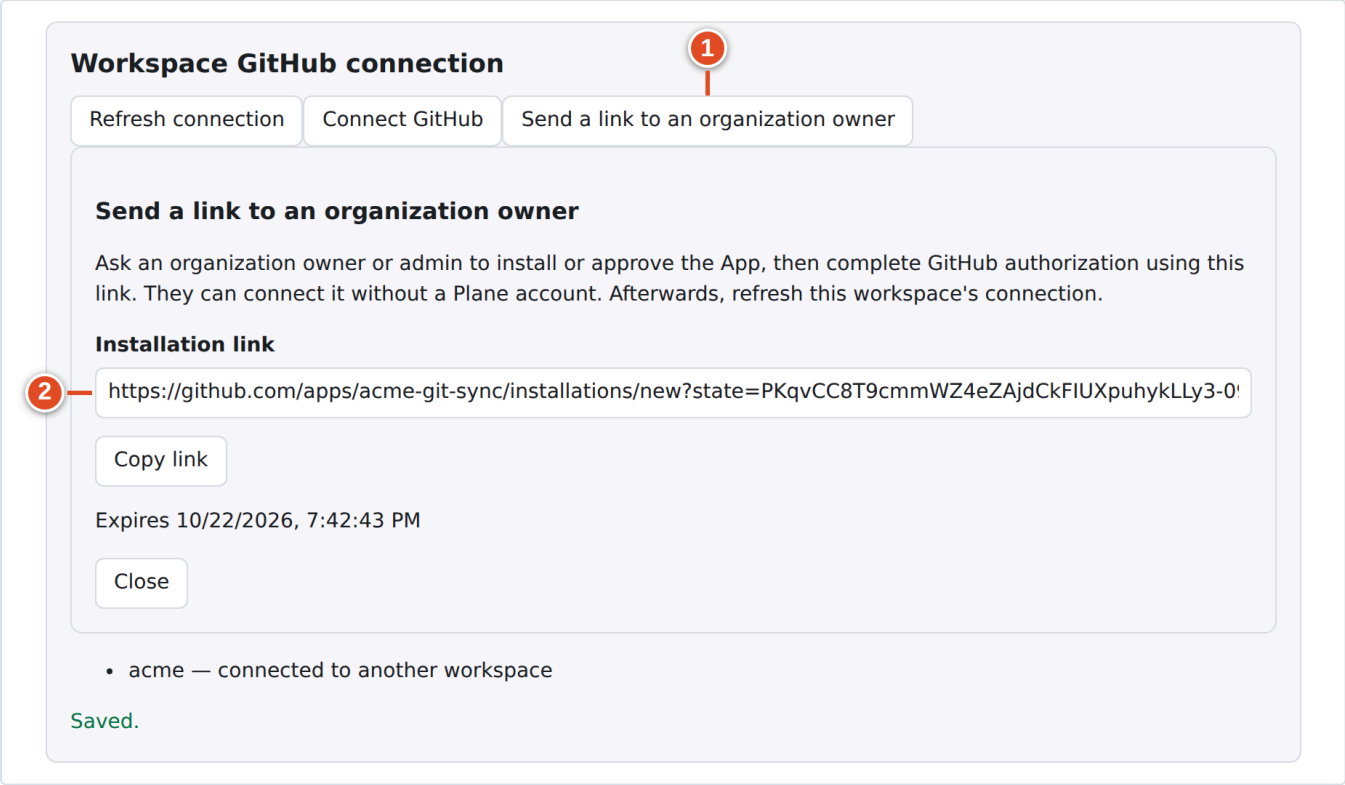


Figure 24.5 Screenshot (demo instance): Send a link to an organization owner — (1) organization field; (2) create the link.

The link is bound to that one organization. If the owner uses it for another organization they get “This handoff link was issued for a different GitHub organization…”. It grants no Plane access.

Disconnect

Disconnect on the installation card asks “Disconnect *login*? Its mappings will be disabled.” After confirming, the installation is removed from this workspace’s allow-list. Every mapping that used it is **Disabled** until it is connected again.

Connection messages

Message	Meaning
state: This connection link has expired, was already used, or belongs to another Plane account. Start again.	Install links last 30 minutes, OAuth links 10 minutes, and each works once
not_yours: Your GitHub account cannot access that installation…	You chose an installation your personal account cannot see
not_owner: Only an active GitHub organization owner or admin can connect its installation…	Use a handoff link
taken: That installation is connected to another workspace.	One workspace per installation
not_configured: Ask the instance admin to configure the GitHub App client ID, client secret and App slug.	God mode incomplete
installation: GitHub did not return a valid installation…	Start again and pick the intended account
Repository access updated and mappings rechecked.	You changed repository selection on GitHub (**Redirect on update**)

Map a repository

Goal: a repository synced with one Plane project

Before you start:

- An installation is connected.
- Your personal GitHub account is linked and can access the repository (Triage, Write, Maintain or Admin).
- The Plane project exists.

Steps

1. Scroll to **Map a repository**.
2. **GitHub repository** ① — pick from “Repositories your account can access in this workspace’s connected installations.” Use **Refresh repositories** after changing the installation on GitHub.
3. **Plane project** ② — the project the repository’s work belongs to.
4. **Open state** / **Closed state** — leave **Project default** unless you need specific states (see below).

5. **Sync direction** ③ — **Two-way** or **GitHub to Plane only**. “Two-way mirrors issues both ways; GitHub to Plane only never writes back to GitHub.”
6. Options ④:
   - **Smart Commits** — pre-ticked from the instance default; lets commit messages run `#comment`, `#time` and state commands.
   - **Post link-back comments on GitHub** — on by default; posts “Linked Plane work item: …” on PRs and issues when a link is first made.
   - **Report problems as comments on GitHub and Plane** — off by default; posts a short notice when a Smart Commit command or a reference fails (“Git sync could not complete this action: …”).
   - **GitHub autolink references (unavailable)** — disabled in this release.
7. Select **Add mapping** ⑤.

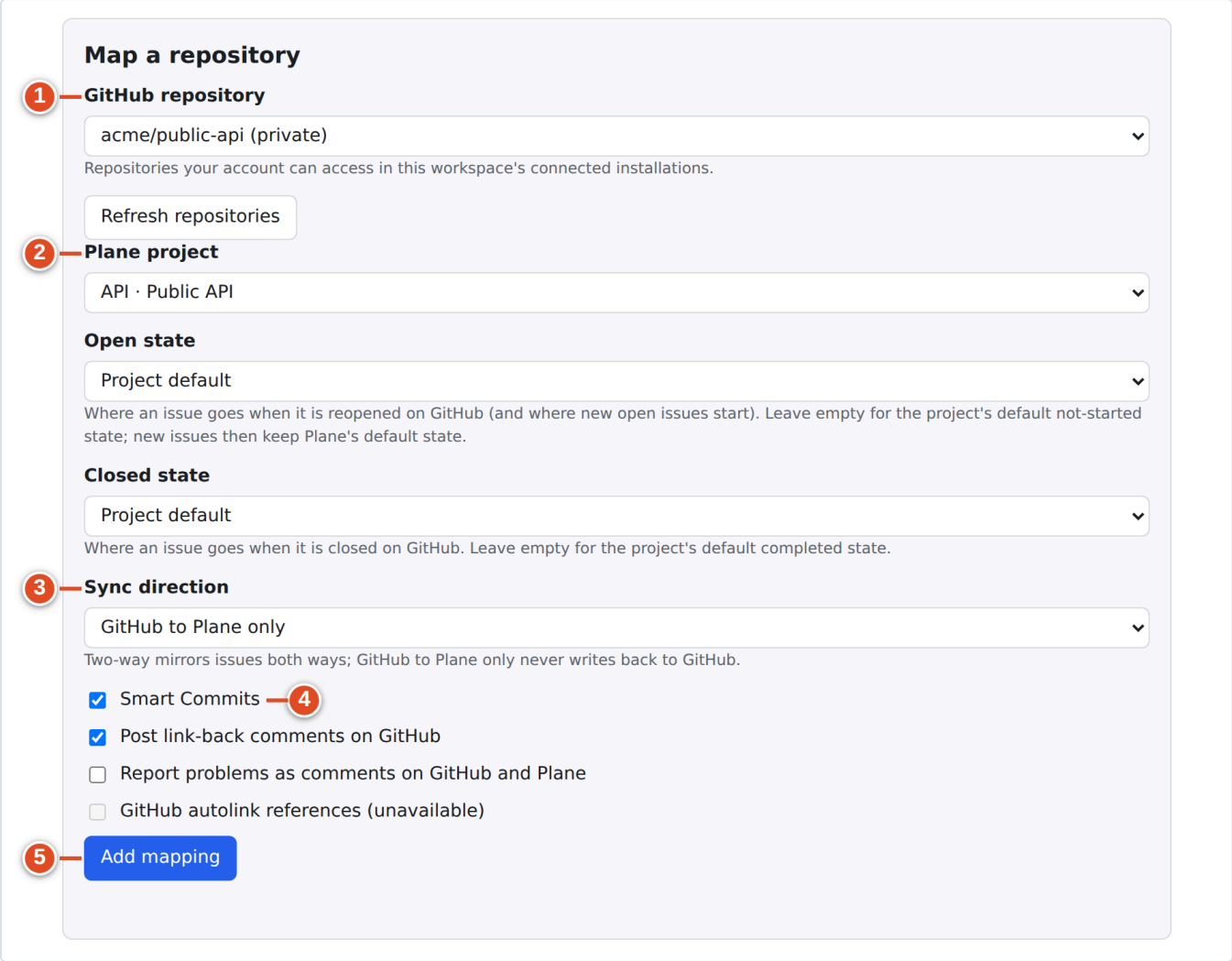


Figure 24.6 Screenshot (demo instance): Map a repository — (1) GitHub repository; (2) Plane project; (3) Sync direction; (4) options; (5) Add mapping.

Result: the panel **Repository mapped** appears with a **Backfill** offer, and the repository appears under **Mapped repositories**.

Open and closed states

These states apply to *mirrored GitHub issues* (chapter 13), not to PR automation.

Field	Used when	Project default means
Open state	"Where an issue goes when it is reopened on GitHub (and where new open issues start)."	The project's default not-started state. New issues keep Plane's default state.
Closed state	"Where an issue goes when it is closed on GitHub."	The project's default completed state

If you choose both, they must be different ("Choose different states for open and closed."). Triage states are never offered.

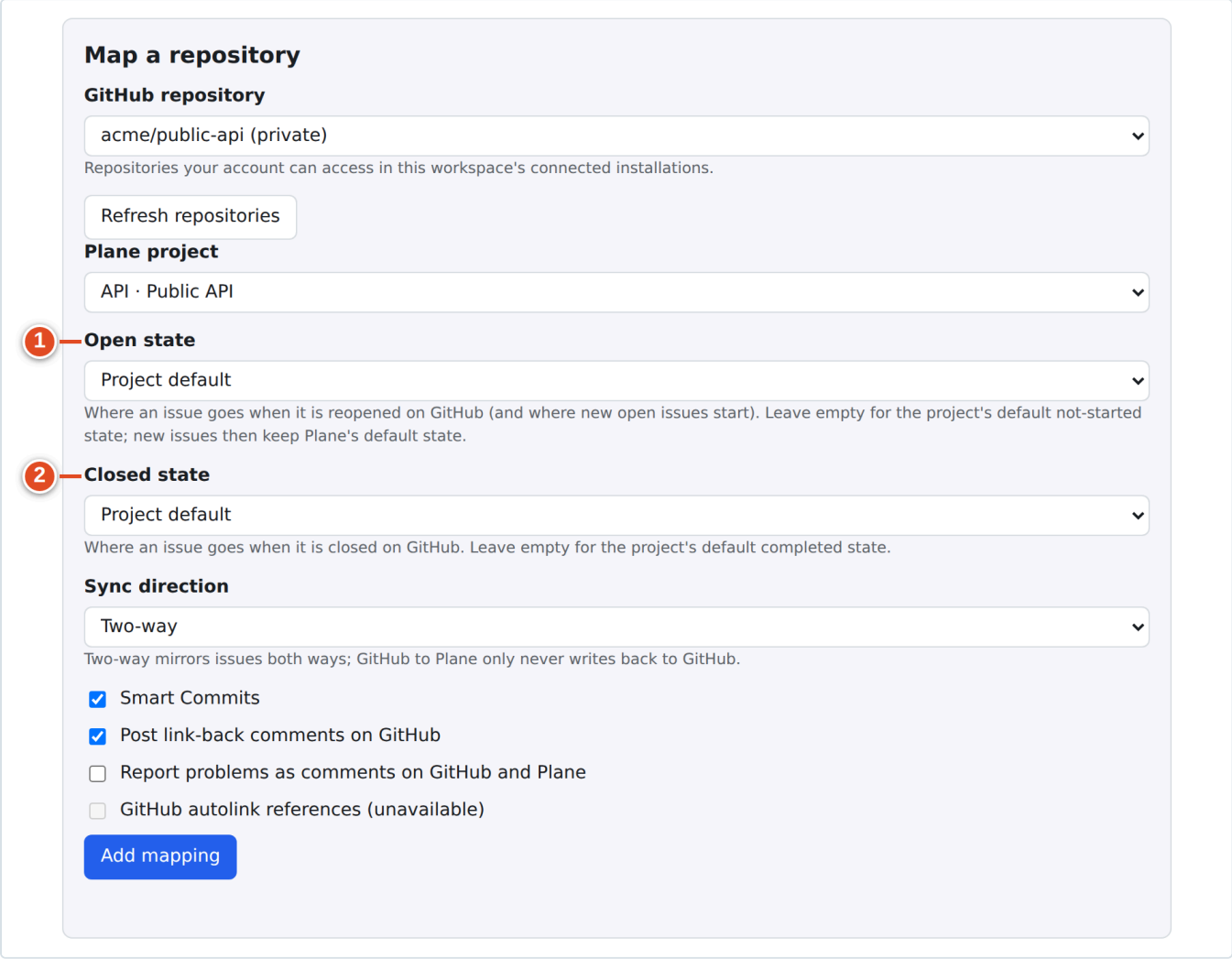


Figure 24.7 Screenshot (demo instance): Open and closed states — (1) Open state; (2) Closed state. Project default leaves Plane's default.

Backfill existing work

Right after **Add mapping**, git-sync offers: “Link this repository’s open pull requests and issues to the Plane work items they mention.”

- **Link existing open PRs and issues now** scans open PRs and issues in batches and links them by the same rules as new events. It reports “Checked *n* pull requests and *m* issues: *k* links made, *u* references did not match a work item.” If there is more, it adds “More items remain; continue to check the next batch.” with **Continue**.
- **Not now** closes the offer. You can run it later from the mapping’s **Settings** tab (**Run backfill**).

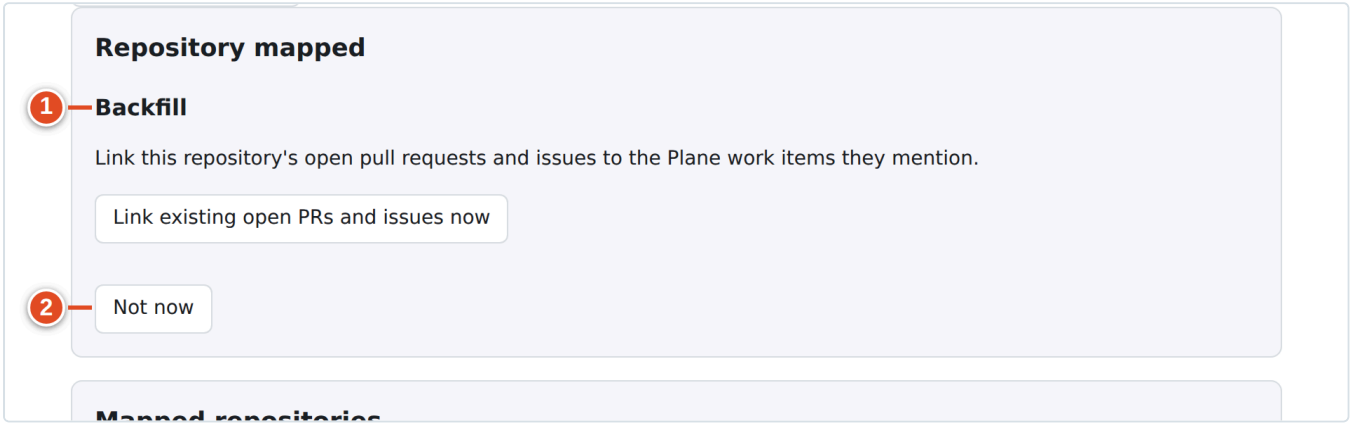


Figure 24.8 Screenshot (demo instance): right after Add mapping — (1) the backfill offer; (2) Not now.

Backfill only **links** open items that already mention a work item key. It does not import closed history, does not create work items and does not mirror issues that lack the `plane` label.

Mapped repositories

The table lists **Repository**, **Plane project**, **Direction** (“Two-way” / “GitHub to Plane”), a **Sync health** card (chapter 28), **Edit** and **Remove**.

- **Edit** opens the mapping’s tabs (chapters 25–26).
- **Remove** asks “Remove the mapping for *owner/repo*? Its rules and conflict policies are deleted too.” Links already made stay in Plane; nothing is deleted on GitHub.

> **Note:** The browser asks first (text recorded from the running UI): “Remove the mapping for acme/website? Its rules and conflict policies are deleted too.”

If it went wrong

Symptom	Cause	Fix
“Connect your personal GitHub account to select a repository.”	No personal link	**My GitHub account** → **Connect personal account**
“Reconnect your personal GitHub account to select a repository.”	Your token was rejected	**Reconnect**
“No repositories available. Connect an installation or ask its owner to grant repository access.”	No connected installation, or it does not include the repository	Connect it; on GitHub add the repository to the installation; **Refresh repositories**
A repository you expect is missing	Your own access is below Triage, or the installation does not include it	Ask for access; adjust the installation
Save refused: “That GitHub installation is not allowed for this workspace”	The repository’s installation is connected elsewhere or not allow-listed	Connect it here or ask the instance admin
A project is missing	The project is archived, or you are not a member	Join the project
Mapping shows **Disabled**	Installation disconnected or removed	Reconnect, then **Save changes** on the mapping

What you learned

- **Workspace settings** → **GitHub Sync** (admins only) has the tabs **Repositories**, **User links**, **My GitHub account** and **Audit log**.
- **Connect GitHub** installs the App and binds that installation to this workspace. Non-owners use a 7-day, organization-bound handoff link.
- **Map a repository** pairs a repository with a project: repository, project, open/closed states, **Sync direction**, Smart Commits, link-back and problem reports.
- **Backfill** links open PRs and issues that already mention keys. **Remove** deletes the mapping with its rules and policies.

Source files

- `src/ui/client/Workspace.tsx` — `MappingsPage`, `MappingForm`, `BackfillPanel`
- `src/ui/client/Connect.tsx` — connection panel, handoff dialog, callback notices
- `src/ui/client/health.tsx` — setup checklist
- `src/ui/install-flow.ts` — install, claim and handoff flows
- `src/ui/mapping-check.ts` — save-time verification
- `src/operations/tooling.ts` — backfill

25 · Mapping settings and conflict policies

Every mapped repository has its own pages. Open them with **Edit** in **Mapped repositories**. The heading is the repository name (for example `acme/website`), with four tabs:

Tab	Address	Purpose
Settings	`/git-sync/ui/workspaces/<slug>/mappings/<id>`	The mapping form plus **Backfill**
Conflict policies	`…/mappings/<id>/policies`	Which side wins for each mirrored field
Rules	`…/mappings/<id>/rules`	What happens in Plane on each GitHub event (chapter 26)
Test rules	`…/mappings/<id>/dry-run`	Try rules without writing anything (chapter 26)

If the mapping no longer exists you see “That mapping does not exist in this workspace.” and **Back to repositories**.

Settings

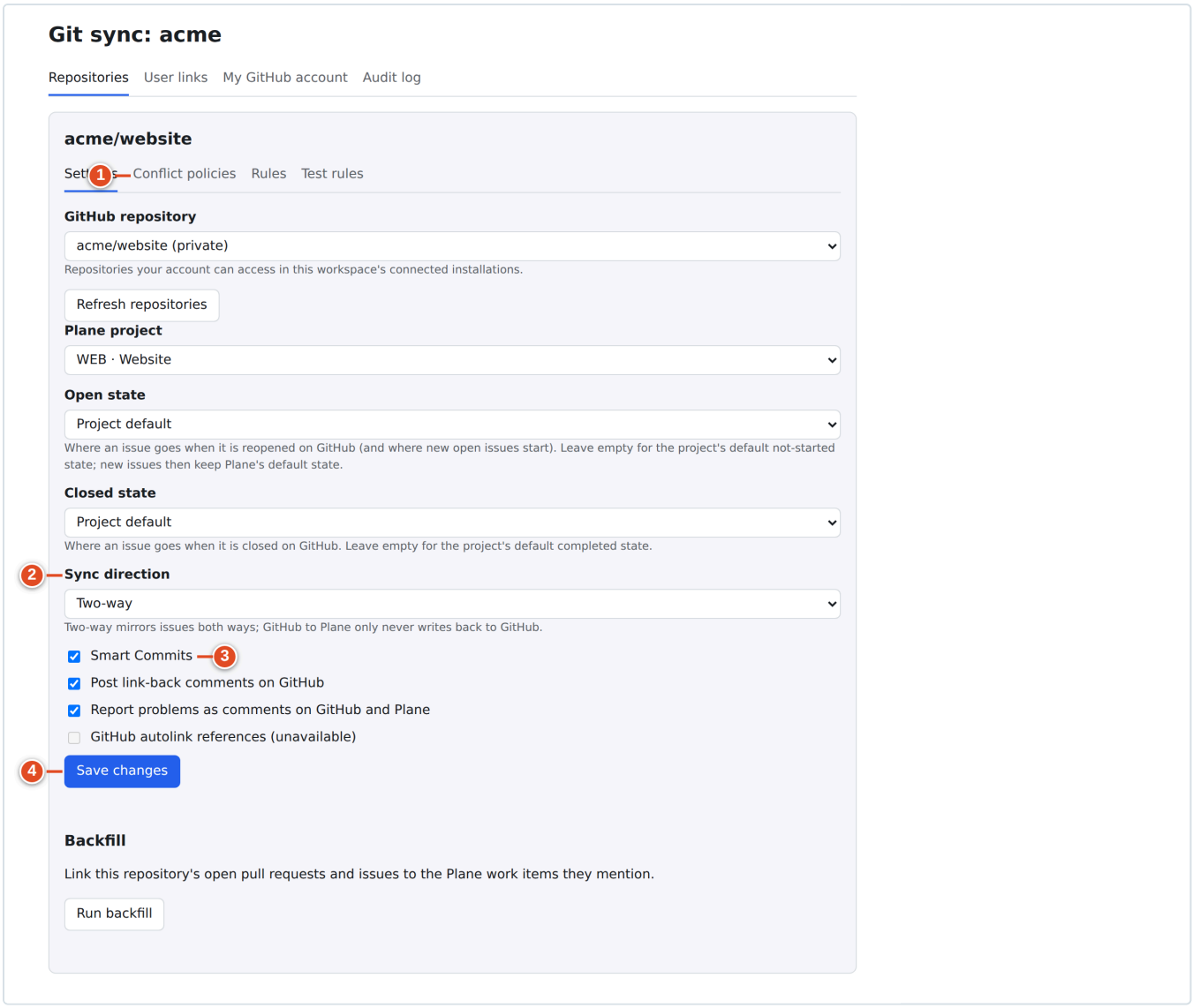


Figure 25.1 Screenshot (demo instance): Mapping Settings tab — (1) tabs; (2) Sync direction; (3) Smart Commits and comment options; (4) Save.

The **Settings** tab shows the same fields as **Map a repository** (chapter 24), filled with the current values, and a **Save changes** button.

Field	Default for a new mapping	Changing it later
GitHub repository	—	Possible, to another repository you can access in a connected installation. Use with care: links and history stay with the old numbers.

Field	Default for a new mapping	Changing it later
Plane project	—	Possible. Custom open/closed states are reset to **Project default**, because states belong to a project.
Open state / Closed state	**Project default**	Affects mirrored issues from then on
Sync direction	**Two-way**	**GitHub to Plane only** stops every Plane-originated mirror write at once
Smart Commits	Instance default (on)	Takes effect for the next push
Post link-back comments on GitHub	On	Only new links get a comment
Report problems as comments on GitHub and Plane	Off	See below
GitHub autolink references (unavailable)	Off, disabled	Not available in this release

Below the form, **Backfill** offers **Run backfill** at any time, with **Continue** for further batches (chapter 24).

Saving re-verifies authority. On every save, git-sync re-checks three things:

- that the project belongs to the workspace,
- that the App's installation (allow-listed for this workspace) can read the repository, and
- for a changed repository, that your own GitHub account can access it.

If something changed meanwhile, the save fails with "The mapping changed during verification. Reload it and try again". Saving is also how you re-enable a mapping listed as "Save the mapping to verify its installation".

Sync direction in practice

	Two-way	GitHub to Plane only
GitHub issue labelled `plane` → creates/updates a Plane work item	Yes	Yes
Plane work item labelled `github` → creates a GitHub issue	Yes	**No** (the label is ignored)
Plane edits (title, description, state, assignees, labels, comments) → GitHub issue	Yes, subject to conflict policies	**No**, never
PR linking, Development panel, Smart Commits, lifecycle rules	Yes	Yes (these write to Plane)
Link-back comments and problem reports on GitHub	If enabled	If enabled

Choose **GitHub to Plane only** when GitHub is the place where issues are managed and Plane is a planning view, or for repositories outside your team's control.

Link-back comments

When git-sync first links a PR or a mirrored issue to a work item, it posts on GitHub. Real text from the demo run:

```
Linked Plane work item:
- WEB-42:
http://plane.example.com/acme/projects/bb57e81d-…/issues/dd4d0dfb-…
```

Text git-sync posted to GitHub in the demo run.

For mirrored issues the text is "Linked to Plane work item: <url>". Each link gets one comment only, the first time. Turn the option off if your team finds the comments noisy; the Development panel in Plane works either way.

Report problems

With **Report problems as comments on GitHub and Plane** on, git-sync tells the author when something they wrote could not be carried out, instead of only recording it in the audit log. Examples:

- a Smart Commit with an unknown state, a bad `#time` duration, or a key that matches no work item;

- an unsigned commit when signatures are required;
- a PR referencing a key that does not exist.

Real text from the demo run (a commit comment on GitHub):

```
Git sync could not complete this action:

- ` WEB-35: #shipped matches no Plane state. `
- ` WEB-35: #time needs a duration such as 1h 30m. `
```

Text git-sync posted to GitHub in the demo run.

The matching Plane comment on WEB-35 reads “Git sync could not complete this action: WEB-35: #shipped matches no Plane state.” Each problem is reported once. Further failures in a burst say “Additional failures are recorded in the sync audit log.”

> **Tip:** Turn **Report problems** on for repositories where developers use Smart Commits. Without it, a mistyped `#in-reveiw` silently does nothing visible.

Conflict policies

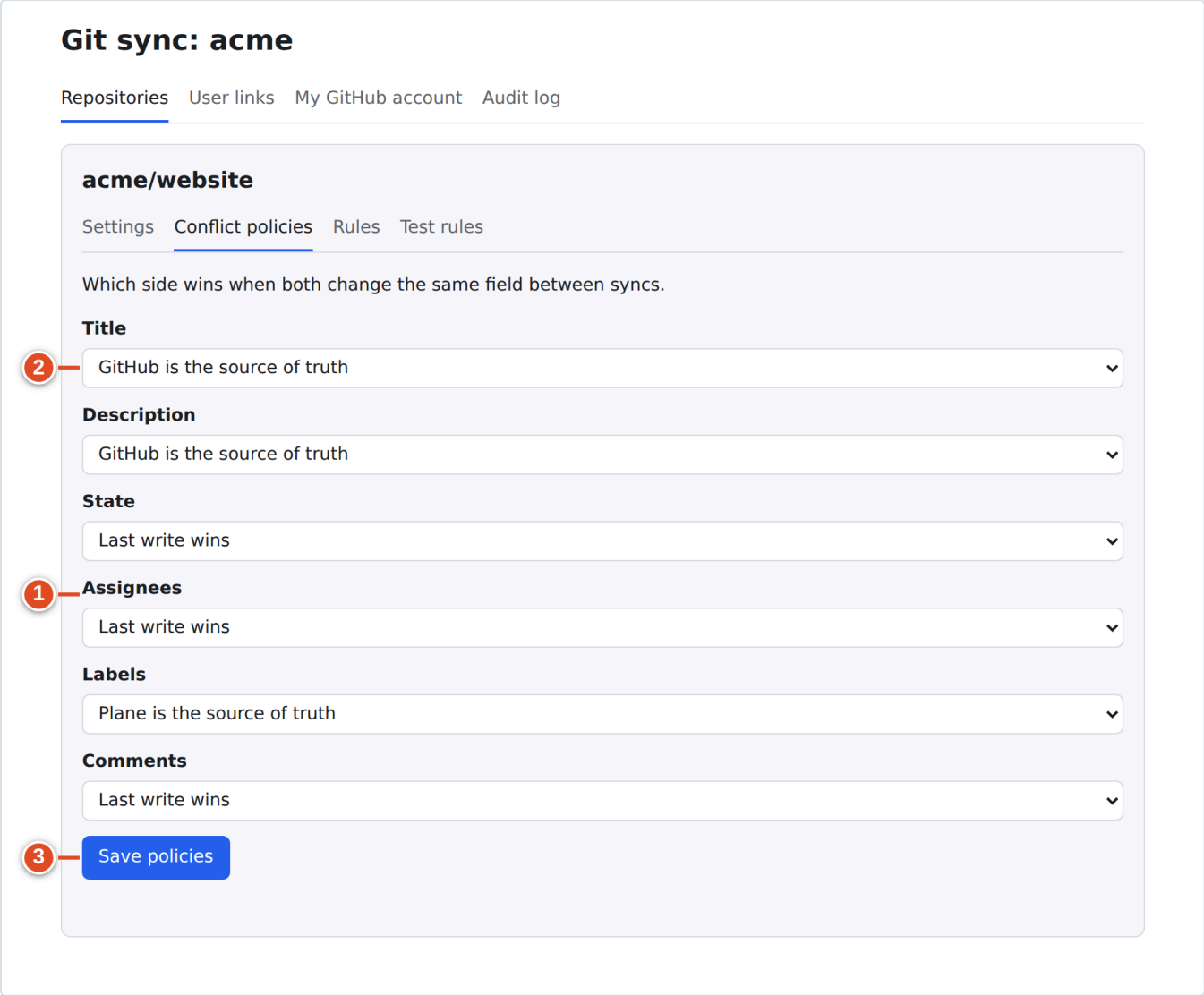


Figure 25.2 Screenshot (demo instance): Conflict policies — (1) one row per field; (2) policy choice; (3) Save policies.

A mirrored issue (chapter 13) exists on both sides. “Which side wins when both change the same field between syncs.” is decided per field:

Field	Default	Options
Title	**GitHub is the source of truth**	**Last write wins**, **GitHub is the source of truth**, **Plane is the source of truth**
Description	**GitHub is the source of truth**	same
State	**Last write wins**	same
Assignees	**Last write wins**	same

Field	Default	Options
Labels	**Last write wins**	same
Comments	**Last write wins**	same

What each policy does to a change:

Policy	A change made on GitHub	A change made in Plane
Last write wins	Copied to Plane	Copied to GitHub
GitHub is the source of truth	Copied to Plane	**Not** copied. Plane keeps its edit until the next GitHub change to that field overwrites it.
Plane is the source of truth	**Not** copied	Copied to GitHub

Last write wins does not compare clocks. Because neither Plane nor GitHub delivers events in strict order, git-sync copies the partner's **current** value when it processes an event. Whichever change was made last is what both sides end up with.

Sync direction overrides policies. With **GitHub to Plane only**, nothing from Plane is ever applied, whatever a field's policy says.

Goal: let Plane own the title and description

Steps

1. Open the mapping → **Conflict policies**.
2. Set **Title** and **Description** to **Plane is the source of truth** (or **Last write wins**).
3. Select **Save policies** (the button shows **Saving…**, then **Saved.**).

Result: from the next edit on, title and description edits in Plane overwrite GitHub, and GitHub edits to those fields no longer reach Plane.

> **Note:** The default makes GitHub own title and description *even on a two-way mapping*. Teams are often surprised that renaming a mirrored item in Plane does nothing on GitHub. That is this policy, not a fault.

> **Warning:** Policies apply to **mirrored issues** only. PR links, Smart Commits and rules write to Plane regardless of policies.

What you learned

- Each mapping has **Settings**, **Conflict policies**, **Rules** and **Test rules** tabs.
- **Settings** re-verifies the project, installation and repository on every save; saving re-enables a mapping waiting for verification.
- **GitHub to Plane only** stops all Plane-originated writes; link-back comments and problem reports are separate switches.
- Conflict policies decide per field which side's change is copied; title and description default to GitHub.

Source files

- `src/ui/client/Workspace.tsx` — `MappingPage`, `MappingForm`, `PoliciesEditor`
- `src/ui/api.ts` — `PATCH /mappings/:id`, `GET/PUT /mappings/:id/conflict-policies`
- `src/ui/validate.ts` — `DEFAULT_CONFLICT_POLICIES`, `parseConflictPolicies`
- `src/sync/mirror.ts` — `shouldApply`
- `src/github/problems.ts` — problem reports

26 · Rules: lifecycle table, branch-aware rules, YAML, versions and Test rules

Rules decide what git-sync does **in Plane** when something happens on GitHub. A typical rule: “when a PR is merged into `main`, move its work items to Done”. Every mapping has one ordered list of rules. You edit it on the mapping’s **Rules** tab and try it safely on **Test rules**.

“Rules decide what happens on each GitHub event. State mappings are rules too, for example when: pull_request.merged then set_state: completed:Done.”

How rules are evaluated

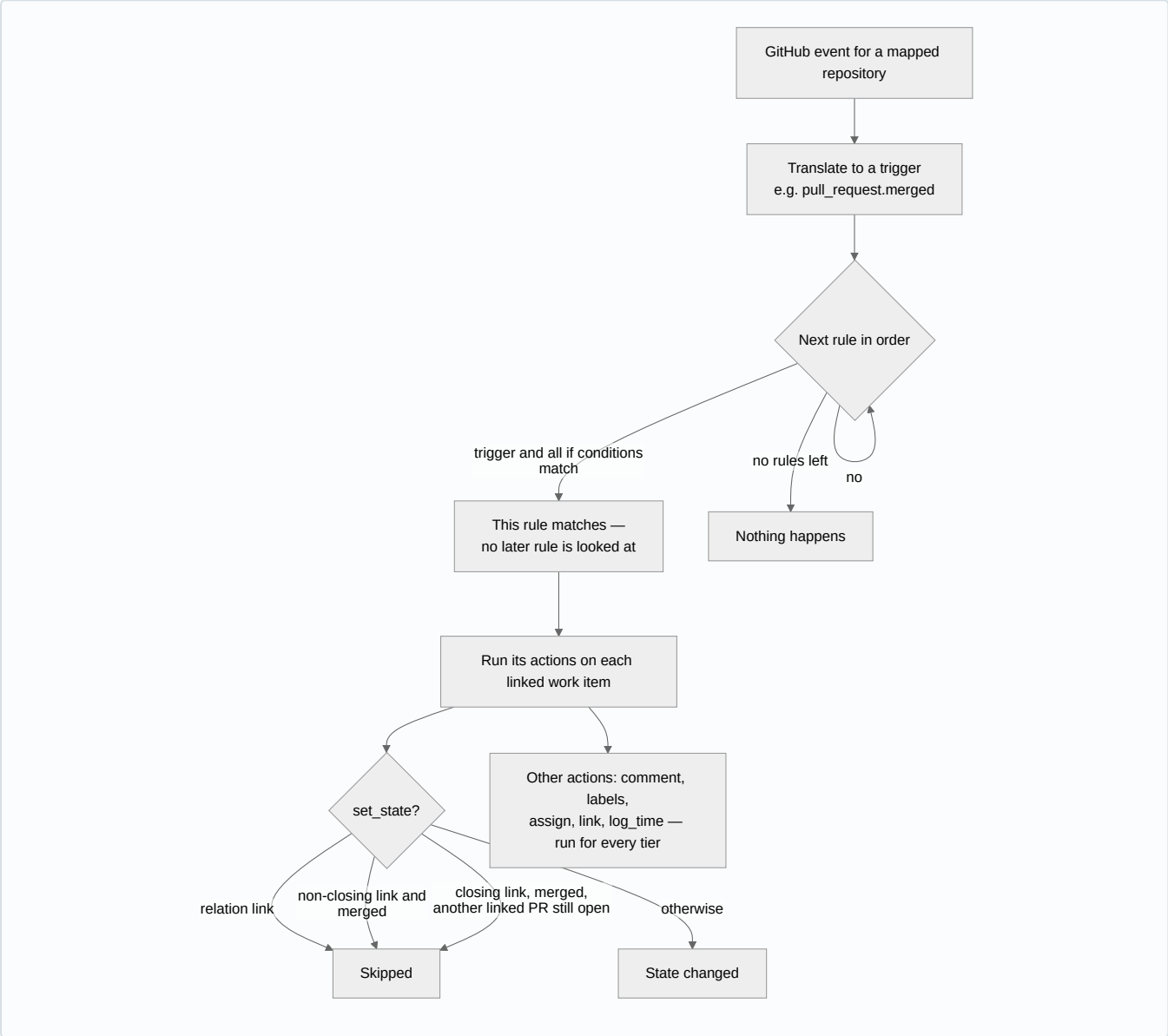


Figure 26.10 Diagram: rule evaluation. The first matching rule wins; reference tiers gate only state changes.

Key points:

1. **First match wins.** Rules are checked from the top. The first rule whose `when` trigger and every `if` condition match is used, and its actions run. No other rule runs for that event.
2. **Targets are the linked work items.**
   - For PR events: every work item the PR links to (by branch, title, body, stored links or `WEB-NEW`).
   - For pushes: the items named in the commit messages.
3. *Tiers gate `set_state` only* (chapter 9 explains tiers):
   - a **relation** link (`Relates to WEB-42`, or a bare key in a PR body) never changes state;
   - a **non-closing** link (`Refs WEB-42`) changes state on every trigger **except** `pull_request.merged`;
   - a **closing** link changes state on merge only when at least one linked PR is merged and *none is still open* (the merge guard; a PR that cannot be read counts as open, and the check runs only at that merge).
4. **Other actions run for every tier.** That covers comments, labels, assignees, links and logged time.
5. **Finished work is never reopened by a new branch.** `branch.created` will not move a Done or Cancelled item back.
6. **Never into Triage.** A state target in the triage group is refused.

The shipping defaults

If you never save rules, this table applies (chapter 14 shows it from a developer's view):

Lifecycle point	Trigger	Default target
Branch pushed	`branch.created`	`started:In Progress`
PR drafted	`pull_request.drafted`	`started:In Progress`
PR opened	`pull_request.opened` (also *ready for review*)	`started:In Progress`
Review requested	`pull_request.review_requested`	`started:In Review`
Changes requested	`pull_request.changes_requested`	no change
PR approved	`pull_request.approved`	no change
Ready for merge	`pull_request.ready_for_merge`	no change

Lifecycle point	Trigger	Default target
PR merged	`pull_request.merged`, base branch matching `^main$`	`completed:Done`
PR closed without merge	`pull_request.closed_unmerged`	no change
PR reopened	`pull_request.reopened`	no change

State targets

A target can be written three ways:

Form	Example	Resolves to
`group:Name`	`started:In Review`	The state named *In Review* in the *started* group. If no such name exists but the group has exactly one state, that one. Otherwise the rule fails and is audited “state … does not exist”.
`group`	`completed`	The group’s default state, otherwise the group’s first state
`Name`	`In QA`	The state with that name, only if exactly one state in the project has it (case-insensitive)

Groups are `backlog`, `unstarted`, `started`, `completed`, `cancelled` (and `triage`, which automation refuses).

> **Warning:** A project that has several started states but none called **In Review** makes the review-requested default fail. Either create **In Review** or change that row of the table.

Rules tab — lifecycle mapping table

Open the mapping → **Rules**. The status line says either “Using shipping defaults (not saved yet).” or “Saved version *n*.” Two editing modes are offered: **Lifecycle mapping table** and **Raw YAML editor**.

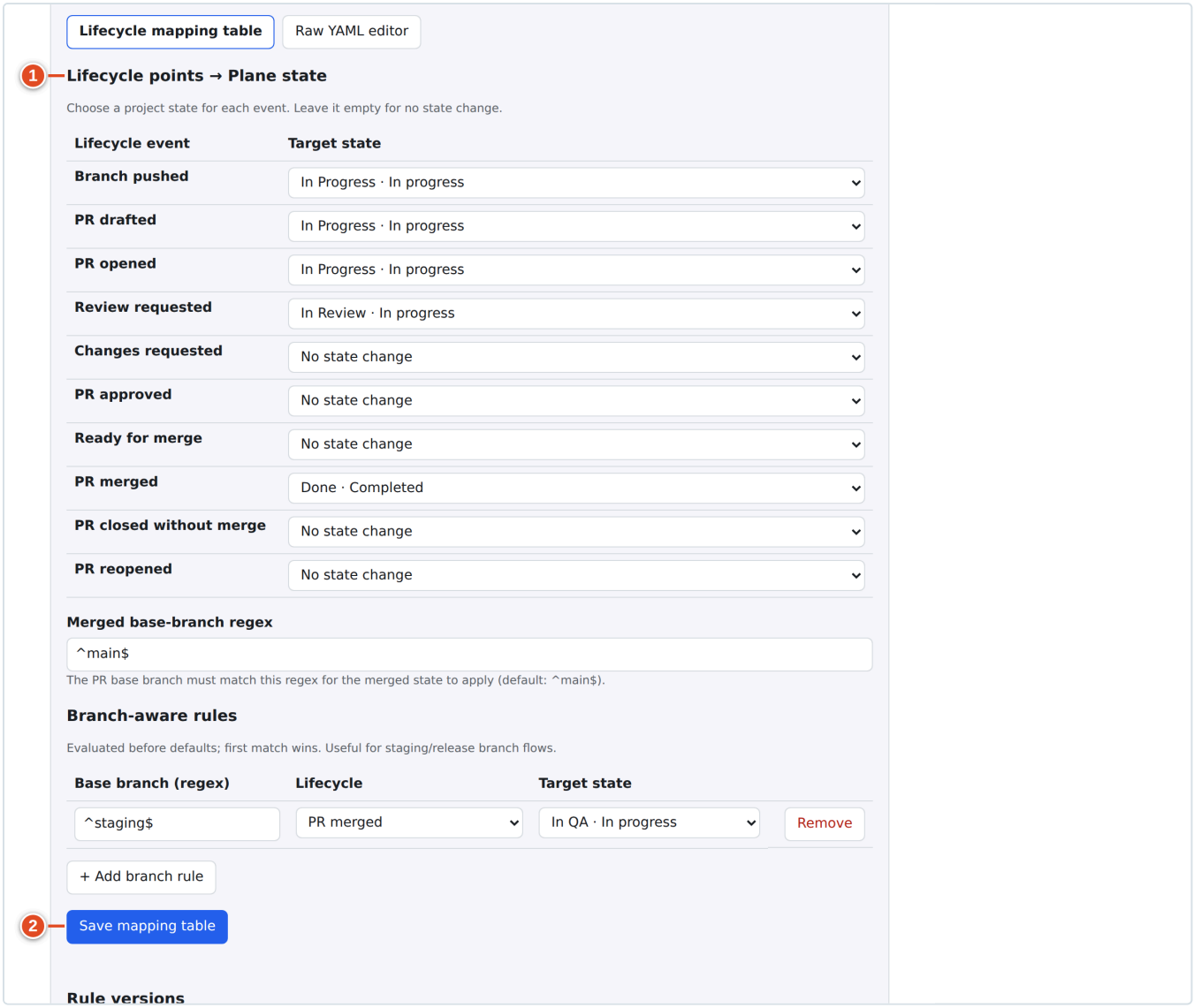


Figure 26.1 Screenshot (demo instance): Rules — (1) lifecycle points → Plane state table; (2) Save mapping table.

Goal: change what a lifecycle point does

Steps

1. Under **Lifecycle points → Plane state** (“Choose a project state for each event. Leave it empty for no state change.”), pick a **Target state** for any **Lifecycle event**:
   - Branch pushed, PR drafted, PR opened, Review requested, Changes requested, PR approved, Ready for merge, PR merged, PR closed without merge, PR reopened.
   - The picker offers each project state (“*In Review* · In progress”), each group (“*Completed (any matching state)*”) and **No state change**.
2. Optionally change **Merged base-branch regex** (“The PR base branch must match this regex for the merged state to apply (default: ^main$).”). For example `^(main|master)$`.
3. Select **Save mapping table**.

Result: a new rule version is saved and used from the next event.

> **Note: Ready for merge** fires once per head commit when the PR is open and not a draft, has at least one standing approval and no standing “changes requested”, and all checks and the combined commit status are successful (neutral and skipped count as success). Map it to a state such as *Ready to merge* if your team wants the board to show it.

Branch-aware rules

“Evaluated before defaults; first match wins. Useful for staging/release branch flows.”

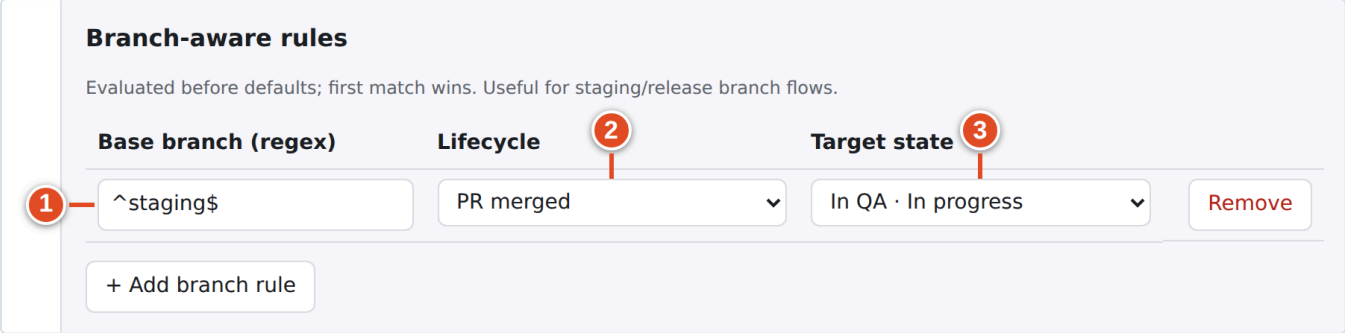


Figure 26.2 Screenshot (demo instance): Branch-aware rules — (1) Base branch (regex); (2) lifecycle point; (3) target state.

Goal: merging into `staging` moves work to In QA, merging into `main` to Done

Steps

1. Select **+ Add branch rule**.
2. **Base branch (regex)**: `^staging$`. ①
3. **Lifecycle**: PR merged. ② (For *Branch pushed* the base column shows “N/A (branch has no base)”.)
4. **Target state**: `In QA` (or `started:In QA`). ③
5. Keep the table’s **PR merged** row at Done with **Merged base-branch regex** `^main$`.
6. **Save mapping table**.

Result: the compiled YAML puts the branch rule first:

```
- name: merged into ^staging$
  when: pull_request.merged
  if:
    base_branch: ^staging$
  then:
    - set_state: started:In QA
- name: "default: branch_pushed"
  when: branch.created
  then:
    - set_state: started:In Progress
# … the other defaults …
- name: "default: merged"
  when: pull_request.merged
  if:
    base_branch: ^main$
  then:
    - set_state: completed:Done
```

In the demo run, PR #126 merged into `staging` moved WEB-39 to **In QA**; PR #121 merged into `main` moved WEB-42 to **Done**.

Rules tab — raw YAML editor

Choose **Raw YAML editor** for anything the table cannot express: labels, comments, links, assignees, logged time, issue/check/deployment triggers.

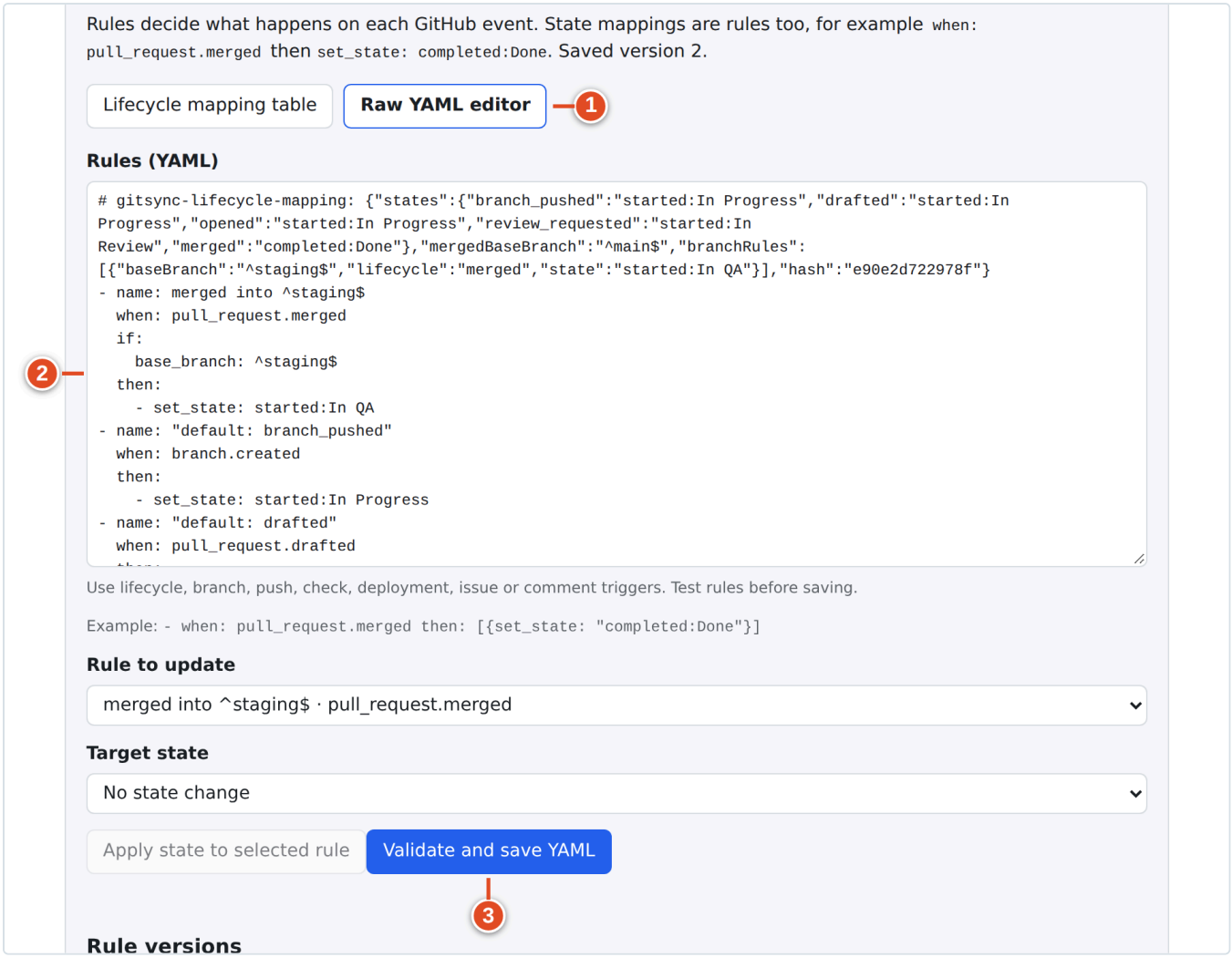


Figure 26.3 Screenshot (demo instance): Rules (YAML) — (1) the compiled rules; (2) Validate and save YAML.

Goal: write and save a custom rule

Steps

1. In **Rules (YAML)** (“Use lifecycle, branch, push, check, deployment, issue or comment triggers. Test rules before saving.”), edit the list.
2. Test it on **Test rules** first (below).
3. Select **Validate and save YAML**. Invalid rules are refused with a message naming the rule, for example `Rule at index 2: unknown trigger pull_request.merge`.

To change only the state of one rule without retyping YAML, choose **Rule to update** (“Rule *n* · *triggers*”), pick a **Target state**, and select **Apply state to selected rule**.

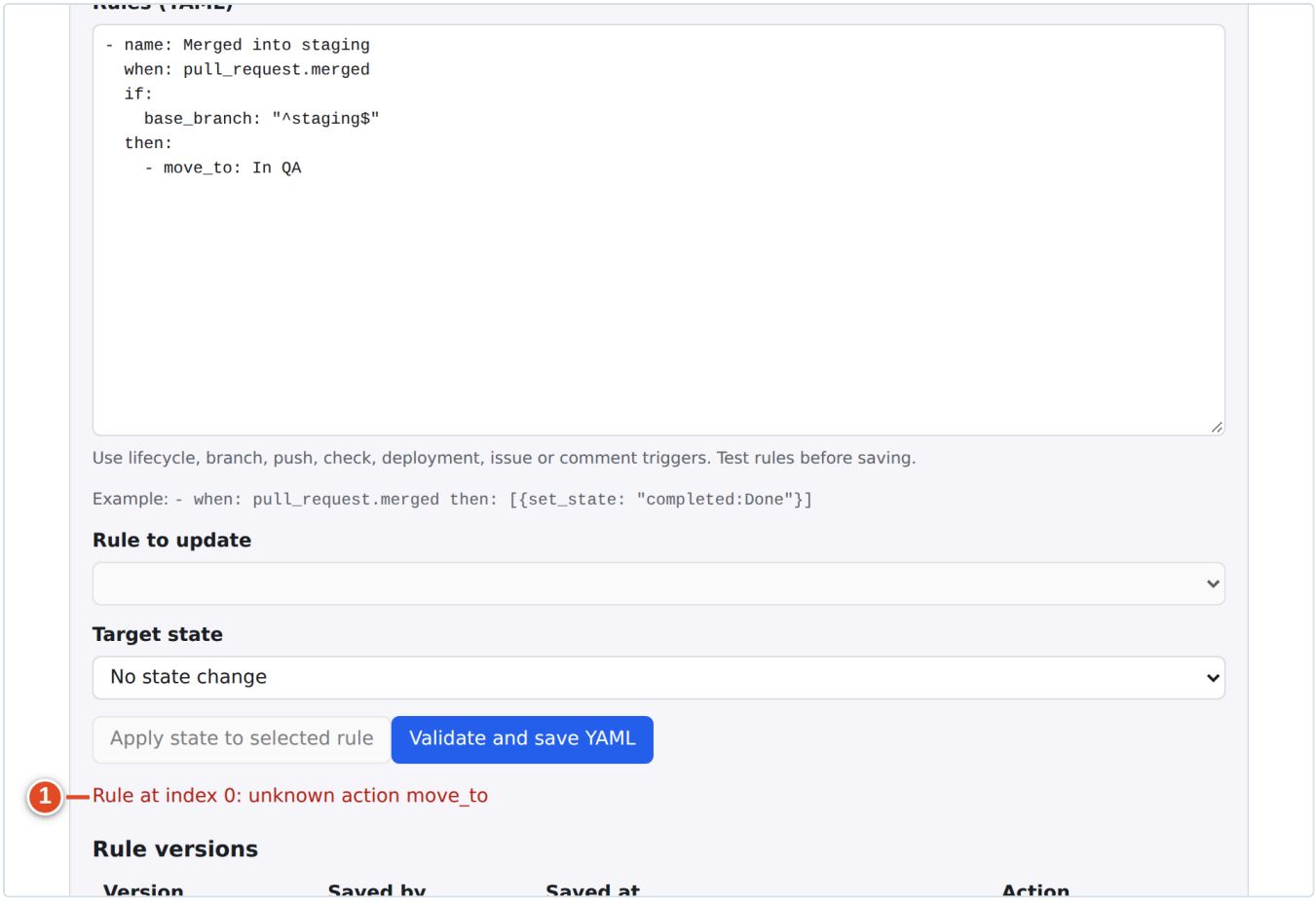


Figure 26.4 Screenshot (demo instance): an invalid rule is refused — (1) the validation error names the rule.

> **Warning:** Switching between modes can replace work.
>
> - Leaving YAML for the table with custom rules asks: “The current YAML contains custom rules or manual edits that cannot be represented in the table. Switching to table mode will replace your rules with defaults upon save. Discard custom YAML?”
> - Saving the table over custom YAML asks: “Saving the mapping table will overwrite your custom YAML rules. Overwrite?”

Rule format

```
- name: Optional label shown in the audit log      # optional
  when: pull_request.merged                        # one trigger or a list
  if:                                              # optional; all must match
    base_branch: "^release/"
  then:                                            # one or more actions, one
key each
    - set_state: "completed:Done"
    - comment: "Released in {{release}}"
```

Triggers (`when`)

Trigger	Fires on
`pull_request.drafted`, `.opened`, `.review_requested`, `.changes_requested`, `.approved`, `.ready_for_merge`, `.merged`, `.closed_unmerged`, `.reopened`	PR lifecycle points
`pull_request.*`	Any of the above
`branch.created`	First push of a new branch
`push`	Any push (items named in commit messages)
`commit.pushed_to_default`	A push to the default branch
`check_suite.<conclusion>` e.g. `check_suite.success`, `check_suite.failure`; `check_suite.*`	CI results
`deployment_status.<state>` e.g. `deployment_status.success`; `deployment_status.*`	Deployments
`issue.<action>` e.g. `issue.opened`, `issue.closed`; `issue.*`	Issue events (GitHub or Plane)
`comment.<action>` e.g. `comment.created`; `comment.*`	Issue comments

Conditions (`if`)

Key	Value	Matching
`base_branch`	regex	PR base branch
`head_branch`	regex	PR head branch, pushed branch, deployment ref
`labels_any`	list	At least one label present (exact case)
`labels_all`	list	All labels present (exact case)
`author`	login or list	GitHub login of the event sender, case-insensitive
`repo`	`owner/name` or list	Case-insensitive

Key	Value	Matching
`project`	Plane project UUID or list	Exact

Actions (`then`)

Action	Value	Effect in Plane	Gated by tier?
`set_state`	state target	Move the item	**Yes**
`add_label`	label name	Add a label (created if missing)	No
`remove_label`	label name	Remove a label	No
`assign`	GitHub login	Add the linked Plane member as assignee (skipped if unlinked)	No
`comment`	text	Comment on the item	No
`add_link`	`{url, title}`	Add a link to the item	No
`log_time`	duration	Comment “Time logged: *duration*” (Plane CE has no worklog)	No
`create_issue`	`{project, title}`	Create a work item in that project (identifier or UUID); once per event	No

Text values may use `{{path}}` templates filled from the event context, for example `{{issue.title}}` or `{{comment.url}}`. A template that cannot be resolved fails the rule, which is audited.

Example: label and announce failing builds

```
- name: Failing CI
  when: check_suite.failure
  then:
    - add_label: ci-failing
    - comment: "CI failed on the linked pull request."
- name: CI fixed
  when: check_suite.success
  then:
    - remove_label: ci-failing
```

Rule versions

Every save (table or YAML) creates a new version. **Rule versions** lists **Version**, **Saved by**, **Saved at** and **Action**; the newest is "*n* (current)".

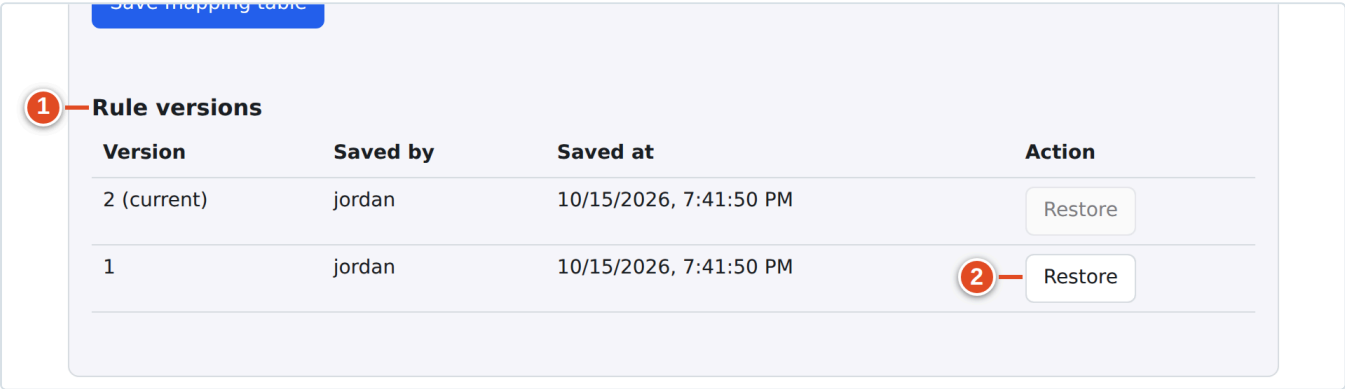


Figure 26.5 Screenshot (demo instance): Rule versions — (1) version list; (2) Restore.

Restore asks "Restore version *n*? This creates a new version and replaces unsaved editor changes." Restoring never deletes history; it saves the old rules as a new current version.

Test rules (dry run)

"Run a delivery through the rules without writing anything. Leave the rules empty to use this mapping's saved rules."

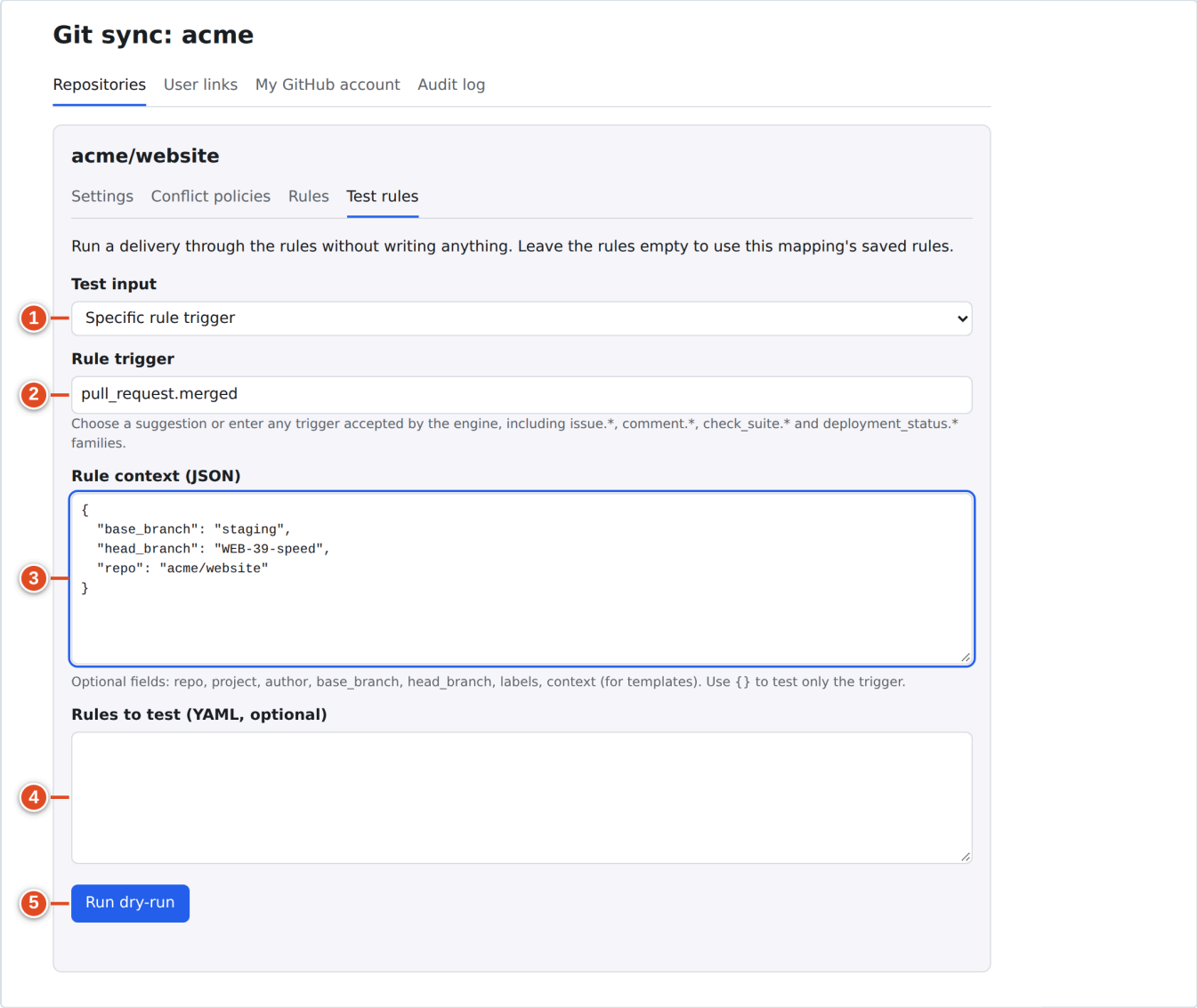


Figure 26.6 Screenshot (demo instance): Test rules — (1) event; (2) conditions; (3) Rules to test (YAML, optional); (4) Run dry-run.

Goal: prove a rule does what you expect before saving it

Steps

1. **Test input**: choose one of two modes.
   - **Webhook delivery**: choose **Source** (GitHub or Plane) and **Event**. Then either paste a **Payload (JSON)**, or enter **Recorded delivery (audit entry id)** (“Uses the payload stored in the audit log instead of the one below.”). Audit entry ids are on the **Audit log**.
   - **Specific rule trigger**: type a **Rule trigger** (suggestions offered) and a **Rule context (JSON)** (“Optional fields: repo, project, author, base_branch, head_branch, labels, context (for templates). Use {} to test only the trigger.”).
2. Optionally paste **Rules to test (YAML, optional)**. If empty, the saved rules (or the defaults) are used.
3. Select **Run dry-run**.

Example input (specific rule trigger):

```
Rule trigger:  pull_request.merged
Rule context:  {"base_branch": "staging", "repo": "acme/website"}
```

Result:

- A match shows “Trigger: pull_request.merged. Matched rule 1 (merged into ^staging$).” followed by the actions as JSON.
- No match shows “Trigger: … No rule matched.” and **No actions would run.**
- A delivery that is not an automation event shows “That delivery is not an automation event.”

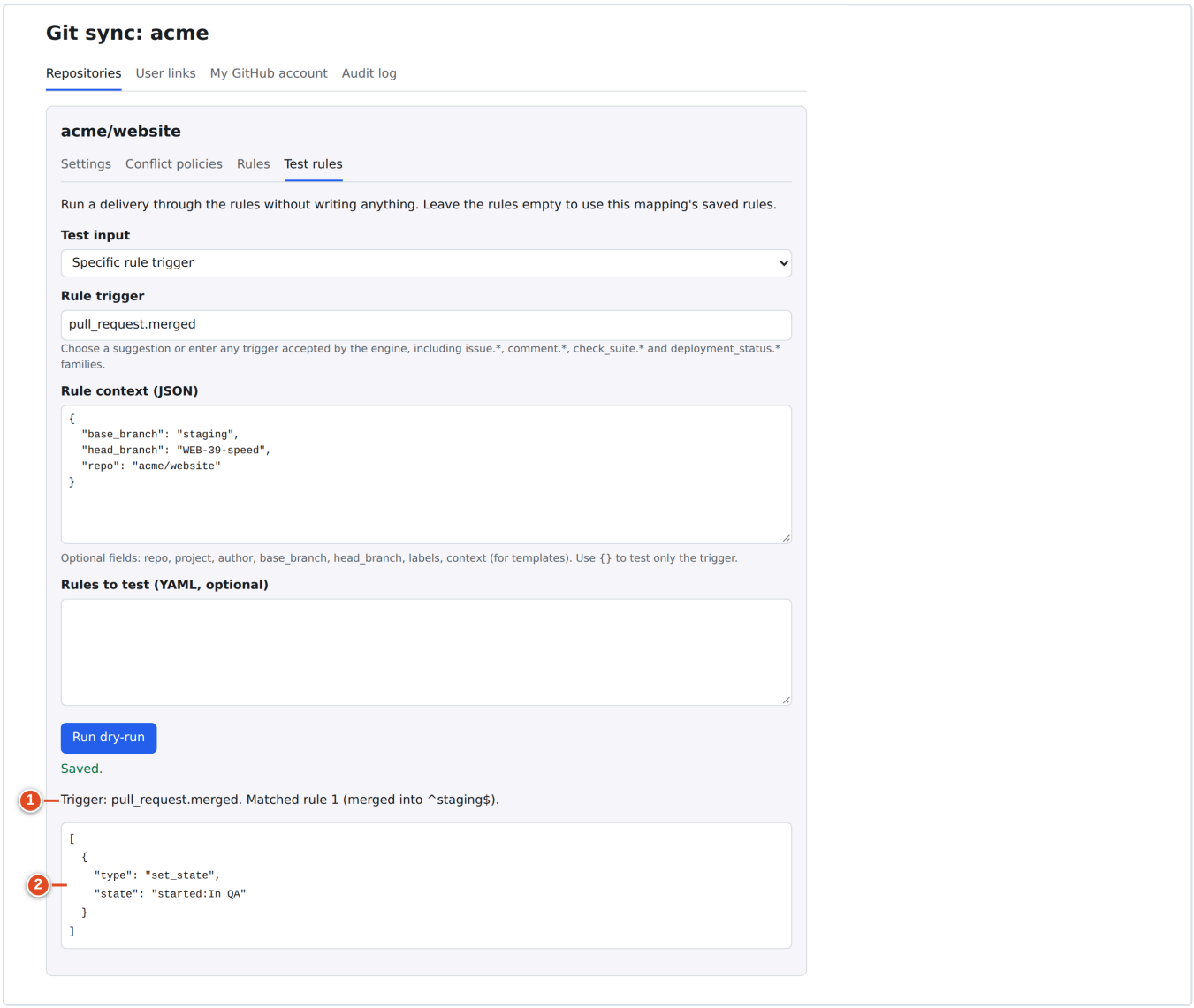


Figure 26.7 Screenshot (demo instance): Dry-run result — (1) matched rule; (2) actions that would run. Nothing was changed.

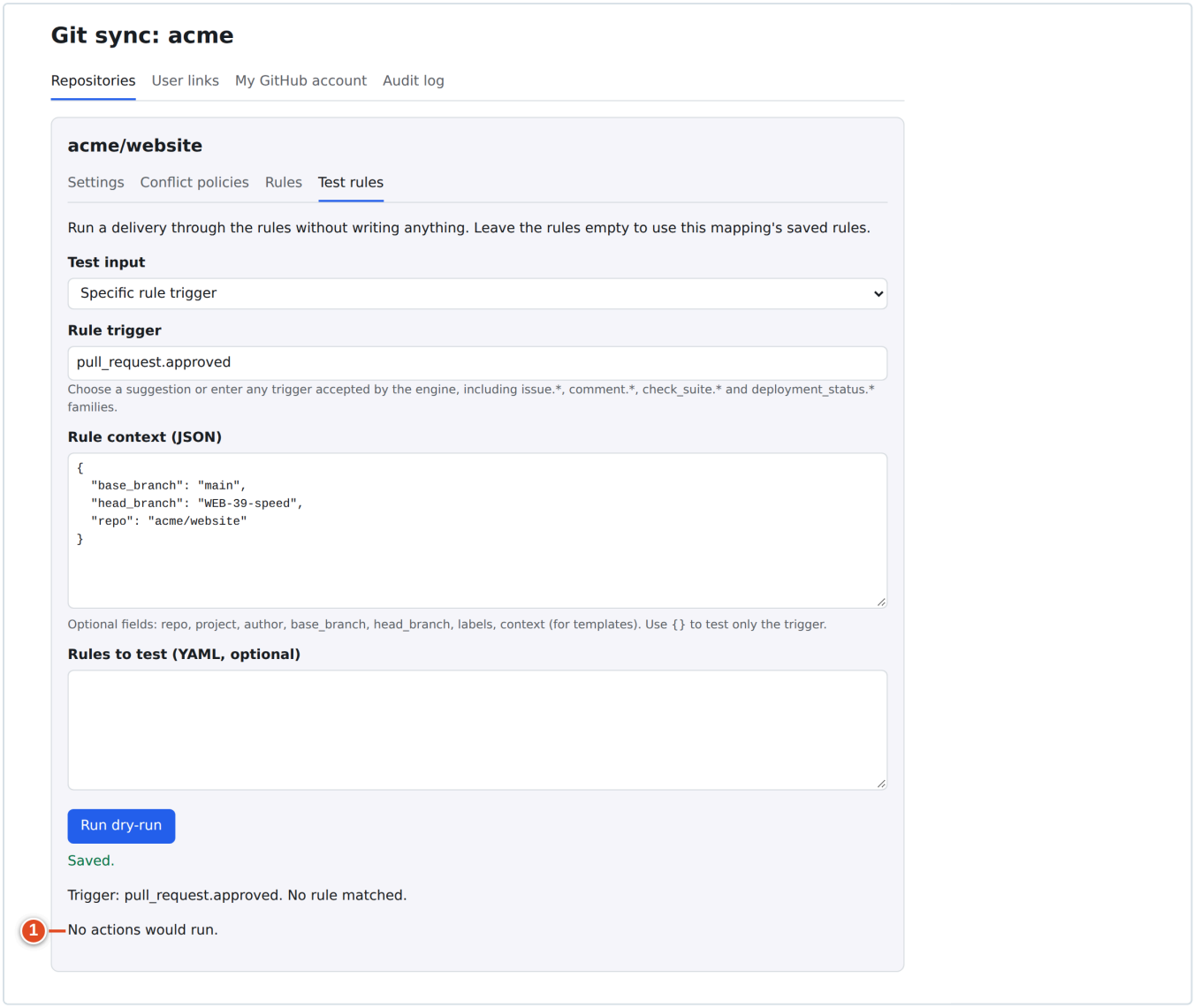


Figure 26.8 Screenshot (demo instance): Dry-run with no match — (1) No actions would run.

To try rules before saving them, paste them into **Rules to test (YAML, optional)**. The dry-run then uses those rules instead of the saved ones and still writes nothing:

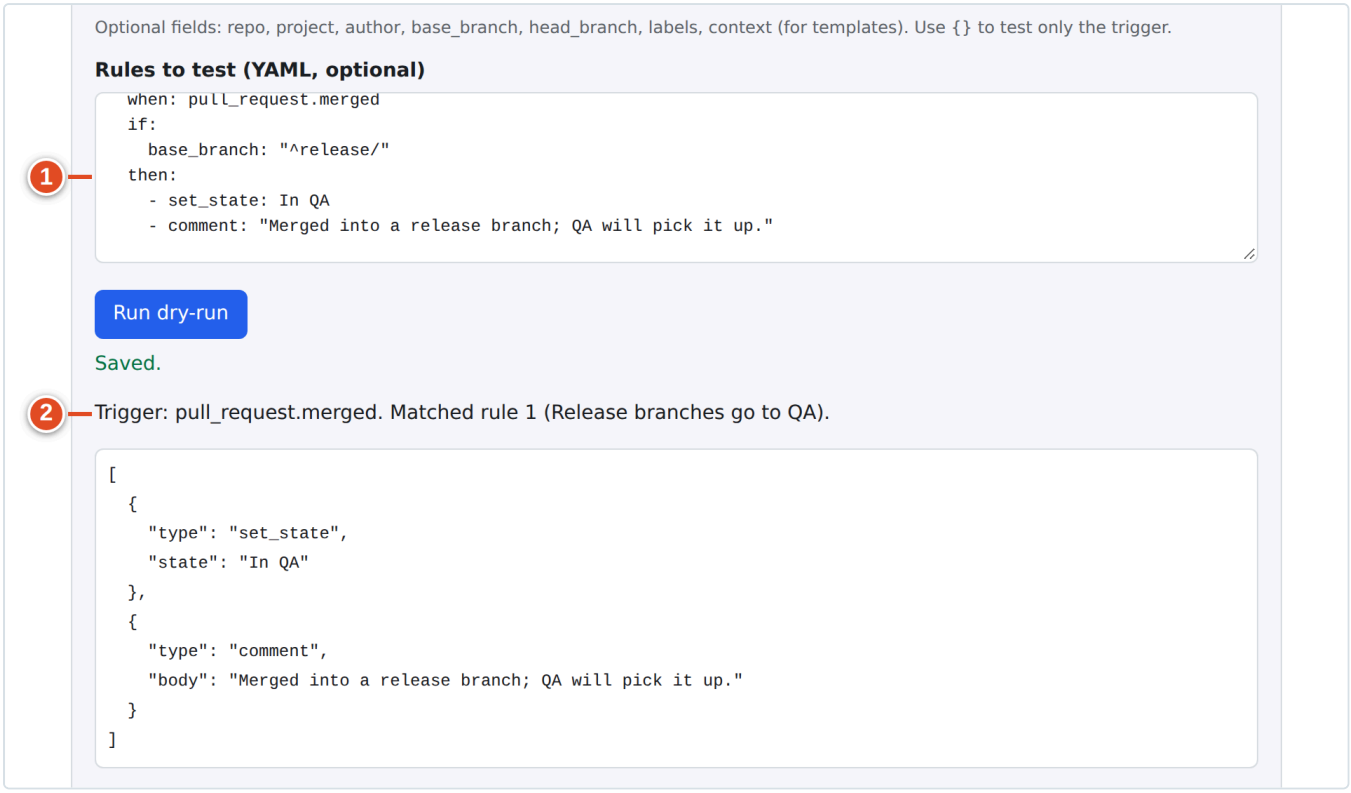


Figure 26.9 Screenshot (demo instance): Testing unsaved rules: (1) Rules to test (YAML, optional); (2) the rule that matched.

A dry run shows rule *selection* and the actions. It does not check that a state exists in the project or that the write would be allowed for a given tier. Those are checked when the rule really runs, and failures appear in the audit log as **Automation rule matched** with outcome *failed* or *ignored*.

If it went wrong

Symptom	Likely cause	Check
A PR merge did not move the item to Done	Base branch does not match `^main$`; link is non-closing/relation; another linked PR still open; no matching rule	Audit log: *Automation rule matched* outcome *ignored*, or no entry; **Test rules** with the base branch
“Automation rule matched … failed: state … does not exist”	Target state name not in this project	Fix the target or create the state
A branch rule never fires	It sits after a broader rule that matches first	Move it up; branch rules from the table are always first
Labels condition never matches	Case differs	Labels compare exactly
Custom YAML disappeared	Saved the table over it	**Rule versions → Restore**

What you learned

- Rules are an ordered list; the first matching rule wins and its actions run on the linked items.
- Tiers gate only `set_state`; the merge guard protects multi-PR work.
- The lifecycle table and branch-aware rules compile to YAML; the YAML editor exposes every trigger, condition and action.
- Every save is a version you can restore; **Test rules** shows matches and actions without writing anything.

Source files

- `src/rules/engine.ts`, `src/rules/types.ts` — grammar and evaluation
- `src/github/pr-automation.ts` — `QLL_DEFAULTS`, `compileLifecycleRules`, state resolution, tier gating, ready for merge
- `src/app/rules.ts`, `src/app/rule-events.ts` — live execution and event translation
- `src/ui/client/Workspace.tsx`, `rule-editor.ts` — Rules and Test rules tabs
- `src/ui/dry-run.ts` — dry-run endpoint

27 · User links

A **user link** pairs a Plane member with a GitHub account in one workspace. git-sync uses user links to:

- recognize **who** did something on GitHub. The webhook's authenticated sender (never the commit author name) is translated to a Plane member for Smart Commit provenance ("via *sha* by *login*"), assignee and `@mention` translation, and rule `assign` actions;
- write **Plane → GitHub** mirror changes *as that person* when their personal token works, instead of as the App;
- fill the **Branch name** `{username}` placeholder and the repository picker for that member.

> **Note:** In this release every write **into Plane** is made by the service account (the bot), even when the GitHub user is linked. A link improves attribution in GitHub and in git-sync's records; it does not make Plane show the member as the author.

There are three ways a link is made:

How	Who	Stores a personal token?	Chapter
The member uses **Connect personal account** (GitHub OAuth)	The member	Yes, encrypted	8
An admin uses **Link a user** on this page	Workspace admin	No	this chapter
Automatic email match: an unlinked GitHub sender whose **public** GitHub email equals exactly one member's Plane email	git-sync	No	this chapter

Open the page

Workspace settings → GitHub Sync → User links, or `/git-sync/ui/workspaces/<slug>/users`.

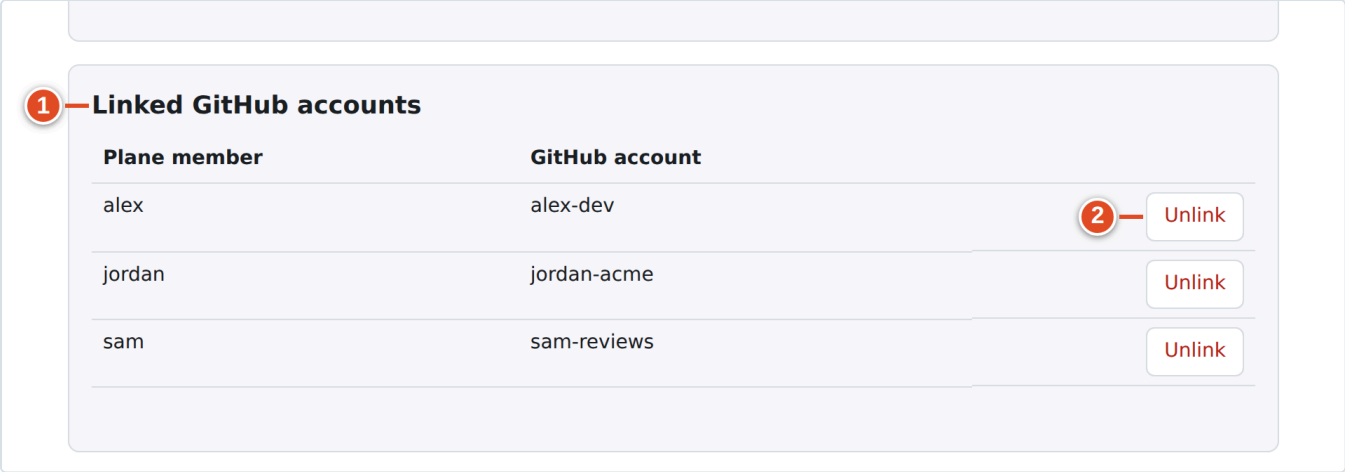


Figure 27.1 Screenshot (demo instance): User links — (1) Linked GitHub accounts; (2) needs re-authorization flag; (3) Unlink.

Linked GitHub accounts lists **Plane member** and **GitHub account** for every link. In the demo:

- Jordan Admin (`jordan-acme`) and Alex Developer (`alex-dev`) connected themselves;
- Sam Reviewer (`sam-reviews`) was linked by the admin.

A row marked “Needs re-authorization. Reconnect using this member’s own Plane session.” means GitHub refused that member’s stored token, for example because they revoked the App on GitHub. Until they **Reconnect** on **My GitHub account**, their mirror writes fall back to the App. Admins cannot reconnect for them.

With no links, the page says “No Plane users are linked to GitHub accounts yet. Unlinked users are attributed to the service account.”

Link a user manually

Goal: tell git-sync that a Plane member is a given GitHub account

Before you start: the person must be a member of this workspace. Ask them for their GitHub username.

Steps

1. Under **Link a user**, choose **Plane member**. ①
2. Type **GitHub username**. ②
3. Select **Look up id**. ③ git-sync asks GitHub for the account and fills **GitHub user id** (“Numeric and stable across renames.”). If GitHub does not know the login you see **Lookup failed**.
4. Select **Link account**. ④

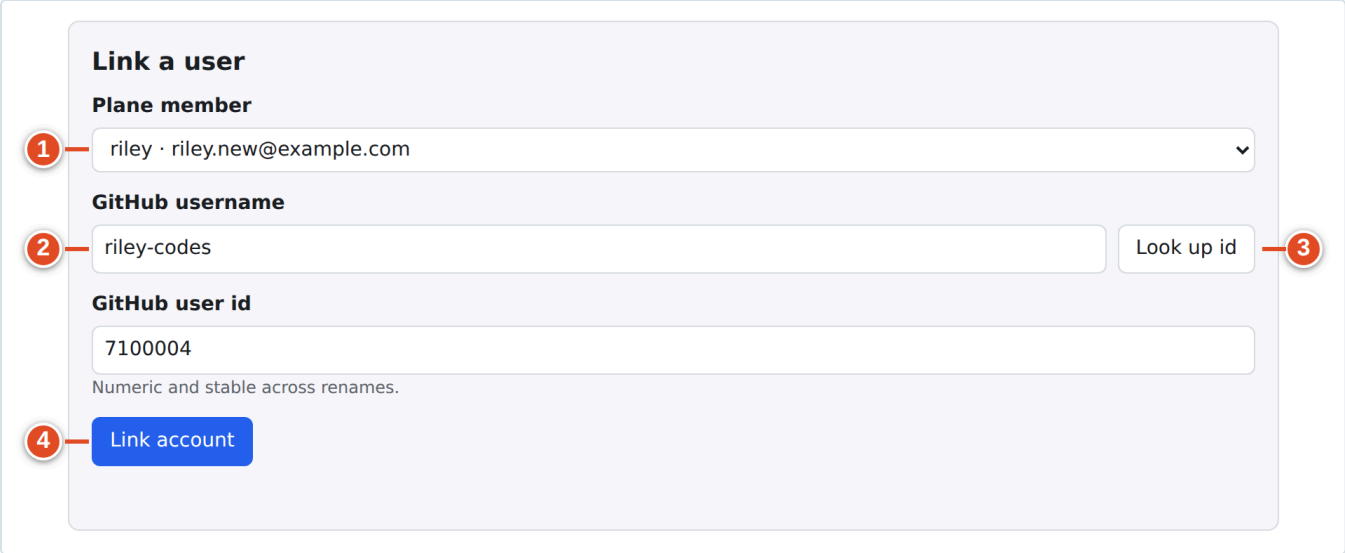


Figure 27.2 Screenshot (demo instance): Link a user — (1) Plane member; (2) GitHub login; (3) Look up id; (4) Link account.

After **Link account** the new link appears in the list at once:

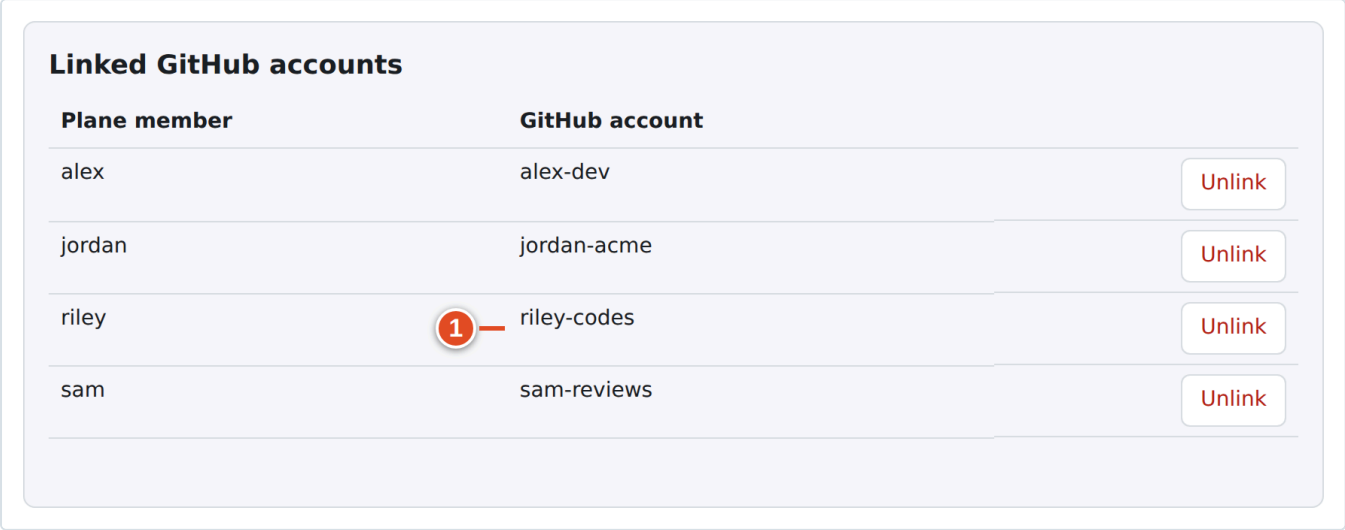


Figure 27.3 Screenshot (demo instance): After Link account: (1) the new link in Linked GitHub accounts.

Result: the link appears in the list with GitHub's canonical spelling of the login.

Checks git-sync makes on save:

Refusal	Why
"That GitHub account does not exist"	The login is unknown on your GitHub host
"githubId does not match that GitHub login"	The id you typed is not that account's id; use **Look up id**

Refusal	Why
“That Plane user is not a member of this workspace”	Only members can be linked
“githubLogin is not a valid GitHub username”	Typo
A uniqueness error	That GitHub account is already linked to another member; one GitHub account can be linked to only one member per workspace

Links are keyed by GitHub’s **numeric id**, so a later rename on GitHub does not break them.

Unlink

Unlink asks “Unlink *login*? Any credentials stored for this user are deleted.” After confirming:

- the link is removed and the member’s stored GitHub token (if any) is deleted;
- git-sync remembers the unlink, so *automatic email matching will not re-link* that GitHub account. Only an explicit **Link account** or the member’s own **Connect personal account** links it again;
- future events from that GitHub account are attributed to the service identity and audited as **Service account attribution**.

> **Note:** The browser asks first (text recorded from the running UI): “Unlink alex-dev? Any credentials stored for this user are deleted.”

Automatic email matching

When an event arrives from a GitHub sender with no link, git-sync tries one careful match:

1. It reads the sender’s **public** GitHub profile email.
2. It compares it with the email addresses of the workspace’s Plane members.
3. If **exactly one** member matches, it links them and audits **Account matched by email**.
4. If there is no public email, no match or several matches, it links nobody and audits the reason.

Bots (`*[bot]`, `ghost`, accounts of type Bot) are never matched, nothing is guessed, and an unlinked account is never matched again automatically.

Member accounts in the setup checklist

While a workspace has no mappings, **Set up GitHub sync** shows “Member accounts: *n* linked of *m*.” It links to this page. Aim for every developer who pushes code to be linked before you rely on Smart Commits.

If it went wrong

Symptom	Cause	Fix
Smart Commit comments say “by *login*” but the audit shows **Service account attribution**	That GitHub login is not linked in this workspace	Link it
A member’s mirror edits on GitHub appear as the App	No personal token, or token needs re-authorization	Member uses **Connect personal account** / **Reconnect**
Assignee from GitHub not copied to Plane	The GitHub assignee is not linked	Link the user; unlinked assignees are dropped
An unlinked person keeps getting re-linked	It does not happen: unlinks are remembered	—

What you learned

- User links map Plane members to stable GitHub ids for attribution, assignees, mentions and the repository picker.
- Members connect themselves via OAuth; admins can link by username with **Look up id**; careful public-email matching links only exact single matches.
- **Unlink** deletes stored credentials and stops email re-matching.
- Plane-side writes still come from the bot in this release.

Source files

- `src/ui/client/Workspace.tsx` — `UserLinksPage`
- `src/ui/api.ts` — `GET /user-links`, `PUT/DELETE /user-links/:planeUserId`
- `src/ui/mapping-check.ts` — `createGithubUserLookup`, `createPlaneMemberCheck`
- `src/app/identity.ts` — email matching
- `src/db/ledgers.ts` — user-link tombstones

28 · Sync health and the audit log

You will get questions like “why didn’t my PR move WEB-42?” or “is the integration working at all?”. Answer them with two tools:

- **Sync health** — a quick status board per mapping.
- **Audit log** — a diary of every delivery git-sync received and every decision it made.

This chapter explains both, for workspace admins and for the instance admin.

Three levels of “healthy”

Level	Where	Proves
Liveness	`GET /git-sync/healthz` → `{"status":"ok"}`	The HTTP service answers. Nothing about the database, GitHub or Plane.
Instance health	God mode → **Connection health** and **Setup checks** (chapter 23)	Database reachable, credentials decryptable, App registered correctly, deliveries seen
Mapping health	Workspace → **Mapped repositories** → **Sync health**	This repository’s deliveries, writes, errors, rate limits

Sync health cards

Each row in **Mapped repositories** has a **Sync health** card. The page refreshes it every 30 seconds; **Refresh sync health** refreshes it now.

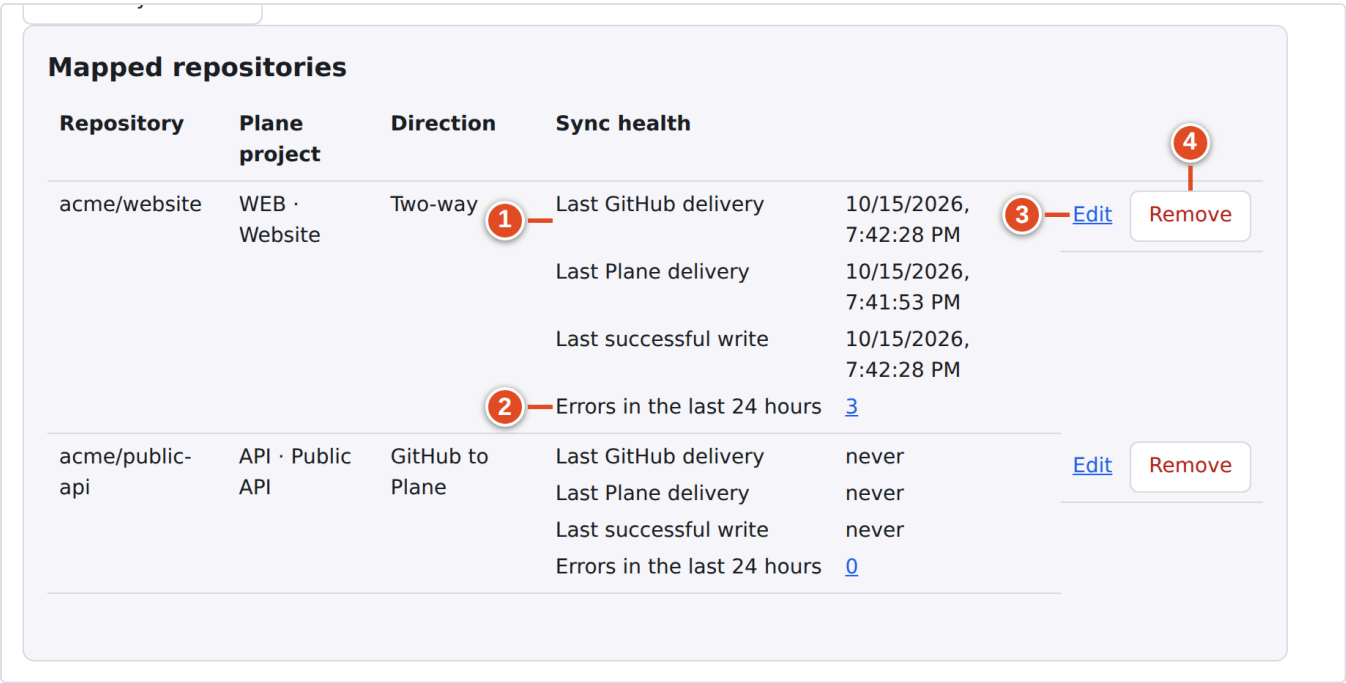


Figure 28.1 Screenshot (demo instance): Mapped repositories with sync health — (1) Last GitHub delivery; (2) Errors in the last 24 hours; (3) Edit; (4) Remove.

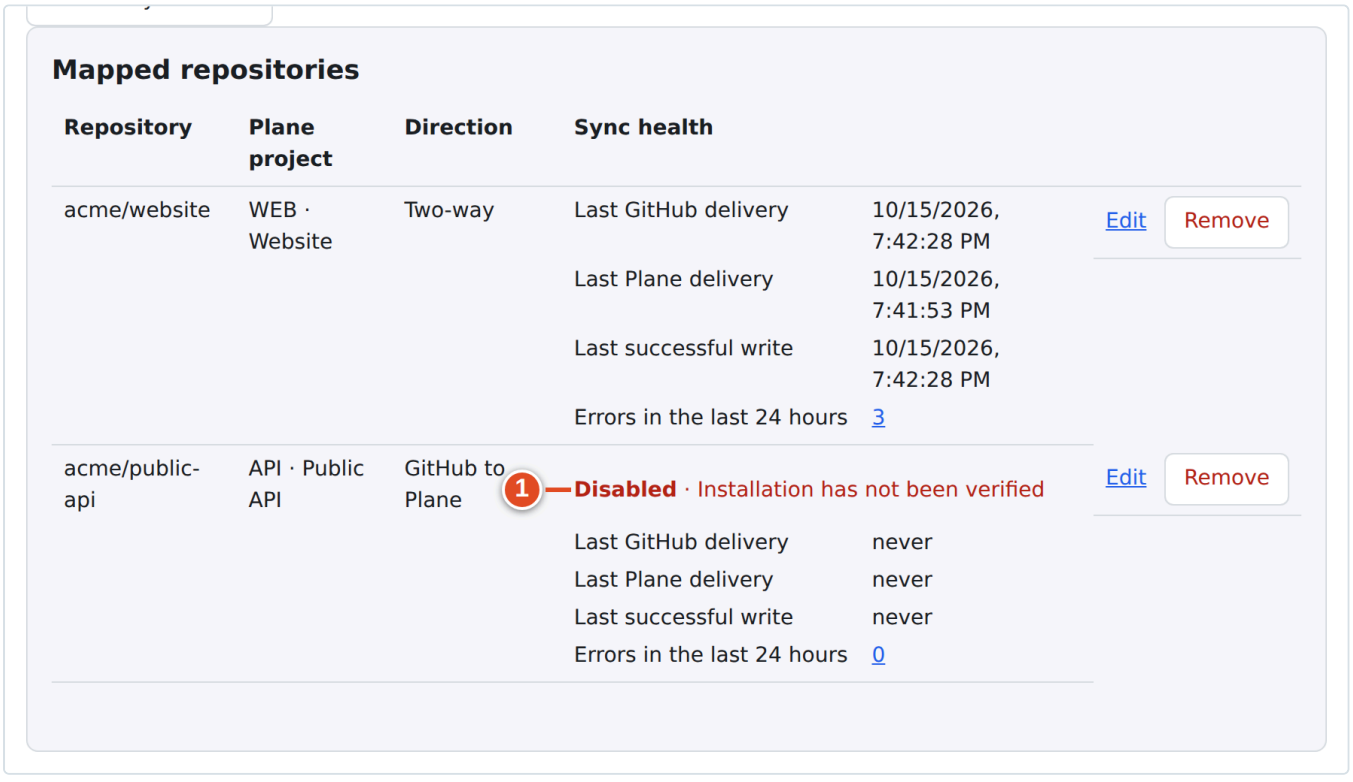
Line	Meaning	Worry when
Last GitHub delivery	Newest verified GitHub webhook for this repository	**never**, or hours old while people are pushing
Last Plane delivery	Newest verified Plane webhook for this workspace	**never**, or old while people edit work items (see the banner below)
Last successful write	Newest write git-sync completed to Plane or GitHub for this mapping	Deliveries are recent but writes are not
Errors in the last 24 hours	Count of failed audit entries for this mapping; the number links to the audit log filtered to them	Not zero — open it
Rate-limited until	Shown only while GitHub or Plane has paused git-sync; writes resume at that time	Often, or far in the future
Disabled · *reason*	Shown only when the mapping cannot sync	Always — fix the reason



Mapped repositories

Repository	Plane project	Direction	Sync health		
acme/website	WEB · Website	Two-way	Last GitHub delivery	10/15/2026, 7:43:22 PM	Edit Remove
			Last Plane delivery	10/15/2026, 7:41:53 PM	
			Last successful write	10/15/2026, 7:43:22 PM	
			Errors in the last 24 hours	3	
			(1) Rate-limited until	10/15/2026, 8:28:22 PM	
acme/public-api	API · Public API	GitHub to Plane	**Disabled** · Installation has not been verified		Edit Remove
			Last GitHub delivery	never	
			Last Plane delivery	never	
			Last successful write	never	
			Errors in the last 24 hours	0	

Figure 28.2 Screenshot (demo instance): Sync health while GitHub rate-limits the App — (1) Rate-limited until.



Mapped repositories

Repository	Plane project	Direction	Sync health		
acme/website	WEB · Website	Two-way	Last GitHub delivery	10/15/2026, 7:42:28 PM	Edit Remove
			Last Plane delivery	10/15/2026, 7:41:53 PM	
			Last successful write	10/15/2026, 7:42:28 PM	
			Errors in the last 24 hours	3	
acme/public-api	API · Public API	GitHub to Plane	(1) **Disabled** · Installation has not been verified		Edit Remove
			Last GitHub delivery	never	
			Last Plane delivery	never	
			Last successful write	never	
			Errors in the last 24 hours	0	

Figure 28.3 Screenshot (demo instance): a disabled mapping — (1) Disabled and its reason.

Banners at the top of the workspace page

Banner	Meaning	Do
“The sync database is unreachable. Settings and activity cannot be loaded.”	git-sync cannot reach its Postgres	Operator: check `plane-db` and git-sync logs
“Plane changes have stopped arriving. Ask a workspace admin to re-enable the Git sync webhook in Plane’s webhook settings.”	Reconciliation found Plane changes newer than the last Plane delivery by more than 30 minutes. Plane has probably switched the webhook off.	Re-enable it (chapter 29)
“GitHub App credentials could not be decrypted. Ask an instance admin to repair the GitHub App settings.”	Stored secrets unreadable	Chapter 30

The audit log

Open **Audit log** in the workspace tabs (`/git-sync/ui/workspaces/<slug>/audit`). Entries are newest first.

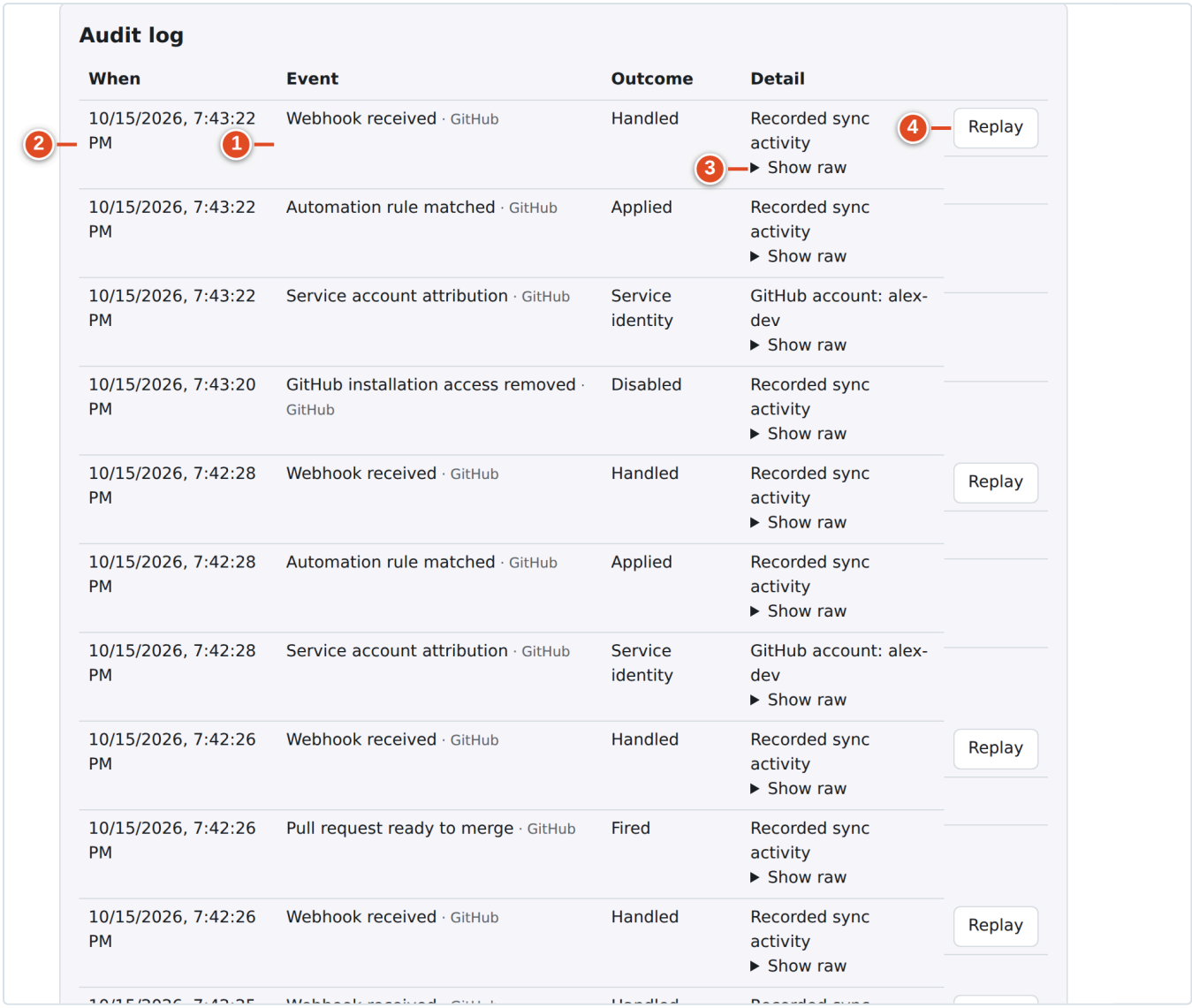


Figure 28.4 Screenshot (demo instance): Audit log — (1) entry kind and outcome; (2) time; (3) Show raw; (4) Replay.

Column	Content
When	Time recorded
Event	A readable label plus "· GitHub" or "· Plane" for the source
Outcome	What happened: *handled*, *applied*, *registered*, *retained*, *ignored*, *failed*, *partial*, *fired*, *created*, *service_identity*, …
Detail	Short summary: the error text if any, otherwise "GitHub account: *login*", "Rule: *name*" or "Recorded sync activity"; **Show raw** expands the full entry

Buttons per entry:

Button	On	Effect
Show raw	Every entry	Expands the stored JSON detail (payload excerpts, ids, error)
Replay	Received deliveries (**Webhook received**)	Asks “Replay this delivery? It runs again under a new delivery id and may write to Plane and GitHub.” and queues the stored payload again
Retry / **Retry scheduled**	**Sync write failed** entries	Schedules that failed write again (chapter 29)

Load older entries pages further back.

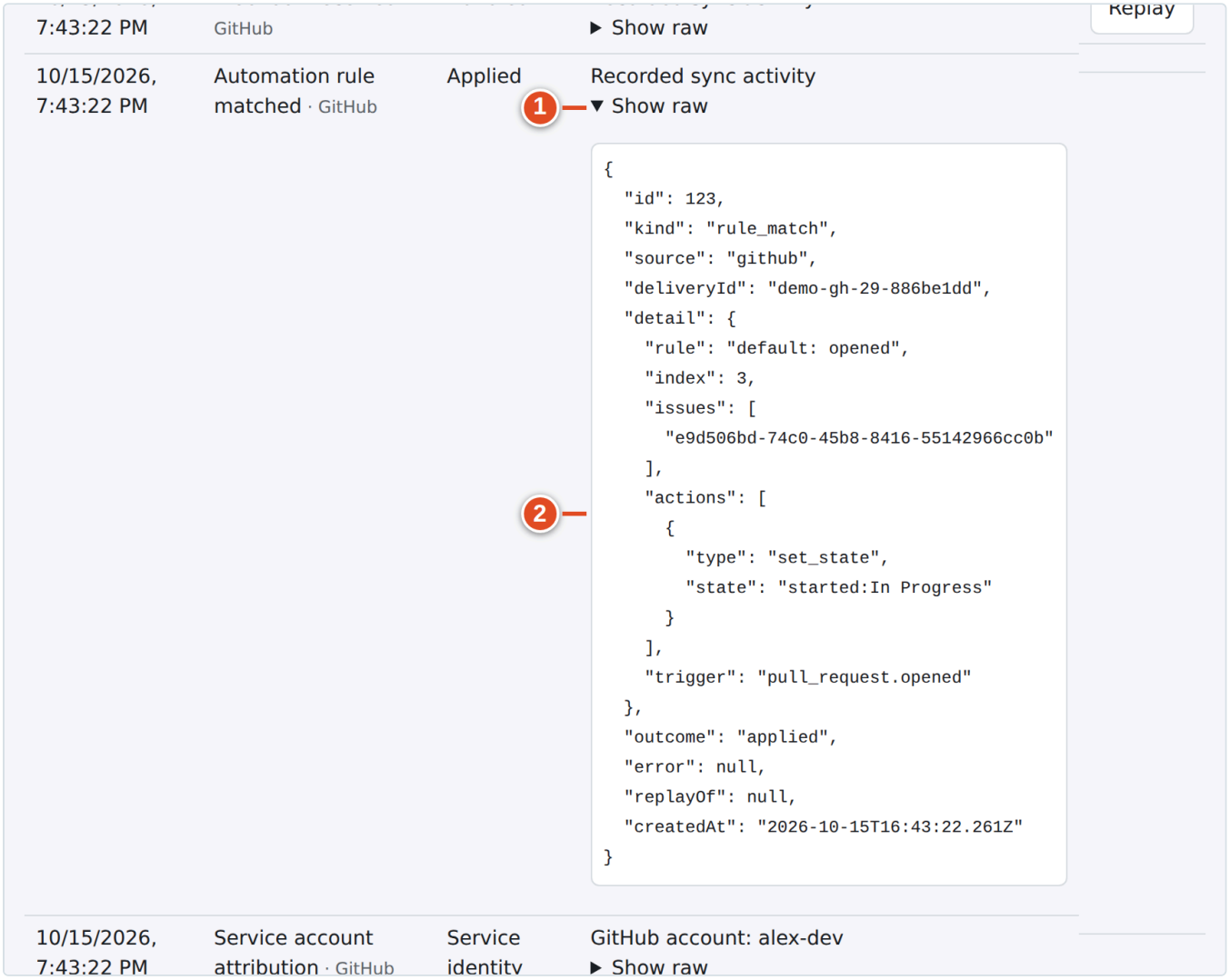


Figure 28.5 Screenshot (demo instance): Show raw — (1) the stored event detail.

> **Warning:** Raw entries can contain issue text, comments and GitHub logins. Keep screenshots of them private.

Common event labels

Label	Meaning
Webhook received	A verified delivery was accepted (raw kind `inbound_delivery`)
Branch or commit push / Pull request updated / GitHub issue updated / Plane work item updated / Comment updated / Commit status updated	Processing of that kind of event
Smart Commit action completed	One Smart Commit command was applied
Automation rule matched	A rule matched: *applied* (wrote), *ignored* (matched but every target was skipped by tier or merge guard), *failed* (e.g. unknown state)
Pull request ready to merge	Ready-for-merge fired for a head commit
Work item creation requested by pull request	A `WEB-NEW` PR created an item
Mirrored work item created	Issue mirroring created a partner
Work item reference not found	A key in a PR matched nothing
Service account attribution	The sender was unlinked or a bot, or had no stored token, so the service identity was used
Account matched by email	Automatic user link (chapter 27)
Sync write / Sync write failed / Sync write retry scheduled	Outbox write results
GitHub installation is not approved / GitHub installation access removed	Mapping authority problems
Plane webhook has stopped arriving	The silent-webhook detector fired
Development link updated / Deployment status updated	Development panel data changed

Filtering

The page has *no filter controls*. Filters are URL parameters, and the health links set them for you:

Parameter	Example	Shows
`errors=1`	`/git-sync/ui/workspaces/acme/audit?errors=1`	Failed entries only
`mappingId=<id>`	`…/audit?mappingId=1&errors=1`	One mapping
`hours=<n>`	`…/audit?errors=1&hours=24&mappingId=1`	The last *n* hours (up to 8760)

The **Errors in the last 24 hours** number opens `?mappingId=<id>&errors=1&hours=24`, and the page says “Errors in the last 24 hours for mapping *id*.”

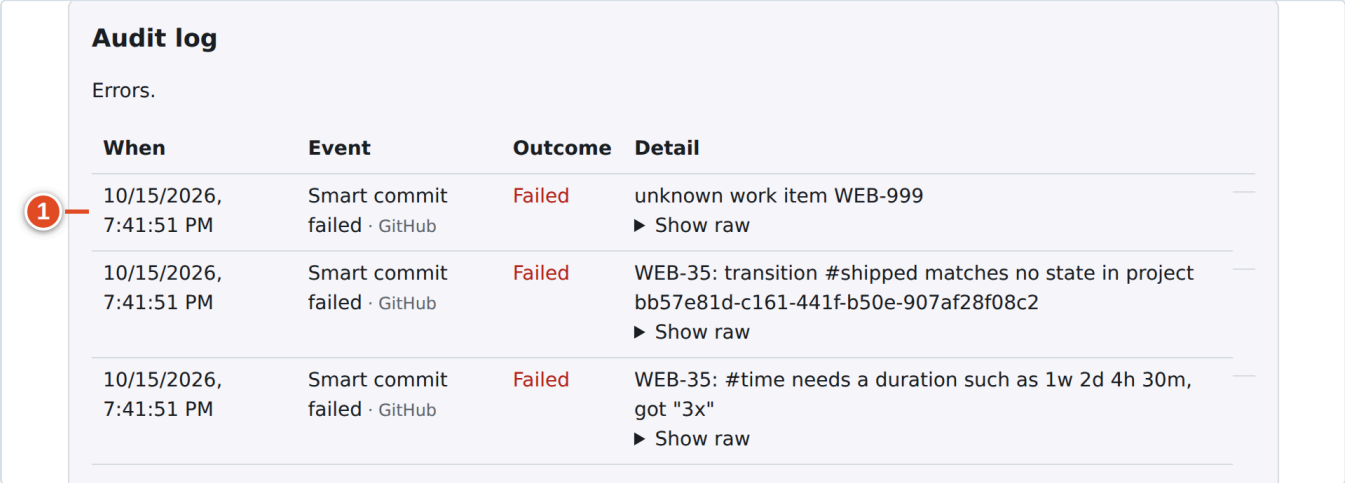


Figure 28.6 Screenshot (demo instance): errors only (URL parameter errors=1) — (1) failed Smart Commit commands.

The other URL parameters work the same way. For example, `?mappingId=1&hours=24` shows one mapping’s last day:

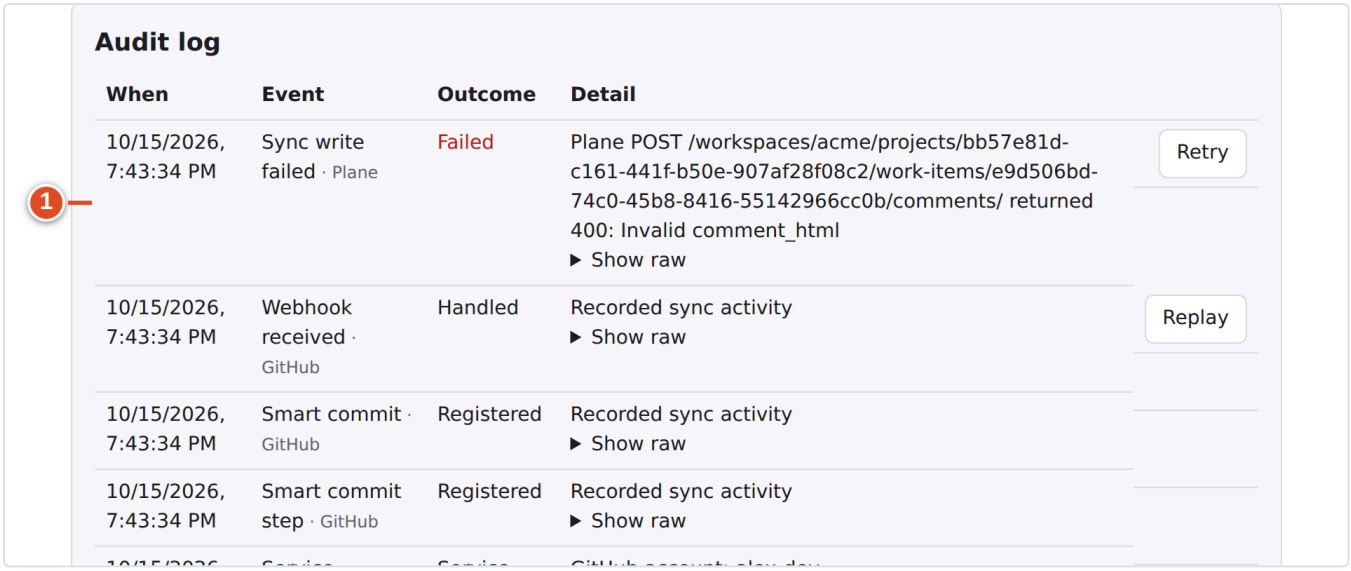


Figure 28.7 Screenshot (demo instance): Audit log filtered by URL parameters (mappingId=1, hours=24): (1) the filter.

Reading a story from the audit log

Real entries from the demo run, oldest first, for one push to WEB-42's branch:

Entry (Event column)	Outcome	Meaning
Smart commit source	retained	The commit was recorded for processing
Service account attribution	service_identity	`alex-dev` is linked, but no Plane token is stored, so Plane writes use the bot
Smart commit target / Smart commit command / Smart commit step	retained / registered	`#comment` for WEB-42 was turned into a durable write
Smart commit	registered	All commands of the commit are registered
Webhook received	handled	The push delivery itself (recorded when processing ends)
Smart Commit action completed	applied	The comment was written in Plane
Automation rule matched (**Show raw**: `"rule": "default: branch_pushed"`)	applied	WEB-42 moved to In Progress

Kinds without a friendly label are shown with their internal name made readable, for example `smart_commit_failed` → “Smart commit failed”.

Instance audit (God mode)

God mode → **GitHub Sync** → **Instance audit** (`/git-sync/ui/admin/audit`) shows activity across the instance:

- By default: “Showing activity with no workspace.” These are deliveries that could not be tied to a workspace, such as installation events, unmapped repositories and signature problems.
- With `?scope=all`: “Showing instance and workspace activity.” — every workspace, with a **Workspace** column and **View workspace activity** links.
- The **Failures in the last 24 hours** link in **Connection health** opens it filtered to errors.

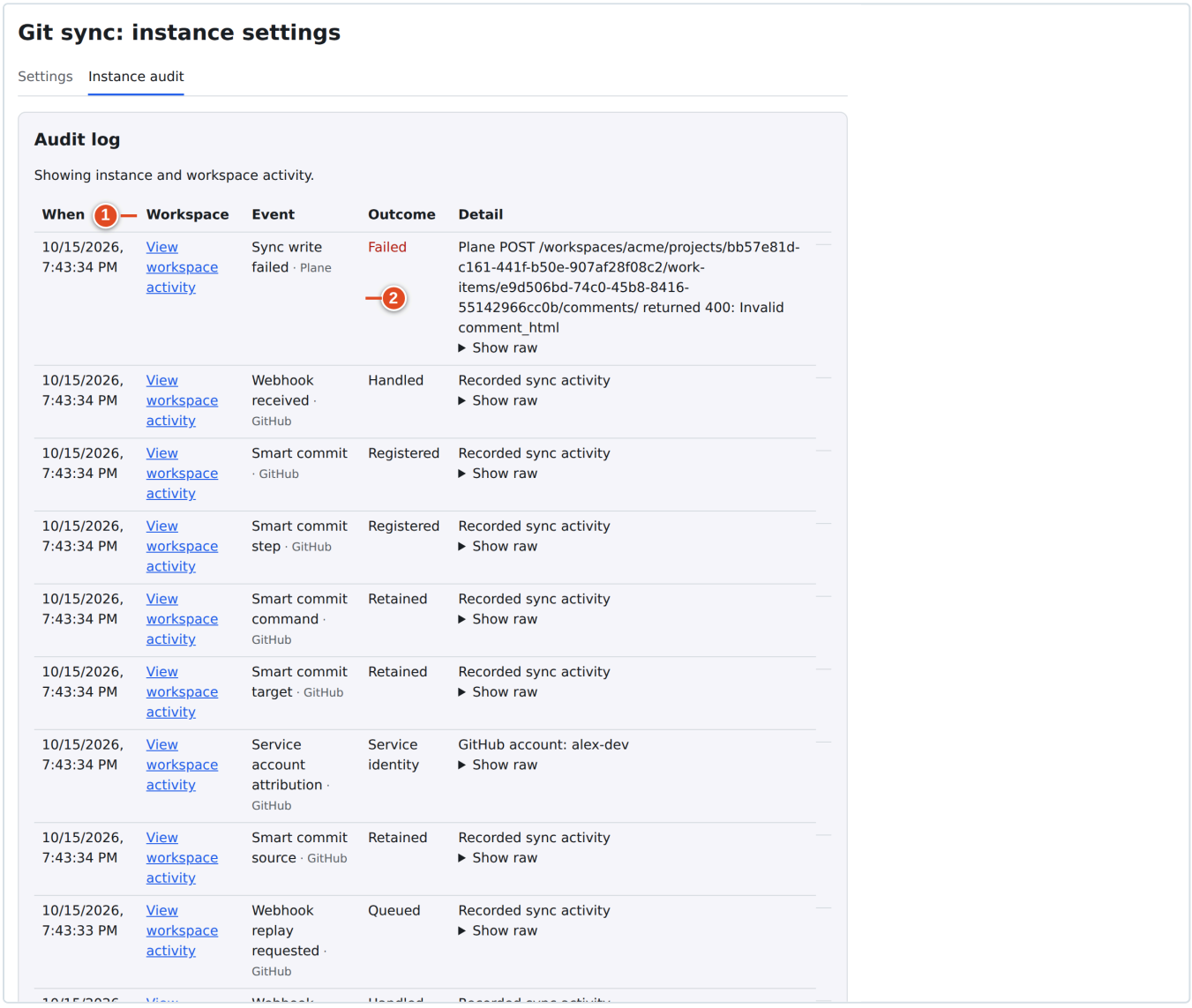


Figure 28.8 Screenshot (demo instance): Instance audit across all workspaces — (1) workspace column; (2) entries.

Without `?scope=all` the instance audit shows only activity that belongs to no workspace:

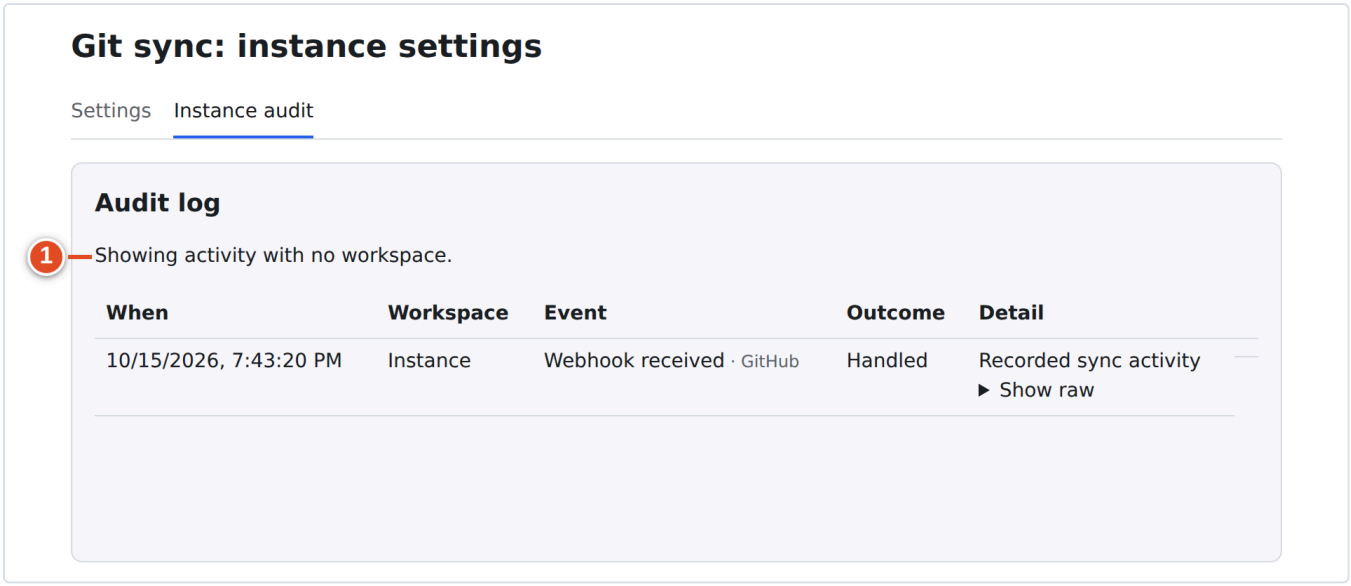


Figure 28.9 Screenshot (demo instance): Instance audit by default: (1) only activity with no workspace (none yet here); add ?scope=all for every workspace.

What to send when asking for help

When a developer reports a problem (chapter 17 tells them what to collect), gather:

1. The work item key, the PR / commit link, and the time.
2. The mapping's health card (screenshot without raw data).
3. The audit entries around that time, with **Show raw** expanded, shared privately.
4. God mode **Setup checks** results.
5. For operators: `docker compose … logs --tail 200 git-sync`, with secrets removed.

What you learned

- `/healthz` proves liveness only; use God mode checks and per-mapping health cards for real status.
- Health cards show last deliveries, last write, 24-hour errors, and, only when relevant, **Rate-limited until** and **Disabled**.
- The audit log explains every decision. **Show raw** reveals details, **Replay** re-runs a delivery, **Retry** re-runs a failed write.
- Filters are URL parameters (`errors`, `mappingId`, `hours`, `scope=all`); there are no on-page filter controls.

Source files

- `src/ui/client/health.tsx` — cards, banners, checklist
- `src/ui/client/Workspace.tsx` — `AuditLogPage`
- `src/ui/client/audit.tsx` — event labels
- `src/ui/client/router.ts` — audit URL filters
- `src/ui/api.ts` — `GET /health`, `GET /audit`, `POST /audit/:id/replay`
- `src/sync/reconcile.ts` — silent-webhook detection

29 · Operations: update, rotate, replay, back up, uninstall

This chapter is the operator's runbook. All commands run *on the Plane server, in your Plane directory*, with the filenames your deployment uses. Wherever you see `docker compose …`, use the *full start command the installer printed* (both Compose files and the env file), for example:

```
cd /root/plane && docker compose -f docker-compose.yaml -f docker-compose.override.yaml --env-file plane.env up -d
```

Daily checks

Check	How	Good
Service alive	`curl -i https://plane.example.com/git-sync/healthz`	`200` `{"status":"ok"}`
Container healthy	`docker compose … ps git-sync`	`healthy`
Recent logs	`docker compose … logs --tail 100 git-sync`	No repeated errors
Instance status	God mode → **GitHub Sync** → **Connection health**, **Setup checks**	Database reachable, checks Verified
Per-repository status	Workspace → **Mapped repositories** health cards	Recent deliveries, few errors



Figure 29.1 Screenshot (terminal): the health endpoint of a running service.

Update git-sync

Goal: run a newer git-sync, keeping all settings and data

Before you start: back up (below). Write down the exact installer options this deployment uses (`--plane-dir`, `--env-file`, `--proxy external --gateway-network …`, `--keep-plane-ui`).

Steps

1. Enter the checkout and pull:

```
cd ~/git-sync
git pull
```

2. Re-run the installer with the same options:

```
sudo ./install.sh --plane-dir /root/plane          # plus your other options
```

3. Check `/git-sync/healthz`, God mode **Setup checks** and a mapping's health card.

Result:

- The image `git-sync:local` is rebuilt and the service restarted.
- Database migrations run automatically on start; they are ordered, recorded, and safe with several replicas.
- `git-sync.env`, `GITSYNC_TOKEN_KEY`, the database and stored secrets are kept. The installer does **not** ask for the Plane secrets again.

If it went wrong:

- Keep the previous checkout revision, and the previous image if you tagged it. To roll back, check out the old revision and re-run the installer.
- A schema-incompatible rollback needs the database backup taken before the update: there are no down-migrations.
- Existing Plane UI fork images are **reused**, not rebuilt. If a release changes the fork patches, remove the old `plane-frontend-gitsync:v1.4.2` / `plane-admin-gitsync:v1.4.2` images before re-running, so they are rebuilt.

> **Note:** "Change a setting" uses the same command: re-run `./install.sh` with the new option, for example `--public-url`. Existing values are kept unless you supply new ones.

Upgrading Plane itself

The UI fork is pinned to Plane **v1.4.2**. If you upgrade Plane:

1. The installer keeps Plane's stock UI automatically for versions it does not recognize. git-sync keeps working; use the direct `/git-sync/ui/…` links.
2. To get the menus back, the fork's eight files must be re-applied and rebuilt for the new release (see `apps/README.md`, `scripts/verify-fork-patch.mjs`).
3. Always start Plane with the installer's printed command, not Plane's `setup.sh`, so the override (and git-sync) is included.

Rotate the bot's Plane API key

Goal: replace `PLANE_SERVICE_PAT` without downtime

Steps

1. Signed in **as the bot**, create a new Personal Access Token (chapter 21, step 2).
2. Re-run the installer with the new key in the environment, without putting it in your shell history:

```
read -r -s -p 'New bot PAT: ' GITSYNC_PLANE_PAT
export GITSYNC_PLANE_PAT
sudo --preserve-env=GITSYNC_PLANE_PAT ./install.sh --plane-dir /root/plane
unset GITSYNC_PLANE_PAT
```

(On rootless Docker, run without `sudo` as the user who owns Plane.) 3. The installer verifies the key with Plane, writes it and the bot's user ID to `git-sync.env`, and restarts git-sync. 4. Revoke the old token in Plane after checking that writes still work (make a test Smart Commit or edit a mirrored issue).

If the stored key was revoked first, an interactive re-run prints "Plane rejected the API key" and asks for a new one.

> **Warning:** Never change `GITSYNC_TOKEN_KEY` when rotating the PAT. It is the encryption key for stored secrets, not a Plane credential.

Rotate the Plane webhook secret

git-sync accepts the current secret and, during a rotation, one previous secret.

1. Note the current value of `PLANE_WEBHOOK_SECRET` in `git-sync.env`.
2. In Plane: **Workspace settings** → **Webhooks** → the git-sync webhook → regenerate the secret key. Copy it.

3. In `git-sync.env` set `PLANE_WEBHOOK_SECRET=<new>` and `PLANE_WEBHOOK_SECRET_PREVIOUS=<old>`. Then run `docker compose … up -d git-sync`.
4. After an hour, delete `PLANE_WEBHOOK_SECRET_PREVIOUS` and run `up -d git-sync` again.

Keep the gap between steps 2 and 3 short, at a quiet time. A Plane delivery that matches neither secret is refused with `401`, and Plane never resends it.

Rotate GitHub App credentials

Credential	How
Private key	GitHub: *Generate a private key*. God mode: paste it into **Private key (PEM)**, leave other secrets empty, **Save GitHub App**. Then delete the old key on GitHub.
Client secret	GitHub: generate a new one. God mode: **OAuth client secret** → **Save GitHub App**. Delete the old one on GitHub.
Webhook secret	Change it on GitHub and in God mode **Webhook secret** close together. There is no “previous” GitHub secret, so deliveries signed with the other secret in between fail with `401`; redeliver them (below).

New credentials are used within about five seconds of saving; no restart is needed.

The encryption key (`GITSYNC_TOKEN_KEY`)

- It encrypts the App’s private key, webhook secret and client secret, members’ GitHub tokens, and in-flight OAuth verifiers.
- It must decode to exactly 32 bytes. The installer generates it once and *always keeps it*.
- **Do not rotate it.** There is no re-encryption command. With a different key, every stored secret becomes unreadable: God mode shows **GitHub App credentials cannot be decrypted. Save the App secrets again.**, GitHub webhooks get `503`, and members must reconnect.
- If the key was lost, chapter 30 explains recovery: restore the key, or re-enter every secret.

Redeliver GitHub webhooks

GitHub does **not** retry a delivery that received an error. After any outage (git-sync down, `503` while secrets were unreadable, `401` during a secret change):

1. GitHub → **Settings** → **Developer settings** → **GitHub Apps** → your App → **Advanced** → **Recent deliveries**.

2. Redeliver every delivery marked failed since the problem began.

Redelivery is safe: git-sync deduplicates by delivery id, and writes are idempotent.

Replay a delivery from the audit log

If a delivery **did** reach git-sync but was processed wrongly (for example, before you fixed a missing state or a rule), a workspace admin can replay it:

1. **Audit log** → find the **Webhook received** entry.
2. **Replay** → confirm “Replay this delivery? It runs again under a new delivery id and may write to Plane and GitHub.”
3. The entry **Webhook replay requested** appears, followed by the new processing.

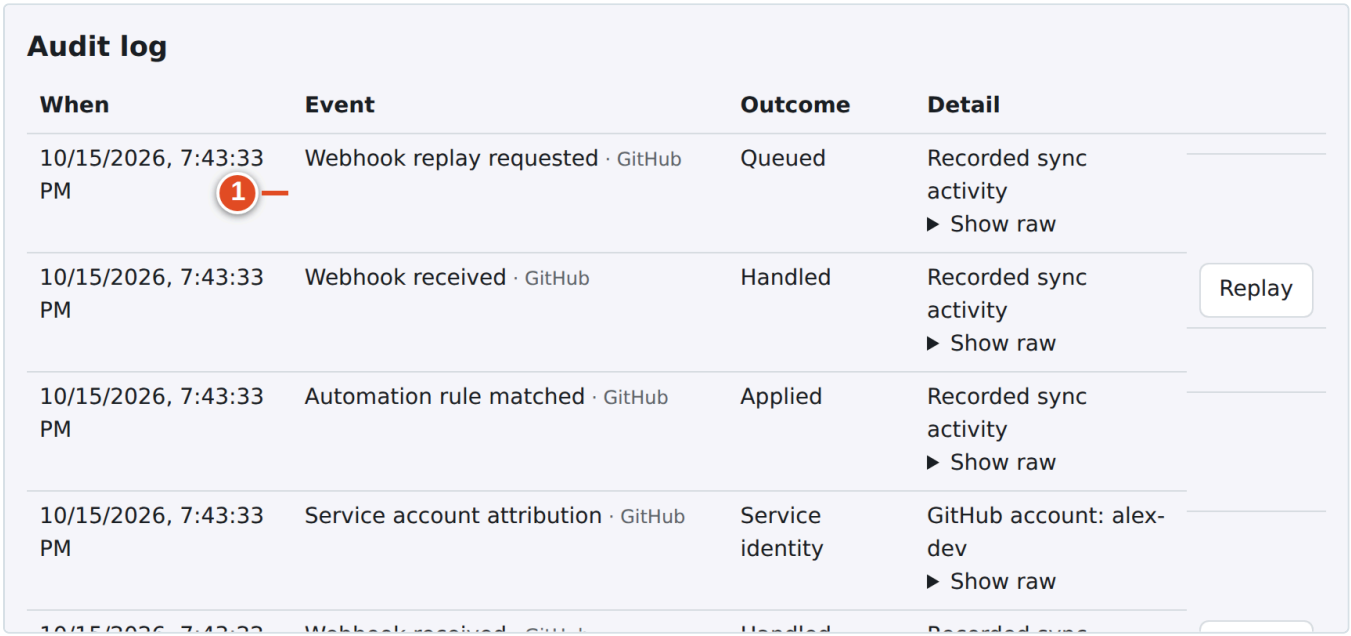


Figure 29.2 Screenshot (demo instance): Replay — (1) the confirmation that the delivery was queued again.

Writes that already succeeded are recognized and not repeated. Only deliveries whose raw payload is still retained (audit payload retention: 90 days) can be replayed.

Retry a failed write

When an outbound write failed permanently (for example a permission was missing, or a state was deleted), fix the cause and then:

1. **Audit log** → open errors (`?errors=1`) → the **Sync write failed** entry.
2. Select **Retry**. It changes to **Retry scheduled**.

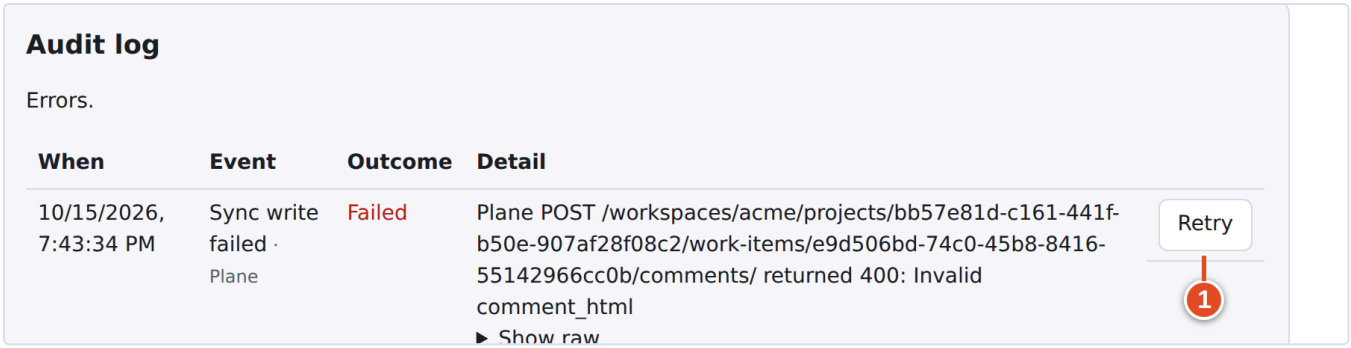


Figure 29.3 Screenshot (demo instance): a failed write — (1) Retry.

The retry re-checks the mapping's authority first. A write whose installation has since been revoked is still refused. A write that is merely rate-limited retries by itself at the deadline; leave it alone.

Re-enable a Plane webhook that Plane switched off

Plane v1.4.2 switches a webhook off after about 5 failed connection attempts over 30–50 minutes, and emails its creator. git-sync notices with the next reconciliation (default every 60 minutes). It audits **Plane webhook has stopped arriving** and shows the banner "Plane changes have stopped arriving…".

1. Fix what made git-sync unreachable; check `/git-sync/healthz`.
2. Plane: **Workspace settings** → **Webhooks** → the git-sync webhook → switch it on.
3. Changes made while it was off are **not** resent. Re-save the affected work items in Plane (or make the matching change on GitHub). The audit entry's `latestChangeAt` and `lastDeliveryAt` show the gap.

When Valkey alone is down, Plane deliveries are kept in Postgres and replayed every minute once it is back, so nothing is lost. If both Valkey and Postgres are down, Plane deliveries get `503` and are lost.

Backups

Back these up together:

What	Where
The `gitsync` database	Plane's Postgres: `docker compose … exec -T plane-db pg_dump -U <plane superuser> gitsync > gitsync.sql`
`git-sync.env` (contains `GITSYNC_TOKEN_KEY` and secrets)	Plane's directory, mode 0600
`git-sync/` directory (App key file, credential-state marker)	Plane's directory

What	Where
`docker-compose.override.yaml` and any external proxy config	Plane's directory / your proxy

> **Warning:** A database backup without the matching `GITSYNC_TOKEN_KEY` cannot decrypt its secrets. Store them together, privately. The installer's `.bak-<time>` files also contain secrets.

Take the backup at a quiet time, or stop git-sync briefly (`docker compose … stop git-sync`). Valkey jobs are not in the database; note the backup time, so that after a restore you know which GitHub deliveries to redeliver.

Uninstall

Goal: remove git-sync and keep its data

```
sudo ./install.sh --uninstall          # add --plane-dir/--env-file as used
at install
```

This removes the `git-sync` service and the installer-managed override entries and proxy route. It restarts Plane's proxy and apps without git-sync ("Restarting Plane's proxy and apps without git-sync"). The `gitsync` database, `git-sync.env` and the `git-sync/` directory are **kept**, so a later install picks everything up again.

Goal: remove git-sync and delete its data

```
sudo ./install.sh --uninstall --purge
```

The installer asks "Delete git-sync's database, settings and stored GitHub credentials for good? [y/N]".

- **Deleted:** the `gitsync` database and role, `git-sync.env` and the state directory.
- **Not deleted:** Plane's database, Plane's data, and the timestamped `.bak-` files, which still contain secrets. Delete those yourself.

After either form:

- Remove any route you added to an **external** proxy yourself.
- Uninstall the GitHub App from your organizations on GitHub if you no longer need it.
- Delete the Plane webhook in **Workspace settings → Webhooks**.
- Revoke the bot's API key.

What you learned

- Update with `git pull` and the same `./install.sh` options; settings, secrets and the encryption key are kept.
- Rotate the bot PAT through the installer, and the Plane webhook secret with the `_PREVIOUS` overlap. Rotate App secrets in God mode. Never rotate `GITSYNC_TOKEN_KEY`.
- Use GitHub **Recent deliveries** for failed GitHub webhooks, **Replay** for deliveries that arrived, and **Retry** for failed writes.
- Back up the database, `git-sync.env` and `git-sync/` together. `--uninstall` keeps data; `--purge` deletes git-sync's data only.

Source files

- `install.sh`, `src/install/env-file.ts`, `src/install/main.ts` — update, rotation, uninstall
- `deploy/APPLY.md` — sections 9–12
- `src/operations/tooling.ts`, `src/operations/outbox.ts` — replay and retry
- `src/sync/reconcile.ts`, `src/queue/cron.ts` — silent-webhook detection, spool replay
- `src/index.ts` — `/git-sync/healthz`

30 · Admin troubleshooting

Start from the symptom, then check the smallest part that could cause it. Reinstalling rarely fixes a wrong callback URL or a wrong account. Before you change anything, collect the evidence that chapter 28 lists (health card, audit entries, setup checks, logs).

A quick triage order

1. `curl -i https://<plane>/git-sync/healthz` — is the service up?
2. God mode → **Connection health** and **Setup checks** — database, credentials, App registration, deliveries.
3. Workspace → the mapping's **Sync health** card — deliveries arriving? **Disabled**? **Rate-limited until**?
4. Workspace → **Audit log** (`?errors=1`) — what did git-sync decide and why?
5. `docker compose … logs --tail 200 git-sync` — startup and connection errors.

Installation and pages

Symptom	Likely cause	Fix
Installer errors	—	See chapter 21, “Troubleshooting the installer”
GitHub Sync page inside Plane is blank	Proxy adds `X-Frame-Options: DENY` or `frame-ancestors 'none'` to `/git-sync/*`	Exclude `/git-sync/*` from the header in every proxy layer (chapter 21, step 6)
GitHub Sync missing from Plane menus	Stock UI kept (`--keep-plane-ui` or non-v1.4.2 Plane), or Plane was started with `setup.sh`	Use direct links; start Plane with the installer’s printed command
Members ask where **GitHub Sync** is	It is admin-only in Workspace settings	Send them `/git-sync/ui/workspaces/<slug>/account`
Not authorised on `/git-sync/ui/admin`	No God mode session in this browser	Sign in to God mode, reload
Not authorised “Workspace settings need a workspace admin.”	You are a member, not an admin	Ask an admin, or use **My GitHub account**
404 for every `/git-sync/` URL	Proxy route missing (external proxy) or git-sync not running	Add `handle /git-sync/* { reverse_proxy git-sync:3100 }` before the Plane route; check the container
`409` “This workspace address belonged to a Plane workspace that was deleted. An instance admin must remove its old git-sync data before it can be used again.”	A new workspace reuses a deleted workspace’s slug	Instance admin: on the God mode page, open the browser console and run `fetch('/git-sync/api/instance/workspaces/<slug>/stale-data', {method: 'DELETE'})` (API only, no button). This deletes git-sync’s old data for that slug after checking the live workspace.

GitHub App and credentials

Symptom	Likely cause	Fix
App credentials: Needs attention “No GitHub App is configured yet…”	Nothing saved	Enter the App (chapter 23)
GitHub App credentials cannot be decrypted. Save the App secrets again. / **App credentials** “cannot be read or decrypted” / GitHub deliveries answered `503`	`GITSYNC_TOKEN_KEY` changed or lost, or database restored without its key	See “Credentials cannot be decrypted” below
App permissions: Not verified “The App registration could not be read from GitHub…”	Wrong App ID or private key, wrong edition/host, outbound network blocked	Check App ID (a number), re-paste the PEM, check **GitHub edition**, test `curl https://api.github.com` from the container’s network
App permissions: Needs attention “…missing required permission levels…”	Registration lacks a permission	Fix on GitHub (chapter 22); owners approve; **Refresh connection**
App webhook events: Needs attention “Subscribe the App to the missing webhook events.”	An event not ticked	Tick all nine
Signed GitHub deliveries: Not verified	No GitHub event received	Make a PR change; then GitHub **Advanced → Recent deliveries**: `404` = wrong webhook URL, `401` = secret mismatch, `503` = decrypt problem, timeout = proxy/firewall
Plane request rate: Needs attention “The active client request limit differs from PLANE_API_RATE_PER_MINUTE…”	Running service and config disagree	Restart git-sync with the full Compose command
Plane request rate: Not verified	Normal on Plane v1.4.2	Compare Plane’s `API_KEY_RATE_LIMIT` with `PLANE_API_RATE_PER_MINUTE` yourself; keep them equal (e.g. `600/minute` ↔ `600`)

Symptom	Likely cause	Fix
`redirect_uri is not associated with this application` on GitHub	Callback URL or Client ID wrong	Callback must be `<PUBLIC_ORIGIN>/git-sync/oauth/github/callback`; use the App's Client ID
Connect GitHub disabled: "Ask the instance admin to configure the GitHub App client ID, client secret and App slug."	Missing slug/Client ID/secret	Complete **GitHub App** in God mode

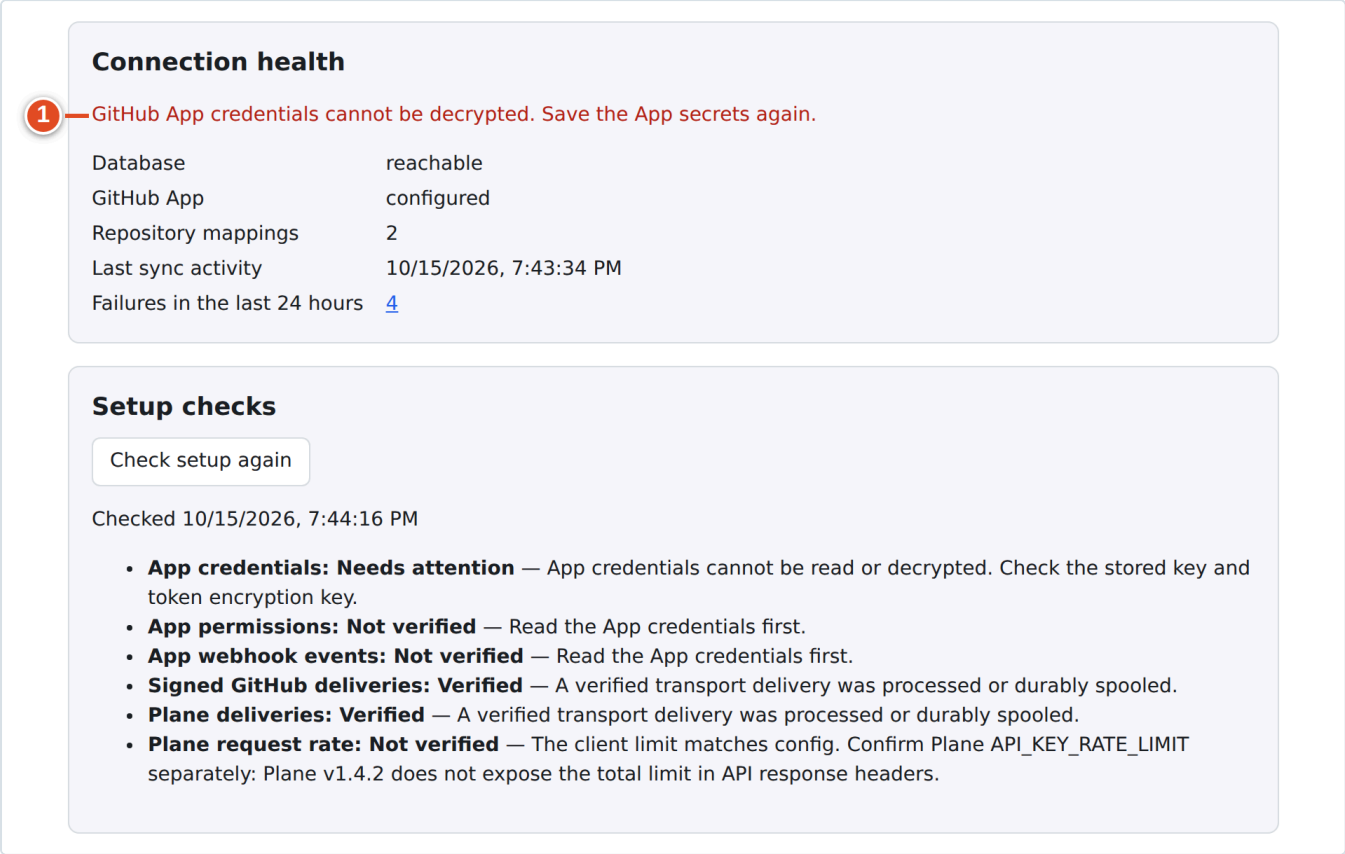


Figure 30.1 Screenshot (demo instance): stored App credentials that cannot be decrypted (GITSYNC_TOKEN_KEY changed) — (1) the error.

Credentials cannot be decrypted

Goal: restore working credentials after a key mismatch

Option A — you still have the right key (best):

1. Find the `GITSYNC_TOKEN_KEY` that matches the database, for example in a backup of `git-sync.env` or an installer `.bak-` file.
2. Put it back in `git-sync.env` and run `docker compose … up -d git-sync`.
3. **Check setup again**, then redeliver failed GitHub deliveries (chapter 29).

Option B — the key is lost:

1. In God mode → **GitHub App**, re-enter **every** secret: **Private key (PEM)**, **Webhook secret** and **OAuth client secret**. Generate a new private key and client secret on GitHub if you do not have them. **Save GitHub App**.
2. Every member must **Reconnect** on **My GitHub account**; their old tokens cannot be read.
3. Redeliver failed GitHub deliveries.

Connecting and mapping

Symptom	Likely cause	Fix
Repository picker empty: “No repositories available…”	No connected installation, or the repository is not included in it	Connect; add the repository to the installation on GitHub; **Refresh repositories**
“Connect your personal GitHub account to select a repository.”	Admin has no personal link	**My GitHub account** → **Connect personal account**
“taken: That installation is connected to another workspace.” / “That installation is already allowed for another workspace; remove it there first”	One installation per workspace	Disconnect it there, or ask the instance admin to remove the allow-list entry
“not_owner: Only an active GitHub organization owner or admin can connect its installation…”	You are not an owner	**Send a link to an organization owner**
“state: This connection link has expired, was already used, or belongs to another Plane account. Start again.”	Link older than 30 min (install) / 10 min (OAuth), reused, or a different Plane session	Start again in one browser
Mapping card shows **Disabled · Installation is not allowed for this workspace**	Installation disconnected/removed/uninstalled	Reconnect or allow-list it, then **Save changes** on the mapping
Disabled · Save the mapping to verify its installation	Mapping without a verified installation	Open it and **Save changes**
“The mapping changed during verification. Reload it and try again”	Someone else edited it meanwhile	Reload, save again

Sync not happening

Symptom	Likely cause	Fix
Plane edits never reach GitHub	Mapping is **GitHub to Plane only**; field policy is **GitHub is the source of truth** (default for title/description); no Plane webhook; the editor is the service account	Check direction and policies (chapter 25); check **Last Plane delivery**; never use a personal PAT as the service PAT
“Plane changes have stopped arriving…” banner	Plane switched its webhook off	Re-enable it; re-save affected items (chapter 29)
Last GitHub delivery: never	Webhook URL/secret wrong, App not installed on that repository	GitHub **Recent deliveries**; installation repository selection
A GitHub issue never appears in Plane	Missing `plane` label, or repository not mapped	Add the label; map the repository
A Plane item never appears on GitHub	Missing `github` label, mapping is **GitHub to Plane only**, or label added by the service account	Add the label as a human user on a two-way mapping
PR does not move the item to Done	Base branch not `^main$`; relation/non-closing reference; another linked PR open; state missing	Audit: **Automation rule matched** *ignored* or *failed*; **Test rules** (chapter 26)
Review requested does nothing	Project has no **In Review** state and several started states	Create **In Review** or change the rule
Smart Commits do nothing	**Smart Commits** off on the mapping; bad grammar; unknown state; sender’s commits unsigned with `REQUIRE_VERIFIED_COMMITS=true`	Enable; turn on **Report problems**; audit `?errors=1`
The signed-commit checkbox is ticked but unsigned commits still run	The checkbox is not enforced in this release	Set `REQUIRE_VERIFIED_COMMITS=true` in `git-sync.env`, `up -d git-sync`

Symptom	Likely cause	Fix
Everything is slow / **Rate-limited until** keeps appearing	GitHub or Plane rate limits	Raise Plane's `API_KEY_RATE_LIMIT` and `PLANE_API_RATE_PER_MINUTE` together; for GitHub, wait — writes resume by themselves
Writes appear as the bot in Plane, not the person	Expected in this release	— (chapter 27)
A deleted Plane item left its GitHub issue open	Deleting is not closing; Plane v1.4.2 sends no delete webhook	Reconciliation marks the link orphaned and comments on GitHub; close the issue yourself if intended

Queues, database and the service

Symptom	Likely cause	Fix
"The sync database is unreachable."	Postgres down, or `DATABASE_URL` wrong	Check `plane-db`; git-sync retries migrations every 5 s and keeps accepting Plane webhooks into Valkey meanwhile
Container restarts repeatedly	Bad `git-sync.env` value (e.g. `GITSYNC_TOKEN_KEY` not 32 bytes, `PLANE_SERVICE_USER_ID` empty, conflicting GitHub edition settings)	Read the first log lines; fix the value; `up -d git-sync`
`healthz` is OK but nothing syncs	Liveness only; workers start after migrations	God mode **Setup checks**, health cards, logs
GitHub events lost during an outage	GitHub never retries errors	Redeliver from **Recent deliveries**

Asking for help

Share, privately:

- the Plane version and git-sync commit (`git rev-parse --short HEAD` in the checkout);
- the non-secret installer options;
- the God mode **Setup checks** results;

- the failing page;
- relevant audit entries;
- log lines with secrets removed.

Never share `git-sync.env`, `.bak-` files, raw tokens or the PEM. Do not run the installer with `bash -x`: it prints secrets.

What you learned

- Triage in order: `healthz` → God mode checks → health card → audit log → logs.
- Most failures are configuration: proxy framing, callback URLs, the nine events, one-installation-per-workspace, direction and policies, and the merge rule's base branch.
- A key mismatch is fixed by restoring the matching `GITSYNC_TOKEN_KEY`, or by re-entering every secret and asking members to reconnect.
- The signed-commit checkbox is not enforced; use the environment variable.

Source files

- `src/ui/setup-check.ts` — check texts
- `src/ui/client/Connect.tsx` — connection messages
- `src/ui/client/health.tsx` — banners
- `src/ui/api.ts` — error texts, stale-data route
- `src/config/env.ts` — configuration validation
- `deploy/APPLY.md`, `INSTALL.md` — runbook and installer troubleshooting

Appendix A · All routes

Every address the git-sync service answers lives under `/git-sync`, on the same origin as Plane. The reverse proxy that serves Plane forwards `/git-sync/*` to the git-sync container on port 3100. In this appendix, `<slug>` is the workspace slug (for example `acme`), and the IDs are Plane IDs.

Who may call:

- **Member**: any signed-in Plane member of that workspace (Plane web session cookie).
- **Workspace admin**: Plane workspace role Admin.
- **Instance admin**: a Plane God mode session.
- **Signed**: requests verified with an HMAC signature, no cookie.
- **Anyone**: no authentication.

Every call that changes something (POST, PUT, PATCH, DELETE) must also carry the service's own `Origin`, so cross-site requests are refused.

Pages (browser)

Address	Page	Who may open it
`/git-sync/ui/workspaces/<slug>`	**Git sync: <slug>**, **Repositories** tab (connection, mapped repositories, map a repository)	Workspace admin
`/git-sync/ui/workspaces/<slug>/mappings/<id>`	Mapping **Settings**	Workspace admin
`/git-sync/ui/workspaces/<slug>/mappings/<id>/policies`	Mapping **Conflict policies**	Workspace admin
`/git-sync/ui/workspaces/<slug>/mappings/<id>/rules`	Mapping **Rules**	Workspace admin
`/git-sync/ui/workspaces/<slug>/mappings/<id>/dry-run`	Mapping **Test rules**	Workspace admin
`/git-sync/ui/workspaces/<slug>/users`	**User links**	Workspace admin
`/git-sync/ui/workspaces/<slug>/audit`	**Audit log**; optional `?mappingId=<id>`, `?errors=1`, `?hours=<1–8760>`	Workspace admin
`/git-sync/ui/workspaces/<slug>/account`	**My GitHub account**	Member
`/git-sync/ui/workspaces/<slug>/projects/<projectId>/work-items/<issueId>/development`	The **Development** panel of one work item (Plane shows it in the work-item sidebar)	Member who can see the work item
`/git-sync/ui/admin`	**Git sync: instance settings**	Instance admin
`/git-sync/ui/admin/audit`	**Instance audit**; `?scope=all` adds every workspace's activity, and the same filters as above apply	Instance admin

Inside Plane, the patched navigation opens these pages in a frame:

- **Workspace settings → GitHub Sync** (`/<slug>/settings/git-sync/`) for workspace admins.
- God mode's **GitHub Sync** (`/god-mode/git-sync/`) for instance admins.

Members do not see the **GitHub Sync** menu item, so they open `/git-sync/ui/workspaces/<slug>/account` directly.

Webhooks, callbacks and health

Method	Address	Who calls it	Purpose
POST	`/git-sync/webhooks/github`	GitHub (Signed: `X-Hub-Signature-256` with the App's webhook secret)	GitHub App events
POST	`/git-sync/webhooks/plane`	Plane (Signed: `X-Plane-Signature` with the Plane webhook secret; the previous secret is also accepted during rotation)	Plane workspace events
GET	`/git-sync/oauth/github/callback`	GitHub, redirecting the browser	Finishes **Connect personal account** (the App's *Callback URL*)
GET	`/git-sync/install/github/callback`	GitHub, redirecting the browser	Finishes an App installation (the App's *Setup URL*)
GET	`/git-sync/healthz`	Anyone	Liveness check used by the container health check

Configuration API (`/git-sync/api`)

Personal account (Member)

Method	Path	Purpose
GET	`/workspaces/<slug>/github/account`	The viewer's link state
POST	`/workspaces/<slug>/github/account/connect`	Start GitHub authorization (returns the authorize URL)
DELETE	`/workspaces/<slug>/github/account`	Disconnect personal account

Workspace (Workspace admin)

Method	Path	Purpose
GET	`/workspaces/<slug>/github/connection`	Connected installations and pending approvals
POST	`/workspaces/<slug>/github/connect`	Start an App installation (**Connect GitHub**)
POST	`/workspaces/<slug>/github/connect/handoff`	Create a link for an organization owner
GET	`/workspaces/<slug>/github/claimable`	Installations the admin may claim
POST	`/workspaces/<slug>/github/installations/<installationId>/claim`	Claim an installation for this workspace
DELETE	`/workspaces/<slug>/github/installations/<installationId>`	**Disconnect** an installation
GET	`/workspaces/<slug>/github/repositories`	Repositories for the picker (through the admin's own GitHub link)
GET	`/workspaces/<slug>/github/user?login=<login>`	**Look up id** of a GitHub login
GET	`/workspaces/<slug>/github/repository?name=<owner/repo>`	Look up one repository
GET	`/workspaces/<slug>/defaults`	Instance defaults for new mappings
GET	`/workspaces/<slug>/projects`	Plane projects
GET	`/workspaces/<slug>/members`	Plane members
GET	`/workspaces/<slug>/projects/<projectId>/states`	A project's states
GET / POST	`/workspaces/<slug>/mappings`	List / add mappings

Method	Path	Purpose
PATCH / DELETE	`/workspaces/<slug>/mappings/<id>`	Edit / remove a mapping
POST	`/workspaces/<slug>/mappings/<id>/backfill`	Backfill an existing repository
GET / PUT	`/workspaces/<slug>/mappings/<id>/rules`	Current rules / save a new version
GET	`/workspaces/<slug>/mappings/<id>/rules/versions`	**Rule versions**
POST	`/workspaces/<slug>/mappings/<id>/rules/versions/<version>/restore`	**Restore** a version
GET / PUT	`/workspaces/<slug>/mappings/<id>/conflict-policies`	Conflict policies
POST	`/workspaces/<slug>/dry-run`	**Run dry-run** (changes nothing)
GET	`/workspaces/<slug>/user-links`	Linked GitHub accounts
PUT / DELETE	`/workspaces/<slug>/user-links/<planeUserId>`	**Link account** / **Unlink**
GET	`/workspaces/<slug>/health`	Sync health
GET	`/workspaces/<slug>/audit`	Audit log entries
POST	`/workspaces/<slug>/audit/<id>/replay`	**Replay** a stored delivery
POST	`/workspaces/<slug>/outbound-writes/<id>/retry`	Retry a failed write

Development panel (Member)

Method	Path	Purpose
GET	`/workspaces/<slug>/projects/<projectId>/work-items/<issueId>/development`	Panel data
POST	`/workspaces/<slug>/projects/<projectId>/work-items/<issueId>/branches`	**Create branch** (project members only)

Instance (Instance admin)

Method	Path	Purpose
GET	`/instance/setup-check`	**Setup checks**
GET	`/instance/health`	**Connection health**
GET / PUT	`/instance/github-app`	**GitHub App** (secrets never returned; blank keeps the stored one)
GET / PUT	`/instance/magic-words`	**Magic words**
GET / PUT	`/instance/settings`	**Defaults**
GET	`/instance/installations`	**Workspace GitHub App installations**
PUT / DELETE	`/instance/installations/<installationId>`	Allow-list / remove an installation
GET	`/instance/workspaces`	Workspaces (from Plane)
GET	`/instance/workspaces/<slug>/projects`	A workspace's projects (for manual recovery)
POST	`/instance/workspaces/<slug>/mappings`	**Add manual mapping**
PATCH	`/instance/workspaces/<slug>/mappings/<id>`	Edit a mapping as instance admin
DELETE	`/instance/workspaces/<slug>/stale-data`	Remove the data of a deleted workspace whose slug was reused
GET	`/instance/disabled-mappings`	**Disabled repository mappings**
GET	`/instance/audit`	Instance audit

Source files

- `src/ui/client/router.ts` (pages)
- `src/ui/api.ts` (configuration API)
- `src/ui/dev-panel.ts` (Development panel)
- `src/api/handler.ts` (webhooks)
- `src/ui/oauth-flow.ts`, `src/ui/install-flow.ts` (callbacks)
- `src/index.ts` (`/git-sync/healthz`)
- `src/app/main.ts` (disabled mappings)

Appendix B · Settings and environment variables

git-sync keeps its configuration in three places:

1. **Environment variables** in `git-sync.env`. The installer writes this file next to Plane's own `plane.env`, and it is read when the container starts.
2. **Instance settings** that an instance admin edits in God mode.
3. **Mapping settings** that a workspace admin edits per repository.

Environment variables (`src/config/env.ts`)

Changing an environment variable takes effect only after the git-sync container restarts.

Variable	Required	Default	Meaning
`DATABASE_URL`	yes	—	PostgreSQL connection for git-sync's own schema (the installer reuses Plane's Postgres server)
`REDIS_URL`	yes	—	Valkey/Redis for the job queues (Plane's Valkey)
`QUEUE_PREFIX`	no	`gitsync`	Prefix of the queue keys
`PORT` / `HOST`	no	`3100` / `0.0.0.0`	Where the service listens
`PLANE_BASE_URL`	yes	—	Plane's API as seen from the container (internal address)
`PLANE_SERVICE_PAT`	yes	—	API key of the bot service account in Plane

Variable	Required	Default	Meaning
`PLANE_SERVICE_USER_ID`	yes	—	Plane user ID of that bot; its own changes are recognized and not echoed back
`PLANE_WEBHOOK_SECRET`	yes	—	Secret of the Plane workspace webhook
`PLANE_WEBHOOK_SECRET_PREVIOUS`	no	—	Old Plane webhook secret, still accepted while you rotate it
`PLANE_SESSION_COOKIE_NAME`	no	`session-id`	Must match Plane's `SESSION_COOKIE_NAME`
`PLANE_API_RATE_PER_MINUTE`	no	`60`	Requests per minute git-sync may send to Plane; must equal Plane's `API_KEY_RATE_LIMIT`
`GITSYNC_TOKEN_KEY`	yes	—	32-byte key that encrypts stored secrets and tokens. **Never change it** on a running install, or stored App credentials and member tokens can no longer be decrypted.
`PUBLIC_ORIGIN`	no	`https://tasks.qelal.et`	The public URL of Plane (used in links, callbacks and the CSRF origin check). The installer always sets it.
`GITHUB_APP_ID`	no[1]	empty	GitHub App ID (bootstrap)
`GITHUB_PRIVATE_KEY_PATH`	no[1]	empty	Path of the App's `.pem` file inside the container
`GITHUB_WEBHOOK_SECRET`	no[1]	empty	The App's webhook secret

Variable	Required	Default	Meaning
`GITHUB_CREDENTIAL_MARKER_PATH`	no	`/var/lib/gitsync/credentials/bootstrap-retired`	Records that bootstrap credentials were replaced in the UI
`GITHUB_EDITION`	no	`github`	`github`, `ghe_cloud` or `ghes`
`GITHUB_GHE_SUBDOMAIN`	with `ghe_cloud`	—	Enterprise name before `.ghe.com`
`GITHUB_GHES_BASE_URL`	with `ghes`	—	Enterprise Server URL
`GITHUB_API_BASE_URL`	no	derived	Legacy way to set the API host
`GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET`	no[2]	—	OAuth client of the App, for **Connect personal account**
`GITHUB_APP_SLUG`	no	—	App slug, for the **Connect GitHub** install link
`REQUIRE_VERIFIED_COMMITS`	no	`false`	`1`, `true` or `yes`: Smart Commit commands run only for commits GitHub marks as verified. **This is the switch that is enforced** (see Appendix E).

[1] The three App variables are set together or left empty together. When they are empty, the App is configured in God mode and stored encrypted in the database. App credentials saved in God mode take priority over the environment.

[2] Both are set together. They can also be entered in God mode.

Installer-only variables

Variable	Used by	Meaning
`GITSYNC_PLANE_PAT`	`install.sh -y`	The bot's Plane API key, for a non-interactive install
`GITSYNC_PLANE_WEBHOOK_SECRET`	`install.sh -y`	Plane webhook secret
`GITSYNC_GITHUB_WEBHOOK_SECRET`	`install.sh -y` with `--github-app-id`	GitHub webhook secret

In Plane's own `plane.env`, the installer relies on two variables:

- `WEBHOOK_ALLOWED_HOSTS` must include `git-sync.internal`, so that Plane may call `http://git-sync.internal:3100/git-sync/webhooks/plane`.
- `API_KEY_RATE_LIMIT` must match `PLANE_API_RATE_PER_MINUTE`.

Instance settings (God mode → GitHub Sync)

Section	Field	Default	Notes
GitHub App	**App ID**, **Private key (PEM)**, **Webhook secret**, **App slug**, **OAuth client ID**, **OAuth client secret**, **GitHub edition** (plus **Enterprise Cloud subdomain** or **Enterprise Server URL**)	—	Secrets are encrypted and never shown again; leave a secret empty to keep the stored one
Defaults	**Enable Smart Commits for newly mapped repositories**	on	Pre-ticks **Smart Commits** in the mapping form
Defaults	**Require signed commits for Smart Commits**	the value of `REQUIRE_VERIFIED_COMMITS`	Stored and shown, but **not applied** by this release; use the environment variable
Defaults	**Reconciliation interval (minutes)**	60	How often missed webhooks are reconciled (1–10080)
Defaults	**Echo-suppression TTL (minutes)**	10	How long an applied change is remembered so it is not synced back (1–10080)
Magic words	closing, non-closing and relation lists	see Appendix C	Lowercase letters and spaces, up to 40 characters, 1–100 per tier, one tier per word
Workspace GitHub App installations	**Installation ID**, **Workspace**, **Account / Org (optional)**	—	One installation belongs to one workspace

Database-only retention settings, with no UI:

- Webhook deliveries are kept 30 days.
- Processed commits are kept 180 days.
- Audit payloads are kept 90 days.

- Finished outbound writes are kept 30 days.

Mapping settings (Workspace settings → GitHub Sync)

Field	Values	Default
GitHub repository / **Plane project**	one repository ↔ one project	—
Sync direction	**Two-way**, **GitHub to Plane only**	Two-way
Open state / **Closed state**	a project state or **Project default**	Project default
Smart Commits	on / off	the instance default
Post link-back comments on GitHub	on / off	on
Report problems as comments on GitHub and Plane	on / off	off
GitHub autolink references (unavailable)	disabled	—
Branch name template	`{key}`, `{title-slug}`, `{username}`	`{key}-{title-slug}` (no UI to change it)
Conflict policies (title, description, state, assignees, labels, comments)	**Last write wins**, **GitHub is the source of truth**, **Plane is the source of truth**	title and description: GitHub is the source of truth; the rest: last write wins
Rules	lifecycle table, branch-aware rules or YAML	the shipped defaults (Appendix D)

Source files

- `src/config/env.ts`
- `src/install/env-file.ts`, `install.sh`
- `src/ui/validate.ts`
- `src/ui/client/Admin.tsx`, `src/ui/client/Workspace.tsx`
- `src/db/migrations/*.sql`

Appendix C · Reference, magic-word and Smart Commit grammar

Every table in this appendix is **generated**. `manual/scripts/grammar-tables.mjs` runs each example through the real parsers in `src/parse/references.ts` and `src/github/smart-commits.ts`, then writes the result here. The tables are rebuilt with every manual build, so they always match the code. The examples come from `tests/parse/references.test.ts`, `tests/github/smart-commits.test.ts`, `tests/a8/grammar.test.ts` and the chapters of this manual.

Work-item keys

A key is a project prefix, a hyphen and a number: `WEB-42`, `API2-103`. The prefix starts with a letter, followed by 1–11 letters or digits, so it is 2–12 characters long. The key must not be glued to letters or digits on either side, and no `_` may follow the number. Leading zeros are dropped, so `WEB-0042` is `WEB-42`. A recognized key still has to exist in Plane: git-sync looks it up, and an unknown key is ignored and written to the audit log.

Prefixes that are never read as work-item keys (`NOT_PROJECTS` in `src/parse/references.ts`): `UTF`, `SHA`, `ISO`, `AES`, `RSA`, `CVE`, `CWE`, `RFC`, `TLS`, `SSL`, `MD`, `ES`, `PEP`, `USB`, `HTTP`, `IPV`, `SSH`, `GPT`, `RS`, `CRC`, `UTC`, `WIN`.

Where git-sync looks, and what a bare key means

Place	Case of a bare key	Tier of a bare key	Tier after a magic word
Pull request head branch	any case	closing	the word’s tier
Pull request title	UPPERCASE only	closing	the word’s tier
Pull request body	UPPERCASE only	relation	the word’s tier
Commit message (push rules only)	UPPERCASE only	relation	the word’s tier

A key that follows a magic word may be in any case. The pull request handler reads only the branch, title and body. Commit messages never link a pull request; they are read only by Smart Commits and by `push` / `commit.pushed_to_default` rules.

When the same key appears several times, the **strongest tier wins** (closing > non-closing > relation). If two places give the same tier, the order branch > title > body > commit decides which source is kept.

Default magic words

Tier	Default words and phrases (from `src/parse/magicWords.ts`)
closing	`close`, `closes`, `closed`, `closing`, `fix`, `fixes`, `fixed`, `fixing`, `resolve`, `resolves`, `resolved`, `resolving`, `complete`, `completes`, `completed`, `completing`, `implement`, `implements`, `implemented`, `implementing`
non-closing	`ref`, `refs`, `references`, `part of`, `contributes to`, `toward`, `towards`
relation	`relates to`, `related to`

An instance admin can replace these lists in God mode, under **Magic words**. Each entry may contain only lowercase letters and single spaces, up to 40 characters. Each tier needs 1–100 entries, and a word may belong to one tier only.

A magic word is followed by an optional colon and then **at least one space**. Several keys can follow one word, joined by `,`, `and`, `&` or spaces.

Reference examples

Text	Where	Result (real parser)	Why
`WEB-42-checkout-button`	branch	WEB-42 (closing)	bare key in a branch
`feature/web-42-login`	branch	WEB-42 (closing)	branch keys are case-insensitive
`WEB-42 Keep the checkout button visible`	title	WEB-42 (closing)	bare key in a PR title
`web-42 keep the button visible`	title	no reference	lowercase bare key in a title
`Closes WEB-42`	body	WEB-42 (closing)	closing magic word
`fixes web-42`	body	WEB-42 (closing)	key after a magic word is case-insensitive
`Refs WEB-41`	body	WEB-41 (non-closing)	non-closing magic word
`Part of WEB-41`	body	WEB-41 (non-closing)	two-word non-closing phrase
`Relates to WEB-40`	body	WEB-40 (relation)	relation magic word
`See WEB-40 for context`	body	WEB-40 (relation)	bare uppercase key in a body
`see web-40`	body	no reference	lowercase bare key in a body
`Closes WEB-42, WEB-43 and API2-7`	body	WEB-42 (closing); WEB-43 (closing); API2-7 (closing)	several keys after one word
`Fixes WEB-1 & WEB-2`	body	WEB-1 (closing); WEB-2 (closing)	& joins keys
`Closes: WEB-42`	body	WEB-42 (closing)	optional colon, then a space

Text	Where	Result (real parser)	Why
`Closes:WEB-42`	body	WEB-42 (relation)	no space after the colon: only the bare key counts
`Closes WEB-0042`	body	WEB-42 (closing)	leading zeros are normalised
`X-42 WEB_42 WEB-42more 42-WEB`	body	no reference	not keys
`UTF-8 SHA-256 CVE-2024`	body	no reference	deny-listed prefixes
`Closes WEB-42⏎skip WEB-42`	body	skipped: WEB-42	skip removes the key from this parse
`Fixes WEB-42⏎ignore WEB-43⏎Fixes WEB-43`	body	WEB-42 (closing); skipped: WEB-43	ignore works like skip
`WEB-42 Move the button`	commit	WEB-42 (relation)	bare key in a commit message

Inputs	Result (real parser)	Why
branch: `web-42-checkout` body: `Refs WEB-42`	WEB-42 (closing, from branch)	branch (closing) beats body Refs (non-closing)
title: `Checkout tweaks` body: `Refs WEB-42⏎Relates to WEB-42`	WEB-42 (non-closing, from body)	non-closing beats relation
title: `WEB-NEW Add rate limiting`	create in: WEB	WEB-NEW asks for a new item
title: `Tidy styles` body: `web-NEW and WEB-new`	create in: WEB	only uppercase NEW counts

`skip KEY` and `ignore KEY` count only in a pull request body, are case-insensitive and cover one key each. The key is dropped from this parse wherever it appears (branch, title or body), but a link that git-sync already stored is **not** removed.

`WEB-NEW` in a title or body asks git-sync to create a new work item in the project whose identifier is `WEB`. `NEW` must be uppercase. This happens on the `opened`, `edited` and `reopened` events.

Smart Commit grammar

`<anything> KEY [KEY…] <anything> #command [args] #command [args] …`

- Each line of a commit message is read on its own. A line needs at least one key **before** its first `#command`.
- `#comment <text>` takes the rest of the line as the comment. A `#tag` or key inside the comment stays text.
- `#time <duration>` accepts `w`, `d`, `h` and `m` units, with decimals; 1w = 5d and 1d = 8h. It is posted as a Plane comment, `Time logged: <duration> (<n> minutes)`.
- Any other `#word` is a workflow-state command.

Commit message	Result (real parser)
`WEB-42 #comment Checked at 320, 375 and 390 px`	keys WEB-42 · comment “Checked at 320, 375 and 390 px”
`Fix layout WEB-42 #time 1h 30m`	keys WEB-42 · time 1h 30m = 90 min
`WEB-42 #time 1w 2d 4h 30m`	keys WEB-42 · time 1w 2d 4h 30m = 3630 min
`WEB-42 #time 1.5h`	keys WEB-42 · time 1.5h = 90 min
`WEB-42 #in-review #time 45m #comment Ready for Sam`	keys WEB-42 · state #in-review · time 45m = 45 min · comment “Ready for Sam”
`WEB-33 WEB-34 #in-progress #comment Shared validation helper`	keys WEB-33, WEB-34 · state #in-progress · comment “Shared validation helper”
`WEB-42 #comment see #12 and WEB-7 — this stays text`	keys WEB-42 · comment “see #12 and WEB-7 — this stays text”
`WEB-42 #time 3x`	keys WEB-42 · error: #time needs a duration such as 1w 2d 4h 30m, got “3x”
`WEB-42 #comment`	keys WEB-42 · error: #comment has no text
`#comment no key before the command`	nothing runs
`Done. #comment WEB-42 after the command`	nothing runs
`WEB-42 updated the docs`	nothing runs
`WEB-42 #done⏎WEB-43 #comment Second line, own key`	keys WEB-42 · state #done ‖ keys WEB-43 · comment “Second line, own key”

Commit message	Result (real parser)
`WEB-42⏎#comment key on the line above`	nothing runs

State commands

A state command is matched against the states of the work item's own project, in three steps:

1. The full state name with spaces as hyphens (`#in-review`).
2. The same name ignoring hyphens and apostrophes (`#inreview`).
3. The leading whole words of exactly one state's name.

If two states match at the same step, the command is refused as ambiguous. A command never moves an item into a triage state.

States of the example project: Backlog, Todo, In Progress, In Review, In QA, Done, Cancelled.

Command	Result (real resolver)
`#in-review`	→ In Review
`#In-Review`	→ In Review
`#inreview`	→ In Review
`#done`	→ Done
`#todo`	→ Todo
`#in-qa`	→ In QA
`#in`	refused: ambiguous
`#progress`	refused: no such state
`#cancel`	refused: no such state
`#shipped`	refused: no such state

Source files

- `src/parse/references.ts`, `src/parse/magicWords.ts`
- `src/github/smart-commits.ts`
- `src/ui/validate.ts` (magic-word validation)
- `manual/scripts/grammar-tables.mjs` (this appendix's generator)

Appendix D · Rule reference: triggers, conditions, actions

Each mapping has a list of rules, stored as YAML in versions. Every event is checked against the list from the top. **The first rule whose trigger and conditions match is the only one that runs.** If no rule matches, nothing happens.

The **Rules** tab writes this YAML for you: its lifecycle table and branch-aware rules compile to it. You can also edit the YAML directly in **Rules (YAML)**. The `yaml` and `example` blocks in this appendix were checked with the real parser (`parseRules` in `src/rules/engine.ts`).

```yaml
- name: Merged into staging moves to QA     # optional, shown in the audit
log
  when: pull_request.merged                 # one trigger, or a list of
triggers
  if:                                       # optional; every condition must
hold
    base_branch: "^staging$"
  then:                                     # one or more actions, run in
order
    - set_state: started:In QA
```

Shipped defaults

A new mapping uses the lifecycle table below until an admin saves something else.

Lifecycle point	Trigger	Plane state
Branch pushed	`branch.created`	`started:In Progress`
Draft PR opened	`pull_request.drafted`	`started:In Progress`
PR opened / ready for review	`pull_request.opened`	`started:In Progress`
Review requested	`pull_request.review_requested`	`started:In Review`
Merged (base branch matches `^main$`)	`pull_request.merged`	`completed:Done`

Lifecycle point	Trigger	Plane state
Changes requested, approved, ready for merge, closed without merge, reopened	—	no change

Triggers

Trigger	When it fires	Which work items it acts on
`pull_request.drafted`	A PR is opened as a draft or converted to draft	Items linked to the PR
`pull_request.opened`	A PR is opened, or a draft is marked ready for review	Items linked to the PR
`pull_request.review_requested`	A reviewer is requested	Items linked to the PR
`pull_request.changes_requested`	A review asks for changes	Items linked to the PR
`pull_request.approved`	A review approves, but the PR is not yet ready for merge	Items linked to the PR
`pull_request.ready_for_merge`	The PR is open and not a draft, has at least one standing approval and no standing change request, and all checks and statuses passed. Fires once per head commit.	Items linked to the PR
`pull_request.merged`	The PR is merged	Items linked to the PR (merge guard applies)
`pull_request.closed_unmerged`	The PR is closed without merging	Items linked to the PR
`pull_request.reopened`	The PR is reopened	Items linked to the PR
`pull_request.*`	Any of the above	—
`branch.created`	The first push of a new branch (not the default branch, not a `revert-…` branch)	Keys in the branch name; finished or cancelled items are never moved back
`push`	Any push	Keys in the branch name and commit messages
`commit.pushed_to_default`	A push to the repository's default branch	Keys in the commit messages

Trigger	When it fires	Which work items it acts on
`check_suite.<conclusion>`, e.g. `check_suite.success`, `check_suite.failure`	A check suite completes; `check_suite.*` matches all	Items linked to the PR of the checked commit
`deployment_status.<state>`, e.g. `deployment_status.success`	A deployment status is reported; `deployment_status.*` matches all	Items linked to the PR of the deployed commit
`issue.<action>`, e.g. `issue.closed`, `issue.labeled`	An event on a mirrored issue (GitHub or Plane)	The mirrored work item
`comment.<action>`, e.g. `comment.created`	A comment on a mirrored issue or linked PR	The mirrored or linked work item

Conditions (`if:`)

Key	Value	Matches when
`base_branch`	regular expression	The PR's target branch matches, for example `^main$` or `^release/`
`head_branch`	regular expression	The source branch matches, for example `^hotfix/`
`labels_any`	list of label names	The GitHub object has at least one of the labels
`labels_all`	list of label names	It has all of the labels
`author`	login or list	The GitHub user who caused the event (case-insensitive)
`repo`	`owner/name` or list	The repository (case-insensitive)
`project`	Plane project ID or list	The work item's project (exact)

An event that has no value for a condition, such as a push with no base branch, does not match a rule that needs it.

Actions (`then:`)

Action	Example	Effect in Plane
`set_state`	`set_state: started:In Review`	Moves the item. The target is `group:Name`, a group alone (`completed`), or a state name. If the name is missing but the group has exactly one state, that state is used.
`add_label` / `remove_label`	`add_label: needs-qa`	Adds or removes a project label; a missing label is created
`assign`	`assign: sam-reviews`	Assigns the Plane member linked to that GitHub login
`comment`	`comment: "Deployed to {{deployment.environment}}"`	Adds a comment
`add_link`	`add_link: {url: "{{deployment.url}}", title: Staging}`	Adds a link
`log_time`	`log_time: 30m`	Adds the comment `Time logged: 30m` (Plane CE has no time tracking)
`create_issue`	`create_issue: {project: WEB, title: "Follow up {{issue.title}}"}`	Creates one new item in that project (ID or identifier only)

Text values can use `{{…}}` placeholders from the event:

- `deployment.environment` and `deployment.url` for deployment triggers.
- `issue.number`, `issue.title` and `issue.url` for `issue.*` and `comment.*` triggers.
- `comment.body` and `comment.url` for `comment.*` triggers.

If a placeholder is missing, the rule fails and the failure is written to the audit log.

Tier gating

How strongly a work item is linked limits **only** `set_state`:

- **Relation** links never change state.
- **Non-closing** links (`Refs WEB-41`) change state on every trigger except `pull_request.merged`.
- **Closing** links change state on merge only when at least one linked PR merged and **no** linked PR is still open. A PR that cannot be read counts as open. This merge guard is checked only at the moment of the merge.

All other actions run for every tier.

Examples

Branch-aware delivery (staging first, then main):

```
- name: Merged into staging moves to QA
  when: pull_request.merged
  if: {base_branch: "^staging$"}
  then: [{set_state: "started:In QA"}]
- name: Merged into main is done
  when: pull_request.merged
  if: {base_branch: "^main$"}
  then: [{set_state: "completed:Done"}]
```

Note a failing build and a deployment on the work item:

```
- when: check_suite.failure
  then: [{add_label: build-failing}, {comment: "CI failed on this work
item's pull request."}]
- when: deployment_status.success
  if: {head_branch: "^release/"}
  then: [{add_link: {url: "{{deployment.url}}", title: "Release
candidate"}}]
```

Hotfixes skip review:

```
- when: [pull_request.opened, pull_request.drafted]
  if: {head_branch: "^hotfix/", labels_any: [urgent]}
  then: [{set_state: "started:In Review"}, {assign: jordan-acme}]
```

> **Note:** Saving YAML replaces the whole list, including the defaults. Copy the default rules you still want into your YAML, or build the rules with the lifecycle table, which compiles them for you.

Source files

- `src/rules/engine.ts`, `src/rules/types.ts`
- `src/github/pr-automation.ts` (defaults, lifecycle compiler, executor, ready-for-merge)
- `src/app/rules.ts`, `src/app/rule-events.ts`
- `src/ui/dry-run.ts` (**Test rules**)

Appendix E · Known limitations and defects

This appendix lists what git-sync does **not** do in this release, and the defects found while writing this manual. Each limitation was checked in the code. Where older documents said otherwise, the code wins. The evidence is in `manual/notes/discrepancies.md`.

Limitations by design or not yet built

Limitation	What to do instead	Evidence
Time is a comment only. `#time` posts `Time logged: <duration> (<n> minutes)`, and the rule action `log_time` posts `Time logged: <duration>`. Plane CE has no time tracking to write to.	Read the comments, or keep time in another tool	`src/github/smart-commits.ts`, `src/github/pr-automation.ts`
Plane-side changes are authored by the bot. Smart Commit comments, rule changes and mirrored edits in Plane show the git-sync service account. The commit's GitHub login appears in the comment ("via 3b2335d by alex-dev").	Connect your GitHub account anyway: changes you make in Plane then reach GitHub as you, and git-sync can recognize you	D-07
The signed-commit checkbox is not enforced. God mode's **Require signed commits for Smart Commits** is saved and shown, but the push handler reads only the environment variable.	Set `REQUIRE_VERIFIED_COMMITS=true` in `git-sync.env` and restart git-sync	D-01, `src/app/push.ts`
GitHub autolink references are unavailable. The option is shown disabled.	Keys in GitHub text are still linked by the Development panel and link-back comments	`Workspace.tsx`
The branch name template cannot be edited. It is always `{key}-{title-slug}`, which can also use `{username}`.	—	D-09
The Development panel has no commit list. It shows pull requests, builds, deployments, previews and branches.	Open the branch or PR on GitHub	`DevPanel.tsx`

Limitation	What to do instead	Evidence
Previews come only from deployments. A deployment URL on `*.vercel.app`, `*.netlify.app`, `*.pages.dev` or `*.amplifyapp.com` becomes a preview. A Markdown "… Preview" link in a PR body is **not** detected.	Have your host report a GitHub deployment	D-11
Commit messages never link a pull request. Only the branch name, title and body do.	Put the key in the branch or PR title	D-06
The merge guard is checked once. If WEB-36 has two PRs and one merges while the other is open, the item stays put. Closing the other PR later does not move it.	Move the item by hand, or merge the last PR	D-08
Deleting a Plane item does not close the GitHub issue. Plane v1.4.2 sends no delete webhook. Reconciliation (hourly by default) marks the link orphaned and comments on GitHub; archived items are recorded silently.	Close the GitHub issue yourself	D-15
Comment edits and deletions are not mirrored. Only new comments are copied.	Post a follow-up comment	`src/sync/mirror.ts`
Unmapped GitHub assignees are dropped, not guessed. Unmapped `@mentions` become links to the GitHub profile.	Link every member under **User links**	D-16
Never synced: priority, dates, estimates, cycles, modules, parent and sub-items, relations, attachments, pages, views and pull requests (PRs are never mirrored as issues).	—	Chapter 13

Limitation	What to do instead	Evidence
Members cannot see the GitHub Sync menu. In Plane, **Workspace settings → GitHub Sync** is for admins only.	Members open `/git-sync/ui/workspaces/<slug>/account` directly	`packages/constants/src/settings/workspace.ts`
The audit log has no filter controls. Filters work only as URL parameters (`?errors=1` , `?mappingId=` , `?hours=`).	Use the **Errors in the last 24 hours** link, or edit the URL	D-12
Saving YAML replaces the default rules.	Keep the defaults in your YAML, or use the lifecycle table	Appendix D
The review-requested default needs an In Review state. If the project has no In Review state but has several started states, the rule fails and the failure is audited.	Add an **In Review** state in the Started group	D-20
The Plane request-rate setup check can say “Needs attention”. It compares git-sync’s active client limit with `PLANE_API_RATE_PER_MINUTE` . Plane CE v1.4.2 does not report its own limit, so git-sync cannot verify Plane’s `API_KEY_RATE_LIMIT` .	Keep `PLANE_API_RATE_PER_MINUTE` equal to Plane’s `API_KEY_RATE_LIMIT` , then restart git-sync	`src/ui/setup-check.ts`
Installer wording. `install.sh --help` says “adds Git Sync to Plane’s menus”, but the menu is labelled **GitHub Sync**.	—	D-22

Defect found while making this manual

1. **The link-back comment for a `WEB-NEW` pull request shows the project, not the new key.** When a PR titled `WEB-NEW Add rate limiting to the sign-up form` created WEB-46 in the demo run, git-sync commented on GitHub:

   ```
   Linked Plane work item:
   - WEB: http://plane.example.com/acme/projects/…/issues/…
   ```

   The comment should say `WEB-46`. The link URL is correct; only the label is wrong. The cause is that `createRequested` in `src/app/pr.ts` (around line 167) passes the project identifier as the item's identifier. *Found in the demo run; text in `manual/notes/demo-output/README.md`.*

Source files

- `manual/notes/discrepancies.md` (D-01 … D-23)
- `src/app/pr.ts`, `src/app/push.ts`, `src/sync/mirror.ts`, `src/sync/dev-info.ts`, `src/sync/github.ts`

Appendix F · Version and how this manual was made

Item	Value
Product	git-sync, the two-way GitHub sync for self-hosted Plane
git-sync version	0.1.0 (`package.json`)
Source commit	`95cc855` (branch `docs/user-manual` , based on `master` at `e0b3a58`)
Plane	Community Edition v1.4.2, with the git-sync UI patches (`plane-frontend-gitsync` , `plane-admin-gitsync` images)
Manual date	15 October 2026
Publisher	Bespo EMS PLC

When git-sync changes, rebuild the manual. If a later release changes a screen or a rule, the code is the authority: check it with `npm run manual:verify` .

Where the pictures come from

Every image caption starts with its source:

- **Screenshot (test Plane)** and **Screenshot (GitHub)** were captured for the developer handbook from the running test Plane CE v1.4.2 and the public GitHub pages of the test repository (`docs/handbook/HANDBOOK_NOTES.md`).

- **Screenshot (demo instance)** was captured for this manual from the **real git-sync service code and user interface**, run locally:
  - It uses a throwaway in-memory PostgreSQL database (PGlite), seeded with the Acme / Website demo data.
  - Only the two network edges are stand-ins: a small in-memory Plane API and an in-memory GitHub API (`manual/demo/server.ts`).
  - Every page, button, message, audit entry and Development-panel row in these pictures was produced by git-sync's own code from realistic webhook deliveries. The deliveries were signed and sent through the real webhook endpoints, then processed by the real handlers and the ordered outbox.
  - No real Plane, GitHub account or production system was touched.
- **Diagram** and **Illustration** are drawings. An Illustration stands in for a screen that could not be captured.
- **[[SCREENSHOT NEEDED]]** marks a screen that needs a signed-in browser that was not available, such as GitHub's own forms or a Plane work item showing the Development section. Each placeholder names the capture command for later.

All screenshots are redacted before capture: email addresses, IP addresses, tokens, long IDs, the App slug, the Client ID and personal avatars are hidden or replaced. Orange numbered callouts match the numbered steps in the text.

Rebuilding

From the repository root, on the `docs/user-manual` branch:

```
npm ci
npm run manual:all          # seed the demo instance, capture, build and
verify
# or step by step:
npm run manual:seed         # start the demo instance with its seed data
npm run manual:capture      # all screenshots; add: -- --only <id>
npm run manual:build        # dist/git-sync-user-manual.pdf, .html and
cheat-sheet.pdf
npm run manual:verify       # labels, grammar tables, secrets, links,
coverage
```

Playwright, Chromium and markdown-it are installed with `npm i --no-save`, so `package.json` and the lock file do not change. The grammar tables are regenerated by `manual/scripts/grammar-tables.mjs` from the real parsers. `manual/scripts/check-labels.mjs` checks that every bold UI label appears in the source code or in a verified capture.

Source files

- `manual/demo/server.ts`, `manual/demo/seed.ts`, `manual/demo/events.ts`
- `manual/scripts/` (capture, build, checks)
- `manual/REVIEW_NOTES.md`, `manual/reports/qa-report.md`

Index of UI labels

Every on-screen label printed in bold in this manual, with the pages where it is explained.

+ Add branch rule 304
A command 129
Accept 83, 84
accessibility 52
Account / Org 270
Account / Org (optional) 269, 362
Account matched by email 101, 319
account or organization 279
Acme 11, 15, 208
Action 310
activity 42
Activity 59
Add access token 235
Add link 57
Add Link 57
Add manual mapping 271, 356
Add mapping 213, 285, 287
Add page 83
Add relation 55
Add selected work items 55
Add sub-work item 55
Add view 75
Add webhook 238
Add work item 45, 66, 83
Admin 223
admins 97
Agree on what Done means 195
alex 159
All 89
all 167
Allow-list an installation 213, 269
Allow-list installation 269
Allow-listed at 270
analytics 77
Analytics 86, 87
and 167
App credentials 265, 269, 343
App credentials: Needs attention 343
App ID 258, 267, 269, 362
App permissions 260, 265, 266, 269
App permissions: Needs attention 261, 343
App permissions: Not verified 343
App slug 259, 267, 362
App webhook events 265, 266, 269, 278
App webhook events: Needs attention 261, 343
Apply state to selected rule 306
APPROVAL 187
Archive 60
archived 160
as a comment 178
as the bot 235, 335
Assigned 34
assignee 51
Assignees 52, 297
Assignment 92
at least one 126, 167
Attach 57
attachment 57
Audit 161
Audit log 206, 278, 288, 311, 321, 324, 337, 341
authenticated webhook sender 134
Authorize 254
Automation rule matched 116, 314, 346
Automations 86
avatar 24
Back to repositories 290
Backfill 285, 288, 290, 293
Backlog 189
bare 109
base 110
base branch 17
Base branch (regex) 304
Blocked by 50, 55, 71
Blocking 55
Board 65
body 109, 117, 121
bold 10, 211
both 134
branch 17, 199
Branch already exists 148, 177
Branch created 147, 177
Branch name 316
Branch pushed 178
branch_created 147
branch-aware rule 169
Branches 116, 142, 145, 150, 179
Builds 142, 150, 182, 202
Calendar 65
Callback URL 255
Cancel 104

Cancelled 50, 171
cancelled 158
cannot 96
Check run 257
Check setup again 265, 267, 269, 344
check suite 167
Check suite 257
Checkout experience 15, 79
Checks 144, 256, 257
Client ID 259
Client secret 259
close 63
Close 170
Closed state 155, 157, 158, 162, 284, 286, 293, 363
closing 113, 121, 124, 146, 178, 180, 190, 191
Closing words 272
Cmd + K 35
command palette 35
comment 59, 131
Comment 59
Comments 27, 28, 93, 159, 298
comments 42
Commit message 109
Commit statuses 256, 257
commits 17
completed 158
Compose plugin 229
Conflict policies 156, 290, 298, 299, 350, 363
conflict policy 156
connect 277
Connect GitHub 254, 269, 279, 288, 344, 353, 360
Connect personal account 99, 100, 104, 106, 204, 213, 254, 255
connected 222
Connection health 250, 264, 321, 330, 333, 341, 356
Contents 256
Contents: write 147
Continue 21, 22, 23, 287, 293
contribute 63
Copied 147
Copy branch name 17, 114, 142, 147, 148, 151, 176, 193
Copy failed; select the name and copy it 147
Copy link 282
Create branch 17, 18, 29, 114, 142, 147, 151, 174
Create GitHub App 258
Create work item 83
Created vs Resolved 86, 87
Ctrl + / 36
Ctrl + K 35
current 298
Cycle 15, 79
cycle 77, 78
Cycles 35, 78
Database 265
Decline 83, 84
Defaults 135, 227, 264, 356, 362
Delete 60
deleted 160
Deleted 339
Deployment status 257
Deployments 142, 144, 145, 150, 182, 210, 256
deployments 144
description 42
Description 44, 297, 298
Detail 325
Development 9, 11, 17, 19, 44, 108, 114, 115
did 337
Direction 287
Disabled 283, 288, 331, 341
Disabled · <reason> 270
Disabled · Installation is not allowed for this workspace 345
Disabled · Save the mapping to verify its installation 345
Disabled repository mappings 264, 275, 356
disables 270
Discard 62
Disconnect 103, 208, 280, 283, 353
Disconnect personal account 99, 103, 106, 208
Display 34, 66, 73
display name 24
Display name 24
Done 17, 50, 91, 118, 122, 125, 126, 165
draft 180
Drafts 19, 61
due date 53
Due date 53
Duplicate of 55
each 138
Echo suppression 161
Echo-suppression TTL (minutes) 275, 362
Edit 287, 290
Effect 190
Email notification preferences 27
Enable Smart Commits for newly mapped repositories 127, 274, 362
encrypted 227
Enter 35, 46
Enterprise Cloud subdomain 260, 267, 362
Enterprise Cloud with data residency 260, 267
Enterprise Server 260, 267
Enterprise Server URL 260, 267, 362
Errors in the last 24 hours 154, 322, 328, 381
Escape 35, 40, 46
estimate 54
Estimate 54, 85

Estimates 77
Event 311, 325
every 248, 345
exactly one 132, 319
except 111, 301
Expires 282
external 339
fail 182
Failures in the last 24 hours 265, 330
filter 65
Finish connecting 281
First name 24
from now on 242
full name 24
G then J 61
G then X 89, 94
Gantt 65
General 255
Git sync 11, 221
Git sync: <slug> 98, 277, 350
Git sync: instance settings 11, 221, 263, 350
git-sync 9, 17
git-sync does 173, 175, 177, 178, 180, 182, 183, 184
git-sync is installed 242
git-sync.internal 228, 237
GitHub 9
github 53, 64, 86
GitHub account 98, 103, 317
GitHub App 95, 222, 254, 264, 265, 267, 275, 344
GitHub App credentials cannot be decrypted. Save the App secrets again 227, 265, 336, 343
GitHub autolink references (unavailable) 257, 285, 293, 363
GitHub Bot 233
GitHub edition 260, 267, 343, 362
GitHub is connected to this workspace 280
GitHub is the source of truth 156, 157, 215, 297, 298, 346, 363
GitHub repository 271, 284, 292, 363
GitHub repository ID 271
GitHub Sync 33, 41, 97, 106, 107, 204, 212, 221
GitHub to Plane only 154, 157, 205, 213, 285, 293, 294, 298
GitHub user id 317
GitHub username 317
GitHub.com 260, 267
Group by 73
Grouping 65
head branch 17, 109
High 174
Home 19, 32
HTTPS 229
identifier 16
In Plane 135
in Plane 300
In Progress 17, 50, 63, 90, 111, 115, 116, 122
In QA 169, 171, 305
In Review 17, 49, 50, 90, 122, 127, 158, 165
inbox 89
Include 206
inside Docker 225
Install 254, 279
Installation 270
Installation ID 269, 270, 362
Installation is not allowed for this workspace 270
Installation link 282
installed 222
instance admin 222
instance administrator 263
Instance audit 263, 330, 350
Intake 16, 35, 77, 83, 84
into Plane 316
invitation 21
Issue comment 257
Issue mirroring 18
Issues 154, 256, 257
Join 28
Join Project 28
Joining a project 21
kept 339
key 16
Keys 129, 218
Labels 52, 159, 298
Last GitHub delivery 322
Last GitHub delivery: never 346
Last name 24
Last Plane delivery 322, 346
Last successful write 322
Last sync activity 265
Last write wins 156, 157, 189, 215, 297, 298, 363
layout 65
leading 132
Lifecycle 304
Lifecycle event 303
Lifecycle mapping table 302
lifecycle points 164
link 57, 74
Link a user 213, 316, 317
Link account 317, 318, 319, 354
Link existing open PRs and issues now 287
Linked GitHub accounts 317
Links 149
links 287
List 65
Load older entries 326
login 91, 159
Look up id 317, 318, 320, 353

Lookup failed 317
magic word 109, 120
magic words 17
Magic words 120, 122, 264, 356, 362, 365
main area 31
Make a copy 60
Manage repository access on GitHub 280
Manual mapping recovery 264
map 277
Map a repository 271, 274, 278, 284, 288, 292
mapped 18
Mapped repositories 285, 290, 321, 333
Members 257
mention 48, 91
Mentions 27, 28, 89, 93
Merge 170
merge guard 188
Merged base-branch regex 303, 304
Metadata 256
Mirror pair 152
mirror pairs 152
mirrored issues 298
Mirroring 218
Module 15
module 77, 79
Modules 80, 81
more than one 132
My GitHub account 11, 19, 95, 96, 97, 98, 100, 103
N then I 40, 45
Name 159
Needs attention 265, 266
Never 166, 348
Never send 206
new 116, 145, 159
New GitHub App 255
New work item 46
No actions would run 312
No GitHub repository is mapped to this project 96, 142, 198
No personal GitHub account connected 98, 100, 103, 106, 204
No state change 303
non-closing 301
Non-closing words 272
none 126, 167
Not 298
Not authorised 263, 278, 342
Not now 287
Not verified 265, 266
notification 27
Notifications 27, 93
number 108
numeric id 319
OAuth client ID 259, 267, 362
OAuth client secret 259, 267, 336, 345, 362
October customer experience 15, 78
off 135
Off 255
On GitHub 135
once per head commit 167
one 96, 112
one extra container 220
One line is one command context 129
open 167
Open in new tab 60
Open state 155, 157, 189, 284, 286, 293, 363
opt-in 152
Order by 73
ordering 65
Organization 279
orphaned 160
Outcome 325
own 220
owner or admin 279
Page 16, 193
page 77, 81
Pages 81
parent 55
Parent 55
pass 182
Payload (JSON) 311
Pending 83
pending 182
Pending organization approval 282
per workspace 238
Permissions 256
personal 279
Personal Access Tokens 235
Personal account 279
Personal account settings need workspace membership 99
Plane 9
plane 53, 64, 86
Plane Community Edition v1.4.2 12, 229
Plane deliveries 266
Plane is the source of truth 156, 157, 215, 297, 298, 363
Plane member 317
Plane project 271, 284, 287, 293, 363
Plane request rate 266
Plane request rate: Needs attention 343
Plane request rate: Not verified 343
Plane webhook has stopped arriving 338
Points 84
Post link-back comments on GitHub 115, 153, 173, 198, 215, 285, 293, 363
PR merged 304
Preferences 19, 26

prefix 108
Previews 142, 144, 145, 150, 182, 202
Priority 51
Private key 259
Private key (PEM) 259, 267, 336, 345, 362
Profile 24
Project 15, 270
Project default 157, 284, 286, 293, 363
Project navigation 31
Projects 28, 35
properties 42
Property changes 27, 28, 91
provenance 178
PRs: DECLINED 142
PRs: MERGED 142, 150, 188
PRs: OPEN 142, 150, 180
public 316, 319
pull request 17
Pull request 257
Pull request review 257
Pull requests 115, 142, 180, 189, 199, 256
purpose 194
push 133, 200, 218
Push 257
Push rules 117
pushed 209
Quicklinks 32
Rate-limited until 322, 331, 341, 347
Raw YAML editor 302, 305
reaction 59
ready for merge 164, 186
Ready for merge 304
Ready for review 164
Reason 270
Recent deliveries 340, 346, 347
Recents 32
reconciliation 160, 226
Reconciliation interval (minutes) 274, 362
Reconnect 99, 103, 106, 204, 208, 213, 255, 260
Recorded delivery (audit entry id) 311
Redirect on update 255, 261, 284
Reference 152
reference 164
Refresh connection 260, 282, 343
Refresh mappings 270
Refresh repositories 284, 288, 345
Refresh sync health 321
Register 254
registration 266
REJECTED 184
Relates to 55
relation 199, 301
Relation phrases 273
relation tier 57
Relations 55
Remove 270, 287, 288
Reopen 170
Replay 162, 189, 199, 205, 206, 210, 216, 326
Report problems 135, 139, 206, 275, 295, 346
Report problems as comments on GitHub and Plane 118, 135, 173, 199, 215, 285, 293, 294
Repositories 98, 278, 288, 350
Repository 270, 287
repository is mapped 222
Repository mapped 285
Repository mappings 265
Request 279, 282
Request user authorization (OAuth) during installation 261
Require signed commits 140
Require signed commits for Smart Commits 135, 274, 275, 362, 379
Restore 310, 354
Retries 161
Retry 206, 326, 331, 337, 340
Retry scheduled 326, 337
reused 334
Rule context (JSON) 311
Rule to update 306
Rule trigger 311
Rule versions 310, 354
rules 164
Rules 169, 171, 290, 299, 300, 302, 350, 363
Rules (YAML) 306, 371
Rules to test (YAML, optional) 311, 313
run again 135
Run backfill 287, 293
Run dry-run 311, 354
Save 45
Save changes 24, 270, 288, 292, 345
Save defaults 275
Save GitHub App 267, 336, 345
Save magic words 122, 273
Save mapping table 303, 304
Save policies 298
Save the mapping to verify its installation 270
Save to Drafts 45, 61
Saved 298
Saved at 310
Saved by 310
saved view 65, 75, 77
screenshot 206
secret key 238
Secrets are never printed 248
Send a link to an organization owner 282, 345

Send me everything 238
Send owner handoff link 282
separate 226
service account 24, 90, 226, 229
Service account attribution 102, 134, 319, 320
service identity 134
Set up GitHub sync 278, 320
Settings 24, 27, 33, 93, 235, 263, 272, 287
Setup checks 229, 250, 264, 265, 275, 321, 331, 333
Setup URL 255
Shift + C 79
Shift + E 85
Shift + M 81
Show raw 206, 325, 326, 329, 331
Show sub-work items 73
sidebar 31
Sign up 233
Signed GitHub deliveries 266
Signed GitHub deliveries: Not verified 261, 343
Signing in 21
Smart Commit 127, 152
Smart Commits 18, 117, 127, 133, 173, 178, 209, 274
Source 311
Specific rule trigger 311
Spreadsheet 65
standing approval 167
start date 53
Start date 53
Start work 114, 142, 145, 176, 214
started 123, 191
State 15, 49, 297
state 49
State change 27, 91, 93
state changes 166
state group 15, 49
State Group 72
state token 132
states 77
Status 257
sticky 32
sub-work item 55
Subscribe 60, 92
Subscribe to events 257
subscription 27
Subscription 92
suggested team convention 193
Sync direction 153, 162, 285, 288, 293, 298, 363
Sync health 287, 321, 341
Sync write failed 326, 337
Target state 303, 304, 306
template 146
Test input 311
Test rules 171, 215, 290, 299, 300, 306, 314, 315
That installation is connected to another workspace 96
The instance admin must configure the GitHub App OAuth credentials 99
theme 25
then 215
tier 109, 111
timezone 25, 206
title 109, 121, 199
Title 297, 298
title or body 123
Todo 111, 166
Triage 83, 158
Two-way 153, 154, 213, 285, 293, 294, 363
Unlink 319, 320, 354
Unresolved 270
Unsubscribe 60, 92
Urgent 51
user link 316
User links 101, 103, 105, 204, 222, 278, 288, 350
v1.4.2 335
Validate and save YAML 306
Verified 135, 265, 269
Version 310
View 16
View workspace activity 330
Views 75
WEB 211
WEB-40 166
WEB-41 166
WEB-42 166
WEB-44 154
Webhook 161, 255
Webhook delivery 311
Webhook received 326, 337
Webhook replay requested 337
webhook secret 254
Webhook secret 259, 267, 336, 345, 362
Webhook URL 256
Website 11, 15, 35, 173, 208, 211
Website delivery overview 16
Website working agreement 16, 81
When 206, 325
when 215
who 316
without 190, 243
Work item 15
Work item completed 27, 91, 93
Work item reference not found 118
Work items 28, 35, 66, 87
Workspace 15, 269, 270, 271, 330, 362
workspace admin 222

Workspace GitHub App installations 264, 356, 362
Workspace GitHub connection 103, 279
Workspace invites 22, 33
Workspace navigation 31
Workspace settings 97, 221
workspace switcher 31
Yes 309
You cannot see this work item 142
You do 173, 174, 175, 176, 178, 180, 182, 183
your 95, 142
Your personal GitHub account is connected 106
Your work 19, 32, 34, 208